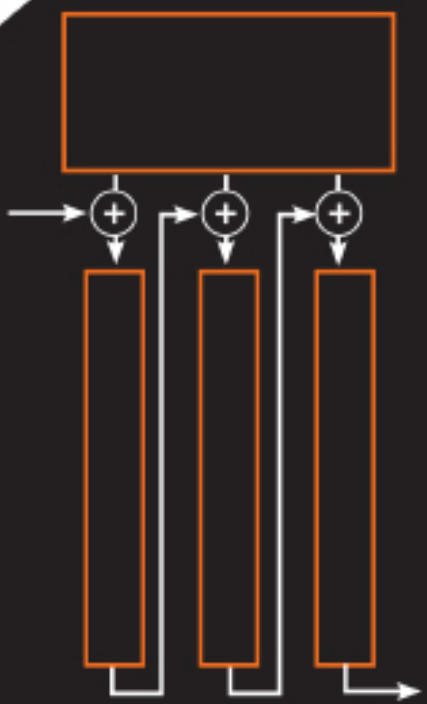
Revised Edition

Algebraic Coding Theory

Elwyn Berlekamp

$g^p - g \equiv 0 \bmod f$



$(1+S)\,\sigma^{(k)} \equiv \omega^{(k)} \bmod z^{2k+1}$

World Scientific

Revised Edition

Algebraic Coding Theory

This page intentionally left blank

Revised Edition

Algebraic Coding Theory

Elwyn Berlekamp
UC Berkeley, emeritus

World Scientific

NEW JERSEY • LONDON • SINGAPORE • BEIJING • SHANGHAI • HONG KONG • TAIPEI • CHENNAI

Published by

World Scientific Publishing Co. Pte. Ltd.

5 Toh Tuck Link, Singapore 596224

USA office: 27 Warren Street, Suite 401-402, Hackensack, NJ 07601

UK office: 57 Shelton Street, Covent Garden, London WC2H 9HE

British Library Cataloguing-in-Publication Data
A catalogue record for this book is available from the British Library.

First published by McGraw-Hill in 1968
Revised Edition by Aegean Park Press in 1984

ALGEBRAIC CODING THEORY
Revised Edition

Copyright © 2015 by World Scientific Publishing Co. Pte. Ltd.

All rights reserved. This book, or parts thereof, may not be reproduced in any form or by any means, electronic or mechanical, including photocopying, recording or any information storage and retrieval system now known or to be invented, without written permission from the publisher.

For photocopying of material in this volume, please pay a copying fee through the Copyright Clearance Center, Inc., 222 Rosewood Drive, Danvers, MA 01923, USA. In this case permission to photocopy is not required from the publisher.

ISBN 978-981-4635-89-9

Printed in Singapore

TO JENNIFER

This page intentionally left blank

PREFACE TO THE REVISED EDITION

When the first edition of Algebraic Coding Theory appeared in 1968, the semiconductor industry was still in its infancy. By today's standards, digital processing was expensive and slow. But digital communication channels were also slow, and so were the transfer speeds for computer files. It had long been known that increasing speeds and capacities inevitably entailed some increases in the raw probability of error. But Claude Shannon (1948) had shown that the use of a modest amount of redundancy made it theoretically possible for decoders to correct all of the bit errors with such a high probability that all other major system parameters could be significantly improved. Lower power, larger files, higher transfer speeds, AND much lower error rates could all be attained, concurrently. To accomplish such goals, the key was to allow more raw errors at the bit level, but to discover and deploy sufficiently sophisticated encoding and decoding to correct them, attaining virtually error-free performance at the system level.

So the quest for error-correcting codes took off. Even before Shannon's work became known outside of Bell Telephone Laboratories, one of his colleagues named Richard Hamming designed a set of rudimentary linear single-error-correcting binary codes able to correct a single bit error in each block of bits. Several more years passed before more powerful error-correcting block codes were designed by others. Most notable among these were Reed–Muller (1954), Bose–Chaudhuri Hocquenghem (BCH) (1959–1960), and Reed–Solomon (RS) (1960). Some of these papers presented algebraic methods for constructing interesting codes, but offered no specific ways to decode them. Then Peterson (1961) provided engineers with a tutorial background on the relevant mathematics as well as specific algorithms to encode and decode BCH and RS codes. But in most situations, these algebra-based block encoders and decoders were not yet considered practical, and a substantially different "convolutional" approach presented by Wozencraft–Jacobs (1965), Fano

(1963), and others, gained popularity in some sectors of the research community.

In 1965 and 1966, I initiated a graduate engineering course in coding theory at the University of California at Berkeley. While teaching it, I discovered some new mathematical algorithms which offered significant improvements for decoding of algebraic codes. The first edition of Algebraic Coding Theory was an outgrowth of that work. The book was also influenced by another course I had taught in digital design, which aroused my interest in the details of computations over finite fields, including arithmetic operations, the factorization of polynomials, and solutions of equations involving matrices with constant diagonals. When the first edition was written, binary BCH codes were considered more promising than nonbinary RS codes. I determined the asymptotic number of information symbols in long BCH codes of specified distances. The result was a singular function of interest in mathematical analysis, nearly two decades before the study of fractals became popular.

Other topics introduced in Algebraic Coding Theory were received with different degrees of enthusiasm by different audiences at different times. The algorithm for factoring rational polynomials mod p was included in Knuth's volume 2 (1969). An enhanced version due to Wang and Roschild (1975) was embedded in a pioneering symbolic manipulation program called MACSYMA, and it became popular in computer science, although the first important mathematics textbook to feature it did not appear until Artin (2010). On the other hand, the algorithm for solving the key equation to decode BCH and RS codes soon received widespread recognition, largely because Massey (1969) revised my exposition and showed that the algorithm could also be used to find the shortest binary feedback shift register whose output sequence is known. There was also considerable interest in Algebraic Coding Theory in the Soviet Union, and the book was translated into Russian and published in Moscow by Mir in 1971.

Engineering applications to communications and to computer memory systems evolved along different paths. In the 1960s, speeds of 5 kilobits per second and bit error rates of 10^{-5} were considered adequate for voice communications, while delays longer than 100 milliseconds were unacceptable. These constraints limited block codes to relatively short lengths, particularly since relatively slow digital circuitry of that era consumed much of the available delay. So most communications engineers turned their attention to improved modulation schemes and to non-algebraic coding schemes, notably Viterbi (1971) codes. On the other hand, computer files generally required near-zero block error rates, usually lower than 10^{-12}.

Transfer rates and clock speeds were typically a few megabits/second. In addition to a constant delay of many bits, many systems could tolerate an additional inter-block delay if it occurred only rarely. So long block codes, often with many more information bits than check bits in each block, became widely used on magnetic disk storage devices. Following an early idea of Forney (1967), on some deep space probes NASA deployed a concatenation including both an inner convolutional code with Viterbi decoding and outer RS code.

In 1973, Solomon Golomb, my wife Jennifer Berlekamp, and I founded a company called Cyclotomics, which focused on applications of algebraic codes. Our first contract was to write software to add appropriate redundancy to archival tapes for the U.S. Census Bureau, some of whose earlier archival tapes, written in the 1950s, had become unreadable. It turned out that most errors occurred in bursts affecting many fewer consecutive bytes than bits. This made byte-oriented RS codes much more attractive than BCH codes. In the 1970s and 1980s, simple interleaved RS codes which correct only up to two character errors/block, using a rudimentary version of Algorithm 7.4 of this book, came into widespread use on compact discs.

At Cyclotomics we found that system constraints varied greatly from project to project, as summarized in an overview I published in Berlekamp (1980). To address these varied challenges, we invented a wide range of solutions including such novel digital architectures as our “GF1” computer and later hypersystolic decoders. We also introduced novel synchronization schemes, helical interleaving, burst forecasting, readable erasures, and bit-serial RS encoders. In many cases, these nonbinary encoders turned out to be surprisingly simpler than binary encoders of comparable redundancy. We obtained many patents, including one on the Welch–Berlekamp decoding algorithm [patented 1986 and described in Berlekamp (1996)] whose advantages included much faster speeds in cases with typical noise levels than with worst-case noise levels. More information about these publications and patents, as well as about my colleagues and collaborators, can be found on my website.

Cyclotomics grew to 40 people. In 1985, it was acquired by Eastman Kodak, and renamed “Kodak Berkeley Research”. Our business, which already included controllers for optical disk memories and communications with telescopes orbiting in space, expanded to include “Cinema Digital Sound” for the movie industry. In the mid-1990s, Kodak Berkeley Research was merged into a bigger division of Kodak and moved to Rochester, NY. By then, my primary research interests had changed to other subjects.

Since the appearance of the second edition of *Algebraic Coding Theory* in 1984, digital electronics have continued to advance at an exponential pace as predicted by Gordon Moore's "Law". The difference between several racks of electronics and one board first shrank to become the difference between one board and one chip. Another decade later, it had shrunk to the difference between a small fraction of a custom chip and an even tinier fraction of a chip. The algorithmic sophistication that once made the difference between feasibility and infeasibility is no longer as economically important as it once was. Some technically superior designs have been displaced by more popular designs which are more intelligible to less educated practitioners.

Academic interests in *Algebraic Coding Theory* have remained strong. Parts of the original edition of this book were used as a text in graduate courses at several universities, mostly in Electrical Engineering departments. As the popularity of these courses grew, several competing books appeared. Applications proliferated. Although the Welch–Berlekamp (WB) algorithm remains technically the best of all known methods for decoding high-rate RS codes, it is now only rarely found in the traditional applications of communications and memories.

But new applications arose elsewhere. When theoretical computer scientists became interested in probabilistic proofs, they found that Reed–Solomon codes were the best way to implement them. In this application, capability to correct errors beyond the traditional bounds was more important than speed. Madhu Sudan (1997) devised a brilliant extension of the Welch–Berlekamp algorithm to improve performance at low code rates. For this work, Sudan received the 2002 Nevanlinna Prize. This is the most prestigious award in applied mathematics given by the International Congress of Mathematics, comparable to their more famous Fields Medals in Pure Mathematics.

There are still numerous opportunities to identify and solve interesting problems at the interface of discrete mathematics, computer science and electrical engineering. *Algebraic Coding Theory* remains a fundamental reference in this increasingly important area.

==========

REFERENCES (mentioned in this preface which do not appear in the 2nd edition)

Artin, M., 2010, "Algebra" 2nd edition, Pearson Education.

Berlekamp, E. R., and L. R. Welch, 1986, Error Correction of Algebraic Block Codes, US Patent 4,633,470.

Berlekamp, E. R., 1996, "Bounded Distance+1 Soft-Decision Reed-Solomon Decoding", *IEEE Trans. Inform. Theory*, **IT42**: 704–720.

Forney, G. D., 1967, *Concatenated Codes*, MIT Press: Cambridge, Massachusetts.

Sudan, M., 1997, "Decoding of Reed Solomon codes beyond the error-correction bound," *J. Complexity*, special issue dedicated to Shmuel Winograd, **13**(1): 180–193.

Wang, P. S. and Rothschild, L. P., 1975, "Factoring multivariate polynomials over the integers," *Math. Comp.* **29**: 935–950.

This page intentionally left blank

PREFACE TO SECOND EDITION

Much has happened since *Algebraic Coding Theory* first appeared fifteen years ago. At that time, most people viewed the subject as a collection of mathematical topics. However, the number of applications grew so much during the 1970s that there is now widespread interest in Algebraic Coding Technology, the art of designing and implementing algebraic codes to solve engineering problems.

Applications have proliferated. Many digital communication systems now use error-correcting codes. Large modern memory devices also use codes. Most semiconductor memories employ bit error-correcting codes, and many larger disc memories employ burst-correcting codes. A wide variety of codes described in this book are used on various optical and magnetic memories. A short Reed-Solomon code was recently implemented in a single integrated circuit within a commercially available consumer product, a digital audio device which plays optically-recorded compact discs at a speed of about 2 Megabits per second. More expensive devices have been built to handle more powerful codes at faster speeds. Some magnetic tape machines intended for use in recording, playing, and editing digital color TV at broadcast studios contain sophisticated special-purpose computers that correct 1400-bit dropouts and multiple bursts of errors at speeds of 120 Megabits per second. Even more ambitious decoders are now under construction.

The technology changes dramatically as newer integrated circuits and other electronic devices appear every few years, but the fundamental principles and algorithms which provide the best foundations for algebraic coding remain relatively unchanged. The graduate student or engineer who seeks to enter the field today will find this book even more valuable than did his counterpart in 1968. The modern student will also encounter more applications of algebraic coding theory in other areas, such as cryptography and mathematical games.

Misprints and errors in the original edition have been corrected. Many of the circuit diagrams have been revised. The original bibliography has been expanded to provide references to the more important recent publications, which extend some of the advanced topics introduced in the latter chapters of this book.

This page intentionally left blank

PREFACE

Since the appearance of Shannon's classic papers in 1948 and 1949, a great deal of research has been devoted to the problem of designing efficient schemes by which information can be coded for reliable transmission across channels which are corrupted by noise. From a practical standpoint, the essential limitation of all coding and decoding schemes proposed to date has **not** been Shannon's **capacity but** the complexity (and **cost**) **of** the **decoder.** For this reason, efforts have been directed toward the design of coding and decoding schemes which could be easily implemented.

The important work of Reed and Solomon (1960), Hocquenghem (1959), Bose and Chaudhuri (1960), Gorenstein and Zierler (1961), and Peterson (1961) marked the advent of a new approach to this problem. By associating each digit of certain codes with an element in a Galois field, it was found possible to derive an algebraic equation whose roots represent the locations of the channel errors. The decoding problem could then be reduced to the computational problem of setting up this algebraic equation and finding its roots. The computational effort required to implement this approach turned out to be orders of magnitude less than the effort which would be required by the direct techniques of decoding by exhaustive search. Using new techniques which are introduced in this book, it is now possible to build algebraic decoders which are orders of magnitude simpler than any that have previously been considered.

The reader who yearns for a "simple" or "elementary" proof of the Bose-Chaudhuri and Hocquenghem theorem must be warned that this book makes no attempt to supply the object of his quest. The real importance of the BCH construction does not stem from the theorem that one can design t-error-correcting codes, for any t. As shown in Chap. 13, the minimum distances of the BCH codes are asymptotically disappointing. Even for modest block lengths, the quadratic-residue codes are often as good or better. The importance of BCH codes stems from the fact that they are capable of correcting all patterns of up to t errors (and many patterns of more errors) *by a simple, easily instrumented decoding algorithm.* There is frequently a conflict between proofs which some people consider conceptually "simple" and proofs which lead to simple instrumentation. In this book I have attempted to provide the proofs which lead to the simplest implementations. The mathematician should not be dismayed by such "practical" considerations, for the solution of these practical problems often leads us to a deeper study of such "abstract" mathematical topics as Ore's p polynomials and Stickelberger's theorem.

This book is an attempt to provide two things:

1 *A comprehensive coverage of algebraic coding theory,* including BCH codes (which include the Reed and Solomon codes), the Srivastava codes, a new class of negacyclic codes for the Lee metric, certain other classes of codes, e.g., Example 5.7, and the best known algebraic methods for decoding these codes.
2 *An introductory coverage of the interfaces between algebraic coding theory and surrounding areas.* The coverage of those topics which are very close to algebraic coding theory, e.g., constructive but nonalgebraic decoding algorithms, such as threshold decoding, is correspondingly greater than the coverage of those topics which lie at a greater distance from algebraic coding theory, e.g., modulation theory. No mention is made of topics which are essentially disjoint from algebraic coding theory, e.g., codes to correct synchronization errors.†

The approximate subject matter of the various chapters of this book is indicated by the positioning of the numbers in Fig. 0.1. For example, the subject matter of Chap. 6 lies somewhere near the interfaces of algebraic coding theory and modern algebra, elementary number theory, and shift-register sequences. Engineering courses are commonly slanted toward the topics listed on the right of Fig. 0.1; mathematics courses, toward the left. It is hoped that this book will prove valuable to readers with varying spectra of interest.

This book is intended for use as a text for beginning graduate students or advanced undergraduates. No prerequisites are assumed beyond an acquaintance with vectors, matrices, and the concepts of dimension, rank and nullity. Of course, any additional knowledge of modern algebra, information theory, or communication theory which the student might possess will prove beneficial, but the teacher should not overestimate the overlap between algebraic coding theory and these surrounding areas. The material covered in this book has surprisingly little in common with the material covered in the standard abstract algebra courses, e.g., Birkhoff and MacLane (1965) or van der Waerden (1931). Most of the algebra presented in this book deals not with questions of existence and uniqueness but with practical algorithms and techniques for solving equations in finite fields. As such, it is best categorized as a branch of the emerging field of discrete numerical analysis.

The reader who feels a danger of becoming bogged down in details may wish to skip the starred sections on his first reading.

† Stiffler (1969) provides a good coverage of synchronization codes.

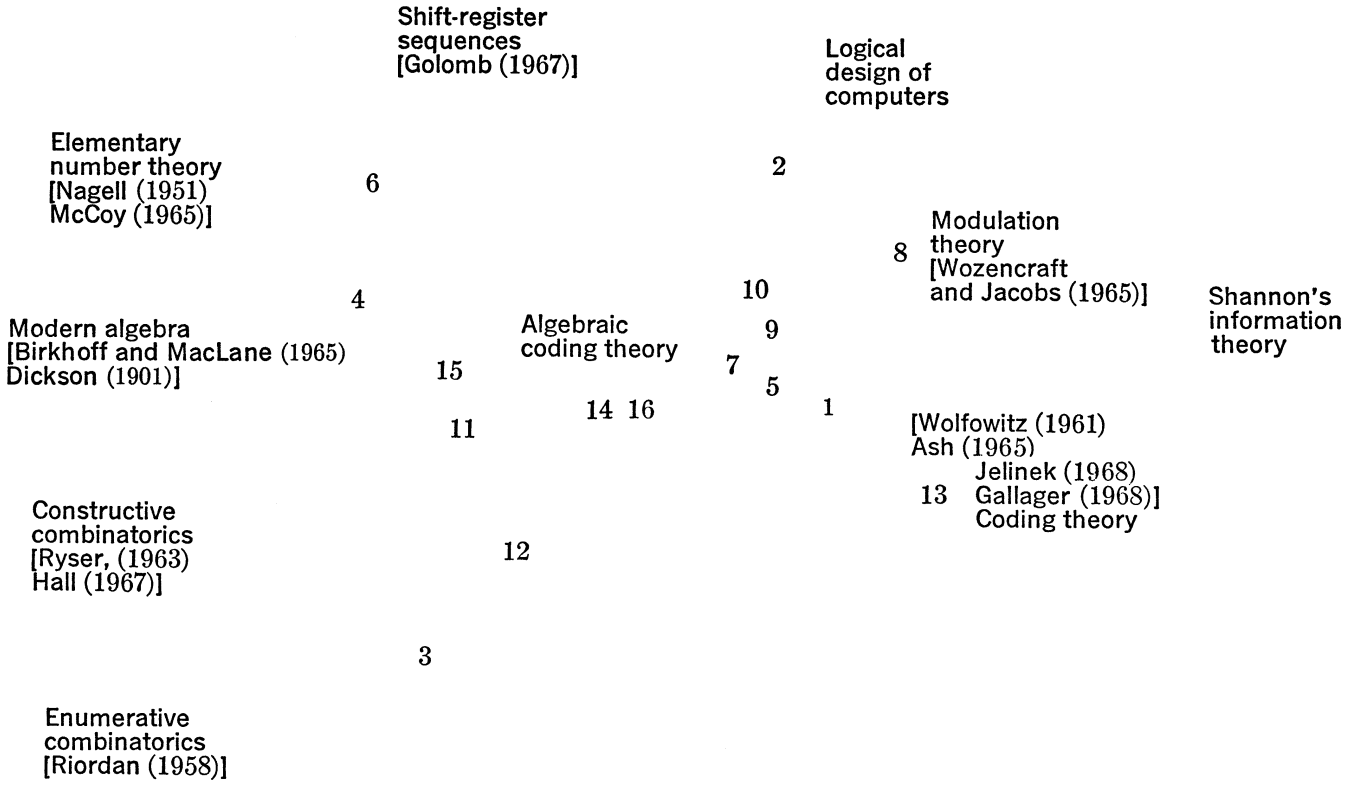


Fig. 0.1 Approximate relationship between the chapters of this book and certain surrounding subjects.

Although I have attempted to reduce the errors in this book insofar as possible, there are no doubt many remaining. I shall be grateful to any reader who points them out to me so that they can be corrected in subsequent printings. I will offer one dollar reward to the first finder of any mistake, whether it be a misstated theorem or a mere typographical error. Needless to say, this offer does not apply to dates or titles in references to unpublished papers, for the author cannot predict with certainty when the cited work will appear or what its final title will be. I reserve the right to count repetitions of the same error as a single mistake.

Notation Matrices are always represented by script letters, such as $\mathcal{M}$. Row vectors are always represented by boldface letters, such as $\mathbf{R}$. The transpose of $\mathbf{R}$ is denoted by $\mathbf{R}^t$. Since there is no occasion to attempt to take the tth power of a vector, this is unambiguous. A sequence of polynomials is always indexed with superscripts, such as $f^{(1)}(x)$, $f^{(2)}(x)$, $f^{(3)}(x)$, . . . , where $f^{(n)}(x) = \sum_i f_i^{(n)} x^i$. When the indeterminate is understood, this sequence of polynomials is sometimes abbreviated as $f^{(1)}$, $f^{(2)}$, $f^{(3)}$, . . .

I have used the decimal numbering system for everything in this book except figures, tables, and problems. For example, the reader will find Eq. (3.37) after Theorem 3.36 but before Sec. 3.4. Similarly, Theorem 6.69 is after Definition 6.689 but before Lemma 6.691.

Elwyn R. Berlekamp

ACKNOWLEDGMENTS

This book grew out of a course which I taught at the University of California, Berkeley, in the spring of 1966. Much of it is based on my own research during the previous two years. During that time, I have benefited from numerous discussions with several persons, most notably G. Solomon of the Jet Propulsion Laboratory and R. C. Bose and I. Chakravarti of the University of North Carolina, where I visited in the fall of 1966. J. Brillhart and N. J. A. Sloane reviewed early versions of the manuscript and supplied many detailed corrections and valuable suggestions. L. D. Baumert, G. D. Forney, R. L. Graham, S. W. Golomb, E. N. Gilbert, Mrs. F. J. MacWilliams, J. L. Massey, H. F. Mattson, H. O. Pollak, and J. H. Spencer also supplied several helpful suggestions.

I would also like to express my thanks to the American Telephone and Telegraph Company and to the University of North Carolina Press for permission to republish modified versions of my papers Factoring Polynomials over Finite Fields, The Enumeration of Information Symbols in BCH Codes, and Negacyclilc Codes for the Lee Metric. The former two papers appeared in the *Bell System Technical Journal*, October, 1967; the latter, in R. C. Bose and T. A. Dowling (eds.) "Proceedings of the Conference on Combinatorial Mathematics and Its Applications."

This page intentionally left blank

CONTENTS

Preface to the Revised Edition vii

Preface to the Second Edition xiii

Preface xv

Acknowledgments xix

1. Basic Binary Codes 1
- 1.1 Repetition Codes and Single-Parity-Check Codes 1
- 1.2 Linear Codes 4
- 1.3 Hamming Codes 8
- 1.4 Manipulative Introduction to Double-Error-Correcting BCH Codes 12

Problems 20

2. Arithmetic Operations Modulo an Irreducible Binary Polynomial 21
- 2.1 A Closer Look at Euclid's Algorithm 21
- ⋆2.2 Logical Circuitry 30
- ⋆2.3 Multiplicative Inversion 36
- ⋆2.4 Multiplication 44
- ⋆2.5 The Solution of Simultaneous Linear Equations 51
- ⋆2.6 Special Methods for Solving Simultaneous Linear Equations When the Coefficient Matrix is Mostly Zeros 63

Problems 69

3. The Number of Irreducible *q*-ary Polynomials of Given Degree 70
- 3.1 A Brute-Force Attack 70
- 3.2 Generating Functions 73
- 3.3 The Number of Irreducible Monic *q*-ary Polynomials of Given Degree—A Refined Approach 76
- ⋆3.4 The Moebius Inversion Formulas 81

Problems 85

⋆ STARRED SECTIONS MAY BE SKIPPED ON FIRST READING.

4. The Structure of Finite Fields 87
4.1 Definitions 87
4.2 Multiplicative Structure of Finite Fields 88
4.3 Cyclotomic Polynomials 90
4.4 Algebraic Structure of Finite Fields 96
4.5 Examples 105
⋆4.6 Algebraic Closure 111
⋆4.7 Determining Minimal Polynomials 112
Problems 117

5. Cyclic Binary Codes 119
5.1 Reordering the Columns of the Parity-Check Matrix of Hamming Codes 119
5.2 Reordering the Columns of the Parity-Check Matrix of Double-Error-Correcting Binary BCH Codes 125
5.3 General Properties of Cyclic Codes 129
5.4 The Chien Search 132
5.5 Outline of General Decoder for Any Cyclic Binary Code 136
5.6 Example 138
5.7 Example 139
5.8 Equivalence of Cyclic Codes Defined in Terms of Different Primitive nth Roots of Unity 141
Problems 144

6. The Factorization of Polynomials Over Finite Fields 146
6.1 A General Algorithm 146
⋆6.2 Determining the Period of a Polynomial 150
⋆6.3 Trinomials Over GF(2) 153
6.4 Factoring $x^n - 1$ Explicitly 154
⋆6.5 Determining the Degrees of the Irreducible Factors of the Cyclotomic Polynomials 156
⋆6.6 Is the Number of Irreducible Factors of $f(x)$ Over GF(q) Odd or Even? 159
⋆6.7 Quadratic Reciprocity 171
Problems 173

7. Binary BCH Codes for Correcting Multiple Errors 176
7.1 Examples 176
7.2 The Key Equation for Decoding Binary BCH Codes 178
7.3 Heuristic Solution of the Key Equation 180
7.4 An Algorithm for Solving the Key Equation Over Any Field 184
⋆7.5 Relation to Matrix Decoding Methods 188
⋆7.6 Simplifications in Algorithm 7.4 for Binary BCH Codes 192
7.7 Implementation of Binary BCH Decoders 195
Problems 199

8. Nonbinary Coding 200
8.1 Modulation Schemes 200
8.2 Weight Functions 204
Problem 206

9. Negacyclic Codes for the Lee Metric 207
9.1 Error Locations and the Error-Locator Polynomial 207
9.2 Double-Error-Correcting Codes 209
9.3 Negacyclic Codes 211
Problems 217

10. Gorenstein-Zierler Generalized Nonbinary BCH Codes for the Hamming Metric 218
10.1 The Generalized BCH Codes and An Algorithm to Decode Them 218
10.2 Examples 221
⋆10.3 Alternate BCH Codes and Extended BCH Codes 223
10.4 Decoding Erasures as Well as Errors 229
10.5 Decoding More Than *t* Errors 231
10.6 Examples 237
Problem 240

11. Linearized Polynomials and Affine Polynomials 241
11.1 How to Find Their Roots 241
11.2 The Least Affine Multiple 245
⋆11.3 Abstract Properties of Linearized and Affine Polynomials 247
⋆11.4 Transformations of $\tilde{f}(z)$ 255
⋆11.5 Root Counting 257
⋆11.6 Low-Weight Codewords in Certain Codes 262
Problems 271

12. The Enumeration of Information Symbols in BCH Codes 273
12.1 Converting the Problem to an Enumeration of Certain Integers mod n 273
⋆12.2 Converting the Problem for Primitive BCH Codes to an Enumeration of Certain *q*-ary Sequences 275
⋆12.3 Sequence Enumeration Theorems 278
⋆12.4 Examples 284
⋆12.5 The Enumeration of Information Symbols in Nonprimitive BCH Codes 287
12.6 Asymptotic Results 290
⋆12.7 Actual Distance 294
Problems 296

13. The Information Rate of the Optimum Codes 298
13.1 The Hamming-Rao High-Rate Volume Bound 298
⋆13.2 Perfect Codes 302
⋆13.3 The Bound $d \leq n + 1 - k$ 309
13.4 Plotkin's Low-Rate Average Distance Bound 311
⋆13.5 Equidistant Codes 315
13.6 The Elias Bound 318

13.7 The Gilbert Bound 321
13.8 Asymptotic Error Bounds and Finite Special Cases 324

14. Codes Derived by Modifying or Combining Other Codes 331
14.1 Extending a Code by Annexing More Check Digits 333
14.2 Puncturing a Code by Deleting Check Digits 334
14.3 Augmenting Additional Codewords to the Code 335
14.4 Expurgating Codewords from the Code 335
14.5 Lengthening the Code by Annexing More Message Digits 336
14.6 Shortening the Code by Omitting Message Digits 336
14.7 Subfield Subcodes 336
14.8 Direct-Product Codes and Their Relatives 338
14.9 Concatenated Codes 347

15. Other Important Coding and Decoding Methods 350
15.1 Srivastava Codes—Noncyclic Codes with an Algebraic Decoding Algorithm 350
15.2 Residue Codes—Good Codes Which are Hard to Decode 352
15.3 Reed-Muller Codes—Weak Codes Which are Easy to Decode 361
15.4 Threshold Decoding—The Best Known Algorithm for Decoding Certain Codes 367
15.5 Orthogonalizable Codes Based on Finite Geometries 375
15.6 Convolutional Codes—A Survey 388
Problems 394

16. Weight Enumerators 397
16.1 The Relationship between Weight Enumerators and the Probability of Decoding Failure 397
16.2 The MacWilliams-Pless Equations for the Weight Enumerators of Dual Codes 400
⋆16.3 Weight Restrictions 407
⋆16.4 Kasami's Weight Enumerators for Certain Subcodes of the Second-Order RM Code 417
⋆16.5 The Weight Enumerator for the Reed-Solomon Codes 429

Appendix A 441

Appendix B 442

References 443

References to the Second Edition 453

Index 461

Revised Edition

Algebraic Coding Theory

This page intentionally left blank

1
Basic Binary Codes

1.1 REPETITION CODES AND SINGLE-PARITY-CHECK CODES

Suppose that we wish to transmit a sequence of binary digits across a noisy channel. If we send a one, a one will probably be received; if we send a zero, a zero will probably be received. Occasionally, however, the channel noise will cause a transmitted one to be mistakenly interpreted as a zero or a transmitted zero to be mistakenly interpreted as a one. Although we are unable to prevent the channel from causing such errors, we can reduce their undesirable effects with the use of coding. The basic idea is simple. We take a set of k message digits which we wish to transmit, annex to them r check digits, and transmit the entire block of $n = k + r$ channel digits. Assuming that the channel noise changes sufficiently few of these n transmitted channel digits, the r check digits may provide the receiver with sufficient information to enable him to detect and correct the channel errors.

Given any particular sequence of k message digits, the transmitter must have some rule for selecting the r check digits. This is called the

encoding problem. Any particular sequence of n digits which the encoder might transmit is called a *codeword.* Although there are 2^n different binary sequences of length n, only 2^k of these sequences are codewords, because the r check digits within any codeword are completely determined by the k message digits. The set consisting of these 2^k codewords of length n is called the *code.*

No matter which codeword is transmitted, any of the 2^n possible binary sequences of length n may be received if the channel is sufficiently noisy. Given the n received digits, the decoder must attempt to decide which of the 2^k possible codewords was transmitted.

Among the simplest examples of binary codes are the *repetition codes,* with $k = 1$, r arbitrary, and $n = k + r = r + 1$. The code contains two codewords, the sequence of n zeros and the sequence of n ones. We may call the first digit the *message digit;* the other r digits, *check digits.* The value of each check digit (each 0 or 1) in a repetition code is identical to the value of the message digit. The decoder might use the following rule. Count the number of zeros and the number of ones in the received bits. If there are more received zeros than ones, decide that the all-zero codeword was sent; if there are more ones than zeros, decide that the all-one codeword was sent. If the number of zeros equals the number of ones, do not decide.

It is evident that this decoding rule will decode correctly in all cases when the channel noise changes less than half of the digits in any one block. If the channel noise changes exactly half of the digits in any one block, the decoder will commit a *decoding failure;* i.e., he will not decode the received word into any of the possible transmitted message words. If the channel noise changes more than half of the digits in any one block, the decoder will commit a *decoding error;* i.e., he will decode the received word into the wrong codeword. If channel errors occur infrequently, the probability of a decoding failure or a decoding error for a repetition code of long block length is very small indeed.

Although we consider a decoding failure to be preferable to a decoding error, we would much prefer correct decoding to either of these two events. Of course, it is often possible to make certain tradeoffs between decoding errors and decoding failures by modifying the decoding algorithm. For example, consider the binary repetition code of block length $n = 5$. First suppose that all received sequences containing 0, 1, or 2 ones are decoded as the all-zero codeword and all received sequences containing 3, 4, or 5 ones are decoded as the all-one codeword. This decoding algorithm decodes every possible received word into one of the possible transmitted codewords; it is a *complete* decoding algorithm. Complete decoding algorithms have no possibility of decoding failure. However, we might instead use an alternate, *incomplete* decoding algo-

rithm for the same binary repetition code of block length $n = 5$. For example, we might decode all received sequences containing 0 or 1 one into the all-zero codeword and all sequences containing 4 or 5 ones into the all-one codeword. This algorithm fails to decode sequences containing 2 or 3 ones. Although this incomplete decoding algorithm has a positive probability of decoding failure, the incomplete algorithm has a considerably lower probability of decoding error.

In some applications, a decoding error is tantamount to disaster. For example, it may result in an incorrect command being received by a missile or a spaceship. A decoding failure, on the other hand, may result only in the command's being ignored. This may represent only a minor nuisance, which can be overcome simply by repeating the command. In such applications one prefers a very incomplete decoding algorithm, which intentionally refuses to decode any sufficiently ambiguous received word.

In other applications, however, there may be no opportunity to repeat undecoded messages. In such cases, decoding failure is almost as severe a catastrophe as decoding error. In applications of this type, one wishes to maximize the probability of correct decoding. Even though the evidence presented by the received word may be rather inconclusive, a complete decoding algorithm will never give up. No matter what sequence is received, it will make some decision as to the transmitted codeword, even if this decision is only an intelligent guess.

For many codes, it is very difficult to expand known incomplete decoding algorithms into complete decoding algorithms. However, it is usually quite easy to derive good incomplete decoding algorithms from the complete decoding algorithm.

We have seen that a complete decoding algorithm for the binary repetition codes may be formulated without difficulty. If the block length is sufficiently large, the probability of decoding error is very small. However, these codes have very low *information rate*, $R = k/n$. All but one of the digits in a repetition code are check digits. We are usually more interested in codes which have a higher information rate.

Extreme examples of such high-rate codes are the *single-parity-check codes*, which contain only one check digit. This check digit is taken to be the mod 2 sum of the code's $n - 1$ information digits. These information digits are added according to the binary rules: $0 + 0 = 0$, $0 + 1 = 1$, $1 + 0 = 1$, $1 + 1 = 0$. The binary sum of a number of binary digits is seen to be 0 or 1, accordingly as the number of ones among these digits is even or odd. From this it follows that the total number of ones (including the check digit) in every codeword of a single-parity-check code is even. If the received word contains an even number of ones, the decoder may decode it without change, but if the received word

contains an odd number of ones, the decoder should not decode it. This incomplete-decoding rule will decode correctly only if no channel errors occur in the transmitted block. A single channel error (or, for that matter, any odd number of channel errors) will be detected as a decoding failure. Any combination of two channel errors (or, for that matter, any nonzero even number of channel errors) will cause a decoding error.

These two examples, the repetition codes and the single-parity-check codes, provide the extreme, relatively trivial, cases of binary codes. The repetition codes have enormous error-correction capability, but only one message digit per block. The single-parity-check codes have very high information rate, but since they contain only one check digit per block, they are unable to do more than detect an odd number of channel errors. In order to interpolate between these two extreme classes to find codes which have moderate rates and moderate error-correction capabilities, we shall consider the more general class of linear codes, of which repetition codes and single-parity-check codes are special cases.

1.2 LINEAR CODES

In a code containing several message digits and several check digits, each check digit must be some function of the message digits. In the simple case of single-parity-check codes, the single parity check was chosen to be the binary sum of all the message digits. If there are several parity checks, it is wise to set each check digit equal to the binary sum of some subset of the message digits. For example, we construct a binary code of block length $n = 6$, having $k = 3$ message digits and $r = 3$ check digits. We shall label the three message digits C_1, C_2, and C_3 and the three check digits C_4, C_5, and C_6. We choose these check digits from the message digits according to the following rules:

$$\begin{aligned} C_4 &= C_1 + C_2 \\ C_5 &= C_1 \quad + C_3 \\ C_6 &= \quad C_2 + C_3 \end{aligned}$$

or, in matrix notation,

$$\begin{bmatrix} C_4 \\ C_5 \\ C_6 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} C_1 \\ C_2 \\ C_3 \end{bmatrix}$$

The full codeword consists of the digits C_1, C_2, C_3, C_4, C_5, C_6. Every codeword must satisfy the parity-check equations

$$\begin{aligned} C_1 + C_2 \quad + C_4 \quad &= 0 \\ C_1 \quad + C_3 \quad + C_5 \quad &= 0 \\ C_2 + C_3 \quad + C_6 &= 0 \end{aligned}$$

or, in matrix notation,

$$\begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix} \mathbf{C}^t = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Here $\mathbf{C}^t$ denotes the column vector which is the transpose of the row vector

$$\mathbf{C} = [C_1, C_2, C_3, C_4, C_5, C_6]$$

More succinctly, we may write this equation as

$$\mathcal{H}\mathbf{C}^t = \mathbf{0}$$

where $\mathbf{0}$ is the three-dimensional column vector whose components are all zeros and $\mathcal{H}$ is the *parity-check matrix*

$$\mathcal{H} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

The $2^3 = 8$ codewords in the code are seen to be

000000 001011 010101 011110 100110 101101 110011 111000

After the message sequence is encoded into the full codeword, the codeword is transmitted across the noisy channel. The channel adds to this the noise word, $\mathbf{E} = [E_1, E_2, E_3, E_4, E_5, E_6]$, where†

$$E_i = \begin{cases} 0 & \text{if channel does not change } i\text{th digit} \\ 1 & \text{if channel changes } i\text{th digit} \end{cases}$$

The received word is given by the sequence $\mathbf{R} = [R_1, R_2, R_3, \ldots, R_6]$ where $R_i = C_i + E_i$, or, in vector notation, $\mathbf{R} = \mathbf{C} + \mathbf{E}$. The decoder begins by computing the *syndrome* digits, which are defined by the equation

$$\mathbf{s}^t = \mathcal{H}\mathbf{R}^t$$

For the present example, this equation is

$$\begin{bmatrix} s_1 \\ s_2 \\ s_3 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix} \mathbf{R}^t$$

Since the syndrome digits are defined by the same equations as the

† We use the terms *word* and *vector* interchangeably.

parity-check equations, the syndrome digits reveal the pattern of parity-check failures on the received codeword.

In order to decode, the decoder must eventually answer the question, "What was the transmitted codeword **C**?" However, it turns out to be easier to first answer the intermediate question, "What was the channel error pattern **E**?" If the decoder is able to find the correct error vector **E**, then **C** may be found from the formula $\mathbf{C} = \mathbf{R} - \mathbf{E}$. (In the binary case, of course, $+ = -$, difference = sum, and $\mathbf{C} = \mathbf{R} - \mathbf{E}$ means $\mathbf{C} = \mathbf{R} + \mathbf{E}$.)

If **R** is the received word, then the set of possible error patterns is just the set of vectors which have the same syndrome as **R**. For, if $\mathcal{H}\mathbf{R}^t \neq \mathcal{H}\mathbf{E}^t$, then $\mathcal{H}(\mathbf{R} - \mathbf{E})^t \neq \mathbf{0}$, so $\mathbf{R} - \mathbf{E}$ could not have been a transmitted codeword. However, if $\mathcal{H}\mathbf{R}^t = \mathcal{H}\mathbf{E}^t$, then $\mathcal{H}(\mathbf{R} - \mathbf{E})^t = \mathbf{0}$, and the error pattern **E** occurred if the codeword $\mathbf{C} = \mathbf{R} - \mathbf{E}$ was transmitted.

For example, suppose the received word is a codeword. Then $\mathcal{H}\mathbf{R}^t = \mathbf{0}$, and $\mathcal{H}\mathbf{E}^t = \mathbf{0}$, so that the error pattern must also be a codeword.

The set of n-dimensional binary vectors which have the zero syndrome is precisely the set of codewords. If **x** and **y** are two codewords, then $\mathcal{H}\mathbf{x}^t = \mathcal{H}\mathbf{y}^t = \mathbf{0}$, so $\mathcal{H}\mathbf{x}^t - \mathcal{H}\mathbf{y}^t = \mathbf{0}$, and $\mathbf{x} - \mathbf{y}$ is also a codeword. Thus, the difference of any pair of codewords is another codeword. For this reason, the set of codewords is said to form a *linear code*, or a *group code*. More generally, if **x** and **y** have the same syndrome (not necessarily zero), then $\mathcal{H}\mathbf{x}^t = \mathcal{H}\mathbf{y}^t$, $\mathcal{H}(\mathbf{x}^t - \mathbf{y}^t) = \mathbf{0}$, and $\mathbf{x} - \mathbf{y}$ is a codeword. The set of all vectors having the same syndrome is said to be a *coset* of the group of codewords. As we have just shown, the difference between any two vectors in the same coset is a codeword. Conversely, the sum of some vector **x** and any codeword gives another vector in the same coset as **x**. Thus, the number of vectors in any coset is identical to the number of codewords, which is 2^k. *The possible error patterns are the words in the same coset as the received word.*

Thus, the decoder may immediately exclude all error patterns which do not lie in the same coset as the received word, i.e., error patterns which have different syndromes. However, all error patterns which are in the same coset as the received word are possible. None of those possible error patterns may be excluded with certainty. But since channel errors are relatively infrequent, some of the error patterns within a given coset are much less likely than others. In general, error patterns having a small number of ones are more likely than error patterns having many ones. More precisely, we define the *weight* of any n-dimensional binary vector as the number of ones in that vector. Within any coset, we may select a vector having least weight as *coset leader*. *One most probable error pattern is the leader of the coset containing the received word.*

In the present example,

$$\mathcal{H} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

The cosets are the rows of Table 1.1. The first row gives the coset having the **0** syndrome; this is the set of codewords. In each subsequent coset, a coset leader is listed in the first column. Every other entry in the table is the sum of the codeword at the top of its column and the coset leader at the left of its row.

In general, we use the terminology of Definitions 1.21.

Definitions 1.21 A code is said to be a *linear* code or a *group* code iff† its codewords are the set of vectors **C** which satisfy an equation $\mathcal{H}\mathbf{C}^t = \mathbf{0}$. The matrix $\mathcal{H}$ is called the *parity-check matrix*. The code's block length n is equal to the number of columns of $\mathcal{H}$. The *syndrome* **s** of any word **R** is defined by the equation $\mathbf{s}^t = \mathcal{H}\mathbf{R}^t$. A *coset* consists of all the words having a given syndrome. The *weight* of any word is the number of ones among its n digits. Within each coset, a word having the least weight is chosen as *coset leader.*

All linear codes have the important properties asserted in Theorem 1.22.

Theorem 1.22 If **R** is the received word, the set of possible error words is the coset containing **R**. A most probable error word is the leader of the coset containing **R**. A linear code may be decoded as follows. Compute the syndrome, find the leader of the coset hav-

† The abbreviation iff means "if and only if."

Table 1.1 Slepian's standard array

Syndrome	*Words*							
000	000000	001011	010101	011110	100110	101101	110011	111000
001	000001	001010	010100	011111	100111	101100	110010	111001
010	000010	001001	010111	011100	100100	101111	110001	111010
011	001000	000011	011101	010110	101110	100101	111011	110000
100	000100	001111	010001	011010	100010	101001	110111	111100
101	010000	011011	000101	001110	110110	111101	100011	101000
110	100000	101011	110101	111110	000110	001101	010011	011000
111	001100	000111	011001	010010	101010	100001	111111	110100

ing this syndrome, and subtract this coset leader from the received word to find a most likely transmitted codeword.

Two major problems arise at once: (1) how does one choose the $\mathcal{H}$ matrix and (2) how does one determine the coset leader, given the syndrome?

For small n, these questions may be answered by an exhaustive search. For large n, k, and r, however, both of these problems are generally unsolved. However, a great many "good" methods of choosing the parity-check matrix and of finding the coset leader from the syndrome are known. We shall present the most promising of these methods in the subsequent sections of this book.

The most trivial examples of good linear codes are the repetition codes and the single-parity-check codes which we presented in Sec. 1.1. The parity-check matrix for the single-parity-check code has only one row and n columns

$$\mathcal{H} = [1111 \cdots 11]$$

The parity-check matrix for the repetition code has $n - 1$ rows and n columns. With $n = 5$,

$$\mathcal{H} = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

1.3 HAMMING CODES†

At extremely low rates or extremely high rates, it is relatively easy to find good linear codes. In order to interpolate between these two extremes, we might adopt either of two approaches: (1) start with the low-rate codes and gradually increase k by adding more and more codewords, attempting to maintain a large error-correction capability, or (2) start with good high-rate codes and gradually increase the error-correction capability, attempting to add only a few additional parity-check constraints.

Historically, the second approach has proved more successful.

† All of the perfect single-error-correcting binary group codes were first discovered by Hamming. The Hamming code of length 7 was first published as an example in the paper by Shannon (1948). The generalization of this example was mentioned by Golay (1949) prior to the appearance of the paper by Hamming (1950). The Hamming codes had been anticipated by Fisher (1942) in a different context.

This is the approach we shall follow. We begin by constructing certain codes to correct single errors, the Hamming codes.

The syndrome of a linear code is related to the error pattern by the equation $\mathbf{s}^t = \mathcal{H}\mathbf{E}^t$. In general, the right side of this equation may be written as E_1 times the first column of the $\mathcal{H}$ matrix, plus E_2 times the second column of the $\mathcal{H}$ matrix, plus E_3 times the third column of the $\mathcal{H}$ matrix, plus . . . . For example, if

$$\mathbf{s}^t = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix} [E_1, E_2, E_3, E_4, E_5, E_6]^t$$

then

$$\begin{bmatrix} s_1 \\ s_2 \\ s_3 \end{bmatrix} = E_1 \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} + E_2 \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} + E_3 \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} + E_4 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + E_5 \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + E_6 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

The syndrome is thus seen to be the sum of those columns of the $\mathcal{H}$ matrix where channel errors occur. If any column of the $\mathcal{H}$ matrix is **0**, then an error in that position will have no effect on the syndrome. The code is unable even to detect an error in such a position. If two columns of the $\mathcal{H}$ matrix are identical, then a single error in one of these two positions has the same syndrome as a single error in the other. Both single-error patterns then lie in the same coset. Since only one of them may be chosen as coset leader, only one of them can be corrected. The other single-error pattern will be incorrectly decoded. Thus, a linear code is incapable of correcting all patterns of single errors unless the columns of its $\mathcal{H}$ matrix are distinct and nonzero.

Conversely, let us suppose that all columns of the $\mathcal{H}$ matrix are distinct and nonzero. Then a single error in any position results in a different syndrome than a single error in any other position. Every single-error pattern is a coset leader, and the code is capable of correcting all patterns of single errors.

Theorem 1.31 A linear binary code is capable of correcting all patterns of not more than one channel error iff all columns of its $\mathcal{H}$ matrix are distinct and nonzero.

To decode single errors, the decoder computes the syndrome of the received word. If the syndrome is zero, the decoder assumes that no errors occurred. If the syndrome is nonzero and equal to some column of the $\mathcal{H}$ matrix, then the decoder assumes that an error occurred in that position of the code. If the syndrome is nonzero and not equal to any column of the $\mathcal{H}$ matrix, then this incomplete decoding procedure fails.

Decoding failures and errors can occur only if two or more channel errors occur. For example, suppose

$$\mathcal{H} = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 \end{bmatrix}$$

If the received word is $\mathbf{R} = 101000101$, then the decoder may compute $\mathbf{s} = 1100$. Since $\mathbf{s}^t$ is equal to the fifth column of the $\mathcal{H}$ matrix, we decide that

$$\mathbf{E} = [000010000] \quad \text{and} \quad \mathbf{C} = [101010101]$$

If, instead, the received word is $\mathbf{R} = [111001111]$, then $\mathbf{s} = 1101$. Since $\mathbf{s}^t$ is not equal to any column of the $\mathcal{H}$ matrix, the decoding procedure fails, detecting two or more channel errors.

Let us now ask, "What is the longest block length possible for a linear single-error-correcting code having at most r check digits?" Since the number of check digits is equal to the number of linearly independent rows of the $\mathcal{H}$ matrix, this question is virtually identical to the question, "What is the maximum number of distinct nonzero columns which can occur in a binary matrix having at most r rows?" The answer is obviously $2^r - 1$. The columns of the $\mathcal{H}$ matrix must consist of the $2^r - 1$ nonzero binary r-tuples, arranged in any order. The linear code defined by such a matrix is called a *Hamming code.*

For every r, there exists a binary Hamming code containing r check digits. The block length of the code is $n = 2^r - 1$, and the number of information digits is given by $k = n - r = 2^r - 1 - r$. The code is able to correct a single error in any digit. Furthermore, since every possible nonzero syndrome is equal to some column of the $\mathcal{H}$ matrix, decoding failure never occurs. The single-error decoding procedure is complete. Every coset has a leader whose weight is either zero or one. No error pattern of weight ≥ 2 can be detected or corrected.

In order to provide protection against error patterns of weight 2, an additional overall parity check is sometimes annexed to a Hamming code. The resulting code has block length $n = 2^m$, $r = m + 1$ check digits, and $k = 2^m - 1 - m$ information digits. For example, the Hamming code of block length 15 has the parity-check matrix

$$\mathcal{H} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

Annexing an overall parity check gives the *extended* Hamming code of block length 16, whose parity-check matrix is

$$\mathcal{H} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

Every column of the parity-check matrix of an extended Hamming code has a 1 in its top row, and the sum of any two columns has a 0 in its top row. For this reason, the syndrome of any error pattern of weight 2 will be different from any column of the $\mathcal{H}$ matrix. It is also nonzero. Any pattern of two channel errors must cause a decoding failure. Thus, the extended Hamming codes are able to correct all channel-error patterns of weight ≤ 1, and detect all channel-error patterns of weight 2.

Although the columns of the $\mathcal{H}$ matrix of a Hamming code may be arranged in any order, some arrangements lead to much simpler instrumentations than others. For this reason, we postpone all questions concerning the implementation of Hamming codes until Sec. 5.1.

The Hamming code having block length $n = 2^m - 1$ has rate given by

$$R = \frac{k}{n} = \frac{2^m - 1 - m}{2^m - 1} = 1 - \frac{m}{2^m - 1}$$

Although we can construct Hamming codes having larger and larger block lengths, the longer codes have rates approaching 1. Thus, although they are a major step forward from the single-parity-check codes, the Hamming codes are still basically a class of very high-rate codes. They are unable to correct any pattern of two or more errors. In order to correct such error patterns, we must search for codes having a greater number of check digits.

The relative complexity of the simplest known double-error-correcting binary codes contrasts sharply with the simplicity of the Hamming codes. A great deal of work in constructive coding theory followed the appearance of Hamming's pioneering paper in 1950. Many new types of codes were introduced, but, with a few important exceptions, most of the new constructions were specialized codes for very specialized purposes. Even the next simplest problem, the construction of relatively high-rate binary codes which would correct any pattern of double errors, remained unsolved. When double-error-correcting codes finally were discovered by Bose and Chaudhuri in 1960 and Hocquenghem in 1959, the generalization to t-error-correcting codes followed immediately, for all t.

In many ways, the gap between the Hamming codes of 1950 and the BCH codes of 1960 represents even more than a decade of research. In fact, most of Hamming's results had been anticipated in a slightly different context by Fisher in 1942, in a paper which was well known to Bose!

In any case, the conceptual gap between the Hamming codes and the double-error-correcting BCH codes is considerable. For that reason, the student is urged to study the next section with care. In my opinion, it is the most important section of this book.

1.4 MANIPULATIVE INTRODUCTION TO DOUBLE-ERROR-CORRECTING BCH CODES

We have seen that a linear code is characterized by its parity-check matrix $\mathcal{H}$. We have also seen that the syndrome of the received sequence is the sum of the columns of $\mathcal{H}$ corresponding to the error positions. Hence, a linear code is capable of correcting all single-error patterns iff all columns of $\mathcal{H}$ are different and nonzero. If $\mathcal{H}$ has m rows and can correct single errors, then $n \leq 2^m - 1$. The Hamming codes achieve this bound.

Each digit of a Hamming code may be labeled by a nonzero binary m-tuple, which is equal to the corresponding column of the $\mathcal{H}$ matrix. The m syndrome digits then reveal directly the label of the error (if there is only one) or the binary vector sum of the labels (if there are several).

This labeling idea is so useful that we shall continue to assume that

$$\boxed{n = 2^m - 1}$$

and that the columns of $\mathcal{H}$ have been labeled accordingly. Now suppose that we wish to correct all patterns of two or fewer errors. Obviously we need a greater redundancy; that is, $\mathcal{H}$ must have more rows. Proceeding naïvely, we suspect that we may need about twice as many parity checks to correct two errors as we need to correct one, so we shall try to find a parity-check matrix $\mathcal{H}$ with $2^m - 1$ columns and $2m$ rows.

Since we suspect that it will be a good idea to make the columns of $\mathcal{H}$ as different as possible, we start out by choosing the first m rows to correspond to our Hamming code. (Why not? It worked for Hamming.) For example, with $m = 5$, $n = 31$, we would like to have a double-error-correcting code with

$$\mathcal{H} = \begin{bmatrix}
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \cdots 1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \cdots 1 & 1 \\
0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \cdots 1 & 1 \\
0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \cdots 1 & 1 \\
1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \cdots 0 & 1 \\
f_1(1) & f_1(2) & f_1(3) & \cdots & & & & & & & & & & & & & \\
f_2(1) & f_2(2) & f_2(3) & & & & & & & & & & & & & & \\
f_3(1) & f_3(2) & f_3(3) & & & & & & & & & & & & & & \\
f_4(1) & f_4(2) & f_4(3) & & & & & & & & & & & & & & \\
f_5(1) & f_5(2) & f_5(3) & & & & & & & & & & & & & &
\end{bmatrix} \tag{1.41}$$

Considered in this way, we seek a function $f(\xi)$ which maps nonzero 5-tuples onto 5-tuples.

The last five rows of this matrix will form a Hamming code iff the function f is a permutation. Heuristically, this prospect seems hopeful. If either the first m checks or the second m checks can correct a single error, then perhaps somehow they can jointly correct two errors.

In order to investigate the possible choices of our function f, we shall have to develop some facility for manipulating binary 5-tuples. We would like to be able to add, subtract, multiply, and divide 5-tuples, in some sense. In order to do this, it is convenient to interpret each 5-tuple as a binary polynomial of degree ≤ 4, for example,

$$
\begin{array}{lll}
0\ 0\ 0\ 0\ 0 \leftrightarrow & & 0 \\
0\ 0\ 0\ 0\ 1 \leftrightarrow & & 1 \\
0\ 0\ 0\ 1\ 0 \leftrightarrow & & x \\
0\ 1\ 0\ 1\ 1 \leftrightarrow & x^3 & + x + 1 \\
1\ 1\ 0\ 1\ 1 \leftrightarrow & x^4 + x^3 & + x + 1
\end{array}
$$

We notice that the addition of two such binary polynomials corresponds to vector addition of the binary 5-tuples, as we desired. Multiplication, however, generally gives us a polynomial of degree greater than 4. For example,

$$
\begin{aligned}
&(x^3 + x + 1)(x^4 + x^3 + x + 1) \\
&\qquad = x^7 + x^6 + x^5 + 3x^4 + 2x^3 + x^2 + 2x + 1 \\
&\qquad = x^7 + x^6 + x^5 + x^4 + x^2 + 1
\end{aligned}
$$

since in binary $2 = 0$ and $3 = 1$.

Clearly we need some method of reducing a polynomial of degree >4 back to a polynomial of degree ≤ 4. One obvious method is to reduce modulo some binary polynomial $M(x)$ of degree 5; that is, we divide by $M(x)$ and keep only the remainder. For example, let

$$M(x) = x^5 + x^2 + 1$$

Dividing (in binary) gives

$$
\begin{array}{r|l}
 & x^2 + x + 1 \\
\hline
x^5 + x^2 + 1 & x^7 + x^6 + x^5 + x^4 + x^2 + 1 \\
 & x^7 + x^4 + x^2 \\
\hline
 & x^6 + x^5 + 1 \\
 & x^6 + x^3 + x \\
\hline
 & x^5 + x^3 + x + 1 \\
 & x^5 + x^2 + 1 \\
\hline
 & x^3 + x^2 + x
\end{array}
$$

so that

$$x^7 + x^6 + x^5 + x^4 + x^2 + 1 = (x^2 + x + 1)(x^5 + x^2 + 1) + x^3 + x^2 + x$$

or

$$x^7 + x^6 + x^5 + x^4 + x^2 + 1 \equiv x^3 + x^2 + x \bmod x^5 + x^2 + 1$$

The symbol $\equiv$ is read "is congruent to." In general, we shall say that

$$A(x) \equiv a(x) \bmod M(x)$$

iff there exists a polynomial $c(x)$ such that

$$A(x) = c(x)M(x) + a(x)$$

Notice that all the multiplications and additions of polynomials are performed in binary. Some authors emphasize this fact by writing $A(x) \equiv a(x) \bmod (2, M(x))$. Note that it makes no difference whether one first reduces the coefficients mod 2 then the polynomial mod $M(x)$ or vice versa.

The usefulness of this definition is due to the following properties.

Congruence properties 1.42 If

$$a(x) \equiv A(x) \bmod M(x)$$

and

$$b(x) \equiv B(x) \bmod M(x)$$

then

$$a(x) \pm b(x) \equiv A(x) \pm B(x) \bmod M(x)$$

and

$$a(x)b(x) \equiv A(x)B(x) \bmod M(x)$$

Proofs

$$a(x) = c(x)M(x) + A(x)$$
$$b(x) = d(x)M(x) + B(x)$$
$$a(x) \pm b(x) = [c(x) \pm d(x)]M(x) + A(x) \pm B(x)$$
$$a(x)b(x) = [c(x)d(x)M(x) + c(x)B(x) + d(x)A(x)]M(x) + A(x)B(x)$$

Q.E.D.

Furthermore, if $a(x)$ and $A(x)$ both have degrees less than $M(x)$, and if $a(x) \equiv A(x) \bmod M(x)$, then $a(x) = A(x)$. [Proof: $a(x) - A(x) =$

$c(x)M(x)$, but no nonzero multiple of $M(x)$ can have degree less than the degree of $M(x)$.]

Hence every polynomial of degree less than $M(x)$ lies in a distinct residue class mod $M(x)$. This gives $2^{\deg M}$ different residue classes. Are there any others? No, because of the following algorithm.

Division algorithm 1.43

For integers†	*For polynomials with coefficients in some given field*‡
Given a and M, $\exists$ unique integers q and A such that	Given $a(x)$ and $M(x)$, $\exists$ unique polynomials $q(x)$ and $A(x)$ such that
$a = qM + A \qquad 0 \leq A < M$	$a(x) = q(x)M(x) + A(x)$ $\deg A(x) < \deg M(x)$

Having shown that we can add, subtract, and multiply mod $M(x)$, we wonder when it is possible to divide. This question is answered by a corollary to Euclid's algorithm.

Euclid's algorithm 1.44

For integers	*For polynomials with coefficients in some given field*
Given a and b, $\exists$ integers A and B such that	Given $a(x)$ and $b(x)$, $\exists$ polynomials $A(x)$ and $B(x)$ such that
$aA + bB = (a,b)$	$a(x)A(x) + b(x)B(x) = (a(x),b(x))$
where (a,b) is the greatest common divisor of a and b.	where $(a(x),b(x))$ is the monic common factor of $a(x)$ and $b(x)$ with greatest degree.

The detailed proof, by iteration of the division algorithm, will be given in Sec. 2.1.

In particular, it is not always legitimate to divide by $a(x)$ mod $M(x)$ if $a(x)$ and $M(x)$ have a common factor $d(x)$. For if $a(x) = r(x)d(x)$ and

† The symbol $\exists$ means "there exists."

‡ A field is a set of elements, including 0 and 1, any pair of which may be added, subtracted, multiplied, or divided (except by 0) to give a unique result in the field. A further study of fields will be undertaken in Chap. 4.

$M(x) = s(x)d(x)$, then $a(x)s(x) = r(x)M(x) \equiv 0 \bmod M(x)$. If it were possible to divide by $a(x)$, we could then deduce that $s(x) \equiv 0 \bmod M(x)$, which is not true.

However, if $(a(x),b(x)) = 1$, then, according to Euclid, we can find $A(x)$ and $B(x)$ so that

$$a(x)A(x) + b(x)B(x) = 1$$

Whence $a(x)A(x) \equiv 1 \bmod b(x)$.

Evidently division by $a(x)$ is equivalent to multiplication by $A(x)$.

In particular, if $M(x)$ is an *irreducible polynomial*, i.e., it has no divisors other than scalars and scalar multiples of itself, then for every $a(x) \not\equiv 0$, we may divide by $a(x) \bmod M(x)$. As we shall prove in Sec. 2.1, polynomials may be factored into products of irreducible polynomials in a unique way, just as integers may be uniquely factored into products of primes.

Let us now verify that the binary polynomial

$$M(x) = x^5 + x^2 + 1$$

is indeed irreducible. Obviously x is not a factor. Let us try $x + 1$

$$\begin{array}{r|l} x+1 & x^4 + x^3 + x^2 \\ & \hline x^5 + x^2 + 1 \\ x^5 + x^4 & \\ \hline x^4 & \\ x^4 + x^3 & \\ \hline x^3 + x^2 & \\ x^3 + x^2 & \\ \hline 1 & \end{array}$$

Since it has a remainder of 1, it fails. Having no success with linear factors, we try quadratics. Clearly x^2 fails, as does $x^2 + x = x(x + 1)$ and $x^2 + 1 = x^2 + 2x + 1 = (x + 1)^2$. So we try the only other possibility,

$$\begin{array}{r|l} x^2+x+1 & x^3 + x^2 \\ & \hline x^5 + x^2 + 1 \\ x^5 + x^4 + x^3 & \\ \hline x^4 + x^3 + x^2 & \\ x^4 + x^3 + x^2 & \\ \hline 1 & \end{array}$$

Hence, all factors must have degrees ≥ 3. But the product of any two such factors has degree ≥ 6. We deduce that our $M(x)$ is irreducible.

We conclude that we may add, multiply, and divide (except by

zero) mod $x^5 + x^2 + 1$. By interpreting all binary 5-tuples as residue classes of $M(x)$, we have considerably more reasonable options of choosing the function f which will tell us how to choose the second five rows of our parity-check matrix so as to get a double-error-correcting binary group code with block length 31 and rate 21/31.

Let us suppose that the two errors occur in positions β_1 and β_2. Writing β_1 and β_2 in binary and considering them as residue classes mod $M(x)$, we shall have the correspondence $\beta_i \leftrightarrow \beta^{(i)}(x)$, where the $\beta^{(i)}(x)$ are binary polynomials of degree less than 5.

The first five parity-check equations will tell us $\beta_1 + \beta_2$; the second set of parity-check equations will tell us $f(\beta_1) + f(\beta_2)$.

The decoder's problem will be to determine β_1 and β_2, given

$$\beta_1 + \beta_2 = \zeta_1$$

and

$$f(\beta_1) + f(\beta_2) = \zeta_2$$

Let us now consider several possible choices for f. The simplest possibility would be a constant multiplication

$$f(\beta) \equiv a(x)\beta(x) \bmod M(x)$$

This idea is clearly a poor one, since in this case we have $\zeta_2 = a\zeta_1$, whence the two equations are dependent. The second five parity checks (giving ζ_2) tell the decoder nothing new.

Similarly, $f(\beta) = \beta + a$ is an unfruitful idea, for any a. In that case, $\zeta_2 = \zeta_1$.

Next, we consider powers. Initially we try

$$f(\beta) = \beta^2$$

The two equations for the decoder are then

$$\begin{aligned} \beta_1 + \beta_2 &= \zeta_1 \\ \beta_1^2 + \beta_2^2 &= \zeta_2 \end{aligned}$$

To the reader unfamiliar with fields of characteristic 2 (which we discuss in Chap. 4) these may look like two independent equations in two unknowns. However, appearances are deceptive, for

$$\begin{aligned} \zeta_1^2 = (\beta_1 + \beta_2)^2 &= \beta_1^2 + 2\beta_1\beta_2 + \beta_2^2 \\ &= \beta_1^2 + \beta_2^2 = \zeta_2 \end{aligned}$$

Hence, $\zeta_2 = \zeta_1^2$. The second equation is the square of the first, so the second five parity-check equations still tell us nothing new.

Not to be discouraged, however, we try

$$f(\beta) = \beta^3$$

The decoder's equations then become

$$\beta_1 + \beta_2 = \zeta_1$$
$$\beta_1^3 + \beta_2^3 = \zeta_2$$

Manipulating, we have

$$\begin{aligned}\zeta_2 = \beta_1^3 + \beta_2^3 &= (\beta_1 + \beta_2)(\beta_1^2 - \beta_1\beta_2 + \beta_2^2) \\ &= \zeta_1(\beta_1^2 + \beta_1\beta_2 + \beta_2^2) \\ &= \zeta_1(\beta_1\beta_2 - \zeta_1^2)\end{aligned}$$

so that

$$\beta_1 + \beta_2 = \zeta_1$$
$$\beta_1\beta_2 = \zeta_1^2 + \frac{\zeta_2}{\zeta_1} \qquad \text{if} \qquad \zeta_1 \neq 0$$

Both β_1 and β_2 must satisfy the quadratic equation

$$\beta(\zeta_1 + \beta) = \zeta_1^2 + \frac{\zeta_2}{\zeta_1}$$
$$\beta^2 + \zeta_1\beta + \left(\zeta_1^2 + \frac{\zeta_2}{\zeta_1}\right) = 0$$

or

$$1 + \zeta_1\beta^{-1} + \left(\zeta_1^2 + \frac{\zeta_2}{\zeta_1}\right)\beta^{-2} = 0 \tag{1.45}$$

Hence, the two errors are given by the two roots of this quadratic equation. If exactly two errors were made, their location numbers both satisfy this equation. Since this equation then has two roots among the field of binary polynomials mod $M(x)$, the decoder will always be able to find the two error location numbers.

If there was only one error, then

$$\beta_1 = \zeta_1 \qquad \beta_1^3 = \zeta_2$$

Hence, in this case the lone error satisfies the equation

$$\beta + \zeta_1 = 0 \qquad \text{or} \qquad 1 + \zeta_1\beta^{-1} = 0$$

Finally, the decoder can always decode if there were no errors, because in that case

$$\zeta_1 = \zeta_2 = 0$$

For reasons which will become apparent later (Sec. 7.2), it is more convenient to deal with polynomials whose roots are the *reciprocals*, i.e., multiplicative inverses, of the locations of the errors, rather than the locations of the errors themselves. This so-called *error-locator poly-*

nomial, or *locator*, is denoted by $\sigma(z)$ and defined by

$$\sigma(z) = \prod_{\substack{\beta = \text{error} \\ \text{locations}}} (1 - \beta z) \tag{1.46}$$

It is also convenient to use the symbol S_k to denote the sum of the kth powers of the locations of the errors. In the present case we have

$$S_1 = \Sigma\beta = \zeta_1 \qquad S_3 = \Sigma\beta^3 = \zeta_2$$

In this notation, our expressions for the locator become

$$\sigma(z) = \begin{cases} 1 & \text{if there are 0 errors, in which case } S_1 = S_3 = 0 \\ 1 + S_1 z & \text{if there is 1 error, in which case } S_1 \neq 0;\ S_3 = S_1^3 \\ 1 + S_1 z + \left(S_1^2 + \dfrac{S_3}{S_1}\right) z^2 & \text{if there are 2 errors, in which case } S_1 \neq 0;\ S_3 \neq S_1^3 \end{cases} \tag{1.47}$$

Since these cases can be distinguished by whether S_1 and/or $S_3 + S_1^3$ are 0, it is evident that the decoder can determine the error locator if there are no more than two errors. If there are three or more errors, then this decoding algorithm will decode incorrectly or fail to decode. For example, if $S_1 = 0$ but $S_3 \neq 0$, the decoding algorithm fails, detecting three or more errors. In Chap. 16, we shall see that this deficiency can be repaired by Algorithm 16.481.

The choice of $f(\xi) = \xi^3$ is thus seen to provide a suitable method of choosing the lower five rows of the $\mathfrak{IC}$ matrix in order to yield a double-error-correcting binary code of block length 31, having 10 parity checks. The first five parity checks then give the sum of the error location numbers; the second five parity checks give the sum of the cubes of the error location numbers. The decoding procedure consists of three major steps: (1) compute the parity checks, obtaining S_1 and S_3; (2) compute the error polynomial $\sigma(z)$; (3) find the reciprocals of the roots of $\sigma(z)$ and change the corresponding positions of the received word.

For the first major step of the decoding procedure, we might imagine 10 separate sets of mod 2 adders, each having as inputs a certain subset of the 31 received digits. Since major step 1 is conceptually simple, we postpone consideration of several known shortcuts of these parity-check calculations until Sec. 5.2.

The second major step evidently requires the instrumentation of the addition, multiplication, division, and squaring operations over the

field of binary polynomials mod $M(x)$. We shall devote Chap. 2 to a more detailed investigation of these arithmetic operations.

The third major step requires the decoder to find the reciprocal roots of an algebraic equation over the field of binary polynomials mod $M(x)$. We shall consider the details of this step in Sec. 5.4.

PROBLEMS

1.1. Consider the three Hamming codes defined by these parity-check matrices:

$$\text{I:}\quad \mathfrak{IC} = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

$$\text{II:}\quad \mathfrak{IC} = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{III:}\quad \mathfrak{IC} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

(*a*) For each of these three codes, decode the following received words: $\mathbf{R}$ = [1110000], $\mathbf{R}$ = [1111000].

(*b*) Show that two of these three matrices define identical codes. *Hint:* Show that the rows of either are linear combinations of rows of the other.

1.2. How many codewords are there in the codes defined by each of the following parity-check matrices?

$$\text{I:}\quad \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

$$\text{II:}\quad \begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 \end{bmatrix}$$

1.3. For the code defined by the parity-check matrix II of Prob. 1.2, find the leader of the coset containing the following word:

(*a*) $\mathbf{R}$ = [111101000]
(*b*) $\mathbf{R}$ = [110101011]
(*c*) $\mathbf{R}$ = [100010001]
(*d*) $\mathbf{R}$ = [010010010]

1.4. Fill in the last five rows of the matrix in Eq. (1.41).

1.5. If the received word (using the code of Prob. 1.4) is

01101011101101011100010110111111

what is S_1? What is S_3? If there are no more than two errors, what is the error-locator polynomial?

1.6. (*a*) Verify that the binary polynomial $x^4 + x^3 + x^2 + x + 1$ is irreducible.

(*b*) Using this polynomial, construct the $\mathfrak{IC}$ matrix for a double-error-correcting BCH code with block length $n = 15$ having $r = 8$ check digits and $k = 7$ information digits.

2

Arithmetic Operations Modulo an Irreducible Binary Polynomial

2.1 A CLOSER LOOK AT EUCLID'S ALGORITHM

In the previous section we indicated that the decoding of binary BCH codes requires arithmetic operations in the field of binary polynomials mod some irreducible binary polynomial $M(x)$. From both the theoretical and practical standpoints, Euclid's algorithm plays a key role in this development.

From the theoretical standpoint, Euclid's algorithm is used to prove that the factorization of polynomials into irreducible polynomials is unique (except for scalar multiples) over any field and that a polynomial of degree d cannot have more than d roots in any field. This fact is needed to prove that the error locator polynomial $\sigma(z)$ cannot have more roots than its degree. If it did, then the entire decoding procedure sketched in Sec. 1.4 would be invalid, for several different pairs of error locations might conceivably be reciprocal roots of the same quadratic equation.

From the practical standpoint, Euclid's algorithm is important

because one of its modifications, the method of convergents of continued fractions, provides the basis for one of the most efficient methods for implementing division in finite fields. This method, apparently new, will be detailed in this section and the next.

Euclid's algorithm is based on the observation that any divisor of R and r must also divide their sum and their difference. Furthermore, since any divisor of r also divides any nonzero multiple of r, such as ar, then any divisor of R and r must also divide $R \pm ar$. Conversely, any divisor of r and $R \pm ar$ must also divide $(R \pm ar) \mp ar = R$. Hence, if we let (R,r) denote the greatest common divisor (hereafter called gcd) of R and r, then we have $(R,r) = (r, R \pm ar)$. Consequently, starting from an original pair of elements R and r, we can find a new pair of elements which have the same gcd. If the multiplier a is judiciously chosen, the problem of finding the gcd of the new pair of elements will be easier than the original problem.

Notice that the argument of the preceding paragraph applies both to integers and to polynomials with coefficients in any given field. In the former case, the gcd is taken to mean the largest of the integers which are common factors; in the latter case, the gcd is taken to mean a common factor of highest degree. The uniqueness of the gcd (up to a constant factor) for polynomials will follow later as a consequence of the algorithm. In either case, the essential properties of the gcd are

$$(R,r) = (r,R) \qquad (R,r) = (r, R \pm ar) \qquad (r,0) = r$$

In the case of integers, the usual choice of the multiplier a, assuming $r < R$, is $a = [R/r]$ (which denotes the greatest integer less than or equal to R/r). This choice yields the result that $0 \leq R - ar < r$. More generally, if we are given the integers r_{n-1} and r_{n-2}, with $r_{n-1} \leq r_{n-2}$, then we may define a_n and r_n so that

$$r_{n-2} = a_n r_{n-1} + r_n \qquad 0 \leq r_n < r_{n-1}$$

which is accomplished by the choice

$$a_n = \left[\frac{r_{n-2}}{r_{n-1}}\right]^{\dagger} \qquad r_n = r_{n-2} - a_n r_{n-1}$$

Starting from the initial pair of numbers $R = r_{-2}$ and $r = r_{-1}$, we have

$$\begin{aligned}
& & 0 < r_{-1} < r_{-2} \\
r_{-2} &= a_0 r_{-1} + r_0 & 0 < r_0 < r_{-1} \\
r_{-1} &= a_1 r_0 + r_1 & 0 < r_1 < r_0 \\
r_0 &= a_2 r_1 + r_2 & 0 < r_2 < r_1 \\
& \cdots\cdots\cdots\cdots\cdots\cdots & \\
r_k &= a_{k+2} r_{k+1} + r_{k+2} & 0 < r_{k+2} < r_{k+1} \\
& \cdots\cdots\cdots\cdots\cdots\cdots & \\
r_{n-2} &= a_n r_{n-1} + 0 & 0 = r_n < r_{n-1}
\end{aligned}$$

† Here $[x]$ denotes the greatest integer not exceeding x.

Since the r's form a decreasing sequence of nonnegative integers, they must eventually terminate when, for some n, $r_n = 0$. Evidently,

$$(r_{-2},r_{-1}) = (r_{-1},r_0) = \cdots = (r_{n-1},0) = r_{n-1}$$

Next let us find sequences of b's and B's such that

$$r_{n-1} = b_k r_k + B_k R_k \qquad \text{for all } k$$

where here we have set $R_k = r_{k-1}$. Since $r_k = r_{k-2} - a_k r_{k-1}$, we have $r_k = R_{k-1} - a_k r_{k-1}$, so that

$$\begin{aligned} b_k r_k + B_k R_k &= b_k(R_{k-1} - a_k r_{k-1}) + B_k r_{k-1} \\ &= (B_k - a_k b_k) r_{k-1} + b_k R_{k-1} \end{aligned}$$

Hence, we may start by setting $B_{n-1} = 0$, $b_{n-1} = 1$ and then work backwards, with

$$B_{k-1} = b_k \qquad b_{k-1} = B_k - a_k b_k$$

Working all the way back, we eventually succeed in obtaining B_{-1} and b_{-1} which satisfy the equation

$$B_{-1} r_{-2} + b_{-1} r_{-1} = r_{n-1} = (r_{-2},r_{-1})$$

In this form of the algorithm, one computes a_0, r_0, a_1, r_1, . . . , a_{n-1}, r_{n-1}, a_n, $r_n = 0$ and then $b_n = 0$, $b_{n-1} = 1$, b_{n-2}, b_{n-3}, . . . , b_1, b_0, b_{-1}. ($B_{-1} = b_0$.) The weakness of this method is that all the partial results a_0, r_0, a_1, r_1, . . . , a_{n-1}, r_{n-1} must be remembered in order to compute the desired multipliers b_0 and b_{-1}. If the algorithm starts with a pair of large numbers, this list of partial results may run to a considerable length.

An alternate procedure which avoids the storage of partial results is given in Algorithm 2.11.

Another version of Euclid's algorithm 2.11 Given r_{-2} and r_{-1}. Define

$$\begin{aligned} p_{-2} &= 0 \qquad p_{-1} = 1 \\ q_{-2} &= 1 \qquad q_{-1} = 0 \end{aligned}$$

and then compute a_k, r_k, p_k, and q_k by the rules

$$\begin{aligned} r_{k-2} &= a_k r_{k-1} + r_k \qquad 0 \le r_k < r_{k-1} \\ p_k &= a_k p_{k-1} + p_{k-2} \\ q_k &= a_k q_{k-1} + q_{k-2} \end{aligned}$$

Stop when $r_n = 0$.

We may easily verify that

$$\begin{aligned} q_k r_{k+1} + q_{k+1} r_k &= q_k(-a_{k+1} r_k + r_{k-1}) + (a_{k+1} q_k + q_{k-1}) r_k \\ &= q_{k-1} r_k + q_k r_{k-1} \end{aligned}$$

and that

$$p_k r_{k+1} + p_{k+1} r_k = p_k(-a_{k+1} r_k + r_{k-1}) + (a_{k+1} p_k + p_{k-1}) r_k$$
$$= p_{k-1} r_k + p_k r_{k-1}$$

and that

$$q_{k+1} p_k - p_{k+1} q_k = (a_{k+1} q_k + q_{k-1}) p_k - (a_{k+1} p_k + p_{k-1}) q_k$$
$$= -(q_k p_{k-1} - p_k q_{k-1})$$

Starting from $k = -1$ and proceeding by induction according to these three formulas, we conclude that, for all $k \geq -1$,

$$q_{k-1} r_k + q_k r_{k-1} = r_{-1}$$
$$p_{k-1} r_k + p_k r_{k-1} = r_{-2}$$
$$q_k p_{k-1} - p_k q_{k-1} = (-1)^k$$

Upon applying these formulas with $k = n$, we have $r_n = 0$, and therefore

$$q_n r_{n-1} = r_{-1}$$
$$p_n r_{n-1} = r_{-2}$$
$$r_{n-1}(q_n p_{n-1} - p_n q_{n-1}) = (-1)^n r_{n-1}$$

$$\boxed{r_{-1} p_{n-1} - r_{-2} q_{n-1} = (-1)^n r_{n-1} = (-1)^n (r_{-2}, r_{-1})} \qquad (2.12)$$

Furthermore, we can make certain statements about the magnitudes of p_{n-1} and q_{n-1}. Since $a_n > 0$, it is evident that

$$p_{n-1} < p_n = \frac{r_{-2}}{r_{n-1}} \quad \text{and} \quad q_{n-1} < q_n = \frac{r_{-1}}{r_{n-1}}$$

Notice that in this approach, at most seven numbers need be remembered at each step: two r's, two p's, two q's, and one a. There is no need to recall any of the other partial results. When the algorithm terminates with $r_n = 0$, the desired multipliers p_{n-1} and q_{n-1} as well as the gcd (r_{n-1}) have all been computed.

This method is called the *continued-fractions version* of Euclid's algorithm. The reason for this nomenclature is that

$$\frac{r_{-2}}{r_{-1}} = a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cfrac{1}{a_3 + \cfrac{1}{\ddots + \cfrac{1}{a_n}}}}}$$

It can also be shown that the quotients p_k/q_k represent the successive convergents of this continued fraction; that is,

$$\frac{p_k}{q_k} = a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cfrac{1}{\cdots + \cfrac{1}{a_k}}}}$$

The reader interested in learning more about continued fractions will find an excellent introduction to this theory in chap. 5 of McCoy (1965). None of those results will be required here.

Euclid's algorithm for polynomials is very similar to Euclid's algorithm for integers. We start from the initial polynomials

$$r^{(-2)} \quad \text{and} \quad r^{(-1)}$$

which denote

$$r^{(-2)}(x) = \sum_i r_i^{(-2)} x^i \quad \text{and} \quad r^{(-1)}(x) = \sum_i r_i^{(-1)} x^i$$

Here $r_i^{(-2)}$ and $r_i^{(-1)}$ are elements in a given field, and deg $r^{(k)}$ denotes the largest i for which $r_i^{(k)} \neq 0$. If deg $r^{(k-1)} <$ deg $r^{(k-2)}$, then we may use the division algorithm for polynomials to compute $a^{(k)}$ and $r^{(k)}$ which satisfy

$$r^{(k-2)} = a^{(k)} r^{(k-1)} + r^{(k)} \qquad \deg r^{(k)} < \deg r^{(k-1)}$$

The details of the division algorithm for polynomials are slightly more complicated than the details of the division algorithm for integers. In either case, we have a dividend D and a divisor d and we wish to find a quotient Q and a remainder R. As it arises in Euclid's algorithm, $D = r^{(k-2)}$; $d = r^{(k-1)}$; $Q = a^{(k)}$, and $R = r^{(k)}$. We must find Q and R so that deg $R <$ deg d. This is accomplished by an iteration. We start by setting $R^{(0)} = D$; $Q^{(0)} = 0$. Then if deg $R^{(n)} \geq$ deg d, we set

$$\begin{aligned} \Delta^{(n)} &= \frac{\text{leading coefficient of } R^{(n)}}{\text{leading coefficient of } d} \, x^{\deg R^{(n)} - \deg d} \\ R^{(n+1)} &= R^{(n)} - \Delta^{(n)} d \\ Q^{(n+1)} &= Q^{(n)} + \Delta^{(n)} \end{aligned} \tag{2.13}$$

Clearly deg $R^{(n+1)} <$ deg $R^{(n)}$, and $dQ^{(n+1)} + R^{(n+1)} = dQ^{(n)} + R^{(n)}$.

The procedure may be continued until $\deg R^{(n)} < \deg d$. We then take

$$Q = Q^{(n)} \qquad R = R^{(n)}$$

Since $dQ^{(n)} + R^{(n)}$ is independent of n, it is equal to $dQ^{(0)} + R^{(0)} = R^{(0)} = D$, as required.

Euclid's algorithm for polynomials thus consists of a sequence of iterated steps, each of which computes a new quotient $a^{(k)}$ and a new remainder $r^{(k)}$. Each of these divisions is itself an iteration, as just described. In the next section, however, we shall see that these iterations and subiterations can be combined into a unified overall procedure which is readily implemented.

By performing successive divisions, Euclid's algorithm for polynomials generates a sequence of remainder polynomials having smaller and smaller degrees. Eventually this sequence must terminate with $r^{(n)} = 0$. We then have

$$(r^{(-2)},r^{(-1)}) = (r^{(-1)},r^{(0)}) = \cdots = (r^{(n-1)},0) = r^{(n-1)}$$

As with integers, the conventional practice is then to work backward to obtain the polynomials $b^{(n-2)}$, $b^{(n-1)}$, . . . , $b^{(0)}$, and $b^{(-1)}$ which satisfy the equation

$$b^{(0)}r^{(-2)} + b^{(-1)}r^{(-1)} = r^{(n-1)}$$

As in the case of integers, this conventional method has the serious liability that it may require an enormous scratch pad of storage to remember all the r's and a's. For this reason, the continued-fractions version again provides an attractive alternative. Defining

$$p^{(-2)} = 0 \qquad p^{(-1)} = 1$$
$$q^{(-2)} = 1 \qquad q^{(-1)} = 0$$

we compute

$$p^{(k)} = a^{(k)}p^{(k-1)} + p^{(k-2)}$$

and

$$q^{(k)} = a^{(k)}q^{(k-1)} + q^{(k-2)}$$

As with integers, the p's and q's may again be computed along with the a's and r's, thus avoiding the need for a large scratch storage. As with integers, the algorithm will eventually conclude with

$$r^{(n)} = 0$$
$$p^{(n)}r^{(n-1)} = r^{(-2)}$$
$$q^{(n)}r^{(n-1)} = r^{(-1)}$$
$$r^{(-1)}p^{(n-1)} - r^{(-2)}q^{(n-1)} = (-1)^n r^{(n-1)}$$

Since deg $a^{(n)} > 0$, it is evident that

$$\deg p^{(n-1)} < \deg \frac{r^{(-2)}}{r^{(n-1)}} \qquad \text{and} \qquad \deg q^{(n-1)} < \deg \frac{r^{(-1)}}{r^{(n-1)}}$$

Several consequences follow from Euclid's algorithm. First we observe that any common factor of $r^{(-2)}$ and $r^{(-1)}$ must also divide $r^{(-1)}p^{(n-1)}$ and $r^{(-2)}q^{(n-1)}$ and hence that any common factor of $r^{(-2)}$ and $r^{(-1)}$ must also divide $r^{(n-1)}$. This justifies calling $r^{(n-1)}$ the gcd. Second, we observe that if there were two different gcd's of $r^{(-2)}$ and $r^{(-1)}$, then each must divide the other. In the domain of integers, this implies uniqueness of the gcd. In the domain of polynomials over the field F, this implies that the gcd of two polynomials is unique except for a constant scalar factor, i.e., an element of F. This nonuniqueness may be removed by normalizing the gcd's to *monic* polynomials, whose leading coefficients are 1. We can then assert that any two polynomials over F have a unique monic gcd.

If f is a polynomial which has no divisors except scalars and scalar multiples of itself, then f is said to be an *irreducible polynomial.* From Euclid's algorithm, we can show that if an irreducible polynomial f divides the product of the polynomials $g^{(1)}$ and $g^{(2)}$, then either f divides $g^{(1)}$ or f divides $g^{(2)}$. For, if f does not divide $g^{(1)}$ then $(f,g^{(1)}) = 1$, and we can write $pf + qg^{(1)} = 1$. Multiplying by $g^{(2)}$ gives $pfg^{(2)} + qg^{(1)}g^{(2)} = g^{(2)}$. Since f divides both terms on the left side of this equation, it must divide $g^{(2)}$, as claimed. By induction, we may then deduce that if f is an irreducible polynomial which divides the product of several polynomials, $\prod_k g^{(k)}$, then for some k, f must divide $g^{(k)}$. Finally we can then deduce that if $f^{(i)}$ and $g^{(k)}$ are all irreducible and $\prod_i f^{(i)} = \prod_k g^{(k)}$, then each $f^{(i)}$ divides some $g^{(k)}$ and each $g^{(k)}$ divides some $f^{(i)}$. Thus, except for constant scalar factors, the factorization of any polynomial into irreducible polynomials is unique, ignoring different orderings of the factors. This is equivalent to Theorem 2.14.

Theorem 2.14 Any monic polynomial over the field F can be uniquely factored into monic irreducible polynomials over the field F.

This result holds for any field F. The analog of this theorem in the domain of integers states that any integer may be uniquely factored into the product of primes.

Since the degree of the product of several polynomials is the sum

of their degrees,† it is evident that every polynomial of degree 1 is irreducible. It is further evident that a polynomial of degree d cannot have more than d linear factors. Furthermore, we claim that the linear polynomial $x - \xi$ is a factor of the polynomial $f(x)$ iff $f(\xi) = 0$, that is, iff ξ is a *root* of the polynomial $f(x)$. For if we apply the division algorithm to the dividend $f(x)$, using $x - \xi$ for the divisor, we shall obtain a quotient $q(x)$ and a remainder r. Since the remainder will have degree smaller than the divisor, the remainder must be a constant, independent of x. Since

$$f(x) = (x - \xi)q(x) + r$$

is true for all x, we may set $x = \xi$, obtaining $r = f(\xi)$, as claimed. From this we deduce Theorem 2.15.

Theorem 2.15 A polynomial of degree d over the field F has at most d roots in any field containing F.

Some readers no doubt consider this assertion so obvious as to question the need of any proof of the sort we have just given. Perhaps such readers can acquire the appropriate amount of skepticism by observing that the claim is not true in general rings, which have addition, subtraction, and multiplication but no division. For example, the quadratic polynomial $x^2 - 1$ has four roots mod 15, namely 1, 4, 11, and 14.

This frequently happens with composite moduli. If p_i are primes and e_i are arbitrary positive integers, then the congruence $A \equiv B \bmod \prod_i p_i^{e_i}$ and the set of simultaneous congruences $A \equiv B \bmod p_i^{e_i}$ for all i are both equivalent to the statement that $A - B$ is a multiple of $\prod_i p_i^{e_i}$. Thus, the congruence $x^2 \equiv 1 \bmod 15$ is equivalent to the pair of congruences $x^2 \equiv 1 \bmod 5$ and $x^2 \equiv 1 \bmod 3$. Each of these has two solutions, so altogether we have four solutions, $x \equiv \pm 1 \bmod 3$ and $x \equiv \pm 1 \bmod 5$, with any combination of signs.

In general, congruences with respect to composite moduli may be broken down as follows.

The Chinese remainder theorem for integers 2.16 Given distinct primes $p_1, p_2, \ldots, p_k$ and integers $c_1, c_2, \ldots, c_k$, the simultaneous congruences $A \equiv c_i \bmod p_i^{e_i}$ have a unique solution mod $\prod_i p_i^{e_i}$.

† In order to have $\deg(fg) = \deg f + \deg g$ even when $f = 0$, we must define $\deg 0 = -\infty$.

The Chinese remainder theorem for polynomials 2.17 Given distinct irreducible polynomials $f^{(1)}(x), f^{(2)}(x), \ldots, f^{(k)}(x)$ and arbitrary polynomials $g^{(1)}(x), g^{(2)}(x), \ldots, g^{(k)}(x)$, the simultaneous congruences $h(x) \equiv g^{(i)}(x) \bmod [f^{(i)}(x)]^{e_i}$ have a unique solution for $h(x)$ mod $\prod_i f^{(i)}(x)^{e_i}$.

Proof for polynomials Since $\prod_{j \neq i} f^{(j)}(x)^{e_j}$ has no factors in common with $f^{(i)}(x)^{e_i}$, we may use Euclid's algorithm to find a polynomial $a^{(i)}(x)$ such that $a^{(i)}(x) \prod_{j \neq i} f^{(j)}(x)^{e_j} \equiv 1 \bmod f^{(i)}(x)^{e_i}$. We then set $h(x) = \sum_i g^{(i)}(x) a^{(i)}(x) \prod_{j \neq i} f^{(j)}(x)^{e_j}$. It is obvious that $h(x) \equiv g^{(i)}(x) \bmod f^{(i)}(x)^{e_i}$ for all i. If $H(x)$ also solves these simultaneous congruences, then $H(x) - h(x)$ is divisible by $f^{(i)}(x)^{e_i}$ for $i = 1, 2, \ldots k$, so $H(x) \equiv h(x) \bmod \prod_i f^{(i)}(x)^{e_i}$. Q.E.D.

The proof of the Chinese remainder theorem for integers is directly analogous to the above proof of the Chinese remainder theorem for polynomials.

The consequences of Euclid's algorithm for polynomials are much the same as the consequences of Euclid's algorithm for integers. In general, however, the details of the two are quite different. Despite the differences between the two, in one set of important special cases a direct correspondence can be made.

Theorem 2.18 If the integer d is the gcd of the integers m and n, then the polynomial $x^d - 1$ is the gcd of the polynomials $x^m - 1$ and $x^n - 1$, over any field.

Proof In general we have the rule for summing geometric series

$$\left(\sum_{i=0}^{a-1} y^i\right)(y - 1) = \sum_{i=1}^{a} y^i - \sum_{i=0}^{a-1} y^i = y^a - 1$$

Setting $y = x^{r_{k-1}}$ and multiplying by x^{r_k} gives

$$x^{r_k}\left[\sum_{i=0}^{a_k-1} (x^{r_{k-1}})^i\right](x^{r_{k-1}} - 1) = x^{a_k r_{k-1} + r_k} - x^{r_k}$$

So, corresponding to the integer division step $r_{k-2} = a_k r_{k-1} + r_k$,

we have the polynomial division

$$x^{r_{k-2}} - 1 = \left\{x^{r_k}\left[\sum_{i=0}^{a_k-1} (x^{r_{k-1}})^i\right]\right\} (x^{r_{k-1}} - 1) + (x^{r_k} - 1)$$

Consequently, the remainders $r^{(k)} = x^{r_k} - 1$ for all k. Euclid's algorithm for the polynomials continued in direct correspondence with Euclid's algorithm for integers. When $r_n = 0$, we have $r^{(n)} = x^0 - 1 = 0$. At the step before this, $r^{(n-1)} = x^d - 1$, which is the gcd of the original polynomials $x^m - 1$ and $x^n - 1$. Q.E.D.

Theorems 2.14 to 2.18 are very important, but the utility of the continued fractions version of Euclid's algorithm (2.11) is not confined to these theoretical results. This algorithm also provides the basis for the implementation of one of the arithmetic operations (division) necessary to decode BCH codes. In order to gain more insight into these operations, we now consider the manner in which they are implemented.

★† 2.2 LOGICAL CIRCUITRY

The three basic elements used in logical design are the AND gate, the OR gate, and the inverter, which are represented as shown in Fig. 2.01. The AND and OR gates may have several inputs, each of which carries a binary signal having either the value 0 or the value 1. The output of the AND gate is zero unless all its inputs are ones, in which case the output of the AND gate is also one. The output of the OR gate is one unless all of its inputs are zero, in which case the output of the OR gate is also zero. The inverter, in contrast to the AND and OR gates, has only one input, and its output is the opposite of its input. If its input signal has value 0, the output has value 1; if the input signal has value 1, the output has value 0.

In practice, circuits having the logical properties of these three elements may be constructed out of transistors, resistors, diodes, vacuum tubes, and/or other components. Depending on the detailed properties

† Starred sections of this book may be skimmed or omitted on first reading.

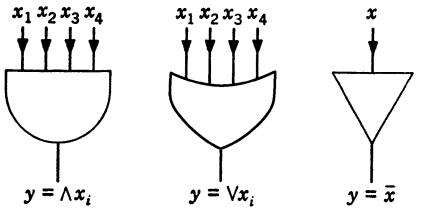


Fig. 2.01 AND, OR, and NOT elements.

of these components, the overall design will be subject to certain restrictions, called *design constraints*. For example, there will be maximum numbers of inputs to AND and OR gates and a maximum number of elements through which signals can propagate successively without additional amplification. Typically, every inverter is equipped with an amplifier, but AND and OR gates are not. Design constraints then specify how many AND and/or OR gates may be successively encountered between inverters and in what orders. Since the design constraints depend heavily on the properties of the components, we shall not consider design constraints much further here. If some of our circuits do not satisfy particular design constraints, it may be necessary to insert additional amplifiers (or pairs of successive inverters) into the circuits at certain crucial points.

One useful combination of AND, OR, and NOT elements is the two-input binary adder, shown in Fig. 2.02. This element is denoted by the symbol $\oplus$.

By cascading two-input binary adders in any of several ways, one may construct N-input binary adders, as shown in Fig. 2.03. No matter how they are cascaded, $N - 1$ two-input adders are required, because each two-input adder merely reduces the number of signals to be summed by 1. Nevertheless, the arrangement shown in Fig. 2.03*c* is generally superior to that shown in Fig. 2.03*b*, since no signal must propagate through more than $1 + \log_2 N$ two-input adders.

Another useful combination of AND, OR, and NOT elements is the flip-flop shown in Fig. 2.04. The essence of the flip-flop is the loop including two inverters and two OR gates. When the inputs to both of these OR gates from outside the loop have value 0, then one side of the loop carries the value 1, and the other side carries the value 0. If the set command is zero, then the signals around the loop remain constant, independent of the input bit. However, if the set command is one, then, depending on the value of the input, one or the other of the OR gates (depending on the value of the input bit) brings a one into the loop.

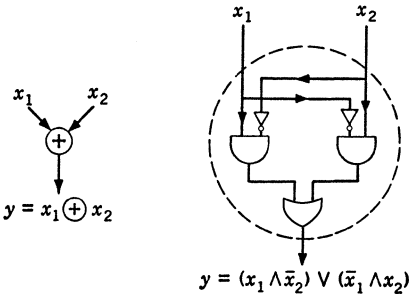


Fig. 2.02 The two-input binary adder.

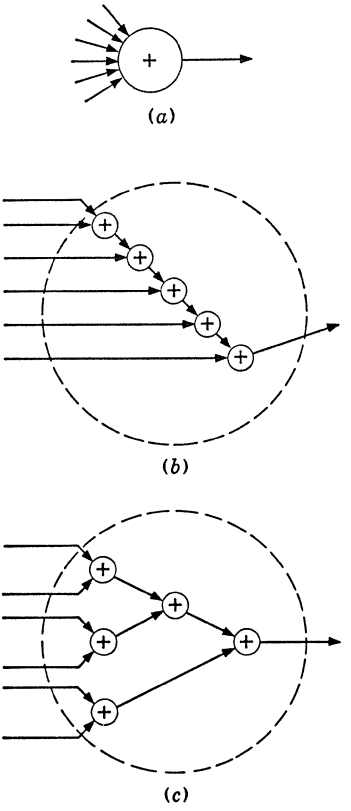


Fig. 2.03 Six-input binary adder (*a*) may be realized either by (*b*) or (*c*).

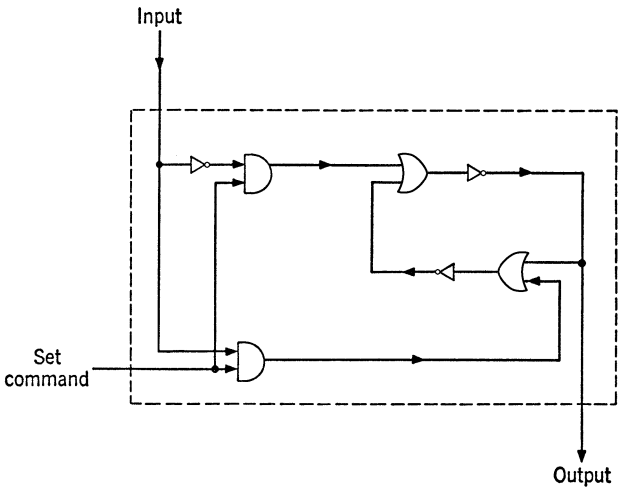


Fig. 2.04 A flip-flop.

The output, which is taken from the right side of the loop, then becomes equal to the input. After the set command again becomes zero, the loop signals retain their new values. Thus the flip-flop is a memory device. Its output now is equal to the value that the input was when the set command signal was most recently one.

A more complicated flip-flop is shown in Fig. 2.05. The essential part is again the loop consisting of two inverters and two OR gates. However, the flip-flop of Fig. 2.05 has been provided with a larger set of possible inputs, namely, x, y, and z. Each of these inputs is gated with the periodic *clock signal*, which alternately assumes the values 0 and 1 for certain prescribed lengths of time. Thus, this flip-flop can be set only at a certain phase of the clock. When the clock is in the proper phase, if the x control is one, the flip-flop will be set equal to the x input; if the y control is one, the flip-flop will be set equal to the y input; if the z control is one, the flip-flop will be set equal to the z input. It is assumed that the controls never attempt to set the flip-flop to two different inputs simultaneously.

It is often useful to send identical control signals to the inputs of several flip-flops, as in Fig. 2.06. Here the boxes represent flip-flops of the type shown in Fig. 2.05. The A boxes represent one register of flip-

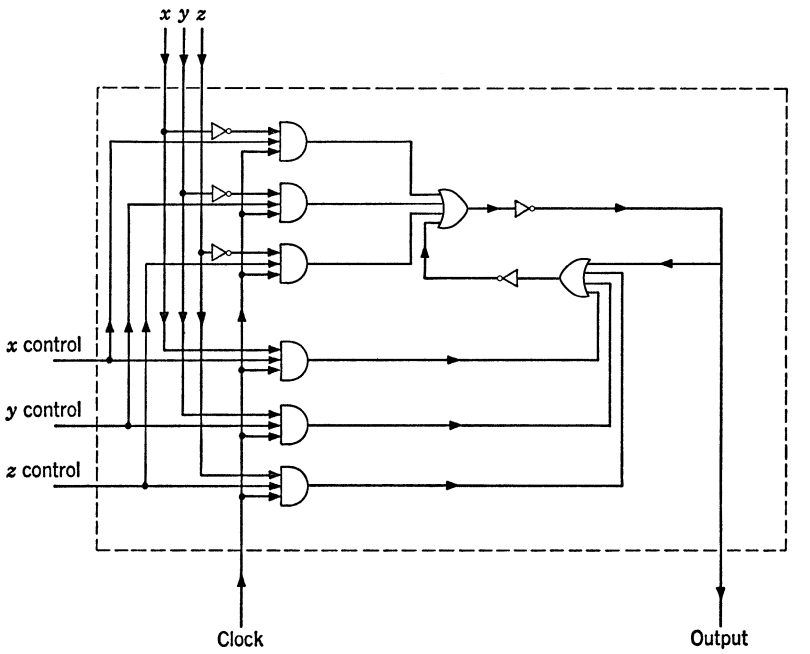


Fig. 2.05 A three-input flip-flop.

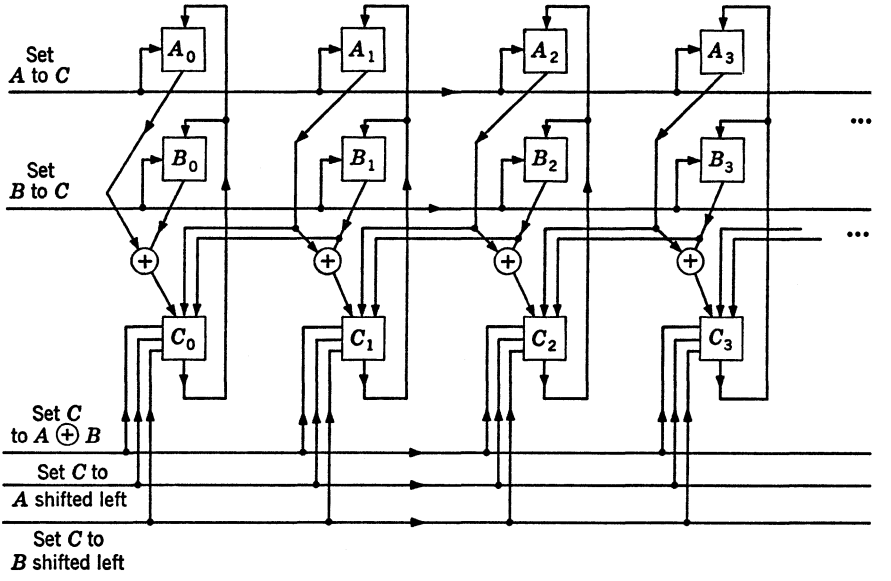


Fig. 2.06 Shifting a register or adding it to another register.

flops; the B boxes, another register; the C boxes, a third register. Each C flip-flop has three possible inputs, namely, the left shift of the A register, or the left shift of the B register, or the bit by bit sum of the A and B registers. Notice that the control signals "set C to $A \oplus B$," "set C to A shifted left," and "set C to B shifted left" are fed into every flip-flop in the C register. Thus, by using the proper command signals, it is possible to operate on the entire C register at once.

The circuit shown in Fig. 2.06 is a two-phase circuit, because the implicit clock inputs to registers A, B, and C are not identical. Registers A and B may be set only during one phase of the clock; register C, during the other phase. This use of the two-phase clock prevents any races and resultant malfunctioning which might occur if one attempted to set a register at the same time its output was being used to reset something else.

In order to shift the A register left, one proceeds in two steps. First, one sets the C register to the A register shifted left, and then one sets the A register to the C register. The entire operation is accomplished in two phases, or one complete clock cycle. Likewise, one may add the A register into the B register, or add the B register into the A register, or shift the B register left. Each of these operations may also be done in one clock cycle. Similarly in Fig. 2.07, the M register is on one phase of the clock cycle; the L register, on the other phase. With this circuit,

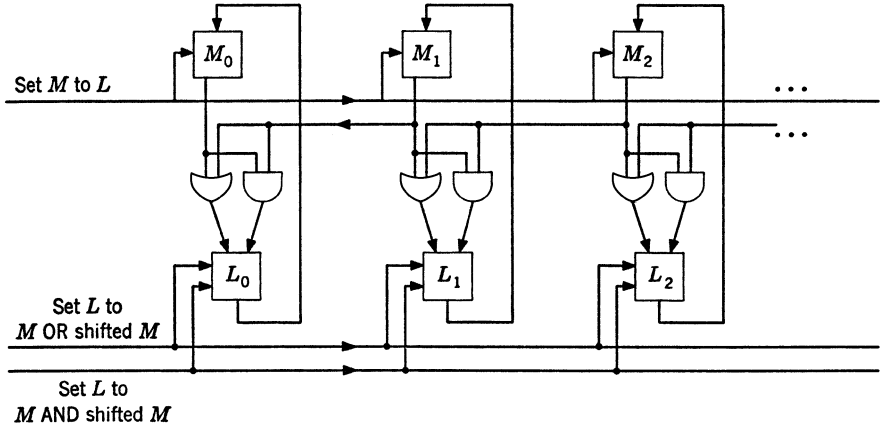


Fig. 2.07 ANDing or ORing a register with a shifted version of itself.

one may do such operations as "AND the left shift of M into M," or "OR the left shift of M into M." Each of these operations also requires one clock cycle. Finally, Fig. 2.08 shows the circuitry for a slightly more complicated operation, namely, "set those bits of B to the left of the leftmost 0 in M to the corresponding bits of C." This command is the second phase of the full cycle command, "add those bits of A to the left of the leftmost 0 in M into the corresponding bits of B." It is not difficult to design similar circuits to execute other simple logical commands of this sort.

Since the diagrams of the type shown in Figs. 2.06 to 2.08 generally become rather involved, there is a tendency to use the shorthand dia-

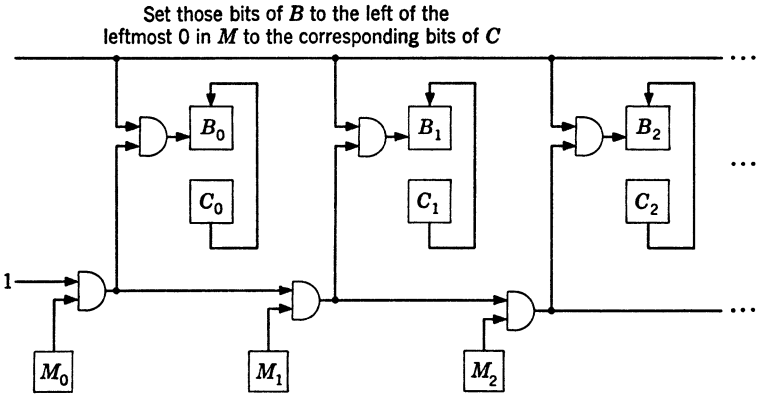


Fig. 2.08 Circuitry for implementing a more complicated command.

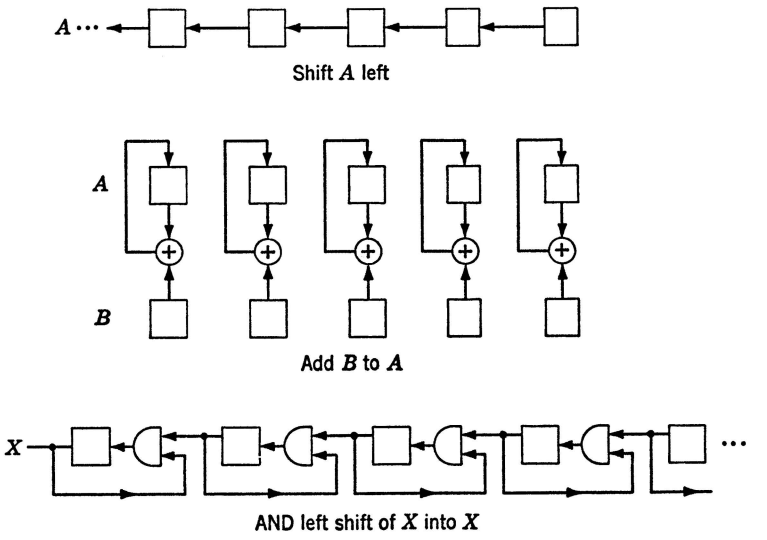


Fig. 2.09 Abbreviated logical-circuit diagrams.

grams shown in Fig. 2.09. In this notation, the controlling set commands and the registers on the secondary phase of the clock are omitted.

★ 2.3 MULTIPLICATIVE INVERSION

Let us now consider the problem of finding the multiplicative inverse of an element in the field of residue classes mod an irreducible binary polynomial $M(x)$ of degree m. Given the residue class containing $r(x)$, a polynomial of degree $<m$, we wish to find the polynomial $p(x)$ of degree $<m$ such that the product satisfies

$$r(x)p(x) \equiv 1 \bmod M(x)$$

or equivalently, $r(x)p(x) + M(x)q(x) = 1$ for some polynomial $q(x)$. Since $M(x)$ is irreducible, the gcd of M and r is 1. We may therefore apply the continued-fractions version of Euclid's algorithm as described in Sec. 2.1. Starting with $r^{(-2)} = M$, $r^{(-1)} = r$, $p^{(-2)} = 0$, $p^{(-1)} = 1$, $q^{(-2)} = 1$, $q^{(-1)} = 0$, we use the division algorithm to find $a^{(k)}$ and $r^{(k)}$ such that

$$r^{(k-2)} = a^{(k)}r^{(k-1)} + r^{(k)} \qquad \deg r^{(k)} < \deg r^{(k-1)}$$

We then set

$$q^{(k)} = a^{(k)}q^{(k-1)} + q^{(k-2)}$$
$$p^{(k)} = a^{(k)}p^{(k-1)} + p^{(k-2)}$$

The iteration is to be continued until $r^{(n)} = 0$. The solution is then given by $q = q^{(n-1)}$, $p = p^{(n-1)}$ with $\deg q < \deg r$, $\deg p < \deg M = m$. Since we wish to find only p (and do not particularly care about q), we may dispense with the q's entirely.

Before designing the logical circuits, let us work an example. Suppose $r(x) = x^4 + x + 1$ and $M(x) = x^5 + x^2 + 1$. One method of computing successive a's and r's and p's follows.

Calculation 2.31

$$
\begin{aligned}
r^{(-2)} &= 1x^5 + 0x^4 + 0x^3 + 1x^2 + 0x + 1\\
r^{(-1)} &= 0x^5 + 1x^4 + 0x^3 + 0x^2 + 1x + 1\\
\\
r^{(-2)} &= 1x^5 + 0x^4 + 0x^3 + 1x^2 + 0x + 1\\
xr^{(-1)} &= 1x^5 + 0x^4 + 0x^3 + 1x^2 + 1x\\
\\
r^{(-2)} + xr^{(-1)} &= 0x^5 + 0x^4 + 0x^3 + 0x^2 + 1x + 1\\
xr^{(-1)} &= 1x^5 + 0x^4 + 0x^3 + 1x^2 + 1x
\end{aligned}
$$

$$\boxed{a^{(0)} = x}$$

$$
\begin{aligned}
r^{(0)} &= 0x^4 + 0x^3 + 0x^2 + 1x + 1\\
r^{(-1)} &= 1x^4 + 0x^3 + 0x^2 + 1x + 1\\
\\
xr^{(0)} &= 0x^4 + 0x^3 + 1x^2 + 1x\\
r^{(-1)} &= 1x^4 + 0x^3 + 0x^2 + 1x + 1\\
\\
x^2r^{(0)} &= 0x^4 + 1x^3 + 1x^2\\
r^{(-1)} &= 1x^4 + 0x^3 + 0x^2 + 1x + 1\\
\\
x^3r^{(0)} &= 1x^4 + 1x^3\\
r^{(-1)} &= 1x^4 + 0x^3 + 0x^2 + 1x + 1\\
\\
x^3r^{(0)} &= 1x^4 + 1x^3\\
r^{(-1)} + x^3r^{(0)} &= 0x^4 + 1x^3 + 0x^2 + 1x + 1\\
\\
x^2r^{(0)} &= 1x^3 + 1x^2\\
r^{(-1)} + x^3r^{(0)} &= 1x^3 + 0x^2 + 1x + 1\\
\\
x^2r^{(0)} &= 1x^3 + 1x^2\\
r^{(-1)} + (x^3 + x^2)r^{(0)} &= 0x^3 + 1x^2 + 1x + 1\\
\\
xr^{(0)} &= 1x^2 + 1x\\
r^{(-1)} + (x^3 + x^2)r^{(0)} &= 1x^2 + 1x + 1\\
\\
xr^{(0)} &= 1x^2 + 1x\\
r^{(-1)} + (x^3 + x^2 + x)r^{(0)} &= 0x^2 + 0x + 1
\end{aligned}
$$

$$\boxed{a^{(1)} = x^3 + x^2 + x}$$

$$\begin{aligned} r^{(0)} &= 1x + 1 \\ r^{(1)} &= 0x + 1 \end{aligned}$$

$$\begin{aligned} r^{(0)} &= 1x + 1 \\ xr^{(1)} &= 1x \end{aligned}$$

$$\begin{aligned} r^{(0)} + xr^{(1)} &= 0x + 1 \\ xr^{(1)} &= 1x \end{aligned}$$

$$\begin{aligned} r^{(0)} + xr^{(1)} &= 1 \\ r^{(1)} &= 1 \end{aligned}$$

$$\begin{aligned} r^{(0)} + (x+1)r^{(1)} &= 0 \\ r^{(1)} &= 1 \end{aligned}$$

$$\boxed{a^{(2)} = x + 1}$$

$$\begin{aligned} r^{(2)} &= 0 \\ r^{(1)} &= 1 \end{aligned}$$

In this computation we evidently have

$$\begin{aligned} r^{(-2)} &= x^5 + x^2 + 1 \\ r^{(-1)} &= x^4 + x + 1 \\ r^{(0)} &= x + 1 & a^{(0)} &= x \\ r^{(1)} &= 1 & a^{(1)} &= x^3 + x^2 + x \\ r^{(2)} &= 0 & a^{(2)} &= x + 1 \end{aligned}$$

Starting from $p^{(-2)} = 0$ and $p^{(-1)} = 1$, we may compute the p's to obtain

$$\begin{aligned} p^{(-2)} &= 0 \\ p^{(-1)} &= 1 \\ p^{(0)} &= x \\ p^{(1)} &= 1 + x^2 + x^3 + x^4 \\ p^{(2)} &= 1 + x^2 + x^5 \end{aligned}$$

In order to avoid remembering the a's, it is wise to do these computations in phase with the computations of the r's. Thus, we may write

Calculation 2.32

$$\begin{aligned} p^{(-1)} &= 1 \\ p^{(-2)} &= 0 \end{aligned}$$

$$\begin{aligned} xp^{(-1)} &= 1x \\ p^{(-2)} &= 0 + 0x \end{aligned}$$

$$
\begin{aligned}
xp^{(-1)} &= 1x \\
xp^{(-1)} + p^{(-2)} &= 0 + 1x \\
\\
p^{(-1)} &= 1 + 0x \\
xp^{(-1)} + p^{(-2)} = p^{(0)} &= 0 + 1x \\
\\
p^{(-1)} &= 1 + 0x + 0x^2 \\
xp^{(0)} &= 0x + 1x^2 \\
\\
p^{(-1)} &= 1 + 0x + 0x^2 + 0x^3 \\
x^2p^{(0)} &= 0x^2 + 1x^3 \\
\\
p^{(-1)} &= 1 + 0x + 0x^2 + 0x^3 + 0x^4 \\
x^3p^{(0)} &= 0x^3 + 1x^4 \\
\\
p^{(-1)} + x^3p^{(0)} &= 1 + 0x + 0x^2 + 0x^3 + 1x^4 \\
x^3p^{(0)} &= 0x^3 + 1x^4 \\
\\
p^{(-1)} + x^3p^{(0)} &= 1 + 0x + 0x^2 + 0x^3 + 1x^4 \\
x^2p^{(0)} &= 0x^2 + 1x^3 + 0x^4 \\
\\
p^{(-1)} + (x^3 + x^2)p^{(0)} &= 1 + 0x + 0x^2 + 1x^3 + 1x^4 \\
x^2p^{(0)} &= 0x^2 + 1x^3 + 0x^4 \\
\\
p^{(-1)} + (x^3 + x^2)p^{(0)} &= 1 + 0x + 0x^2 + 1x^3 + 1x^4 \\
xp^{(0)} &= 0x + 1x^2 + 0x^3 + 0x^4 \\
\\
p^{(-1)} + (x^3 + x^2 + x)p^{(0)} &= 1 + 0x + 1x^2 + 1x^3 + 1x^4 \\
xp^{(0)} &= 0x + 1x^2 + 0x^3 + 0x^4 \\
\\
p^{(1)} &= 1 + 0x + 1x^2 + 1x^3 + 1x^4 \\
p^{(0)} &= 0 + 1x + 0x^2 + 0x^3 + 0x^4 \\
\\
xp^{(1)} &= 1x + 0x^2 + 1x^3 + 1x^4 + 1x^5 \\
p^{(0)} &= 0 + 1x + 0x^2 + 0x^3 + 0x^4 + 0x^5 \\
\\
xp^{(1)} &= 1x + 0x^2 + 1x^3 + 1x^4 + 1x^5 \\
p^{(0)} + xp^{(1)} &= 0 + 0x + 0x^2 + 1x^3 + 1x^4 + 1x^5 \\
\\
p^{(1)} &= 1 + 0x + 1x^2 + 1x^3 + 1x^4 + 0x^5 \\
p^{(0)} + xp^{(1)} &= 0 + 0x + 0x^2 + 1x^3 + 1x^4 + 1x^5 \\
\\
p^{(1)} &= 1 + 0x + 1x^2 + 1x^3 + 1x^4 + 0x^5 \\
p^{(0)} + (x + 1)p^{(1)} = p^{(2)} &= 1 + 0x + 1x^2 + 0x^3 + 0x^4 + 1x^5
\end{aligned}
$$

$$p^{(1)} = 1 + 0x + 1x^2 + 1x^3 + 1x^4 + 0x^5 + 0x^6$$
$$xp^{(2)} = 1x + 0x^2 + 1x^3 + 0x^4 + 0x^5 + 1x^6$$

The coefficients of the calculations for the r's and p's are as follows:

Calculation 2.33

```
1 0 0 1 0 1,1      initial position
0 1 0 0 1 1,0

1 0 0 1 0 1,1
1 0 0 1 1,0 0

0 0 0 0 1 1,1      computing r(0) and p(0)
1 0 0 1 1,0 1

0 0 0 1 1,1 0
1 0 0 1 1,0 1

0 0 1 1,1 0 0
1 0 0 1 1,0 1

0 1 1,1 0 0 0
1 0 0 1 1,0 1

1 1,1 0 0 0 0
1 0 0 1 1,0 1

1 1,1 0 0 0 1      computing r(1) and p(1)
0 1 0 1 1,0 1

1 1,1 0 0 0 1
1 0 1 1,0 1 0

1 1,1 0 0 1 1
0 1 1 1,0 1 0

1 1,1 0 0 1 1
1 1 1,0 1 0 0

1 1,1 0 1 1 1
0 0 1,0 1 0 0

1 1,1 0 1 1 1
0 1,0 1 0 0 0
```

1 1,1 0 1 1 1	
1,0 1 0 0 0 0	
0 1,1 0 1 1 1	
1,0 0 0 1 1 1	computing $r^{(2)}$ and $p^{(2)}$
1,1 0 1 1 1 0	
1,0 0 0 1 1 1	
0,1 0 1 1 1 0	
1,1 0 1 0 0 1	
,1 0 1 1 1 0 0	
1,1 0 1 0 0 1	terminal position

(Calculation 2.33 is a contraction of Calculations 2.31 and 2.32.)

Here we have arranged the r's with their leading coefficients leftmost and the p's with their leading coefficients rightmost. In this manner, we are able to do the computations with only two principal registers, each containing $m + 2$ bits. The rules are as follows.

Multiplicative inversion algorithm 2.34 Initially, the top register is loaded with the irreducible polynomial $M(x)$, followed by a comma and a one. The bottom register is loaded with the coefficients of the polynomial $r(x)$, followed by a comma and a zero. The value of k is initially -1. We then proceed as follows.

If the leftmost bit of the top register is zero, shift the top register (including its comma) one position left.

If the leftmost bit of the bottom register is zero, shift the bottom register (including its comma) one position left.

If the leftmost bits of both the top and bottom registers are ones and k is even, then add that part of the bottom register to the left of its comma into the top register, and add that part of the top register to the right of its comma into the bottom register.

If the leftmost bits of both the top and bottom registers are ones and k is odd, then add that part of the top register to the left of its comma into the bottom register, and add that part of the bottom register to the right of its comma into the top register.

Whenever the commas are aligned and a shift is executed, the parity of k changes. When k is even, the bottom comma is left of (or coincident with) the top comma; when k is odd, the bottom comma is right of (or coincident with) the top comma.

When either comma is shifted out of the left end of its register, the calculation is completed. The register whose comma was shifted out contains the coefficients $p_0, p_1, \ldots, p_{m-1}, 0, 0$; the other register contains $1, M_0, M_1, \ldots, M_m$.

This method combines the iterations of the division algorithm and Euclid's algorithm into a unified procedure, each step of which is merely a shift or an add. The total number of shifts is exactly $2m + 1$, since each shift moves one comma or the other and the top comma is moved a total of $m + 1$ places and the bottom comma, a total of m places. Since every add is directly followed by a shift, it is immediately evident that there are no more than $2m$ add operations. We conclude that the entire procedure requires no more than $4m + 1$ operations altogether.

There are various logical circuits which can be designed to implement this procedure. One method of keeping track of the commas is to use a separate register whose contents are all ones except for a lone string of zeros, the first and last positions of which represent the commas. Shifting operations cause each digit of the marker register to be replaced by the logical AND or OR of itself and its right neighbor, depending on which register is being shifted and on the parity of k. Since there is only one zero in the marker register iff there are no two adjacent zeros in that register, the condition of coincident commas may be checked by ORing together the logical AND of the even positions of the marker register and the logical AND of the odd positions of the marker register. As an illustration, we show the first several steps of the Calculation 2.33 with the marker register used instead of the commas.

Calculation 2.35

With commas	*With marker register*
1 0 0 1 0 1,1	1 0 0 1 0 1 1
	1 1 1 1 1 1 0 1
0 1 0 0 1 1,0	0 1 0 0 1 1 0
1 0 0 1 0 1,1	1 0 0 1 0 1 1
	1 1 1 1 1 0 0 1
1 0 0 1 1,0 0	1 0 0 1 1 0 0
0 0 0 0 1 1,1	0 0 0 0 1 1 1
	1 1 1 1 1 0 0 1
1 0 0 1 1,0 1	1 0 0 1 1 0 1
0 0 0 1 1,1 0	0 0 0 1 1 1 0
	1 1 1 1 1 0 1 1
1 0 0 1 1,0 1	1 0 0 1 1 0 1
0 0 1 1,1 0 0	0 0 1 1 1 0 0
	1 1 1 1 0 0 1 1
1 0 0 1 1,0 1	1 0 0 1 1 0 1

```
0 1 1,1 0 0 0 0        0 1 1 1 0 0 0
                     1 1 1 0 0 0 1 1
1 0 0 1 1,0 1          1 0 0 1 1 0 1

1 1,1 0 0 0 0 0        1 1 1 0 0 0 0
                     1 1 0 0 0 0 1 1
1 0 0 1 1,0 1          1 0 0 1 1 0 1

1 1,1 0 0 0 1          1 1 1 0 0 0 1
                     1 1 0 0 0 0 1 1
0 1 0 1 1,0 1          0 1 0 1 1 0 1

1 1,1 0 0 0 1          1 1 1 0 0 0 1
                     1 1 0 0 0 1 1 1
1 0 1 1,0 1 0          1 0 1 1 0 1 0
```

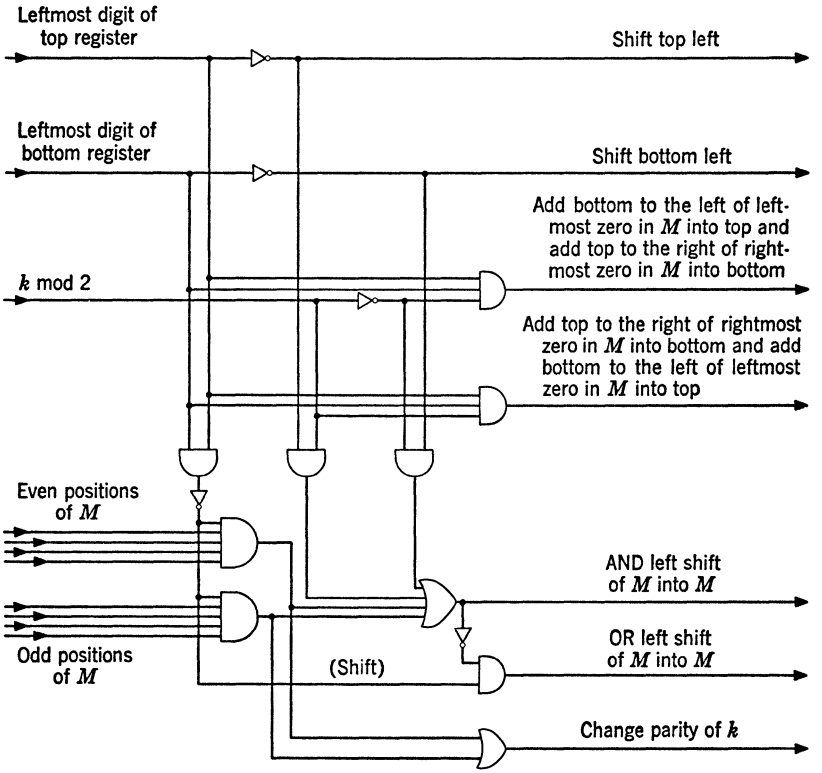


Fig. 2.10 Control circuitry for computing multiplicative inverses modulo an irreducible binary polynomial via a variant of Euclid's algorithm. (Here M refers to the marker register, not to the polynomial $M(x)$, whose coefficients are the initial contents of the top register.)

Notice that the termination of the algorithm is marked by the appearance of a zero in the leftmost digit of the marker register.

If one uses synchronous logic and a two-phase clock, the cheapest implementation of this procedure requires up to $4m + 1$ clock cycles to complete the calculation. If one is willing to pay for additional components, the calculation can be speeded up. For example, consecutive adds and shifts may be done simultaneously, thus cutting the time by a factor of 2. Or if one is interested in extreme speed, one may build separate logic for each step of the procedure. Such a circuit could accomplish the entire calculation in one clock cycle (or, more precisely, as fast as signals propagate through m stages of logic), but the cost of its logic would be proportional to m. For large m, a circuit whose cost and computing time are both linear in m is probably the best choice for most applications.

Using the techniques described in the previous section, it is a relatively straightforward matter to design circuits to execute each of the rules in Algorithm 2.34. The control logic for such circuits is given in Fig. 2.10.

★ 2.4 MULTIPLICATION

When considering the multiplication of residue classes mod $M(x)$, where $M(x)$ is an irreducible binary polynomial of degree m, it is helpful to introduce the symbol α to denote the residue class containing x. Then α^2 represents the residue class containing x^2, and, in general, if $r(x)$ is any polynomial, then $r(\alpha)$ represents the residue class containing $r(x)$. Since $M(x) \equiv 0 \bmod M(x)$, we must have $M(\alpha) = 0$. The element represented by the symbol α is therefore a root of the polynomial $M(x)$. Hence, we have an obvious isomorphism between the field containing the 2^m residue classes mod $M(x)$ and the field containing the binary field and all polynomials in α, where α is a root of the irreducible binary polynomial $M(x)$.

Any element Y in this field may be expressed uniquely as a polynomial of degree $<m$ in α, $Y = \sum_{i=0}^{m-1} Y_i\alpha^i$, where the Y_i are binary numbers. The element Y may be conveniently stored in an m-bit register, whose components contain the binary numbers $Y_{m-1}, Y_{m-2}, \ldots, Y_0$.

2.41 MULTIPLICATION OF A REGISTER BY A WIRED CONSTANT

Let us first consider the multiplication of the field element in the Y register by a constant field element A. We may assume that A is represented by some binary polynomial in α. Since $Y = \sum_{i=0}^{m-1} Y_i\alpha^i$, we

have $YA = \sum_{i=0}^{m-1} Y_i(A\alpha^i)$. Expressing $A\alpha^i$ as a polynomial of degree $<m$ in α gives $A\alpha^i = \sum_{j=0}^{m-1} A_{i,j}\alpha^j$, so that

$$\begin{aligned} YA &= \sum_{i=0}^{m-1} Y_i \sum_{j=0}^{m-1} A_{i,j}\alpha^j \\ &= \sum_{j=0}^{m-1} \left(\sum_{i=0}^{m-1} Y_i A_{i,j} \right) \alpha^j \end{aligned}$$

Thus, multiplication of the field element Y by the field element A is equivalent to multiplication of the m-dimensional binary row vector $\mathbf{Y} = [Y_{m-1}, Y_{m-2}, \ldots, Y_0]$ by the $m \times m$ matrix whose components are $A_{i,j}$. The rows of this matrix represent the products $A\alpha^{m-1}, A\alpha^{m-2}, \ldots, A$.

For example, let $M(x) = x^5 + x^2 + 1$. Suppose we wish to multiply the contents of the Y register by the field element $A = \alpha^3 + \alpha$. We first compute

$$\begin{aligned} A\alpha &= \alpha^4 + \alpha^2 \\ A\alpha^2 &= \alpha^5 + \alpha^3 = \alpha^3 + \alpha^2 + 1 \\ A\alpha^3 &= \alpha^4 + \alpha^3 + \alpha \\ A\alpha^4 &= \alpha^5 + \alpha^4 + \alpha^2 = \alpha^4 + 1 \end{aligned}$$

The multiplication $Z = YA$ is equivalent to

$$[Z_4,Z_3,Z_2,Z_1,Z_0] = [Y_4,Y_3,Y_2,Y_1,Y_0] \begin{bmatrix} 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

This multiplication may readily be accomplished by the circuit of Fig. 2.11.

If we instead wish to multiply the contents of the Y register by the field element α, we build a circuit which multiplies by the matrix

$$[\alpha] = \begin{bmatrix} 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

This circuit is shown in Fig. 2.12.

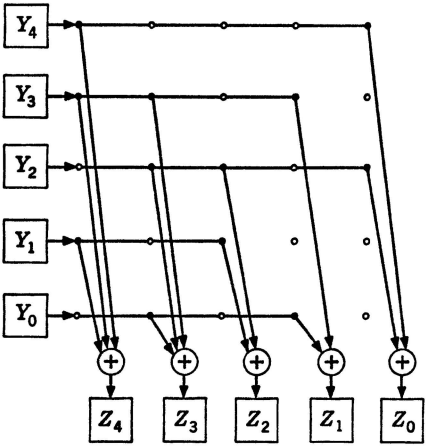


Fig. 2.11 Circuit for multiplying by $\alpha^3 + \alpha$.

Similarly, to multiply the arbitrary field element in the Y register by the element α^{-1}, we build a circuit which multiplies by the matrix

$$[\alpha^{-1}] = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

This circuit is shown in Fig. 2.13.

In general, we may let $W(M)$ denote the weight of $M(x)$, that is, the number of nonzero coefficients in the polynomial $M(x)$. If the field is represented with respect to α, a root of $M(x)$, then the multiplication of a register by α or α^{-1} requires only $W(M) - 2$ two-input adders. For many values of m it is possible to select an irreducible binary polynomial of degree m which has weight 3.† In these cases, multiplication by α and α^{-1} may be accomplished with only one two-input adder.

† Further discussion of irreducible binary polynomials of weight 3 may be found in Sec. 6.3.

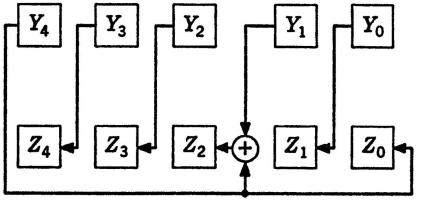


Fig. 2.12 Circuit for multiplying by α.

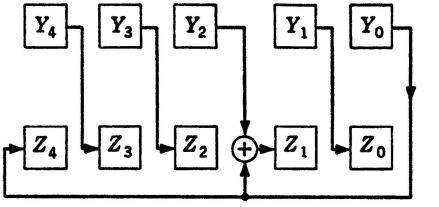


Fig. 2.13 Circuit for multiplying by α^{-1}.

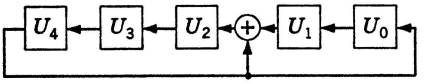


Fig. 2.14 Circuit for multiplying U by α.

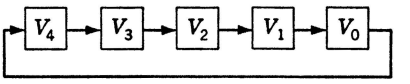


Fig. 2.15 Circuit for cyclically shifting V right.

2.42 MULTIPLICATION OF TWO REGISTERS

In order to multiply together the two field elements in the U and V registers, we must compute the product $Z = UV$. Each of the m bits of the product is a linear combination of the m^2 binary products $U_k V_j$. The operation may therefore be represented by an $m \times m \times m$ tensor. This multiplication may be implemented directly, as suggested by Bartee and Schneider (1963). Depending on the irreducible polynominal $M(x)$, this implementation might require almost as many as $m^3 - m$ two-input adders. Even with a wise choice of irreducible polynomial, the number of two-input adders tends to be so large that the method is quite expensive for large m. If one does not require the extreme speed (one clock cycle) of the Bartee-Schneider method, then longer, less expensive implementations are preferable. One of the most direct of these is the following, which is very similar to the method described by Peterson (1961).

Let the multiplicand U be stored in a feedback shift register which is wired to replace U by $U\alpha$ upon command. For example, let α be a root of $M(x) = x^5 + x^2 + 1$. In abbreviated shorthand notation, we have the circuit shown in Fig. 2.14. The multiplier V is stored in a register, which may be cyclically shifted to the right as in Fig. 2.15. The product Z is accumulated in a register to which the U register can be added, as in Fig. 2.16.

To perform the multiplication, the Z register is initially set to zero. Depending on the lowest bit of the multiplicand V_0, U is either added or not added into Z. If $V_0 = 1$, then U is added into Z; if $V_0 = 0$, Z is left unchanged. The V register is then shifted right, the U register is multiplied by α, and the process is repeated. At the nth step, U con-

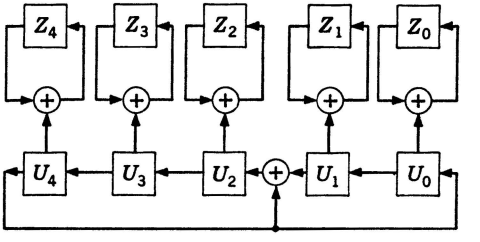


Fig. 2.16 Circuit for adding multiples of U into Z.

tains α^n times the original multiplicand, V_0 contains V_n, the nth bit of the original multiplier, and Z contains $\sum_{i=0}^{n} V_i(U\alpha^i)$. After m such steps, the multiplication is complete. Z contains the product of the original U and V registers. The V register has been cycled completely around to its original position, but the U register now contains $U\alpha^{m-1}$. If it is necessary to recover the original multiplicand for subsequent use, additional circuitry is needed. The original multiplicand may be recovered in one step by a wired multiplication by the constant $\alpha^{-(m-1)}$. Often this multiplication requires so many adders that it is less expensive to build an extra register in which the original U is preserved.

A cheaper method of recovering the original multiplicand is to wire the multiplication by α^{-1} and repeat this multiplication m times. This multiplication requires only one two-input adder [if $M(x)$ has weight 3], but it requires m clock cycles. Various intermediate solutions are also possible, such as wiring the multiplication by α^{-2} and executing it $m/2$ times. Another solution is to wire the multiplications of the U register by α, α^2, and α^{-2} and to wire double shifts of the V register in both directions and a single right shift of the V register. One may then accumulate the product as

$$Z = V_0U + V_2(U\alpha^2) + V_4(U\alpha^4) + \cdots + V_{m-1}(U\alpha^{m-1}) + \cdots + V_5(U\alpha^5) + V_3(U\alpha^3) + V_1(U\alpha)$$

This method completes the entire multiplication in m steps and allows the original multiplier and multiplicand to be recovered in one or two additional steps. However, this method requires somewhat more complicated controls.

2.43 LOG TABLES

As a final method of doing multiplications, divisions, powers, and roots, we consider the use of log tables. Such tables may be constructed for the nonzero elements in any finite field. For example, if α is a root of the irreducible binary polynomial $M(x) = x^5 + x^2 + 1$, then multiplication by α is readily accomplished by the feedback shift register shown in Fig. 2.17.

Starting this register from the state $1 = 0\alpha^4 + 0\alpha^3 + 0\alpha^2 + 0\alpha + 1$, we may obtain the successive powers of α as the successive states of this shift register. The first 31 powers happen to include all 31 nonzero

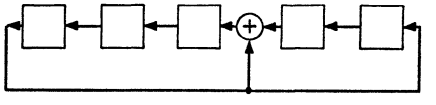


Fig. 2.17 Feedback shift register for multiplying by α.

elements of the field, with $\alpha^{31} = \alpha^0 = 1$. Thus, we may represent every element in this field as a power of α, thereby obtaining the table of logs and antilogs shown in Appendix A.

The use of such tables greatly simplifies hand calculations involving multiplications, divisions, powers, and roots of field elements. For example, we compute the product

$$\begin{aligned} 1\alpha^4 + 0\alpha^3 + 1\alpha^2 + 1\alpha + 0 &= 10110 = \alpha^{28} \\ 0\alpha^4 + 0\alpha^3 + 1\alpha^2 + 0\alpha + 1 &= \underline{00101 = \alpha^5} \\ & \qquad \alpha^{33} = \alpha^2 = 00100 \end{aligned}$$

or the quotient

$$\frac{10110}{00101} = \frac{\alpha^{28}}{\alpha^5} = \alpha^{23} = 01111$$

or the power

$$10111^6 = (\alpha^{26})^6 = (\alpha^{-5})^6 = \alpha^{-30} = \alpha = 00010$$

or the root

$$\begin{aligned} 11001^{1/4} = (\alpha^{25})^{1/4} &= (\alpha^{25})^8 \qquad \text{since } 4 \cdot 8 \equiv 1 \bmod 31 \\ &= \alpha^{200} = \alpha^{31 \cdot 6 + 14} = \alpha^{14} = 11101 \end{aligned}$$

In general, we can construct such tables iff every nonzero element in the field is a power of one so-called *primitive* element. In Theorem 4.24 we shall prove that every finite field contains such an element. Consequently, we may construct such log and antilog tables for any finite field.

Although such tables are extremely useful for hand computations, they are of dubious value for circuitry implementations. One obvious approach would be to have two permanently stored memories, each containing $2^m - 1$ words of m bits each. One memory would contain logs, and the other memory would contain the antilogs. With the use of these memories, all multiplications and divisions could be reduced to conventional addition and subtraction of the exponents mod $2^m - 1$. Unfortunately, for large m, the number of bits in such a memory grows as $m(2^m - 1)$, which in many applications is more expensive than the direct methods described in Sec. 2.42.

If some method were known by which one could readily calculate the logarithm of a field element (by doing computations on the bits of its representation as a polynomial in α, instead of looking the answer up in a table), then the logarithmic approach might provide the basis for a feasible implementation of the multiplication and division operations. In general, however, no such direct methods of calculating logarithms are

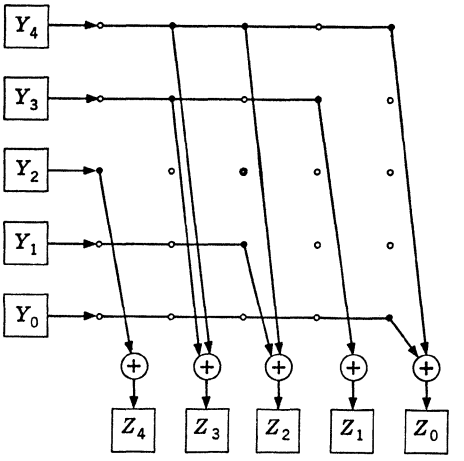


Fig. 2.18 Circuit for computing $Z = Y^2 \bmod x^5 + x^2 + 1$.

known, although Probs. 4.3 and 4.4 furnish a few partial results in this direction.

2.44 SQUARES AND SQUARE ROOTS

Another special case of multiplication which is worth considering separately is the multiplication of the field element in the Y register by itself, $Z = Y^2$. Since we can wire the multiplication of a row vector by a matrix, we can readily implement any operation which is linear over the field elements. In particular, this includes such operations as the square. In most fields, the square is a nonlinear operation, but in a field which includes the binary field, we have $(\beta + \gamma)^2 = \beta^2 + \gamma^2$. Hence the square is a linear operation, and the square of any element in the field may be expressed as an appropriate linear combination of the squares of basis elements. For example, if α is a root of $\alpha^5 + \alpha^2 + 1$, we have

$$(\alpha^4)^2 = \alpha^8 = \alpha^3\alpha^5 = \alpha^3(\alpha^2 + 1) = \alpha^5 + \alpha^3 = \alpha^3 + \alpha^2 + 1$$
$$(\alpha^3)^2 = \alpha^6 = \alpha\alpha^5 = \alpha(\alpha^2 + 1) = \alpha^3 + \alpha$$
$$(\alpha^2)^2 = \alpha^4$$
$$(\alpha^1)^2 = \alpha^2$$
$$(\alpha^0)^2 = 1$$

so the transformation $Z = Y^2$ may be taken as $\mathbf{Z} = \mathbf{Y}\mathbb{S}$ where

$$\mathbb{S} = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

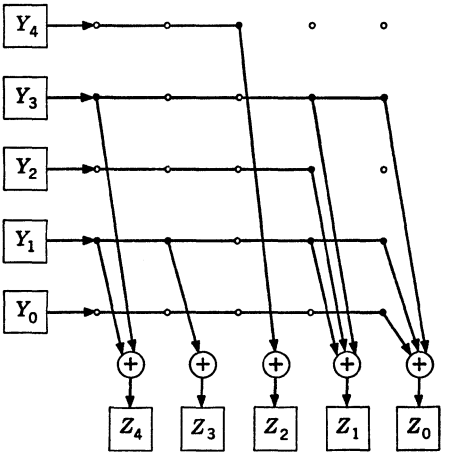


Fig. 2.19 Circuit for computing $Z = \sqrt{Y} \bmod x^5 + x^2 + 1$.

The matrix multiplication may be implemented by the circuit shown in Fig. 2.18. The inverse of this matrix is given by

$$\mathcal{S}^{-1} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Evidently, the square root of a field element may be extracted by multiplying by this matrix. That is, the transformation $Z = \sqrt{Y}$ may be performed as $\mathbf{Z} = \mathbf{Y}\mathcal{S}^{-1}$. This may be implemented by the circuit shown in Fig. 2.19.

★ 2.5 THE SOLUTION OF SIMULTANEOUS LINEAR EQUATIONS

In later chapters, we shall see that several problems which arise in coding theory may be reduced to the solution of a set of simultaneous equations. Often these equations are linear over the binary field. In some cases, however, they may be over some other field, such as GF(5), the field of integers modulo the prime 5. The addition and multiplication tables in GF(5) are

+	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

*	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

The simultaneous linear equations which we wish to solve are sometimes of the form

$$\begin{aligned}
\mathfrak{L}_{0,0}Z_0 + \mathfrak{L}_{1,0}Z_1 + \mathfrak{L}_{2,0}Z_2 + \cdots + \mathfrak{L}_{m-1,0}Z_{m-1} &= 0 \\
\mathfrak{L}_{0,1}Z_0 + \mathfrak{L}_{1,1}Z_1 + \mathfrak{L}_{2,1}Z_2 + \cdots + \mathfrak{L}_{m-1,1}Z_{m-1} &= 0 \\
\mathfrak{L}_{0,2}Z_0 + \mathfrak{L}_{1,2}Z_1 + \mathfrak{L}_{2,2}Z_2 + \cdots + \mathfrak{L}_{m-1,2}Z_{m-1} &= 0 \\
&\cdots \\
\mathfrak{L}_{0,m-1}Z_0 + \mathfrak{L}_{1,m-1}Z_1 + \mathfrak{L}_{2,m-1}Z_2 + \cdots + \mathfrak{L}_{m-1,m-1}Z_{m-1} &= 0
\end{aligned}$$

where we are given $\mathfrak{L}_{i,j}$ for $i,j = 0, 1, \ldots, m-1$ and wish to find $Z_0, Z_1, Z_2, \ldots, Z_{m-1}$. These equations may be written in matrix notation as $\mathbf{Z}\mathfrak{L} = \mathbf{0}$, which is an abbreviation for

$$[Z_0, Z_1, \ldots, Z_{m-1}] \begin{bmatrix} \mathfrak{L}_{0,0} & \mathfrak{L}_{0,1} & \mathfrak{L}_{0,2} & \cdots & \mathfrak{L}_{0,m-1} \\ \mathfrak{L}_{1,0} & \mathfrak{L}_{1,1} & \mathfrak{L}_{1,2} & \cdots & \mathfrak{L}_{1,m-1} \\ \mathfrak{L}_{2,0} & \mathfrak{L}_{2,1} & \mathfrak{L}_{2,2} & \cdots & \mathfrak{L}_{2,m-1} \\ \cdots & & & & \\ \mathfrak{L}_{m-1,0} & \mathfrak{L}_{m-1,1} & \mathfrak{L}_{m-2,2} & \cdots & \mathfrak{L}_{m-1,m-1} \end{bmatrix} = [0,0,0, \ldots, 0]$$

In the equation $\mathbf{Z}\mathfrak{L} = \mathbf{0}$ the dot product of the $\mathbf{Z}$ vector with each column of the $\mathfrak{L}$ matrix is 0. This must remain true if we exchange any pair of columns in the $\mathfrak{L}$ matrix, or if we add any column in the $\mathfrak{L}$ matrix into any other column, or if we multiply any column of the $\mathfrak{L}$ matrix by any nonzero scalar element. Since each of these column operations is reversible, we may perform any combination of these column operations on the $\mathfrak{L}$ matrix in any order. These column operations cannot alter the solution vector(s) $\mathbf{Z}$.

By appropriate choices of column operations, the $\mathfrak{L}$ matrix may be reduced to any of several *canonical forms*. The form which is most convenient for the present problem is the reduced triangular idempotent form, which we denote by $\check{\mathfrak{L}}$. This form is defined as follows:

Definition 2.51 An $m \times m$ matrix $\check{\mathfrak{L}}$ is in reduced triangular idempotent form iff every entry below the main diagonal is zero, every entry on the main diagonal is zero or one, and every entry in the same column as a main-diagonal zero or the same row as a main-diagonal one is zero.†

† A reduced triangular idempotent matrix has the same columns as the more common column-reduced standard echelon matrix, but the columns are permuted in a different order.

The following matrix is an example of a triangular idempotent matrix over GF(5)

$$\begin{bmatrix} 0 & 2 & 4 & 0 & 3 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2 & 3 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

In symbols instead of words, the definition requires that $\check{\mathfrak{L}}_{i,j} = 0$ if $i > j$, that $\check{\mathfrak{L}}_{i,j} = 0$ or 1 if $i = j$, and that $\check{\mathfrak{L}}_{i,j} = 0$ if $\check{\mathfrak{L}}_{i,i} = 1$ or if $\check{\mathfrak{L}}_{j,j} = 0$. Equivalently, $\check{\mathfrak{L}}_{i,j} \neq 0$ only if $\check{\mathfrak{L}}_{j,j} = 1$ and either $i = j$ or $i < j$ and $\check{\mathfrak{L}}_{i,i} = 0$.

The term *triangular* arises from the requirement that $\check{\mathfrak{L}}_{i,j} = 0$ if $i > j$. The term *idempotent* arises from the following theorem.

Theorem 2.52 If $\check{\mathfrak{L}}$ is a square matrix in reduced triangular idempotent form, it satisfies the idempotent equation $\check{\mathfrak{L}}^2 = \check{\mathfrak{L}}$.

Proof $\check{\mathfrak{L}}_{i,j}$ is nonzero only if $\check{\mathfrak{L}}_{j,j} = 1$; $\check{\mathfrak{L}}_{j,k}$ is nonzero only if $j = k$ or $j < k$ and $\check{\mathfrak{L}}_{j,j} = 0$. We conclude that the product, $\check{\mathfrak{L}}_{i,j}\check{\mathfrak{L}}_{j,k}$, is zero unless $j = k$. Therefore,

$$\sum_j \check{\mathfrak{L}}_{i,j}\check{\mathfrak{L}}_{j,k} = \check{\mathfrak{L}}_{i,k}\check{\mathfrak{L}}_{k,k}$$

If $\check{\mathfrak{L}}_{k,k}$ is 0, then so is $\check{\mathfrak{L}}_{i,k}$; if $\check{\mathfrak{L}}_{k,k}$ is nonzero, it is 1. In either case, $\check{\mathfrak{L}}_{i,k}\check{\mathfrak{L}}_{k,k} = \check{\mathfrak{L}}_{i,k}$, and

$$\sum_j \check{\mathfrak{L}}_{i,j}\check{\mathfrak{L}}_{j,k} = \check{\mathfrak{L}}_{i,k} \qquad \text{for all } i \text{ and } k$$

This is equivalent to the matrix equation $\check{\mathfrak{L}}^2 = \check{\mathfrak{L}}$. Q.E.D.

As a consequence of this result, we may readily characterize the row vectors $\mathbf{Z}$ which satisfy the equation $\mathbf{Z}\check{\mathfrak{L}} = \mathbf{0}$. For later use, we also obtain a characterization of the row vectors $\mathbf{U}$ which satisfy the equation $\mathbf{U}\check{\mathfrak{L}} = \mathbf{U}$. The theorem follows.

Theorem 2.53 If $\check{\mathfrak{L}}$ is in reduced triangular idempotent form, then the row vector $\mathbf{Z}$ is a solution to the equation $\mathbf{Z}\check{\mathfrak{L}} = \mathbf{0}$ iff $\mathbf{Z}$ is a linear combination of rows of the matrix $\check{\mathfrak{L}} - \mathfrak{I}$, where $\mathfrak{I}$ is the identity matrix. Similarly, the row vector $\mathbf{U}$ is a solution to the equation $\mathbf{U}(\check{\mathfrak{L}} - \mathfrak{I}) = \mathbf{0}$ iff $\mathbf{U}$ is a linear combination of rows of the matrix $\check{\mathfrak{L}}$. $\mathbf{U}$ is such a linear combination iff the product of each component of $\mathbf{U}$ and the corresponding diagonal component of $(\check{\mathfrak{L}} - \mathfrak{I})$ is 0.

Proof If $\check{\mathcal{E}}^2 = \check{\mathcal{E}}$, then $\check{\mathcal{E}}^2 - \check{\mathcal{E}} = 0$, and $(\check{\mathcal{E}} - \mathcal{I})\check{\mathcal{E}} = 0$. Therefore, if $\mathbf{Z}$ is any linear combination of rows of the $\check{\mathcal{E}} - \mathcal{I}$ matrix, then $\mathbf{Z}\check{\mathcal{E}} = \mathbf{0}$. Similarly, if $\check{\mathcal{E}}^2 = \check{\mathcal{E}}$, then $\check{\mathcal{E}}^2 - \check{\mathcal{E}} = 0$ and $\check{\mathcal{E}}(\check{\mathcal{E}} - \mathcal{I}) = 0$. Therefore, if $\mathbf{U}$ is a linear combination of rows of the $\check{\mathcal{E}}$ matrix, $\mathbf{U}$ satisfies the equation $\mathbf{U}(\check{\mathcal{E}} - \mathcal{I}) = 0$.

The only nonzero rows of the $\check{\mathcal{E}} - \mathcal{I}$ matrix have -1 on the main diagonal, so the rank of the $\check{\mathcal{E}} - \mathcal{I}$ matrix is equal to the nullity of $\check{\mathcal{E}}$. Therefore, linear combinations of rows of $\check{\mathcal{E}} - \mathcal{I}$ are the only solutions of the equation $\mathbf{Z}\check{\mathcal{E}} = \mathbf{0}$. The only nonzero columns of the $\check{\mathcal{E}}$ matrix have 1 on the main diagonal, so the rank of the $\check{\mathcal{E}}$ matrix is equal to the nullity of the $\check{\mathcal{E}} - \mathcal{I}$ matrix. Therefore, linear combinations of the rows of $\check{\mathcal{E}}$ are the only solutions of the equation $\mathbf{U}(\check{\mathcal{E}} - \mathcal{I}) = \mathbf{0}$.

The product of each component of $\mathbf{U}$ and the corresponding diagonal component of $\check{\mathcal{E}} - \mathcal{I}$ is 0 iff $\mathbf{U}$ has a zero in each column in which the $\check{\mathcal{E}}$ matrix is all zero. Since every nonzero column of the triangular $\check{\mathcal{E}}$ matrix includes a main-diagonal one, $\mathbf{U}$ is a linear combination of the rows of $\check{\mathcal{E}}$ iff $\mathbf{U}$ has a zero in each column in which $\check{\mathcal{E}}$ is zero. Q.E.D.

Finally, we must prove Theorem 2.54.

Theorem 2.54 Any square matrix may be reduced to triangular idempotent form by appropriate column operations.

Although there are several ways of going about this reduction, we outline a relatively indirect procedure which lends itself most readily to implementation.

Since we may exchange any pair of columns, we may permute the columns in any order we desire. Thus, we define the operation "rotate columns leftward" by the diagrammatic permutation shown in Fig. 2.20. Similarly, we define the operation "rotate rows upward" by the diagrammatic permutation shown in Fig. 2.21.

At this point the alert reader should be alarmed! We are permitted only column operations, not row operations. However, our procedure will perform this permutation of rows a total of m times, thereby ending up back where it started. Since the operation of rotating rows commutes with each of the column operations, the net effect of the m row rotations is nil. Why then do we bother rotating the rows at all? Because certain decisions in the procedure must be made on the basis of the values of elements in a certain row, and it is much easier to permute the rows

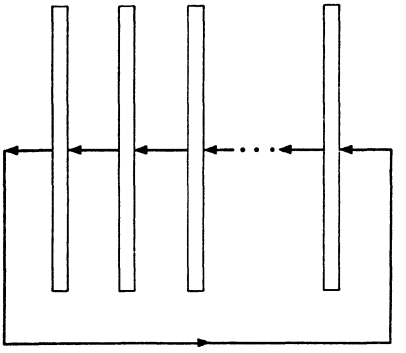


Fig. 2.20 Rotate columns leftward.

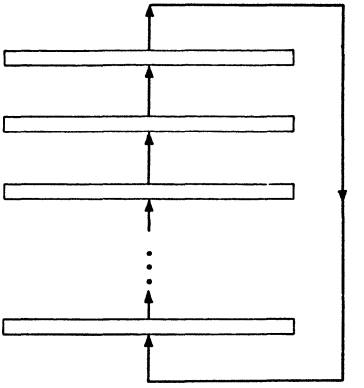


Fig. 2.21 Rotate rows upward.

into a register of specially equipped hardware than to build hardware sufficiently sophisticated to make control decisions on the contents of any row.

Matrix-reduction algorithm 2.55 The procedure consists of one basic set of steps, which is repeated m times:

1. If the top component of the leftmost column is nonzero, do nothing. If the top component of the leftmost column is zero, exchange that column with the leftmost of the columns whose top component is nonzero and whose main-diagonal component is zero. If there is no such column, exchange the leftmost column with the leftmost of the columns whose top component is nonzero and whose main-diagonal component is nonzero. If there is no such column (in which case the top row is all zero), do nothing.
2. Divide the leftmost column by its top component. (If that component is zero, do nothing, as in that case the whole top row is zero.)
3. Zero the top entry of each column except the leftmost column by subtracting an appropriate multiple of the leftmost column.
4. Rotate rows upward and rotate columns leftward.

In the binary case, step 2 may be omitted. Each of the other steps may be executed in one clock cycle. Abbreviated circuits to control the exchange of step 1 are shown in Figs. 2.22 and 2.23. A circuit to accomplish step 3 is shown in Fig. 2.24.

In general, after the basic set of steps has been performed k times, the first $m - k$ columns contain zeros in their bottom k rows, and the

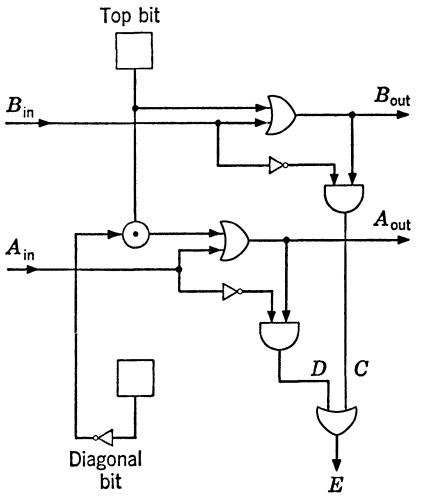


Fig. 2.22 Detailed design of one cell of Fig. 2.23. Input wire A carries a 0 iff no leftmore column contains a 1 in the top position and a 0 in the diagonal position. If the present column has a 1 in the top bit and a 0 in the diagonal bit, A_{out} is set to 1; otherwise A_{out} is allowed to be the same as A_{in}. Input wire B carries a 0 iff all A's are 0 and no leftmore column contains a 1 in the top position.

If either the A signal or the B signal changes from 0 to 1, then a 1 appears at D or C respectively and causes a 1 at E, indicating that this column is to be exchanged with the leftmost column.

The connections between the columns are shown in Fig. 2.23. Note that the signal propagating down the A wire, back, and then down the B wire changes from 0 to 1 only at the column which is to be exchanged. If the upper-left-corner element is 1, then this signal carries 1 everywhere; if every element of the top row is 0, then this signal carries 0 everywhere. In either of these two cases, no change occurs, and no column is exchanged with the leftmost column.

The actual exchange of columns requires many gates not shown. This can be accomplished in a tedious but straightforward manner, using the E signals in each column as control signals for gating the exchange.

lower right $k \times k$ submatrix is triangular idempotent. The reader is invited to prove this assertion, by induction on k. For example, after four repetitions of the basic steps, a 7×7 matrix over GF(5) might appear as follows:

$$\begin{bmatrix} 0 & 0 & 3 & 1 & 4 & 0 & 2 \\ 1 & 2 & 2 & 3 & 4 & 0 & 0 \\ 4 & 3 & 2 & 4 & 3 & 4 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

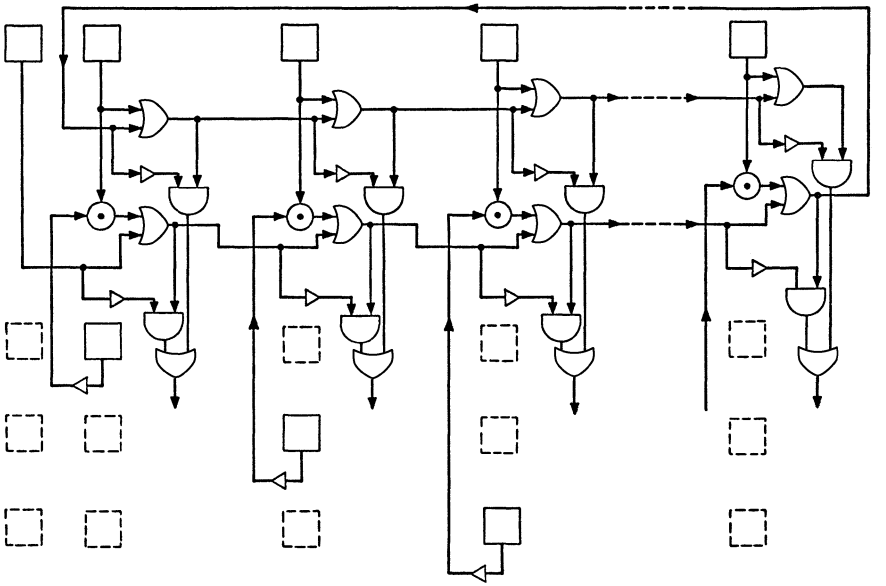


Fig. 2.23 Circuitry for deciding which column of a binary matrix to exchange with the leftmost column.

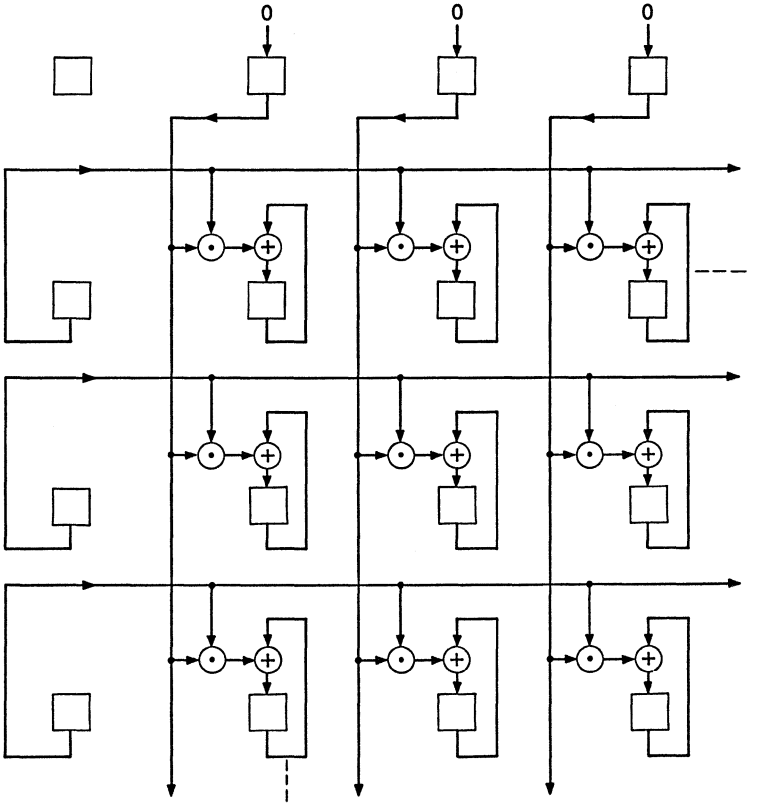


Fig. 2.24 Circuit for performing the operation "zero top entry of each column except the leftmost column by subtracting an appropriate multiple of the leftmost column" on a binary matrix.

Let us now perform the next cycle of basic steps. First, we exchange the first and fifth columns.

$$\begin{bmatrix} 4 & 0 & 3 & 1 & 0 & 0 & 2 \\ 4 & 2 & 2 & 3 & 1 & 0 & 0 \\ 3 & 3 & 2 & 4 & 4 & 4 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Second, we normalize the first column.

$$\begin{bmatrix} 1 & 0 & 3 & 1 & 0 & 0 & 2 \\ 1 & 2 & 2 & 3 & 1 & 0 & 0 \\ 2 & 3 & 2 & 4 & 4 & 4 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Third, we subtract multiples of the first column from the other columns.

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 2 & 4 & 2 & 1 & 0 & 3 \\ 2 & 3 & 1 & 2 & 4 & 4 & 2 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Finally, we rotate rows upward and columns leftward.

$$\begin{bmatrix} 2 & 4 & 2 & 1 & 0 & 3 & 1 \\ 3 & 1 & 2 & 4 & 4 & 2 & 2 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 2 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

In this example, the bottom k rows were unaffected by the first three steps of the procedure. In general, this may not be the case, as we show by another example.

Let us start with the matrix

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 3 & 4 \\ 1 & 2 & 2 & 3 & 4 & 0 & 0 \\ 4 & 3 & 2 & 4 & 3 & 4 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

We exchange the first and sixth columns, and then normalize the first column.

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 4 \\ 0 & 2 & 2 & 3 & 4 & 1 & 0 \\ 3 & 3 & 2 & 4 & 3 & 4 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 2 \\ 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

We then add the first column to the seventh column, and rotate rows and columns to obtain

$$\begin{bmatrix} 2 & 2 & 3 & 4 & 1 & 0 & 0 \\ 3 & 2 & 4 & 3 & 4 & 4 & 3 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 3 & 1 \\ 0 & 0 & 0 & 0 & 0 & 2 & 2 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Thus, we see that any square matrix may be reduced to triangular idempotent form by appropriate column operations which may be readily implemented. For an $m \times m$ binary matrix, these column operations may be performed in $3m$ clock cycles by the circuit which exchanges an arbitrary column with the first column (under the control of the circuit of Figs. 2.22 and 2.23), the circuit of Fig. 2.24, and a circuit which performs the permutation of Figs. 2.20 and 2.21.

In many cases of interest, we shall have m linear equations in m unknowns which are not of the form $\mathbf{Z}\mathfrak{L} = \mathbf{0}$ but of the form $\mathbf{Z}\mathfrak{L} = \mathbf{U}$,

where $\mathbf{U}$ is a given m-dimensional row vector and $\mathfrak{L}$ is a given $m \times m$ matrix. To find $\mathbf{Z}$, we first form the augmented-matrix equation $[\mathbf{Z}, -1]\begin{bmatrix}\mathfrak{L}\\ \mathbf{U}\end{bmatrix} = \mathbf{0}$. This is an abbreviation for

$$[Z_0, Z_1, \cdots, Z_{m-1}, -1]\begin{bmatrix} \mathfrak{L}_{0,0} & \mathfrak{L}_{0,1} & \mathfrak{L}_{0,2} & \cdots & \mathfrak{L}_{0,m-1} \\ \mathfrak{L}_{1,0} & \mathfrak{L}_{1,1} & \mathfrak{L}_{1,2} & \cdots & \mathfrak{L}_{1,m-1} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ \mathfrak{L}_{m-1,0} & \cdots & \cdots & \cdots & \mathfrak{L}_{m-1,m-1} \\ U_0 & U_1 & \cdots & \cdots & U_{m-1} \end{bmatrix} = [0,0, \ldots ,0]$$

By appropriate column operations on the augmented matrix, we can reduce the square $\mathfrak{L}$ matrix to triangular idempotent form. Each column operation performed on the $\mathfrak{L}$ matrix must, of course, also be performed on the augmented row. (If one uses the procedure suggested previously, including row rotations, then only the m rows of the unaugmented $\mathfrak{L}$ matrix should be rotated.)

As the column operations transform the square $\mathfrak{L}$ matrix into a reduced triangular idempotent matrix $\check{\mathfrak{L}}$, they transform the augmented row $\mathbf{U}$ into the reduced augmented row $\check{\mathbf{U}}$. Of course, the reduced augmented row need have no special form; all its components might be nonzero.

If $\check{\mathbf{U}}$, the reduced augmented row, contains a nonzero element ξ in a column in which the reduced $\check{\mathfrak{L}}$ matrix is all zero, then the dot product of this column of the augmented $\check{\mathfrak{L}}$ matrix with the augmented $\mathbf{Z}$ vector is

$$Z_0 0 + Z_1 0 + Z_2 0 + \cdots + Z_{m-1} 0 - 1\xi = -\xi \neq 0$$

This equation is a contradiction. Evidently, the original equations $[\mathbf{Z}, -1]\begin{bmatrix}\mathfrak{L}\\ \mathbf{U}\end{bmatrix} = \mathbf{0}$, like the reduced $[\mathbf{Z}, -1]\begin{bmatrix}\check{\mathfrak{L}}\\ \check{\mathbf{U}}\end{bmatrix} = \mathbf{0}$, are incompatible. They have no solutions.

However, if the reduced augmented row $\check{\mathbf{U}}$ contains zero in every column in which the diagonal component of $\check{\mathfrak{L}} - \mathfrak{I}$ is nonzero, then $\check{\mathbf{U}}$ contains a zero in every all-zero column of $\check{\mathfrak{L}}$. $\check{\mathbf{U}}$ is a linear combination of the rows of $\check{\mathfrak{L}}$, and $\check{\mathbf{U}}$ satisfies the equation $\check{\mathbf{U}}\check{\mathfrak{L}} = \check{\mathbf{U}}$. Thus, one solution of the equation $[\mathbf{Z}, -1]\begin{bmatrix}\check{\mathfrak{L}}\\ \check{\mathbf{U}}\end{bmatrix} = \mathbf{0}$ is given by the choice $\mathbf{Z} = \check{\mathbf{U}}$.

We summarize these results in Theorem 2.56.

Theorem 2.56 Let $\mathfrak{L}$ be a given $m \times m$ matrix, and let $\mathbf{U}$ be a given m-dimensional row vector. Then the equation $\mathbf{Z}\mathfrak{L} = \mathbf{U}$ is equivalent to the augmented-matrix equation $[\mathbf{Z}, -1]\begin{bmatrix}\mathfrak{L}\\ \mathbf{U}\end{bmatrix} = \mathbf{0}$. All solutions $\mathbf{Z}$ of these equations may be obtained as follows:

1. By appropriate column operations on the augmented $\mathfrak{L}$ matrix, reduce the $\mathfrak{L}$ matrix to the triangular idempotent form $\check{\mathfrak{L}}$. These column operations will change the augmented row $\mathbf{U}$ into some other augmented row $\check{\mathbf{U}}$.
2. Subtract ones from the main diagonal of $\check{\mathfrak{L}}$, thereby forming the matrix $\check{\mathfrak{L}} - \mathfrak{I}$.
3. Multiply each component of $\check{\mathbf{U}}$ by the corresponding diagonal component of $\check{\mathfrak{L}} - \mathfrak{I}$. If any of the m products is nonzero, the equation $\mathbf{Z}\mathfrak{L} = \mathbf{U}$ has no solutions. If all the m products are 0, then $\mathbf{Z} = \check{\mathbf{U}}$ is one solution of the equation $\mathbf{Z}\mathfrak{L} = \mathbf{U}$, and all other solutions may be found by adding to $\check{\mathbf{U}}$ any linear combination of nonzero rows of $\check{\mathfrak{L}} - \mathfrak{I}$.

Example 2.57 We consider a system of five binary equations in five unknowns

$$[Z_0,Z_1,Z_2,Z_3,Z_4]\begin{bmatrix}0&0&0&0&0\\1&1&1&1&1\\1&1&1&1&1\\1&1&1&1&0\\1&1&1&1&0\end{bmatrix} = [00001]$$

The augmented matrix is

$$\begin{bmatrix}0&0&0&0&0\\1&1&1&1&1\\1&1&1&1&1\\1&1&1&1&0\\1&1&1&1&0\\0&0&0&0&1\end{bmatrix}$$

This matrix may be reduced by adding the third column into each of the other columns, and then adding the fifth column into the third column, giving

$$\begin{bmatrix}\check{\mathfrak{L}}\\ \check{\mathbf{U}}\end{bmatrix} = \begin{bmatrix}0&0&0&0&0\\0&0&1&0&0\\0&0&1&0&0\\0&0&0&0&1\\0&0&0&0&1\\0&0&1&0&1\end{bmatrix}$$

However, it is more instructive to follow through the details of the way this reduction would be performed by the circuitry in Figs. 2.22 to 2.24. The detailed calculation is shown on p. 62.

	Exchange columns		Clear top row		Rotate rows and columns (to next line)
00000		00000		00000	
11111		11111		11111	
11111		11111		11111	
11110	→	11110	→	11110	
11110		11110		11110	
00001		00001		00001	
11111		11111		10000	
11111		11111		10000	
11101		11101		10010	
11101		11101		10010	
00000		00000		00000	
00010		00010		00010	
00001		10000		10000	
00101		10100		10100	
00101		10100		10100	
00000		00000		00000	
00001		10000		10000	
00100		00100		00100	
01001		10001		10000	
01001		10001		10000	
00000		00000		00000	
00001		00001		00001	
00001		00001		00001	
01000		10000		10001	
00001		10000		10000	
00000		00000		00000	
00010		00010		00010	
00010		00010		00010	
00001		10000		10000	
00011		10010		10010	
00000			10000		
00100			01100		
00100	Add identity		00000		
00001			00011		
00001	→		00000		
00101			00101		

One solution is seen to be 00101. Altogether there are eight solutions, obtained by adding to 00101 any linear combination of the rows: 10000, 01100, 00011.

★ 2.6 SPECIAL METHODS FOR SOLVING SIMULTANEOUS LINEAR EQUATIONS WHEN THE COEFFICIENT MATRIX IS MOSTLY ZEROS

If a large proportion of the m^2 entries in the $m \times m$ $\mathfrak{L}$ matrix are zero, then it may be advantageous to solve the equation $Z\mathfrak{L} = 0$ by special, direct methods. As an example, consider this system of 10 binary equations in 10 binary unknowns:

$$[Z_0,Z_1,Z_2, \ldots ,Z_9] \begin{bmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} = [0,0,0, \ldots , 0]$$

The nonzero entries in each column of the $\mathfrak{L}$ matrix represent the subscripts of a set of Z's which must sum to 0. These indices are as follows:

Equation number	*Variables*		
0	0	4	9
1	1	4	7
2	0	4	8
3	8	9	
4	1	3	5
5	2	5	7
6	2	3	6
7	2	7	
8	6	7	8
9	0	5	

Let us first eliminate those equations which contain no more than two unknowns. According to equation number 3, $Z_8 + Z_9 = 0$, so $Z_9 = Z_8$. In general, we adopt the policy of expressing the Z's with the higher subscripts in terms of Z's with lower subscripts and then eliminat-

ing the Z's with the higher subscripts from all the remaining equations. Thus, we start by substituting 8 for 9 everywhere that 9 occurs:

Equation number	*Variables*			
0	0	4	~~9~~ 8	
1	1	4	7	
2	0	4	8	
3	8	⑨		
4	1	3	5	
5	2	5	7	
6	2	3	6	
7	2	7		
8	6	7	8	
9	0	5		

We have circled the 9 in equation number 3 to indicate that this variable, Z_9, is now defined by this equation.

Continuing, we find the next equation containing only two variables, which is equation number 7: $Z_2 + Z_7 = 0$. We thus circle the 7 in equation 7 and substitute 2 for 7 in equations 1, 5, and 8. In equation 5, the substituted 2 cancels with another 2 already there. Continuing

Equation number	*Variables*			
0	0	4	~~9~~ 8	8
1	1	4	~~7~~ 2	2
2	0	4	8	
3	8	⑨		
4	1	3	5	
5	~~2~~	5	~~2~~	
6	2	3	6	
7	2	⑦		
8	6	~~7~~	8	2
9	0	5		

our first pass through the equations, the next equation we encounter with only two variables is equation 9. We circle the 5 there and substitute 0 for 5 elsewhere.

Equation number	*Variables*			
0	0	4	~~8~~	8
1	1	4	~~2~~	2
2	0	4	8	
3	8	(9)		
4	1	3	~~5~~	0
5	~~5~~	~~5~~	~~5~~	0
6	2	3	6	
7	2	(7)		
8	6	~~7~~	8	2
9	0	(5)		

We now scan the equations again to see if there are any equations containing no circled elements and no more than two variables. The only such equation is number 5, which contains only the variable Z_0, which we circle. Since $Z_0 = 0$ by equation 5, we can eliminate the 0 subscript everywhere else that it occurs.

Equation number	*Variables*			
0	~~0~~	4	~~8~~	8
1	1	4	~~2~~	2
2	~~0~~	4	8	
3	8	(9)		
4	1	3	~~5~~	~~0~~
5	~~5~~	~~5~~	~~5~~	(0)
6	2	3	6	
7	2	(7)		
8	6	~~7~~	8	2
9	~~0~~	(5)		

We then make another pass through the equations. This time equation 0 enables us to define Z_8, and then equation 4 enables us to define Z_3.

Equation number	*Variables*
0	~~8~~ 4 ~~8~~ (8)
1	1 4 ~~7~~ 2
2	~~8~~ ~~4~~ ~~8~~
3	8 (9)
4	1 3 ~~5~~ ~~8~~
5	~~2~~ ~~5~~ ~~7~~ (0)
6	2 3 6
7	2 (7)
8	6 ~~7~~ ~~8~~ 2 4
9	~~8~~ (5)

Equation number	*Variables*
0	~~8~~ 4 ~~8~~ (8)
1	1 4 ~~7~~ 2
2	~~8~~ ~~4~~ ~~8~~
3	8 (9)
4	1 (3) ~~5~~ ~~8~~
5	~~2~~ ~~5~~ ~~7~~ (0)
6	2 ~~3~~ 6 1
7	2 (7)
8	6 ~~7~~ ~~8~~ 2 4
9	~~8~~ (5)

We are now down to only three equations:

Equation 1: $Z_1 + Z_4 + Z_2 = 0$
Equation 6: $Z_2 + Z_6 + Z_1 = 0$
Equation 8: $Z_6 + Z_2 + Z_4 = 0$

Each of these equations has three unknowns. Fortunately, two of these equations have a pair of variables in common, so we may continue to reduce the number of active variables in various ways. One method is to define Z_4 via equation 1 as $Z_4 = Z_1 + Z_2$, which is then substituted into equation 8.

Equation number	*Variables*
0	~~8~~ 4 ~~8~~ (8)
1	1 (4) ~~7~~ 2
2	~~8~~ ~~4~~ ~~8~~
3	8 (9)
4	1 (3) ~~5~~ ~~8~~
5	~~2~~ ~~5~~ ~~7~~ (0)
6	2 ~~3~~ 6 1
7	2 (7)
8	6 ~~7~~ ~~8~~ ~~2~~ ~~4~~ 1
9	~~8~~ (5)

Equation 8 then defines Z_6, and finally equation 6 defines Z_2.

Equation number	*Variables*
0	~~0~~ 4 ~~8~~ ⑧
1	1 ④ ~~7~~ 2
2	~~0~~ ~~4~~ ~~5~~
3	8 ⑨
4	1 ③ ~~5~~ ~~6~~
5	~~2~~ ~~5~~ ~~7~~ ⓪
6	2 ~~5~~ ~~6~~ ~~4~~
7	2 ⑦
8	⑥ ~~7~~ ~~5~~ ~~2~~ ~~4~~ 1
9	~~0~~ ⑤

Equation number	*Variables*
0	~~0~~ 4 ~~8~~ ⑧
1	1 ④ ~~7~~ 2
2	~~0~~ ~~4~~ ~~5~~
3	8 ⑨
4	1 ③ ~~5~~ ~~6~~
5	~~2~~ ~~5~~ ~~7~~ ⓪
6	② ~~5~~ ~~6~~ ~~4~~
7	2 ⑦
8	⑥ ~~7~~ ~~5~~ ~~2~~ ~~4~~ 1
9	~~0~~ ⑤

This completes the reduction of the matrix. Each Z_k, except Z_1, is defined in terms of other Z's with smaller subscripts. The value of Z_1 is evidently arbitrary. If we choose $Z_1 = 0$, then we get the trivial solution $\mathbf{Z} = \mathbf{0}$. However, if we choose $Z_1 = 1$, then we may find the nontrivial solution. We find the circled 0 in equation 5, where we read, "$Z_0 = 0$." We find the circled 2 in equation 6, where we read, "$Z_2 = 0$." We find the circled 3 in equation 4, where we read "$Z_3 = Z_1$." Since $Z_1 = 1$, $Z_3 = 1$. Similarly, we read off the values of each of the other Z's in succession:

k's for which $Z_k = 0$	*k's for which* $Z_k = 1$
0	1
2	3
5	4
7	6
	8
	9

Finally, we may verify that this is indeed a solution, $\mathbf{Z} =$ [0101101011].

With a little practice, the reader should be able to do examples of the following type in less than 5 minutes.

Example 2.61 Solve $Z\mathfrak{L} = 0$, where

$$\mathfrak{L} = \begin{bmatrix} 0&0&0&0&0&0&0&0&0&0&0&0&0&0&0 \\ 0&1&1&0&0&0&0&0&0&0&0&0&0&0&0 \\ 0&0&1&0&1&0&0&0&0&0&0&0&0&0&0 \\ 0&0&0&1&0&0&1&0&0&0&0&0&0&0&0 \\ 0&0&0&0&1&0&0&0&1&0&0&0&0&0&0 \\ 0&0&0&0&0&1&0&0&0&0&1&0&0&0&0 \\ 0&0&0&0&0&0&1&0&0&0&0&0&1&0&0 \\ 0&0&0&0&0&0&0&1&0&0&0&0&0&1&0 \\ 1&1&0&0&0&0&0&0&1&0&0&0&0&0&0 \\ 0&0&1&1&0&0&0&0&0&1&0&0&0&0&0 \\ 0&0&0&0&1&1&0&0&0&0&1&0&0&0&0 \\ 0&0&0&0&0&0&1&1&0&0&0&1&0&0&0 \\ 0&0&0&0&0&0&0&0&1&1&0&0&1&0&0 \\ 0&0&0&0&0&0&0&0&0&0&1&1&0&1&0 \\ 0&0&0&0&0&0&0&0&0&0&0&0&1&1&0 \\ 0&0&0&0&0&0&0&0&0&0&0&0&0&1&0 \end{bmatrix}$$

Solution

8			(8)				
1	8		(1)	~~8~~			
2	1	9	2	~~1~~	(9)		
3	9		(3)	~~9~~	2		
4	2	10	~~4~~	~~2~~	~~10~~	(5)	
5	10		5	(10)			
6	3	11	(6)	~~3~~	~~11~~	2	~~7~~
7	11	→	7	(11)			
8	4	12	~~8~~	4	(12)		
9	12		~~9~~	~~12~~	2	(4)	
10	5	13	~~10~~	~~5~~	(13)		
11	13		~~11~~	~~13~~	(7)		
12	6	14	~~12~~	~~6~~	~~14~~	~~4~~	~~2~~
13	14		~~13~~	(14)			
14	7	15	~~14~~	~~7~~	(15)		

If $Z_2 = 1$, then

$= 0$	1	5	7	8	10	11	13	14	15
$= 1$	2	3	4	6	9	12			

If $Z_2 = 0$, then

$$Z_1 = Z_2 = Z_3 = \cdots = Z_{15} = 0$$

In either case, Z_0 is arbitrary.

When m is very large, it is advantageous to keep an additional table in which we record the number of the equations in which each variable appears. This greatly speeds up the operation of substituting Z_j for Z_k, for now we need not search through all the equations to find the occurrence of Z_k. The numbers of the equations in which Z_k appears may be looked up in the second table. There is a certain amount of extra work required to update the second table as the calculation proceeds, but this proves small compared with the searching time saved.

PROBLEMS

2.1. Use the continued-fractions version of Euclid's algorithm to find binary polynomials A and B such that

$$A(x)(x^5 + x^3 + 1) + B(x)(x^6 + x^5 + x^4 + x^2 + 1) = 1$$

2.2. Find all solutions (if there are any) of the equation $\mathbf{Z}\mathfrak{L} = \mathbf{U}$, where $\mathfrak{L}$ is the 4×4 matrix over the integers mod 7:

$$\mathfrak{L} = \begin{bmatrix} 1 & 3 & 3 & 6 \\ 0 & 4 & 0 & 2 \\ 5 & 1 & 2 & 6 \\ 2 & 0 & 4 & 5 \end{bmatrix} \quad \text{and} \quad \mathbf{U} = [0014]$$

2.3. Find all solutions of the binary equations $\mathbf{Z}\mathfrak{L} = \mathbf{0}$, where

$$\mathfrak{L} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

3

The Number of Irreducible q-ary Polynomials of Given Degree

We have seen that we can construct a double-error-correcting binary BCH code of block length $2^m - 1$ if we can find an irreducible binary polynomial of degree m. If we cannot find an irreducible binary polynomial of degree m, we cannot construct such a code by that method. A priori, we have no guarantee that there exist irreducible binary polynomials of all degrees. One method of proving that there do exist irreducible binary polynomials of every degree is to enumerate them, obtain an explicit formula for the number of them, and show that the answer is always positive. Although this approach may appear to complicate the problem unnecessarily, the results justify the effort. The techniques also prove valuable for solving other problems in subsequent chapters.

3.1 A BRUTE-FORCE ATTACK

In order to enumerate the irreducible binary polynomials by degree, let us first try brute-force methods. We know that there are two binary

polynomials of degree 1, namely, x and $x + 1$. Both are irreducible. There are four binary polynomials of degree 2, three of which factor. The complete listing is as follows:

$$\begin{aligned} x^2 &= x^2 \\ x^2 + 1 &= (x + 1)^2 \\ x^2 + x &= x(x + 1) \\ x^2 + x + 1 &= \text{irreducible} \end{aligned}$$

There are eight binary cubics, and, in general, there are 2^d binary polynomials of degree d, corresponding to the two choices each of $f_0, f_1, f_2, \ldots, f_{d-1}$ in the expression $f(x) = \left(\sum_{i=0}^{d-1} f_i x^i\right) + x^d$. We may factor the eight binary cubics as follows:

$$\begin{aligned} x^3 &= x^3 \\ x^3 + 1 &= (x + 1)(x^2 + x + 1) \\ x^3 + x &= x(x + 1)^2 \\ x^3 + x + 1 &= \text{irreducible} \\ x^3 + x^2 &= x^2(x + 1) \\ x^3 + x^2 + 1 &= \text{irreducible} \\ x^3 + x^2 + x &= x(x^2 + x + 1) \\ x^3 + x^2 + x + 1 &= (x + 1)^3 \end{aligned}$$

Of the eight binary cubics, two factor into the product of a quadratic and a linear factor; four factor into the product of three linear factors, and the other two are irreducible. In general, there are $n + 1$ binary polynomials of degree n which factor into n linear factors, corresponding to the $n + 1$ choices of $k = 0, 1, 2, \ldots, n$ in the expression

$$f(x) = x^k(x + 1)^{n-k} \tag{3.11}$$

If we let I_n denote the number of irreducible binary polynomials of degree n, then we have $I_1 = 2, I_2 = 1, I_3 = 2$. Let us now compute I_4 without tabulating all 16 binary quartics. In order to count the number of ways the quartic might factor, we must consider all possible factorizations. The quartic can factor into the product of an irreducible polynomial of degree 3 and a polynomial of degree 1 in $I_3 I_1 = 4$ different ways. It can factor into the product of two irreducible quadratics in $I_2^2 = 1$ way. It can factor into the product of an irreducible quadratic and two linear factors; this can happen in 1 way with distinct linear factors and in $I_2 I_1 = 2$ ways with the same linear factor repeated, a total of 3 ways. Finally, the

binary quartic can factor into the product of four linear factors in five ways. We summarize these possibilities as follows:

	Factorization	*Number of ways*
$4 =$	$3+1$	4
	$2+2$	1
	$2+1+1$	3
	$1+1+1+1$	5
		13

$I_4 = 2^4 - 13 = 3$

Knowing I_4, we may go on to compute I_5:

	Factorization	*Number of ways*
$5 =$	$4+1$	6
	$3+2$	2
	$3+1+1$	6
	$2+2+1$	2
	$2+1+1+1$	4
	$1+1+1+1+1$	6
		26

$I_5 = 2^5 - 26 = 6$

We may then compute I_6:

	Factorization	*Number of ways*
$6 =$	$5+1$	12
	$4+2$	3
	$4+1+1$	9
	$3+3$	3
	$3+2+1$	4
	$3+1+1+1$	8
	$2+2+2$	1
	$2+2+1+1$	3
	$2+1+1+1+1$	5
	$1+1+1+1+1+1$	7
		55

$I_6 = 2^6 - 55 = 9$

If we are sufficiently patient and careful, we can go on to compute $I_7, I_8, \ldots$. Fortunately, however, we can avoid most of these calculations by using a more sophisticated bookkeeping scheme known as the *method of generating functions.*

3.2 GENERATING FUNCTIONS

In general, if we have a sequence $A_0, A_1, A_2, \ldots$, we may define the generating function as the formal power series

$$A(z) = \sum_{i=0}^{\infty} A_i z^i$$

The A_i may be rational numbers or elements in any given field. Two generating functions, $A(z) = \sum_{i=0}^{\infty} A_i z^i$ and $B(z) = \sum_{i=0}^{\infty} B_i z^i$, are said to be equal iff $A_i = B_i$ for all i. If we wish to make the weaker assertion that $A_i = B_i$ for $i = 0, 1, 2, \ldots, k-1$, then we say that $A(z) \equiv B(z) \bmod z^k$.

The sum or difference of the two generating functions, $A(z) \pm B(z)$, is given by

$$\sum_{i=0}^{\infty} (A_i \pm B_i) z^i$$

and the product is given by

$$A(z)B(z) = \sum_{i=0}^{\infty} \Big(\sum_{j=0}^{i} A_j B_{i-j} \Big) z^i$$

We say that $C(z) = A(z)/B(z)$ iff $C(z)B(z) = A(z)$. If it exists, the quotient of two generating functions is unique, for if $D(z)B(z) = C(z)B(z)$, then $0 = D(z)B(z) - C(z)B(z) = [D(z) - C(z)]B(z)$, and either $D(z) = C(z)$ or $B(z) = 0$, as the reader is invited to verify. If $A_0 = 0$, then $B(z)A(z) \equiv 0 \bmod z$ for any $B(z)$, and the equation $B(z)A(z) = 1$ has no solution for $B(z)$. However, if $A_0 \neq 0$, then

$$\frac{A(z)}{A_0} = 1 + \sum_{i=1}^{\infty} \left(\frac{A_i}{A_0} \right) z^i$$

and

$$\frac{A_0}{A(z)} = \frac{1}{1 + \sum_{i=1}^{\infty} \left(\frac{A_i}{A_0} \right) z^i} = \sum_{n=0}^{\infty} \left[- \sum_{i=1}^{\infty} \left(\frac{A_i}{A_0} \right) z^i \right]^n$$

Therefore,

$$\frac{1}{A(z)} = \frac{1}{A_0} \sum_{n=0}^{\infty} \left[- \sum_{i=1}^{\infty} \left(\frac{A_i}{A_0} \right) z^i \right]^n$$

We conclude that the generating function $A(z)$ has a multiplicative inverse iff $A_0 \neq 0$.

We say that $A(z)$ is an *even* generating function iff $A_i = 0$ for all odd i. We say that $A(z)$ is an *odd* generating function iff $A_i = 0$ for all even i. The sum or difference of two even generating functions is even, and the sum or difference of two odd generating functions is odd. The product of two even generating functions, or of two odd generating functions, is even, but the product of an even generating function and an odd generating function is odd. The multiplicative inverse of an even generating function, if it exists, is even. Any generating function may be written as the sum of an even generating function and an odd generating function: $A(z) = \hat{A}(z) + \check{A}(z)$. I use the accents $\hat{}$ and $\check{}$ to denote even and odd functions, respectively. Notice the resemblance of these accents to the graphs of the even cosine function and the odd sine function.†

It is important to notice that these properties of generating functions do not depend in any way on the convergence or divergence of the series $\sum_{i=0}^{\infty} A_i z^i$ for any actual values of z. For example, if $A = i!$, then this series diverges for all nonzero complex values of z. Furthermore, there is no need for the coefficients A_i to be real or complex numbers. We can use generating functions over *any* field.

Given any generating function

$$A(z) = \sum_{n=0}^{\infty} A_n z^n$$

we may define its *formal derivative*

$$A'(z) = \sum_{n=1}^{\infty} n A_n z^{n-1}$$

If

$$C(z) = A(z)B(z) = \sum_{n=0}^{\infty} \left(\sum_{k=0}^{n} A_k B_{n-k} \right) z^n$$

† The reader is warned to avoid confusing $\check{f}(z)$, the odd part of $f(z)$, with $\tilde{f}(z)$, the reciprocal polynomial of $f(z)$.

then

$$\begin{aligned} C'(z) &= \sum_{n=1}^{\infty} n \Big(\sum_{k=0}^{n} A_k B_{n-k} \Big) z^{n-1} \\ &= \sum_{n=1}^{\infty} \Big[\sum_{k=0}^{n} (k + n - k) A_k B_{n-k} \Big] z^{n-1} \\ &= \sum_{n=1}^{\infty} \Big[\sum_{k=0}^{n} k A_k B_{n-1-(k-1)} + \sum_{k=0}^{n} (n - k) A_k B_{n-k} \Big] z^{n-1} \\ &= A'(z)B(z) + A(z)B'(z) \end{aligned}$$

By induction we may extend this result to the formula for the derivative of the product of the k generating functions $A^{(1)}(z)$, $A^{(2)}(z)$, . . . , $A^{(k)}(z)$.

Theorem 3.21

$$\Big[\prod_{i=1}^{k} A^{(i)}(z) \Big]' = \sum_{j=1}^{k} [A^{(j)}(z)]' \prod_{i \neq j} A^{(i)}(z)$$

Corollary 3.22

$$\frac{\Big[\prod_{i=1}^{k} A^{(i)}(z) \Big]'}{\prod_{i=1}^{k} A^{(i)}(z)} = \sum_{j=1}^{k} \frac{[A^{(j)}(z)]'}{A^{(j)}(z)}$$

Thus, formal derivatives behave according to the familiar rules of ordinary derivatives. The only difference is that formal derivatives do not involve any limiting operations.

Formal derivatives may also be applied to generating functions which are ordinary polynomials. In this case we obtain the important result of Theorem 3.23.

Theorem 3.23 Let $g(z)$ be an irreducible polynomial with nonzero derivative $g'(z)$.† Then $g(z)$ is a *repeated factor* of the polynomial $f(z)$ iff $g(z)$ divides $(f(z),f'(z))$, the gcd of $f(z)$ and $f'(z)$.

† In a finite field, every irreducible polynomial has nonzero derivative, although in certain infinite fields of finite characteristic there do exist irreducible polynomials which have derivatives that are identically 0.

Proof Let $g(z)$ be an irreducible factor of $f(z)$, so that $f(z) = g(z)h(z)$. Taking formal derivatives gives $f'(z) = g'(z)h(z) + g(z)h'(z)$. If $g(z)$ divides $f'(z)$, then it also divides $g'(z)h(z)$. But deg $g'(z) <$ deg $g(z)$, so $g(z)$ cannot divide $g'(z)$. In order to divide $f'(z)$, $g(z)$ must divide $h(z)$. Therefore $g(z)$ is a repeated factor of $f(z)$ iff it is a factor of both $f(z)$ and $f'(z)$, which happens iff $g(z)$ divides the gcd of $f(z)$ and $f'(z)$. Q.E.D.

This theorem is true in all fields. For example, let $f(x)$ be the binary polynomial $x^5 + x^4 + x^2 + x$. We then have

$$\begin{aligned} f(x) &= x^5 + x^4 + x^2 + x \\ f'(x) &= 5x^4 + 4x^3 + 2x + 1 \\ &= x^4 + 1 \\ (f(x),f'(x)) &= x^2 + 1 = (x + 1)^2 \end{aligned}$$

We verify this by factoring $f(x)$ into irreducible factors:

$$f(x) = x(x + 1)^2(x^2 + x + 1)$$

3.3 THE NUMBER OF IRREDUCIBLE MONIC q-ARY POLYNOMIALS OF GIVEN DEGREE—A REFINED APPROACH

Suppose we let A_k be the number of binary polynomials of degree k which are powers of a given irreducible binary polynomial $f(x)$, where deg $f = m$. Since the jth power of the given irreducible binary polynomial has degree jm, we conclude that

$$A_k = \begin{cases} 1 & \text{if } k \text{ is a multiple of } m \\ 0 & \text{if } k \text{ is a nonmultiple of } m \end{cases}$$

Allowing the polynomial $f(x)^0 = 1$, which has degree 0, we have $A_0 = 1$. The generating function for the A's is given by

$$\begin{aligned} A(z) &= 1 + z^m + z^{2m} + z^{3m} + \cdots \\ &= \frac{1}{1 - z^m} \end{aligned}$$

We call $A(z)$ the *enumerator by degree* of the powers of $f(x)$.

Similarly, if $g(x)$ is an arbitrary polynomial of degree n, then the enumerator by degree of the powers of $g(x)$ is $B(z) = 1/(1 - z^n)$.

Let C_k be the number of binary polynomials of degree k whose only irreducible factors are $f(x)$ and $g(x)$. Every such polynomial must be the product of some power of $g(x)$ and some power of $f(x)$. If the power of $g(x)$ has degree i, then the power of $f(x)$ must have degree

$k - i$. Thus $C_k = \sum_i B_i A_{k-i}$ and $C(z) = A(z)B(z)$. More generally, if $A(z)$ is the enumerator by degree of some set of polynomials $f^{(1)}(x)$, $f^{(2)}(x)$, . . . , and if $B(z)$ is the enumerator by degree of another set of polynomials $g^{(1)}(x)$, $g^{(2)}(x)$, . . . , and if $f^{(i)}(x)$ and $g^{(j)}(x)$ are relatively prime for all i and j, then $C(z) = A(z)B(z)$ is the enumerator by degree of the set of products $f^{(n)}(x)g^{(m)}(x)$. As before, $C_k = \sum_i B_i A_{k-i}$.

By induction, we may extend this result to as many sets as we like. The result is Theorem 3.31.

Theorem 3.31 Let S_1, S_2, . . . be sets of polynomials which have the property that any two polynomials in different sets are relatively prime. Let $S_1 * S_2 * \cdots$ be the set of polynomials which are products of a polynomial from each set. Then the enumerator by degree of $S_1 * S_2 * \cdots$ is $\prod_i A^{(i)}(z)$, where $A^{(i)}(z)$ is the enumerator by degree of S_i.

For example, let S_1 be the powers of the binary polynomial x; let S_2 be the powers of the binary polynomial $x + 1$. Then $S_1 * S_2$ is the set of binary polynomials which factor into linear factors. According to the theorem, the enumerator of this set is $[1/(1 - z)]^2$. We may verify this by the calculation

$$\frac{1}{(1-z)^2} = \frac{d}{dz}\frac{1}{1-z} = \frac{d}{dz}\left(\sum_{n=0}^{\infty} z^n\right) = \sum_{n=0}^{\infty} nz^{n-1} = \sum_{k=0}^{\infty} (k+1)z^k$$

Thus, there are $k + 1$ binary polynomials of degree k which are products of linear factors. This agrees with the argument preceding Eq. (3.11).

Similarly, the enumerator by degree of the binary polynomials which are products of x, $x + 1$, $x^2 + x + 1$ is

$$\left(\frac{1}{1-z}\right)^2 \frac{1}{1-z^2}$$

Likewise, the enumerator by degrees for binary polynomials which are products of powers of a set of irreducible polynomials which includes I_1 irreducible polynomials of degree 1, I_2 irreducible polynomials of degree 2, I_3 irreducible polynomials of degree 3, . . . must be

$$\prod_{m=1}^{\infty} \left(\frac{1}{1-z^m}\right)^{I_m}$$

However, the set of binary polynomials which are products of powers of irreducible binary polynomials is precisely the set of all binary polynomials. We know that there are 2^k binary polynomials of degree k, so the enumerator by degree of the set of all binary polynomials is

$$\sum_{k=0}^{\infty} 2^k z^k = \frac{1}{1-2z}$$

We thus have the binary case of the general theorem.

Theorem 3.32 Let I_m be the number of distinct irreducible monic polynomials of degree m over a finite field of q elements. Then

$$\frac{1}{1-qz} = \prod_{m=1}^{\infty} \left(\frac{1}{1-z^m}\right)^{I_m}$$

Proof

$$\left\{\begin{matrix}\text{The set of all} \\ \text{monic polynomials}\end{matrix}\right\} = \left\{\begin{matrix}\text{the set of products of powers} \\ \text{of irreducible monic polynomials}\end{matrix}\right\}$$

Q.E.D.

The theorem is seen to be a quantitative refinement of the unique-factorization theorem for monic polynomials over a finite field of q elements.

If we expand both sides of this theorem, we soon find ourselves doing the same sort of calculations we encountered with the brute-force approach. In the binary case, for example, we might know that $I_1 = 2$, $I_2 = 1$, $I_3 = 2$. We might then find I_4 by equating the coefficients of z^4 in the equation

$$\begin{aligned} 1 + 2z + 4z^2 + 8z^3 + 16z^4 + \cdots \\ = (1 + z + z^2 + z^3 + z^4 \cdots)^2(1 + z^2 + z^4 + \cdots) \\ (1 + z^3 + z^6 + \cdots)^2(1 + z^4 + \cdots)^{I_4} \cdots \end{aligned}$$

The calculation of the coefficient of z^4 on the right-hand side of this equation involves about the same amount of work as the brute-force approach in Sec. 3.1. In this sense, then, the generating-function formulation tells us nothing new; it merely furnishes us with a shorthand in which to abbreviate the complicated calculations of the brute-force method.

In another sense, however, the generating function does furnish

us with something new, for it suggests manipulations by which we may transform these complicated rules into other forms. For example, we may invert both sides of Theorem 3.32 to obtain

$$1 - qz = \prod_{m=1}^{\infty} (1 - z^m)^{I_m} \tag{3.33}$$

Equation (3.33) has an interesting interpretation. Since $\prod_{m=1}^{\infty} (1 + z^m)^{I_m}$ is the enumerator by degree for polynomials which are products of distinct irreducible factors, $\prod_{m=1}^{\infty} (1 - z^m)^{I_m}$ is the difference between the enumerator by degree for polynomials which are products of an even number of distinct irreducible factors and the enumerator by degree for polynomials which are products of an odd number of distinct irreducible factors. The one monic polynomial of degree 0 is the product of no irreducible factors, and each of the q monic irreducible polynomials of degree 1 is the product of one irreducible factor. Thus, Eq. (3.33) is equivalent to Theorem 3.34.

Theorem 3.34 If $m \geq 2$, the number of monic polynomials of degree m which are products of an even number of distinct irreducible factors is equal to the number of monic polynomials of degree m which are products of an odd number of distinct irreducible factors.

Theorem 3.34 is true in every finite field.

To obtain more information about I_m, we perform further manipulations on Eq. (3.33). We may apply Corollary 3.22 to obtain

$$-\frac{q}{1 - qz} = \sum_{m=1}^{\infty} -\frac{mI_m z^{m-1}}{1 - z^m}$$

Multiplying by $-z$ gives

$$\frac{qz}{1 - qz} = \sum_{m=1}^{\infty} mI_m \frac{z^m}{1 - z^m}$$

$$\sum_{k=1}^{\infty} q^k z^k = \sum_{m=1}^{\infty} mI_m \sum_{\substack{k, \\ m|k}} z^k = \sum_{k=1}^{\infty} \sum_{\substack{m, \\ m|k}} mI_m z^k$$

We thus obtain Theorem 3.35.

Theorem 3.35

$$q^k = \sum_{\substack{m, \\ m|k}} mI_m$$

The sum is taken over all m which divide k, including 1 and k itself.

A number of properties of I_m may be deduced directly from this equation. First, we see that $I_1 = q$. Next, since $I_m \geq 0$ for all m, we deduce that $I_k \leq (q^k - q)/k$, with equality iff k is prime. In particular, $I_2 = (q^2 - q)/2 > 0$; $I_3 = (q^3 - q)/3 > 0$. We may also get a crude lower bound on I_k by arguing that

$$q^k = \sum_{\substack{m, \\ m|k}} mI_m \leq kI_k + \sum_{i=0}^{k/2} q^i < kI_k + q^{k/2+1}$$

so

$$I_k > \frac{q^k - q^{k/2+1}}{k} = \frac{q^k}{k}(1 - q^{-k/2+1})$$

This shows that $I_k > 0$ for all $k > 2$. Since we have seen that $I_1 > 0$ and $I_2 > 0$, we conclude that there exist irreducible monic polynomials of every degree over every finite field. If $f(x)$ is an irreducible polynomial of degree m over the integers mod p, the p^m residue classes mod $f(x)$ form a finite field of p^m elements. This proves Theorem 3.36.

Theorem 3.36 If p is any prime and m is any integer, there exists a finite field having p^m elements.

Although this argument does not give us a practical method for finding any irreducible polynomial of large degree, it does tell us how many there are. Over any finite field, a randomly chosen polynomial of large degree, m, will be irreducible with probability about $1/m$. More precisely, we have shown that

$$\frac{1}{k}(1 - q^{-k/2+1}) < \frac{I_k}{q^k} \leq \frac{1}{k}(1 - q^{-k+1}) \tag{3.37}$$

In order to obtain a closed-form expression for I_k, we must use the Moebius inversion formula.

★ 3.4 THE MOEBIUS INVERSION FORMULAS

The formula

$$q^k = \sum_{\substack{m, \\ m|k}} mI_m$$

is a special case of the formula

$$f(n) = \sum_{\substack{d, \\ d|n}} g(d)$$

where $g(d)$ is an arbitrary function. We wish to find a formula which explicitly expresses g in terms of f. It turns out that the proof of this formula (Theorem 3.41), once stated, is much shorter than its derivation. We shall now derive the desired formula by a brute-force approach that does not anticipate the answer. If we let

$$n = \prod_{i=1}^{j} p_i^{e_i}$$

where the p_i are primes, then d divides n iff d is of the form

$$d = \prod_{i=1}^{j} p_i^{k_i}$$

where $0 \leq k_i \leq e_i$. The formula

$$f(n) = \sum_{\substack{d, \\ d|n}} g(d)$$

now becomes

$$f\left(\prod_{i=1}^{j} p_i^{e_i}\right) = \sum_{k_1=0}^{e_1} \sum_{k_2=0}^{e_2} \cdots \sum_{k_j=0}^{e_j} g\left(\prod_{i=1}^{j} p_i^{k_i}\right)$$

Since this formula is valid for all values of n, we may also use this formula with

$$n = p_1^{e_1-1} \prod_{i=2}^{j} p_i^{e_i}$$

In this case we obtain

$$f\left(p_1^{e_1-1} \prod_{i=2}^{j} p_i^{e_i}\right) = \sum_{k_1=0}^{e_1-1} \sum_{k_2=0}^{e_2} \cdots \sum_{k_j=0}^{e_j} g\left(\prod_{i=1}^{j} p_i^{k_i}\right)$$

Subtracting gives

$$\sum_{k_1=e_1-1}^{e_1} (-1)^{e_1-k_1} f\left(p_1^{k_1} \prod_{i=2}^{j} p_i^{e_i}\right) = \sum_{k_2=0}^{e_2} \cdots \sum_{k_j=0}^{e_j} g\left(p_1^{e_1} \prod_{i=2}^{j} p_i^{k_i}\right)$$

Thus, we have replaced one of the sums on g by a sum on f. Continuing in this manner gives

$$\sum_{k_1=e_1-1}^{e_1} \sum_{k_2=e_2-1}^{e_2} (-1)^{e_1+e_2-k_1-k_2} f\left(p_1^{k_1} p_2^{k_2} \prod_{i=3}^{j} p_i^{e_i}\right) = \sum_{k_2=0}^{e_2} \cdots \sum_{k_j=0}^{e_j} g\left(p_1^{e_1} p_2^{e_2} \prod_{j=3}^{j} p_i^{k_i}\right)$$

and finally

$$\sum_{k_1=e_1-1}^{e_1} \sum_{k_2=e_2-1}^{e_2} \cdots \sum_{k_j=e_j-1}^{e_j} (-1)^{\sum_i e_i - \sum_i k_i} f\left(\prod_{i=1}^{j} p_i^{k_i}\right) = g\left(\prod_{i=1}^{j} p_i^{e_i}\right)$$

Substituting $m_i = e_i - k_i$, this becomes

$$g\left(\prod_{i=1}^{j} p_i^{e_i}\right) = \sum_{m_1=0}^{1} \sum_{m_2=0}^{1} \cdots \sum_{m_j=0}^{1} (-1)^{\sum_i m_i} f\left(\prod_{i=1}^{j} p_i^{e_i-m_i}\right)$$

This is an explicit expression of g in terms of f. We may also write this formula as

$$g\left(\prod_{i=1}^{j} p_i^{e_i}\right) = \sum_{m_1=0}^{1} \sum_{m_2=0}^{1} \cdots \sum_{m_j=0}^{1} \mu\left(\prod_{i=1}^{j} p_i^{m_i}\right) f\left(\prod_{i=1}^{j} p_i^{e_i-m_i}\right)$$

where

$$\mu\left(\prod_{i=1}^{i} p_i^{m_i}\right) = (-1)^{\sum_i m_i}$$

when each $m_i = 0$ or 1. If we define

$$\mu\left(\prod_{i=1}^{j} p_i^{m_i}\right) = 0$$

whenever any $m_i \geq 2$, then we may increase the upper limits of summation to any desired limits. In particular, we may write

$$g\left(\prod_{i=1}^{j} p_i^{e_i}\right) = \sum_{m_1=0}^{e_1} \sum_{m_2=0}^{e_2} \cdots \sum_{m_j=0}^{e_j} \mu\left(\prod_{i=1}^{j} p_i^{m_i}\right) f\left(\prod_{i=1}^{j} p_i^{e_i-m_i}\right)$$

This is equivalent to Theorem 3.41.

The Moebius inversion formula 3.41 If

$$f(n) = \sum_{\substack{d, \\ d|n}} g(d)$$

then

$$g(n) = \sum_{\substack{d, \\ d|n}} \mu(d) f\left(\frac{n}{d}\right)$$

where $\mu(d)$ is the Moebius function, defined by

$$\mu(d) = \begin{cases} 1 & \text{if } d = 1 \\ (-1)^k & \text{if } d \text{ is the product of } k \text{ distinct primes} \\ 0 & \text{if } d \text{ contains any repeated prime factors} \end{cases}$$

The Moebius inversion formula may also be expressed in other forms. If one substitutes $c = n/d$, one has

$$g(n) = \sum_{\substack{c, \\ c|n}} \mu\left(\frac{n}{c}\right) f(c)$$

The most difficult part of the derivation of the Moebius inversion formula is motivating the definition of the Moebius function. Once this function has been defined, it is easily verified that

$$\sum_{\substack{d, \\ d|n}} \mu(d) = \begin{cases} 1 & \text{if } n = 1 \\ 0 & \text{if } n > 1 \end{cases}$$

This is equivalent to the property that

$$\sum_{\substack{n, \\ d|n|m}} \mu\left(\frac{m}{n}\right) = \begin{cases} 1 & \text{if } d = m \\ 0 & \text{if } d \text{ is a proper divisor of } m \end{cases}$$

In other words, as Rota (1964) has so eloquently pointed out, the Moebius function is the inverse of the delta function on the lattice of integers, with divisibility taken as the partial ordering. Once this is established, there is no problem in proving that if

$$f(n) = \sum_{\substack{d, \\ d|n}} g(d)$$

then

$$g(m) = \sum_{\substack{n, \\ n|m}} f(n)\mu\left(\frac{m}{n}\right)$$

The proof takes one line:

$$\sum_{\substack{n, \\ n|m}} \sum_{\substack{d, \\ d|n}} g(d)\mu\left(\frac{m}{n}\right) = \sum_{\substack{d, \\ d|m}} g(d) \sum_{\substack{n, \\ d|n|m}} \mu\left(\frac{m}{n}\right) = g(m)$$

Similarly, we may prove another form of the theorem.

Multiplicative Moebius inversion theorem 3.42 If

$$f(n) = \prod_{\substack{d, \\ d|n}} g(d) \quad \text{and} \quad g(n) \neq 0, \quad \text{for } n|m, n < m$$

then

$$g(m) = \prod_{\substack{n, \\ n|m}} f(n)^{\mu(m/n)}$$

Proof

$$\prod_{\substack{m, \\ n|m}} \prod_{\substack{d, \\ d|n}} g(d)^{\mu(m/n)} = \prod_{\substack{d, \\ d|m}} \prod_{\substack{n, \\ d|n|m}} g(d)^{\mu(m/n)} = \prod_{\substack{d, \\ d|m}} g(d)^{\sum_{\substack{n, \\ d|n|m}} \mu\left(\frac{m}{n}\right)} = g(m)$$

Q.E.D.

For the enumeration of irreducible polynomials, the original form of the Moebius inversion formula proves most useful. Theorem 3.43 is the immediate consequence of Theorems 3.35 and 3.41.

Theorem 3.43

$$I_k = \frac{1}{k} \sum_{\substack{d, \\ d|k}} \mu(d) q^{(k/d)}$$

If $q = 2$, we have $I_1 = 2$, $I_2 = \frac{1}{2}(2^2 - 2) = 1$, $I_3 = \frac{1}{3}(2^3 - 2) = 2$, $I_4 = \frac{1}{4}(2^4 - 2^2) = 3$, $I_5 = \frac{1}{5}(2^5 - 2) = 6$, ... Even for large

values of k the formula is quite tractable. For example,

$$I_{60} = \tfrac{1}{60}(2^{60} - 2^{30} - 2^{20} - 2^{12} + 2^{10} + 2^6 + 2^4 - 2^2)$$
$$= 19{,}215{,}358{,}392{,}200{,}893$$

In general, the Moebius inversion formula expresses I_k as the sum of 2^j terms, where j is the number of distinct prime factors of k.

PROBLEMS

3.1. Consider the generating function

$$f(z) = \Big(\sum_{n=0}^{\infty} n!z^n\Big)^2$$

Prove that

$$2\sum_{k=0}^{n}(k+1)^2k!(n-k)! = (n+1)\sum_{j=0}^{n} j!(n+1-j)! \qquad \text{for all } n$$

by computing df/dz two different ways. Are you disturbed by the divergence of the series for $f(z)$? Check the above identity when $n = 3$ by showing that both sides are 256.

3.2. (*a*) The binary polynomials mod $x^5 + x^2 + 1$ constitute a finite field having 32 elements. How many irreducible monic quadratic polynomials are there over this field?

(*b*) The double-error-correcting binary BCH code described in Sec. 1.4 has block length $n = 31$, 10 check digits, and 21 message digits. There are 2^{10} possible syndromes. How many of these syndromes correspond to a coset of weight 0,1, 2, ≥ 3? (Four questions.) How does the decoding procedure fail when the received word lies in a coset of weight ≥ 3?

3.3. Obtain the enumerators by degree for each of the following subsets of monic polynomials over a field of order q.

(*a*) Perfect squares

$$\frac{1}{1 - qz^2}$$

(*b*) Products of distinct irreducible factors

$$\frac{1 - qz^2}{1 - qz}$$

Hint: An arbitrary monic polynomial can be uniquely factored into a perfect square and a square-free factor.

(*c*) Products of an even number of distinct irreducible factors

$$\frac{1 - qz + (1 - qz^2)/(1 - qz)}{2}$$

Hint: Review Eq. (3.33).

(*d*) Products of an odd number of distinct irreducible factors

$$\frac{qz - 1 + (1 - qz^2)/(1 - qz)}{2}$$

(*e*) Products of nonlinear irreducible factors

$$\frac{(1 - z)^q}{1 - qz} = \prod_{m=2}^{\infty} \left(\frac{1}{1 - z^m}\right)^{I_m}$$

(*f*) Products of distinct nonlinear irreducible factors

$$\frac{1 - qz^2}{(1 - qz)(1 + z)^q}$$

(*g*) Products of an even number of distinct nonlinear irreducible factors

$$\frac{(1 - qz^2)/(1 - qz)(1 + z)^q + (1 - qz)/(1 - z)^q}{2}$$

(*h*) Products of an odd number of distinct nonlinear irreducible factors

$$\frac{(1 - qz^2)/(1 - qz)(1 + z)^q - (1 - qz)/(1 - z)^q}{2}$$

3.4 Express each generating function obtained in Prob. 3.3 as an explicit power series in z. You may assume $q = 2$. *Hint:* Use partial-fraction expansions.

3.5. How many different necklaces of length m can be made from beads of q given colors? Two necklaces are considered the same iff one can be rotated into the other. For example, –001101– is the same as –101001– but different from –001011–. The necklace –001011– can be obtained by picking –001101– up and turning it over, but it cannot be obtained by a rotation.

3.6. Prove that the expected number of irreducible factors of a randomly chosen polynomial of large degree, n, over a finite field of order q is approximately $\ln n$.

Hint: Let $F(y,z) = \prod_{m=1}^{\infty} (1 - yz^m)^{-I_m}$. Interpret $F(y,z)$ as a bivariate generating function, and evaluate $\partial F/\partial y$ at $y = 1$.

4
The Structure of Finite Fields

4.1 DEFINITIONS

Various equivalent definitions of a field are possible. I prefer a slightly redundant, relatively informal definition.

Field definition 4.11 A field is a set of elements, including 0 and 1, any pair of which may be added or multiplied (denoted by $+$ and $*$, respectively) to give a unique result in the field. The addition and multiplication are associative and commutative, and multiplication distributes over addition in the usual way: $u * (v + w) = u * v + u * w$. Every field element u has a unique negative $-u$ such that $u + (-u) = 0$. Every nonzero field element u has a unique reciprocal field element $1/u$, such that $u * (1/u) = 1$. For every field element u, $0 + u = u = 1 * u$, and $0 * u = 0$.

These properties of a field are not all independent; some of them may be derived from others. The reader interested in minimizing the

number of field axioms will find further discussions of this question in books by Birkhoff and MacLane (1965) or van der Waerden (1931). For our purposes it is most convenient to postulate all the properties given above.

The *order of a field* is the number of elements in the field. If the order is infinite, we call the field an *infinite field;* if the order is finite, we call the field a *finite field.* The rational numbers, the real numbers, and the complex numbers are examples of infinite fields. If p is a prime, the integers mod p form a finite field of order p. If $f(x)$ is an irreducible polynomial of degree m with coefficients in the field of integers mod p, then the residue classes† mod $f(x)$ form a finite field of order p^m. In Sec. 4.4, we shall discover that all finite fields are essentially of this type.

4.2 MULTIPLICATIVE STRUCTURE OF FINITE FIELDS

If a field contains an element α, then it must also contain the *powers* of α: $\alpha, \alpha^2, \alpha^3, \ldots$. Since the field must contain the reciprocal of every nonzero element, it must also contain $\alpha^{-1}, \alpha^{-2}, \alpha^{-3}, \ldots$. If the powers of α are not all distinct, then there exist two integers m and n such that $m > n$ and $\alpha^m = \alpha^n$ or $\alpha^{m-n} = 1$. The least positive integer n for which $\alpha^n = 1$ is called the *order* of α. If the order of α is n, then $1, \alpha, \alpha^2, \ldots, \alpha^{n-1}$ are all distinct, for if $\alpha^m = \alpha^k$ for some $0 \leq m < k < n$, then $\alpha^{k-m} = 1$ even though $0 < m - k < n$, contradicting the definition of order. Thus the order of an element is the number of distinct powers of that element. If an element has an infinite number of distinct powers, we say that its order is infinite. The order of 0 is undefined.

An infinite field may contain some elements of infinite order and other elements of finite order, e.g., the field of real numbers contains two elements of finite order, $+1$, which has order 1, and -1, which has order 2. All other nonzero real numbers have infinite order. In the field of complex numbers, the only elements of finite order are of the form $e^{2k\pi i/n}$, where k and n are integers and $i = \sqrt{-1}$. Thus, in certain infinite fields, only exceptional elements have finite order. In a finite field, however, each element can have only a finite number of distinct powers. Therefore, every nonzero element in a finite field must have finite order.

If α is an element of order n, then we may easily compute α^m. Using the division algorithm, we may write $m = jn + k$, where $0 \leq k < n$. We then have $\alpha^m = \alpha^{jn+k} = \alpha^{nj}\alpha^k = 1^j\alpha^k = \alpha^k$. Thus, $\alpha^m = \alpha^k$ iff $m \equiv k \bmod n$. This proves Theorem 4.21.

† The reader unfamiliar with this terminology should think of *residue classes* mod $f(x)$ as the polynomials of degree $<$ deg f, with arithmetic operations defined mod $f(x)$.

Theorem 4.21 If α has order n, then $\alpha^m = 1$ iff m is a multiple of n.

We next show that if two field elements have relatively prime orders, then the order of their product is the product of their orders.

Theorem 4.22 If α has order n and β has order m and $(m,n) = 1$, then $\alpha\beta$ has order mn.

Proof If $(\alpha\beta)^k = 1$, then $\alpha^k = \beta^{-k}$, $\alpha^{mk} = \beta^{-mk} = 1$, and $\beta^{-nk} = \alpha^{nk} = 1$

Since $\alpha^{mk} = 1$ and $(n,m) = 1$, k must be a multiple of n; since $\beta^{-nk} = 1$ and $(n,m) = 1$, k must be a multiple of m. Therefore, $(\alpha\beta)^k = 1$ only if k is a multiple of mn. On the other hand, if k is a multiple of mn, then $(\alpha\beta)^k = \alpha^k\beta^k = 1$. Q.E.D.

Let us next compute the order of α^k, given that the order of α is n. If (n,k) is the gcd of n and k, then $(\alpha^k)^{n/(n,k)} = (\alpha^n)^{k/(n,k)} = 1$, so the order of α^k must be a divisor of $n/(n,k)$. On the other hand, if $\alpha^{km} = 1$, then km must be a multiple of $n = [n/(n,k)](n,k)$. Since $n/(n,k)$ is relatively prime to k, it must divide m. This proves Theorem 4.23.

Theorem 4.23 If the order of α is n, the order of α^k is $n/(n,k)$.

We say that α is a *primitive nth root of unity* iff the order of α is n. In a field of order q, we say that α is a *primitive field element* iff the order of α is $q - 1$.

Theorem 4.24 A finite field of order q must contain a primitive field element whose order is $q - 1$ and whose powers include all nonzero field elements.

Proof Let n be the maximum order of the nonzero field elements, and let α be an element of order n. Since each of the n distinct powers of α is a nonzero field element, $n \leq q - 1$. Let $\beta \neq 0, 1$ be another field element, and let d be the order of β. If $d \nmid n$, then there is a prime p and an integer $e \geq 1$ such that $p^e | d$, $p^{e-1} | n$, and $p^e \nmid n$. By Theorem 4.22, $\alpha^{p^{e-1}}\beta^{d|p^e}$ has order np.

This contradiction shows that $d|n$. Therefore, every nonzero field element is a root of the polynomial $x^n - 1$. This polynomial has degree n, so according to Theorem 2.15, it has at most n roots in the field. Therefore $q - 1 \leq n$. Since we have already shown that $n \leq q - 1$, we conclude that $n = q - 1$. Q.E.D.

This theorem has a corollary, which is a generalization of Fermat's theorem.

Theorem 4.25 Every element in a field of order q satisfies the equation

$$x^q - x = 0$$

Proof The nonzero field elements all satisfy $x^{q-1} - 1 = 0$; the 0 field element satisfies the equation $x = 0$. All field elements satisfy the equation $x(x^{q-1} - 1) = x^q - x = 0$. Q.E.D.

4.3 CYCLOTOMIC POLYNOMIALS

Every factorization of the polynomial $x^q - x$ gives us a partition of the elements in a finite field of order q. If $x^q - x = f(x)g(x)$, then every element in the field must either be a root of $f(x)$ or a root of $g(x)$. We have already mentioned the factorization $x^q - x = x(x^{q-1} - 1)$, which separates the zero element from the nonzero elements. We now wish to separate the nonzero elements according to their orders by factoring the polynomial $x^{q-1} - 1$, which is a special case of the polynomial $x^n - 1$. If we assume that the field contains an element α of order n, then $1, \alpha, \alpha^2, \alpha^3, \ldots, \alpha^{n-1}$ are all roots of the equation $x^n - 1 = 0$. The polynomial $x^n - 1$ has degree n. According to Theorem 2.15, no polynomial of degree n can have more than n roots in any field. The powers of α must include *all* nth roots of unity. Therefore, in any field which contains a primitive nth root of unity, we have the factorization of Theorem 4.31.

Theorem 4.31

$$x^n - 1 = \prod_{i=0}^{n-1} (x - \alpha^i) = \prod_{i=1}^{n} (x - \alpha^i)$$

Similarly, if $n = kd$, then $\alpha^k, \alpha^{2k}, \alpha^{3k}, \ldots, \alpha^{dk}$ are all roots of the equation $x^d - 1 = 0$. This equation has degree d, and so it can have

no more than d roots in any field. Therefore, if α has order n, and d divides n, the powers of α include *all* dth roots of unity. Furthermore, every field element whose order is a divisor of n must be a power of α, since an element of order d is a dth root of unity.

Every power of α has order which divides n, and every field element whose order divides n is a power of α. This suggests that we partition the powers of α according to their orders:

$$x^n - 1 = \prod_{\substack{d, \\ d|n}} \prod_{\substack{\beta, \\ \beta \text{ a field} \\ \text{element of order } d}} (x - \beta)$$

The polynomial whose roots are the field elements of order d is called the *cyclotomic polynomial*, denoted by $Q^{(d)}(x)$. Theorem 4.32 is an immediate consequence of this definition.

Theorem 4.32

$$x^n - 1 = \prod_{\substack{d, \\ d|n}} Q^{(d)}(x)$$

Applying the multiplicative form of the Moebius inversion theorem 3.42 gives Theorem 4.33.

Theorem 4.33

$$Q^{(n)}(x) = \prod_{\substack{d, \\ d|n}} (x^d - 1)^{\mu(n/d)} = \prod_{\substack{d, \\ d|n}} (x^{n/d} - 1)^{\mu(d)}$$

Using the properties of the Moebius function, we can derive many properties of the cyclotomic polynomials by straightforward manipulation.

Properties of the cyclotomic polynomials 4.34

4.341.

$$Q^{(mp^k)}(x) = Q^{(pm)}(x^{p^{k-1}})$$

4.342. If p prime and $p \nmid m$, then

$$Q^{(pm)}(x) = \frac{Q^{(m)}(x^p)}{Q^{(m)}(x)}$$

4.343. If $n = \prod_i p_i^{e_i}$, p_i primes, then

$$\deg Q^{(n)}(x) = \varphi(n)$$

where $\varphi(n)$ is *Euler's phi function* given by $\varphi(n) = \prod_i p_i^{e_i-1}(p_i - 1)$

4.344. If $n \geq 2$,

$$Q^{(n)}(x) = \prod_{\substack{d,\\ d|n}} (1 - x^{n/d})^{\mu(d)}$$

4.345. If $n \geq 3$ and n odd,

$$Q^{(2n)}(x) = Q^{(n)}(-x)$$

4.346. If $n \geq 2$,

$$x^{\varphi(n)} Q^{(n)}(x^{-1}) = Q^{(n)}(x)$$

4.347

$$Q^{(n)}(1) = \begin{cases} 0 & \text{iff } n = 1 \\ p & \text{iff } n \text{ is a power of the prime } p \\ 1 & \text{iff } n \text{ has two or more prime factors} \end{cases}$$

4.348

$$Q^{(n)}(-1) = \begin{cases} 0 & \text{iff } n = 2 \\ -2 & \text{iff } n = 1 \\ 2 & \text{iff } n \text{ is a power of 2, } n \geq 4 \\ p & \text{iff } n \text{ is 2 times a power of } p \\ 1 & \text{otherwise} \end{cases}$$

Proofs

4.341

$$Q^{(mp^k)}(x) = \prod_{\substack{d,\\ d|p^k m}} (x^{mp^k/d} - 1)^{\mu(d)}$$

$$= \prod_{\substack{d,\\ d|pm}} [(x^{p^k})^{m/d} - 1]^{\mu(d)} = Q^{(pm)}(x^{p^{k-1}})$$

4.342

$$Q^{(pm)}(x) = \Big[\prod_{\substack{d,\\ d|m}} (x^{pm/d} - 1)^{\mu(d)}\Big]\Big[\prod_{\substack{d,\\ d|pm\\ d\nmid m}} (x^{pm/d} - 1)^{\mu(d)}\Big]$$

$$= \Big[\prod_{\substack{d,\\ d|m}} [(x^p)^{m/d} - 1]^{\mu(d)}\Big]\Big[\prod_{\substack{d,\\ d|m}} (x^{m/d} - 1)^{-\mu(d)}\Big]$$

$$= \frac{Q^{(m)}(x^p)}{Q^{(m)}(x)}$$

4.343. From Property 4.341,

$$\varphi(p^k m) = p^{k-1}\varphi(pm)$$

From Property 4.342,

$$\varphi(pm) = p\varphi(m) - \varphi(m) = (p - 1)\varphi(m)$$

Therefore

$$\varphi\Big(\prod_{i=1}^{j} p_i^{e_i}\Big) = p_j^{e_j-1}(p_j - 1)\varphi\Big(\prod_{i=1}^{j-1} p_i^{e_i}\Big)$$

$$= \prod_{i=j+1-k}^{j} p_i^{e_i-1}(p_i - 1)\varphi\Big(\prod_{i=1}^{j-k} p_i^{e_i}\Big)$$

$$= \prod_{i=1}^{j} p_i^{e_i-1}(p_i - 1) \qquad \text{by induction on } k$$

4.344

$$Q^{(n)}(x) = \prod_{\substack{d,\\ d|n}} (x^{n/d} - 1)^{\mu(d)} = (-1)^{\sum\limits_{\substack{d,\\ d|n}} \mu(d)} \prod_{\substack{d,\\ d|n}} (1 - x^{n/d})^{\mu(d)}$$

$$= \begin{cases} -\prod\limits_{\substack{d,\\ d|n}} (1 - x^{n/d})^{\mu(d)} & \text{if } n = 1 \\ \prod\limits_{\substack{d,\\ d|n}} (1 - x^{n/d})^{\mu(d)} & \text{if } n > 1 \end{cases}$$

4.345. According to Properties 4.342 and 4.344,

$$Q^{(2n)}(x) = \frac{Q^{(n)}(x^2)}{Q^{(n)}(x)} = \prod_{\substack{d,\\ d|n}} \left(\frac{1 - x^{2n/d}}{1 - x^{n/d}}\right)^{\mu(d)}$$

$$= \prod_{\substack{d,\\ d|n}} (1 + x^{n/d})^{\mu(d)} = Q^{(n)}(-x)$$

4.346. If α has order n, so does α^{-1}. Therefore, $Q^{(n)}(\alpha^{-1}) = 0$ iff $Q^{(n)}(\alpha) = 0$. Therefore $Q^{(n)}(x)$ and $x^{\varphi(n)}Q^{(n)}(x^{-1})$ have identical roots. Both have degree $\varphi(n)$. If $n \geq 2$, both are monic. Hence,

$$Q^{(n)}(x) = x^{\varphi(n)}Q^{(n)}(x^{-1})$$

and

$$Q^{(n)}(x) = \sum_{i=0}^{\varphi(n)} Q_i^{(n)}x^i = x^{\varphi(n)}Q^{(n)}(x^{-1}) = \sum_{i=0}^{\varphi(n)} Q_{\varphi(n)-i}^{(n)}x^i$$

Therefore, $Q_i^{(n)} = Q_{\varphi(n)-i}^{(n)}$ for all i.

4.347. Since the conditions are mutually exclusive, we need only prove the "if" clauses.

(*a*) $Q^{(1)}(1) = 1 - 1 = 0$

(*b*) $Q^{(p^k)}(1) = Q^{(p)}(1^{p^{k-1}}) = Q^{(p)}(1)$

$$Q^{(p)}(x) = \frac{x^p - 1}{x - 1} = \sum_{i=0}^{p-1} x^i$$

$$Q^{(p)}(1) = \sum_{i=0}^{p-1} 1^i = p$$

(*c*) According to Property 4.342,

$$Q^{(pm)}(1) = \frac{Q^{(m)}(1^p)}{Q^{(m)}(1)} = 1$$

4.348

(*a*) $Q^{(2)}(x) = x + 1$ $\qquad Q^{(2)}(-1) = 0$

(*b*) $Q^{(1)}(x) = x - 1$ $\qquad Q^{(1)}(-1) = -2$

(*c*) $Q^{(2^k)}(x) = x^{2^{k-1}} + 1$ $\qquad Q^{(2^k)}(-1) = 1 + 1 = 2$

(*d*) $Q^{(2p^k)}(x) = Q^{(p^k)}(-x)$ $\qquad Q^{(2p^k)}(-1) = Q^{(p^k)}(1) = p$

(*e*) If $n = p^k$, p odd, then $Q^{(p^k)}(-1) = Q^{(p)}(-1) = \sum_{i=0}^{p-1} (-1)^i = 1$. If $n = pm$, p odd, $(p,m) = 1$, then

$$Q^{(pm)}(-1) = \frac{Q^{(m)}([-1]^p)}{Q^{(m)}(-1)} = 1 \qquad \text{Q.E.D.}$$

One immediate consequence of Property 4.343 is Theorem 4.35.

Theorem 4.35 Any finite field of order q contains exactly $\varphi(q-1)$ primitive field elements.

Using Property 4.343, the degree of any cyclotomic polynomial may be determined immediately. If the order of α is n, then the order of α^k is also n iff $(n,k) = 1$. Thus, if α is an element of order n, we may write

$$Q^{(n)}(x) = \prod_{\substack{k,\\(k,n)=1\\1\le k<n}} (x - \alpha^k)$$

From this we see that Euler's phi function of n is equal to the number of integers less than n and relatively prime to n.

In practice, the above properties of cyclotomic polynomials prove quite helpful in simplifying calculations. Using Property 4.341, the calculation may be reduced to a case where n is the product of distinct primes. Then, for $n > 1$ we may write

$$Q^{(n)}(x) \equiv \prod_{\substack{d,\\d|n}} (1 - x^{n/d})^{\mu(d)} \bmod x^{\varphi(n)/2+1} \tag{4.36}$$

Working mod $x^{\varphi(n)/2+1}$ enables us to determine the first half of the coefficients of the cyclotomic polynomial with a minimum of effort. Using Property 4.346, the other half of the coefficients may then be determined by symmetry. Properties 4.347 and 4.348 provide helpful checks on the result.

Examples 4.37 Let us determine $Q^{(36)}$ and $Q^{(105)}$.

$$Q^{(36)}(x) = Q^{(6)}(x^6) \qquad \varphi(6) = 2 \qquad Q^{(6)} = 1 - x + x^2$$

$$\boxed{Q^{(36)} = 1 - x^6 + x^{12}}$$

$$Q^{(105)}(x) = ? \qquad \varphi(105) = \varphi(3 \cdot 5 \cdot 7) = 2 \cdot 4 \cdot 6 = 48$$

$$Q^{(105)} = \frac{(1-x^3)(1-x^5)(1-x^7)(1-x^{105})}{(1-x)(1-x^{15})(1-x^{21})(1-x^{35})}$$

$$\equiv (1 + x + x^2)(1 - x^5)(1 - x^7)(1 + x^{15})(1 + x^{21}) \bmod x^{25}$$

$$\equiv (1 + x + x^2)(1 - x^5 - x^7 + x^{12} + x^{15} - x^{20} + x^{21} - x^{22}) \bmod x^{25}$$

$$\boxed{\begin{aligned} Q^{(105)} = {} & 1 + x + x^2 - x^5 - x^6 - 2x^7 - x^8 - x^9 + x^{12} + x^{13} + x^{14} \\ & + x^{15} + x^{16} + x^{17} - x^{20} - x^{22} - x^{24} - x^{26} - x^{28} + x^{31} \\ & + x^{32} + x^{33} + x^{34} + x^{35} + x^{36} - x^{39} - x^{40} - 2x^{41} - x^{42} \\ & - x^{43} + x^{46} + x^{47} + x^{48} \end{aligned}}$$

4.4 ALGEBRAIC STRUCTURE OF FINITE FIELDS

We begin our discussion of the algebraic structure of finite fields by considering the additive properties of the field's unit element, 1. Since the field contains 1, it must contain $\sum_{i=1}^{2} 1 = 1 + 1$ and $\sum_{i=1}^{3} 1 = 1 + 1 + 1$ and $\sum_{i=1}^{n} 1 = 1 + 1 + \cdots + 1$. If these field integers are not all distinct, then there exist two rational integers m and n such that $m > n$ and $\sum_{i=1}^{m} 1 = \sum_{i=1}^{n} 1$ in the field, or $\sum_{i=1}^{m-n} 1 = 0$ in the field.

Definition 4.401 The least positive rational integer c for which $\sum_{i=1}^{c} 1 = 0$ in the field is called the *characteristic* of the field.

If $\sum_{i=1}^{n} 1$ is nonzero for every integer n, then the field is said to have characteristic ∞ (or, in the older terminology still used by some writers, characteristic 0).

The field integers are closed under multiplication, because†

$$\left(\sum_{i=1}^{n} 1\right)\left(\sum_{i=1}^{m} 1\right) = \left(\sum_{i=1}^{nm} 1\right)$$

If $\left(\sum_{i=1}^{mn} 1\right) = 0$ and $\left(\sum_{i=1}^{n} 1\right) \neq 0$, then we can multiply by $1/\left(\sum_{i=1}^{n} 1\right)$ and obtain $\sum_{i=1}^{m} 1 = 0$. Hence, if $\sum_{i=1}^{mn} 1 = 0$, then either $\sum_{i=1}^{m} 1 = 0$ or $\sum_{i=1}^{n} 1 = 0$. The conclusion is Theorem 4.402.

Theorem 4.402 The characteristic of any field must be ∞ or a prime number p.

If the field has characteristic ∞, then the order of the field must also be ∞. If the field has characteristic p, however, its order may be

† In expressions such as this, the multiplication operation is implicit: $\left(\sum_{i=1}^{n} 1\right)\left(\sum_{i=1}^{m} 1\right)$ means $\left(\sum_{i=1}^{n} 1\right) * \left(\sum_{i=1}^{m} 1\right)$.

finite or infinite. For example, the integers mod p form a finite field of characteristic p. An example of an infinite field of characteristic p is the set of all rational functions in x, that is, $f(x)/g(x)$, where $f(x)$ and $g(x)$ are polynomials in x over the field of integers mod p.

In any field of characteristic p, the field integers are closed under subtraction and division as well as multiplication and addition. For, given any positive integer $k < p$, we may use Euclid's algorithm for integers to find integers j and r such that $jk + pr = 1$, where $0 < j < p$. In the field, we must then have $\left(\sum_{i=1}^{j} 1\right)\left(\sum_{i=1}^{k} 1\right) = 1$, whence the field integer $\sum_{i=1}^{j} 1$ is the reciprocal of the field integer $\sum_{i=1}^{k} 1$. This gives Theorem 4.403.

Theorem 4.403 In a field of characteristic p, the field integers form a subfield of order p isomorphic to the field of integers mod p.

We next show that every field inherits a considerable amount of structure from its prime subfield. The first result in this direction is Theorem 4.404.

Theorem 4.404 In any field of characteristic p, $x^p - a^p = (x - a)^p$.

Proof$_1$ Let $f(x) = x^p - a^p$, and suppose $f(x) = g(x)h(x)$, where $g(x)$ and $h(x)$ are relatively prime and $0 < \deg g(x) \leq \deg h(x) < p$, so that $g'(x)$ and $h'(x)$ are both nonzero. Then $f'(x) = 0 = g(x)h'(x) + g'(x)h(x)$. Since $h(x)$ divides $g(x)h'(x)$ and $(g(x),h(x)) = 1$, $h(x)$ divides $h'(x)$. But this is impossible, because $\deg h'(x) < \deg h(x)$. Therefore no two factors of $f(x)$ are relatively prime, so $f(x)$ must be the power of an irreducible polynomial. Since $f(a) = 0$, the irreducible polynomial $x - a$ is a factor of $f(x)$, and $f(x)$ is therefore a power of $x - a$. Since $\deg f(x) = p$, $f(x) = (x - a)^p$. Q.E.D.

Proof$_2$ Since $(x - a)^p$ is a polynomial of degree p in x, we may write $(x - a)^p = \sum_k \binom{p}{k} [-a]^k x^{p-k}$, where $\binom{p}{k}$ is an integer called a *binomial coefficient*. It is obvious that $\binom{p}{0} = \binom{p}{p} = 1$. It can

be shown by induction on p and k that

$$\binom{p}{k} = \frac{p(p-1)\cdots(p-k+1)}{1\cdot 2\cdots k}$$

If $0 < k < p$, the numerator contains a factor of p but the denominator does not, so $\binom{p}{k} \equiv 0 \bmod p$. This proves Theorem 4.404. Q.E.D.

One consequence of this theorem is that in a field of characteristic p, no element can have order which is a multiple of p. For, if $x^{kp} = 1$, then $x^{kp} - 1 = (x^k - 1)^p = 0$, so $x^k - 1 = 0$. Another consequence of this theorem is Theorem 4.405.

Theorem 4.405 If $w_1, w_2, \ldots, w_k$ are elements in a field of characteristic p, then

$$\Big(\sum_{i=1}^{k} w_i\Big)^{p^n} = \Big(\sum_{i=1}^{k} w_i{}^{p^n}\Big) \quad \text{for all } n$$

Proof We first prove the theorem by induction on n in the special case when $k = 2$. Our induction hypothesis is $(w_1 + w_2)^{p^n} = w_1{}^{p^n} + w_2{}^{p^n}$. This assertion is trivially true when $n = 0$. In general, we compute

$$\begin{aligned}
(w_1 + w_2)^{p^{n+1}} &= [(w_1 + w_2)^{p^n}]^p \\
&= (w_1{}^{p^n} + w_2{}^{p^n})^p && \text{by the induction hypothesis} \\
&= (x - a)^p && \text{if } x = w_1{}^{p^n} \text{ and } a = -w_2{}^{p^n} \\
&= x^p - a^p && \text{by Theorem 4.404} \\
&= (w_1{}^{p^n})^p + (w_2{}^{p^n})^p && \text{by present definition of } x \text{ and } a \\
&= w_1{}^{p^{n+1}} + w_2{}^{p^{n+1}}
\end{aligned}$$

Thus, if $k = 2$, the theorem is true for all n. Setting $w_1 = y$, $w_2 = z$, we have $(y + z)^{p^n} = y^{p^n} + z^{p^n}$. For any n, we now proceed to show that

$$\Big(\sum_{i=1}^{k} w_i\Big)^{p^n} = \sum_{i=1}^{k} w_i{}^{p^n} \qquad \text{by induction on } k$$

We compute

$$\begin{aligned}
\Big(\sum_{i=1}^{k+1} w_i\Big)^{p^n} &= \Big(\sum_{i=1}^{k} w_i + w_{k+1}\Big)^{p^n} \\
&= (y+z)^{p^n} \qquad \text{if } y = \sum_{i=1}^{k} w_i \text{ and } z = w_{k+1} \\
&= y^{p^n} + z^{p^n} \qquad \text{by previous arguments} \\
&= \Big(\sum_{i=1}^{k} w_i\Big)^{p^n} + w_{k+1}^{p^n} \qquad \text{by present definition of } y \text{ and } z \\
&= \Big(\sum_{i=1}^{k} {w_i}^{p^n}\Big) + w_{k+1}^{p^n} \qquad \text{by induction hypothesis} \\
&= \sum_{i=1}^{k+1} {w_i}^{p^n} \qquad \text{Q.E.D.}
\end{aligned}$$

One corollary to this theorem is a special case of Fermat's theorem.

Corollary 4.406 If k is an integer in a field of characteristic p, it satisfies the equation $k^{p^n} = k$ for all n.

Proof

$$k^{p^n} = \Big(\sum_{i=1}^{k} 1\Big)^{p^n} = \sum_{i=1}^{k} 1^{p^n} = \sum_{i=1}^{k} 1 = k \qquad \text{Q.E.D.}$$

On the other hand, the polynomial $x^p - x$ can have at most p roots in any field. Therefore, Theorem 4.407 is true.

Theorem 4.407 An element in a field of characteristic p is an integer iff it satisfies the equation $x^p - x = 0$.

Thus, if w is not an integer, $w^p \neq w$. However, w^p is still closely related to w, as indicated by Theorem 4.408.

Theorem 4.408 If $f(x)$ is a polynomial over the integers mod p, and if w is an element in a field of characteristic p which satisfies the equation $f(w) = 0$, then $f(w^{p^n}) = 0$ for all n.

Proof If $\sum_i f_i w^i = 0$, then

$$0 = \Big(\sum_i f_i w^i\Big)^{p^n} = \sum_i (f_i w^i)^{p^n} = \sum_i {f_i}^{p^n} w^{ip^n} = \sum_i {f_i}^{p^n} (w^{p^n})^i = \sum_i f_i (w^{p^n})^i$$

because each f_i is an integer. Q.E.D.

Since the elements $w, w^p, w^{p^2}, w^{p^3}, \ldots$ are a subset of the powers of w, the number of elements of this form depends only on the order of w. The precise relationship is given in Theorem 4.409.

Theorem 4.409 If w is an element of order n in a finite field of characteristic p, then $w^{p^m} = w$, where m is the multiplicative order of the integer p modulo the integer n. The m elements $w, w^p, w^{p^2}, \ldots, w^{p^{m-1}}$ are all distinct.

Proof $w^{p^k} = w^{p^i}$ iff $w^{p^k - p^i} = 1$, which happens iff $p^k - p^i$ is a multiple of n, which happens iff $p^k \equiv p^i \bmod n$, which happens iff $p^{k-i} \equiv 1 \bmod n$, which happens iff $k - i$ is a multiple of m, the multiplicative order of $p \bmod n$. Q.E.D.

Thus, if w is a root of the equation $f(x) = 0$, where $f(x)$ is a polynomial over the field integers, then $w, w^p, w^{p^2}, \ldots, w^{p^{m-1}}$ are also roots of this same equation. Are there necessarily any others? No, for the following theorem shows us that there exists a polynomial of degree m over the field integers whose only roots are $w, w^p, w^{p^2}, \ldots, w^{p^{m-1}}$.

Theorem 4.410 Let w be an element of order n in a finite field of characteristic p, and let m be the multiplicative order of $p \bmod n$. Then the coefficients of the mth degree polynomial $f(x) = \prod_{i=0}^{m-1} (x - w^{p^i})$ are field integers. This polynomial is irreducible over the integers mod p.

Proof

$$
\begin{aligned}
[f(x)]^p &= \prod_{i=0}^{m-1} (x - w^{p^i})^p \\
&= \prod_{i=0}^{m-1} (x^p - w^{p^{i+1}}) \\
&= \prod_{i=1}^{m} (x^p - w^{p^i}) \\
&= \prod_{i=0}^{m-1} (x^p - w^{p^i})
\end{aligned}
$$

because $w^{p^m} = w = w^{p^0}$. In other words, we have the identity $[f(x)]^p = f(x^p)$. Writing $f(x)$ as

$$f(x) = \sum_{i=0}^{m} f_i x^i$$

we have

$$[f(x)]^p = \Big(\sum_{i=0}^{m} f_i x^i\Big)^p = \sum_{i=0}^{m} f_i{}^p x^{pi}$$

and

$$f(x^p) = \sum_{i=0}^{m} f_i x^{pi}$$

These two polynomials must be identical. Therefore, for every i, we have $f_i{}^p = f_i$, so f_i is a field integer.

If $f(x) = g(x)h(x)$, where $g(x)$ and $h(x)$ are also monic polynomials over the integers mod p, then since $f(w) = 0$, either $g(w) = 0$, or $h(w) = 0$. If $g(w) = 0$ and $g(x)$ is a polynomial over the field integers, then $g(w^p) = 0$, $g(w^{p^2}) = 0$, . . . , $g(w^{p^{m-1}}) = 0$, so $g(x)$ has degree m, and $g(x) = f(x)$. Similarly, if $h(w) = 0$, then $h(x) = f(x)$. Q.E.D.

Since all polynomials over the field integers which have w as a root must also have w^p, w^{p^2}, . . . , $w^{p^{m-1}}$ as roots, every polynomial over the integers which has w as a root must be a multiple of the polynomial $f(x) = \prod_{i=0}^{m-1} (x - w^{p^i})$. This polynomial is called the *minimal polynomial of* w, and its degree, m, is called the *degree of* w. Since the elements w^p, w^{p^2}, . . . , $w^{p^{m-1}}$ must have the same minimal polynomial as w, these elements are called the *conjugates of* w. This is the same reason that the familiar complex numbers $+i$ and $-i$ are called conjugates; they are both roots of the same irreducible real polynomial, $x^2 + 1$.

Since the minimal polynomial of w is an irreducible polynomial of degree m over the field integers, the p^m polynomials in w of degree less than m over the field integers are distinct. They form a field. Although we have used this fact several times in previous chapters, this theorem is so important that we take this opportunity to review its proof.

Theorem 4.411 If w is a field element of degree m, the polynomials over the field integers of degree $<m$ in w form a subfield of order p^m.

Proof We first note that the p^m polynomials of degree $<m$ in w over the field integers are all distinct, for if two of them were equal, w would be a root of their difference. This difference would be a polynomial of degree $<m$, contradicting the assumption that w has degree m.

The sum and difference of any two polynomials of degree $<m$ in w is also a polynomial of degree $<m$ in w. The product of two polynomials of degree $<m$ in w is another polynomial in w, which may be reduced modulo the minimal polynomial of w to obtain a polynomial of degree $<m$ in w. The multiplicative inverse of any polynomial of degree $<m$ in w may be found by applying Euclid's algorithm to this polynomial and the minimal polynomial of w. Since the minimal polynomial of w is irreducible, their gcd must be 1, and the equation $a(x)g(x) + b(x)f(x) = 1$ becomes $a(x)g(x) \equiv 1 \bmod f(x)$. If $f(x)$ is the minimal polynomial of w, we have $a(w)g(w) = 1$, so $a(w)$ is the multiplicative inverse of $g(w)$. Thus, the polynomials in w of degree $<m$ over the field integers form a field of order p^m. Every element in this field may be expressed in the form $u = \sum_{i=0}^{m-1} a_i w^i$, where $a_0, a_1, \ldots, a_{m-1}$ are integers mod p. Q.E.D.

We can now show that every finite field must be of this type.

Theorem 4.412 The order of a finite field is a power of its characteristic.

Proof Suppose the field has order q and characteristic p. According to Theorem 4.24, the field contains a primitive element α of order $q - 1$. The number of conjugates of α is the multiplicative order of $p \bmod q - 1$, which we denote by m. Since $q - 1$ divides $p^m - 1$, $q - 1 \leq p^m - 1$, and $q \leq p^m$. Since the degree of α is m, the p^m polynomials of degree $<m$ in α are all distinct, and $p^m \leq q$. Therefore $q = p^m$. Q.E.D.

If every element x in a field of order q satisfies $x^{q^{n-1}} = x$, then we also have $(x^{q^{n-1}})^q = x^q = x$, or $x^{q^n} = x$. Hence, by induction on n, we have another generalization of Fermat's theorem.

Theorem 4.413 Every element in a field of order q satisfies the equation $x^{q^n} - x = 0$ for every n.

Given an irreducible polynomial $f(x)$ of degree d over a field of order q, the set of residue classes mod $f(x)$ forms a field of order q^d. If α denotes the residue class containing x, then $f(\alpha) = 0$. Thus, $f(x)$ has a root in the field of order q^d even though $f(x)$ is irreducible over the field of order q. Since α is an element in the field of order q^d, it satisfies Theorem 4.413. For every n, $\alpha^{q^{dn}} - \alpha = 0$. The element α also satisfies the equation $f(\alpha) = 0$. Thus, α is a common root of the polynomials $x^{q^d} - x$ and $f(x)$. Using the division algorithm, we may write $x^{q^{dn}} - x = g(x)f(x) + r(x)$, where the degree of the remainder, $r(x)$, is less than $\deg f(x) = d$. In particular, if we set $x = \alpha$ we have $\alpha^{q^{dn}} - \alpha = g(\alpha)f(\alpha) + r(\alpha)$, or $0 = 0 + r(\alpha)$. But all polynomials of degree $<n$ in α represent different elements in the field of order q^{dn}, so $r(\alpha) = 0$ implies that $r(x) = 0$. This proves Theorem 4.414.

Theorem 4.414 Every irreducible polynomial of degree d over a field of order q divides $x^{q^k} - x$ if k is any multiple of d.

For example, every irreducible binary polynomial of degree 1 or 3 must divide $x^{2^3} + x$. The product of all irreducible binary polynomials of degrees 1 and 3 is $x(x + 1)(x^3 + x + 1)(x^3 + x^2 + 1)$. This product must also divide $x^8 + x$. Since the degree of this product is 8, we must have $x(x + 1)(x^3 + x + 1)(x^3 + x^2 + 1) = x^{2^3} + x$, which may be verified by direct calculation.

In general, in a field of order q, the product of all distinct monic irreducible polynomials whose degrees divide k divides $x^{q^k} - x$. The degree of $x^{q^k} - x$ is q^k; the degree of the product of all distinct monic irreducible polynomials whose degrees divide k is the sum of the degrees of all distinct monic irreducible polynomials whose degrees divide k. For each m that divides k, there are I_m distinct monic irreducible polynomials of degree m. Therefore, the degree of the product of all distinct monic irreducible polynomials whose degrees divide k is $\sum_{\substack{m, \\ m|k}} mI_m$. According to Theorem 3.35, this equals q^k. Therefore, we have Theorem 4.415.

Theorem 4.415 In a field of order q, $x^{q^k} - x$ factors into the product of all monic irreducible polynomials whose degrees divide k.

In particular, over the field of integers mod p, $x^{p^k} - x$ factors into the product of all monic irreducible polynomials whose degrees divide k. In a field of order p^k, $x^{p^k} - x$ factors into p^k linear factors.

Since the integers mod p must be a subfield of the field of order p^k, we may equate these two factorizations in the field of order p^k obtaining Theorem 4.416.

Theorem 4.416 If $f(x)$ is an irreducible polynomial of degree m over the integers mod p and m divides k, then a field of order p^k must contain m roots of $f(x)$.

From this result we may deduce that the finite field of order p^k is unique. To prove this, let $f(x)$ be any particular irreducible polynomial of degree k over the integers mod p. Then any finite field of order p^k must contain k roots of $f(x)$. If we call one of these roots α, then we may express every element in the field as a polynomial in α of degree $<k$. This gives us Theorem 4.417.

Theorem 4.417 If p is any prime and k is any integer, there exists a *unique* finite field of order p^k. This field is called the *Galois field* of order p^k and is denoted by $\mathrm{GF}(p^k)$.

From an engineering standpoint, it is misleading to overstress the uniqueness of $\mathrm{GF}(p^k)$, for this field may have many different representations, as we shall see in an example in Sec. 4.5. The design and cost of circuitry to perform calculations in $\mathrm{GF}(p^k)$ depend critically on the representation. For this reason, some engineers prefer to think of different representations of $\mathrm{GF}(p^k)$ as different fields. This viewpoint is particularly justified in situations where the cost of transforming from one representation to another is large.

The monic irreducible polynomials whose degrees divide k are a subset of the monic irreducible polynomials whose degrees divide j iff j is a multiple of k. Therefore, if q is any prime power, we have Theorem 4.418.

Theorem 4.418 $\mathrm{GF}(q^k)$ is a subfield of $\mathrm{GF}(q^j)$ iff j is a multiple of k.

If $f(x)$ is an irreducible polynomial of degree m over $\mathrm{GF}(q)$, then every root of $f(x)$ lies in $\mathrm{GF}(q^m)$. If w is one such root, then the other roots of $f(x)$ are w^q, w^{q^2}, . . . , $w^{q^{m-1}}$. These elements are called the *q-conjugates* of w. In general, if q is a power of the prime p, the q-conjugates are a subset of the conventional p-conjugates.

4.5 EXAMPLES

As a straightforward illustration of these theorems, we shall investigate the factorization of the binary polynomial $x^4 - x$.

$$\begin{aligned} x^4 - x &= x(x^3 - 1) \\ &= xQ^{(1)}(x)Q^{(3)}(x) \\ &= x(x - 1)(x^2 + x + 1) \end{aligned}$$

This cyclotomic factorization is valid over any field. Over the binary field, + and − are interchangeable, and we have

$$x^4 - x = x(x + 1)(x^2 + x + 1)$$

The factor $x^2 + x + 1$ is the irreducible binary quadratic. In order to factor this irreducible polynomial, we must extend the field to include its roots. If we let ξ denote one of its roots, then

$$\xi^2 + \xi + 1 = 0 \qquad \text{or} \qquad \xi^2 = \xi + 1$$

The other root of this quadratic is the binary conjugate of ξ, namely, ξ^2, which we denote by ∂. In GF(4), we then have the complete factorization,

$$x^4 - x = x(x + 1)(x + \xi)(x + \partial)$$

Here

$$\partial = \xi + 1 = \xi^2$$

and

$$\xi = \partial + 1 = \partial^2$$

The four elements of the field may be represented as the four binary polynomials of degree <2 in ξ (or ∂); the three nonzero field elements may be represented as the successive powers of ξ (or ∂).

As a more revealing example, we investigate the factorization of the binary polynomial $x^{2^4} - x$, and several resulting representations of GF(16). We begin with the cyclotomic factorization

$$\begin{aligned} x^{16} - x &= x(x^{15} - 1) \\ &= xQ^{(1)}(x)\quad Q^{(3)}(x)\qquad\quad Q^{(5)}(x)\qquad\qquad Q^{(15)}(x) \\ &= x(x - 1)\ (x^2 + x + 1)\ (x^4 + x^3 + x^2 + x + 1)\ (x^8 - x^7 + x^5 - x^4 + x^3 - x + 1) \end{aligned}$$

This factorization is valid over any field. The reader is invited to verify that

$$Q^{(15)}(x) = x^8 - x^7 + x^5 - x^4 + x^3 - x + 1$$

In the binary field, $Q^{(15)}$ must factor into a pair of primitive irreducible quartics.† One method of finding these factors is the process of elimination. They must both be of the form $x^4 + Ax^3 + Bx^2 + Cx + 1$, where A, B, and C are binary numbers. Since 1 is not a root of this polynomial, we must have $A + B + C = 1$, so that either all three or only one of the three binary numbers A, B, C are 1. The choice of all three is ruled out, since that gives $Q^{(5)}(x)$. Finally,

$$x^4 + x^2 + 1 = (x^2 + x + 1)^2$$

which leaves us with only the two correct factors

$$Q^{(15)}(x) = (x^4 + x^3 + 1)(x^4 + x + 1)$$

An easier method of finding the two irreducible factors of $Q^{(15)}(x)$ is based on the fact that if $u \in \mathrm{GF}(q)$ and $x \in \mathrm{GF}(q^m)$, then the transformation $x \to x + u$ maps conjugates into conjugates and irreducible polynomials into irreducible polynomials. Thus, in the present case, since $Q^{(5)}(x)$ is irreducible, so is $Q^{(5)}(x + 1) = (x + 1)^4 + (x + 1)^3 + (x + 1)^2 + (x + 1) + 1 = (x^4 + 1) + (x^3 + x^2 + x + 1) + (x^2 + 1) + (x + 1) + 1 = x^4 + x^3 + 1$. Since this irreducible quartic does not divide $Q^{(5)}(x)$, it must divide $Q^{(15)}(x)$. The other factor of $Q^{(15)}(x)$ is then seen to be the *reciprocal* of this polynomial, namely, $x^4(x^{-4} + x^{-3} + 1) = x^4 + x + 1$.

By either method, we obtain the complete factorization over GF(2),

$$x^{16} - x = x(x + 1)(x^2 + x + 1)(x^4 + x^3 + x^2 + x + 1)(x^4 + x^3 + 1)(x^4 + x + 1)$$

By extending the binary field to include ξ and ∂, the roots of $x^2 + x + 1$, we obtain the larger field GF(4). Over GF(4), we may refine the factorization to obtain

$$x^{16} - x = x(x + 1)(x + \xi)(x + \partial)(x^2 + \xi x + 1)(x^2 + \partial x + 1)(x^2 + \xi x + \xi)(x^2 + \partial x + \partial)(x^2 + x + \xi)(x^2 + x + \partial)$$

The reader should notice how the ξ's and ∂'s appear in conjugate pairs, much as the i's and $-i$'s appear in conjugate pairs when one factors real polynomials into complex factors, and for the same reason. The imaginary numbers i and $-i$ are complex conjugates because they are roots of the same real irreducible polynomial, $x^2 + 1$; ξ and ∂ are conjugates because they are roots of the same irreducible binary polynomial, $x^2 + x + 1$.

† Following Dickson (1901) and Albert (1956), all coding theorists now define a *primitive polynomial* of degree m as an irreducible factor of $Q^{(q^m-1)}$ over GF(q). Some writers in other specialties still use the classical terminology, according to which a polynomial is primitive iff its coefficients are relatively prime rational integers.

Evidently, $x^{4^2} - x$ factors into the four irreducible linear factors and the six irreducible quadratic factors over GF(4), as the reader should verify by the process of elimination.

In order to further refine the factorization, we must extend the field even further, to include the other 12 elements in GF(16) not included in GF(4). We denote these additional 12 elements by various Greek letters having no particular significance

$$
\begin{array}{lllll}
x^{16} - x = & xQ^{(1)}(x) & Q^{(3)}(x) & Q^{(5)}(x) & Q^{(15)}(x) \\
= & x(x+1) & (x^2+x+1) & (x^4+x^3+x^2+x+1) & (x^8+x^7+x^5+x^4+x^3+x+1) \\
= & x(x+1) & (x^2+x+1) & (x^4+x^3+x^2+x+1) & (x^4+x^3+1)(x^4+x+1) \\
= & x(x+1) & (x+\xi) & (x^2+\xi x+1) & (x^2+\xi x+\xi)(x^2+\partial x+\partial) \\
 & & \times (x+\partial) & \times (x^2+\partial x+1) & \times (x^2+x+\xi)(x^2+x+\partial) \\
= & x(x+1) & (x+\xi) & (x+\delta)(x+\mu) & (x+\beta)(x+\vartheta)(x+\lambda)(x+\zeta) \\
 & & \times (x+\partial) & \times (x+\gamma)(x+\theta) & \times (x+\alpha)(x+\pi)(x+\varphi)(x+\rho)
\end{array}
$$

Since α is one of the eight roots of $Q^{(15)}(x)$, it is a primitive element, and we may express each of the 15 nonzero field elements as a power of α, as we have done in columns 1 and 2 of Table 4.1.

Since α is a root of the irreducible GF(4) quadratic $x^2 + x + \xi$, we may express every element in GF(16) as a polynomial over GF(4) of degree <2 in α. This is done in column 3.

The successive rows of column 3 can be obtained from the feedback shift register of Fig. 4.1 if one is careful to do all the GF(4) arithmetic correctly. One must use the facts that $\xi^2 = \xi + 1 = \partial$ and $\partial^2 = \partial + 1 = \xi$.

From column 3, we can identify the remaining elements of GF(16). α^4 is clearly π, the conjugate of α with respect to GF(4). α^5 is ξ, and α^{10} is ∂. α^2 and α^8 are conjugates of α with respect to GF(2) but not with respect to GF(4). Hence they are roots of the same irreducible binary quartic but of a different irreducible GF(4) quadratic. These elements are φ and ρ (in an order which we must arbitrarily specify).

Since α and π are roots of the quadratic, $x^2 + x + \xi = 0$, they are also roots of $\partial x^{-2}(x^2 + x + \xi) = x^{-2} + \partial x^{-1} + \partial = 0$, whence α^{-1} and $\pi^{-1} = \alpha^{-4}$ are roots of the quadratic $x^2 + \partial x + \partial = 0$. These elements are λ and ζ. Similarly, α^{-2} and α^{-8} are the roots of $x^2 + \xi x + \xi$, which are β and ϑ. Finally, we must determine whether the GF(4) conjugates α^3 and α^{12} are roots of $x^2 + \xi x + 1$ and the conjugates α^6 and α^9 are roots of $x^2 + \partial x + 1$, or vice versa. Computing $[(\alpha^3)^2 + 1]/\alpha^3$ gives ∂,

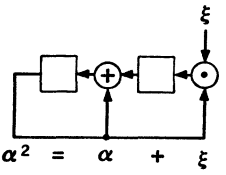


Fig. 4.1 Feedback shift register over GF(4).

Table 4.1

(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)	(15)	(16)	(17)
Representation as a power of α	*Representation as a negative power of* α	*Representation as a polynomial over* GF(4) *of* deg <2 *in* α	*Minimal* GF(4) *polynomials*	*Elements of* GF(16)	*Minimal* GF(2) *polynomials*	*Representation as a polynomial over* GF(2) *of* deg <4 *in* α	*Binary* $\log_\alpha$	*Representation as a power of* β	*Representation as a polynomial over* GF(2) *of* deg <4 *in* β	*Representation as a binary linear combination of* β^8, β^4, β^2, β	*Attempted representation as a power of* γ	*Representation in cosets of powers of* γ	*Representation in cosets of powers of* γ	*Representation as a polynomial over* GF(2) *of* deg <4 *in* γ	*Binary trace*	GF(4) *trace*
α^0	α^{-15}	0 0	x	0	x	0000		β^0	0000	0000	γ^0	0	0	0000	0	0
α^1	α^{-14}	1 0	$x+1$	1	$x+1$	0001	0000	β^{13}	0001	1111		γ^0	γ^0	0001	0	0
α^2	α^{-13}	$1\ \xi$	$x+\xi$	α	x^4	0010	0001	β^{11}	0110	0011		$\alpha\gamma^0$	$\pi\gamma^4$	1010	0	1
α^3	α^{-12}	$\partial\ \xi$	$x+\partial x+1$	ϕ	x^4	0100	0010	β^9	0101	0110		$\phi\gamma^0$	$\lambda\gamma^2$	0110	0	1
α^4	α^{-11}	$1\ 1$	$x+\xi$	π	$x^4+x^3+x^2+x+1$	0011	0100	β^7	0111	1100	γ	γ	γ	0010	1	∂
α^5	α^{-10}	$0\ \xi$	$x+\xi$	ξ	x^2+x+1	0110	0101	β^5	1011	0101		$\alpha\gamma$	π	1011	0	1
α^6	α^{-9}	$\xi\ 0$	$x^2+\xi x$	δ	$x^4+x^3+x^2+x+1$	1100	0110	β^3	1000	1011	γ^2	$\phi\gamma$	$\lambda\gamma^3$	1100	0	0
α^7	α^{-8}	$\xi\ \partial$	$x^2+\xi x+\xi$	β	x^4+x^3+x+1	1011	0111	β^1	0010	0001		$\alpha\gamma^2$	$\pi\gamma$	1001	1	ξ
α^8	α^{-7}	$1\ \partial$	$x^2+x+\partial$	ρ	x^4	0101	1000	β^{14}	1100	1001		$\phi\gamma^2$	$\lambda\gamma^4$	0111	0	1
α^9	α^{-6}	$\xi\ \xi$	$x^2+\xi x+1$	μ	$x^4+x^3+x^2+x+1$	1010	1001	β^{12}	0011	1110	γ^3	γ^3	γ^3	1000	1	ξ
α^{10}	α^{-5}	$0\ \partial$	$x+\partial$	∂	x^2+x+1	0111	1010	β^{10}	1010	1010		$\alpha\gamma^3$	$\pi\gamma^2$	1101	0	0
α^{11}	α^{-4}	$\partial\ 0$	$x^2+\partial x+\partial$	λ	$x^4+x^3+x^2+1$	1110	1011	β^8	1110	1000		$\phi\gamma^3$	λ	1110	1	∂
α^{12}	α^{-3}	$\partial\ 1$	$x^2+\partial x+1$	θ	$x^4+x^3+x^2+x+1$	1111	1100	β^6	1111	0111	γ^4	γ^4	γ^4	1111	1	∂
α^{13}	α^{-2}	$\xi\ 1$	$x^2+\xi x+\xi$	ϑ	$x^4+x^3+x^2+x+1$	1101	1101	β^4	1001	0100		$\alpha\gamma^4$	$\pi\gamma^3$	0101	1	ξ
α^{14}	α^{-1}	$\partial\ \partial$	$x^2+\partial x+\partial$	ζ	x^4+x^3+x+1	1001	1110	β^2	0100	0010		$\phi\gamma^4$	$\lambda\gamma$	0011	1	∂

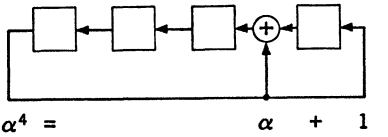


Fig. 4.2 Feedback shift register wired to multiply by α.

from which we conclude that the latter possibility is correct. Therefore, α^3 and α^{12} are γ and θ; α^6 and α^9 are δ and μ. Thus we have 16 elements, as listed in column 5. They are the roots of the corresponding irreducible GF(4) polynomials listed in column 4. Notice that we could exchange the names of any element and its GF(4) conjugate in column 5 without altering column 4.

In column 6, we have listed the minimal binary polynomials of each of the 16 elements. These entries are immediately obtained from the factorizations of the irreducible binary quartics into irreducible GF(4) quadratics. Note that all binary conjugates, i.e., any element, its square, its fourth power, and its eighth power, are roots of the same irreducible binary polynomial.

Since α is a root of the irreducible binary quartic, $x^4 + x + 1$, we may ignore the subfield GF(4) and express every field element as a binary polynomial in α of degree <4. This is done in column 7. The successive powers of α are readily found as the successive states of the feedback shift register wired to multiply by α, as shown in Fig. 4.2.

For most purposes, the representations given in columns 7 and 1 are the most convenient. In digital circuitry, the representation of column 7 is used almost exclusively. For hand computations, however, it is helpful to keep both columns 1 and 7 available. Multiplications, divisions, powers, and roots are most conveniently accomplished in terms of the representation of column 7.

Column 8 lists the logs of the nonzero elements (base α) in binary notation. This notation has several advantageous properties. For example, the binary logarithm of the multiplicative inverse of a field element is obtained by taking the bit-by-bit complement of the binary logarithm of the element. The only peculiarity is the occurrence of a sequence of all ones as the complement of the all-zero sequence, which represents the binary logarithm of 1. The all-one sequence must also be understood to represent the logarithm of 1. The all-one sequence and the all-zero sequence are equivalent representations of the same number, just as $+0$ and -0 are really the same number. Essentially, the arithmetic of binary logarithms is the so-called *ones-complement* arithmetic, which is commonly used in certain types of computer indexing and arithmetic units.

Another, more important advantage of the binary logarithm nota-

tion is that the binary logarithm of each of the binary conjugates of an element may be obtained as cyclic shifts of the binary logarithm of the element. For example, the conjugates α^{11}, $\alpha^{22} = \alpha^{7}$, α^{14}, $\alpha^{28} = \alpha^{13}$ have the binary logarithms 1011, 0111, 1110, 1101, each of which is a cyclic left shift of its predecessor. Similarly, GF(4) conjugates have binary logarithms which are double cyclic shifts of each other.

In fields of characteristic p, p-ary logarithms have the same advantageous properties as binary logarithms have in fields of characteristic 2.

The choice of the primitive element α, in terms of which we expressed the other elements of the field, was somewhat ad hoc. In column 9, we adopt another choice, namely, β, a root of the primitive irreducible binary quartic $x^4 + x^3 + 1$. In column 10, we list the field elements as binary polynomials of degree <4 in β. In column 11, we represent the field elements in yet another way, as binary linear combinations of the elements, β^8, β^4, β^2, and β. Similarly we might represent all the field elements in terms of binary linear combinations of any four independent field elements, i.e., four elements for which all 16 binary combinations are distinct or, equivalently, field elements for which no nonzero binary combination is zero. Some sets of four field elements, such as α, α^2, α^4, and α^8, which satisfy the linear equation $\alpha + \alpha^2 + \alpha^4 + \alpha^8 = 0$, are not independent. A set of four linearly dependent field elements cannot be used as a basis for representing the field.

The four field elements used as a basis for the representation of column 11, namely, $\lambda = \beta^8$, $\vartheta = \beta^4$, $\zeta = \beta^2$, and β, include a complete set of binary conjugates. In terms of this basis, one may therefore compute squares and square roots of field elements merely by permuting the digits of the representation. Squaring is accomplished by a cyclic shift to the left; square rooting, by a cyclic shift to the right. With respect to these operations, column 11 is like column 8. There, however, the similarity ends. The representation of column 11, like the representations of columns 7, 10, and 15, is linear; addition of elements may be performed on a component-by-component basis. Column 8 does not share this property; in fact, addition of two elements, given only their representations in column 8, constitutes a formidable problem. In columns 7, 10, 11, and 15, addition is trivial, but multiplication is harder. Squaring is easiest in terms of the representation of column 11, but multiplication of two arbitrary registers is more difficult. For these reasons, a representation as a binary polynomial in some root of some irreducible binary polynomial is most frequently used in practice.

In column 12, we attempt to represent the field elements as powers of a nonprimitive element γ, which is a root of $Q^{(5)}$, not $Q^{(15)}$. The representation fails, because $\gamma^5 = 1$, and there are only five distinct powers of γ. In columns 13 and 14, however, we succeed in representing

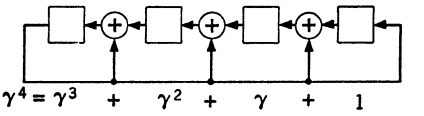


Fig. 4.3 Feedback shift register wired to multiply by γ.

the field in terms of the powers of γ and their cosets. The different choices of the leading coset element in columns 13 and 14 illustrate the fact that two cosets are either disjoint or identical.

In column 15, we represent the field elements as binary polynomials of degree <4 in γ. Since γ is a root of an irreducible binary quartic, this representation is successful, even though this polynomial is not primitive.

It is interesting to consider the feedback shift register of Fig. 4.3, wired to multiply a field element in this representation by γ. Given any starting state, the register will proceed through each of the other four states in the same coset as the starting state and then return to its original state.

In columns 16 and 17, we list the traces of the elements of GF(16) with respect to the subfields GF(2) and GF(4). The trace, often abbreviated Tr, in $GF(q^m)$ with respect to $GF(q)$ is defined by

$$\mathrm{Tr}\ (x) = \sum_{i=0}^{m-1} x^{q^i}$$

Since $[\mathrm{Tr}\ (x)]^q = \mathrm{Tr}\ (x^q) = \mathrm{Tr}\ (x)$, $\mathrm{Tr}\ (x) \in GF(q)$. As we shall see in Theorem 6.69, the trace gives us information about the solutions of certain quadratic and cubic equations in the field.

★ 4.6 ALGEBRAIC CLOSURE

Since $k!$ divides $j!$ whenever $k < j$, $GF(p) \subset GF(p^{2!}) \subset GF(p^{3!}) \subset GF(p^{4!}) \subset \cdots$. I define $GF(p^{\infty!})$ by the rule $\xi \in GF(p^{\infty!})$ iff $\xi \in GF(p^{n!})$ for all sufficiently large n. Every element of $GF(p^{\infty!})$ has finite order, even though the order of $GF(p^{\infty!})$ is ∞.

Unlike the finite fields of characteristic p, $GF(p^{\infty!})$ is *algebraically closed*, as indicated by Theorem 4.61.

Theorem 4.61 Every polynomial of degree d over $GF(p^{\infty!})$ has d roots in $GF(p^{\infty!})$.

Proof If $f(x)$ is a polynomial over $GF(p^{\infty!})$, then there exists a k such that all coefficients of $f(x)$ lie in $GF(p^k)$. If $g(x)$ is an irreduc-

ible factor of $f(x)$ over GF(p^k) and deg $g(x) = i$, then all roots of $f(x)$ lie in GF(p^{ki}), which is a subfield of GF($p^{n!}$) for all sufficiently large n. Hence all roots of $f(x)$ lie in GF($p^{\infty!}$). Q.E.D.

★ 4.7 DETERMINING MINIMAL POLYNOMIALS

If w is a root in GF(q^m) of an irreducible polynomial of degree m over GF(q), then every element in GF(q^m) may be expressed as a polynomial over GF(q) of degree $<m$ in w. Each element in GF(q^m) is a root of some minimal polynomial over GF(q), and the degree of this minimal polynomial is a divisor of m. Often one wishes to determine this minimal polynomial.

For example, suppose w is a root in GF(2^6) of the cyclotomic polynomial $Q^{(9)}(x) = x^6 + x^3 + 1$. Since the multiplicative order of 2 mod 9 is 6, w has 6 distinct conjugates in GF(2^6), and $Q^{(9)}(x)$ is irreducible over GF(2). Suppose $u = w^3 + w + 1$. What is the minimal polynomial of u?

The most straightforward approach is to express u, u^2, u^3, . . . , u^6 as polynomials in w and to determine which linear combination of these is 0. In the present example, we have

$$
\begin{aligned}
1 &= 1 \\
u &= 1 + w + w^3 \\
u^2 &= w^2 + w^3 \\
u^3 &= 1 + w^2 + w^3 + w^4 + w^5 \\
u^4 &= 1 + w^3 + w^4 \\
u^5 &= w^3 + w^4 + w^5 \\
u^6 &= w + w^2 + w^3 + w^4 + w^5
\end{aligned}
$$

Solving the equations

$$
[M_0,M_1,M_2,M_3,M_4,M_5,M_6]
\begin{bmatrix}
1 & 0 & 0 & 0 & 0 & 0 \\
1 & 1 & 0 & 1 & 0 & 0 \\
0 & 0 & 1 & 1 & 0 & 0 \\
1 & 0 & 1 & 1 & 1 & 1 \\
1 & 0 & 0 & 1 & 1 & 0 \\
0 & 0 & 0 & 1 & 1 & 1 \\
0 & 1 & 1 & 1 & 1 & 1
\end{bmatrix}
= [0,0,0,0,0,0]
$$

we obtain $\mathbf{M} = [1,1,1,0,0,1,1]$, so the minimal polynomial of u is $1 + x + x^2 + x^5 + x^6$.

This method has the advantage of great generality. It is also easily programmed on a computer, and it is quite feasible even for large fields. In certain special cases, however, hand computations may be

considerably shortened by the use of appropriate shortcuts which we now present.

If we know that the minimal polynomial of w is $M(x)$, then for $k \in \mathrm{GF}(q)$ the minimal polynomial of $w-k$, which we call $G(x)$, must have the property that $G(x) = 0$ iff $M(x+k) = 0$. Evidently, $G(x) = M(x+k)$. If k is an integer, we may compute $G(x)$ directly from the binomial theorem

$$M(x+k) = \sum_n M_n(x+k)^n = \sum_n M_n \sum_i \binom{n}{i} x^{n-i}k^i$$

In a field of characteristic p, these calculations are further simplified if we know the congruences of the binomial coefficients mod p. These congruences were first discovered by Lucas in 1878.

Lucas' theorem 4.71 If p is prime and

$$N = \sum_{i=0}^{I} N_i p^i \qquad 0 \le N_i < p$$

$$K = \sum_{i=0}^{I} K_i p^i \qquad 0 \le K_i < p$$

then

$$\binom{N}{K} \equiv \prod_{i=0}^{I} \binom{N_i}{K_i} \bmod p$$

Proof According to Theorem 4.404,

$$(1+x)^p = \sum_{k=0}^{p} \binom{p}{k} x^k \equiv 1 + x^p \bmod p$$

Continuing,

$$(1+x)^{np+A} \equiv (1+x^p)^n(1+x)^A \bmod p$$

$$\sum_{m=0}^{np+A} \binom{np+A}{m} x^m \equiv \left[\sum_{i=0}^{n} \binom{n}{i} x^{pi}\right]\left[\sum_{j=0}^{A} \binom{A}{j} x^j\right] \bmod p$$

We now equate the coefficients of x^{kp+B}. This gives

$$\binom{np+A}{kp+B} \equiv \begin{cases} 0 & \text{if } 0 \le A < B < p \\ \binom{n}{k}\binom{A}{B} & \text{if } 0 \le B \le A < p \end{cases}$$

Recalling that $\binom{A}{B} = 0$ if $A < B$, we may assert that

$$\binom{np + A}{kp + B} \equiv \binom{n}{k}\binom{A}{B} \bmod p$$

for any $0 \leq A < p$, $0 \leq B < p$.

Using this fact, the proof of the theorem may be readily completed by a straightforward induction on I. Q.E.D.

Remark The theorem is false if p is not prime. For example, base 10, $\binom{14}{12} \equiv 1$, but $\binom{1}{1}\binom{4}{2} \equiv 6$; base 4, $\binom{10}{2} \equiv 2$, but $\binom{1}{0}\binom{0}{2} \equiv 0$.

Corollary 4.72 $\binom{N}{K} \equiv 0 \bmod p$ unless $K_i \leqq N_i$ for all i.

Example 4.73 Suppose w is an element in GF(2^{11}) and that the minimal polynomial of w over GF(2) is $M(x) = x^{11} + x^8 + x^5 + x^2 + 1$. What is the minimal polynomial of $w + 1$?

Answer We work in binary, obtaining

	1011	1010	1001	1000	111	110	101	100	11	10	1	0
$(x+1)^{1011} =$	1	1	1	1	0	0	0	0	1	1	1	1
$(x+1)^{1000} =$				1								1
$(x+1)^{101} =$							1	1			1	1
$(x+1)^{10} =$										1		1
$1 =$												1
	1	1	1	0	0	0	1	1	1	0	0	1

$$G(x) = x^{11} + x^{10} + x^9 + x^5 + x^4 + x^3 + 1$$

Given $M^{(1)}(x)$, the minimal polynomial of α, one often wishes to determine $M^{(k)}(x)$, the minimal polynomial of α^k. For some values of k, $M^{(k)}(x)$ may be determined with relative ease. In particular, if the order of α is n, and if k is congruent to a power of q mod n, then $M^{(k)}(x) = M^{(1)}(x)$. Another easy case arises if $-k$ is congruent to a power of the characteristic mod n, since then $M^{(k)}(x) = M^{(-1)}(x)$. If $M^{(1)}(x) = \sum_i M_i^{(1)} x^i$, then $M^{(-1)}(x) = \sum_i M_{m-i}^{(1)} x^i$. In general, $M^{(-1)}(x)$ may be found by reversing the coefficients of $M^{(1)}(x)$.

Methods also exist for determining $M^{(k)}(x)$ from $M^{(1)}(x)$ for other values of k. These methods have been described by Daykin (1965), Golomb (1967), and others. As an example of these techniques, we shall show how $M^{(3)}(x)$ may be determined from $M^{(1)}(x)$ in a field of characteristic 2. Since the roots of $M^{(3)}(x)$ are the cubes of the roots of $M^{(1)}(x)$, $M^{(1)}(x)$ must divide $M^{(3)}(x^3)$. Similarly, if ∂ is a primitive cube root of unity [$\partial \in \mathrm{GF}(4)$], then $M^{(1)}(\partial x)$ and $M^{(1)}(\partial^2 x)$ must also divide $M^{(3)}(x^3)$. On the other hand, every root of $M^{(3)}(x^3)$ is either a root of $M^{(1)}(x)$ or $M^{(1)}(\partial x)$ or $M^{(1)}(\partial^2 x)$. Therefore, we have Theorem 4.74.

Theorem 4.74 $M^{(1)}(x)M^{(1)}(\partial x)M^{(1)}(\partial^2 x)$ is a power of $M^{(3)}(x^3)$.

Knowing $M^{(1)}(x)$, we can compute $M^{(1)}(x)M^{(1)}(\partial x)M^{(1)}(\partial^2 x)$. Since $\partial^3 = 1$, the polynomial $M^{(1)}(\partial x)$ may be expressed as a quadratic polynomial in ∂, with coefficients that are polynomials in x.

$$M^{(1)}(\partial x) = A + B\partial + C\partial^2$$

where $A = A(x^3)$, $B = xB(x^3)$, $C = x^2C(x^3)$.

$$\begin{aligned} M^{(1)}(x)M^{(1)}(\partial x)M^{(1)}(\partial^2 x) &= (A + B + C)(A + B\partial + C\partial^2)(A + B\partial^2 + C\partial) \\ &= ABC + A^3 + B^3 + C^3 \end{aligned}$$

For example, suppose we wish to determine $M^{(3)}(x)$ when $M^{(1)}(x) = x^6 + x^4 + x^2 + x + 1$. We have $A = 1 + x^6$; $B = x + x^4$; $C = x^2$. Following Golomb (1967), we do the computations in Table 4.2, where we list only the exponents. We start by listing in column A the exponents of $M^{(1)}(x)$ which are $\equiv 0 \bmod 3$; in column B, we list the exponents $\equiv 1 \bmod 3$; in column C, we list the exponents $\equiv 2 \bmod 3$. We then compute the cross terms in the expression A^3. In general, $A = \sum_i A_i x^i$ and

Table 4.2 Cubic transformation of $x^6 + x^4 + x^2 + x + 1$

		A	B	C
Given		0, 6	1, 4	2
Cubic cross terms	A^3		4	2
3	B^3	3		2
$\frac{ABC}{3}$		3	1, 4	2
Totals mod 2		0, 6	4	

$A^3 = \left(\sum_i A_i x^i\right)^3 = \left(\sum_i A_i x^i\right)\left(\sum_j A_j x^j\right)^2 = \left(\sum_i A_i x^i\right)\left(\sum_j A_j x^{2j}\right) = \sum_i A_i x^{3i}$ $+ \sum_{i \neq j}\sum A_i A_j x^{i+2j}$. The cross terms are all of the form x^{i+2j}, for $A_i = A_j = 1$; $i \neq j$. In the present example, the cross terms are $0 + 2 \cdot 6 = 12$ and $6 + 2 \cdot 0 = 6$. Dividing these cubic cross terms by 3, we obtain 4 and 2, which are listed in the table. Similarly, we compute the cubic cross terms for B^3, which are $1 + 2 \cdot 4 = 9$ and $4 + 2 \cdot 1 = 6$. Dividing these by 3, we obtain 3 and 2, which are listed in the table. Finally, we compute the terms of the product ABC. In the present example, there are four such terms: $0 + 1 + 2 = 3$, $0 + 4 + 2 = 6$, $6 + 1 + 2 = 9$, and $6 + 4 + 2 = 12$. Dividing these by 3 gives 1, 2, 3, and 4, which are listed in the table. We then sum all entries in the table mod 2, obtaining 0, 4, 6. We conclude that $M^{(1)}(x)M^{(1)}(\partial x)M^{(1)}(\partial^2 x) = 1 + (x^3)^4 + (x^3)^6$, so $1 + x^4 + x^6$ is a power of $M^{(3)}(x)$. Since $M^{(3)}(x)$ is irreducible, $M^{(3)}(x) = 1 + x^2 + x^3$.

In practice, $M^{(1)}(x)M^{(1)}(\partial x)M^{(1)}(\partial^2 x)$ is usually equal to the first power of $M^{(3)}(x^3)$. This is always the case when the degree of $M^{(1)}(x)$ is odd, and it is usually the case when the degree of $M^{(1)}(x)$ is even. However, it may happen that $M^{(1)}(x)M^{(1)}(\partial x)M^{(1)}(\partial^2 x)$ is the square of $M^{(3)}(x^3)$, as we have just seen.

As a final example of this technique, we compute the minimal polynomials of all elements in GF(2^5), starting from $M^{(1)}(x) = 1 + x^2 + x^5$. In Table 4.3, we compute $M^{(3)}(x)$. We then repeat the transformation to compute $M^{(9)}(x)$. Knowing $M^{(1)}(x)$, $M^{(3)}(x)$, and $M^{(9)}(x)$, it is a trivial matter to compute $M^{(-1)}(x)$, $M^{(-3)}(x)$, and $M^{(-9)}(x)$. This gives us all six irreducible binary quintics, as listed in Table 4.4. Similar

Table 4.3 Cubic transformation of $1 + x^2 + x^5$ and $1 + x^2 + x^3 + x^4 + x^5$

		A	B	C
$M^{(1)}(x)$		0		2, 5
$\frac{\text{Cubic cross terms}}{3}$	C^3	3	4	
$\frac{ABC}{3}$				
$M^{(3)}(x)$		0, 3	4	2, 5
$\frac{\text{Cubic cross terms}}{3}$	A^3		1	2
	C^3	3	4	
$\frac{ABC}{3}$		3, 3	4	2
$M^{(9)}(x)$		0	1, 4	5, 2

Table 4.4 Minimal polynomials of elements in GF(2^5)

Cyclic shift of binary expansion of exponent									
00001	$M^{(1)}(x)$	$= M^{(2)}(x)$	$= M^{(4)}(x)$	$= M^{(8)}(x)$	$= M^{(16)}(x)$				$= 1 + x^2 + x^5$
00011	$M^{(3)}(x)$	$= M^{(6)}(x)$	$= M^{(12)}(x)$	$= M^{(24)}(x)$	$= M^{(17)}(x)$				$= 1 + x^2 + x^3 + x^4 + x^5$
00101	$M^{(5)}(x)$	$= M^{(10)}(x)$	$= M^{(20)}(x)$	$= M^{(9)}(x)$	$= M^{(18)}(x)$				$= 1 + x + x^2 + x^4 + x^5$
00111	$M^{(7)}(x)$	$= M^{(14)}(x)$	$= M^{(28)}(x)$	$= M^{(25)}(x)$	$= M^{(19)}(x)$	$= M^{(-3)}(x)$			$= 1 + x + x^2 + x^3 + x^5$
01011	$M^{(11)}(x)$	$= M^{(22)}(x)$	$= M^{(13)}(x)$	$= M^{(26)}(x)$	$= M^{(21)}(x)$	$= M^{(-5)}(x)$			$= 1 + x + x^3 + x^4 + x^5$
01111	$M^{(15)}(x)$	$= M^{(30)}(x)$	$= M^{(29)}(x)$	$= M^{(27)}(x)$	$= M^{(23)}(x)$	$= M^{(-1)}(x)$			$= 1 + x^3 + x^5$
00000					$M^{(0)}(x)$				$= 1 + x$

methods may be used to find $M^{(5)}(x)$ directly from $M^{(1)}(x)$, starting from the fact that $M^{(5)}(x^5)$ is a root of

$$M^{(1)}(x)M^{(1)}(\xi x)M^{(1)}(\xi^2 x)M^{(1)}(\xi^3 x)M^{(1)}(\xi^4 x)$$

where ξ is a primitive fifth root of unity. Analogous methods may be used to find $M^{(7)}(x)$, ... Although these techniques prove quite helpful for hand calculations with polynomials of relatively small degree, it usually proves easiest to do the calculations with polynomials of large degrees by computer, using the general matrix methods described at the beginning of this section.

PROBLEMS

4.1. [Lehmer (1936)] Prove the following facts about $|Q_i^{(n)}|$, the magnitudes of the coefficients of the cyclotomic polynomials

$$Q^{(n)}(x) = \sum_{i=0}^{\varphi(n)} Q_i^{(n)} x^i$$

(*a*) If n is the product of two distinct primes, then for all i, $|Q_i^{(n)}| = 0$ or 1.
(*b*) If $n < 105$, then for all i, $|Q_i^{(n)}| = 0$ or 1.
(*c*) Given any number A, there exist n and i such that $|Q_i^{(n)}| \geq A$. *Hint:* Let $n = \prod_{j=1}^{t} p_j$, where $t \geq A + 1$ and p_j are distinct primes such that $p_t/2 < p_1 < p_2 < \cdots < p_t$. Show that $Q_{p_t}^{(n)} = 1 - t$.
(*d*) Given any number A, there exist primes p, q, r such that $|Q_i^{(pqr)}| \geq A$ for some i. *Hint:* For given $p > 2A$, Dirichlet's theorem guarantees the existence of q and r such that $q \equiv 2 \bmod p$, $2r \equiv -1 \bmod pq$. Show that $Q^{(pqr)}_{(p-3)(qr+1)/2} = (p-1)/2$.

4.2. Let α, a primitive element in GF(2^{2m}), be a root of $M^{(1)}(x)$. In each of the following cases, determine the minimal polynomial in α over GF(2^2) and the minimal polynomial in α over GF(2^m).
(*a*) $2m = 6$, $M^{(1)}(x) = x^6 + x + 1$
(*b*) $2m = 8$, $M^{(1)}(x) = x^8 + x^4 + x^3 + x^2 + 1$
(*c*) $2m = 10$, $M^{(1)}(x) = x^{10} + x^3 + 1$

4.3. For $m = 3, 4, 5$, as in Prob. 4.2, for each $\xi \in \mathrm{GF}(2^m)$, find the integer $n(\xi)$ such that $\alpha + \xi = \alpha^n$. Also find integers $i(\xi)$ and $j(\xi)$ such that $\alpha + \xi = \beta^i\gamma^j$ where $\beta = \alpha^{2^m+1}$ and $\gamma = \alpha^{2^m-1}$.

4.4. Show that multiplication and division of arbitrary elements in $\mathrm{GF}(2^{2m})$, expressed in the form $A\alpha + B$; $A,B, \in \mathrm{GF}(2^m)$, may be reduced to addition and subtraction of integers mod $2^m \pm 1$ with the aid of four tables, none of which contains more than 2^m entries. *Hint:* If $A \neq 0$, $A\alpha + B = A(\alpha + B/A)$.

4.5. [Berlekamp (1968c)] Let q be a prime power; m, a prime; α, a primitive element of $\mathrm{GF}(q^m)$. Let $\mathrm{Tr}(x) = \sum_{i=0}^{m-1} x^{q^i}$. For each element $\xi \in \mathrm{GF}(q)$, define

$$S_\xi = \{k | 0 \leq k < t(q^m - 1) \quad \text{and} \quad \mathrm{Tr}(\alpha^k) = \xi\}$$

Clearly $\cup S_\xi = \{0, 1, 2, \ldots, t(q^m - 1) - 1\}$. Show that for each ξ, S_ξ contains no arithmetic progression of more than t terms.

4.6. Let $R^{(n)}(x) = \sum_k \binom{n-1-k}{k} x^k$, so that $R^{(0)} = 0$, $R^{(1)} = R^{(2)} = 1$, $R^{(3)} = 1 + x$, $R^{(4)} = 1 + 2x$, $R^{(5)} = 1 + 3x + x^2$, $R^{(6)} = 1 + 4x + 3x^2$, ... Prove the following:

(*a*) $R^{(n+1)} = R^{(n)} + xR^{(n-1)}$

(*b*) $R^{(n+m)} = R^{(m+1)}R^{(n)} + xR^{(m)}R^{(n-1)}$

(*c*) If $d|n$, $R^{(d)}|R^{(n)}$

(*d*) If $f(x)$ is an irreducible polynomial, and $f|R^{(d)}$ but $f \nmid R^{(i)}$ for any $i < d$, then $f \nmid R^{(n)}$ unless $d|n$.

(*e*) If $d = \gcd(n,m)$, then $\gcd(R^{(n)},R^{(m)}) = R^{(d)}$.

(*f*) $R^{(2n+1)} = [R^{(n+1)}]^2 + x[R^{(n)}]^2$

(*g*) $R^{(2n)} = [R^{(n)}]^2 + 2xR^{(n)}R^{(n-1)}$

(*h*) Over GF(2), $R^{(2a)} = 1$, $R^{(2a+1)} = \sum_{k=0}^{a-1} x^{2i-1}$, $R^{(2a-1)} = 1 + \sum_{i=0}^{a-1} x^{2i}$, and $xR^{(2a+1)}R^{(2a-1)} = x^{2a} + x$.

(*i*) Over GF(2), if $f(x)$ is an irreducible polynomial, and $f|R^{(n)}$ but $f \nmid R^{(i)}$ for any $i < n$, then $\deg f = d$, where d is the least integer j such that $2^j \equiv \pm 1 \bmod n$.

(*j*) For what values of $n \leq 20$ is $R^{(2n+1)}(x)$ irreducible over GF(2)?

(*k*) Using Lucas' theorem, write out $R^{(101)}(x)$ as a binary polynomial of degree 50.

5
Cyclic Binary Codes

5.1 REORDERING THE COLUMNS OF THE PARITY-CHECK MATRIX OF HAMMING CODES

In Sec. 1.3, we saw how to construct single-error-correcting codes having $n = 2^m - 1$ digits, of which m are check digits and $n - m$ are information digits. Each of the n columns of the parity-check matrix must contain a different nonzero binary m-tuple, which is called the *location number* of that digit. As long as the n different digits of the code are assigned different nonzero location numbers, it does not matter how these location numbers are ordered. For example, the parity-check matrix for a single-error-correcting code of block length 15 might be given as

$$\mathcal{H}' = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

In order to correct additional errors, we must add more rows to this matrix. In order to find intelligent methods for adding those additional rows and in order to decode the resulting codes, we found it helpful to consider each of these n error location numbers as a nonzero element in GF(2^m), represented as a binary polynomial of degree $<m$ in a root of some irreducible binary polynomial of degree m. We shall now show that this viewpoint is also helpful for designing efficient encoders and decoders, even in the single-error-correcting case.

In the present example, with $m = 4$, we might choose the primitive polynomial to be $x^4 + x + 1$. The leftmost column then has location number 1; the second column, location number α; the third column, location number $\alpha + 1$; . . . ; the eleventh column, $\alpha^3 + \alpha + 1$; . . . . If we denote the error location numbers by the field elements η_i', $i = 0, 1, 2, \ldots, 14$, we have $\eta_0' = 1$, $\eta_1' = \alpha$, $\eta_2' = \alpha + 1$, $\eta_3' = \alpha^2$, $\eta_4' = \alpha^2 + 1$, $\eta_5' = \alpha^2 + \alpha$, . . . , $\eta_{14}' = \alpha^3 + \alpha^2 + \alpha + 1$. The 15-dimensional binary vector $\mathbf{R} = [R_0, R_1, \ldots, R_{14}]$ then has the syndrome $\mathcal{H}'\mathbf{R}^t$, which is the Galois field representation of the sum of the corresponding location numbers, $\mathcal{H}'\mathbf{R}^t = \sum_{i=0}^{14} R_i\eta_i'$.

As long as the n different digits of the code are assigned different nonzero location numbers in GF(2^m), the code will correct single errors, no matter how these location numbers are ordered. For example, since α is the root of a primitive irreducible binary quartic, every nonzero element in GF(16) is a power of α. Therefore, we might choose to assign the successive digits of the code location numbers corresponding to successive powers of α, $\eta_i = \alpha^i$. This assignment of location numbers has the great advantage that the syndrome of the received vector $\mathbf{R} = [R_0, R_1, R_2, \ldots, R_{14}]$ becomes a polynomial in α: $\mathcal{H}\mathbf{R}^t = \sum_{i=0}^{14} R_i\alpha^i = R(\alpha)$, where we associate the vector $\mathbf{R}$ with the polynomial $R(x) = \sum_{i=0}^{14} R_ix^i$. This polynomial provides a very economical method of evaluating the syndrome. By the division algorithm, we may write $R(x) = M(x)Q(x) + r(x)$, where $r(x)$ has degree less than the degree of $M(x)$. If $M(x)$ is the irreducible binary polynomial of degree m which is the minimal polynomial of α, then $M(\alpha) = 0$, and $R(\alpha) = r(\alpha)$. Thus, after the binary polynomial $R(x)$ is divided by the binary polynomial $M(x)$, the syndrome is given by the remainder polynomial $r(x)$, evaluated at $x = \alpha$.

Before designing circuitry to divide an arbitrary binary polynomial $R(x)$ by the constant binary polynomial $M(x)$, let us first consider an example. Let $R(x) = x^{14} + x^{12} + x^{11} + x^{10} + x^8 + x^4 + x^3 + x$ and let $M(x) = x^4 + x + 1$. In longhand, the division is

```
10011|0000010111010001 1010
     00000
     -----
      00001
      00000
      -----
       00010
       00000
       -----
        00101
        00000
        -----
         01011
         00000
         -----
          10111
          10011
          -----
           01000
           00000
           -----
            10001
            10011
            -----
             00100
             00000
             -----
              01000
              00000
              -----
               10000
               10011
               -----
                00111
                00000
                -----
                 01111
                 00000
                 -----
                  11110
                  10011
                  -----
                   11011
                   10011
                   -----
                    10000
                    10011
                    -----
                     0011
```

The quotient is represented by the first digits of every other row, 0000010100100111, which are the coefficients of the quotient as a binary polynomial of degree 15, namely, $x^{10} + x^8 + x^5 + x^2 + x + 1$. The remainder is $0011 = x + 1$.

This division may be implemented with the circuit shown in Fig. 5.01. The four flip-flops store the differences at successive steps. In

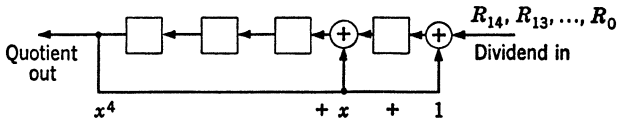


Fig. 5.01 Division circuit.

the previous example, the successive states of the flip-flops are 0000, 0001, 0010, 0101, 1011, 0100, 1000, 0010, . . . , 0011. When the division is complete, the coefficients of the remainder appear in the flip-flops.

If the digits $R_{14}, R_{13}, \ldots, R_0$ are transmitted in that order, the decoder may do the division as the digits are being received from the channel, as in Fig. 5.02.

After the entire block of n digits has been received, the polynomial $R(x)$ appears in the top register, and the remainder $r(x)$ appears in the feedback shift register. The sum of the locations of the errors S_1 is given by $S_1 = r(\alpha)$.

If there is one error in the block, then S_1 gives the location of the error. By providing sufficient gating circuitry, the error may be corrected immediately. However, this requires gating into each of the n digits of the received-word buffer. A less expensive approach is to postpone the correction until the erroneous digit leaves the buffer. If $S_1 = \alpha^j$, we must allow $n - 1 - j$ correct digits to leave the buffer before changing the erroneous digit as it passes out. This may be readily accomplished by disconnecting the input to the feedback shift register and letting it run free. Each time the buffer is shifted, the contents of the feedback shift register are multiplied by α.

In order to operate continuously, the decoder uses two feedback shift registers, one of which performs the division on the incoming block while the other times the correction of the erroneous digit in the outgoing block. The complete decoder is shown in Fig. 5.03.

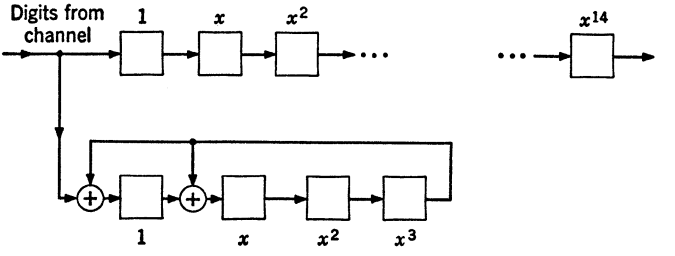


Fig. 5.02 Circuit for dividing received word by fixed polynomial.

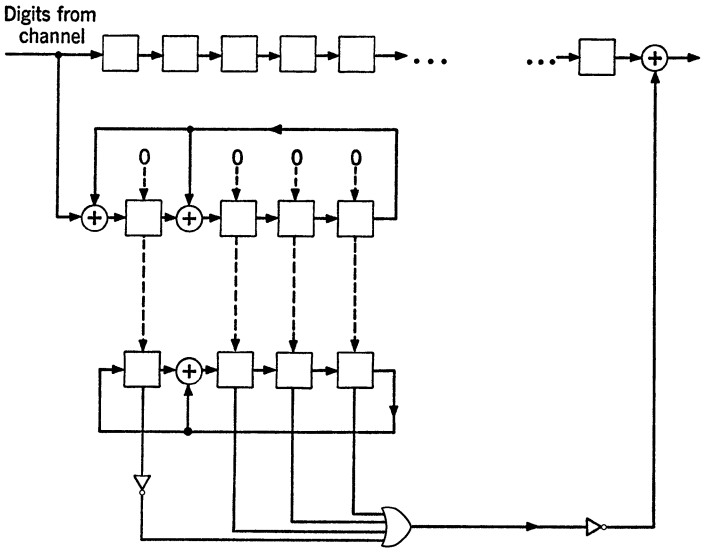


Fig. 5.03 Complete decoder for Hamming code.

At the end of each block, the field element S_1 appearing in the top feedback shift register is transferred to the lower feedback shift register and shifted, while the top feedback shift register is reset to zero, as shown by the dotted lines. The bottom register then contains $S_1\alpha$. As the digits at locations $\alpha^{-1}, \alpha^{-2}, \alpha^{-3}, \ldots, \alpha, 1$ leave the buffer, the bottom register contains $S_1\alpha, S_1\alpha^2, S_1\alpha^3, \ldots, S_1\alpha^{-1}, S_1$. As the digit at location α^{-j} leaves the buffer, the wires leading into the OR gate carry $1 + S_1\alpha^j$. If $1 + S_1\alpha^j \neq 0$, then $S_1 \neq \alpha^{-j}$. In this case the error is not located at location α^{-j}, so the digit at that location passes out of the buffer unchanged. On the other hand, if $1 + S_1\alpha^j = 0$, then $S_1 = \alpha^{-j}$ and the error is corrected as it leaves the buffer.

The encoders for Hamming codes may also be readily implemented. Given $n - m$ message digits, the encoder may transmit these as $C_{n-1}, C_{n-2}, \ldots, C_m$. The last m digits, $C_{m-1}, C_{m-2}, \ldots, C_0$, must then be chosen so that the total codeword satisfies all the parity-check equations. With the locations of the code labeled as the successive powers of α, these m parity checks are equivalent to the requirement that $C(\alpha) = 0$, which is equivalent to the requirement that $C(x)$ be a multiple of the minimal polynomial of α, which we shall now call $g(x)$. The proper choices for the check digits $C_{m-1}, C_{m-2}, \ldots, C_0$ are then seen to be the coefficients of the remainder polynomial when $\sum_{i=m}^{n-1} C_i x^i$ is divided by $g(x)$.

If the division of $\sum_{i=m}^{n-1} C_i x^i$ by $g(x)$ were implemented according to the method used in the decoder, the remainder would not be available until the feedback shift register had received all n inputs $C_{n-1}, C_{n-2}, \ldots, C_m, 0, 0, \ldots, 0$. Fortunately, there is another division circuit which we can use in order to obtain the digits of the remainder without waiting for the last m 0's to pass through the shift register. In order to understand the alternate division circuit, it is helpful to first consider the redundant three-register circuit shown in Fig. 5.04.

The top shift register buffers the incoming polynomial; the middle register divides it by $g(x)$. In this example, $g(x) = x^4 + x + 1$. The bottom register contains the sum of the top two registers, as the reader may verify by checking that if it contains the sum of the top two registers before any shift, it will also contain the sum of the top two registers after that shift. The sequences of digits fed back into both the middle and bottom registers are identical. Either of these sequences gives the digits of the quotient polynomial. After receiving the input sequence $C_{n-1}, C_{n-2}, \ldots, C_m, 0, 0, \ldots, 0$, the top buffer register contains zeros in its first m positions. The middle and bottom registers will therefore have identical contents, each equal to the remainder of $\sum_{i=m}^{n-1} C_i x^i$ divided by $g(x)$. Thus, in order to compute the remainder, we may omit the top two registers altogether. After the sequence $C_{n-1}, C_{n-2}, \ldots, C_m$ has been fed into the bottom register, this register contains the remainder. Thus the encoder may consist of this register alone, together with appropriate timing and switching circuitry, as shown in Fig. 5.05.

Initially the three switches are placed in their up positions, as

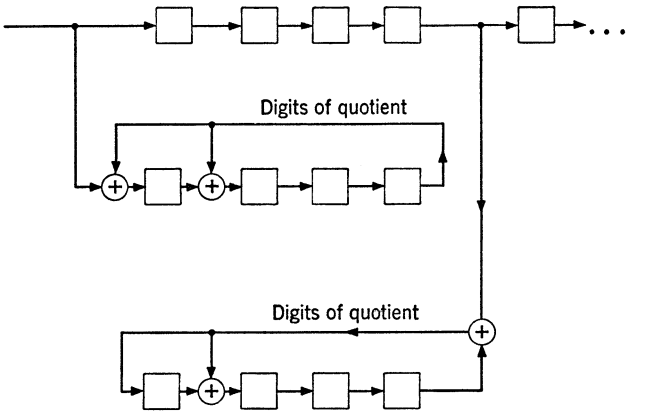


Fig. 5.04 Duplicate division circuit.

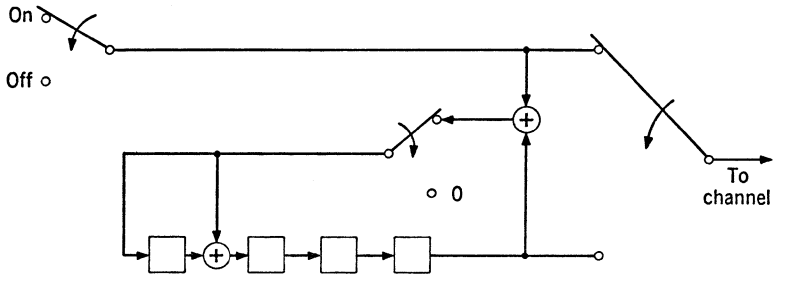


Fig. 5.05 Encoder for a Hamming code.

shown, and the shift register contains all zeros. The message source is turned on, and it emits the digits $C_{n-1}, C_{n-2}, \ldots, C_m$, which are sent down the channel and into the feedback shift register. Then the switches are all turned to their down positions. The message source is turned off, the inputs to the feedback shift register are set to zero, and the contents of the shift register are sent down the channel. After all the digits in the shift register have been sent down the channel, the switches may be returned to their up positions to begin the transmission of the next block.

5.2 REORDERING THE COLUMNS OF THE PARITY-CHECK MATRIX OF DOUBLE-ERROR-CORRECTING BINARY BCH CODES

Labeling the successive digits of the code according to the successive powers of a primitive field element also proves advantageous for multiple-error-correcting BCH codes. In the case of double-error-correcting binary codes, the second m rows of each column of the $\mathcal{H}$ matrix are taken as the cube of the location number in the first m rows of that column. The $\mathcal{H}$ matrix then has n columns and $2m$ rows.

$$\mathcal{H} = \left|\begin{array}{c|c|c|c|c|c|c|c|c|c|c} 1 & \alpha & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 & \cdots & \alpha^{-3} & \alpha^{-2} & \alpha^{-1} \\ \hline 1 & \alpha^3 & \alpha^6 & \alpha^9 & \alpha^{12} & \alpha^{15} & \alpha^{18} & \cdots & \alpha^{-9} & \alpha^{-6} & \alpha^{-3} \end{array}\right|$$

For example, if $n = 15$ and α is a root of $x^4 + x + 1$, we have

$$\mathcal{H} = \left|\begin{array}{ccccccccccccccc} 0&0&0&1&0&0&1&1&0&1&0&1&1&1&1 \\ 0&0&1&0&0&1&1&0&1&0&1&1&1&1&0 \\ 0&1&0&0&1&1&0&1&0&1&1&1&1&0&0 \\ 1&0&0&0&1&0&0&1&1&0&1&0&1&1&1 \\ \hline 0&1&1&1&1&0&1&1&1&1&0&1&1&1&1 \\ 0&0&1&0&1&0&0&1&0&1&0&0&1&0&1 \\ 0&0&0&1&1&0&0&0&1&1&0&0&0&1&1 \\ 1&0&0&0&1&1&0&0&0&1&1&0&0&0&1 \end{array}\right| \tag{5.21}$$

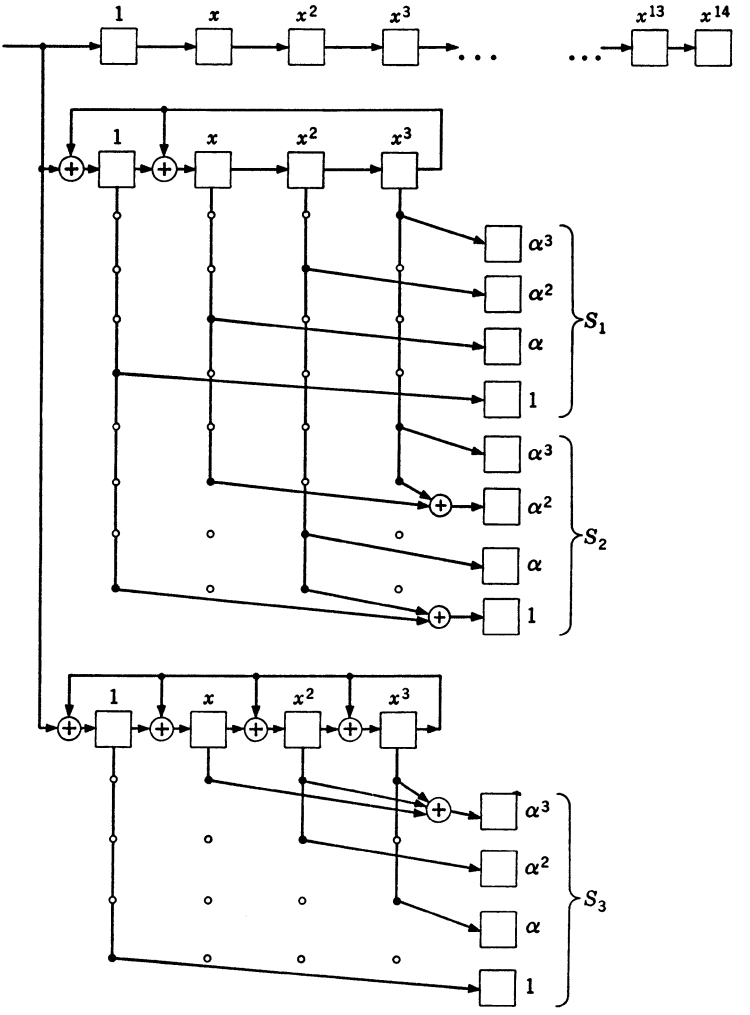


Fig. 5.06 First part of decoder for double-error-correcting BCH code.

The syndrome, $\mathbf{S}^t = \mathcal{H}\mathbf{R}^t$, consists of $2m$ digits. The first m digits of the syndrome give S_1, the sum of the error locations; the second m digits of the syndrome give S_3, the sum of the cubes of the error locations. If the received vector is represented by the polynomial $R(x) = \sum_{i=0}^{n-1} R_i x^i$, then $S_1 = R(\alpha)$, and $S_3 = R(\alpha^3)$. These two power sums may be computed from the received word separately. To compute S_1, we divide $R(x)$ by $M^{(1)}(x)$, the minimal polynomial of α, to obtain the remainder $r^{(1)}(x)$. We then have $S_1 = r^{(1)}(\alpha)$. To compute S_3, we divide $R(x)$ by

$M^{(3)}(x)$, the minimal polynomial of α^3, to obtain the remainder $r^{(3)}(x)$. We then compute $S_3 = r^{(3)}(\alpha^3)$. In general, the calculation of S_3 from $r^{(3)}$ may require as many as m parity checks, and each of these parity checks has at most m inputs.†

In the present example, the minimal polynomial of α^3 is $x^4 + x^3 + x^2 + x + 1$. The polynomials $r^{(1)}(x)$ and $r^{(3)}(x)$ may be obtained by running the received word through the horizontal feedback shift registers shown in Fig. 5.06. As the decoder receives the digits from the channel, they are fed into the 15-digit received-word buffer, shown horizontally at the top of Fig. 5.06. After all 15 digits of the block are received, this buffer contains the polynomial $R(x)$, representing the received word. As the received channel digits are fed into this register, they are simultaneously fed into the two feedback shift registers, wired according to the minimal polynomials of α and α^3. These registers are initially zero. As the digits arrive from the channel, these registers divide the polynomial $R(x)$ by $M^{(1)}$ and $M^{(3)}$, the minimal polynomials of α and α^3, respectively. When the entire block of 15 digits has arrived, these registers contain the remainders $r^{(1)}(x)$ and $r^{(3)}(x)$.

The power sums S_1, S_2, and S_3 are then obtained from the remainders $r^{(1)}(x)$ and $r^{(3)}(x)$, via the wired matrix multiplications shown, and placed in the vertical registers in the figure. The matrix used to obtain S_1 from $r^{(1)}(x)$ turns out to be the identity matrix; for this reason, we could just as well take $S_1 = r^{(1)}(\alpha)$ directly. However, the evaluations of $S_2 = r^{(1)}(\alpha^2)$ and of $S_3 = r^{(3)}(\alpha^3)$ require more circuitry. The connections of these matrix multiplications may readily be obtained from the first m columns of the expanded $\mathfrak{IC}$ matrix, which for the present example are

$$1 \quad \alpha \quad \alpha^2 \quad \alpha^3 \quad \Leftrightarrow \quad \begin{matrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{matrix}$$

$$1 \quad \alpha^2 \quad \alpha^4 \quad \alpha^6 \quad \Leftrightarrow \quad \begin{matrix} 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 \end{matrix}$$

$$1 \quad \alpha^3 \quad \alpha^6 \quad \alpha^9 \quad \Leftrightarrow \quad \begin{matrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{matrix}$$

† It is also possible to obtain S_3 by dividing $R(x^3)$ by $M^{(1)}(x)$. This alternative method eliminates the need for a register wired to divide by $M^{(3)}(x)$, but it requires many more shifts.

Since α^2 is a conjugate of α, it has the same minimal polynomial, and S_2 may therefore be obtained from the same remainder as S_1. For that reason, one generally does not include S_2 in the parity-check matrix, although there is no harm in doing so. The expanded parity-check matrix (which includes the squares of the location numbers, as well as the location numbers and their cubes) has more rows than the conventional parity-check matrix. However, the extra rows are linear combinations of the first m rows, and the rank of the expanded parity-check matrix is the same as the rank of the conventional parity-check matrix. Similarly, the parity-check matrix could be expanded even more, to include additional conjugates of S_1 and S_3, such as S_4, S_8, or S_6. S_4 and S_8 could be obtained from the remainder $r^{(1)}(x)$, and S_6 could be obtained from the remainder $r^{(3)}(x)$.

To encode, we start with the given message digits $C_{n-1}, C_{n-2}, \ldots, C_{2m}$. The encoder must then find the digits $C_{2m-1}, C_{2m-2}, \ldots, C_0$ such that the resulting polynomial $\sum_{i=0}^{n-1} C_i x^i$ satisfies both $C(\alpha) = 0$ and $C(\alpha^3) = 0$. The first condition is satisfied iff $C(x)$ is a multiple of the minimal polynomial of α; the second condition, iff $C(x)$ is a multiple of the minimal polynomial of α^3. Since $C(x)$ must be a multiple of both of these irreducible polynomials, it must be a multiple of their product, which we call $g(x)$. For the double-error-correcting BCH code of block length 15, with α taken as a root of $x^4 + x + 1$, the minimal polynomial of α^3 is $x^4 + x^3 + x^2 + x + 1$, and $g(x) = (x^4 + x + 1)(x^4 + x^3 + x^2 + x + 1) = x^8 + x^7 + x^6 + x^4 + 1$. The encoder may be built in essentially the same manner as the encoder for single-error-correcting codes. For this particular case, the encoder is as shown in Fig. 5.07.

Seven message digits are sent down the channel and into the shift register. The switches are then turned to the down positions. The eight check digits in the feedback shift register are sent down the channel,

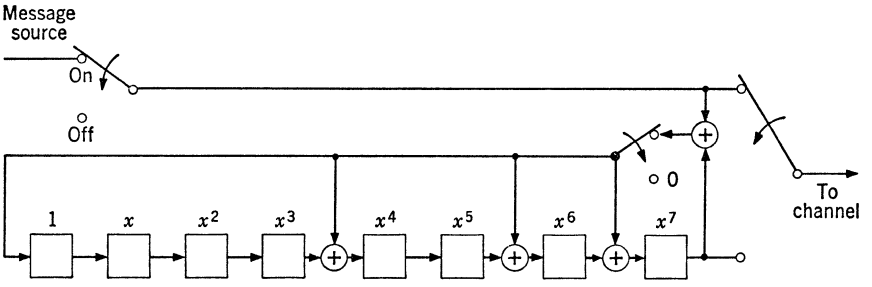


Fig. 5.07 Encoder for cyclic code using generator polynomial.

and the shift register is refilled with zeros. The switches are then returned to the up positions to begin the transmission of a new block.

There is an alternative encoder for this code which requires only a seven-stage shift register. In order to understand the design of this alternative encoder, it is helpful to consider cyclic codes.

5.3 GENERAL PROPERTIES OF CYCLIC CODES

For both single- and double-error-correcting BCH codes, we were able to find a polynomial $g(x)$ which had the property that a binary polynomial $C(x)$, of degree $<n$, represents a codeword iff $C(x)$ is a multiple of $g(x)$. This polynomial, $g(x)$, is called the *generator polynomial* of the cyclic code. Let the polynomial $C(x)$ be divided by some multiple of $g(x)$, say $g(x)h(x)$. This gives a quotient $q(x)$ and a remainder $r(x)$, so $C = qgh + r$, and C is a multiple of g iff r is a multiple of g. Thus, if the polynomial $C(x)$ represents a codeword, then any multiple of $C(x) \bmod g(x)h(x)$ represents another codeword. In particular, if g is the product of distinct irreducible polynomials of degrees dividing m, and $g \neq x$, then g divides $x^{2^m-1} - 1 = x^n - 1$. Taking $h(x) = (x^n - 1)/g(x)$, we conclude that if the polynomial $C(x)$ represents a codeword, then so does every multiple of $C(x) \bmod x^n - 1$.

If the polynomial $C(x) = C_0 + C_1x + C_2x^2 + \cdots + C_{n-1}x^{n-1}$ is multiplied by $x \bmod x^n - 1$, the result is $C_{n-1} + C_0x + \cdots + C_{n-2}x^{n-1}$. The codeword represented by the polynomial $xC(x) \bmod x^n - 1$ is seen to be a cyclic shift of the codeword represented by the polynomial $C(x)$. Since every cyclic shift of a codeword therefore gives another codeword, we say that the code is a *cyclic code.* Evidently, if $g(x)$ is any divisor of $x^n - 1$, then the set of multiples of $g(x) \bmod x^n - 1$ forms a linear cyclic code. This is true even if $n + 1$ is not a power of 2.

Conversely, if the codewords of a linear cyclic code are represented by a set of polynomials, then for every code polynomial $C(x)$, the code contains all cyclic shifts of $C(x)$. The kth cyclic right shift of the polynomial $C(x)$ is the polynomial $x^kC(x) \bmod x^n - 1$. Since the sum of codewords of a linear code is also a codeword of the same linear code, the code must contain all multiples of $C(x) \bmod x^n - 1$. Using Euclid's algorithm, we may write the gcd of $C(x)$ and $x^n - 1$ as a multiple of $C(x) \bmod x^n - 1$. Hence, a linear cyclic code contains the gcd of every codeword polynomial and $x^n - 1$. If we let $g(x)$ denote the codeword polynomial of lowest degree, we conclude that $g(x)$ must divide $x^n - 1$ and that every codeword polynomial must be a multiple of $g(x)$.

We conclude that the codeword polynomials of any linear cyclic code consist of the multiples of some generator polynomial mod $x^n - 1$, where the generator polynomial $g(x)$ is a divisor of $x^n - 1$.

It is also helpful to use polynomials to represent the rows of the $\mathcal{H}$ matrix. For example, with α a root of $x^4 + x + 1$ and

$$\mathcal{H} = \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|} \hline 1 & \alpha & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 & \alpha^7 & \alpha^8 & \alpha^9 & \alpha^{10} & \alpha^{11} & \alpha^{12} & \alpha^{13} & \alpha^{14} \\ \hline \end{array}$$

$$= \begin{array}{|*{15}{c|}} \hline 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 \\ \hline 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 \\ \hline 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ \hline 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 \\ \hline \end{array}$$

we represent the rows of $\mathcal{H}$ by the polynomials $h^{(0)}(x)$, $h^{(1)}(x)$, $h^{(2)}(x)$, and $h^{(3)}(x)$, given by

$$\begin{aligned} h^{(0)}(x) &= x^{11} + x^8 + x^7 + x^5 + x^3 + x^2 + x + 1 \\ h^{(1)}(x) &= x^{12} + x^9 + x^8 + x^6 + x^4 + x^3 + x^2 + x \\ h^{(2)}(x) &= x^{13} + x^{10} + x^9 + x^7 + x^5 + x^4 + x^3 + x^2 \\ h^{(3)}(x) &= x^{14} + x^{10} + x^7 + x^6 + x^4 + x^2 + x + 1 \end{aligned}$$

Suppose that the received word is $\mathbf{C} = [C_0, C_1, C_2, \ldots, C_{14}]$. The dot product of $\mathbf{C}$ with the first row of the $\mathcal{H}$ matrix is $C_3 + C_6 + C_7 + C_9 + C_{11} + C_{12} + C_{13} + C_{14}$. Since the polynomial $C(x)$ has the high powers of x on the right and the polynomial $h^{(0)}(x)$ has the high powers of x on the left, the dot product of $\mathbf{C}$ with the first row of the $\mathcal{H}$ matrix is the coefficient of x^{14} in the polynomial product $h^{(0)}(x)C(x)$. Similarly, the dot product of the code vector $\mathbf{C}$ and the second row of the $\mathcal{H}$ matrix is the coefficient of x^{14} in the polynomial product $h^{(1)}(x)C(x)$, etc.

The polynomial $h(x) = (x^n - 1)/g(x)$ has the property that $h(x)g(x) \equiv 0 \bmod x^n - 1$. Since every codeword polynomial is a multiple of $g(x)$, each codeword polynomial satisfies the equation $h(x)C(x) \equiv 0 \bmod x^n - 1$. Moreover, each codeword polynomial satisfies the equations $x^j h(x)C(x) \equiv 0 \bmod x^n - 1$ for all j. Since the sum of parity checks is a parity check, and since $x^j h(x)$ is a parity-check polynomial for all j, we conclude that any polynomial multiple of $h(x) \bmod x^n - 1$ constitutes another parity-check polynomial. Altogether, there are $2^{\deg g}$ multiples of $h(x)$ having degree less than n. If the code satisfied more than $2^{\deg g}$ parity checks, then one could find more than deg g independent parity-check equations, and the code would have to contain more than deg g check digits. This leads us to the following conclusion.

Theorem 5.31 The code polynomials of a linear cyclic code of block length n consist of all the multiples of a single code-generating polynomial $g(x)$. The parity-check polynomials which this code satisfies consist of all the multiples of a single parity-check polynomial, $h(x) = (x^n - 1)/g(x)$.

For example, the single-error-correcting binary cyclic code of block length $n = 15$ generated by $g(x) = x^4 + x + 1$ has $h(x) = x^{11} + x^8 + x^7 + x^5 + x^3 + x^2 + x + 1$. The parity checks given on the previous page are seen to be

$$\begin{aligned} h^{(0)}(x) &= h(x) \\ h^{(1)}(x) &= xh(x) \\ h^{(2)}(x) &= x^2h(x) \\ h^{(3)}(x) &= (x^3 + 1)h(x) \end{aligned}$$

As another example, the double-error-correcting binary cyclic code of block length $n = 15$ generated by $g(x) = (x^4 + x + 1)(x^4 + x^3 + x^2 + x + 1)$ has $h(x) = (x + 1)(x^2 + x + 1)(x^4 + x^3 + 1) = x^7 + x^6 + x^4 + 1$. The rows of the usual parity-check matrix, as given in Eq. (5.21), are seen to be

$$\begin{aligned} h^{(0)}(x) &= (x^4 + x^3 + x^2 + x + 1)h(x) \\ h^{(1)}(x) &= x(x^4 + x^3 + x^2 + x + 1)h(x) \\ h^{(2)}(x) &= x^2(x^4 + x^3 + x^2 + x + 1)h(x) \\ h^{(3)}(x) &= (x^3 + 1)(x^4 + x^3 + x^2 + x + 1)h(x) \\ h^{(4)}(x) &= (x^2 + 1)(x^4 + x + 1)h(x) \\ h^{(5)}(x) &= (x + 1)(x^4 + x + 1)h(x) \\ h^{(6)}(x) &= (x^4 + x + 1)h(x) \\ h^{(7)}(x) &= (x^3 + x^2 + x + 1)(x^4 + x + 1)h(x) \end{aligned}$$

As we saw in Sec. 5.2, the above form of the $\mathcal{H}$ matrix is particularly useful for computing S_1 and S_3, which are needed by the decoder in order to find the location numbers of the error positions. However, there is no reason why the encoder should be required to use this form of the $\mathcal{H}$ matrix rather than some other equivalent form. Any set of eight independent multiples of $h(x)$ will generate an identical code. One particularly simple set is provided by the polynomials $h^{(i)}(x) = x^ih(x)$; $i = 0, 1, \ldots, 7$. In order to satisfy these parity checks, we must have $\sum_{i=0}^{7} C_{j+i}h_{7-i} = 0$ for $j = 7, 6, 5, \ldots, 0$. Since $h_7 = 1$, this is equivalent to the requirement that

$$C_j = \sum_{i=1}^{7} C_{j+i}h_{7-i} \qquad \text{for } j = 7, 6, 5, \ldots, 0$$

These equations can be implemented by an encoder of the same form as the one shown in Fig. 5.08, where $h(x) = 1 + x^4 + x^6 + x^7$.

To begin, the deg h message digits are sent down the channel and into the shift register simultaneously. The message source is then turned off, and the remaining $n -$ deg h check digits are computed according to

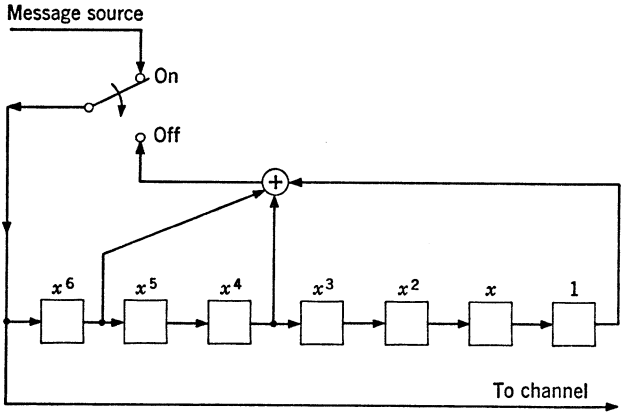


Fig. 5.08 Encoder for cyclic code using check polynomial.

the recursion rule,

$$C_j = \sum_{i=1}^{\deg h} C_{j+i} h_{(\deg h)-i}$$

As each such check digit is computed and sent down the channel, it is also sent back into the shift register so that it may be used to compute subsequent check digits, if needed. After all the $n - \deg h$ check digits have been sent, the switch is returned to its up position to begin the next block.

This method of encoding is seen to require a shift-register having $\deg h$ stages. When $\deg h < \deg g$, the encoder of Fig. 5.08 may be preferable to the encoder of Fig. 5.07.

5.4 THE CHIEN SEARCH

In Sec. 5.1 we found that even after the decoder knows the error locations, it is difficult to correct the errors immediately. It turns out to be simpler to wait until the erroneous digits leave the received-word buffer, and then to correct them as they leave.

One approach to the correction problem, due to Chien (1964), avoids the explicit solution of the algebraic equation $\sigma(z^{-1}) = 0$. As the digit at location X_i leaves the buffer, the decoder may calculate the polynomial $\sigma(X_i^{-1})$ to see whether or not this is zero. If $\sigma(X_i^{-1}) \neq 0$, then the digit at location X_i is unchanged; if $\sigma(X_i^{-1}) = 0$, then the error at location X_i is corrected. This method is not restricted to single- and double-error-correcting codes. Once the decoder has found the coefficients of the

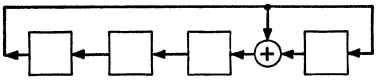


Fig. 5.09 Circuit which multiplies by α.

error locator $\sigma(z) = \sum_{i=0}^{t} \sigma_i z^i$, whose roots are the reciprocals of the error locations, then Chien's search may be used to test each of the locations $\alpha^{-1}, \alpha^{-2}, \alpha^{-3}, \ldots, 1$, to see if the digit in the position now leaving the buffer is a reciprocal root of the error locator. For double-error-correcting binary BCH codes, the error locator $\sigma(z)$ may be computed according to Eq. (1.47); for multiple-error-correcting BCH codes, the error locator may be computed by more involved procedures which we shall present in Chap. 7.

Once the decoder has found the coefficients $\sigma_1, \sigma_2, \sigma_3, \ldots, \sigma_t$, the Chien search proceeds as follows. First, the decoder computes $\sigma(\alpha) = \sum_i \sigma_i \alpha^i$. Next, the decoder computes $\sigma(\alpha^2) = \sum_i \sigma_i \alpha^{2i}$, then $\sigma(\alpha^3) = \sum_i \sigma_i \alpha^{3i}, \ldots$. In order to calculate these polynomials quickly, the decoder uses $t + 1$ registers. At the kth step, these registers contain the quantities $1, \sigma_1\alpha^k, \sigma_2\alpha^{2k}, \sigma_3\alpha^{3k}, \ldots, \sigma_t\alpha^{tk}$. The value of $\sigma(\alpha^k)$ is given by the sum of the contents of these registers. In order to proceed to the $(k + 1)$st step, the decoder multiplies the register containing $\sigma_1\alpha^k$ by α; the register containing $\sigma_2\alpha^{2k}$, by α^2; ... ; and the register containing $\sigma_t\alpha^{tk}$, by α^t. All these multiplications in GF(2^m) may be performed simultaneously. For certain relatively small values of t and m, it is feasible to build circuitry to multiply the registers containing $1, \sigma_1\alpha^k, \sigma_2\alpha^{2k}, \ldots, \sigma_t\alpha^{tk}$ by the *wired constants* $1, \alpha, \alpha^2, \ldots, \alpha^t$ in a single clock cycle. For example, in GF(2^4) with $\alpha^4 = \alpha + 1$, multiplication by α may be accomplished by the circuit of Fig. 5.09. Multiplication by α^2 may be accomplished by the circuit of Fig. 5.10. The Chien search may be accomplished by the circuit of Fig. 5.11.

Initially, the top register of the circuit in Fig. 5.11 is loaded with σ_2 and the bottom register with σ_1. At each clock cycle, the top register is multiplied by α^2 and the bottom register by α. After k clock cycles, the top register contains $\sigma_2\alpha^{2k}$, and the bottom register contains $\sigma_1\alpha^k$. The

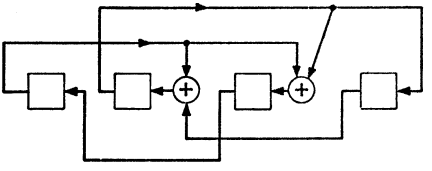


Fig. 5.10 Circuit which multiplies by α^2.

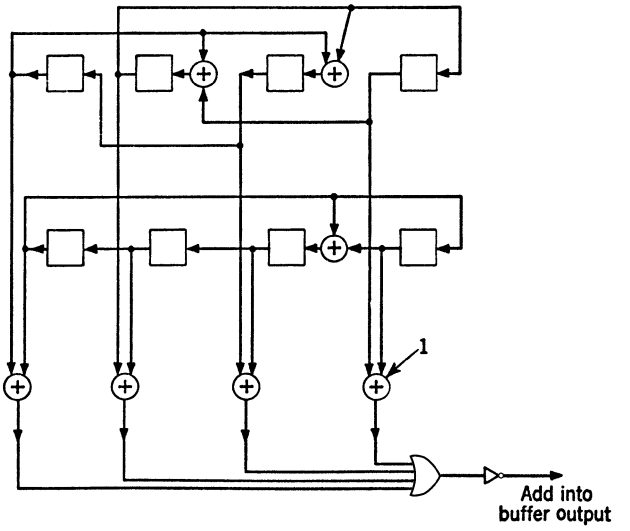


Fig. 5.11 Chien searcher for double-error-correcting code.

adders evaluate the polynomial $\sigma(\alpha^k) = 1 + \sigma_1\alpha^k + \sigma_2\alpha^{2k}$. If this polynomial is zero, then α^{-k} is a reciprocal root of the error polynomial, and a one is added into the erroneous digit at location α^{-k} which is now leaving the buffer. If $\sigma(\alpha^k) \neq 0$, then α^{-k} is not a reciprocal root of the error polynomial, and the digit at location α^{-k} leaves the buffer unchanged.

For larger values of t and m, the cost of building wired nets to multiply by α^t in GF(2^m) in one clock cycle becomes substantial. For moderate values of t, we may multiply by α^t by executing t successive multiplications by α. This method requires only registers wired to multiply by α, but it requires too many shifts if t is large. In these cases, it is usually more economical to allow m clock cycles for each of the multiplications. Each multiplication may be done by any of the methods of Sec. 2.4. It is still feasible to do all the t multiplications simultaneously. A circuit which does this is shown in Fig. 5.12. This circuit uses a single *master* control unit and t identical *slave multipliers*, one of which is shown in Fig. 5.13. Initially, the first slave accumulator is loaded with σ_1; the first slave multiplier, with α. The second slave accumulator is initially loaded with σ_2; the second slave multiplier, with α^2, ... The ith slave accumulator is initially loaded with σ_i; the ith multiplier register, with α^i. The master control unit sends identical control signals to every slave multiplier, causing each slave to place the contents of its accumulator into its multiplicand register, to multiply its multiplicand register by its multiplier register, and to accumulate the new product in its

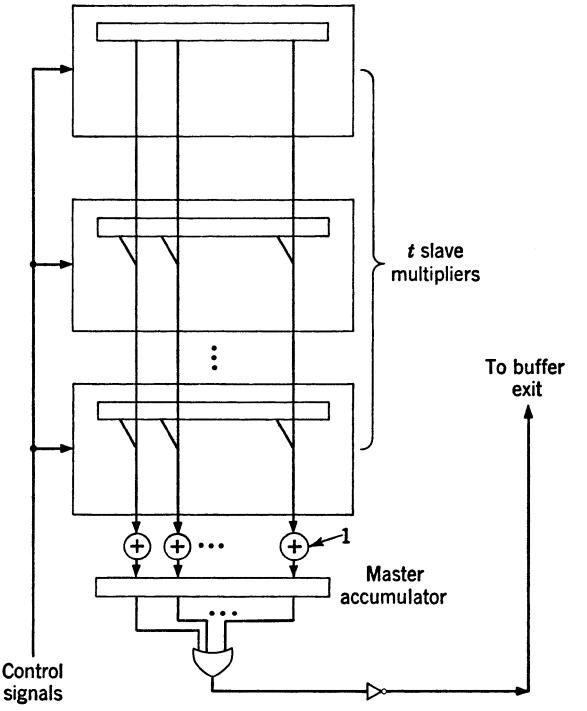


Fig. 5.12 Overall design of a Chien searcher for a t-error-correcting code.

accumulator. The sum of all the slave accumulators is then computed by the master adders and placed in the master accumulator. This is $\sigma(\alpha)$. Continuing the procedure, the Chien searcher successively computes $\sigma(\alpha)$, $\sigma(\alpha^2)$, . . . , $\sigma(\alpha^{-1})$, $\sigma(1)$. As the digit at location α^i leaves the buffer, it is corrected iff $\sigma(\alpha^{-i}) = 0$.

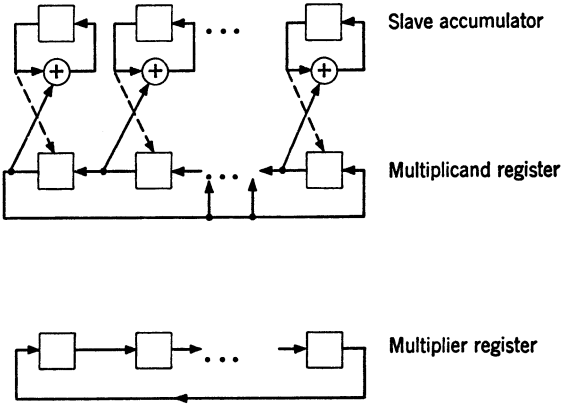


Fig. 5.13 Typical slave multiplier.

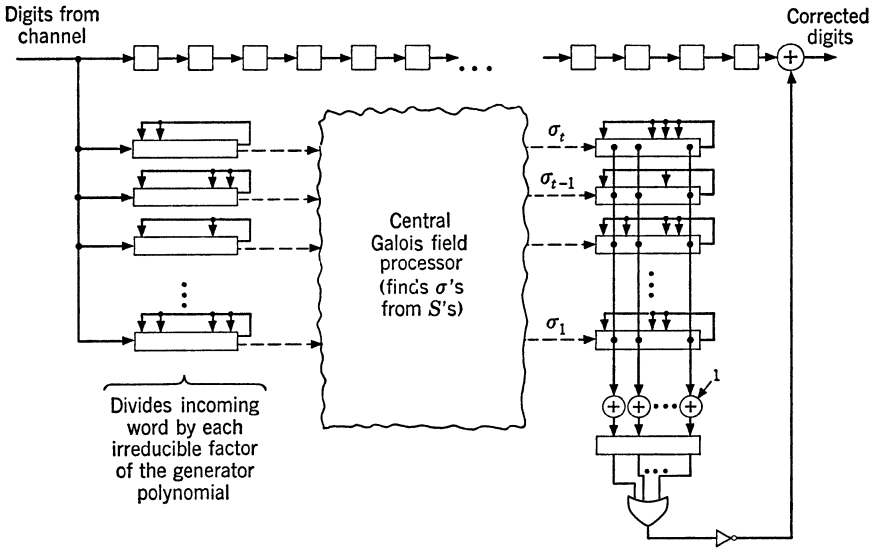


Fig. 5.14 General outline of a decoder for a cyclic code.

5.5 OUTLINE OF GENERAL DECODER FOR ANY CYCLIC BINARY CODE

A sketch of an overall design for any binary cyclic code is shown in Fig. 5.14. The decoder consists of four principal parts, a buffer of $2n$ digits, shift registers wired to divide the incoming word by each irreducible factor of the generator polynomial, a central Galois field processor, and a Chien searcher. At a typical instant of time, the buffer will hold parts of three successive blocks, as shown in Fig. 5.15. The first i digits of the buffer hold the first i digits of the incoming word; the next n digits of the buffer hold the entire buffered word; the last $n - i$ digits of the buffer hold the last $n - i$ digits of the outgoing word. The Chien searcher is in the process of computing $\sigma(\alpha^i)$ in order to determine whether or not the next digit to leave the buffer should be corrected. The shift registers wired to divide the incoming word by each irreducible factor of the generator polynomial are busily doing just that. The central processor is engaged in trying to find the error-locator polynomial for the buffered word.

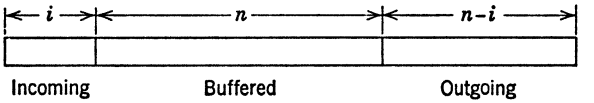


Fig. 5.15 Buffer contents at a typical time.

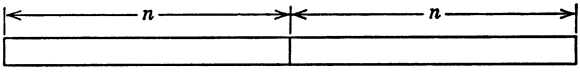


Fig. 5.16 Buffer contents at end of block.

When all the n digits of the incoming block have been received, then all the digits of the outgoing block have left. The buffer then appears as in Fig. 5.16. The buffered block then becomes the outgoing block, and the incoming block becomes the buffered block. The coefficients of the error-locator polynomial are read out of the central GF processor and into the Chien searcher, as the remainders of the received word divided by each irreducible factor of the generator polynomial are read into the central GF processor. As the next n digits of the incoming word arrive from the channel, the central GF computer must compute the coefficients of the error-locator polynomial for the buffered word.

The registers which divide the incoming polynomial by the irreducible factors of the generator polynomial and the Chien searcher must both operate in synchronization with the buffer. The central GF computer, however, is totally disconnected from the rest of the decoder except at those brief instants between blocks when it outputs the error locator it has just computed and inputs the syndrome for the next block. It is not really essential that input and output of this central processor be executed simultaneously. If the central processor is so fast that it is able to compute the error locator before the new received word arrives, then the buffer size may be reduced. For example, suppose that the central GF processor is able to compute the error locator in half the time required for n digits to be received from the channel. In that case, the buffer must be capable of storing only $3n/2$ digits. After a complete word is received, the central processor computes its error locator by the time the beginning of this word is ready to leave the buffer. The error locator is then fed into the Chien searcher, and the central processor sits idle until the rest of the incoming word is received.

In the extreme case of Hamming codes, the central processor may be eliminated altogether. The buffer may be reduced to n digits. The error-locator polynomial for single errors is given by $\sigma(z) = 1 + \sigma_1 z$, where $\sigma_1 = S_1 = r^{(1)}(\alpha)$. The entire decoder is shown in Fig. 5.03.

For double-error-correcting binary BCH codes, the central processor must do some GF computations. Given $r^{(1)}(x)$ and $r^{(3)}(x)$, the remainders of the received word divided by $M^{(1)}(x)$ and $M^{(3)}(x)$, the minimal polynomials of α and α^3, the central processor must find the error locator. First, it may compute $S_1 = r^{(1)}(\alpha)$, $S_2 = r^{(1)}(\alpha^2)$, $S_3 = r^{(3)}(\alpha^3)$. Then, it must do the arithmetic for the computation $\sigma_2 = (S_1S_2 + S_3)/S_1$ or set $\sigma_2 = 0$ if S_1 and S_3 are 0. This decoding algorithm is sufficient

to correct double errors, but it is not complete. The code has cosets of weight 3 which this algorithm fails to decode.

In general, if $M^{(k)}(x)$, the minimal polynomial of α^k, is a factor of the generator polynomial, then one piece of input data which the central processor receives is $r^{(k)}(x)$, the remainder of the received word divided by $M^{(k)}(x)$. The central processor may begin by computing $S_k = r^{(k)}(\alpha^k)$ and any desired binary conjugates of S_k, such as $S_{2k} = S_k^2 = r^{(k)}(\alpha^{2k})$ or $S_{4k} = S_k^4 = r^{(k)}(\alpha^{4k})$. These calculations are relatively straightforward. The harder problem is to compute the σ's from the S's.

This outlined decoder in Fig. 5.14 is applicable to any binary cyclic code. As examples, we present two different double-error-correcting binary codes. In these examples, the calculation of the σ's from the S's is relatively easy.

5.6 EXAMPLE

Design an encoder for the cyclic binary code of block length 23, generated by the minimal polynomial of a primitive twenty-third root of unity. Design a decoder which corrects all patterns of two or fewer errors.

SOLUTION

The powers of 2 mod 23 are 1, 2, 4, 8, 16, 9, 18, 13, 3, 6, 12. Since the multiplicative order of 2 mod 23 is 11, the cyclotomic polynomial $Q^{(23)}(x)$ must factor into two irreducible binary polynomials, each of degree 11. If α is a primitive twenty-third root of unity, then α^{-1} is *not* a conjugate of α, so the two irreducible factors of $Q^{(23)}(x)$ are reciprocal polynomials. With this information, and a moderate amount of trial and error, we may determine the factors of $Q^{(23)}(x)$ to be $x^{11} + x^9 + x^7 + x^6 + x^5 + x + 1$ and $x^{11} + x^{10} + x^6 + x^5 + x^4 + x^2 + 1$. Taking $g(x) = x^{11} + x^9 + x^7 + x^6 + x^5 + x + 1$, we design the encoder shown in Fig. 5.17.

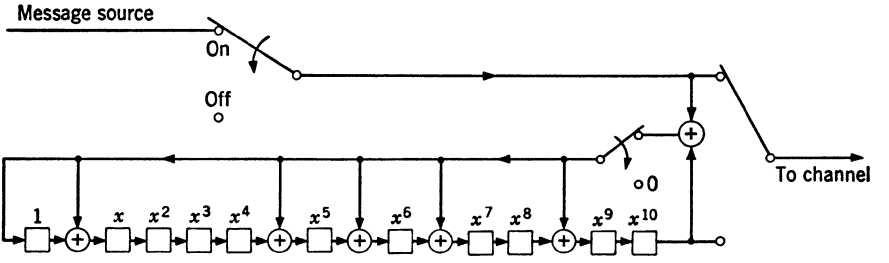


Fig. 5.17 Encoder for double-error-correcting BCH code of block length 23.

The decoder begins by dividing the received word by the minimal polynomial of α, $M^{(1)}(x)$, which in this case is equal to $g(x)$. Since 3 is a power of 2 mod 23, $M^{(3)}(x) = M^{(1)}(x)$. Thus, $r^{(3)}(x)$, the remainder of the division by $M^{(3)}(x)$, is identical to $r^{(1)}(x)$. As soon as the divisions are complete, the decoder may determine $S_1 = r^{(1)}(\alpha)$ and $S_3 = r^{(3)}(\alpha^3) = r^{(1)}(\alpha^3)$. Knowing S_1 and S_3, the decoder may find the error-locator polynomial from the usual formula for double-error-correcting binary BCH codes

$$\sigma(z) = \begin{cases} 1 & \text{if } S_1 = S_3 = 0 \quad \text{coset of weight 0} \\ 1 + S_1 z & \text{if } S_3 = S_1^3 \quad \text{includes all cosets of weight 1} \\ 1 + S_1 z + \left(S_1^2 + \dfrac{S_3}{S_1}\right) z^2 & \text{if } S_1 \neq 0 \text{ and } S_3 \neq S_1^3 \quad \text{includes all cosets of weight 2} \end{cases} \tag{1.47}$$

In this example, it is convenient to represent the elements of GF(2^{11}) as polynomials of degree <11 in α, where α is a root of the code's generator polynomial. The calculations of S_1^2 and S_3/S_1 may be accomplished by the methods described in Chap. 2. The errors may then be located by calculating $\sigma(\alpha^{-i})$, $i = 0, 1, 2, \ldots, 22$, via a Chien search. The reader should have no difficulty in designing all the circuitry for the complete decoder.

It should be noted that although the coefficients of the error-locator polynomial might be any elements in GF(2^{11}), the only meaningful reciprocal roots of this polynomial are twenty-third roots of unity. If the error-locator polynomial does not have all its roots among the twenty-third roots of unity, then this procedure fails to decode. If this happens, the decoder knows that more than two channel errors have occurred.

Although the incomplete decoding algorithm cannot correct any triple-error pattern, it will be shown in Sec. 15.2 that the complete decoding algorithm for the code can correct all patterns of three or fewer channel errors.

5.7 EXAMPLE

Design an encoder for the cyclic binary code of block length 17, generated by the minimal polynomial of a primitive seventeenth root of unity.

Design a decoder which corrects all patterns of two or fewer channel errors.

SOLUTION

The powers of 2 mod 17 are 1, 2, 4, 8, −1, −2, −4, −8. Since the multiplicative order of 2 mod 17 is 8, the cyclotomic polynomial $Q^{(17)}(x)$ must factor into two irreducible binary polynomials, each of degree 8. If α is a primitive seventeenth root of unity, then α^{-1} is a conjugate of α, so each of the two irreducible factors of $Q^{(17)}(x)$ is self-reciprocal. With this information, we deduce that both factors of $Q^{(17)}(x)$ are of the form $x^8 + Ax^7 + Bx^6 + Cx^5 + Dx^4 + Cx^3 + Bx^2 + Ax + 1$. Dividing this polynomial by $x + 1$ gives a remainder of D; dividing by $x^3 + 1$ gives a remainder of $(1 + B + C)x^2 + Dx + (1 + B + C)$; dividing by $x^5 + 1$ gives a remainder of $Dx^4 + (C + 1)x^3 + (A + B)x^2 + (A + B)x + (C + 1)$. Since $x + 1$ must not divide this irreducible octic, $D = 1$; since $x^2 + x + 1$ must not divide it, $B + C = 1$; since $x^4 + x^3 + x^2 + x + 1$ must not divide it, either $C = 1$ or $A = B$ or both. Thus, we are able to determine that the factors of $Q^{(17)}(x)$ are $x^8 + x^7 + x^6 + x^4 + x^2 + x + 1$ and $x^8 + x^5 + x^4 + x^3 + 1$. Since the latter factor has the fewer nonzero coefficients, we shall obtain circuitry with fewer connections if we choose this irreducible octic as the generator polynomial of the code. We set $g(x) = x^8 + x^5 + x^4 + x^3 + 1$. The reader should be able to design an encoder for this code without difficulty. It is also a straightforward matter to design circuitry which divides the received word by $M^{(1)}(x) = g(x)$, to obtain $r^{(1)}(x)$. From this, the decoder may easily find $S_1 = r^{(1)}(\alpha)$. In order to correct two errors, we would like to also know S_3. In this example, however, S_3 is nowhere to be found. Since 3 is not a power of 2 mod 17, α^3 is not a root of the code's generator polynomial. No combination of parity checks on the received word will yield S_3. In order to correct double errors, the decoder must adopt another attack. Indeed, it is not at all obvious (a priori) that it is possible to correct double errors with this code. This code has only eight check digits, which is the same number as the double-error-correcting binary BCH code of block length 15. Since eight check digits are just enough to correct any pair of two or fewer errors in a block of 15 binary digits, the conservative reader is understandably skeptical about a code with eight check digits which purports to correct all patterns of two or fewer errors in a block of 17 binary digits. Yet even skeptics should not give up until they have tried all reasonable possibilities.

It is apparent that the decoder cannot locate two errors merely from the knowledge of S_1. He cannot find S_3, so he must look for additional information elsewhere. He can find $S_2 = r^{(1)}(\alpha^2)$, $S_4 = r^{(1)}(\alpha^4)$, $S_8 = r^{(1)}(\alpha^8)$, $S_{16} = S_{-1} = r^{(1)}(\alpha^{-1})$. The most useful of these quantities

is S_{-1}. If there are two errors, S_1 and S_{-1} provide the decoder with two independent equations in the two unknowns

$$X_1 + X_2 = S_1$$
$$\frac{1}{X_1} + \frac{1}{X_2} = S_{-1}$$

Since $S_{-1} = (X_1 + X_2)/X_1X_2$, $X_1X_2 = S_1/S_{-1}$ and $\sigma(z) = 1 + S_1z + (S_1/S_{-1})z^2$. Thus, the decoder can find the error-locator polynomial if there are two channel errors. If there is only one channel error, then $\sigma(z) = 1 + S_1z$. Therefore, the decoder can decode if there is only one channel error. However, it is conceivable that the decoder may be unable to tell whether there are one or two channel errors. If there is one channel error, then $S_{-1} = S_1^{-1}$. Conceivably this might occur even when there are two channel errors. If this were the case, then $\sigma(z) = 1 + S_1z + S_1^2z^2 = (\partial + S_1z)(\partial^2 + S_1z)$, where $\partial \in \mathrm{GF}(4)$ is a primitive third root of unity. The reciprocal roots of $\sigma(z)$ are the error locations, $X_1 = S_1/\partial$ and $X_2 = S_1/\partial^2$. But $1/S_{-1} = X_1X_2/(X_1 + X_2) = S_1 = X_1 + X_2$, so $X_1X_2 = (X_1 + X_2)^2 = S_1^2$. Since X_1 and X_2 are seventeenth roots of unity, S_1^2, S_1, and $\partial = S_1/X_1$ are also seventeenth roots of unity. But $\partial^{17} = \partial^2 \neq 1$. From this contradiction we may conclude that if there are two channel errors, then $S_{-1} \neq S_1^{-1}$. We may therefore correct all patterns of two or fewer channel errors with decoding algorithm 5.71.

Decoding algorithm 5.71

$$\sigma(z) = \begin{cases} 1 & \text{if } S_1 = S_{-1} = 0 \\ 1 + S_1z & \text{if } S_1 \neq 0 \text{ and } S_{-1} = \dfrac{1}{S_1} \\ 1 + S_1z + \dfrac{S_1}{S_{-1}}z^2 & \text{if } S_1 \neq 0,\ S_{-1} \neq 0,\ S_{-1} \neq \dfrac{1}{S_1} \end{cases}$$

All the calculations of Algorithm 5.71 are performed in $\mathrm{GF}(2^8)$. The reader should have no difficulty in designing the necessary circuitry.

5.8 EQUIVALENCE OF CYCLIC CODES DEFINED IN TERMS OF DIFFERENT PRIMITIVE nTH ROOTS OF UNITY

The generator polynomial of a binary cyclic code of odd block length n must be a divisor of $x^n - 1 = \prod_{i=1}^{n} (x - \alpha^i)$, where α is a primitive nth

root of unity. If m is the multiplicative order of 2 mod n, then $\alpha \in \mathrm{GF}(2^m)$. Since $g(x)$ divides $x^n - 1$, the roots of $g(x)$ are a subset of the roots of $x^n - 1$. Therefore, every root of $g(x)$ is a power of α. Thus, we may define a cyclic code of block length n by specifying a primitive nth root of unity and K, the subset of integers mod n for which α^k is a root of $g(x)$. Since all conjugates of roots of $g(x)$ are also roots of $g(x)$, the set K must be closed under multiplication by 2 mod n. For example, if $1 \in K$, then $2 \in K$, $4 \in K$, $8 \in K$, . . . , corresponding to the fact that α^1, α^2, α^4, α^8, . . . are binary conjugates. Similarly, $3 \in K$ iff $6 \in K$. The number of check digits in the code is deg $g(x) = |K|$, the number of integers in the set K. The number of information digits in the code is $n - |K|$.

In order to define the generator polynomial $g(x)$ by the equation

$$g(x) = \prod_{k \in K} (x - \alpha^k)$$

we must specify both α, a particular primitive nth root of unity, and K, the set of powers of α which are roots of $g(x)$. Although different codes arise from different choices of α, these different codes are *equivalent* in the sense that they are permutations of each other. The precise statement is as follows.

Theorem 5.81 Let α be a particular primitive nth root of unity in a finite field of characteristic 2, let K be a subset of the integers mod n which is closed under multiplication by 2, let $g(x) = \prod_{k \in K} (x - \alpha^k)$, let j be any integer prime to n, and let $\check{g}(x) = \prod_{k \in K} (x - \alpha^{jk}) = \prod_{k \in jK} (x - \alpha^k)$. Let $c(x) = \sum_{k=0}^{n-1} c_k x^k$ be any binary polynomial of degree $< n$, and let $\check{c}(x) = \sum_{k=0}^{n-1} \check{c}_k x^k$ be the permuted polynomial, defined by $\check{c}_k = c_m$ where $m \equiv jk \bmod n$. Then $c(x)$ is a multiple of $g(x)$ iff $\check{c}(x)$ is a multiple of $\check{g}(x)$.

EXAMPLE Let $n = 31$, let α be a root of $x^5 + x^2 + 1$, let $K = 1, 2, 4, 8, 16, 3, 6, 12, 24, 17$, and let $j = 7$. The factors of $x^{31} - 1$ are given in Sec. 4.7. We take $g(x) = M^{(1)}(x)M^{(3)}(x) = 1 + x^3 + x^5 + x^6 + x^8 + x^9 + x^{10}$ and $\check{g}(x) = M^{(7)}(x)M^{(21)}(x)$. Since $11 \equiv 2 \cdot 21 \bmod 31$, $M^{(21)}(x) = M^{(11)}(x)$ and $\check{g}(x) = M^{(7)}(x)M^{(11)}(x) = 1 + x^3 + x^4 + x^9 + x^{10}$. One multiple of $g(x)$ is $c(x) = (1 + x)g(x) = 1 + x + x^3 + x^4 + x^5 + x^7 + x^8 + x^{11}$, which is permuted into $1 + x^9 + x^{27} + x^5 + x^{14} + x^1 + x^{10} + x^6 = \check{g}(x)(1 + x + x^3 + x^7 + x^{12} + x^{13} + x^{14} + x^{15} + x^{16} + x^{17})$.

Proof Let i be the multiplicative inverse of j mod n, so that $ij \equiv 1 \bmod n$. Then all the following are equivalent

$c(x)$ is a multiple of $g(x)$
$c(\alpha^k) = 0 \quad$ for all $k \in K$
$c(\alpha^{ik}) = 0 \quad$ for all $ik \in K$
$c(\alpha^{ik}) = 0 \quad$ for all $k \in jK$
$\check{c}(\alpha^k) = 0 \quad$ for all $k \in jK$
$\check{c}(x)$ is a multiple of $\check{g}(x)$ Q.E.D.

Corollary 5.82 Any cyclic binary code of odd block length is invariant under the permutation

$c_k \rightarrow c_i \qquad i \equiv 2k \bmod n$

Corollary 5.82 is useful in the study of the weight structure of cyclic binary codes. Any group of permutations which permutes codewords into codewords partitions the codewords into classes, each class consisting of the codewords which are permutations of each other. Since all codewords in each class have the same weight, one need only examine one codeword in each class to obtain a complete enumeration of the codewords by weight. The larger the permutation group which leaves the code invariant, the smaller the number of classes which must be examined. MacWilliams (1965) and Goethals (1965, 1966) have adeptly exploited the permutation group generated by the obvious cyclic shift and the permutation of Corollary 5.82 to obtain complete weight enumerations of all cyclic binary codes of block length up to 43.

If α and $\check{\alpha}$ are two primitive nth roots of unity with different minimal polynomials, then Theorem 5.81 asserts that the codewords in the code generated by the polynomial $g(x) = \prod_{k \in K} (x - \check{\alpha}^k)$ are permutations of the codewords in the code generated by the polynomial $g(x) = \prod_{k \in K} (x - \alpha^k)$. The error patterns in one code which are decoded correctly by a given decoding algorithm are permutations of error patterns in the other code which are correctly decoded by the same decoding algorithm; the error patterns in one code which cause decoding failure are permutations of error patterns in the other code which cause decoding failure; the error patterns in one code which are incorrectly decoded are permutations of error patterns in the other code which are incorrectly decoded. Thus, with respect to the permutation, the two codes are completely equivalent.

In many actual applications, however, permutated error patterns are *not* equiprobable. Errors usually have at least a small predilection to occur together in bursts. On most channels, a pattern of three consecutive errors is at least slightly more probable than a particular pattern of three errors scattered throughout the block. For these reasons, it may often be preferable to define the code in terms of one primitive nth root of unity rather than another. For example, we saw in Example 5.7 that $Q^{(17)}(x) = g(x)\check{g}(x)$, where $g(x) = 1 + x^3 + x^4 + x^5 + x^8$ and $\check{g}(x) = 1 + x + x^2 + x^4 + x^6 + x^7 + x^8$. Either of these two factors may be chosen as the generator polynomial of a cyclic binary code of block length 17 having eight check digits. The code generated by $g(x)$ will incorrectly decode a pattern of three consecutive errors in positions 3, 4, and 5 as a double error in positions 0 and 8. This is easily seen from the fact that there is a codeword (the generator polynomial itself) of weight 5 which has ones in positions 0, 3, 4, 5, and 8. On the other hand, it turns out that no codeword of weight 5 in the code generated by $\check{g}(x)$ has ones in three consecutive positions. Therefore, a double-error-correcting decoder for the code generated by $\check{g}(x)$ will fail to decode a triple consecutive error. If it uses Algorithm 5.71, it will end up with a purported quadratic error-locator polynomial $\sigma(x)$ which does not have any roots among the seventeenth roots of unity.

Thus, if the channel errors have any tendency at all to occur in bursts, there may be good reason to define a cyclic code in terms of one primitive nth root of unity rather than another. In general, the number of connections in the encoding and decoding circuitry will also depend on the particular primitive nth root of unity which is selected. The detailed criteria on which this selection should be based depend on the economics of the particular logical circuitry being used, as well as on the detailed burst statistics of the channel. The choices depend on the explicit factorization of $x^n - 1$ and $Q^{(n)}(x)$. Methods for finding these factors are presented in Chap. 6.

PROBLEMS

5.1. [Abramson (1959)] Show that the extended code formed by annexing an overall parity check to the end of a cyclic Hamming code is capable of correcting all single errors and all double adjacent errors which do not include the overall parity-check position. Is it possible to rearrange the columns in such a way that the code can correct all single and double adjacent errors?

5.2. [Fire (1959)] Let e and c be arbitrary

$$n = \text{least common multiple of } e \text{ and } c$$

Let $p(x)$ be an irreducible factor of $Q^{(e)}(x)$ over GF(q).

$$m = \text{order of } q \bmod e$$
$$g(x) = p(x)(x^c - 1)$$

Prove that the cyclic code of length n generated by $g(x)$ contains no codeword of the form $C(x) = x^iA(x) - x^jB(x) \bmod x^n - 1$ where $\deg A(x) < m$ and $\deg A(x) + \deg B(x) < c$. *Hint:* If $C(x) \equiv 0 \bmod x^c - 1$, then $A(x) = B(x)$ and $j \equiv i \bmod c$.

5.3. [Massey (1964)] A code is said to be *reversible* iff $[C_{n-1}, C_{n-2}, \ldots, C_1, C_0]$ is a codeword whenever $[C_0, C_1, \ldots, C_{n-2}, C_{n-1}]$ is a codeword.

(*a*) Prove that a cyclic code is reversible iff the reciprocal of every root of its generator polynomial is also a root of the generator polynomial.

(*b*) Prove that if -1 is a power of $q \bmod n$, then every cyclic code of length n over GF(q) is reversible.

5.4. Exhibit a reversible cyclic double-error-correcting binary code of length $n = 4^i + 1$ and rate $R = 1 - 4i/n$.

5.5. If n is prime, if m is the multiplicative order of $2 \bmod n$, and if $j = (n - 1)/m$, then prove the following assertions:

(*a*) There are 2^{j+1} cyclic binary codes of length n.

(*b*) There are $\sum_{\substack{l \\ l \mid j}} \frac{1}{l} \sum_{\substack{d \\ d \mid l}} \mu(d) 2^{l/d}$ binary cyclic codes of length n which are inequivalent with respect to Theorem 5.81.

6

The Factorization of Polynomials over Finite Fields

6.1 A GENERAL ALGORITHM

We present here an algorithm for factoring a given polynomial, $f(x) = \sum_{k=0}^{m} f_k x^k$, $f_k \in \mathrm{GF}(q)$, into powers of irreducible polynomials. If q is small, this algorithm proves effective even if m is quite large.

1. We construct the $m \times m$ matrix Q over $\mathrm{GF}(q)$, whose ith row represents $x^{q(i-1)}$ reduced modulo $f(x)$. Specifically,

$$x^{qi} \equiv \sum_{k=0}^{m-1} \mathsf{Q}_{i+1,k+1} x^k \bmod f(x)$$

The Q matrix may be computed with a shift register wired to multiply by $x \bmod f(x)$. The register is started at 1, which is the first row of Q. After q shifts, it contains the second row of Q, . . . , etc. After $q(m-1)$ shifts, it contains the last row of Q.

Given any polynomial $g(x)$ of degree $<m$ over $\mathrm{GF}(q)$, $g(x) = \sum_{i=0}^{m-1} g_i x^i$, we may compute the residue of $[g(x)]^q \bmod f(x)$ by multi-

plying the row vector $[g_0, g_1, \ldots, g_{m-1}]$ by the $\mathcal{Q}$ matrix. This follows from the observation that

$$[g(x)]^q = g(x^q) = \sum_{i=0}^{m-1} g_i x^{qi} \equiv \sum_{i=0}^{m-1} \left(\sum_{k=0}^{m-1} g_i \mathcal{Q}_{i+1,k+1} x^k \right)$$

$$= \sum_{k=0}^{m-1} \left(\sum_{i=0}^{m-1} g_i \mathcal{Q}_{i+1,k+1} \right) x^k$$

Similarly, we could compute $[g(x)]^q - g(x) \bmod f(x)$ by multiplying the row vector $[g_0, g_1, \ldots, g_{m-1}]$ by the matrix $\mathcal{Q} - \mathcal{I}$, where $\mathcal{I}$ is the $m \times m$ identity matrix over GF(q).

2. We find a set of row vectors which span the null space of $\mathcal{Q} - \mathcal{I}$. This may be done by appropriate column operations on the matrix $\mathcal{Q} - \mathcal{I}$, as described in Secs. 2.5 and 2.6. Each such row vector in the null space of $\mathcal{Q} - \mathcal{I}$ represents a polynomial $g(x)$ which satisfies the equation $[g(x)]^q - g(x) \equiv 0 \bmod f(x)$, and, conversely, each $g(x)$ which satisfies this equation is represented by a row vector in the null space of $\mathcal{Q} - \mathcal{I}$.
3. We select any of the polynomials $g(x)$ found in step 2 and apply Euclid's algorithm to determine the greatest common divisor of $f(x)$ and $g(x) - s$ for each $s \in \mathrm{GF}(q)$.† We then have the factorization of Theorem 6.11.

Theorem 6.11

$$f(x) = \prod_{s \in \mathrm{GF}(q)} [\gcd(f(x), g(x) - s)]$$

Remark If $g(x)$ is a scalar, then this factorization degenerates into

$$f(x) = \gcd(f(x), 0) \prod_{s \neq 0} \gcd(f(x), s)$$

$$= f(x) \prod_{s \neq 0} 1$$

However, if $g(x)$ has positive degree, then the factorization is nontrivial. In general, this factorization is incomplete; the gcd $(f(x), g(x) - s)$ may be reducible.

Proof of Theorem 6.11 Since $[g(x)]^q - g(x) \equiv 0 \bmod f(x)$, $f(x)$ divides $[g(x)]^q - g(x) = \prod_{s \in \mathrm{GF}(q)} [g(x) - s]$. Therefore $f(x)$ also

† In practice, there is no need to perform all of Euclid's algorithm q separate times to determine all the gcd's. A shortcut will be seen in Example 6.12.

divides $\prod_{s \in GF(q)} [\gcd (f(x), g(x) - s)]$. On the other hand, gcd $(f(x), g(x) - s)$ divides $f(x)$. If $s \neq t$, and $s,t \in GF(q)$, then $g(x) - s$ and $g(x) - t$ are relatively prime, as are gcd $(f(x), g(x) - s)$ and gcd $(f(x), g(x) - t)$. Therefore, $\prod_{s \in GF(q)} [\gcd (f(x), g(x) - s)]$ divides $f(x)$. Assuming both polynomials to be monic, they must be equal since each divides the other. Q.E.D.

Example 6.12 Let $f(x)$ be the binary polynomial 1110001110001, or $f(x) = 1 + x + x^2 + x^6 + x^7 + x^8 + x^{12}$. The successive powers of x are

		100000000000
100000000000		001000000000
010000000000		000010000000
001000000000		000000100000
000100000000		000000001000
000010000000		000000000010
000001000000	$\mathcal{Q} =$	111000111000
000000100000		001110001110
000000010000		111011011011
000000001000		101010010010
000000000100		110010011100
000000000010		001100100111
000000000001 $\Rightarrow$		
111000111000		000000000000
011100011100		011000000000
001110001110		001010000000
000111000111		000100100000
111011011011		000010001000
100101010101		000001000010
101010010010	$\mathcal{Q} - \mathcal{I} =$	111000011000
010101001001		001110011110
110010011100		111011010011
011001001110		101010010110
001100100111		110010011110
		001100100110

If we number the columns of $\mathcal{Q} - \mathcal{I}$ from 0 to 11, then the upper right quarter of the $\mathcal{Q} - \mathcal{I}$ matrix may be zeroed if we add the third column to the sixth column, the first, second, and fourth columns to the eighth column, and the fifth column to the tenth

column. The lower right quarter of the $\mathcal{Q} - \mathcal{I}$ matrix then becomes

$$\mathcal{R} = \begin{matrix} 011000 \\ 111110 \\ 011001 \\ 010110 \\ 011110 \\ 001110 \end{matrix}$$

The equation $[g_6,g_7, \ldots ,g_{11}]\mathcal{R} = \mathbf{0}$ is found to have solutions $[g_6,g_7, \ldots ,g_{11}] = [A,0,0,A,0,A]$ where $A = 0$ or 1. The first six coordinates of g are then readily found from the equation $\mathbf{g}(\mathcal{Q} - \mathcal{I}) = \mathbf{0}$, with solutions $\mathbf{g} = [B,A,0,A,A,0,A,0,0,A,0,A]$ where $A,B \in \mathrm{GF}(2)$. Finally we apply Euclid's algorithm to $f(x) = 1110001110001$ and $g(z) = s10110100101$. By letting $t = s + 1$, and leaving s as an indeterminate, we may effectively find the gcd of 111000111001 and 010110100101 with the same computation that computes the gcd of 111000111001 and 110110100101:

$$\begin{array}{r} 1110001110001 \\ \underline{s10110100101} \\ 1t001110101 \\ \underline{s10110100101} \\ s0t11101 \\ \underline{1t001110101} \\ 1t0s1s01 \\ \underline{s0t11101} \\ tttt0t \end{array}$$

If $t = 0$, the gcd is 10011101; if $s = 0$, the gcd of 1110001110001 and 010110100101 is equal to the gcd of 111101 and 11001001, which is 111101 itself. Both 111101 and 10011101 are irreducible, and the factorization is complete: $1 + x + x^2 + x^6 + x^7 + x^8 + x^{12} = (1 + x + x^2 + x^3 + x^5)(1 + x^3 + x^4 + x^5 + x^7)$ over GF(2).

In general, suppose $f(x) = \prod_{i=1}^{n} [p^{(i)}(x)]^{e_i}$, where each $p^{(i)}(x)$ is an irreducible polynomial over GF(q). Then $f(x)$ divides $\prod_{s \in \mathrm{GF}(q)} [g(x) - s]$ iff each $[p^{(i)}(x)]^{e_i}$ divides $g(x) - s_i$ for some $s_i \in \mathrm{GF}(q)$. On the other hand, given any set of scalars $s_1, s_2, \ldots , s_n \in \mathrm{GF}(q)$, then the Chinese remainder theorem 2.17 guarantees the existence of a unique $g(x) \bmod f(x)$

such that $g(x) \equiv s_i \bmod [p^{(i)}(x)]^{e_i}$ for all i. Since there are q^n choices of $s_1, s_2, \ldots, s_n$, there are exactly q^n solutions of the equation $[g(x)]^q - g(x) \equiv 0 \bmod f(x)$. This proves Theorem 6.13.

Theorem 6.13 The number of distinct irreducible factors of $f(x)$ is equal to the dimension of the null space of $\mathcal{Q} - \mathcal{I}$.

In particular, the polynomial $f(x)$ is the power of an irreducible polynomial iff the null space of $\mathcal{Q} - \mathcal{I}$ has dimension 1. In this case, the only solutions of $[g(x)]^q - g(x) \equiv 0 \bmod f(x)$ are scalars in GF(q), and the null space of $\mathcal{Q} - \mathcal{I}$ contains only vectors of the form [s,0,0, . . . ,0]. If the null space of $\mathcal{Q} - \mathcal{I}$ has dimension n, it has a basis consisting of n monic polynomials: $g^{(1)}(x), g^{(2)}(x), \ldots, g^{(n)}(x)$. Without loss of generality, we may assume that $g^{(n)}(x) = 1$ and that the other $n - 1$ basis polynomials have positive degree.

When we apply Euclid's algorithm to $f(x)$ and $g^{(1)}(x) - s$, we obtain a factorization of $f(x)$. If this gives fewer than n factors of $f(x)$, then we may compute the gcd of $g^{(2)}(x) - s$ and each known factor of $f(x)$. By this process, we may continue to refine the factorization of $f(x)$. The following argument shows that this process must eventually yield n irreducible powers which are factors of $f(x)$.

Let $\mathcal{C}$ be the $n \times n$ matrix over GF(q) defined by the equations $g^{(j)}(x) \equiv \mathcal{C}_{i,j} \bmod [p^{(i)}(x)]^{e_i}$. Then $\mathcal{C}$ must be nonsingular. For if $\sum_j A_j \mathcal{C}_{i,j} = 0$ for all i, then $\sum_j A_j g^{(j)}(x) \equiv 0 \bmod [p^{(i)}(x)]^{e_i}$ for all i, whence $\sum_j A_j g^{(j)}(x) = 0$, contradicting the linear independence of $g^{(1)}(x), g^{(2)}(x), \ldots, g^{(n)}(x)$. When we apply Euclid's algorithm to $f(x)$ and $g^{(j)}(x) - s$, we obtain a factorization of $f(x)$ into as many different factors as there are distinct elements in the jth column of $\mathcal{C}$. The irreducible powers $[p^{(i)}(x)]^{e_i}$ and $[p^{(k)}(x)]^{e_k}$ are separated iff $\mathcal{C}_{i,j} \neq \mathcal{C}_{k,j}$. Since $\mathcal{C}$ is nonsingular, for every i and k there exists some j such that $\mathcal{C}_{i,j} \neq \mathcal{C}_{k,j}$. Thus any two irreducible-power factors of $f(x)$ will be separated by some $g^{(j)}(x)$.

Any irreducible power with nonzero derivative may be factored by applying Euclid's algorithm to the polynomial and its derivative.

If q is very large, this algorithm can be improved by several embellishments, some of which are discussed by Knuth (in press), vol. II.

★ 6.2 DETERMINING THE PERIOD OF A POLYNOMIAL

Given a polynomial $f(x)$, we are often interested in determining its period, i.e., the smallest j for which $f(x)$ divides $x^j - 1$. If $f(x)$ is an irreducible polynomial of degree m over GF(q), then the period of $f(x)$ is equal to the multiplicative order of its roots, which is a factor of $q^m - 1$.

How is the period of a factorable polynomial related to the periods of its irreducible factors? Let us first suppose that $f(x)$ is an irreducible polynomial of period n and try to find the period of a power of $f(x)$, $[f(x)]^m$. Since n is a factor of $q^{\deg f} - 1$, it is a nonmultiple of p, the characteristic of GF(q). Since the period of $f(x)$ is n, $f(x)$ divides $x^k - 1$ iff k is a multiple of n. If $k = p^i j$, where j is a nonmultiple of p, then $x^k - 1 = (x^j - 1)^{p^i}$. The polynomial $x^j - 1$ has no repeated roots, because it divides $x^{q^m-1} - 1$ for that m which is the multiplicative order of the integer $q \bmod j$. Thus, every irreducible factor of $x^k - 1$ has multiplicity p^i. We conclude that if the irreducible polynomial $f(x)$ has period n, then the period of $[f(x)]^m$ is n times the least power of $p \geq m$.

Next suppose that $f(x)$ is the product of several irreducible factors $f^{(1)}(x)$, $f^{(2)}(x)$, . . . which have multiplicities m_1, m_2, ... Since $x^k - 1$ divides $x^N - 1$ iff k divides N, we conclude that the period of $f(x)$ is the least common multiple of the periods of $[f^{(1)}(x)]^{m_1}$, $[f^{(2)}(x)]^{m_2}$, ... This, in turn, is the least common multiple of the periods of the $f^{(i)}(x)$ times the least power of p which is greater than or equal to all the m_i.

Theorem 6.21 If $f(x) = \prod_i [f^{(i)}(x)]^{m_i}$, where the $f^{(i)}(x)$ are irreducible polynomials of periods n_i over GF(q), of characteristic p, then the period of $f(x)$ is the least common multiple of the n_i times the least power of p which is not less than any of the m_i.

Thus, the problem of determining the period of a polynomial reduces to the problem of determining the period of each of its irreducible factors. What is the best method of determining n, the period of an irreducible polynomial $f(x)$? If $f(x)$ has degree m, then n must be a factor of $q^m - 1$, so our first step is to factor the integer $q^m - 1 = \prod_i p_i^{e_i}$, where the p_i are prime integers. We may then determine whether n is a multiple of $p_j^{e_i}$ by computing the residue of $x^{(q^m-1)/p_i} \bmod f(x)$. This computation is accomplished by multiplying together a suitable combination of the residues of x, x^q, x^{q^2}, . . . , $x^{q^{m-1}} \bmod f(x)$. If $x^{(q^m-1)/p_i} \not\equiv 1 \bmod f(x)$, then the period of $f(x)$ is a multiple of $p_j^{e_i}$; if $x^{(q^m-1)/p_i} \equiv 1 \bmod f(x)$, then the period of $f(x)$ is not a multiple of $p_j^{e_i}$. In that case, we then check to see whether the period of $f(x)$ is a multiple of $p_j^{e_i-1}$, $p_j^{e_i-2}$, ... This computation is repeated for all prime factors of $q^m - 1$. From the results of these computations, we may determine n, the period of $f(x)$.

Example 6.22 What is the block length of the shortest cyclic binary code whose generator polynomial is $g(x) = x^9 + x + 1$?

Solution We first compute the $\mathcal{Q}$ matrix, whose rows are the even powers of x reduced mod $g(x)$. This gives

Power	*Residue*
0	100000000
2	001000000
4	000010000
6	000000100
8	000000001
10	011000000
12	000110000
14	000001100
16	000000011

Since the nullity of $\mathcal{Q} - \mathcal{I}$ is 1, $g(x)$ is an irreducible power. Since $g'(x) = x^8 + 1$, which has no factors in common with $g(x)$, $g(x)$ is irreducible. We next compute the residues of x^{2^5}, x^{2^6}, . . . each of which is obtained by multiplying the previous residue by $\mathcal{Q}$. This gives

Power	*Residue*
32	000001111
64	011111111
128	010101010
256	010001000
512	010000000

Of course, it is no surprise to find $x^{2^9} \equiv x$, but there is no harm in making the computation anyway, as a check on the previous residues. We must next decide whether $x^n \equiv 1$ for any n which is a proper divisor of $2^9 - 1 = 511 = 7 \cdot 73$. The choice $n = {}^{511}\!/_{73} = 7$ is hopeless, because we have already seen that $x^8 \not\equiv x$. But we must check x^{73}. In binary, 73 = 1001001, so we compute x^{73} as a product of x, x^8, and x^{64}. This gives

$$\begin{aligned} xx^8x^{64} &\equiv (1 + x)x^{64} \\ &\equiv (1 + x)(x + x^2 + x^3 + x^4 + x^5 + x^6 + x^7 + x^8) \\ &\equiv x + x^9 \equiv 1 \end{aligned}$$

Thus, the period of $x^9 + x + 1$ is $n = 73$.

For moderate to large values of m, the factorization of the integer $q^m - 1$ proves the most difficult part of determining the period of an irreducible polynomial of degree m. Although the novice might be

tempted to attack this factorization as the need arises, the wiser course is to rely on the accumulated literature on the subject, which is based on the cleverest tricks of the world's best number theorists combined with many hours of computer time. Collating the results reported by Brillhart and Selfridge (1967) and the papers which they cite, one can obtain a table of the complete factorization of $2^m - 1$ for all $m \leq 136$ and for many larger m.†

The period of $f(x^t)$ and the periods of its irreducible factors are related to the period of $f(x)$. In order to determine this relation in general, it suffices to consider only certain special cases. First, we may assume that $f(x)$ is irreducible, for if $f(x) = g(x)h(x)$, then $f(x^t) = g(x^t)h(x^t)$, and we may proceed to analyze $g(x^t)$ and $h(x^t)$ separately. Next, we may assume that t is prime, for if $t = rs$, then $f(x^t) = f((x^r)^s)$, and we may proceed in two stages. We may also exclude the trivial case when the prime t is equal to the characteristic of the field. The remaining cases are covered by Theorem 6.23.

Theorem 6.23 Let $f(x)$ be an irreducible polynomial of period n and degree m over GF(q), and let t be a prime, $t \nmid q$. If $t|n$, then every irreducible factor of $f(x^t)$ has period tn. If $t \nmid n$, then one irreducible factor of $f(x^t)$ has period n and the other factors have period tn.

Proof Using the properties of cyclotomic polynomials proved in Sec. 4.3, $f(x^t)|Q^{(n)}(x^t)$. If $t|n$, $Q^{(n)}(x^t) = Q^{(nt)}(x)$, which proves the first assertion. If $t \nmid n$, $Q^{(n)}(x^t) = Q^{(n)}(x)Q^{(nt)}(x) = Q^{(nt)}(x)$ $\left(\prod_{\substack{k, \\ (k,n)=1}} f^{(k)}(x) \right)^{1/m}$, where $f^{(k)}(x)$ is the minimal polynomial of the kth powers of the roots of $f(x)$. Each $f^{(k)}(x)$ is irreducible, and $f^{(k)}(x)$ divides $f(x^t)$ iff there exists a j for which $kt \equiv q^j \bmod n$. Q.E.D.

★ 6.3 TRINOMIALS OVER GF(2)

The methods of Secs. 6.1 and 6.2 may be easily programmed. Using a modern computer, it is quite feasible to factor any binary polynomial of degree up to 1,000.

† Using special techniques introduced by Lucas (1878), it is relatively easy for moderate values of m to determine whether or not the number $2^m - 1$ is prime. If m is prime, this number is called a *Mersenne number;* if $2^m - 1$ is prime, it is called a *Mersenne prime.* Unfortunately, the Lucas procedure often shows that a Mersenne number is composite without giving any information about the factors. In other cases, however, the procedure has enabled Gillies (1964) to prove that certain enormous numbers, including $2^{11,213} - 1$, are prime.

Among the most interesting binary polynomials are the trinomials, $x^n + x^k + 1$. If α is a root in GF(2^n) of an irreducible binary trinomial of degree n, then the first n powers of α provide a particularly attractive basis for representing GF(2^n), since it is then possible to multiply by α using an n-bit feedback shift register whose connections require only one two-input adder (see Sec. 2.41). In coding theory, most practical interest centers on fields of order 2^{12} or smaller. From this viewpoint, the interest in GF(2^n) for $n > 12$ is largely academic. However, there are certain other applications of feedback shift registers, such as the generation of sequences of pseudorandom bits, where there is practical interest in finding primitive binary trinomials of as large degree as possible.

Since the $\mathcal{Q}$ matrix corresponding to a trinomial has very few ones, a basis of the null space of $\mathcal{Q} - \mathcal{I}$ may be determined according to the methods of Sec. 2.6. For this reason, it is much easier to factor a binary trinomial than an arbitrary binary polynomial of comparable degree.

In some applications, the explicit factors of a given polynomial are of little importance. What matters is only whether or not the given polynomial is irreducible. This question can be decided by the method of Sec. 6.1, or by other algorithms [Gleason, MacRae (unpublished)] of comparable complexity. Using MacRae's algorithm and a CDC 6600 computer, Zierler has found all irreducible binary trinomials of degree up to 1,000. Using the latest known results on the factorizations of $2^n - 1$, Brillhart has determined the orders of many of Zierler's irreducible trinomials. The results are given by Brillhart and Zierler (1969).

Although these data prove fascinating to those of us who enjoy numerology, no obvious patterns seem to emerge. Most of the known results, which have been summarized by Golomb (1967), apply only to (rare) specially selected values of n and k. It is not even known whether there exist primitive binary trinomials of arbitrarily large degree. The only general nontrivial theorem about binary trinomials, due to Swan (1962), gives only the parity of the number of irreducible factors of $x^n + x^k + 1$. We shall present this theorem in Corollary 6.696. It appears very difficult even to determine the number of irreducible binary trinomials of given degree, although the number of irreducible polynomials in various larger ensembles may be determined with relative ease, as indicated by Eq. (3.37) and Probs. 3.3 and 3.4.

6.4 FACTORING $x^n - 1$ EXPLICITLY

As one important example of the algorithm introduced in Sec. 6.1, we consider the factorization of the polynomial $x^n - 1$ over GF(q), where n and q are relatively prime. In this case, the $\mathcal{Q}$ matrix becomes $\mathcal{Q}_{i+1,j+1} = 1$

iff $qi \equiv j \bmod n$, $\mathfrak{Q}_{i+1,j+1} = 0$ iff $qi \not\equiv j \bmod n$. As a specific example, suppose $n = 15$ and $q = 2$.

	100000000000000	000000000000000	0
	001000000000000	011000000000000	1
	000010000000000	001010000000000	2
	000000100000000	000100100000000	3
	000000001000000	000010001000000	4
	000000000010000	000001000010000	5
	000000000000100	000000100000100	6
$\mathfrak{Q} =$	000000000000001	$\mathfrak{Q} - \mathfrak{I} =$ 000000010000001	7
	010000000000000	010000001000000	8
	000100000000000	000100000100000	9
	000001000000000	000001000010000	10
	000000010000000	000000010001000	11
	000000000100000	000000000100100	12
	000000000001000	000000000001010	13
	000000000000010	000000000000011	14

By suitably permuting the rows and columns, we can bring $\mathfrak{Q} - \mathfrak{I}$ into the form

0	0000	0000	0000	00	0
0	1100	0000	0000	00	1
0	0110	0000	0000	00	2
0	0011	0000	0000	00	4
0	1001	0000	0000	00	8
0	0000	1100	0000	00	7
0	0000	0110	0000	00	14
0	0000	0011	0000	00	13
0	0000	1001	0000	00	11
0	0000	0000	1100	00	3
0	0000	0000	0110	00	6
0	0000	0000	0011	00	12
0	0000	0000	1001	00	9
0	0000	0000	0000	11	5
0	0000	0000	0000	11	10

A basis for the null space of $\mathfrak{Q} - \mathfrak{I}$ is seen to be

$$g^{(1)}(x) = x + x^2 + x^4 + x^8$$
$$g^{(2)}(x) = x^7 + x^{14} + x^{13} + x^{11}$$
$$g^{(3)}(x) = x^3 + x^6 + x^{12} + x^9$$
$$g^{(4)}(x) = x^5 + x^{10}$$

In general, if $f(x) = x^n - 1$ over GF(q), then we may choose

$$g(x) = \sum_{k \in K} x^k$$

where K is any set of numbers which is closed under multiplication by q mod n. For each $g(x)$, there exist values of s for which $g(x) - s$ has some nontrivial factor in common with $x^n - 1$. Thus, no matrix manipulations are necessary to factor the polynomial $x^n - 1$; we may write down the g's by inspection. The common factor of $x^n - 1$ and $g(x) - s$ may be found by Euclid's algorithm. By reapplying Euclid's algorithm with other g's, we may eventually factor $x^n - 1$ into irreducible factors. In most cases, it is advantageous to start with the cyclotomic factorization of $x^n - 1$ and use Euclid's algorithm and the g's to continue the factorization of the cyclotomic polynomials into their irreducible factors.

★ 6.5 DETERMINING THE DEGREES OF THE IRREDUCIBLE FACTORS OF THE CYCLOTOMIC POLYNOMIALS

Since every element of order n has the same number of conjugates with respect to GF(q), every irreducible factor of the cyclotomic polynomial $Q^{(n)}(x)$ has the same degree over GF(q). This degree is m, which is the multiplicative order of q mod n. Thus, although the explicit factors of $Q^{(n)}(x)$ may be determined only by polynomial manipulations using Euclid's algorithm, the degrees of these irreducible factors may be computed by appropriate manipulations with integers.

The most straightforward approach for determining the order of q mod n is to compute the successive residues, q, q^2, q^3, . . . , mod n, until one finds the least m such that $q^m \equiv 1 \bmod n$. However, if n is large, this procedure becomes tedious. Fortunately, the calculations may be considerably shortened by the use of several simple number theoretic results which we now present.

Theorem 6.51 If $n = \prod_i n_i^{j_i}$, n_i distinct primes, then the order of q mod n is the least common multiple of the orders of q mod $n_i^{j_i}$.

EXAMPLE Suppose $n = 45$, $q = 2$. Since the order of 2 mod 5 is 4 and the order of 2 mod 9 is 6, the order of 2 mod 45 is 12.

Proof of Theorem 6.51 $q^m \equiv 1 \bmod n$ iff $q^m \equiv 1 \bmod n_i^{j_i}$ for every i. $q^m \equiv 1 \bmod n_i^{j_i}$ iff m is a multiple of the order of q mod $n_i^{j_i}$.

Thus, $q^m \equiv 1 \bmod n$ iff m is a multiple of each of the orders of $q \bmod n_i^{j_i}$. The order of $q \bmod n$ is the least such multiple. Q.E.D.

Theorem 6.52 Let n be prime, and let m_j be the order of $q \bmod n^j$. Let $q^{m_2} - 1$ be divisible by n^i but not by n^{i+1}. (This defines i.) Then, for every j, either $m_{j+1} = m_j$ or $m_{j+1} = nm_j$, and

$$m_j = \begin{cases} m_2 & \text{if } 2 \leq j \leq i \\ n^{j-i}m_2 & \text{if } j \geq i \end{cases}$$

EXAMPLE Suppose we wish to know the order of 7 mod $1{,}024 = 2^{10}$. We then take $q = 7$, $n = 2$. We see that $m_1 = 1$, but $m_2 = 2$, and $q^{m_2} - 1 = 48$, which is divisible by 2^4 but not by 2^5. This means $i = 4$. According to the theorem, $m_{10} = 2^{10-4}m_2 = 128$.

Remark If $n = 1{,}093$ or $n = 3{,}511$, the order of 2 mod n^2 is equal to the order of 2 mod n; for all other primes $<2^{24}$, the order of 2 mod n^2 equals n times the order of 2 mod n.

Proof of Theorem 6.52 $q^{m_j} \equiv 1 \bmod n^j$, so $(q^{m_j})^k \equiv 1 \bmod n^j$ and $\sum_{k=0}^{n-1} (q^{m_j})^k \equiv \sum_{k=0}^{n-1} 1 \equiv n \bmod n^j$. Thus, $\sum_{k=0}^{n-1} (q^{m_j})^k \equiv 0 \bmod n$ and $q^{m_j} - 1 \equiv 0 \bmod n^j$. Multiplying these together gives $q^{nm_j} - 1 = (q^{m_j} - 1)\left[\sum_{k=0}^{n-1} (q^{m_j})^k\right] \equiv 0 \bmod n^{j+1}$. Therefore m_{j+1} divides nm_j. Since m_{j+1} must be a multiple of m_j, and n is prime, there are only two possibilities: either $m_{j+1} = m_j$, or $m_{j+1} = nm_j$.

If $q^{m_2} - 1$ is divisible by n^i but not by n^{i+1}, then $m_j = m_2$ for $j = 2, 3, \ldots, i$, and $m_{i+1} = nm_2$. For $j \geq i$, we claim that $q^{m_j} - 1$ is not divisible by n^{j+1}, and therefore, $m_{j+1} = nm_j$. This claim follows by induction on j from the fact that $(q^{nm_j} - 1)/(q^{m_j} - 1)$ cannot be divisible by n^2, because it is congruent to $n \bmod n^j$. Q.E.D.

These two theorems essentially reduce the computation of the order of $q \bmod n$ to the computations of the orders of q mod each prime factor of n. Thus, we need really consider only the computation of $q \bmod n$ when n is prime. The most obvious result is Theorem 6.53.

Theorem 6.53 If n is prime, the order of q mod n is a divisor of $n - 1$.

Proof According to Fermat's theorem 4.25, $q^n \equiv q \bmod n$, for any number q and any prime n. If (as we assume throughout) q and n are relatively prime, then $q^{n-1} \equiv 1 \bmod n$, so $n - 1$ is a multiple of the order of q mod n. Q.E.D.

One is still faced with the computation of $q^d \bmod n$, for various d's which are divisors of $n - 1$. The longest such computation occurs for the largest divisor of $n - 1$, which is $(n - 1)/2$. When $q = 2$ (or a power of 2), this calculation is simplified by Theorem 6.54.

Theorem 6.54 If n is a prime $\equiv \pm 1 \bmod 8$, then $2^{(n-1)/2} \equiv 1 \bmod n$, but if n is a prime $\equiv \pm 3 \bmod 8$, then $2^{(n-1)/2} \equiv -1 \bmod n$.

Proof Let $[x]$ denote the greatest integer less than or equal to x.

$$2^{(n-1)/2} \prod_{k=1}^{(n-1)/2} k = \prod_{k=1}^{(n-1)/2} 2k = \left(\prod_{k=1}^{[(n-1)/4]} 2k \right) \left(\prod_{k=[(n-1)/4]+1}^{(n-1)/2} 2k \right)$$

For each k, $[(n - 1)/4] < k \leq (n - 1)/2$, $-2k \equiv n - 2k \bmod n$, so

$$2^{(n-1)/2} \prod_{k=1}^{(n-1)/2} k$$

$$\equiv \left(\prod_{k=1}^{[(n-1)/4]} 2k \right) (-1)^{(n-1)/2-[(n-1)/4]} \left(\prod_{k=[(n-1)/4]+1}^{(n-1)/2} (n - 2k) \right)$$

$$= (-1)^{(n-1)/2-[(n-1)/4]} \prod_{k=1}^{(n-1)/2} k$$

This is because the even terms in the product $\prod_{k=1}^{(n-1)/2} k$ occur in the product of the $2k$ terms; the odd terms in this product occur in the product of the $n - 2k$ terms. Since $k = 1, 2, \ldots, (n - 1)/2$ are each relatively prime to the prime n, we may cancel these factors, obtaining

$$2^{(n-1)/2} \equiv (-1)^{(n-1)/2-[(n-1)/4]} \bmod n$$

If $n \equiv \pm 1 \bmod 8$, the exponent of -1 is even; if $n \equiv \pm 3 \bmod 8$, the exponent is odd. Q.E.D.

As an example of the use of all these theorems, we shall determine the degrees of the irreducible binary factors of $Q^{(25,025)}$. We first factor $25{,}025 = 5^2 \cdot 1{,}001 = 5^2 \cdot 7 \cdot 11 \cdot 13$. The degree of this polynomial is therefore given by $\varphi(25{,}025) = 4 \cdot 5 \cdot 6 \cdot 10 \cdot 12 = 14{,}400$. The number 7 is a prime $\equiv -1 \bmod 8$, so the order of 2 mod 7 must be a factor of $(n-1)/2 = 3$. We see that the order of 2 mod 7 is 3. The other prime factors of the block length, 5, 11, and 13, are each $\equiv \pm 3 \bmod 8$. It follows that the order of 2 mod any such factor is a divisor of $n-1$ but not a divisor of $(n-1)/2$. Thus, the order of 2 mod 5 is 4; the order of 2 mod 11 is 10, and the order of 2 mod 13 is a divisor of 12 but not a divisor of 6. The only possibilities are 4 and 12, and since $2^4 \equiv 3 \bmod 13$, we decide that the order of 2 mod 13 is 12. We must also compute the order of 2 mod 5^2, which is either 4 or $4 \cdot 5$. We quickly decide in favor of the latter possibility. Thus, we know that the order of 2 mod 5^2 is 20; the order of 2 mod 7 is 3; the order of 2 mod 11 is 10; the order of 2 mod 13 is 12. The order of 2 mod 25,025 is the least common multiple of 20, 3, 10, and 12, which is 60. Therefore, $Q^{(25,025)}$ is the product of 240 irreducible binary factors, each of degree 60.

Using more advanced methods in the theory of cyclotomy, it is possible to prove several other results analogous to Theorem 6.54. As a sample of these results, we state Theorems 6.55 and 6.56.

Theorem 6.55 If n is a prime $\equiv 1 \bmod 3$, then $2^{(n-1)/3} \equiv 1 \bmod n$ iff there exist integers x and y such that $n = x^2 + 27y^2$.

Theorem 6.56 If n is a prime $\equiv 1 \bmod 4$, then $2^{(n-1)/4} \equiv 1 \bmod n$ iff there exist integers x and y such that $n = x^2 + 64y^2$.

Proofs of these theorems are given by Hall (1967) and Storer (1967).

★ 6.6 IS THE NUMBER OF IRREDUCIBLE FACTORS OF $f(x)$ OVER GF(q) ODD OR EVEN?

Although it appears unlikely that any general method for factoring polynomials over finite fields can improve substantially on the algorithm presented in Sec. 6.1, other methods sometimes enable one to obtain certain information about the factors with even less effort. For example, every high-school algebra student knows that the roots of the real quadratic equation $ax^2 + bx + c = 0$ are $x = (-b \pm \sqrt{b^2 - 4ac})/2a$. If q is a power of an odd prime which does not divide a, then this formula remains valid over GF(q). Thus, the question of whether or not a

quadratic equation over GF(q) has two roots in GF(q) depends on whether or not its discriminant, $D = b^2 - 4ac$, is a perfect square in GF(q). In particular, we see that the original quadratic equation has an even number (two) of irreducible factors over GF(q) if its discriminant is a square, but it has an odd number (one) of irreducible factors over GF(q) if its discriminant is a nonsquare. This is the simplest example of a general result called *Stickelberger's theorem.* In order to state the general result, we must define the discriminant of a polynomial of arbitrary degree.

Definition 6.61 The *discriminant* D of a polynomial $f(x) = a \prod_{i=1}^{n} (x - \alpha_i)$ is defined by

$$D(f) = a^{2n-2} \prod_{1 \le i < j \le n} \prod (\alpha_i - \alpha_j)^2$$

We shall see that the discriminant is a special case of a function of two polynomials, called the *resultant,* which is defined as follows.

Definition 6.62† The *resultant* of the two polynomials

$$f(x) = a \prod_{i=1}^{n} (x - \alpha_i) \qquad g(x) = b \prod_{j=1}^{m} (x - \beta_j)$$

is defined by

$$R(f,g) = a^m b^n \prod_{i=1}^{n} \prod_{j=1}^{m} (\alpha_i - \beta_j)$$

The following properties of the resultant are immediate. The proof of (6.622) follows directly from (6.627).

6.621. $R(g,f) = (-1)^{mn} R(f,g)$.

6.622. If $g(x) = f(x)q(x) + r(x)$, then $R(f,g) = a^{m - \deg r} R(f,r)$.

6.623. If $f(x)$ and $g(x)$ have any common factor of positive degree, $R(f,g) = 0$.

6.624. If a and b are constants not both 0, $R(a,b) = 1$.

6.625. If $f(x) = f^{(1)}(x) f^{(2)}(x)$, then $R(f,g) = R(f^{(1)},g) R(f^{(2)},g)$.

† This definition follows van der Waerden (1931). The reader is warned that certain other authors, including Swan, define the resultant as $R_{\text{swan}}(f,g) = R_{\text{vdw}}(g,f)$. The reader may also notice that the factor $(-1)^{n(n-1)/2}$ should be included in van der Waerden's eq. (5), sec. 28.

6.626. If $g(x) = g^{(1)}(x)g^{(2)}(x)$, then $R(f,g) = R(f,g^{(1)})R(f,g^{(2)})$.

6.627. $R(f,g) = a^m \prod_{i=1}^{n} g(\alpha_i) = (-1)^{mn} b^n \prod_{j=1}^{m} f(\beta_j)$.

Using Properties 6.621 to 6.624, we can compute $R(f,g)$ by applying Euclid's algorithm to $f(x)$ and $g(x)$. This shows that the resultant of $f(x)$ and $g(x)$ can be expressed in terms of products, sums, and differences and quotients of their coefficients. In particular, if $f(x)$ and $g(x)$ are polynomials over a field F, then $R(f,g) \in F$,† even if the roots of $f(x)$ and $g(x)$ are not in F.

Theorem 6.63 gives an expression for the discriminant of a *monic* polynomial $f(x)$ of degree n.

Theorem 6.63

$$D(f) = (-1)^{n(n-1)/2} R(f,f') = (-1)^{n(n-1)/2} R(f',f)$$

Proof According to Property 6.627,

$$R(f,f') = \prod_{k=1}^{n} f'(\alpha_k)$$

with

$$f(x) = \prod_{i=1}^{n} (x - \alpha_i) \qquad f'(x) = \sum_j \prod_{i \neq j} (x - \alpha_i)$$

$$f'(\alpha_k) = \sum_j \prod_{i \neq j} (\alpha_k - \alpha_i) = \prod_{i \neq k} (\alpha_k - \alpha_i)$$

$$R(f,f') = \prod_{i \neq k} \prod_{k} (\alpha_k - \alpha_i)$$

$$= \Big[\prod_{1 \leq i < k \leq n} \prod (\alpha_k - \alpha_i) \Big] \Big[\prod_{1 \leq k < i \leq n} \prod (\alpha_k - \alpha_i) \Big]$$

$$= (-1)^{n(n-1)/2} D(f) \qquad \text{Q.E.D.}$$

Using this relation between the discriminant and the resultant, we can derive a formula for the discriminant of a product.

† More generally, if the coefficients of $f(x)$ and $g(x)$ lie in an integral domain D, then $R(f,g)$ is also in D. The proof requires a determinant which I have avoided because its evaluation entails much unnecessary computation.

Theorem 6.64 If $g(x)$ is a monic polynomial of degree i and $h(x)$ is a monic polynomial of degree j, then

$$D(gh) = D(g)D(h)[R(h,g)]^2$$

Proof

$$\begin{aligned}(-1)^{(i+j)(i+j-1)/2}D(gh) &= R(gh,(gh)') \\ &= R(gh,\, g'h + gh') = R(g,\, g'h + gh')R(h,\, g'h + gh') \\ &= R(g,g'h)R(h,gh') \\ &= R(g,g')R(g,h)R(h,g)R(h,h') \\ &= (-1)^{i(i-1)/2}D(g)R(g,h)(-1)^{ij}R(g,h)(-1)^{j(j-1)/2}D(h)\end{aligned}$$

The theorem then follows from the observation that

$$\frac{(i+j)(i+j-1)}{2} = \frac{i(i-1)}{2} + ij + \frac{j(j-1)}{2} \qquad \text{Q.E.D.}$$

By repeated applications of this result, we can compute the discriminant of a product of k monic polynomials, $f^{(1)}(x)$, $f^{(2)}(x)$, . . . , $f^{(k)}(x)$. We have

$$\begin{aligned}D\left(\prod_{i=1}^{j} f^{(i)}\right) &= D\left(\prod_{i=1}^{j-1} f^{(i)}\right) D(f^{(j)}) \left[R\left(\prod_{i=1}^{j-1} f^{(i)}, f^{(j)}\right)\right]^2 \\ &= D\left(\prod_{i=1}^{j-1} f^{(i)}\right) D(f^{(j)}) \prod_{i=1}^{j-1} R(f^{(i)}, f^{(j)})^2\end{aligned}$$

Theorem 6.65

$$D\left(\prod_{i=1}^{k} f^{(i)}\right) = \left[\prod_{i=1}^{k} D(f^{(i)})\right]\left[\prod_{1\le i<j\le k} R(f^{(i)}, f^{(j)})\right]^2$$

We can now compute the discriminant of several polynomials directly. For example, consider the quadratic $ax^2 + bx + c$. We have

$$\begin{aligned}D(ax^2 + bx + c) &= a^2 D\left(x^2 + \frac{b}{a}x + \frac{c}{a}\right) \\ &= -a^2 R\left(2x + \frac{b}{a},\, x^2 + \frac{b}{a}x + \frac{c}{a}\right) = -4a^2\left(\frac{-b^2}{4a^2} + \frac{c}{a}\right) = b^2 - 4ac\end{aligned}$$

The quadratic is a special case of the general trinomial, $x^n + ax^k + b$, whose discriminant has been computed by Swan (1962).

Swan's lemma 6.66 The resultant of two binomials is given by

$$R(x^j - \xi^j, x^k - \nu^k) = (-1)^j(\nu^m - \xi^m)^d$$

where $d = \gcd(j,k)$ and $m = jk/d$.

Proof If α is a primitive jth root of unity then

$$x^j - \xi^j = \prod_{i=1}^{j} (x - \xi\alpha^i)$$

$$R(x^j - \xi^j, x^k - \nu^k) = \prod_{i=1}^{j} [(\xi\alpha^i)^k - \nu^k]$$

$$= (-1)^j\xi^{md} \prod_{i=1}^{j} \left[\left(\frac{\nu}{\xi}\right)^k - \alpha^{ki}\right]$$

Since α^k has order j/d,

$$\prod_{i=1}^{j/d} (x - \alpha^{ki}) = x^{j/d} - 1$$

and

$$R(x^j - \xi^j, x^k - \nu^k) = (-1)^j\xi^{md}\left[\left(\frac{\nu}{\xi}\right)^{jk/d} - 1\right]^d$$

$$= (-1)^j(\nu^m - \xi^m)^d \qquad \text{Q.E.D.}$$

Swan's theorem 6.67 Let $n > k > 0$. Let $d = \gcd(n,k)$; $n = Nd$; $k = Kd$. Then the discriminant of the trinomial $x^n + ax^k + b$ is given by

$$D(x^n + ax^k + b)$$
$$= (-1)^{n(n-1)/2}b^{k-1}(n^Nb^{N-K} - (-1)^N(n - k)^{N-K}k^Ka^N)^d$$

Proof With $f(x) = x^n + ax^k + b$, we have

$$D(f(x)) = (-1)^{n(n-1)/2}R(nx^{n-1} + akx^{k-1}, f(x))$$

$$R(nx^{n-1} + akx^{k-1}, f(x)) = R(x^{k-1},f(x))R(n,f(x))$$
$$R(x^{n-k} + n^{-1}ak, f(x))$$

$$R(x^{k-1},f(x)) = (f(0))^{k-1} = b^{k-1}$$

$$R(n,f(x)) = n^n = n^{Nd}$$

$$R(x^{n-k} + n^{-1}ak, f(x)) = R(x^{n-k} + n^{-1}ak, x^k(x^{n-k} + n^{-1}ak)$$
$$+ a(1 - n^{-1}k)x^k + b)$$
$$= R(x^{n-k} + n^{-1}ak, a(1 - n^{-1}k)x^k + b)$$
$$= (a(1 - n^{-1}k))^{n-k}R(x^{n-k} - \xi^{n-k}, x^k - \nu^k)$$

where we have set

$$\xi = (-n^{-1}ak)^{1/(n-k)} \qquad \nu = (ba^{-1}(n^{-1}k - 1)^{-1})^{1/k}$$

The theorem then follows directly from Swan's lemma 6.66, with $j = n - k$, $m = (N - K)Kd$, $\nu^m = (ba^{-1}(n^{-1}k - 1)^{-1})^{N-K}$, and $\xi^m = (-n^{-1}ak)^K$. Q.E.D.

We now show how the discriminant of $f(x)$ determines whether $f(x)$ has an odd number of irreducible factors or an even number of irreducible factors.

Stickelberger's theorem 6.68 (odd characteristic) Let q be a power of an odd prime, let $f(x)$ be a monic polynomial of degree m over GF(q), with discriminant $D(f) \neq 0$. Let r be the number of irreducible factors of $f(x)$ over GF(q). Then $r \equiv m \bmod 2$ iff $D(f)$ is a square in GF(q).

Proof We first consider the case when $r = 1$; that is, $f(x)$ is irreducible over GF(q). If α is any particular root of $f(x)$, then $\alpha \in \mathrm{GF}(q^m)$, and the other roots of $f(x)$ are q-conjugates of α. Therefore, $\sqrt{D} \in \mathrm{GF}(q^m)$, for in GF($q^m$) we have

$$\sqrt{D} = \prod_{0 \le i < j \le m-1} \prod (\alpha^{q^i} - \alpha^{q^j})$$

$$\sqrt{D}^q = \prod_{1 \le i < j \le m} \prod (\alpha^{q^i} - \alpha^{q^j})$$

$$= \prod_{1 \le i < j \le m-1} \prod (\alpha^{q^i} - \alpha^{q^j}) \prod_{\substack{k, \\ 1 \le k \le m-1}} (\alpha^{q^k} - \alpha^{q^m})$$

$$= (-1)^{m-1} \sqrt{D}$$

If m is odd, then $\sqrt{D}^q = \sqrt{D}$, so $\sqrt{D} \in \mathrm{GF}(q)$. If m is even, then $\sqrt{D}^q \neq \sqrt{D}$, so $D \notin \mathrm{GF}(q)$. This proves the theorem when $r = 1$.

Next suppose $f(x) = \prod_{i=1}^{r} f^{(i)}(x)$, where each $f^{(i)}(x)$ is an irreducible polynomial of degree d_i over GF(q). Then $D = R^2 \prod_{i=1}^{r} D_i$,

where $D_i = D(f^{(i)})$ and $R = \prod_{1 \le i < j \le m} \prod R(f^{(i)}, f^{(j)})$. We then have

$$\sqrt{D} = R \prod_{i=1}^{r} \sqrt{D_i}$$

$$(\sqrt{D})^q = R^q \prod_{i=1}^{r} \sqrt{D_i{}^q}$$

Since $R \in \mathrm{GF}(q)$, $R^q = R$; since $f^{(i)}(x)$ is an irreducible polynomial of degree d_i, this theorem applies, and $\sqrt{D_i{}^q} = (-1)^{d_i - 1} \sqrt{D_i}$. Thus,

$$\sqrt{D^q} = R \prod_{i=1}^{r} (-1)^{d_i - 1} \sqrt{D_i}$$

Since

$$\sum_{i=1}^{r} d_i = m \qquad \sum_{i=1}^{r} (d_i - 1) = m - r$$

and

$$\sqrt{D^q} = R(-1)^{m-r} \prod_{i=1}^{r} \sqrt{D_i}$$

$$= (-1)^{m-r} \sqrt{D}$$

Therefore $\sqrt{D} \in \mathrm{GF}(q)$ iff $m \equiv r \bmod 2$. Q.E.D.

According to Stickelberger's theorem, $D(f(x))$ gives the parity of the number of irreducible factors of $f(x)$ if all irreducible factors of $f(x)$ are distinct. According to Theorems 6.63, 6.623, and 3.23, $D(f(x)) = 0$ iff $f(x)$ has a repeated irreducible factor. These properties of the discriminant of a polynomial are analogous to the properties of the Moebius function of a number, as defined in Theorem 3.41. If a polynomial (number) has any repeated irreducible (prime) factors, its discriminant (Moebius function) vanishes; if all irreducible (prime) factors of a polynomial (number) are distinct, then its discriminant (Moebius function) determines the parity of the number of its irreducible (prime) factors.

Stickelberger's theorem 6.68 expresses the parity (mod 2) of the number of irreducible factors of a polynomial over a field of odd characteristic in terms of the discriminant of the polynomial. In fields of characteristic 2, the discriminant proves insufficient to determine the parity of the number of irreducible factors. However, the number of irreducible factors may be determined from a different symmetric function of the roots, which we now introduce.

Definition 6.689 If

$$f(x) = f_m \prod_{i=1}^{m} (x - \alpha_i)$$

then

$$\rho(f) = \sum_{1 \le i < j \le m} \sum \left[\frac{\alpha_i \alpha_j}{(\alpha_i + \alpha_j)^2} \right]$$

Theorem 6.69 If $f(x)$ is a polynomial of degree m which is the product of r distinct irreducible polynomials over GF(2^n), then

$r \equiv m \bmod 2$ iff $\mathrm{Tr}\,[\rho(f)] = 0$

where

$$\mathrm{Tr}\,(x) = \sum_{i=0}^{n-1} x^{2^i}$$

Remark We postpone the problem of calculating Tr (ρ) from the coefficients of $f(x)$ to Theorem 6.695.

The proof of Theorem 6.69 is based on three lemmas.

Lemma 6.691

$$\mathrm{Tr}\,[\rho(f)] = \sum_{1 \le i < j \le m} \sum \left[\frac{1}{1 + \alpha_j/\alpha_i} + \frac{1}{1 + (\alpha_j/\alpha_i)^q} \right] \qquad q = 2^n$$

Proof In a field of characteristic 2,

$$\frac{\alpha_i \alpha_j}{(\alpha_i + \alpha_j)^2} = \frac{\alpha_j/\alpha_i}{1 + (\alpha_j/\alpha_i)^2} = \frac{1}{1 + \alpha_j/\alpha_i} + \frac{1}{1 + (\alpha_j/\alpha_i)^2}$$

$$\mathrm{Tr}\,(\rho) = \sum_{k=0}^{n-1} \sum_{1 \le i < j \le m} \sum \left[\frac{1}{1 + \alpha_j/\alpha_i} + \frac{1}{1 + (\alpha_j/\alpha_i)^2} \right]^{2^k}$$

$$= \sum_{1 \le i < j \le m} \sum \sum_{k=0}^{n-1} \left[\frac{1}{1 + (\alpha_j/\alpha_i)^{2^k}} + \frac{1}{1 + (\alpha_j/\alpha_i)^{2^{k+1}}} \right]$$

$$= \sum_{1 \le i < j \le m} \sum \left[\frac{1}{1 + \alpha_j/\alpha_i} + \frac{1}{1 + (\alpha_j/\alpha_i)^q} \right] \qquad \text{Q.E.D.}$$

Lemma 6.692 Theorem 6.69 is true if $r = 1$.

Proof If $r = 1$, $f(x)$ is irreducible and $\alpha_i = \alpha^{q^i}$, so

$$\text{Tr}\,(\rho) = \sum_{1\le i<j\le m}\sum \left(\frac{1}{1+\alpha^{q^i-q^i}} + \frac{1}{1+\alpha^{q(q^i-q^i)}}\right)$$

With $J = j - i$, this becomes

$$\begin{aligned}\text{Tr}\,(\rho) &= \sum_{J=1}^{m-1}\sum_{i=1}^{m-J}\left(\frac{1}{1+\alpha^{q^i(q^J-1)}} + \frac{1}{1+\alpha^{q^{i+1}(q^J-1)}}\right)\\ &= \sum_{J=1}^{m-1}\left(\frac{1}{1+\alpha^{q(q^J-1)}} + \frac{1}{1+\alpha^{q^{m+1-J}(q^J-1)}}\right)\end{aligned}$$

Setting $\beta = \alpha^q$, $I = m - J$ gives

$$\begin{aligned}\text{Tr}\,(\rho) &= \sum_{J=1}^{m-1}\frac{1}{1+\beta^{(q^J-1)}} + \sum_{I=1}^{m-1}\frac{1}{1+\beta^{q^m-q^I}}\\ &= \sum_{J=1}^{m-1}\frac{\beta}{\beta+\beta^{q^J}} + \sum_{I=1}^{m-1}\frac{\beta^{q^I}}{\beta^{q^m}+\beta^{q^I}}\end{aligned}$$

Since $\beta^{q^m} = \beta$, we have

$$\text{Tr}\,(\rho) = \sum_{J=1}^{m-1} 1 = m - 1$$

in a field of characteristic 2. Therefore Theorem 6.69 holds if $r = 1$. Q.E.D.

Lemma 6.693 If $f(x)$ and $g(x)$ are relatively prime polynomials over GF(2^n), then Tr $[\rho(fg)]$ = Tr $[\rho(f)]$ + Tr $[\rho(g)]$.

Proof Let $\alpha_1, \alpha_2, \alpha_3, \ldots, \alpha_m$ be the roots of $f(x)$ and let $\alpha_{m+1}, \alpha_{m+2}, \ldots, \alpha_{m+k}$ be the roots of $g(x)$. Then, setting

$$\sum_{1\le i<j\le m+k}\sum = \sum_{1\le i<j\le m}\sum + \sum_{m+1\le i<j\le m+k}\sum + \sum_{i=1}^{m}\sum_{j=m+1}^{m+k}$$

in Lemma 6.691 gives

$$\begin{aligned}\text{Tr}\,[\rho(fg)] = \text{Tr}\,[\rho(f)] + \text{Tr}\,[\rho(g)]\\ + \sum_{i=1}^{m}\sum_{j=m+1}^{m+k}\left[\frac{1}{1+\alpha_j/\alpha_i} + \frac{1}{1+(\alpha_j/\alpha_i)^q}\right]\end{aligned}$$

Therefore, Lemma 6.693 is equivalent to the assertion that

$$\sum_{i=1}^{m} \sum_{j=m+1}^{k} \frac{1}{1 + \alpha_j/\alpha_i} = \sum_{i=1}^{m} \sum_{j=m+1}^{k} \frac{1}{1 + (\alpha_j/\alpha_i)^q} \tag{6.694}$$

We now prove Eq. (6.694) by showing that both sides are equal to

$$\sum_{i=1}^{m} \sum_{j=m+1}^{k} \frac{1}{1 + \alpha_j{}^q/\alpha_i}$$

Since $\alpha_{m+1}, \alpha_{m+2}, \ldots, \alpha_{m+k}$ are a complete set of conjugates, $\alpha_{m+1}^q, \alpha_{m+2}^q, \ldots, \alpha_{m+k}^q$ are a permutation of $\alpha_{m+1}, \alpha_{m+2}, \ldots, \alpha_{m+k}$. Therefore, for every i,

$$\sum_{j=m+1}^{k} \frac{1}{1 + \alpha_j{}^q/\alpha_i} = \sum_{j=m+1}^{k} \frac{1}{1 + \alpha_j/\alpha_i}$$

Similarly, $\alpha_1, \alpha_2, \ldots, \alpha_m$ are a permutation of $\alpha_1{}^q, \alpha_2{}^q, \ldots, \alpha_m{}^q$, so for every j,

$$\sum_{i=1}^{m} \frac{1}{1 + \alpha_j{}^q/\alpha_i{}^q} = \sum_{i=1}^{m} \frac{1}{1 + \alpha_j{}^q/\alpha_i}$$

This proves Eq. (6.694) and Lemma 6.693. Q.E.D.

Proof of Theorem 6.69 Let $f(x) = \prod_{i=1}^{r} f^{(i)}(x)$, where each $f^{(i)}(x)$ is irreducible over GF(2^n). Then in GF(2)

$$\begin{aligned} \mathrm{Tr}\,[\rho(f)] &= \sum_{i=1}^{r} \mathrm{Tr}\,[\rho(f^{(i)})] \qquad \text{by Lemma 6.693} \\ &= \sum_{i=1}^{r} (\deg f^{(i)} - 1) \qquad \text{by Lemma 6.692} \\ &= \deg f - r \qquad \text{Q.E.D.} \end{aligned}$$

Theorem 6.69 enables us to determine the parity of the number of irreducible factors of $f(x)$ from Tr $[\rho(f)]$. Definition 6.689 defines ρ in terms of the roots of $f(x)$, which are often unknown. In practice, we can apply Theorem 6.69 only after we have expressed Tr $[\rho(f)]$ in terms of the coefficients of the polynomial $f(x)$. Theorem 6.695 furnishes one such expression.

Theorem 6.695 If $D(f) \equiv [\eta(f)]^2 - 4\xi(f) \bmod 8$, where $D(f)$ is the discriminant of the polynomial $f(x)$, and D, η, and ξ are represented as polynomials over the rational integers in the variables $f_0, f_1,$

. . . , f_m, then, in GF(2^n),

$$\mathrm{Tr}\,[\rho(f)] = \mathrm{Tr}\left\{\frac{\xi(f)}{[\eta(f)]^2}\right\}$$

EXAMPLES The discriminant of the general quadratic, $f_0 + f_1x + f_2x^2$, is $D(f) = f_1^2 - 4f_0f_2$. With $\eta(f) = f_1$, $\xi(f) = f_0f_2$, Theorems 6.69 and 6.695 assert that the general quadratic over GF(2^n) has an even number of irreducible factors over GF(2^n) iff Tr $(f_0f_2/f_1^2) = 0$.

The discriminant of the general cubic, $f_0 + f_1x + f_2x^2 + f_3x^3$, is $D(f) = f_1^2f_2^2 - 27f_0^2f_3^2 - 4f_0f_2^3 - 4f_1^3f_3 + 18f_0f_1f_2f_3 \equiv (f_1f_2 + f_0f_3)^2 - 4(f_0^2f_3^2 + f_0f_2^3 + f_1^3f_3)$ mod 8.

Theorems 6.69 and 6.695 assert that the general cubic over GF(2^n) has an odd number of irreducible factors over GF(2^n) iff

$$\mathrm{Tr}\left[\frac{f_0^2f_3^2 + f_0f_2^3 + f_1^3f_3}{(f_1f_2 + f_0f_3)^2}\right] = 0$$

Proof of Theorem 6.695 Without loss of generality, we may assume that $f(x)$ is monic. Consider the function

$$\delta(f) = \prod_{1\le i<j\le m}\prod (\alpha_i + \alpha_j)$$

Since δ is a symmetric polynomial in the roots of $f(x)$, $\delta(f)$ may also be expressed as a polynomial in $f_0, f_1, \ldots, f_{m-1}$. The coefficients of $f_0, f_1, \ldots, f_{m-1}$ in this representation of $\delta(f)$ will be rational integers, so the identity which expresses $\prod_{1\le i<j\le m}\prod (\alpha_i + \alpha_j)$ as a polynomial in $f_0, f_1, \ldots, f_{m-1}$ is valid in all fields. This is a consequence of a well-known theorem on symmetric polynomials [van der Waerden (1931, sec. 26)].

Similarly, $[\delta(f)]^2\rho(f)$, which is also a symmetric polynomial in the roots of $f(x)$, may likewise be expressed as a polynomial in $f_0, f_1, \ldots, f_{m-1}$ over the rational integers. Since

$$(\alpha_i - \alpha_j)^2 = (\alpha_i + \alpha_j)^2 - 4\alpha_i\alpha_j$$

$$\begin{aligned} D(f) &= [\delta(f)]^2 \prod_{1\le i<j\le m}\prod \left[1 - \frac{4\alpha_i\alpha_j}{(\alpha_i + \alpha_j)^2}\right] \\ &= [\delta(f)]^2 - [\delta(f)]^2 \sum_{i=1}^{m-1}\sum_{j=i+1}^{m} \frac{4\alpha_i\alpha_j}{(\alpha_i + \alpha_j)^2} \\ &\quad + [\delta(f)]^2 \sum_{i=1}^{m-1}\sum_{j=i+1}^{m}\sum_{I=i}^{m-1}\sum_{\substack{J=I+1,\\ J>j \text{ if } I=i}}^{m} \frac{16\alpha_i\alpha_j\alpha_I\alpha_J}{(\alpha_i + \alpha_j)^2(\alpha_I + \alpha_J)^2} \\ &\quad - 64 \cdots \\ &\equiv [\delta(f)]^2 - 4[\delta(f)]^2\rho(f) \bmod 8 \end{aligned}$$

By hypothesis,

$$D(f) \equiv [\eta(f)]^2 - 4\xi(f) \bmod 8$$

since

$$[\eta(f)]^2 \equiv [\delta(f)]^2 \bmod 4$$
$$\eta(f) \equiv \delta(f) \bmod 2$$

so

$$\eta(f) = \delta(f) + 2\gamma(f)$$
$$[\eta(f)]^2 = [\delta(f)]^2 + 4\delta(f)\gamma(f) + 4[\gamma(f)]^2$$
$$\xi(f) \equiv [\delta(f)]^2\rho(f) + \delta(f)\gamma(f) + [\gamma(f)]^2 \bmod 2$$

Therefore, in GF(2^n),

$$\frac{\xi(f)}{[\eta(f)]^2} = \rho(f) + \frac{\gamma(f)}{\delta(f)} + \left[\frac{\gamma(f)}{\delta(f)}\right]^2$$

and since Tr (x^2) = Tr (x),

$$\text{Tr}\left[\frac{\xi(f)}{[\eta(f)]^2}\right] = \text{Tr}\,[\rho(f)] \qquad \text{Q.E.D.}$$

In general, if $f(x)$ is a polynomial over GF(2^n), $n > 1$, then we can determine the parity of the number of irreducible factors of $f(x)$ only by separately computing $\eta(f)$ and $\xi(f)$ in GF(2^n) and then taking the trace of ξ/η^2. However, if $f(x)$ is a polynomial over GF(2), a shortcut is possible. We may treat $f(x)$ as a polynomial over the rational integers and compute its discriminant mod 8.† If $D \equiv 1 \bmod 8$, then $r \equiv m \bmod 2$ [because $\eta = 1$, $\xi = 0$, and Tr $(\xi/\eta^2) = 0$]; if $D \equiv 5 \bmod 8$, then $r \not\equiv m \bmod 2$ [because $\eta = 1$, $\xi = 1$, and Tr $(\xi/\eta^2) = 1$]. If D is not congruent to 1 or 5 mod 8, then D is even, and $f(x)$ has a repeated factor over GF(2).

Combining the binary version of Stickelberger's theorem with Swan's theorem gives Swan's corollary.

Swan's corollary 6.696 Let $n > k > 0$. Assume exactly one of n,k odd. (If n odd, k odd, then substitute k for $n - k$.) Then $x^n + x^k + 1$ has an even number of irreducible factors over GF(2) in the following cases and only these cases:

1. n even, k odd, $n \not\equiv 2k$, $nk/2 \equiv 0$ or $1 \bmod 4$
2. n odd, k even, $k \not| 2n$, $n \equiv \pm 3 \bmod 8$
3. n odd, k even, $k | 2n$, $n \equiv \pm 1 \bmod 8$

† The binary version of Stickelberger's theorem has been proved by Carlitz (1953), Dalen (1955), and Swan (1962), but the generalization to GF(2^n) given by Theorems 6.69 and 6.695 is apparently new. The special cases of deg $f = 2$ or 3, n arbitrary, appear in Berlekamp, Rumsey, and Solomon (1967).

Proof From Swan's theorem 6.67,

$$D = (-1)^{n(n-1)/2}[n^N - (-1)^N(n-k)^{N-K}k^K]^d$$

By hypothesis, D, d, $n-k$, and $N-K$ are all odd, so mod 8 we have

$$D \equiv (-1)^{n(n-1)/2}[n^N - (-1)^N(n-k)k^K]$$

The theorem then follows from the evaluation of D mod 8 in each of the following four possible cases:

1. n even, k odd, $N > 2$
2. n odd, k even, $K > 2$
3. n odd, k even, $K = 2$
4. n even, k odd, $N = 2$

Since the fourth case implies $D \equiv 5$, it may be discarded. Q.E.D.

★ 6.7 QUADRATIC RECIPROCITY

Given an integer a and a prime p, the *Legendre symbol* (a/p) is defined by Definition 6.71.

Definition 6.71

$$(a/p) = \begin{cases} 1 & \text{if } x^2 = a \text{ has nonzero solutions in GF}(p) \\ -1 & \text{if } x^2 = a \text{ does not have solutions in GF}(p) \\ 0 & \text{if } p \text{ divides } a \end{cases}$$

The Legendre symbol arises in applications of Stickelberger's theorem, in the determination of the multiplicative order of a mod p, in the determination of the degrees of the irreducible factors of $Q^{(a)}(x)$ over GF(p), in the study of quadratic-residue codes, which will be defined in Sec. 15.2, and in many other places in number theory. If the equation $x^2 = a$ has nonzero solutions in GF(p), a is said to be a *quadratic residue* mod p; if the equation $x^2 = a$ has no solution in GF(p), a is said to be a *quadratic nonresidue.*

If a is a quadratic nonresidue, then $x^2 - a$ is irreducible over GF(p), and $\sqrt{a} \in \text{GF}(p^2)$; if a is a quadratic residue, then $x^2 - a$ factors over GF(p), and $\sqrt{a} \in \text{GF}(p)$. In either case, $\sqrt{a} \in \text{GF}(p^2)$. The element $\sqrt{a}$ is in the subfield GF(p) iff $\sqrt{a}^p = \sqrt{a}$. Since $(a^{(p-1)/2})^2 = 1$ and the only two roots of the equation $x^2 - 1 = 0$ are $x = \pm 1$, $a^{(p-1)/2} = \pm 1$ mod p. Therefore,

$$(a/p) \equiv a^{(p-1)/2} \bmod p$$

This congruence is called *Euler's criterion.*

Lemmas 6.721 to 6.723 follow immediately.

Lemma 6.721 $(ab/p) = (a/p)(b/p)$.

Lemma 6.722 $(-1/p) = (-1)^{(p-1)/2}$.

Lemma 6.723 $(a^2/p) = 1$.

Lemma 6.724

$$(2/p) = \begin{cases} 1 & \text{if } p \equiv \pm 1 \bmod 8 \\ -1 & \text{if } p \equiv \pm 3 \bmod 8 \end{cases}$$

The proof of Lemma 6.724 follows directly from Theorem 6.54.

Let us now consider the factorization of the polynomial $f(x) = x^n - 1$ over GF(p), where n and p are distinct odd primes. We know that $x^n - 1 = (x - 1)Q^{(n)}(x)$ and that every irreducible factor of $Q^{(n)}(x)$ over GF(p) has degree m, which is the multiplicative order of p mod n. Thus, the number of factors of $x^n - 1$ over GF(p) is $r = 1 + (n - 1)/m$. By definition of the Legendre symbol, m divides $(n - 1)/2$ iff $(p/n) = 1$. Therefore, $(-1)^{(n-1)/m} = (p/n)$, and $(-1)^r = -(p/n)$, where r is the number of irreducible factors of $x^n - 1$ over GF(p).

The discriminant of the polynomial $f(x) = x^n - 1$ is given by $D(f) = (-1)^{n(n-1)/2}R(nx^{n-1},x^n - 1) = n^n(-1)^{n(n-1)/2}$. Stickelberger's theorem 6.68 states that if $f(x)$ is a polynomial of degree n which is the product of r irreducible factors over GF(p), then $n \equiv r \bmod 2$ iff $(D/p) = 1$, where D is the discriminant of $f(x)$. In other words, $(D/p) = (-1)^{n-r}$. With $f(x) = x^n - 1$, n an odd prime, we get

$$(n^n(-1)^{n(n-1)/2}/p) = (-1)^n(-1)^r = (-1)^n(-(p/n)) = (p/n)$$

This is a theorem which relates the quadratic residue (p/n) to the quadratic residue (n/p). Noticing that $(-1)^{n(n-1)/2} = (-1)^{(n-1)/2}$, we obtain

$$(p/n) = (n^n(-1)^{(n-1)/2}/p) = (n/p)(n^{n-1}/p)((-1)^{(n-1)/2}/p)$$

Since $n^{n-1} = (n^{(n-1)/2})^2$, $(n^{n-1}/p) = 1$. Since $(-1/p) = (-1)^{(p-1)/2}$, $((-1)^{(n-1)/2}/p) = (-1)^{(n-1)(p-1)/4}$. We then have

$$(p/n) = (n/p)(-1)^{(n-1)(p-1)/4}$$

This is equivalent to Theorem 6.73.

Gauss' quadratic-reciprocity theorem 6.73 If n and p are distinct odd primes,

$$(p/n)(n/p) = (-1)^{(n-1)(p-1)/4}$$

This result, due to Gauss, is called the *law of quadratic reciprocity.* The proof used here is due to Swan (1962).

As a practical tool, the law of quadratic reciprocity is very helpful for computations with Legendre symbols.

Example 6.74 What are the degrees of the irreducible factors of $Q^{(257)}$ over GF(19)?

Solution The problem is to compute the multiplicative order of 19 mod 257, which amounts to computing 19^{128} or (19/257). Using quadratic reciprocity, we have

$$\begin{aligned}(19/257) &= (257/19) = (10/19) = (2/19)(5/19) = (2/19)(19/5)\\ &= (2/19)(4/5) = (2/19)\end{aligned}$$

Since $19 \equiv 3 \bmod 8$, $(2/19) = -1$, and

$$19^{128} \equiv -1 \bmod 257$$

Since the order of 19 is a divisor of 256 but not of 128, it must be 256. Therefore, $Q^{(257)}(x)$ is irreducible over GF(19).

Further exercises on quadratic reciprocity may be found in books on number theory, including those by McCoy (1965) and Nagell (1951).

PROBLEMS

6.1. Factor the following polynomials into irreducible factors over GF(2):

(*a*) $x^{16} + x^{11} + x^6 + x^5 + x^4 + x + 1$

(*b*) $x^{16} + x^3 + 1$

(*c*) $x^{23} + 1$

(*d*) $x^{41} + 1$

6.2. Determine the periods of the polynomials in Prob. 6.1.

6.3. The prime p is said to be a *primitive* factor of $2^m - 1$ iff the order of 2 mod p is m; the prime p is said to be an *algebraic* factor of $2^m - 1$ iff the order of 2 mod p is a proper divisor of m.

(*a*) Prove that if p is a primitive factor of $2^m - 1$, then $p \equiv 1 \bmod m$. If m odd, $p \equiv \pm 1 \bmod 8$; if $m \equiv \pm 2 \bmod 8$, $p \equiv 1$ or $3 \bmod 8$; if $m \equiv 0 \bmod 8$, $p \equiv 1 \bmod 2m$.

(*b*) Factor $2^{11} - 1$ and $(2^{25} - 1)/(2^5 - 1) = 1{,}082{,}401$.

(*c*) Prove that if $m \equiv \pm 2 \bmod 8$, then the product of all primitive prime factors of $2^m - 1$ is $\equiv 3 \bmod 8$. *Hint:* Use induction on $Q^{(m)}(2)$.

(*d*) Using Cunningham's tables, find the first counterexample to the following pseudotheorem: if $m \equiv \pm 2 \bmod 8$, and if p is a primitive prime factor of $2^m - 1$, then $p \equiv 3 \bmod 8$.

6.4. Prove or disprove:

(*a*) If p is a prime $\equiv 3 \bmod 8$, then the multiplicative order of 2 mod p is $p - 1$.

(*b*) If p is a prime $\equiv -3 \bmod 8$, then the multiplicative order of 2 mod p is $p - 1$.

6.5. A number b is said to be *primitive mod n* iff the multiplicative order of b mod n is $\varphi(n)$. Prove or disprove:

(*a*) If 2 is primitive mod n, then $Q^{(n)}(x)$ is irreducible over GF(2).

(*b*) If $Q^{(n)}(x)$ is irreducible over GF(2), then n is a power of a prime p, $p \equiv \pm 3 \bmod 8$, either $p|n$ or $p = n + 1$, and 2 is primitive mod p.

(*c*) $Q^{(n)}(x)$ is irreducible over GF(q) iff all of the following are true:

(1) $n = p^e$ or $2p^e$, where p is an odd prime.

(2) $(p,q) = 1$.

(3) Order of q mod p is $p - 1$.

(4) Either $e = 1$ or order of q mod $p^2 = p$(order of q mod p).

6.6. Prove or disprove:

(*a*) For given degree m, there exists at most one cyclotomic polynomial of degree m which is irreducible over GF(2).

(*b*) For what values of $m \leq 30$ do there exist irreducible cyclotomic binary polynomials of degree n, and what are their orders? Start:

m	n
1	1
2	3
3	—
4	5
5	—
6	9
.	
.	
.	?

6.7. For what values of the integers j and k are the following polynomials irreducible over GF(2)?

(*a*) $x^{2 \cdot 3^k} + x^{3^k} + 1$

(*b*) $x^{4 \cdot 5^k \cdot 3^j} + x^{5^k \cdot 3^j} + 1$

(*c*) $x^{4 \cdot 5^k \cdot 3^j} + x^{3 \cdot 5^k \cdot 3^j} + x^{2 \cdot 5^k \cdot 3^j} + x^{5^k \cdot 3^j} + 1$

(*d*) $x^{5 \cdot 31^k} + x^{2 \cdot 31^k} + 1$

(*e*) $x^{6 \cdot 3^k \cdot 7^j} + x^{4 \cdot 3^k \cdot 7^j} + x^{2 \cdot 3^k \cdot 7^j} + x^{3^k \cdot 7^j} + 1$

What are the periods of these polynomials and their factors?

6.8. Prove or disprove the following:

(*a*) No binary trinomial can have an irreducible factor whose period is 5.

(*b*) If any binary trinomial has an irreducible factor of period n, then some binary trinomial of degree $< n$ has an irreducible factor of period n.

(*c*) If n is a prime ≥ 5 and 2 is primitive mod n, then no binary trinomial can have an irreducible factor whose period is n.

(*d*) Under the same conditions as (*c*), the period cannot be n^k, for any k.

(*e*) If there exists a binary trinomial which has an irreducible factor of period n, and if $(k,n) = 1$, then there exists a binary trinomial which has an irreducible factor of period kn.

(*f*) For $n = 1, 3, 5, 7, 9, \ldots, 41$, determine whether or not there exists any binary trinomial which has an irreducible factor of period n. *Hint:* For $n = 17, 23$, and 41, review Secs. 5.7, 5.6, and 5.7, respectively.

6.9. Using Swan's corollary, prove that all binary trinomials of degree n are reducible if:

(*a*) $n \equiv 0 \bmod 8$.

(*b*) $n \equiv 13$ or $19 \bmod 24$, and every factor of n is $\equiv 1 \bmod 6$.

(*c*) n is a prime > 3, $n \equiv \pm 3 \bmod 8$, $n \equiv 3$ or $-1 \bmod 7$.

7
Binary BCH Codes for Correcting Multiple Errors

In order to construct a binary BCH code which is capable of correcting all patterns of t or fewer channel errors, one first selects an irreducible binary factor of the cyclotomic polynomial $Q^{(n)}$. The degree of the irreducible binary factor is the multiplicative order of 2 mod n, which we denote by m. Let $\alpha \in \mathrm{GF}(2^m)$ be a root of the irreducible binary polynomial. The t-error-correcting binary BCH code of block length n is then the cyclic code whose generator polynomial is the product of the distinct minimal polynomials of $\alpha, \alpha^2, \alpha^3, \alpha^4, \ldots, \alpha^{2t-1}, \alpha^{2t}$. For a *primitive* BCH code, n is restricted to be $2^m - 1$; for a *nonprimitive* BCH code, n may be any other odd number.

7.1 EXAMPLES

We may construct all binary BCH codes of block length 31 from Table 4.4. The codes have these generator polynomials

$$t = 1 \qquad g(x) = M^{(1)}(x) = 1 + x^2 + x^5$$

or in shorthand,

$$g = 101001$$
$$t = 2 \qquad g(x) = M^{(1)}(x)M^{(3)}(x) = 1 + x^3 + x^5 + x^6 + x^8 + x^9 + x^{10}$$

or

$$g = 10010110111$$

$$t = 3 \qquad g(x) = M^{(1)}(x)M^{(3)}(x)M^{(5)}(x)$$
$$g = 1111010111110001$$

$$t = 4 \text{ or } 5 \qquad g(x) = M^{(1)}(x)M^{(3)}(x)M^{(5)}(x)M^{(7)}(x)$$
$$g = 101010110110010001101$$

$$t = 6 \text{ or } 7 \qquad g(x) = M^{(1)}(x)M^{(3)}(x)M^{(5)}(x)M^{(7)}(x)M^{(11)}(x)$$
$$g = 111001000101011111011010011$$

$$t = 8, 9, \ldots, \text{or } 15$$
$$g(x) = M^{(1)}(x)M^{(3)}(x)M^{(5)}(x)M^{(7)}(x)M^{(11)}(x)M^{(15)}(x)$$
$$g = 1111111111111111111111111111111$$

The single-error-correcting code has 5 check digits, 26 message digits, and rate 26/31. The double-error-correcting code has rate 21/31. We have already studied these codes in Secs. 5.1 and 5.2.

Notice that the 4-error-correcting BCH code and the 5-error-correcting BCH code of block length 31 are identical. This is because $M^{(9)}(x)$, the minimal polynomial of α^9, is identical to $M^{(5)}(x)$, the minimal polynomial of α^5. Similarly, the 6-error-correcting code is identical to the 7-error-correcting code. The rate of this code is 6/31.

Notice also that the 8- to 14-error-correcting codes are identical to the 15-error-correcting code. This code has rate 1/31. The only two codewords in this code are the sequence of 31 zeros and the sequence of 31 ones. It is no surprise that this code, which repeats its single information digit 31 times, is able to correct all patterns of up to 15 errors.

The repetition code may be easily decoded by the trivial majority-rule decoding algorithm of Sec. 1.1. As we shall show in Sec. 15.4, a generalization of this algorithm may be used to decode certain other low-rate BCH codes. Unfortunately, however, these special techniques for decoding certain low-rate codes are not very useful for decoding moderate-rate and high-rate BCH codes. For this reason, we first present a general algebraic decoding procedure which is applicable to all binary BCH codes. This algorithm is the best procedure known for decoding BCH codes of medium and high rates. It is also able to decode low-rate BCH codes, although in some cases it is inferior to the algorithms of Sec. 15.4.

7.2 THE KEY EQUATION FOR DECODING BINARY BCH CODES

If the encoder transmits the binary BCH codeword

$$C(x) = \sum_{i=0}^{n-1} C_i x^i$$

and the channel noise causes additive errors given by the coefficients of the binary polynomial

$$E(x) = \sum_{i=0}^{n-1} E_i x^i$$

then the received word will be given by

$$R(x) = \sum_{i=0}^{n-1} R_i x^i = \sum_{i=0}^{n-1} C_i x^i + \sum_{i=0}^{n-1} E_i x^i$$

For $j = 1, 2, \ldots, 2t$, the codeword is a multiple of the minimal polynomial of α^j, and therefore

$$R(\alpha^j) = 0 + \sum_{i=0}^{n-1} E_i \alpha^{ji} = \sum_{k=1}^{e} X_k{}^j = S_j$$

where we let the Galois field error locations $X_1, X_2, \ldots, X_e$ denote the positions where $E_i = 1$. If $R(x)$ is divided by $M^{(j)}(x)$, the minimal polynomial of α^j $(1 \leq j \leq 2t)$, then $S_j = R(\alpha^j)$ may be readily computed from the remainder, $r^{(j)}(\alpha^j)$, as in Sec. 5.2. After the decoder has calculated $S_1, S_2, S_3, \ldots, S_{2t}$, the major problem is to find the error locations, $X_1, X_2, \ldots, X_e$, from the equations

$$\sum_{i=1}^{e} X_i{}^j = S_j \qquad j = 1, 2, 3, \ldots, 2t$$

In general, these equations will have many solutions, each corresponding to a different error pattern in the same coset of the additive group of codewords. The decoder must find a solution with as small a value of e as possible.

In order to solve these equations, the decoder first attempts to find the coefficients of the error-locator polynomial, defined by Definition 7.21 [or Eq. (1.46)].

Definition 7.21

$$\sigma(z) = \prod_{i=1}^{e} (1 - X_i z) = 1 + \sum_{j=1}^{e} \sigma_j z^j$$

Once the decoder has determined the error-locator polynomial $\sigma(z)$, the reciprocal roots of $\sigma(z)$ can be found via a Chien search. The

errors can then be corrected, as in Fig. 5.14. The hardest part of the decoding procedure is to find the σ's from the S's.

To relate the σ's and the S's, we introduce the generating function

$$S(z) = \sum_{j=1}^{\infty} S_j z^j = \sum_{j=1}^{\infty} \sum_{i=1}^{e} X_i^j z^j = \sum_{i=1}^{e} \frac{X_i z}{1 - X_i z}$$

We may eliminate the fractions in this equation by multiplying through by $\sigma(z)$. This gives

$$S(z)\sigma(z) = \sum_{i=1}^{e} \frac{X_i z}{1 - X_i z} \prod_{j=1}^{e} (1 - X_j z) = \sum_{i=1}^{e} X_i z \prod_{j \neq i} (1 - X_j z)$$

Adding $\sigma(z)$ to both sides gives

$$[1 + S(z)]\sigma(z) = \sigma(z) + \sum_{i=1}^{e} X_i z \prod_{j \neq i} (1 - X_j z)$$

We define the polynomial $\omega(z) = \sum_{k=0}^{e} \omega_k z^k$ by the equation

$$\omega(z) = \sigma(z) + \sum_{i=1}^{e} X_i z \prod_{j \neq i} (1 - X_j z) \tag{7.22}$$

We then have the equation

$$[1 + S(z)]\sigma(z) = \omega(z)$$

In general, the decoder knows only the coefficients of the first $2t$ powers of z in $S(z)$; he does not know S_{2t+1}, S_{2t+2}, S_{2t+3}, ... In other words, the decoder does not know $S(z)$, but he does know $S(z)$ mod z^{2t+1}. The relevant equation, which I call the *key equation*, is

$$\boxed{[1 + S(z)]\sigma(z) \equiv \omega(z) \bmod z^{2t+1}} \tag{7.23}$$

Given $S(z)$, we wish to find *both* $\sigma(z)$ and $\omega(z)$ from this equation. The unknown polynomials $\sigma(z)$ and $\omega(z)$ both have degrees $\leq e$, the number of errors that actually occurred.

One "physical interpretation" of the key equation has been suggested by Massey (1968). The key equation may be rewritten as

$$S_k + \sum_{i=1}^{k-1} \sigma_i S_{k-i} + \sigma_k = \omega_k$$

or

$$S_k = \omega_k - \sum_{i=1}^{k-1} \sigma_i S_{k-i} - \sigma_k$$

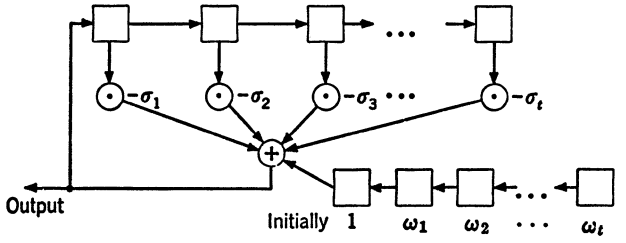


Fig. 7.1 Feedback-shift-register interpretation of the key equation.

This equation gives the kth output of the feedback shift registers of Figs. 7.1 and 7.2, which are wired according to the coefficients of the polynomial $\sigma(z)$ and initially loaded with the coefficients of the polynomial $\omega(z)$. Thus, the key equation may be interpreted as the mathematical formulation of a problem in feedback-shift-register synthesis. Given the output sequence $1 + S(z)$, we wish to determine the connections $\sigma(z)$ and the initial contents $\omega(z)$ of the shortest feedback shift register whose output sequence is $1 + S(z)$.

In our present application, the decoding of binary BCH codes, only $\sigma(z)$ is of any real interest; $\omega(z)$ is just another polynomial of no obvious importance. However, this same key equation arises in several other applications, and we shall encounter it again in Chaps. 9 and 10. In both of these applications of the key equation, as well as in the interpretation as a problem in feedback-shift-register synthesis, the unknown polynomials $\sigma(z)$ and $\omega(z)$ will both have an obvious importance.

Let us now develop an algorithm for solving the key equation over any field.

7.3 HEURISTIC SOLUTION OF THE KEY EQUATION

We wish to solve the key equation

$$(1 + S)\sigma \equiv \omega \bmod z^{2t+1} \tag{7.23}$$

for the polynomials $\sigma(z)$ and $\omega(z)$, given $S(z) \bmod z^{2t+1}$. The problem looks difficult, so we break it up into smaller pieces. We consider the sequence of equations

$$(1 + S)\sigma^{(k)} \equiv \omega^{(k)} \bmod z^{k+1} \tag{7.301}$$

For each $k = 0, 1, 2, \ldots, 2t$, we shall find polynomials

$$\sigma^{(k)} = \sum_i \sigma_i^{(k)} z^i$$

and

$$\omega^{(k)} = \sum_i \omega_i^{(k)} z^i$$

which solve this equation. In general, these equations may have many solutions. Since the degree of σ is the number of errors, a good decoder must attempt to find a solution in which degree σ and degree ω are "small."

If we have a solution to (7.301), then we can hope that this same pair of polynomials, $\sigma^{(k)}$ and $\omega^{(k)}$, might also solve the equation

$$(1 + S)\sigma^{(k)} \stackrel{?}{\equiv} \omega^{(k)} \bmod z^{k+2}$$

In general, we cannot expect to be so lucky. However, we may write

$$(1 + S)\sigma^{(k)} \equiv \omega^{(k)} + \Delta_1^{(k)} z^{k+1} \bmod z^{k+2} \tag{7.302}$$

where we define $\Delta_1^{(k)}$ as the coefficient of z^{k+1} in the product $(1 + S)\sigma^{(k)}$. If $\Delta_1^{(k)} = 0$, then we may evidently proceed by taking $\sigma^{(k+1)} = \sigma^{(k)}$ and $\omega^{(k+1)} = \omega^{(k)}$. In order to define $\sigma^{(k+1)}$ in the case when $\Delta_1^{(k)} \neq 0$, we introduce the auxiliary polynomials $\tau^{(k)}$ and $\gamma^{(k)}$, which will be chosen so as to solve the *auxiliary equation*

$$(1 + S)\tau^{(k)} \equiv \gamma^{(k)} + z^k \bmod z^{k+1} \tag{7.303}$$

Of course we would also like the degrees of $\tau^{(k)}$ and $\gamma^{(k)}$ to be "small." In terms of these auxiliary polynomials, we may define the successive σ's and ω's by

$$\sigma^{(k+1)} = \sigma^{(k)} - \Delta_1^{(k)} z\tau^{(k)} \tag{7.304}$$

$$\omega^{(k+1)} = \omega^{(k)} - \Delta_1^{(k)} z\gamma^{(k)} \tag{7.305}$$

It is readily seen that $\sigma^{(k+1)}$ and $\omega^{(k+1)}$ satisfy the equation

$$(1 + S)\sigma^{(k+1)} \equiv \omega^{(k+1)} \bmod z^{(k+1)+1}$$

if $\sigma^{(k)}$ and $\omega^{(k)}$ satisfy (7.302) and $\tau^{(k)}$ and $\gamma^{(k)}$ satisfy (7.303).

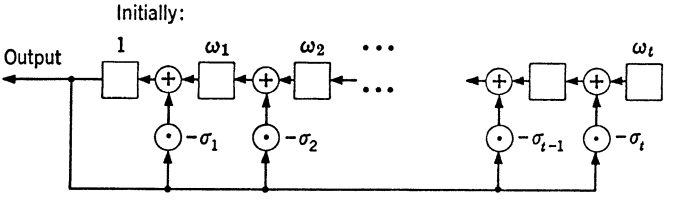


Fig. 7.2 Another feedback-shift-register interpretation of the key equation.

There are two obvious ways to define $\tau^{(k+1)}$ and $\gamma^{(k+1)}$. *Either*

$$\tau^{(k+1)} = z\tau^{(k)} \quad \text{and} \quad \gamma^{(k+1)} = z\gamma^{(k)} \tag{7.306}$$

or

$$\tau^{(k+1)} = \frac{\sigma^{(k)}}{\Delta_1{}^{(k)}} \quad \text{and} \quad \gamma^{(k+1)} = \frac{\omega^{(k)}}{\Delta_1{}^{(k)}} \tag{7.307}$$

Either choice will satisfy the equation

$$(1 + S)\tau^{(k+1)} \equiv \gamma^{(k+1)} + z^{k+1} \bmod z^{k+2}$$

if $\sigma^{(k)}$ and $\omega^{(k)}$ satisfy (7.302) and $\tau^{(k)}$ and $\gamma^{(k)}$ satisfy (7.303). If $\Delta_1{}^{(k)} = 0$, then (7.307) is meaningless, and we are forced to define $\tau^{(k+1)}$ and $\gamma^{(k+1)}$ by (7.306). However, if $\Delta_1{}^{(k)} \neq 0$, then our choice between (7.306) and (7.307) must be based upon our desire to minimize the degrees of $\tau^{(k+1)}$ and $\gamma^{(k+1)}$. The degrees of $\sigma^{(k+1)}$, $\tau^{(k+1)}$, $\omega^{(k+1)}$, and $\gamma^{(k+1)}$ are given by

$$\deg \sigma^{(k+1)} = \begin{cases} \deg \sigma^{(k)} & \text{if } \Delta_1{}^{(k)} = 0 \text{ or if } \deg \sigma^{(k)} > \deg \tau^{(k)} + 1 \\ 1 + \deg \tau^{(k)} & \text{if } \Delta_1{}^{(k)} \neq 0 \text{ and if } \deg \tau^{(k)} > \deg \sigma^{(k)} - 1 \end{cases} \tag{7.308}$$

$$\deg \sigma^{(k+1)} \leq \text{either of above} \quad \text{if } \Delta_1{}^{(k)} \neq 0 \text{ and if } \deg \sigma^{(k)} = 1 + \deg \tau^{(k)}$$

$$\deg \tau^{(k+1)} = \begin{cases} 1 + \deg \tau^{(k)} & \text{if we use (7.306)} \\ \deg \sigma^{(k)} & \text{if we use (7.307)} \end{cases} \tag{7.309}$$

$$\deg \omega^{(k+1)} = \begin{cases} \deg \omega^{(k)} & \text{if } \Delta_1{}^{(k)} = 0 \text{ or if } \deg \omega^{(k)} > 1 + \deg \gamma^{(k)} \\ 1 + \deg \gamma^{(k)} & \text{if } \Delta_1{}^{(k)} \neq 0 \text{ and if } \deg \gamma^{(k)} > \deg \omega^{(k)} - 1 \end{cases} \tag{7.310}$$

$$\deg \omega^{(k+1)} \leq \text{either of above} \quad \text{if } \Delta_1{}^{(k)} \neq 0 \text{ and if } \deg \omega^{(k)} = 1 + \deg \gamma^{(k)}$$

$$\deg \gamma^{(k+1)} = \begin{cases} 1 + \deg \gamma^{(k)} & \text{if we use (7.306)} \\ \deg \omega^{(k)} & \text{if we use (7.307)} \end{cases} \tag{7.311}$$

The degree of $\sigma^{(k+1)}$ is seen to be subject to an "accidental" decrease if $\deg \sigma^{(k)} = 1 + \deg \tau^{(k)}$ and the leading coefficients of $\sigma^{(k)}$ and $\Delta_1{}^{(k)}\tau^{(k)}$ happen to be equal. In order to circumcompute such accidents, we base our choice between (7.306) and (7.307) not on the actual degrees of $\sigma^{(k)}$, $\tau^{(k)}$, $\omega^{(k)}$, and $\gamma^{(k)}$ but on an upper bound $D(k)$ which is independent of such vagaries. We shall define this integral-valued function $D(k)$ in such a way that

$$\deg \sigma^{(k)} \leq D(k) \tag{7.312}$$

$$\deg \tau^{(k)} \leq k - D(k) \tag{7.313}$$

From (7.308), (7.312), and (7.313), we are led to the recursive definition of $D(k)$

$$D(k+1) = \begin{cases} D(k) & \text{if } \Delta_1^{(k)} = 0 \text{ or if } D(k) \geq \dfrac{k+1}{2} \\ k + 1 - D(k) & \text{if } \Delta_1^{(k)} \neq 0 \text{ and } D(k) \leq \dfrac{k+1}{2} \end{cases} \tag{7.314}$$

It is readily seen that if $\deg \sigma^{(k)} \leq D(k)$ and $\deg \tau^{(k)} \leq k - D(k)$, then $\deg \sigma^{(k+1)} \leq D(k+1)$. Similarly, if $\deg \omega^{(k)} \leq D(k)$ and $\deg \gamma^{(k)} \leq k - D(k)$, then $\deg \omega^{(k+1)} \leq D(k+1)$.

In order to ensure that $\deg \tau^{(k+1)} \leq (k+1) - D(k+1)$ and that $\deg \gamma^{(k+1)} \leq (k+1) - D(k+1)$, we must adopt the following rule (7.315) for choosing between (7.306) and (7.307).

$$\text{Use} \begin{cases} (7.306) & \text{if } \Delta_1^{(k)} = 0 \text{ or if } D(k) > \dfrac{k+1}{2} \\ (7.307) & \text{if } \Delta_1^{(k)} \neq 0 \text{ and } D(k) < \dfrac{k+1}{2} \end{cases} \tag{7.315}$$

If $\Delta_1^{(k)} \neq 0$ and $D(k) = (k+1)/2$, then either (7.306) or (7.307) will give us polynomials $\tau^{(k+1)}$ and $\gamma^{(k+1)}$ each having degree $\leq k + 1 - D(k+1)$. When in doubt, procrastinate! We postpone the close decision between (7.306) and (7.307) in this case until we have looked at another consideration.

The initial equations are

$$(1 + S)\sigma^{(0)} \equiv \omega^{(0)} \bmod z$$
$$(1 + S)\tau^{(0)} \equiv \gamma^{(0)} + 1 \bmod z$$

The equations may be solved by the obvious initialization

$$\sigma^{(0)} = \tau^{(0)} = \omega^{(0)} = 1 \qquad \gamma^{(0)} = 0 \qquad D(0) = 0 \tag{7.316}$$

We notice that $\deg \sigma^{(0)} = \deg \tau^{(0)} = \deg \omega^{(0)} = 0 = D(0)$ but that† $\deg \gamma^{(0)} = -\infty < D(0)$. Thus, at least initially, we may do even better than the restrictions

$$\deg \omega^{(k)} \leq D(k)$$
$$\deg \gamma^{(k)} < k - D(k)$$

We require that at least one of these expressions be satisfied with strict inequality. To this end, we introduce the Boolean function $B(k)$, with initial value $B(0) = 0$. [In general, either $B(k) = 0$ or $B(k) = 1$.] We require that

$$\deg \omega^{(k)} \leq D(k) - B(k) \tag{7.317}$$
$$\deg \gamma^{(k)} \leq k - D(k) - [1 - B(k)] \tag{7.318}$$

† Recall that $\deg 0 = -\infty$, as explained in the footnote on page 28.

If we take the proper choice between (7.306) and (7.307) in the case when $\Delta_1^{(k)} \neq 0$ and $D(k) = (k+1)/2$, and if we define $B(k)$ carefully, then we can guarantee that (7.317) and (7.318) hold for all k. By examining (7.310) and (7.311), the proper choice is seen to be

$$\text{Use} \begin{cases} (7.306) & \text{if } \Delta_1^{(k)} \neq 0, D(k) = \dfrac{k+1}{2}, \text{ and } B(k) = 0 \\ (7.307) & \text{if } \Delta_1^{(k)} \neq 0, D(k) = \dfrac{k+1}{2}, \text{ and } B(k) = 1 \end{cases} \tag{7.319}$$

$$B(k+1) = \begin{cases} B(k) & \text{when using (7.306)} \\ 1 - B(k) & \text{when using (7.307)} \end{cases} \tag{7.320}$$

This completes the heuristic derivation of the recursive algorithm. To summarize, we start from the initial conditions (7.316). We then proceed recursively as follows. Define $\Delta_1^{(k)}$ by (7.302), $\sigma^{(k+1)}$ by (7.304), $\omega^{(k+1)}$ by (7.305), and $D(k+1)$ by (7.314). According to (7.315) and (7.319), we then define $\tau^{(k+1)}$ and $\gamma^{(k+1)}$ by (7.306) or (7.307) and $B(k+1)$ according to (7.320). The polynomials defined in this recursive manner are then seen to satisfy Eqs. (7.301), (7.303), (7.312), (7.313), (7.317), and (7.318). We restate the algorithm explicitly as follows.

7.4 AN ALGORITHM FOR SOLVING THE KEY EQUATION OVER ANY FIELD

Initially define $\sigma^{(0)} = 1$, $\tau^{(0)} = 1$, $\omega^{(0)} = 1$, $\gamma^{(0)} = 0$, $D(0) = 0$, $B(0) = 0$. Proceed recursively as follows. If S_{k+1} is unknown, stop; otherwise define $\Delta_1^{(k)}$ as the coefficient of z^{k+1} in the product $(1 + S)\sigma^{(k)}$ and let

$$\begin{aligned} \sigma^{(k+1)} &= \sigma^{(k)} - \Delta_1^{(k)} z\tau^{(k)} \\ \omega^{(k+1)} &= \omega^{(k)} - \Delta_1^{(k)} z\gamma^{(k)} \end{aligned}$$

If $\Delta_1^{(k)} = 0$, or if $D(k) > (k+1)/2$, or if $\Delta_1^{(k)} \neq 0$ and $D(k) = (k+1)/2$ and $B(k) = 0$, set

$$\begin{aligned} D(k+1) &= D(k) \\ B(k+1) &= B(k) \\ \tau^{(k+1)} &= z\tau^{(k)} \\ \gamma^{(k+1)} &= z\gamma^{(k)} \end{aligned}$$

But if $\Delta_1^{(k)} \neq 0$ and either $D(k) < (k+1)/2$ or $D(k) = (k+1)/2$ and $B(k) = 1$, set

$$\begin{aligned} D(k+1) &= k + 1 - D(k) \\ B(k+1) &= 1 - B(k) \\ \tau^{(k+1)} &= \frac{\sigma^{(k)}}{\Delta_1^{(k)}} \\ \gamma^{(k+1)} &= \frac{\omega^{(k)}}{\Delta_1^{(k)}} \end{aligned}$$

Theorem 7.41 For each k,

7.411. $\sigma^{(k)}(0) = \omega^{(k)}(0) = 1$.
7.412. $(1 + S)\sigma^{(k)} \equiv \omega^{(k)} + \Delta_1{}^{(k)} z^{k+1} \bmod z^{k+2}$.
7.413. $(1 + S)\tau^{(k)} \equiv \gamma^{(k)} + z^k \bmod z^{k+1}$.
7.414. $\deg \sigma^{(k)} \leq D(k)$ with equality if $B(k) = 1$.
7.415. $\deg \tau^{(k)} \leq k - D(k)$ with equality if $B(k) = 0$.
7.416. $\deg \omega^{(k)} \leq D(k) - B(k)$ with equality if $B(k) = 0$.
7.417. $\deg \gamma^{(k)} \leq k - D(k) - [1 - B(k)]$ with equality if $B(k) = 1$.

Theorem 7.42 For each k,

$$\omega^{(k)}\tau^{(k)} - \sigma^{(k)}\gamma^{(k)} = z^k$$

Theorem 7.43 If σ and ω are any pair of polynomials which satisfy

$$\sigma(0) = 1 \quad \text{and} \quad (1 + S)\sigma \equiv \omega \bmod z^{k+1}$$
$$D = \max \{\deg \sigma, \deg \omega\}$$

then there exist polynomials U and V such that $U(0) = 1$, $V(0) = 0$, $\deg U \leq D - D(k)$, $\deg V \leq D - [k - D(k)]$, and

$$\sigma = U\sigma^{(k)} + V\tau^{(k)}$$
$$\omega = U\omega^{(k)} + V\gamma^{(k)}$$

Theorem 7.44 If σ and ω are relatively prime, and $\sigma(0) = 1$ and $(1 + S)\sigma \equiv \omega \bmod z^{k+1}$, then

7.441. Either $\deg \sigma \geq D(k) + 1 - B(k) \geq D(k)$, or $\deg \omega \geq D(k)$, or both.
7.442. If $\deg \sigma \leq (k + 1)/2$ and $\deg \omega \leq k/2$, then $\sigma = \sigma^{(k)}$ and $\omega = \omega^{(k)}$.

Proof of Theorem 7.41, excepting the equality clauses of 7.414 to 7.417 These claims were proved in the heuristic derivation of the algorithm. Readers who prefer a direct proof may verify these claims by a straightforward induction on k.

Proof of Theorem 7.42 According to Theorem 7.41,

$$(1 + S)\sigma^{(k)} \equiv \omega^{(k)} \bmod z^{k+1}$$
$$(1 + S)\tau^{(k)} \equiv \gamma^{(k)} + z^k \bmod z^{k+1}$$

Taking the product of these two congruences gives

$$(1 + S)\tau^{(k)}\omega^{(k)} \equiv (1 + S)\sigma^{(k)}(\gamma^{(k)} + z^k)$$

Dividing by $1 + S$ gives

$$\tau^{(k)}\omega^{(k)} \equiv \sigma^{(k)}\gamma^{(k)} + \sigma^{(k)}z^k$$

Since

$$\sigma^{(k)}z^k \equiv \sigma_0{}^{(k)}z^k \equiv z^k \bmod z^{k+1}$$

this becomes

$$\tau^{(k)}\omega^{(k)} - \gamma^{(k)}\sigma^{(k)} \equiv z^k \bmod z^{k+1}$$

According to Theorems 7.415 and 7.416, $\deg \omega^{(k)} + \deg \tau^{(k)} \leq k$. According to Theorems 7.414 and 7.417, $\deg \sigma^{(k)} + \deg \gamma^{(k)} \leq k$. Therefore, $\deg \{\tau^{(k)}\omega^{(k)} - \sigma^{(k)}\gamma^{(k)}\} \leq k$ from which we conclude that

$$\boxed{\tau^{(k)}\omega^{(k)} - \sigma^{(k)}\gamma^{(k)} = z^k}$$

Proof of equality clauses of 7.414 to 7.417 If $B(k) = 0$, then

$$\deg \gamma^{(k)} \leq k - D(k) - 1$$
$$\deg \sigma^{(k)} \leq D(k)$$

and

$$\deg \{\sigma^{(k)}\gamma^{(k)}\} \leq k - 1$$

Since

$$\deg \{\omega^{(k)}\tau^{(k)} - \sigma^{(k)}\gamma^{(k)}\} = k$$

it follows that

$$\deg \{\omega^{(k)}\tau^{(k)}\} = k \quad \text{and} \quad \deg \omega^{(k)} = D(k)$$
$$\deg \tau^{(k)} = k - D(k)$$

Similarly, if $B(k) = 1$, then $\deg \omega^{(k)} \leq D(k) - 1$, $\deg \tau^{(k)} \leq k - D(k)$, and $\deg \{\omega^{(k)}\tau^{(k)}\} \leq k - 1$. It follows that $\deg \{\sigma^{(k)}\gamma^{(k)}\} = k$, $\deg \sigma^{(k)} = D(k)$, and $\deg \gamma^{(k)} = k - D(k)$.

Remarks on Theorem 7.43 This theorem gives us the form of the *general solution* (of any degree) of the equations $\sigma(0) = 1$, $(1 + S)\sigma \equiv \omega \bmod z^{k+1}$. In view of Theorems 7.42 and 7.411, it is evident that

$\sigma^{(k)}$ and $\tau^{(k)}$ are relatively prime. Hence any polynomial f may be expressed in the form

$$f = U\sigma^{(k)} + V\tau^{(k)}$$

Theorem 7.43 asserts that if σ and ω are a solution to the equations $\sigma(0) = 1$, $(1 + S)\sigma \equiv \omega \bmod z^{k+1}$, then the same U and V hold for both expressions $\sigma = U\sigma^{(k)} + V\tau^{(k)}$ and $\omega = U\omega^{(k)} + V\gamma^{(k)}$. Furthermore, Theorem 7.43 asserts that the degrees of U and V are small.

Proof of Theorem 7.43

$$\omega \equiv (1 + S)\sigma \bmod z^{k+1} \qquad \text{(hypothesis)}$$

Multiplying by $\omega^{(k)}$ and then using Theorem 7.412 gives

$$\begin{aligned}(1 + S)\sigma^{(k)}\omega &\equiv (1 + S)\sigma\omega^{(k)} \\ \sigma^{(k)}\omega &\equiv \sigma\omega^{(k)} \\ \sigma^{(k)}\omega - \sigma\omega^{(k)} &= -z^k V \qquad (7.45)\end{aligned}$$

where $V(0) = 0$ and $\deg V \leq D(k) + D - k$. Multiplying the hypothesis by $\tau^{(k)}$ and then using Theorem 7.413 gives

$$\begin{aligned}(1 + S)\tau^{(k)}\omega &\equiv (1 + S)\sigma(\gamma^{(k)} + z^k) \\ \tau^{(k)}\omega &\equiv \sigma(\gamma^{(k)} + z^k) \equiv \sigma\gamma^{(k)} + z^k \\ \tau^{(k)}\omega - \sigma\gamma^{(k)} &= Uz^k \qquad (7.46)\end{aligned}$$

where $U(0) = 1$ and $\deg U \leq D - D(k)$. Subtracting $\tau^{(k)}$ times (7.45) from $\sigma^{(k)}$ times (7.46) gives

$$(\tau^{(k)}\omega^{(k)} - \sigma^{(k)}\gamma^{(k)})\sigma = z^k(U\sigma^{(k)} + V\tau^{(k)})$$

Using Theorem 7.42, this becomes

$$\boxed{\sigma = U\sigma^{(k)} + V\tau^{(k)}}$$

Similarly, subtracting $\gamma^{(k)}$ times (7.45) from $\omega^{(k)}$ times (7.46) gives

$$(\omega^{(k)}\tau^{(k)} - \sigma^{(k)}\gamma^{(k)})\omega = z^k(U\omega^{(k)} + V\gamma^{(k)})$$

Using Theorem 7.42, this becomes

$$\boxed{\omega = U\omega^{(k)} + V\gamma^{(k)}}$$

Proof of Theorem 7.44 Theorem 7.441 is an immediate consequence of Eq. (7.46) and Theorems 7.415 and 7.417.

Consequently, if $\deg \sigma \leq (k+1)/2$ and $\deg \omega \leq k/2$, then $D(k) \leq (k+1)/2$, with strict inequality if $B(k) = 0$. According to Theorems 7.414 and 7.416, we may deduce that

$$\deg \sigma^{(k)} \leq \frac{k+1}{2} \qquad \deg \omega^{(k)} \leq \frac{k}{2} \qquad \deg (\sigma^{(k)}\omega) \leq k + \tfrac{1}{2} < k+1$$

$$\deg (\sigma\omega^{(k)}) \leq k + \tfrac{1}{2} < k+1$$

$$\deg (-z^k V) = \deg (\sigma^{(k)}\omega - \sigma\omega^{(k)}) < k+1 \qquad \deg V < 1$$

Since $V(0) = 0$, this implies $V = 0$. Theorem 7.43 becomes $\sigma = U\sigma^{(k)}$, $\omega = U\omega^{(k)}$. By hypothesis, σ and ω are relatively prime, so $U = 1$. Q.E.D.

★ 7.5 RELATION TO MATRIX DECODING METHODS

Previous methods for calculating the σ's from the S's have relied on the direct solution of simultaneous linear equations. Instead of the key equation $(1 + S)\sigma \equiv \omega \bmod z^{2t+1}$, one has the equivalent matrix equation

$$\begin{bmatrix} 1 & 0 & 0 & 0 & \cdots & \cdots & 0 \\ S_1 & 1 & 0 & 0 & \cdots & \cdots & 0 \\ S_2 & S_1 & 1 & 0 & \cdots & \cdots & 0 \\ S_3 & S_2 & S_1 & 1 & \cdots & \cdots & 0 \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ S_t & \cdots & \cdots & \cdots & S_2 & S_1 & 1 \\ S_{t+1} & S_t & \cdots & \cdots & \cdots & \cdots & S_1 \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ S_{2t} & \cdots & \cdots & \cdots & \cdots & \cdots & S_t \end{bmatrix} \begin{bmatrix} 1 \\ \sigma_1 \\ \sigma_2 \\ \cdots \\ \cdots \\ \sigma_t \end{bmatrix} = \begin{bmatrix} 1 \\ \omega_1 \\ \omega_2 \\ \cdots \\ \cdots \\ \omega_t \\ 0 \\ 0 \\ \cdot \\ \cdot \\ \cdot \\ 0 \end{bmatrix}$$

Given $S_1, S_2, \ldots, S_{2t}$, one might attempt to solve the above equations for $\sigma_1, \sigma_2, \ldots, \sigma_t$ and $\omega_1, \omega_2, \ldots, \omega_t$. First, one determines $\sigma_1, \sigma_2, \ldots, \sigma_t$ from the equations

$$\begin{bmatrix} S_{t+1} & S_t & \cdots & S_2 & S_1 \\ S_{t+2} & S_{t+1} & & & S_2 \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ S_{2t} & \cdots & \cdots & S_{t+1} & S_t \end{bmatrix} \begin{bmatrix} 1 \\ \sigma_1 \\ \sigma_2 \\ \cdots \\ \sigma_{t-1} \\ \sigma_t \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \cdots \\ 0 \end{bmatrix}$$

or equivalently,

$$\begin{bmatrix} S_t & & S_2 & S_1 \\ S_{t+1} & & & S_2 \\ \cdots & \cdots & \cdots & \cdots \\ S_{2t-1} & & & S_t \end{bmatrix} \begin{bmatrix} \sigma_1 \\ \sigma_2 \\ \cdots \\ \sigma_t \end{bmatrix} = - \begin{bmatrix} S_{t+1} \\ S_{t+2} \\ \cdots \\ S_{2t} \end{bmatrix}$$

These equations are typically solved by means of a reduction of the above matrix, as in Sec. 2.5. This method requires many more computations and more storage space than the generating-function approach. As we have seen in Secs. 7.1 to 7.4, there is no need to introduce these matrices at all.

Because a considerable amount of work has been done with these matrices, we shall show how Algorithm 7.4 relates to the matrix methods. If one attempts to solve the above equations for the k unknowns $\sigma_1, \sigma_2, \ldots, \sigma_k$, then the $k \times k$ coefficient matrix is given by

$$\mathcal{M}_k = \begin{bmatrix} S_k & & S_2 & S_1 \\ & & & S_2 \\ S_{2k-1} & & & S_k \end{bmatrix}$$

If the determinant of $\mathcal{M}_k$ is nonzero, then these equations have a unique solution for $\sigma_1, \sigma_2, \ldots, \sigma_k$. The traditional method for finding the σ's, given the S's, has been to select the largest k for which $\mathcal{M}_k$ is nonsingular and then solve the corresponding equations.

The iterative algorithm may be used to determine which $\mathcal{M}_k$'s are singular and which are not, according to Theorem 7.51.

Theorem 7.51

$$\text{Nullity } \mathcal{M}_k = |D(2k+1) - k|$$
$$\text{Rank } \mathcal{M}_k = k - |D(2k+1) - k|$$

Proof For $i = k, k+1, k+2, \ldots, 2k-1$, we may define the polynomial

$$\rho^{(i)} = \begin{cases} \tau^{(i)} & \text{iff } i - D(i) \leq k-1 \\ z^{2k-i-1}\sigma^{(i)} & \text{iff } i - D(i) \geq k \end{cases}$$

Since $i - D(i) \geq k$ iff $2k - i - 1 + D(i) \leq k-1$, we conclude that

$$\deg \rho^{(i)} \leq k-1 \qquad \text{for } i = k, k+1, k+2, \ldots, 2k-1$$

With each polynomial $\rho^{(i)}$, we associate a k-dimensional column vector

$$\underline{\rho}^{(i)} = [\rho_0^{(i)}, \rho_1^{(i)}, \ldots, \rho_{k-1}^{(i)}]^t$$

We then introduce the $k \times k$ matrix ρ by taking these $\rho^{(i)}$ for the successive columns:

$$\rho = \begin{bmatrix} \rho_0^{(k)} & \rho_0^{(k+1)} & \cdots & \rho_0^{(2k-1)} \\ \rho_1^{(k)} & \rho_1^{(k+1)} & \cdots & \rho_1^{(2k-1)} \\ \rho_2^{(k)} & \rho_2^{(k+1)} & \cdots & \rho_2^{(2k-1)} \\ \cdots & \cdots & \cdots & \cdots \\ \rho_{k-1}^{(k)} & \rho_{k-1}^{(k+1)} & \cdots & \rho_{k-1}^{(2k-1)} \end{bmatrix}$$

We claim that the matrix is nonsingular. For, if some linear combination of its columns were zero, then

$$\sum_{i=k}^{2k-1} c_i \underline{\rho}^{(i)} = \mathbf{0}$$

and

$$\sum_{i=k}^{2k-1} a_i z^{2k-i-1} \sigma^{(i)} + \sum_{i=k}^{2k-1} b_i \tau^{(i)} = 0$$

where

$$a_i = \begin{cases} c_i & \text{if } i - D(i) \geq k \\ 0 & \text{otherwise} \end{cases} \qquad b_i = \begin{cases} c_i & \text{if } i - D(i) \leq k-1 \\ 0 & \text{otherwise} \end{cases}$$

If $a_i \neq 0$, then $(1 + S)\rho^{(i)} \equiv z^{2k-i-1}\omega^{(i)} \bmod z^{2k}$, and $\deg z^{2k-i-1}\omega^{(i)} \leq 2k - i - 1 + D(i) \leq k - 1$. If $b_i \neq 0$, then $(1 + S)\tau^{(i)} \equiv \gamma^{(i)} + z^i \bmod z^{i+1}$, and $\deg \gamma^{(i)} \leq k - 1$. Hence, if $b_k = b_{k+1} = \cdots = b_{k+j-1} = 0$ but $b_{k+j} \neq 0$, then

$$(1 + S) \sum_{i=k}^{2k-1} c_i \underline{\rho}^{(i)} \equiv \xi + b_{k+j} z^{k+j} \bmod z^{k+j+1}$$

where $\deg \xi \leq k - 1$. We deduce that if $\sum_{i=k}^{2k-1} c_i \underline{\rho}^{(i)} = 0$, then $b_i = 0$ for $i = k, k + 1, \ldots, 2k - 1$ and $\sum_{i=k}^{2k-1} a_i z^{2k-i-1} \sigma^{(i)} = 0$. But if $a_{k+j} \neq 0$ and $a_{k+j+1} = a_{k+j+2} = \cdots = a_{2k-1} = 0$, then

$$\sum_{i=k}^{2k-1} a_i z^{2k-i-1} \sigma^{(i)} \equiv a_{k+j} z^{k-j-1} \bmod z^{k-j}$$

We conclude that $\sum_{i=k}^{2k-1} c_i \varrho^{(i)} = 0$ iff $c_i = 0$ for all i, and that the ρ matrix is nonsingular.

Since the matrix ρ is nonsingular, the rank of $\mathcal{M}_k$ is the same as the rank of the product matrix $\mathcal{P} = \mathcal{M}_k\rho$. The columns of $\mathcal{P}$ are $\mathcal{M}_k\varrho^{(i)}$, $i = k, k+1, \ldots, 2k-1$. Because of the structure of the matrix $\mathcal{M}_k$, the column $\mathcal{M}_k\varrho^{(i)}$ is given by $\mathbf{P}^{(i)} = [\mathcal{P}_k^{(i)}, \mathcal{P}_{k+1}^{(i)}, \ldots, \mathcal{P}_{2k-1}^{(i)}]^t$, where $\sum_j \mathcal{P}_j^{(i)} z^j = (1+S)\rho^{(i)}$. If $\rho^{(i)} = z^{2k-i-1}\sigma^{(i)}$, then $i - D(i) \geq k$, $\deg z^{2k-i-1}\sigma^{(i)} \leq k-1$ and $(1+S)\rho^{(i)} \equiv z^{2k-i-1}\omega^{(i)} \bmod z^{2k}$, so that $\mathbf{P}^{(i)} = \mathcal{M}_k\varrho^{(i)} = \mathbf{0}$. On the other hand, if $\rho^{(i)} = \tau^{(i)}$, then $\deg \gamma^{(i)} \leq i - D(i) \leq k-1$ and $(1+S)\rho^{(i)} \equiv \gamma^{(i)} \equiv z^i \bmod z^{i+1}$, so that $\mathbf{P}^{(i)} = \mathcal{M}_k\varrho^{(i)} = [0,0, \ldots, 0,1, \ldots]^t$. (The first nonzero entry in $\mathbf{P}^{(i)}$ is a one in the $[i-(k-1)]$st coordinate.) From these arguments it follows that the $\mathcal{P}$ matrix is triangular, with zeros above the main diagonal. Each entry on the main diagonal is either zero or one. If it is zero, the entire column containing it is zero. The null space of $\mathcal{M}_k$ is spanned by the columns $\rho^{(i)} = z^{2k-i-1}\sigma^{(i)}$; the range of $\mathcal{M}_k$ is spanned by the columns of $\mathcal{P}$ for which $\mathcal{P}^{(i)} = (1+S)\tau^{(i)}$. The nullity of $\mathcal{M}_k$ is the same as the nullity of $\mathcal{P}$, which is equal to the number of i's $(k \leq i \leq 2k-1)$ for which $i - D(i) \geq k$, or equivalently, $D(i) \leq i - k$.

From Algorithm 7.4,

$$D(i+1) = \begin{cases} D(i) & \text{if } D(i) > \dfrac{i+1}{2} \text{ or if } \Delta_1^{(i)} = 0 \\ i+1-D(i) & \text{otherwise} \end{cases}$$

It is evident that $D(i)$ is a monotonic nondecreasing function of i and that $D(i) \leq i/2$ only if $D(i) = D(i-1)$.

We next claim that the set of i's for which $D(i) \leq i-k$ must occur consecutively. For if $D(i) \leq i-k < i+1-k \leq D(i+1)$, then $D(i+1) = i+1-D(i) \geq k+1$ and $D(j) = D(i+1) \geq k+1 > j+1-k$ for $j = i+1, i+2, \ldots, 2k-1$.

If $D(2k-1) = k-j, j > 0$, then

$$D(i) = \begin{cases} k-j \leq i-k & \text{for } i = 2k-j, \ldots, 2k-1 \\ > i-k & \text{for } i = k, k+1, \ldots, 2k-j-1 \end{cases}$$

In this case, nullity $\mathcal{M}_k = j = k - D(2k-1)$.

On the other hand, if $D(2k-1) = k+j$, $j > 0$, then there exists an m such that if $D(2k-1) = k-j$, $j > 0$, then

$$D(i) = \begin{cases} k+j > i-k & \text{for } i = m, m+1, \ldots, 2k-1 \\ m-j-k \leq i-k & \text{for } i = m-j, \ldots, m-1 \end{cases}$$

In this case, nullity $\mathcal{M}_k = j = D(2k-1) - k$. We conclude that

Nullity $\mathcal{M}_k = \lvert D(2k-1) - k \rvert$; rank $\mathcal{M}_k = k - \lvert D(2k-1) - k \rvert$

This is equivalent to Theorem 7.51. Q.E.D.

★ 7.6 SIMPLIFICATIONS IN ALGORITHM 7.4 FOR BINARY BCH CODES

Given *any* sequence $S_1, S_2, S_3, \ldots, S_{2t}$, Algorithm 7.4 will find low-degree polynomials $\sigma(z)$ and $\omega(z)$ which solve the key equation (7.23). In the case of binary BCH codes, however, the sequence $S_1, S_2, S_3, \ldots, S_{2t}$ is not arbitrary. The S's must be power-sum symmetric functions of e error locations, $X_1, X_2, \ldots, X_e$. Even if $e > t$, we have $S_k = \sum_{i=1}^{e} X_i^k$, $S_{2k} = \sum_{i=1}^{e} X_i^{2k} = \left(\sum_{i=1}^{e} X_i^k\right)^2 = S_k^2$. Thus, the S's are related to each other by the equations $S_{2k} = S_k^2$. The generating function $S(z)$ must therefore satisfy the equation $[S(z)]^2 = \sum_{k=1}^{\infty} S_{2k} z^{2k} = \hat{S}(z)$, where I again use the accent $\hat{}$ to denote the even part and the accent $\backsim$ to denote the odd part, as in Sec. 3.2. The generating-function equation $\hat{S} = S^2$ leads to a considerable simplification of the iterative algorithm, as we shall now show.

Lemma 7.61 Let $1 + S$ and $1 + R$ be reciprocal generating functions in a field of characteristic 2: $(1+S)(1+R) = 1$. Then $\hat{R} = 0$ iff $\hat{S} = S^2$.

Proof Separating $(1+S)(1+R) = 1$ into even and odd parts gives $(1+\hat{S})(1+\hat{R}) + \tilde{S}\tilde{R} = 1$ and $\tilde{S}(1+\hat{R}) + (1+\hat{S})\tilde{R} = 0$. Subtracting $\tilde{S}$ times the latter from $1+\hat{S}$ times the former gives $[(1+\hat{S})^2 - \tilde{S}^2](1+\hat{R}) = 1+\hat{S}$, from which

$$\hat{R} = \frac{1+\hat{S}}{(1+\hat{S})^2 - \tilde{S}^2} - 1$$

In a field of characteristic 2, $(1+\hat{S})^2 - \tilde{S}^2 = 1 + \hat{S}^2 + \tilde{S}^2 = 1 + (\hat{S}+\tilde{S})^2 = 1 + S^2$ and $\hat{R} = 0$ iff $1 + \hat{S} = 1 + S^2$ or $\hat{S} = S^2$. Q.E.D.

Theorem 7.62 If $\hat{S}(z) = [S(z)]^2$ in a field of characteristic 2, then the quantities defined by Algorithm 7.4 satisfy

$$\omega^{(i)} = \hat{\sigma}^{(i)}$$
$$\gamma^{(i)} = \begin{cases} \bar{\tau}^{(i)} & \text{if } i \text{ even} \\ \hat{\tau}^{(i)} & \text{if } i \text{ odd} \end{cases}$$
$$\Delta_1{}^{(i)} = 0 \quad \text{if } i \text{ odd}$$

Proof The proof is by induction on i. The theorem is true for $i = 0$. We assert that if

$$\omega^{(2k)} = \hat{\sigma}^{(2k)} \quad \text{and} \quad \gamma^{(2k)} = \bar{\tau}^{(2k)}$$

then

$$\omega^{(2k+1)} = \hat{\sigma}^{(2k+1)} \qquad \gamma^{(2k+1)} = \hat{\tau}^{(2k+1)} \qquad \Delta_1{}^{(2k+1)} = 0$$

and therefore

$$\sigma^{(2k+2)} = \sigma^{(2k+1)} \qquad \omega^{(2k+2)} = \omega^{(2k+1)} \qquad \tau^{(2k+2)} = z\tau^{(2k+1)}$$
$$\gamma^{(2k+2)} = z\gamma^{(2k+1)} \qquad \omega^{(2k+2)} = \hat{\sigma}^{(2k+2)} \quad \text{and} \quad \gamma^{(2k+2)} = \bar{\tau}^{(2k+2)}$$

The only nonobvious claim of the previous sentence is that $\Delta_1{}^{(2k+1)} = 0$. To prove this, we write

$$(1 + S)\sigma^{(2k+1)} \equiv \hat{\sigma}^{(2k+1)} + \Delta_1{}^{(2k+1)}z^{2k+2} \bmod z^{2k+3}$$

In view of Lemma 7.61, we may multiply this by $1 + \tilde{R}$ to obtain

$$\sigma^{(2k+1)} \equiv \hat{\sigma}^{(2k+1)} + \tilde{R}\hat{\sigma}^{(2k+1)} + \Delta_1{}^{(2k+1)}z^{2k+2} \bmod z^{2k+3}$$
$$\bar{\sigma}^{(2k+1)} + \tilde{R}\hat{\sigma}^{(2k+1)} \equiv \Delta_1{}^{(2k+1)}z^{2k+2} \bmod z^{2k+3}$$

Since the expression on the left is odd, $\Delta_1{}^{(2k+1)} = 0$. Q.E.D.

Theorem 7.63

$$\deg \sigma^{(k)} = D(k) \qquad \deg \tau^{(k)} = k - D(k)$$

Proof Suppose $\deg \sigma^{(k)} = D(k)$ and $\deg \tau^{(k)} = k - D(k)$. Then $\deg \sigma^{(k)} = 1 + \deg \tau^{(k)}$ only if k is odd, and in this case $\Delta_1{}^{(k)} = 0$. It follows that $\deg \sigma^{(k)} \neq \deg \Delta_1{}^{(k)}z\tau^{(k)}$. According to Algorithm 7.4, $\sigma^{(k+1)} = \sigma^{(k)} - \Delta_1{}^{(k)}z\tau^{(k)}$. Since the two terms on the right have different degrees, $\deg \sigma^{(k+1)} = \max\{\deg \sigma^{(k)}, \deg \Delta_1{}^{(k)}z\tau^{(k)}\} = D(k+1)$. It is easy to verify that $\deg \tau^{(k+1)} = k + 1 - D(k+1)$. Since $\deg \sigma^{(0)} = \deg \tau^{(0)} = 0 = D(0)$, Theorem 7.63 is true by induction on k. Q.E.D.

In view of Theorems 7.62 and 7.63, we may compute the functions $\sigma^{(0)}, \tau^{(0)}, \sigma^{(2)}, \tau^{(2)}, \sigma^{(4)}, \tau^{(4)}, \ldots, \sigma^{(2t)}, \tau^{(2t)}$ by an abbreviated algorithm, in which the ω and $\tilde{\gamma}$ polynomials and the odd-indexed σ and τ polynomials never appear explicitly.

Abbreviated algorithm 7.64 (for binary BCH codes) Initially define $\sigma^{(0)} = 1$, $\tau^{(0)} = 1$. Proceed recursively as follows. If S_{2k+1} is unknown, stop; otherwise define $\Delta_1{}^{(2k)}$ as the coefficient of z^{2k+1} in the product $(1 + S)\sigma^{(2k)}$. Let

$$\sigma^{(2k+2)} = \sigma^{(2k)} + \Delta_1{}^{(2k)} z\tau^{(2k)}$$

$$\tau^{(2k+2)} = \begin{cases} z^2\tau^{(2k)} & \text{if } \Delta_1{}^{(2k)} = 0 \text{ or if } \deg \sigma^{(2k)} > k \\ \dfrac{z\sigma^{(2k)}}{\Delta_1{}^{(2k)}} & \text{if } \Delta_1{}^{(2k)} \neq 0 \text{ and } \deg \sigma^{(2k)} \leq k \end{cases}$$

From this algorithm, it is immediately evident that

$$\tau_0{}^{(2k)} = 0 \qquad \text{if } k > 0 \tag{7.65}$$

We may also simplify the expression for the general solution of the equations $\sigma(0) = 1$, $(1 + S)\sigma \equiv \hat{\sigma} \bmod z^{2t+1}$. According to Theorem 7.43,

$$\sigma = U\sigma^{(2t)} + V\tau^{(2t)}$$

Multiplying by $1 + S$ gives

$$(1 + S)\sigma = U(1 + S)\sigma^{(2t)} + V(1 + S)\tau^{(2t)}$$
$$(1 + S)\sigma = U\hat{\sigma}^{(2t)} + V\hat{\tau}^{(2t)}$$

which is an even function only if

$$U = \hat{U} \qquad \text{and} \qquad V = \hat{V} \tag{7.66}$$

Previous simplifications of the binary case have been based on the matrix

$$\mathfrak{M}_k = \begin{bmatrix} 1 & 0 & 0 & \cdots & \cdots & \cdots & \cdots \\ S_2 & S_1 & 1 & \cdots & \cdots & \cdots & \cdots \\ S_4 & S_3 & S_2 & S_1 & 1 & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ S_{2k-4} & \cdots & \cdots & \cdots & \cdots & \cdots & S_{k-1} \\ S_{2k-2} & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \end{bmatrix}$$

It is a relatively tedious but straightforward problem to show that

$$\text{Nullity } \mathfrak{M}_k = \left| \left\lceil \frac{k - \deg \sigma^{(2k)}}{2} \right\rceil \right| \tag{7.67}$$

Here the brackets denote the greatest integer less than or equal to the quantity inside. ($[\frac{5}{2}] = 2$; $[-\frac{1}{2}] = -1$.) The proof of this theorem is directly parallel to the proof of Theorem 7.51, if one begins by defining

$$\rho^{(2i)} = \begin{cases} \tau^{(2i)} & \text{iff } \deg \tau^{(2i)} \leq k - 1 \\ z^{2k-2i-1}\sigma^{(2i)} & \text{iff } \deg \sigma^{(2i)} \leq 2i - k \end{cases}$$

for $i = 0, 1, 2, \ldots, k - 1$. We leave the remainder of the proof as an exercise for the reader.

As an illustration of Algorithm 7.64, we present the following example.

Example 7.68 Consider the 3-error-correcting binary BCH code of block length 31, with the field represented as polynomials of degree <5 in α, where $\alpha^5 + \alpha^2 + 1 = 0$. Log and antilog tables for this field are given in Appendix A. Suppose $S_1 = 11101 = \alpha^{14}$, $S_3 = 10000 = \alpha^4$, $S_5 = 00010 = \alpha^1$.

$$1 + S = 1 + \alpha^{14}z + \alpha^{28}z^2 + \alpha^4 z^3 + \alpha^{25}z^4 + \alpha^1 z^5 + \alpha^8 z^6 + \cdots$$

$$\sigma^{(0)} = 1 \qquad \tau^{(0)} = 1$$

$$(1 + S)\sigma^{(0)} \equiv 1 + \alpha^{14}z \bmod z^2$$

$$\sigma^{(2)} = 1 + \alpha^{14}z \qquad \tau^{(2)} = \alpha^{17}z$$

$$(1 + S)\sigma^{(2)} \equiv 1 + (\alpha^4 + \alpha^{11})z^3 \bmod z^4$$

$$\alpha^4 = 10000$$

$$\underline{\alpha^{11} = 00111}$$

$$10111 = \alpha^{26} = \alpha^{-5}$$

$$\sigma^{(4)} = 1 + \alpha^{14}z + \alpha^{12}z^2 \qquad \tau^{(4)} = \alpha^5 z + \alpha^{19}z^2$$

$$(1 + S)\sigma^{(4)} \equiv 1 + \alpha^{12}z^2 + (\alpha + \alpha^8 + \alpha^{16})z^5 \bmod z^6$$

$$\alpha = 00010$$

$$\alpha^8 = 01101$$

$$\underline{\alpha^{16} = 11011}$$

$$10100 = \alpha^7$$

$$\sigma^{(6)} = 1 + \alpha^{14}z + 0z^2 + \alpha^{26}z^3 \qquad \tau^{(6)} = \alpha^{24}z + \alpha^7 z^2 + \alpha^5 z^3$$

This calculation may be summarized as follows:

$2k$	$D(2k)$	$\sigma^{(2k)}$	$\tau^{(2k)}$	$\Delta_1^{(2k)}$
0	0	1	1	α^{14}
2	1	$1 + \alpha^{14}z$	$\alpha^{-14}z$	α^{26}
4	2	$1 + \alpha^{14}z + \alpha^{12}z^2$	$\alpha^{-26}z(1 + \alpha^{14}z)$	α^7
6	3	$1 + \alpha^{14}z + 0z^2 + \alpha^{26}z^3$	$\alpha^{-7}z(1 + \alpha^{14}z + \alpha^{12}z^2)$	?

7.7 IMPLEMENTATION OF BINARY BCH DECODERS

As described in Sec. 7.2, a decoder for any binary BCH code can be designed along the general lines suggested in Sec. 5.5 (Fig. 5.14). We

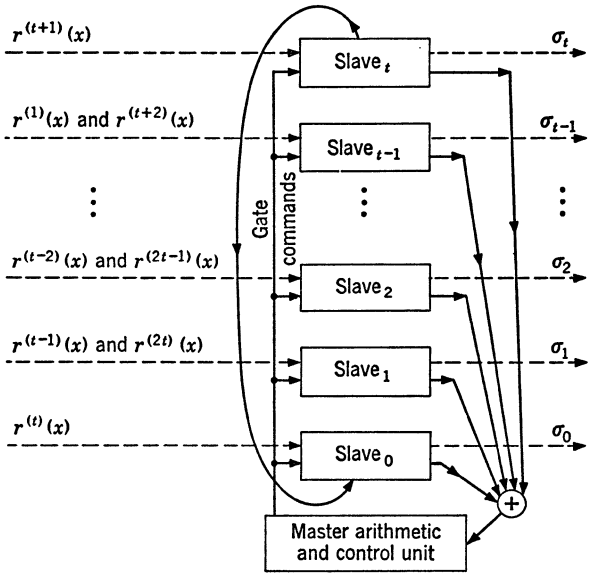


Fig. 7.3 Central GF processor for binary BCH decoder.

may now sketch a design for the central GF processor of Fig. 5.14. One possible overall design is shown in Fig. 7.3. It consists of one master arithmetic and control unit and $t + 1$ separate slave units. The slave units receive all their gate commands directly from the master control unit. The master always sends identical gate commands to all the slaves. The slaves are all identical. Each contains five registers and the facility to multiply two of them together and accumulate the result in its slave accumulator. All these slave accumulators are connected to a master adder, whose output is accessible to the master. Words may be transferred only from each slave to the corresponding register of the adjacent slave. There is no facility by which the master can access data in any of the slaves. Although the master arithmetic unit is able to perform multiplicative inversion, none of the slaves has this ability.

As soon as the last digit of an incoming received word arrives from the channel, the remainders $r^{(1)}(x)$, $r^{(2)}(x)$, . . . , $r^{(2t)}(x)$ are transferred out of the registers at the left of Fig. 5.14 and into the appropriate slave units, as shown by the dotted lines at the left of Fig. 7.3. The slave containing the remainder $r^{(k)}(x)$ then sets one of its registers to the wired constant α^k. It then computes $r_0^{(k)} + r_1^{(k)}\alpha^k$, α^{2k}, $r_0^{(k)} + r_1^{(k)}\alpha^k + r_2^{(k)}\alpha^{2k}$, α^{3k}, until it eventually accumulates the power-sum symmetric function $S_k = r^{(k)}(\alpha^k)$. With all slaves doing this simultaneously, the power-sum symmetric functions $S_1, S_2, S_3, \ldots, S_t$ are obtained in the time required

to perform $m - 1$ Galois field multiplications. In a similar manner, the slaves then compute the other half of the power-sum symmetric functions, $S_{t+1}, S_{t+2}, S_{t+3}, \ldots, S_{2t-1}$. All the relevant S's then appear in appropriate slave registers, as shown in Fig. 7.4, and the master control unit can proceed to execute Algorithm 7.64 or 7.4.

Every step of Algorithms 7.4 and 7.64 can be executed directly by the computer of Fig. 7.3. The contents of the slave registers at a typical stage of the algorithm are shown in Fig. 7.5. The left four registers of slave$_0$ contain S_{k+t+2}, S_{k+1}, 1, and 0; the left four registers of slave$_1$ contain S_{k+t+1}, S_k, $\sigma_1^{(k)}$, and $\tau_1^{(k)}$. Each slave then computes the product of its second and third registers and places the result in its rightmost register. The sum of the rightmost slave registers is then accumulated in the master accumulator. This sum is the quantity $\Delta_1^{(k)}$. The contents of the registers corresponding to the τ column of Fig. 7.5 are then shifted upward to the corresponding registers of the adjacent slaves. The quantity $\Delta_1^{(k)}$ is placed in the master multiplier register. Gate commands are then issued which cause each slave to multiply the contents of its τ register by the contents of the master multiplier. The contents of the σ register are then added to the result to obtain the coefficients of the polynomial, $\sigma^{(k+1)} = \sigma^{(k)} - \Delta_1^{(k)} z\tau^{(k)}$. Then depending on whether $\Delta_1^{(k)} = 0$ and whether or not $D(k) > (k + 1)/2$, the master control unit decides whether or not to move the polynomial $\sigma^{(k)}$ from the

	1	S_{t+1}			
	S_1	S_{t+2}			
	S_2	S_{t+3}			
	S_3	S_{t+4}			
Slave$_3$		S_{2t-2}			
Slave$_2$		S_{2t-1}			
Slave$_1$	S_{t-1}				
Slave$_0$	S_t				

Fig. 7.4 Contents of slave registers at start of Algorithm 7.64 or 7.4.

S_{k+2}	S_{k+t+3}			
S_{k+3}	S_{k+t+4}			
S_{k+4}	⋮			
S_{k+5}				
	S_{2t-1}			
	—			
	1			
	S_1			
	S_2			
	⋮			
	S_{k-3}	⋮	⋮	
	S_{k-2}	$\sigma_3^{(k)}$	$\tau_3^{(k)}$	$S_{k-2}\sigma_3^{(k)}$
S_{k+t}	S_{k-1}	$\sigma_2^{(k)}$	$\tau_2^{(k)}$	$S_{k-1}\sigma_2^{(k)}$
S_{k+t+1}	S_k	$\sigma_1^{(k)}$	$\tau_1^{(k)}$	$S_k\ \sigma_1^{(k)}$
S_{k+t+2}	S_{k+1}	1	0	S_{k+1}

Fig. 7.5 Contents of slave registers at computation of $\Delta_1^{(k+1)}$.

σ column to the τ column. The coefficients of $\sigma^{(k+1)}$ are then moved to the σ column. The polynomial in the τ column (which is either $\sigma^{(k)}$ or $\tau^{(k)}$, as needed) is then used to compute $\tau^{(k+1)}$, which is placed in the τ column. The master control unit computes the value of $D(k+1)$, shifts the S columns upward, and continues with the next iteration of the algorithm.

When the algorithm completes the calculation of $\sigma^{(2t)}$, this error-locator polynomial may then be transferred from the slave registers to the Chien searcher, and the computer of Fig. 7.3 is ready to begin decoding the next received word.

The decoder design sketched in Figs. 5.14 and 7.3 is quite feasible even for rather large binary BCH codes. In general, the cost of a decoder for the t-error-correcting binary BCH code of block length $n = 2^m - 1$ may be expressed as $Am + Bn + Cmt$, where A, B, and C are constants independent of n and t. Am represents the cost of the master processor (which contains several m-bit registers); Bn represents the cost of the buffer to store the received word; Cmt represents all other costs. The cost of a single slave, the cost of one unit of the Chien searcher, and the cost of a register which divides the incoming word by a single irreducible factor of the generator polynomial are all proportional to m, and there are t of each of these components.

The slaves in the computer of Fig. 7.3 need perform only about t multiplications per decoded block, but each unit of the Chien searcher

must perform n multiplications per decoded block. The timing requirements of the decoder are therefore very reasonable: the Chien searcher must perform only one Galois field multiplication and a t-input addition per received channel digit. The other parts of the decoder may operate even slower. For example, if one uses 50-nanosecond switching logic, one can multiply two elements in GF(2^{10}) in 1 microsecond. It is then quite feasible to decode *any* binary BCH code of block length 1,023 at the rate of a million bits (a thousand blocks) per second. The decoding time is independent of the error-correction capability t, although the cost of the decoder does increase with t.

Of course, there are many possible embellishments and modifications of the basic design of Figs. 5.14 and 7.3. One can eliminate slave$_0$ by slightly increasing the complexity of the master. One can also eliminate slave$_t$ and slave$_{t-1}$ by further increases in the complexity of the master program. It is also possible to interpret the computer of Fig. 7.3 in various ways. If one prefers shift registers to polynomials, one may consider the columns of Figs. 7.4 and 7.5 as representing t-stage shift registers over GF(2^m).

PROBLEMS

7.1. If $\sigma^{(2k)}$ and $\tau^{(2k)}$ are defined by Algorithm 7.64, show that $\sigma^{(2k)}\tau^{(2k)} = \text{odd} + z^{2k}$, where odd is an odd function of z.

7.2. Consider the triple-error-correcting binary BCH code of block length 31. Suppose $S_1 = 01010 = \alpha^6$, $S_3 = 01111 = \alpha^{23}$, $S_5 = 0001 = \alpha^0$. Show that $\sigma^{(6)} = 1 + \alpha^6 z + \alpha^{21}z^2 + \alpha^{11}z^3$.

7.3. [Gross (1963)] Show that the extended t-error-correcting binary BCH code formed by annexing an overall parity check to the end of a cyclic binary BCH code is capable of correcting all patterns of t or fewer channel errors and all bursts of $t + 1$ consecutive errors which do not include the overall-parity-check position. For what values of n and t is it also possible to correct bursts of length $t + 1$ which do include the overall-parity-check position?

7.4. Let $1 + S(z)$ be the output sequence of a binary feedback shift register $\sigma(z)$, with initial contents $\omega(z)$. Given each of the following sequences $1 + S(z)$, use Algorithm 7.4 to find a shortest $\sigma(z)$ and $\omega(z)$. In which cases is the answer unique? In which cases is Algorithm 7.64 applicable?

(*a*) $1 + 0z + 1z^2 + 1z^3 + 0z^4 + 0z^5 + 1z^6 + 0z^7 + 0z^8 + 0z^9 + 1z^{10} + \cdots$
$= 1011001001 \ldots$

(*b*) $10001100101 \ldots$

(*c*) $11000000101 \ldots$

(*d*) $11101001110 \ldots$

7.5 What are the next 5 bits in each of the sequences of Prob. 7.4, assuming that each is generated by a shortest feedback shift register?

7.6 [Daykin (1960)] What fraction of the q^{m+n-1} persymmetric† matrices over GF(q) have row rank r? *Hint:* Review Sec. 7.5.

† A *persymmetric* matrix is one with constant minor diagonals $a_{i,j} = a_{k,l}$ whenever $i + j = k + l$.

8
Nonbinary Coding

8.1 MODULATION SCHEMES

Thus far, we have considered only the problem of coding and decoding for the binary symmetric channel, whose inputs are the symbols 0 and 1, and whose outputs are also the symbols 0 and 1. In most applications, the encoded sequence of 0s and 1s is given to a modulator, which converts these symbols into certain functions of time. The resulting time function is used to control the amplitude of the transmitted signal, which, for example, might be the voltage on a wire or the instantaneous power emitted by a radio transmitter.

For example, the symbol 0 might represent the function shown in Fig. 8.1, and the symbol 1 might represent the function shown in Fig. 8.2. The encoded sequence 0001101 would then cause the modulator to transmit the function shown in Fig. 8.3. Additive noise might then cause the received signal to appear as in Fig. 8.4. It is assumed that the receiver is able to quantize the received sequence into the proper time segments, obtaining the signal shown in Fig. 8.5. Each segment

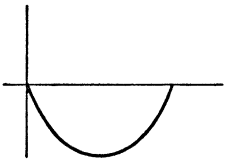


Fig. 8.1 Representation of 0.

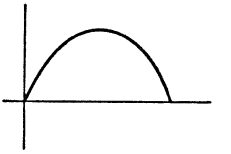


Fig. 8.2 Representation of 1.

of this signal is then demodulated into a 0 or a 1, whichever is more likely. The received sequence of Fig. 8.5 would be demodulated as 0001001. The demodulator would present this sequence to the decoder, which would then attempt to correct the symbol error in the fifth digit.

Although there are many possible choices of signals to represent the symbols 0 and 1, the particular choice of modulation scheme has no

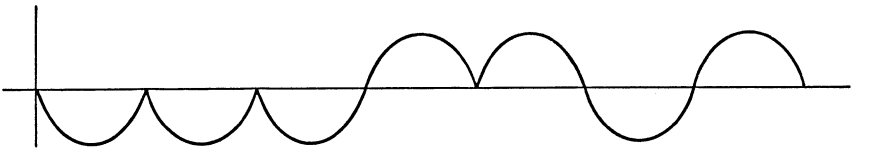


Fig. 8.3 Representation of 0001101.

Fig. 8.4 Noisy version of 0001101.

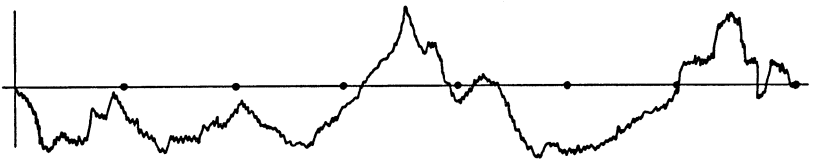


Fig. 8.5 Time-quantized version of noisy representation of 0001101.

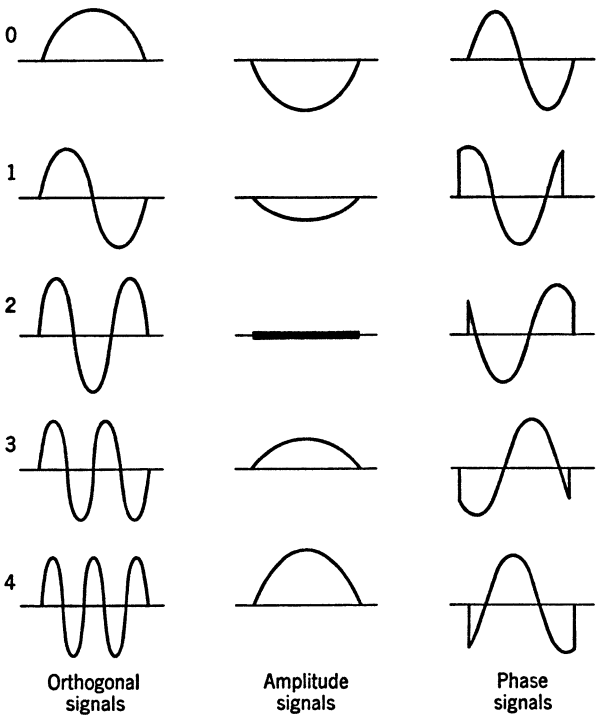


Fig. 8.6 Three modulation schemes for five-symbol alphabet.

real effect on the encoder and the decoder. They operate only on sequences of 0s and 1s, which the encoder feeds into the modulator and the decoder gets from the demodulator.

In the nonbinary case, there are again many modulation schemes possible. We briefly consider three such schemes, using an alphabet of five symbols: 0, 1, 2, 3, 4. These schemes are shown in Fig. 8.6.

If the orthogonal modulation scheme is used, the codeword 140343 would be represented by the signal shown in Fig. 8.7. As the transmitted signal travels across the channel, it is corrupted by the noise. The received signal might be the signal shown in Fig. 8.8. The receiver begins by demodulating the signal and re-encoding it into a sequence of symbols. If the noise is weak, the demodulator will have little trouble in reaching the correct decisions. In the above example, the demodulator will correctly demodulate the first symbol as 1 and the second symbol as 4. Occasionally, however, the noise will be more severe, and the demodulator will err. In the above example, the demodulator will probably decide that the third symbol is 1. This incorrect decision is called a

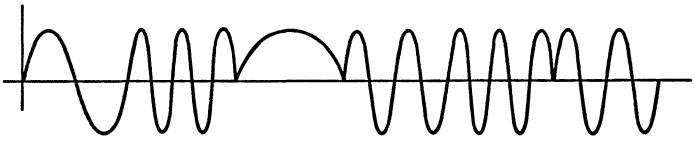


Fig. 8.7 Representation of 140343 using orthogonal modulation.

symbol error. Since the noise was weak during the fourth segment of time, the demodulator correctly decides that the fourth symbol was 3.

During the fifth time segment, the received signal is so weak that the demodulator can do no better than to make a feeble guess. Rather than force the demodulator to choose among equally bad alternatives, we might allow the demodulator to call this letter an *erasure*, which we denote by ?. In some cases we shall permit erasures; in others, we shall not. If erasures are not permitted, then the demodulator's forced guess would be 0. In either event, the last symbol would be correctly demodulated. Thus, the received symbol sequence would be either 1423?3 or 142303, depending on whether or not erasures are permitted. In either case, the decoder must attempt to correct the symbol errors and erasures. Further discussion of erasures is given by Forney (1966a).

In order to construct linear codes for nonbinary channels, it is necessary to introduce an arithmetic structure on the symbols of the input alphabet. If the number of symbols in this alphabet is a prime p or a prime power q, then this arithmetic structure may be taken as GF(p), or GF(q). As in the binary case, the transmitted and received sequences may be considered as vectors over this field. The difference between the received sequence and the transmitted sequence is again the noise vector. In the previous example over GF(5), the transmitted sequence was 140343; the received sequence was 1423?3 or 142303, so the difference sequence was 0020?0 or 002010. As in the binary case, a linear code may be defined as the set of vectors whose transposes lie in the null space of a parity-check matrix $\mathcal{H}$. The received word $\mathbf{R}$ has the syndrome $\mathbf{S}^t = \mathcal{H}\mathbf{R}^t$, which is the same as the syndrome of the error vector, $\mathbf{S}^t = \mathcal{H}\mathbf{E}^t$. The set of codewords is an additive group; the set

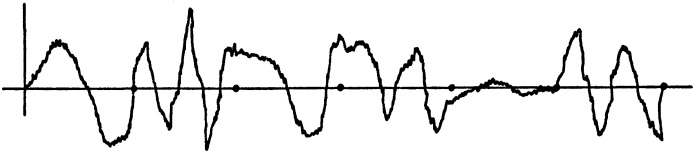


Fig. 8.8 Noisy version of orthogonal modulation of 140343.

of received words having any particular syndrome is a coset of this group. Given any particular received word, the set of possible error vectors is the coset containing the received word. The problem of the decoder is then this: given the syndrome of the received word, find the most probable error vector in the coset whose words have the given syndrome.

8.2 WEIGHT FUNCTIONS

How does the decoder choose the most probable of the possible error words which have a given syndrome? In the binary case, the answer is to select a word of minimum weight, where the weight of a word is defined as the number of ones among the word's n digits. In the non-binary case, as in the binary case, we may choose as coset leader a word of minimum weight. However, depending on the modulation scheme, there are many possible definitions of weight. We consider two such definitions, due to Hamming (1950) and Lee (1958).

Definition 8.21 The *Hamming weight* of a word $C_0, C_1, C_2, \ldots, C_{n-1}$; $C_i \in \mathrm{GF}(q)$, is defined as the sum of the Hamming weights of its digits, where

$$w_H(C_i) = \begin{cases} 0 & \text{if } C_i = 0 \\ 1 & \text{if } C_i \neq 0 \end{cases}$$

Definition 8.22 The *Lee weight* of a word $C_0, C_1, C_2, \ldots, C_{n-1}$; $C_i \in \mathrm{GF}(p)$, is defined as the sum of the Lee weights of its digits, where

$$w_L(C_i) = |C_i| \qquad |C_i| \equiv \pm C_i \bmod p \qquad \text{and} \qquad 0 \leq |C_i| \leq \frac{p}{2}$$

Both the Hamming weight and the Lee weight may also be defined over alphabets containing q letters even if q is not a prime or a prime power. However, difficulties in constructing cyclic (or, as we shall subsequently see, negacyclic) codes arise unless one can work in a finite field. To avoid these difficulties, we consider only alphabets having a number of symbols which is a prime power. For the Lee metric, the modular definition of the weight forces the code constructor to work mod q, rather than in $\mathrm{GF}(q)$. Since the integers mod q form a field iff q is a prime, we shall be able to construct Lee metric codes only for alphabets having a prime number of letters.

The Hamming weight is well suited to orthogonal modulation schemes. In fact, it can be shown that if the letters are modulated as orthogonal signals to which the channel adds white gaussian noise, then *all intersymbol errors are equally likely.* The probability of an error word, then, depends only on the number of nonzero digits in the error word and not on the values of these nonzero error digits. The Hamming metric faithfully weighs the candidate error words, assigning low weights to likely error words and high weights to unlikely error words.

The Lee metric is well suited to phase-modulation schemes. If the phase-modulated signals are transmitted through additive gaussian noise, then it is much more likely that the noise will corrupt the transmitted letter into another letter of only slightly different phase than into a letter of greatly different phase. Nonzero digits in the error word are much more likely to have a value of ± 1 than a value of ± 2. The Lee metric provides a good approximation; generally it assigns lower weights to the more probable error words.

For channels using amplitude modulation against additive gaussian noise, both the Hamming metric and the Lee metric have certain obvious flaws. The probability of confusing the highest amplitude and the lowest amplitude is much less than the probability of confusing two adjacent amplitudes near the middle of the alphabet. However, if the number of symbols in the alphabet is large, then the Lee metric provides a reasonable approximation. The approximation of the Hamming metric is much cruder.

For some channels, still other definitions of weight functions prove useful. However, we shall devote our primary attention to the Hamming metric and the Lee metric.

There are numerous things to be considered in the design of modulation and demodulation systems. One must choose the set of signals and decide on a demodulating rule. In general, this choice will depend critically upon the detailed characteristics of the channel. Noise power, noise spectra, frequency limitations on the transmitter, power limitations on the transmitter, and possible intersymbol interference must all be considered. A great deal of work has been done on these problems, but we shall not consider them any further here. The interested reader is referred to books by Davenport and Root (1958) or Wozencraft and Jacobs (1965). For purposes of understanding the subsequent parts of this book, only a survey knowledge of this area is absolutely necessary. We assume that the modulator, the demodulator, and the weight function (Hamming or Lee) are given and concentrate our attention on the problem of correcting the errors (or errors and erasures) at the demodulator by suitable encoding and decoding.

PROBLEM

8.1. [Gray codes†] Some communication systems operate as follows: First, a block of Rn p-ary source digits is encoded into $\mathbf{C} = [C_0, C_1, \ldots, C_{n-1}]$, a codeword in a good code of rate R and length n over GF(p). The n digits of this codeword are then partitioned into n/s consecutive subblocks, $\mathbf{C} = [\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n/s-1)}]$, where each $\mathbf{x}$ is a sequence of s elements from GF(p), $\mathbf{x} = [x_0, x_1, \ldots, x_{s-1}]$. For each i, $\mathbf{x}^{(i)}$ is then modulated into a single-level signal $y^{(i)} = f(\mathbf{x}^{(i)})$, where $f(\mathbf{x})$ is an integral-valued function assuming all values in the interval $0 \leq f \leq p^s - 1$. The levels $y^{(0)}, y^{(1)}, \ldots, y^{(n/s-1)}$ are transmitted across the additive noise channel. The received levels are then demodulated as $z^{(0)}, z^{(1)}, \ldots, z^{(n/s-1)}$, and converted into the s-digit p-ary sequences $\mathbf{v}^{(0)}, \mathbf{v}^{(1)}, \ldots, \mathbf{v}^{(n/s-1)}$ in such a way that $z^{(i)} = f(\mathbf{v}^{(i)})$. The received vector $\mathbf{R} = [\mathbf{v}^{(0)}, \mathbf{v}^{(1)}, \ldots, \mathbf{v}^{(n/s-1)}] = [R_0, R_1, \ldots, R_{n-1}]$ is then treated as a single word of length n over GF(p) and decoded by a decoding algorithm which succeeds whenever the Lee distance between $\mathbf{C}$ and $\mathbf{R}$ is sufficiently small.

We consider two choices for the function $f(\mathbf{x}) = \sum_{i=0}^{s-1} f_i(\mathbf{x}) p^i$, $0 \leq f_i(\mathbf{x}) < p$, namely

(a) $f_i(\mathbf{x}) = x_i$ for $i = 0, 1, \ldots, s-1$

(b) $f_i(\mathbf{x}) \equiv x_i - x_{i+1} \bmod p$ for $i = 0, 1, \ldots, s-2$
$f_{s-1}(\mathbf{x}) = x_{s-1}$

Show that if rule b is used, then the Lee distance between $\mathbf{x}$ and $\mathbf{v}$ is no greater than $|z - y|$, but that if rule a is used, the Hamming distance between $\mathbf{x}$ and $\mathbf{v}$ can be s even when $|z - y| = 1$.

† In Prob. 8.1b, $\mathbf{x}$ is a *Gray code* representation of the integer f.

9
Negacyclic Codes for the Lee Metric

In this chapter, we shall construct a class of codes which are able to correct all error patterns of sufficiently low Lee weight. Throughout this chapter, we assume that the channel input alphabet consists of the elements of an odd prime field GF(p). Although the reader may at first think that this assumption is overly restrictive, it appears to be necessary. Unlike the Hamming metric, the Lee metric is defined in a modular way which forces the code constructor to work in the ring of integers mod q and not in some other structure, such as GF(q). If the size of the input alphabet is not prime, this modular ring is not a field, and the constructions used in this chapter fail. We shall see later (Theorem 13.25) that in certain very special cases other constructions produce good codes over alphabets of composite orders.

9.1 ERROR LOCATIONS AND THE ERROR-LOCATOR POLYNOMIAL

Let us begin by considering the case of single errors. If the block length is n, then there are $2n$ possible single-error patterns, since we may have a

single Lee error of ± 1 in any of the n positions. Including the all-zero pattern of no errors, we see that there are $2n + 1$ different error patterns of Lee weight ≤ 1. If we wish to correct all these error patterns with a linear code containing r parity-check digits, then each of these $2n + 1$ error patterns must have a different pattern of parity-check failures, and $2n + 1 \leq p^r$, or $n \leq (p^r - 1)/2$.

Following the essential idea of the constructions of Bose-Chaudhuri and Hocquenghem, we next label each digit of the code with a nonzero element in some extension field of GF(p). The previous inequality suggests that we label each digit of the code with *two different* error location numbers from GF(p^r). Upon considering the definition of the Lee metric, and our desire to correct an error of ± 1 in each of the $n = (p^r - 1)/2$ positions, we assign the two location numbers $\pm\alpha^{j-1}$ to the jth digit of the code, where α is a primitive element of GF(p^r). Since $\alpha^n = -1$, no two different positions have a common location number.

As one of the simplest nontrivial examples, we take $p = 5$, $r = 2$, $n = (5^2 - 1)/2 = 12$. The 24 nonzero elements of GF(25) may be represented as the successive powers of α, where α is a root of the primitive quadratic $x^2 + x + 2$. Alternatively, every element of GF(25) may be represented in the form $A_1\alpha + A_0$, where A_1 and $A_0 \in$ GF(5). The relations between these representations are given in Appendix B. We label the 12 digits of the code as follows:

	Code digit position											
	1	2	3	4	5	6	7	8	9	10	11	12
Positive	α^0	α^1	α^2	α^3	α^4	α^5	α^6	α^7	α^8	α^9	α^{10}	α^{11}
location	0	1	−1	−1	−2	−1	0	2	−2	−2	1	−2
number	1	0	−2	2	2	−1	2	0	1	−1	−1	−2
Negative	α^{12}	α^{13}	α^{14}	α^{15}	α^{16}	α^{17}	α^{18}	α^{19}	α^{20}	α^{21}	α^{22}	α^{23}
location	0	−1	1	1	2	1	0	−2	2	2	−1	2
number	−1	0	2	−2	−2	1	−2	0	−1	1	1	2

We now claim that any error pattern of Lee weight t may be specified by giving the locations of the t different errors or by giving the polynomial $\sigma(z)$ whose reciprocal roots are these errors. As examples, we consider these error patterns:

$E(x)$	X_1	X_2	X_3	$\sigma(z)$
x^4	α^4			$1 - \alpha^4 z$
$-x^8$	$-\alpha^8 = \alpha^{20}$			$1 - \alpha^{20} z$
$x^3 + x^7$	α^3	α^7		$(1 - \alpha^3 z)(1 - \alpha^7 z)$
$x^5 - x^9$	α^5	$-\alpha^9 = \alpha^{21}$		$(1 - \alpha^5 z)(1 - \alpha^{21} z)$
$2x^6$	α^6	α^6		$(1 - \alpha^6 z)^2$
$-x^7 - 2x^{10}$	$-\alpha^7 = \alpha^{19}$	$-\alpha^{10} = \alpha^{22}$	α^{22}	$(1 - \alpha^{19} z)(1 - \alpha^{22} z)^2$

The crucial property of the error location numbers $X_1, X_2, \ldots$ is that

$$E(\alpha^j) = \sum_i X_i^j = S_j \qquad \text{for all odd } j$$

The reader who is already familiar with the Gorenstein-Zierler Hamming-metric extension of the BCH codes must be warned that *no error values* are used in our negacyclic codes for the Lee metric. A multiple error in some position is evidenced by a multiple root of the error-locator polynomial. We then have Theorem 9.11.

Theorem 9.11 Each distinct error pattern of Lee weight t corresponds to a distinct error-locator polynomial $\sigma(z)$ whose degree is t.

With appropriate restrictions, the converse is also true.

Theorem 9.12 An error polynomial $\sigma(z)$ of degree t corresponds to an error pattern of Lee weight t if all reciprocal roots of $\sigma(z)$ are $2n$th roots of unity and no root has multiplicity greater than $(p-1)/2$ and no two reciprocal roots of $\sigma(z)$ sum to 0.

9.2 DOUBLE-ERROR-CORRECTING CODES

Let us now construct a code of block length 12 over GF(5) which corrects double errors in the Lee metric. As shown in Appendix B, we may represent the elements of GF(5^2) in terms of α, a root of $x^2 + x + 2$. Following the BCH argument in a heuristic manner, we take the first two rows of the parity-check matrix as the positive-digit location numbers and the second two rows as the cubes of the first two rows:

$$\mathfrak{IC} = \begin{bmatrix} 0 & 1 & -1 & -1 & -2 & -1 & 0 & 2 & -2 & -2 & 1 & -2 \\ 1 & 0 & -2 & 2 & 2 & -1 & 2 & 0 & 1 & -1 & -1 & -2 \\ 0 & -1 & 0 & -2 & 0 & 1 & 0 & 2 & 0 & -1 & 0 & -2 \\ 1 & 2 & 2 & -1 & -1 & -2 & -2 & 1 & 1 & 2 & 2 & -1 \end{bmatrix}$$

The codewords are chosen to satisfy all four of these parity-check equations. A codeword is transmitted, and noise is added. From the first two parity-check equations (the top two rows of the above matrix), the decoder may deduce the sum of the error locations, $S_1 = \Sigma X_i$; from the bottom two rows of the above equations, the decoder may deduce the sum of the cubes of the error locations, $S_3 = \Sigma X_i^3$. If there are no

more than two errors, we have

$$S_1 = X_1 + X_2 \neq 0 \qquad \text{unless } X_1 = X_2 = 0$$
$$S_3 = X_1^3 + X_2^3$$
$$\frac{S_3}{S_1} = X_1^2 - X_1X_2 + X_2^2$$
$$\frac{S_3}{S_1} - S_1^2 = -3X_1X_2$$
$$X_1X_2 = \frac{S_1^3 - S_3}{3S_1}$$

$$\boxed{\sigma(z) = 1 - S_1z + \frac{S_1^3 - S_3}{3S_1} z^2} \tag{9.21}$$

Thus, this code is capable of correcting double errors. To decode, we compute S_1 and S_3 from the parity-check equations and then perform the necessary arithmetic calculations in GF(5^2) to find the error polynomial. If α^j is a reciprocal root of this polynomial, and $0 \leq j < n$, then there is an error of $+1$ in the $(j + 1)$st digit of the received word. If α^j is a reciprocal root of the error-locator polynomial, and $n \leq j < 2n$, then there is an error of -1 in the $(j + 1 - n)$th digit of the received word. A double error in any position of the code manifests itself in a double reciprocal root of the error-locator polynomial. For these double-error-correcting codes, the quadratic error-locator polynomial

$$1 - S_1z + \frac{S_1^3 - S_3}{3S_1} z^2$$

has repeated roots iff

$$S_1^2 = 4\,\frac{S_1^3 - S_3}{3S_1} \qquad \text{or} \qquad 4S_3 = S_1^3$$

This condition also follows from the equations $S_1 = 2X_1$ and $S_3 = 2X_1^3$. Similarly, it may be seen that there is one error only if $(S_1^3 - S_3)/3S_1 = 0$ and zero errors only if $S_1 = 0$.

The reader may wonder why we selected the bottom two rows of the parity-check matrix to be the cubes of the first two rows rather than the squares. If instead one selects the squares, then one does *not* get the desired equations

$$\begin{aligned} X_1 + X_2 &= S_1 \\ X_1^2 + X_2^2 &= S_2 \end{aligned} \tag{9.22}$$

but instead one gets the formidable-looking equations

$$\begin{aligned} X_1 + X_2 &= S_1 \\ X_1|X_1| + X_2|X_2| &= S_2 \end{aligned} \tag{9.23}$$

Here $X_i = \pm X_i$, accordingly as $\log_\alpha X_i$ is between 0 and $n-1$ or between n and $2n-1$. The difficulty arises because $(+X_i)^2 = (-X_i)^2 \neq (-X_i^2)$. A similar problem arises if one includes any even power of the error location numbers as rows of the parity-check matrix. Although this may not necessarily represent a bad choice, it results in the formidable-looking equations which we prefer to avoid.

Our actual choice of the $\mathcal{H}$ matrix has another nice mathematical property which we shall now investigate. Since the successive columns of the first two rows of this matrix represent successive powers of α, the codeword polynomial $C(x)$, of degree <12, satisfies these parity-check equations iff $C(\alpha) = 0$. This can happen iff the code polynomial $C(x)$ is a multiple of the minimal polynomial of α, which is $x^2 + x + 2$. Similarly, the code polynomial satisfies the last two parity checks iff $C(\alpha^3) = 0$, which can happen iff $C(x)$ is a multiple of the minimal polynomial of α^3, which is $x^2 - 2$. Evidently, then, the polynomial $C(x)$ of degree <12 represents a codeword iff $C(x)$ is a multiple of the product, $(x^2 + x + 2)(x^2 - 2)$. Since α is a primitive element of GF(25), $\alpha^{24} = 1$, but $\alpha^k \neq 1$ for any k less than 24. Since $(\alpha^{12})^2 = 1$ and the only two square roots of 1 are ± 1, it is evident that $\alpha^{12} = -1$. A similar argument reveals that $(\alpha^j)^{12} = -1$ for every odd j, including $j = 3$. Thus, both the minimal polynomial of α (namely, $x^2 + x + 2$) and the minimal polynomial of α^3 (namely, $x^2 - 2$) must be divisors of $x^{12} + 1$. Therefore, if $C(x)$ is a codeword represented by

$$C(x) = C_0 + C_1 + C_2x^2 + \cdots + C_{10}x^{10} + C_{11}x^{11}$$

then $C(x)$ is a multiple of $(x^2 + x + 2)(x^2 - 2)$, and so is $xC(x) - C_{11}(x^{12} + 1)$, which is $-C_{11} + C_0x + C_1x^2 + \cdots + C_9x^{10} + C_{10}x^{11}$. For this reason, we call this code a negacyclic code.

9.3 NEGACYCLIC CODES

Definition 9.31 A *negacyclic code* of block length n over GF(p) (p an odd prime and n a nonmultiple of p) is the set of multiples of a *generator polynomial* $g(x)$ which divides $x^n + 1$ over GF(p). The quotient $(x^n + 1)/g(x)$ is called the *check polynomial* $h(x)$.

Since $x^n + 1 = (x^{2n} - 1)/(x^n - 1)$, the roots of $x^n + 1$ are the roots of $x^{2n} - 1$ which are not roots of $x^n - 1$. If α is a primitive root of $x^{2n} - 1$, then the even powers of α are roots of $x^n - 1$, and the odd powers of α are roots of $x^n + 1$. We repeat, the roots of $x^n + 1$ are the odd powers of a primitive $2n$th root of unity. Therefore, both the generator polynomial and the check polynomial of any negacyclic code of block length n may be conveniently described in terms of their roots, which are

odd powers of a single primitive $2n$th root of unity. For the double-error-correcting code of block length 12 over GF(5), which we discussed in the previous section, the roots of the generator polynomial are α, α^3, and their quinary conjugates, α^5 and $(\alpha^3)^5 = \alpha^{15}$. The other eight odd powers of α, namely, α^7, α^9, α^{11}, α^{13}, α^{17}, α^{19}, α^{21}, and α^{23}, are roots of the check polynomial.

This method of describing negacyclic codes is important because it immediately reveals the equations which must be solved in order to find the error locator. If α^j is a root of the generator polynomial, then the decoder may immediately compute the sum of the jth powers of the error locations by evaluating the received polynomial, $R(x) = C(x) + E(x)$ at $x = \alpha^j$, obtaining $R(\alpha^j) = 0 + E(\alpha^j) = \sum_i X_i{}^j = S_j$. Given these S_j, the decoder must then attempt to solve for the error locator $\sigma(z)$ whose reciprocal roots give the error locations. The relation between $\sigma(z)$ and the S's is given by Newton's identities, which may be readily derived as follows. Let

$$\sigma(z) = \prod_i (1 - X_i z) \qquad X_i \text{ not necessarily distinct}$$

Then

$$\sigma'(z) = -\sum_i X_i \prod_{j \neq i} (1 - X_j z)$$

and

$$-\frac{z\sigma'(z)}{\sigma(z)} = \sum_i \frac{X_i z}{1 - X_i z} = \sum_i \sum_{k=1}^{\infty} (X_i z)^k$$

$$= \sum_{k=1}^{\infty} \left(\sum_i X_i{}^k \right) z^k = \sum_{k=1}^{\infty} S_k z^k = S(z)$$

We thus have Newton's identities in generating-function notation,

$$S\sigma + z\sigma' = 0 \tag{9.32}$$

In negacyclic codes, all the coefficients of the even powers of z in the generating function for S are initially unknown. For this reason, it is helpful to eliminate these terms from the equations by separating Newton's identities into their even and odd parts.

Letting

$$\hat{\sigma} = 1 + \sigma_2 z^2 + \sigma_4 z^4 + \cdots$$
$$\check{\sigma} = \sigma_1 z + \sigma_3 z^3 + \cdots$$
$$\hat{S} = S_2 z^2 + S_4 z^4 + \cdots$$
$$\check{S} = S_1 z + S_3 z^3 + \cdots$$

Newton's identities become

$$(\hat{S} + \check{S})(\check{\sigma} + \hat{\sigma}) + z(\check{\sigma} + \hat{\sigma})' = 0$$

which may be broken up into the two equations

$$\hat{S}\hat{\sigma} + \check{S}\check{\sigma} + z\hat{\sigma}' = 0$$

and

$$\hat{S}\check{\sigma} + \check{S}\hat{\sigma} + z\check{\sigma}' = 0$$

Subtracting $\hat{\sigma}$ times the latter equation from $\check{\sigma}$ times the former gives

$$\check{S}(\check{\sigma}^2 - \hat{\sigma}^2) + z(\check{\sigma}\hat{\sigma}' - \hat{\sigma}\check{\sigma}') = 0 \tag{9.33}$$

Under sufficiently restrictive circumstances, Eq. (9.33) can be solved. The major result is Theorem 9.34.

Theorem 9.34 If the roots of the generator polynomial of a negacyclic code of block length un over GF(p) include $a, a^3, a^5, \ldots, a^{2t-1}$, where $2t - 1 < p$ and a is a primitive $2n$th root of unity, then that negacyclic code is capable of correcting all error patterns of Lee weight $\leq t$.

Remarks Before proving this theorem, we present Table 9.1, which lists the parameters of some of the codes which this theorem promises. The codes which have relatively large t and a relatively small number of check digits r usually come in primitive block lengths of the form $n = (p^m - 1)/2$. Nevertheless, there are also moderately good nonprimitive codes of this type having other block lengths; these codes are marked in the table by a dagger.

Proof of Theorem 9.34 Our proof is constructive; it consists of an efficient decoding procedure.
We begin with the equation

$$\check{S}(\check{\sigma}^2 - \hat{\sigma}^2) = z(\hat{\sigma}\check{\sigma}' - \check{\sigma}\hat{\sigma}')$$

Since $\hat{\sigma}(0) = 1$, we may divide through by $\hat{\sigma}^2$ to obtain

$$\check{S}\left(\left(\frac{\check{\sigma}}{\hat{\sigma}}\right)^2 - 1\right) = z\left(\frac{\hat{\sigma}\check{\sigma}' - \check{\sigma}\hat{\sigma}'}{\hat{\sigma}^2}\right) = z\left(\frac{\check{\sigma}}{\hat{\sigma}}\right)'$$

Table 9.1 Values of parameters of some of the negacyclic codes promised by Theorem 9.34

		Error correction									
		1	2	3	4	5					
p	n	r									
5	2	1	2								
5	6†	2	3								
5	12	2	4								
5	62	3	6								
5	312	4	8								
7	3	1	2	3							
7	24	2	4	6							
7	171	3	6	9							
11	5	1	2	3	4	5					
11	15†	2	3	5	7	8					
11	60	2	4	6	8	10					
11	665	3	6	9	12	15					
17	8	1	2	3	4	5	6	7	8		
17	24†	2	3	5	7	8	10	12	13		
17	72†	2	4	6	8	9	11	13	15		
17	144	2	4	6	8	10	12	14	16		
127	63	1	2	3	4	...				62	63
127	8064	2	4	6	8	...				124	126

† Nonprimitive; $n \neq (p^m - 1)/2$.

Introducing the generating function

$$U = \frac{\tilde{\sigma}}{\hat{\sigma}}$$

we have

$$\tilde{S}(U^2 - 1) = zU'$$

or

$$U = \int \frac{1}{z} \tilde{S}(-1 + U^2)\, dz$$

Although this equation may look formidable, its solution is, in fact, trivial, because each coefficient of U is specified in terms of

certain coefficients of $\tilde{S}$ and previously computed coefficients of U

$$U_1 z + U_3 z^3 + U_5 z^5 + \cdots = \int \frac{S_1 z + S_3 z^3 + S_5 z^5 + \cdots}{z} \\ [-1 + (U_1 z + U_3 z^3 + \cdots)^2]\, dz$$

$$U_1 = -S_1$$

$$U_3 = \frac{-S_3 + U_1^2 S_1}{3}$$

$$U_5 = \frac{-S_5 + U_1^2 S_3 + 2U_1 U_3 S_1}{5}$$

. . . . . .

Difficulty is encountered if we attempt to compute the coefficient U_p, since this would require dividing by the integer p, which is zero in GF(p). However, under the hypotheses of the theorem, $2t - 1 < p$, and no such difficulty arises if we compute only U_1, U_3, U_5, . . . , U_{2t-1}. Since U is an odd function of z, we may define the generating function T by the equation $T(z^2) = [1 + zU(z)]^{-1} - 1$. It is evident that $T(0) = 0$, and that $1 + T(z^2) = [1 + zU(z)]^{-1}$. Knowing the coefficients of U_1, U_3, . . . , U_{2t-1}, this equation enables us to compute recursively the coefficients T_1, T_2, . . . , T_t. Since $U = \tilde{\sigma}/\hat{\sigma}$ and $1 + zU = (\hat{\sigma} + z\tilde{\sigma})/\hat{\sigma}$, we define the polynomials

$$\omega(z^2) = \hat{\sigma}(z) \qquad \Phi(z^2) = \hat{\sigma}(z) + z\tilde{\sigma}(z)$$

It is evident that

$$1 + zU(z) = \frac{\Phi(z^2)}{\omega(z^2)}$$

and that

$$1 + T(z^2) = \frac{\omega(z^2)}{\Phi(z^2)}$$

so

$$[1 + T(z)]\Phi(z) \equiv \omega(z) \bmod z^{t+1} \tag{9.35}$$

We further claim that if there are no more than t errors, then Φ and ω satisfy the additional conditions

$$\Phi(0) = \omega(0) = 1 \qquad \deg \Phi \leq \frac{1+t}{2} \qquad \deg \omega \leq \frac{t}{2}$$

and Φ and ω are relatively prime.

The last observation follows from the fact that no two reciprocal roots of σ may sum to zero. Therefore, σ can have no even

factors of positive degree. This implies that $\hat{\sigma}$ and $\check{\sigma}$ are relatively prime, as are Φ and ω.

In view of these conditions, Eq. (9.35) can be solved for Φ and ω by Algorithm 7.4. The solution is evidently given by

$$\Phi = \Phi^{(t)} \qquad \omega = \omega^{(t)}$$

where $\Phi^{(t)}$ and $\omega^{(t)}$ are generated by Algorithm 7.4. We may summarize the decoding procedure as Algorithm 9.36.

Algorithm 9.36 for decoding negacyclic codes of Theorem 9.34

1. Compute U mod z^{2t} from the equation

$$U = \int \frac{S}{z}(U^2 - 1)\, dz$$

2. Compute $1 + T$ mod z^{t+1} from the equation

$$1 + T(z^2) = [1 + zU(z)]^{-1}$$

3. Use Algorithm 7.4 to find

$$\Phi^{(0)}, \omega^{(0)}, \tau^{(0)}, \gamma^{(0)}, \Phi^{(1)}, \omega^{(1)}, \tau^{(1)}, \gamma^{(1)}, \ldots, \Phi^{(t)}, \omega^{(t)}$$

which solve the equations $(1 + T)\Phi^{(t)} \equiv \omega^{(t)} \bmod z^{t+1}$.

4. Set $\hat{\sigma}(z) = \omega^{(t)}(z^2)$; $\check{\sigma}(z) = [\Phi^{(t)}(z^2) - \omega^{(t)}(z^2)]/z$; $\sigma = \hat{\sigma} + \check{\sigma}$.
5. Using a Chien search, evaluate the polynomials $\hat{\sigma}(\alpha^{-j})$ and $\check{\sigma}(\alpha^{-j})$ for $j = n - 1, \ldots, 1, 0$, where α is the primitive $2n$th root of unity whose successive powers give the code's successive location numbers. If $\hat{\sigma}(\alpha^{-j}) = -\check{\sigma}(\alpha^{-j})$, there is a positive error in the code's $(j - 1)$st position; if $\hat{\sigma}(\alpha^{-j}) = +\check{\sigma}(\alpha^{-j})$, there is a negative error in the code's $(j - 1)$st position. In either case, the multiplicity of the error may be determined by evaluating the derivatives of $\hat{\sigma}$ and $\check{\sigma}$. If

$$\left[\frac{d^{(i)}}{dz^{(i)}}\hat{\sigma}(z)\right]_{z=\alpha^{-j}} = \pm\left[\frac{d^{(i)}}{dz^{(i)}}\check{\sigma}(z)\right]_{z=\alpha^{-j}} \qquad \text{for } i < k$$

but

$$\left[\frac{d^{(k)}}{dz^{(k)}}\hat{\sigma}(z)\right]_{z=\alpha^{-j}} \neq \pm\left[\frac{d^{(k)}}{dz^{(k)}}\check{\sigma}(z)\right]_{z=\alpha^{-j}}$$

then the error has multiplicity k. In these equations, the negative signs are used to determine the multiplicity of a positive error, and vice versa.

If high speed is required and computing registers are plentiful, then one may perform Chien searches on all the polynomials $\hat{\sigma}$, $\bar{\sigma}$, $\hat{\sigma}'$, $\bar{\sigma}'$, $\hat{\sigma}''$, $\bar{\sigma}''$, . . . , $d^{(t-1)}\bar{\sigma}/dz^{(t-1)}$.

On the other hand, if computing registers are scarce but time is available, one should perform Chien searches on only polynomials $\hat{\sigma}$ and $\bar{\sigma}$. By judicious programming it is possible to calculate the derivatives only when they are needed, without interrupting the Chien searches on $\hat{\sigma}(\alpha^{-j})$ and $\bar{\sigma}(\alpha^{-j})$.

Although Theorem 9.34 gives us a powerful class of negacyclic codes, together with a practical decoding algorithm, there is a strong desire to remove the restriction that $t \leq (p-1)/2$. One may wish to correct more than $(p-1)/2$ errors. The obvious conjecture for doing this is the code whose roots include further successive odd powers of α. If $\alpha, \alpha^3, \ldots, \alpha^{p-2}, \alpha^p$ prove insufficient, then would you believe that $\alpha, \alpha^3, \alpha^5, \ldots, \alpha^p, \alpha^{p+2}$ might work? Unfortunately, this attempt does not work. The reason for this failure will be revealed by Theorem 11.65.

PROBLEMS

9.1. Show that steps 1 and 2 of Algorithm 9.36 may be combined, and that $T(z^2) = V(z)$ may be calculated directly from $\tilde{S}(z)$ by solving the equation

$$z^2V' - zV = zV^2 + \tilde{S}[(z^2-1)V^2 + 2z^2V + z^2]$$

9.2. Tabulate all 30 powers of 3 in GF(31). With $M^{(i)}(x) = x - 3^i$, we consider the 4-Lee-error-correcting negacyclic code of length 15 over GF(31) generated by $g(x) = M^{(1)}(x)M^{(3)}(x)M^{(5)}(x)M^{(7)}(x) = x^4 + 20x^3 + 15x^2 + 28x + 28$. Assuming there are no more than 4 errors, decode the received sequence $\mathbf{R}$ = [20,20,11,5,16,9,14,22,19,23, 5,17,1,6,15].

Answers

$$\begin{aligned}
\tilde{S}(z) &= 14z + 30z^3 + 6z^5 + 11z^7 + \cdots \\
U(z) &= 17z + 16z^3 + 1z^5 + 0z^7 + \cdots \\
1 + T(z) &= 1 + 14z + 25z^2 + 1z^3 + 3z^4 + \cdots \\
&= 1 + 3^{22}z + 3^{10}z^2 + 3^0z^3 + 3^1z^4 + \cdots \\
\Phi^{(1)} &= 1 + 3^7z \\
\Phi^{(2)} &= 1 + 3^3z \\
\Phi^{(3)} &= 1 + 3^{21}z + 3^{11}z^2 \\
\Phi^{(4)} &= 1 + 3^{20}z + 3^{17}z^2 \\
\omega^{(4)} &= 1 + 19z + 24z^2 \\
\sigma(z) &= 1 + 17z + 19z^2 + 29z^3 + 24z^4 \\
&= (1 - 3^3z)(1 + 3^7z)(1 + 3^9z)^2
\end{aligned}$$

$$\mathbf{E} = [\,0,\ 0,\ 0,\ 1,\ 0,\ 0,\ 0,\ -1,\ 0,\ -2,\ 0,\ 0,\ 0,\ 0,\ 0]$$

$$\mathbf{C} = [20,\ 20,\ 11,\ 4,\ 16,\ 9,\ 14,\ 23,\ 19,\ 25,\ 5,\ 17,\ 1,\ 6,\ 15]$$

10
Gorenstein-Zierler Generalized Nonbinary BCH Codes for the Hamming Metric

10.1 THE GENERALIZED BCH CODES AND AN ALGORITHM TO DECODE THEM

If q is any prime power, we may define cyclic codes over GF(q) in the same way that we defined cyclic codes over GF(2). Some factor of $x^n - 1$ over GF(q) is taken as the generator polynomial. The codewords consist of all multiples of the generator polynomial which have degree $< n$. As in the binary case, the generator polynomial of the t-error-correcting q-ary BCH code of block length n is the product of the distinct minimal polynomials of $\alpha, \alpha^2, \alpha^3, \ldots, \alpha^{2t}$, where α is a primitive nth root of unity over GF(q). $\alpha \in \mathrm{GF}(q^m)$, where m is the multiplicative order of q mod n. GF(q) is called the *symbol field;* GF(q^m) is called the *locator field.*

This generalization of BCH codes to the nonbinary case was first introduced by Gorenstein and Zierler (1961). The special case $n = q - 1$ had been previously introduced by Reed and Solomon (1960) from another point of view.

If the encoder transmits the codeword whose successive digits are the coefficients of the polynomial

$$C(x) = C_0 + C_1x + C_2x^2 + \cdots + C_{n-1}x^{n-1} \qquad C_i \in \mathrm{GF}(q)$$

and the channel noise adds to this an error word given by

$$E(x) = E_0 + E_1x + E_2x^2 + \cdots + E_{n-1}x^{n-1} \qquad E_i \in \mathrm{GF}(q)$$

then the decoder receives the word

$$R(x) = C(x) + E(x)$$

Let α be a primitive nth root of unity over GF(q). If α^k is a root of $C(x)$, then $R(\alpha^k) = E(\alpha^k)$. If the error word consists of an error of value Y_1 at location $X_1 = \alpha^{j_1}$ and an error of value Y_2 at location $X_2 = \alpha^{j_2}$, . . . , then $E(x) = \sum_i Y_i x^{j_i}$ and $E(\alpha^k) = \sum_i Y_i \alpha^{kj_i} = \sum_i Y_i X_i^k = S_k$. Here the X_i are called the *error locations;* the Y_i are called the *error values;* and the S_k are called the *weighted power-sum symmetric functions.*

The $Y_i \in \mathrm{GF}(q)$, but the X_i and the $S_i \in \mathrm{GF}(q^m)$. A t-error-correcting BCH code is constructed in such a way that α, α^2, α^3, . . . , α^{2t} are roots of its generating polynomial,† and therefore they are also roots of every codeword. Thus, as the first step of our BCH decoding procedure, we may calculate $S_k = E(\alpha^k)$ for $k = 1, 2, \ldots, 2t$.

In order to locate and evaluate the errors, it is helpful to consider two polynomials. The first, called the *error locator* $\sigma(z)$, is the polynomial whose reciprocal roots are the error locations

$$\sigma(z) = \prod_i (1 - X_i z) \tag{10.11}$$

The second, called the *error evaluator* $\omega(z)$, is defined by

$$\boxed{\omega(z) = \sigma(z)(+) \sum_i z X_i Y_i \prod_{j \neq i} (1 - X_j z)} \tag{10.12}$$

Notice that Definitions 10.11 and 10.12 reduce to Definitions 7.21 and 7.22 in the binary case. Unlike the error-locator polynomial for the negacyclic codes of Chap. 9, the error-locator polynomial of Eq. (10.11) can never have repeated roots. However, the degree of the error-locator polynomial defined by Eq. (10.11) is still equal to the number of (Hamming-metric) errors, just as the degree of the error-locator polynomial of Chap. 9 was equal to the number of (Lee-metric) errors and the degree of the error polynomial defined by Eq. (7.21) was equal to the number of binary errors. Notice also that the error-evaluator polynomial defined by Eq. (10.12) depends both on the locations and the values of the errors, although the error locator defined by Eq. (10.11) depends *only* on the locations of the errors.

† The alternate, more general, definition α^l, α^{l+1}, . . . , α^{l+2t-1} for $l \neq 1$ is covered in Sec. 10.3.

Since $\sigma(0) = 1$, the generating function for the quotient $\omega(z)/\sigma(z)$ is well defined. Its coefficients are given by

$$\frac{\omega}{\sigma} = 1 + \sum_i \frac{zX_iY_i}{1 - X_iz}$$
$$= 1 + \sum_{k=1} \left(\sum_i Y_iX_i^k \right) z^k = 1 + S(z)$$

where we have defined the generating function $S(z)$ by $S(z) = \sum_{k=1}^{\infty} S_kz^k$. According to the above calculations, $\omega/\sigma = 1 + S$, and

$$\boxed{(1 + S)\sigma = \omega}$$

For the t-error-correcting BCH code, the decoder is able to calculate only $S_1, S_2, \ldots, S_{2t}$. Thus, only the first $2t$ coefficients of the generating function for $S(z)$ are known, and the decoder must attempt to find σ and ω from the key equation

$$\boxed{(1 + S)\sigma \equiv \omega \bmod z^{2t+1}} \tag{10.13}$$

The decoder can solve this equation for the polynomials $\sigma(z)$ and $\omega(z)$ by using Algorithm 7.4. This is the second step of our decoding procedure.

Once the error locator $\sigma(z)$ is known, the decoder may locate the errors by finding its reciprocal roots $\sigma(z) = \prod_i (1 - X_iz)$. The solution of the equation for the X's, given $1, \sigma_1, \sigma_2, \sigma_3, \ldots, \sigma_{\deg \sigma}$ [the coefficients of $\sigma(z) = \Sigma\sigma_jz^j$] is the third step of the decoding procedure.

After the errors have been located, the decoder can evaluate the polynomial $\omega(z)$ at the point $z = X_i^{-1}$, obtaining

$$\omega(X_i^{-1}) = Y_i \prod_{j \neq i} (1 - X_jX_i^{-1})$$

Thus, as the fourth step of the decoding procedure, the decoder can evaluate the errors according to the formula

$$Y_i = \frac{\omega(X_i^{-1})}{\prod_{j \neq i} (1 - X_jX_i^{-1})} = \frac{X_i^{(\deg \sigma)}\omega(X_i^{-1})}{X_i \prod_{j \neq i} (X_i - X_j)} = \frac{\tilde{\omega}(X_i)}{X_i \prod_{j \neq i} (X_i - X_j)} \tag{10.14}$$

†

The accent $\sim$, which denotes the reciprocal polynomial, should not be confused with the accent $\backsim$, which I have used to denote the odd part.

†This usage of $\tilde{\omega}$ differs from the reciprocal polynomial of ω by a power of x when $\deg \omega < \deg \sigma$ (i.e. when $\sum Y_i = 1$).

We may summarize our decoding procedure for nonbinary BCH codes as Algorithm 10.15.

Decoding algorithm 10.15

10.151. Find the weighted power-sum symmetric functions S_1, S_2, . . . , S_{2t}. For $k = 1, 2, 3, \ldots, 2t$, S_k may be found from the formula $S_k = r^{(k)}(\alpha^k)$, where $r^{(k)}(x)$ is the remainder of the received polynomial $R(x)$ divided by the minimal polynomial of α^k over GF(q), $M^{(k)}(x)$.

10.152. Knowing the generating function for $S(z)$ mod z^{2t+1}, use Algorithm 7.4 to solve the key equation (10.13) for the polynomials $\sigma^{(2t)}(z)$ and $\omega^{(2t)}(z)$.

10.153. Using a Chien search over the nth roots of unity over GF(q), find the reciprocal roots of the error-locator polynomial $\sigma(z)$. These are the error locations.

10.154. Find the error values from Eq. (10.14).

10.2 EXAMPLES

Take a code of block length $n = 15$ over GF(2^2), whose syndrome gives S_1, S_2, S_3, and $S_4 = S_1^4$. We have

$q = 4$ symbols per alphabet letter
$n = 15 = 4^2 - 1$ block length
$t = 2$ guaranteed error correction
$k = 9$ information digits
$r = 6$ check digits

Let ξ be a primitive element in GF(2^2) which satisfies $\xi^2 + \xi + 1 = 0$. Let α be a primitive element of GF(2^4) which satisfies $\alpha^4 + \alpha + 1 = 0$ over GF(2) or $\alpha^2 + \alpha + \xi = 0$ over GF(4). $\xi = \alpha^5$. All the elements of GF(2^4) may be represented as powers of α, as in Table 4.1.

Example 10.21

$1 + S = 1 + \alpha^3 z + \alpha^6 z^2 + \alpha^4 z^3 + \alpha^{12} z^4 + \cdots$

k	$D(k)$	$B(k)$	$\sigma^{(k)}$	$\tau^{(k)}$	$\omega^{(k)}$	$\gamma^{(k)}$	$\Delta_1^{(k)}$
0	0	0	1	1	1	0	α^3
1	1	1	$1+\alpha^3 z$	$\alpha^{-3} z$	1	α^{-3}	0
2	1	1	$1+\alpha^3 z$	$\alpha^{-3} z$	1	$\alpha^{-3} z$	α^{-1}
3	2	0	$1+\alpha^3 z+\alpha^{11} z^2$	$\alpha(1+\alpha^3 z)$	$1+\alpha^{11} z^2$	α	0
4	2	0	$1+\alpha^3 z+\alpha^{11} z^2$	$\alpha(1+\alpha^3 z)$	$1+\alpha^{11} z^2$	αz	?

The factorization of $\sigma^{(4)}$ is given by

$$\sigma^{(4)} = (1 + \alpha^4 z)(1 + \alpha^7 z)$$

so

$$X_1 = \alpha^4 \qquad X_2 = \alpha^7$$

$$\tilde{\omega}^{(4)} = z^2 + \alpha^{11} \qquad Y_1 = \frac{(\alpha^4)^2 + \alpha^{11}}{\alpha^4(\alpha^4 + \alpha^7)} = \frac{\alpha^7}{\alpha^4\alpha^3} = 1$$

$$Y_2 = \frac{(\alpha^7)^2 + \alpha^{11}}{\alpha^7(\alpha^4 + \alpha^7)} = \frac{\alpha^{10}}{\alpha^7(\alpha^3)} = 1$$

Example 10.22

$$1 + S = 1 + \alpha^{11}z + \alpha^{10}z^2 + \alpha^5 z^3 + \alpha^{14}z^4 + \cdots$$

k	$D(k)$	$B(k)$	$\sigma^{(k)}$	$\tau^{(k)}$	$\omega^{(k)}$	$\gamma^{(k)}$	$\Delta_1{}^{(k)}$
0	0	0	1	1	1	0	α^{-4}
1	1	1	$1 + \alpha^{11}z$	α^4	1	α^4	α^6
2	1	0	$1 + \alpha^{14}z$	$\alpha^{-6}(1 + \alpha^{11}z)$	$1 + \alpha^{10}z$	α^{-6}	α^6
3	2	1	$1 + \alpha^3 z + \alpha^{11}z^2$	$\alpha^{-6}(1 + \alpha^{14}z)$	$1 + \alpha^5 z$	$\alpha^{-6}(1 + \alpha^{10}z)$	0
4	2	1	$1 + \alpha^3 z + \alpha^{11}z^2$	$\alpha^{-6}(1 + \alpha^{14}z)$	$1 + \alpha^5 z$	$\alpha^{-6}(1 + \alpha^{10}z)$	?

$$\sigma^{(4)} = 1 + \alpha^3 z + \alpha^{11}z^2 = (1 + \alpha^4 z)(1 + \alpha^7 z) \qquad \text{so } X_1 = \alpha^4$$

$$X_2 = \alpha^7$$

$$\omega^{(4)} = 1 + \alpha^5 z$$

$$Y_1 = \frac{X_1 + \alpha^5}{X_1 + X_2} = \frac{\alpha^4 + \alpha^5}{\alpha^4 + \alpha^7} = \frac{\alpha^8}{\alpha^3} = \alpha^5 = \xi$$

$$Y_2 = \frac{X_2 + \alpha^5}{X_1 + X_2} = \frac{\alpha^7 + \alpha^5}{\alpha^4 + \alpha^7} = \frac{\alpha^{13}}{\alpha^3} = \alpha^{10} = \xi^2$$

Example 10.23

$$1 + S = 1 + \alpha^{11}z + \alpha^2 z^2 + \alpha^8 z^3 + \alpha^{14}z^4 + \cdots$$

k	$D(k)$	$B(k)$	$\sigma^{(k)}$	$\tau^{(k)}$	$\omega^{(k)}$	$\gamma^{(k)}$	$\Delta_1{}^{(k)}$
0	0	0	1	1	1	0	α^{11}
1	1	1	$1 + \alpha^{11}z$	α^4	1	α^4	α^{12}
2	1	0	$1 + \alpha^6 z$	$\alpha^3(1 + \alpha^{11}z)$	$1 + \alpha z$	α^3	0
3	1	0	$1 + \alpha^6 z$	$\alpha^3(1 + \alpha^{11}z)$	$1 + \alpha z$	α^3	0
4	1	0	$1 + \alpha^6 z$	$\alpha^3(1 + \alpha^{11}z)$	$1 + \alpha z$	α^3	?

$$X_1 = \alpha^6$$

$$Y_1 = \frac{\alpha^6 + \alpha}{\alpha^6} = \frac{\alpha^{11}}{\alpha^6} = \alpha^5 = \xi$$

★ 10.3 ALTERNATE BCH CODES AND EXTENDED BCH CODES

In Sec. 10.1, we assumed that the roots of the generator polynomial of a t-error-correcting BCH code must include $\alpha, \alpha^2, \alpha^3, \ldots, \alpha^{2t}$. Although convenient, this definition is overly restrictive. More generally, for any l, one may define a t-error-correcting BCH code as the cyclic code whose generator is the product of the distinct minimal polynomials of α^l, $\alpha^{l+1}, \ldots, \alpha^{l+2t-1}$. If $l \neq 1$, this definition gives an alternate t-error correcting BCH code. There is no loss of generality in the assumption that $l > 0$, since the case $l = 0$ is identical to the case $l = n$, where n is the block length.

The decoding procedure for alternate t-error-correcting BCH codes may be derived as follows. In general, one has the identity

$$(1 + S)\sigma = \omega$$

or, equivalently,

$$\Big(1 + \sum_{k=1}^{l-1} S_k z^k + \sum_{k=l}^{\infty} S_k z^k\Big)\sigma = \omega$$

and

$$\Big(\sum_{k=l}^{\infty} S_k z^k\Big)\sigma = \omega - \Big(1 + \sum_{k=l}^{l-1} S_k z^k\Big)\sigma$$

Since the left side is divisible by z^l, so is the right side, and we may define the polynomial

$$\Omega = \frac{\omega - \Big(1 + \sum_{k=1}^{l-1} S_k z^k\Big)\sigma}{z^{l-1}} + \sigma$$

Clearly $\deg \Omega \leq \deg \sigma$, and $\Big(\sum_{k=l}^{\infty} S_k z^{1+k-l}\Big)\sigma = \Omega - \sigma$ or

$$\boxed{\Big(1 + \sum_{k=l}^{l+2t-1} S_k z^{1+k-l}\Big)\sigma \equiv \Omega \bmod z^{2t+1}} \tag{10.31}$$

Algorithm 7.4 may be used to solve Eq. (10.31) for the polynomials $\sigma(z)$ and $\Omega(z)$. The error evaluator is given by the formula

$$\omega = z^{l-1}\Omega + \Big(1 + \sum_{k=1}^{l-1} S_k z^k - z^{l-1}\Big)\sigma$$

The usual formula for the error values, Eq. (10.14), now becomes

$$Y_i = \frac{X_i^{1-l}\Omega(X_i^{-1})}{\prod_{j\neq i}(1 - X_jX_i^{-1})} = \frac{X_i^{-l}\tilde{\Omega}(X_i)}{\prod_{j\neq i}(X_i - X_j)} \tag{10.32}$$

Here $\tilde{\Omega}(z) = z^{\deg \sigma}\Omega(z^{-1})$.

Thus, we may decode an alternate BCH code by Algorithm 10.33.

Decoding algorithm 10.33

10.331. Find the weighted power-sum symmetric functions S_l, $S_{l+1}, \ldots, S_{l+2t-1}$. For $k = l, l+1, \ldots, l+2t-1$, S_k may be found from the formula $S_k = r^{(k)}(\alpha^k)$, where $r^{(k)}(x)$ is the remainder of the received polynomial $R(x)$ divided by the minimal polynomial of α^k over GF(q), $M^{(k)}(x)$.

10.332. Use Algorithm 7.4 to solve Eq. (10.31) for the polynomials $\sigma(z)$ and $\Omega(z)$.

10.333. Using a Chien search over the nth roots of unity over GF(q), find the reciprocal roots of the error-locator polynomial $\sigma(z)$. These are the error locations.

10.334. Find the error values from Eq. (10.32).

Algorithm 10.33 reduces into Algorithm 10.15 when $l = 1$.

The case $l = 0$ also deserves special consideration. If we define

$$\Omega(z) = \sigma(z) + z\sum_i Y_i \prod_{j\neq i}(1 - X_jz) \tag{10.34}$$

then

$$\frac{\Omega(z)}{\sigma(z)} - 1 = z\sum_i \frac{Y_i}{1 - X_iz} = z\sum_i\sum_k Y_iX_i^kz^k = z\sum_{k=0}^{\infty} S_kz^k$$

so

$$\left(1 + \sum_{k=0}^{\infty} S_kz^{k+1}\right)\sigma(z) = \Omega(z)$$

Therefore, Eqs. (10.31) and (10.32) remain valid when $l = 0$.

The alternate BCH code with $l = 0$ is actually a shortened version of the *extended* BCH code, which has the same number of check digits and one additional message digit. In general, an extended BCH code of block length n may be constructed by annexing a single overall parity check to a cyclic BCH code of block length $n - 1$. For example, we may construct an extended double-error-correcting BCH code of block

length 16 by annexing a single parity check to the $\mathcal{H}$ matrix of Eq. (5.21). The parity-check matrix for the extended binary BCH code is then

$$\mathcal{H} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} (\alpha^0)^0 & (\alpha^1)^0 & (\alpha^2)^0 & \cdots & (\alpha^{-2})^0 & (\alpha^{-1})^0 & 0^0 \\ (\alpha^0)^1 & (\alpha^1)^1 & (\alpha^2)^1 & \cdots & (\alpha^{-2})^1 & (\alpha^{-1})^1 & 0^1 \\ (\alpha^0)^3 & (\alpha^1)^3 & (\alpha^2)^3 & \cdots & (\alpha^{-2})^3 & (\alpha^{-1})^3 & 0^3 \end{bmatrix} \tag{10.35}$$

The location number of the overall parity check is the field element 0. All the nonzero locations are powers of α, so it is convenient to represent 0 as a power of α by introducing the convention $\alpha^\infty = 0$. This equation is purely conventional; it cannot be justified by any limiting operation because $\lim_{k\to\infty} \alpha^k$ does not exist. If we also adopt the convention that $0^0 = 1$, then we may interpret the overall binary parity check as $S_0 = \sum_i X_i^0$, which is the parity of the number of errors. In general, the overall parity check is $S_0 = \sum_i Y_i X_i^0 = \sum_i Y_i$, which is the sum of the error values.

The error-locator polynomial for the extended BCH code is defined by Eq. (10.11); the error evaluator is defined by Eq. (10.34), where the sums and products are taken over all error locations, possibly including $X_i = \alpha^\infty$. If there are e errors none of which occurs at location α^∞, then $\deg \sigma = e$ and $\deg \Omega \leq e$, as usual. However, if there are e errors one of which occurs at location α^∞, then $\deg \sigma = e - 1$, and $\deg \Omega = e$, as may be seen from Eq. (10.34) and the observation that

$$\deg z \prod_{j\neq i} (1 - X_j z) = \begin{cases} e & \text{if } X_i = \alpha^\infty \\ e - 1 & \text{if } X_j = \alpha^\infty \text{ for some } j \neq i \end{cases}$$

We summarize this result as a theorem.

Theorem 10.36 In an extended BCH code, $\deg \sigma < \deg \Omega$ iff there is an error at location α^∞.

Extended BCH codes may be decoded by a slight modification of Algorithm 10.33. At Step 10.333, Theorem 10.36 may be invoked to determine whether or not there is an error at location α^∞; the other error locations may be determined by a Chien search. All the error values may still be determined from Eq. (10.32). If there is an error at location α^∞, its value is given by

$$Y = \frac{\tilde{\Omega}(0)}{\prod_{i=1}^{e-1} (-X_i)}$$

where now $\tilde{\Omega}(z) = z^{1+\deg \sigma}\Omega(z^{-1})$.

This decoding procedure for extended BCH codes may also be expressed in another form. Define

$$\tilde{\sigma}^{(n)}(z) = z^{D(n)}\sigma^{(n)}(z^{-1}) \qquad \text{and} \qquad \tilde{\Omega}^{(n)}(z) = z^{D(n)}\tilde{\Omega}^{(n)}(z^{-1})$$

where $\sigma^{(n)}(z)$, $\Omega^{(n)}(z)$, and $D(n)$ are generated by Algorithm 7.4 as it solves the equation

$$\left(1 + \sum_{k=0}^{\infty} S_k z^{k+1}\right) \sigma(z) = \Omega(z)$$

If there are no more than t errors in an extended BCH code, then their locations are given by the roots of $\tilde{\sigma}^{(2t)}(z)$ and their values are given by

$$Y_i = \frac{\tilde{\Omega}^{(2t)}(X_i)}{\prod_{j \neq i} (X_i - X_j)}$$

In this formulation, an error at location α^∞ is located and evaluated in the same manner as any other error.

Every element in GF(q^m) is a location of some digit of the extended narrow-sense BCH code of block length q^m. These codes have even more symmetry than the conventional BCH codes. In addition to the cyclic permutation and the permutation $X_i \to X_i^2$ (Corollary 5.82), the extended BCH codes are invariant under the affine permutation group on GF(q^m). This result is due to Peterson. Theorem 10.37, due to Kasami, is a slight generalization of Peterson's theorem.

Theorem 10.37 Let α be a primitive element in GF(q^m), and let the generator polynomial of a cyclic code of block length $q^m - 1$ over GF(q) be given by $g(x) = \prod_{k \in K} (x - \alpha^k)$, where K is a set of integers mod $q^m - 1$. Assume that $0 \notin K$, and that if $k \in K$ and $\binom{k}{j} \neq 0$,

then $j \in K \cup 0$. Let the field element 0 represent the location of the overall parity check of the extended code. Then the extended code is invariant under the affine permutation group consisting of all permutations of the form

$$X_i \to uX_i + v \qquad u,v \in \mathrm{GF}(q^m) \qquad u \neq 0$$

Remarks Before proving Theorem 10.37, let us see what it implies. In view of the corollary to Lucas' theorem 4.72, $\binom{k}{j} \neq 0$ in a field of characteristic p iff every digit of the p-ary expansion of j is less than or equal to the corresponding digit of the p-ary expansion of k. In abbreviated notation, $\binom{k}{j} \neq 0$ iff $j \subseteq k$.

For a t-error-correcting BCH code, $k \in K$ iff k can be written as $k \equiv iq^s \bmod q^m - 1$, where $0 < i \leq 2t$. If $j \subseteq k$, then $q^{m-s}j$ is congruent to an integer $\leq i$, so $j \in K \cup 0$. Therefore Theorem 10.37 implies Theorem 10.371.

Theorem 10.371 (Peterson) Every extended BCH code of block length q^m over $\mathrm{GF}(q)$ is invariant under the affine permutation group on $\mathrm{GF}(q^m)$.

Example 10.372 According to Theorem 10.37, the code defined by parity-check matrix (10.35) is invariant under the permutation $X_i \to X_i + \alpha$, which permutes the codeword $\mathbf{C} = [C_0, C_1, C_2, \ldots, C_{14}, C_\infty]$ into $[C_4, C_\infty, C_5, C_9, C_0, C_2, C_{11}, C_{14}, C_{10}, C_3, C_8, C_6, C_{13}, C_{12}, C_7, C_1]$.

Proof of Theorem 10.37 Let $X_1, X_2, \ldots$ denote the locations of the nonzero digits in a codeword of the original extended code, and let Y_i represent the values of the digits of the corresponding locations. Let $\check{X}_i = uX_i + v$ be the locations of these digits in the permuted codeword. Then if $k \in K \cup 0$, $S_k = \sum_i Y_i X_i^k = 0$. The permuted codeword is itself a codeword iff $\check{S}_k = \sum_i Y_i(uX_i + v)^k = 0$ for all $k \in K \cup 0$. Using the binomial theorem, $\check{S}_k = \sum_i Y_i \sum_j \binom{k}{j} u^j X_i^j v^{k-j} = \sum_j \binom{k}{j} u^j v^{k-j} S_j$. By the hypotheses of the theo-

rem, if $k \in K \cup 0$ for each j, either $\binom{k}{j} = 0$ or $S_j = 0$. In either case, $\binom{k}{j} S_j = 0$ and $\check{S}_k = \sum_j 0 = 0$, for all $k \in K \cup 0$. Q.E.D.

Kasami, Lin, and Peterson (1967*a*) have shown that if an extended cyclic code is invariant under the affine group, then $k \in K$, $\binom{k}{j} \neq 0$, $\Rightarrow j \in K \cup 0$. In other words, the hypotheses of Theorem 10.37 are necessary as well as sufficient.

A permutation group is said to be *transitive* iff for all ordered pairs of digits, the group contains a permutation which permutes one digit into the other digit. The affine group over GF(q^m) is obviously transitive. So is the cyclic group over the nth roots of unity if $(n,q) = 1$.

The invariance of an extended binary code under a transitive permutation group implies something about the weight structure of the code, according to Theorem 10.38.

Theorem 10.38 (Prange) Let a_i be the number of codewords of weight i in a linear binary code of block length n, and let A_i be the number of codewords of weight i in the extended code, of length $N = n + 1$. If the extended code is invariant under a transitive permutation group, then

$$a_{2j-1} = 2jA_{2j}/N = 2ja_{2j}/(N - 2j) \qquad \text{for all } j$$

Proof The extended code contains exactly $A_{2j} = a_{2j-1} + a_{2j}$ codewords of weight $2j$. Exactly a_{2j-1} of these words have a 1 at location α^∞; a_{2j} of these words have a 0 at location α^∞. The total weight of all the words of weight $2j$ is $2jA_{2j} = 2j(a_{2j-1} + a_{2j})$. Since the extended code is invariant under a transitive permutation group, this total weight must be distributed evenly among the code's N locations. Each location must have a 1 in $2j(a_{2j-1} + a_{2j})/N$ codewords of weight $2j$. The location α^∞ is no different from any other location, so

$$a_{2j-1} = \frac{2jA_{2j}}{N} = \frac{2j(a_{2j-1} + a_{2j})}{N}$$

which is equivalent to the theorem. Q.E.D.

Corollary 10.39 If an extended linear binary code is invariant under a transitive permutation group, the minimum weight of the original (unextended) code is odd.

By Peterson's theorem 10.371, Corollary 10.39 implies that every narrow-sense binary BCH code has odd minimum weight.

10.4 DECODING ERASURES AS WELL AS ERRORS

For many channels, it turns out to be wiser not to force the demodulator to make a choice between sufficiently close alternatives. The best strategy is to demodulate sufficiently weak or sufficiently ambiguous received signals not as any of the q letters in the input alphabet but as an additional letter, ?, called an erasure. In addition to locating and correcting any errors which may be present, the decoder must then also attempt to determine the values of the symbols in the erased locations. Thus, the goal of the decoder is to correct all the errata, which consist of two types: erasures, whose locations are known but whose values are unknown, and errors, whose locations and values are both unknown. It is helpful for the decoder to consider three different locator polynomials:

$$\text{The erasure locator:} \quad \Lambda = \prod_i (1 - X_i z) \qquad X_i = \text{erasures} \tag{10.41}$$

$$\text{The error locator:} \quad \lambda = \prod_i (1 - X_i z) \qquad X_i = \text{errors} \tag{10.42}$$

$$\text{The errata locator:} \quad \sigma = \prod_i (1 - X_i z) \qquad X_i = \text{errata} \tag{10.43}$$

$$\boxed{\sigma = \Lambda\lambda} \tag{10.44}$$

In order to compute parity checks on the received word, the decoder must assign some arbitrary values to the symbols at the erasure locations. If α^k is a root of the code's generator polynomial, the decoder may then evaluate

$$S_k = \sum_{\substack{\text{all} \\ \text{errata}}} Y_i X_i^k = R(\alpha^k) = r^{(k)}(\alpha^k)$$

If X_i is an erasure location, Y_i is the difference between the assigned symbol at location X_i and the symbol in the transmitted codeword at location X_i; if X_i is an error location, Y_i is the difference between the received symbol at location X_i and the symbol in the transmitted codeword at location X_i.

The key equation, $(1 + S)\sigma = \omega$, may be written as $(1 + S)\Lambda\lambda = \omega$. More generally, for the alternate BCH codes discussed in Sec. 10.3, the key equation may be written as

$$\left(1 + \sum_{k=m}^{m+d-2} S_k z^{k+1-m}\right) \Lambda\lambda \equiv \Omega \bmod z^d$$

Here the code's designed distance d is $2t + 1$ for the t-error-correcting code. The S's are the known weighted power-sum symmetric functions of the errata locations; Λ is the known erasure locator; λ is the unknown error locator; Ω is the unknown errata evaluator for the alternate BCH code, as discussed in Sec. 10.3.

To simplify the key equation, we combine the known S's with the known erasure-locator polynomial to obtain Forney's (1965) T's. The T's are defined by the equation

$$1 + \sum_{k=1}^{d-1} T_k z^k = \left(1 + \sum_{k=m}^{m+d-2} S_k z^{k+1-m}\right)\Lambda \bmod z^d \tag{10.45}$$

The decoder may find λ and Ω from the equation

$$\left(1 + \sum_{k=1}^{d-1} T_k z^k\right) \lambda \equiv \Omega \bmod z^d$$

If there are s erasures and t errors, then $\deg \lambda = t$ and $\deg \Omega = s + t$. We have

$$\left(1 + \sum_{k=1}^{s} T_k z^k + \sum_{k=s+1}^{d-1} T_k z^k\right) \lambda \equiv \Omega \bmod z^d$$

$$\left(\sum_{k=s+1}^{d-1} T_k z^k\right) \lambda \equiv \Omega - \left(1 + \sum_{k=1}^{s} T_k z^k\right) \lambda \bmod z^d$$

Since the left side is divisible by z^{s+1}, so is the right side. We define the polynomial η by the equation

$$\eta = \frac{\Omega - \left(1 + \sum_{k=1}^{s} T_k z^k\right) \lambda}{z^s} + \lambda \tag{10.46}$$

Substituting this into the previous equation gives

$$\begin{aligned} \left(\sum_{k=s+1}^{d-1} T_k z^k\right) \lambda &\equiv z^s(\eta - \lambda) \bmod z^d \\ \left(\sum_{k=s+1} T_k z^{k-s}\right) \lambda &\equiv \eta - \lambda \bmod z^d \\ \left(1 + \sum_{k=s+1}^{d-1} T_k z^{k-s}\right) \lambda &\equiv \eta \bmod z^{d-s} \end{aligned} \tag{10.47}$$

If there are t errors, $\deg \lambda = t$, and $\deg \eta \leq t$. If $t < (d - s)/2$, these equations may be solved by Algorithm 7.4.

The overall decoding procedure for correcting erasures as well as errors with an alternate BCH code of designed distance d is summarized in Algorithm 10.48.

Algorithm 10.48

10.481. Compute the weighted power-sum symmetric functions of the errata locations, S_m, S_{m+1}, S_{m+2}, . . . , S_{m+d-2}, compute the erasure-locator polynomial from Eq. (10.41), and define $s = \deg \Lambda$.

10.482. Compute the generating function $T(z)$ from Eq. (10.45).

10.483. Use Algorithm 7.4 to find polynomials $\lambda(z)$ and $\eta(z)$ which solve Eq. (10.47).

10.484. Use a Chien search to find the reciprocal roots of the error-locator polynomial $\lambda(z)$.

10.485. Compute the errata-evaluator polynomial from the equation

$$\Omega = \left(1 + \sum_{k=1}^{s} T_k z^k - z^s\right) \lambda + z^s \eta$$

[This equation follows directly from Eq. (10.46).]

10.486. Find all the errata values from Eq. (10.32).

Algorithm 10.48 will correct any combination of s erasures and t errors if d is larger than $s + 2t$. In this sense, an error may be considered to be twice as harmful as an erasure. Heuristically, one can attribute this to the fact that there are two unknowns (a location and a value) associated with each error but only one unknown (a value) associated with each erasure. One can not rely on this heuristic interpretation too much, however, because the criterion $s + 2t < d$ remains valid even in the binary case, where every error has the known value 1.

10.5 DECODING MORE THAN t ERRORS

Although it is possible to extend decoding Algorithms 10.33 and 10.48 to correct some of the errata patterns which would otherwise cause decoding failure, we restrict our attention in this section to the simplest case, Algorithm 10.15. The reader who wishes to extend the results of this section to alternate BCH codes and decoding algorithms which correct erasures as well as errors will find such a project straightforward but tedious and the results slightly weaker.

If the t-error-correcting q-ary BCH code encounters no more than t errors, then the errors will be correctly located and correctly evaluated by Algorithm 10.15. At Step 10.152, Algorithm 7.4 will terminate with the correct error locator, $\sigma = \sigma^{(2t)}$, and the correct error evaluator, $\omega = \omega^{(2t)}$. In this case, the polynomial $\sigma^{(2t)}$ has $D(2t) = \deg \sigma^{(2t)}$ distinct reciprocal roots among the nth roots of unity, corresponding to the error locations. The decoder will find these reciprocal roots at Step 10.153 of the decoding procedure. At Step 10.154, the decoder will compute

$$Y_i = \frac{\tilde{\omega}(X_i)}{X_i \prod_{j \neq i} (X_i - X_j)}$$

If there are no more than t errors, then each Y_i will be some nonzero element in GF(q), equal to the value of the error at location X_i.

If there are more than t errors, then almost anything can happen. It is possible (though very unlikely) that Algorithm 7.4 terminates with the correct error locator and the correct error evaluator, even though $D(2t) = \deg \sigma^{(2t)} > t$. It is also possible that Algorithm 7.4 terminates with a *legitimate* error locator and a *legitimate* error evaluator corresponding to some error pattern of weight $\leq t$. In that case the decoder completes Steps 10.153 and 10.154 of the decoding procedure without difficulty, and incorrectly "corrects" what it incorrectly believes to be the error pattern. Although wrong, the decoder cannot be blamed for such a mistake, since, on the basis of the received word, the error pattern which it incorrectly corrected is more probable than the actual error pattern, which has greater weight.

Far more likely than either of these events, however, are two other possibilities, failure at Step 10.153 or failure at Step 10.154. Algorithm 7.4 may terminate with an illegitimate polynomial $\sigma^{(2t)}$ which does *not* have deg $\sigma^{(2t)}$ distinct roots among the nth roots of unity. In this case, Step 10.153 ends in failure. Even if all roots of $\sigma^{(2t)}$ are distinct nth roots of unity, the polynomial $\omega^{(2t)}$ may be illegitimate. Although the Y's which the decoder computes at Step 10.154 must lie in the locator field which contains the nth roots of unity over GF(q), they may not lie in the symbol field, which consists of GF(q) itself.† In the event of such a failure at Step 10.153 or 10.154, the decoder has detected that more than t errors have occurred.

In many cases, the decoder can detect illegitimate polynomials $\sigma^{(2t)}$ and $\omega^{(2t)}$ by preliminary calculations immediately after Step 10.152, thus avoiding the relatively lengthy Chien search of Step 10.153. For example, if deg $\sigma^{(2t)} < D(2t)$, then $\sigma^{(2t)}$ cannot be a legitimate locator

† In the special case of Reed-Solomon codes, this type of failure cannot occur, because the symbol field and the locator field are identical.

polynomial of an unextended BCH code. Although this test will detect only a very small fraction of the possible illegitimate polynomials $\sigma^{(2t)}$, it requires no extra computation.

As a second legitimacy test, the decoder could determine the parity of the number of irreducible factors of $\sigma^{(2t)}$ over GF(q^m) via Theorems 6.68 or 6.69 and 6.695. If the number of irreducible factors of $\sigma^{(2t)}$ is not congruent to $D(2t)$ mod 2, then $\sigma^{(2t)}$ is illegitimate. Heuristically, we might conjecture that an illegitimate error locator would be equally likely to have an even or an odd number of irreducible factors, so that a legitimacy test based on Stickelberger's theorem should detect about half of the illegitimate error locators. For binary BCH codes with $1 \ll 2t < 2^{m/2} = \sqrt{(n+1)}$ this conjecture is true, but in the special case of double-error-correcting binary BCH codes, we shall find that a slight modification of this legitimacy test detects all illegitimate error locators and leads to a complete maximum-likelihood decoding algorithm (Algorithm 16.481).

As a third legitimacy test, the decoder could compute the residues $z, z^2, z^{2^2}, z^{2^3}, \ldots, z^{2^{\log n}} \bmod \sigma^{(2t)}(z)$. By multiplying together an appropriate combination of these residues, the decoder could compute the residue of z^{n+i} for some $i \leq \deg \sigma^{(2t)}$. The candidate locator polynomial $\sigma^{(2t)}$ is then legitimate iff $z^{n+i} \equiv z^i \bmod \sigma^{(2t)}(z)$. Although this legitimacy test will detect all illegitimate locator polynomials, it cannot generally provide any guarantee against an illegitimate evaluator polynomial, which will cause the decoder to find "error values" in GF(q^m) − GF(q) at Step 10.486.

In order to test the legitimacy of both $\sigma^{(2t)}$ and $\omega^{(2t)}$ and to prepare for corrective measures if either is illegitimate, we insert an additional step into decoding Algorithm 10.15, between Steps 10.152 and 10.153.

Step 10.1525 Compute the generating function $S^{(2t)}$, defined by

$$1 + S^{(2t)} = \frac{\omega^{(2t)}}{\sigma^{(2t)}} \tag{10.51}$$

For the moment, we avoid specifying the number of coefficients of $S^{(2t)}$ which are to be computed. Since $\omega^{(2t)}$ and $\sigma^{(2t)}$ are given, it is clear that for $i > D(2t)$, the decoder may compute

$$S_i^{(2t)} = -\sum_{j=1}^{D(2t)} S_{i-j}^{(2t)} \sigma_j^{(2t)} \tag{10.52}$$

If the polynomial $\sigma^{(2t)}$ is legitimate, then $\sigma^{(2t)} = \prod_i (1 - X_i z)$,

where the X_i are distinct nth roots of unity and $\sigma^{(2t)}$ divides $1 - z^n$. In this case we may write

$$\sigma^{(2t)} = \frac{1 - z^n}{\xi^{(2t)}}$$

and

$$1 + S^{(2t)} = \frac{\xi^{(2t)}\omega^{(2t)}}{1 - z^n} = \xi^{(2t)}\omega^{(2t)} \sum_{j=0}^{\infty} z^{nj}$$

If deg $\omega^{(2t)} \leq$ deg $\sigma^{(2t)} = D(2t)$, then deg $\xi^{(2t)}\omega^{(2t)} \leq n$, and we have $S_{n+k}^{(2t)} = S_k^{(2t)}$ for $k = 1, 2, 3, \ldots$. Conversely, if $S_{n+k}^{(2t)} = S_k^{(2t)}$ for $k + 1, 2, \ldots, D(2t)$, then $S_{n+k}^{(2t)} = S_k^{(2t)}$ for all k, because $S_{n+D(2t)+i}^{(2t)} = -\sum_{j=1}^{D(2t)} S_{n+D(2t)+i-j}^{(2t)}\sigma_j^{(2t)} = -\sum_{j=1}^{D(2t)} S_{D(2t)+i-j}^{(2t)}\sigma_j^{(2t)} = S_{D(2t)+i}^{(2t)}$ (by induction on i). Since $S_{n+k}^{(2t)} = S_k^{(2t)}$ for all $k > 0$, $1 + S^{(2t)} = \Phi^{(2t)}/(1 - z^n)$, where deg $\Phi^{(2t)} \leq n$. We then have

$$\frac{\omega^{(2t)}}{\sigma^{(2t)}} = \frac{\Phi^{(2t)}}{1 - z^n}$$

or $\sigma^{(2t)}\Phi^{(2t)} = \omega^{(2t)}(1 - z^n)$. This shows that $\sigma^{(2t)}$ divides $\omega^{(2t)}(1 - z^n)$. According to Theorems 7.42 and 7.411, $\sigma^{(2t)}$ and $\omega^{(2t)}$ are relatively prime. Therefore, $\sigma^{(2t)}$ divides $1 - z^n$. We have proved Theorem 10.53.

Theorem 10.53 The reciprocal roots of $\sigma^{(2t)}$ are distinct nth roots of unity iff $S_{n+k}^{(2t)} = S_k^{(2t)}$ for $k = 1, 2, \ldots, D(2t)$.

Thus, the success or failure of Step 10.153 may be anticipated by an inspection of certain coefficients of $S^{(2t)}$ at Step 10.1525. By further investigation of these coefficients, we may anticipate the success or failure of Step 10.154.

Theorem 10.54 If $X_1, X_2, \ldots, X_{D(2t)}$ are distinct and

$$\sigma^{(2t)} = \prod_{i=1}^{D(2t)} (1 - X_i z)$$

then the quantities

$$\frac{\tilde{\omega}(X_i)}{X_i \prod_{j \neq i} (X_i - X_j)} \in \mathrm{GF}(q)$$

iff

$$S_{qk}^{(2t)} = [S_k^{(2t)}]^q \qquad \text{for } k = 1, 2, \ldots, D(2t)$$

Proof If $\sigma^{(2t)} = \prod_{i=1}^{D} (1 - X_i z)$, then a partial-fraction expansion of $\omega^{(2t)}/\sigma^{(2t)}$ gives

$$\frac{\omega^{(2t)}}{\sigma^{(2t)}} = \sum_{k=1}^{\infty} S_k^{(2t)} z^k - 1 = \sum_{i}^{D} \frac{Y_i X_i z}{1 - X_i z} = \sum_{k=1}^{\infty} \left(\sum_{i}^{D} Y_i X_i^k \right) z^k$$

where

$$Y_i = \frac{\tilde{\omega}^{(2t)}(X_i)}{X_i \prod_{j \neq i} (X_i - X_j)} \quad \text{and} \quad S_k{}^{(2t)} = \sum_{i=1}^{D} Y_i X_i{}^k \quad \text{for all } k.$$

In a field including GF(q), we may also write $(S_k{}^{(2t)})^q = \left(\sum_{i=1}^{D} Y_i X_i{}^k \right)^q$ $= (S_k{}^{(2t)})^q = \sum_{i=1}^{D} Y_i{}^q X_i{}^{qk}$. If $Y_i = Y_i{}^q$, then we have $S_{kq}{}^{(2t)} = \sum_{i=1}^{D} Y_i X_i{}^{kq} = \sum_{i=1}^{D} Y_i{}^q X_i{}^{kq} = (S_k{}^{(2t)})^q$ for all k. Conversely, if $S_{kq}{}^{(2t)} = (S_k{}^{(2t)})^q$, then we have both $\sum_{i=1}^{D} Y_i X_i{}^{qk} = (S_k{}^{(2t)})^q$ for $k = 1, 2, \ldots, D$ and $\sum_{i=1}^{D} Y_i{}^q X_i{}^{qk} = (S_k{}^{(2t)})^q$ for $k = 1, 2, \ldots, D$. The unique solution of the first set of simultaneous linear equations is given by

$$Y_i = \frac{\tilde{\Omega}(X_i{}^{-1})}{X_i \prod_{j \neq i} (X_i - X_j)}$$

where Ω is determined by $\deg \Omega \leq D$, $\Omega(0) = 1$, and

$$\left[1 + \sum_{k=1} (S_k{}^{(2t)}) q z^k\right] \left[\prod_{i=1}^{D} (1 - X_i{}^q z)\right] \equiv \Omega \bmod z^{D+1}$$

This same expression furnishes the unique solution to the (identical) second set of equations, whence $Y_i{}^q = Y_i$ and $Y_i \in \mathrm{GF}(q)$, for $i = 1, 2, \ldots, D$. Q.E.D.

Theorems 10.53 and 10.54 provide a method of anticipating the success or failure of Steps 10.153 and 10.154 by computing $S^{(2t)}$ at Step 10.1525. Since the success of Step 10.153 is not guaranteed until one computes $S_{2t+1}^{(2t)}, \ldots, S_{n+D(2t)}^{(2t)}$, this computation may appear prohibitively time-consuming. It is true that the computation of $n + D(2t) -$

$2t$ successive coefficients of $S^{(2t)}$ is approximately as much work as a Chien search over $n + D(2t) - 2t$ successive nth roots of unity and that with only $2t - D(2t)$ more steps, Step 10.153 might be completed via a Chien search. Nevertheless, there are two major advantages to Step 10.1525:

1. The vast majority of illegitimate polynomials $\sigma^{(2t)}$ and $\omega^{(2t)}$ may be detected by computing only a few coefficients of $S^{(2t)}$.
2. Sometimes corrective steps may be taken.

For example, although the decoder does not know S_{2t+1}, he often knows S_{2t+K} for various (small) values of K, because of conjugate constraints $S_{qk} = S_k{}^q$ and cyclic constraints $S_{k+n} = S_k$. After computing $S^{(2t)}$, the decoder then knows $\Delta_K{}^{(2t)} = S_{2t+K} - S_{2t+K}^{(2t)}$ for various values of K. If $\Delta_K{}^{(2t)} \neq 0$, then $\sigma^{(2t)}$ and $\omega^{(2t)}$ are not legitimate. However, these known $\Delta_K{}^{(2t)}$ may enable the decoder to compute the polynomials U and V and thereby determine the true error locator from Theorem 7.43, which states that

$$\sigma = U\sigma^{(2t)} + V\tau^{(2t)}$$

In general, we use Definition 10.55.

Definition 10.55

$$\Delta^{(2t)} = \frac{S - S^{(2t)}}{z^{2t}}$$

(Notice that the first coefficient, $\Delta_1{}^{(2t)}$, coincides with the $\Delta_1{}^{(2t)}$ defined in Algorithm 7.4.) Setting

$$1 + S = \frac{\omega}{\sigma} = \frac{U\omega^{(2t)} + V\gamma^{(2t)}}{U\sigma^{(2t)} + V\tau^{(2t)}} \qquad \text{and} \qquad 1 + S^{(2t)} = \frac{\omega^{(2t)}}{\sigma^{(2t)}}$$

we get

$$\Delta^{(2t)} = \frac{1}{z^n}\left(\frac{\omega}{\sigma} - \frac{\omega^{(2t)}}{\sigma^{(2t)}}\right) = \frac{-V(\omega^{(2t)}\tau^{(2t)} - \sigma^{(2t)}\gamma^{(2t)})}{z^n\sigma^{(2t)}(U\sigma^{(2t)} + V\tau^{(2t)})}$$

$$\Delta^{(2t)} = \frac{-V}{\sigma^{(2t)}(U\sigma^{(2t)} + V\tau^{(2t)})}$$

$$\Delta^{(2t)} = -V\sum_{k=0}^{\infty}[1 - \sigma^{(2t)}(U\sigma^{(2t)} + V\tau^{(2t)})]^k$$

$$\Delta_1{}^{(2t)} = -V_1$$

$$\Delta_2{}^{(2t)} = -V_2 + V_1(U_1 + \sigma_1{}^{(2t)} + V_1\tau_0{}^{(2t)} + \sigma_1{}^{(2t)})$$

$$\begin{aligned}\Delta_3{}^{(2t)} = {} & -V_3 + V_2(U_1 + \sigma_1{}^{(2t)} + V_1\tau_0{}^{(2t)} + \sigma_1{}^{(2t)}) \\ & - V_1\{(U_1 + \sigma_1{}^{(2t)} + V_1\tau_0{}^{(2t)} + \sigma_1{}^{(2t)})^2 \\ & - [\sigma_2{}^{(2t)} + \sigma_1{}^{(2t)}(V_1 + \sigma_1{}^{(2t)} + V_1\tau_0{}^{(2t)}) \\ & + U_2 + U_1\sigma_1{}^{(2t)} + \sigma_2{}^{(2t)} + V_2\tau_0{}^{(2t)} + V_1\tau_1{}^{(2t)}]\}\end{aligned}$$

In a field of characteristic p,

$$\sum_{k=0}^{\infty} \xi^k = \left[\prod_{m=0}^{\infty} (1 - \xi^{p^m}) \right]^{p-1}$$

Using this identity gives Theorem 10.56.

Theorem 10.56

$$\Delta^{(2t)} = -V \left[\prod_{m=0}^{\infty} (\sigma^{(2t)})^{p^m} \right]^{p-1} \left\{ \prod_{m=0}^{\infty} [U^{p^m} (\sigma^{(2t)})^{p^m} + V^{p^m} (\tau^{(2t)})^{p^m}] \right\}^{p-1}$$

In this special case when $U = 1$ and $V = V_1 z$ and $p = 2$, this becomes

$$\Delta^{(2t)} = V \prod_{m=0}^{\infty} [(\sigma^{(2t)})^2 + V \sigma^{(2t)} \tau^{(2t)}]^{2^m}$$

and

$$(\sigma^{(2t)})^2 + V\sigma^{(2t)}\tau^{(2t)} = 1 + (\sigma_1^{(2t)})^2 z^2 + (\sigma_2^{(2t)})^2 z^4 + \cdots + V_1 z[\tau_1^{(2t)} z + (\tau_2^{(2t)} + \tau_1^{(2t)} \sigma_1^{(2t)}) z^2 + \cdots]$$

So

$$\Delta^{(2t)} = V_1 z + [V_1^2 \tau_1^{(2t)} + V_1 (\sigma_1^{(2t)})^2] z^3 + \cdots \tag{10.57}$$

10.6 EXAMPLES

As the first example, we continue Example 7.68, using the 3-error-correcting binary BCH code of block length 31, with the field represented as polynomials of degree <5 in α, where $\alpha^5 + \alpha^2 + 1 = 0$. The power-sum symmetric functions of the error locations were $S_1 = 11101 = \alpha^{14}$, $S_3 = 10000 = \alpha^4$, and $S_5 = 00010 = \alpha^1$. Using Algorithm 7.4, we find $\sigma^{(6)} = \alpha^{26} z^3 + 0z^2 + \alpha^{14} z + 1$ and $\tau^{(6)} = \alpha^{24} z + \alpha^7 z^2 + \alpha^5 z^3$. We have $1 + S = 1 + \alpha^{14} z + \alpha^{28} z^2 + \alpha^4 z^3 + \alpha^{25} z^4 + \alpha^1 z^{15} + \alpha^8 z^6 + \cdots$. Using the formula

$$S_i^{(6)} = \begin{cases} S_i & \text{for } i \leq 6 \\ \sum_{j=1}^{3} S_{i-j}^{(6)} \sigma_i^{(6)} = \alpha^{14} S_{i-1}^{(6)} + \alpha^{26} S_{i-3}^{(6)} & \text{for } i > 6 \end{cases}$$

we have

$$1 + S^{(6)} = 1 + \alpha^{14} z + \alpha^{28} z^2 + \alpha^4 z^3 + \alpha^{25} z^4 + \alpha^1 z^5 + \alpha^8 z^6 + \alpha^{25} z^7 + \alpha^{19} z^8 + \alpha^{20} z^9 + \alpha^2 z^{10} + \cdots$$

However, in view of the cyclic and conjugate constraints, it is known that $S_9 = S_{31+9} = S_{40} = S_5{}^8 = \alpha^8 \neq \alpha^{20} = S_9{}^{(6)}$. We conclude that the poly-

nomial $\sigma^{(6)}$ is illegitimate; it does not have three distinct reciprocal roots in GF(2^5), because $\Delta_3^{(6)} = S_9 - S_9^{(6)} = \alpha^8 - \alpha^{20} = 1 \neq 0$. Corrective measures are in order. If we assume that no more than four errors occurred, then the general solution $\sigma = U\sigma^{(6)} + V\tau^{(6)}$ becomes $\sigma = \sigma^{(6)} + V_1 z\tau^{(6)}$. (Recall that, for binary codes, U must be even and V odd, and if either U or V had degree ≥ 2, then σ would have degree > 4.) According to Eq. (10.57),

$$\Delta_3^{(2t)} = +V_1^2 \tau_1^{(2t)} + V_1(\sigma_1^{(2t)})^2$$

In the present example this becomes

$$1 = V_1^2\alpha^{24} + V_1\alpha^{28}$$

or

$$(V_1\alpha^{-4})^2 + V_1\alpha^{-4} = \alpha^{-1} \tag{10.61}$$

The solutions of this quadratic equation are given by

$$V_1\alpha^{-4} = \alpha^{11} \quad \text{or} \quad \alpha^{11} + 1 = \alpha^{19}$$
$$V_1 = \alpha^{15} \text{ or } \alpha^{23}$$

If $V_1 = \alpha^{15}$, then $\sigma = 1 + \alpha^{14}z + \alpha^8 z^2 + \alpha z^3 + \alpha^{20}z^4$. If $V_1 = \alpha^{23}$, then $\sigma = 1 + \alpha^{14}z + \alpha^{16}z^2 + \alpha^5 z^3 + \alpha^{-3}z^4$.

One method for seeing whether either of these potential error locators is legitimate is to see whether it divides $z^{2^5} - z$. This can be accomplished fairly quickly. Modulo $\tilde{\sigma}(z) = z^4 + \alpha^{14}z^3 + \alpha^8 z^2 + \alpha z + \alpha^{20}$ we may compute

$$\begin{aligned}
z^4 &\equiv \alpha^{14}z^3 + \alpha^8 z^2 + \alpha z + \alpha^{20} \\
z^5 &\equiv \alpha^{16}z^3 + \alpha^{26}z^2 + \alpha^{17}z + \alpha^3 \\
z^6 &\equiv \alpha^5 z^3 + \alpha^8 z^2 + \alpha^{16}z + \alpha^5 \\
z^{2^3} = (z^4)^2 &\equiv \alpha^{28}z^6 + \alpha^{16}z^4 + \alpha^2 z^2 + \alpha^{40} \\
&\equiv \alpha^{-3}z^3 + \alpha^{15}z^2 + \alpha^{23}z + \alpha^{16} \\
z^{2^4} &\equiv \alpha^{-6}z^6 + \alpha^{-1}z^4 + \alpha^{15}z^2 + \alpha \\
&\equiv \alpha^{12}z^3 + \alpha^{23}z^2 + \alpha^4 z + \alpha^{28} \\
z^{2^5} &\equiv \alpha^{24}z^6 + \alpha^{15}z^4 + \alpha^8 z^2 + \alpha^{25} \\
&\equiv z
\end{aligned}$$

Therefore $\sigma = 1 + \alpha^{14}z + \alpha^8 z^2 + \alpha z^3 + \alpha^{20}z^4$ must have four distinct roots in GF(2^5). A Chien search reveals that

$$\sigma = (1 + \alpha^3 z)(1 + \alpha^5 z)(1 + \alpha^{16}z)(1 + \alpha^{27}z)$$

It happens that the other potential error polynomial also has four roots in GF(2^5)

$$1 + \alpha^{14}z + \alpha^{16}z^2 + \alpha^5 z^3 + \alpha^{-3}z^4 = (1 + \alpha^{19}z)(1 + \alpha^{21}z)(1 + \alpha^{22}z)(1 + \alpha^{28}z)$$

It turns out that any attempt to correct 4 errors with the 3-error-correcting binary BCH code of length 31 must always lead to an ambiguous result, because all of the code's cosets of weight 4 contain 2 coset leaders. This property is a consequence of the fact that the code is a punctured Reed-Muller code, which we will study in Sec. 15.3.

As a second example, we consider the 5-error-correcting binary BCH code of block length 31. Suppose $S_1 = \alpha^9$, $S_3 = \alpha^3$, $S_5 = \alpha^{15}$, $S_7 = \alpha^{28}$. With $S_9 = S_5{}^8 = \alpha^{27}$, we then have

$$1 + S(z) = 1 + \alpha^9 z + \alpha^{18} z^2 + \alpha^3 z^3 + \alpha^5 z^4 + \alpha^{15} z^5 + \alpha^6 z^6 + \alpha^{28} z^7 + \alpha^{10} z^8 + \alpha^{27} z^9 + \alpha^{-1} z^{10} + \cdots$$

Algorithm 7.64 terminates with

$$\sigma^{(10)}(z) = 1 + \alpha^9 z + \alpha^{17} z^2 + \alpha^7 z^3 + \alpha^7 z^4 + \alpha^{20} z^5$$
$$\tau^{(10)}(z) = \alpha^7 z + \alpha^{16} z^2 + \alpha^5 z^3 + \alpha^2 z^4 + \alpha^{11} z^4$$

We compute

$$S^{(10)}(z) = \cdots + \alpha^{-1} z^{10} + \alpha^5 z^{11} + \alpha^{12} z^{12} + \alpha^{14} z^{13}$$

Here

$$S_{13}{}^{(10)} = \alpha^{14} \neq (S_{11}{}^{(10)})^4 = (\alpha^5)^4 = \alpha^{20}$$

The error locator $\sigma^{(10)}(z)$ is illegitimate, and corrective measures are in order. If no more than six errors occurred, then according to Eq. (10.57),

$$S_{11} = S_{11}{}^{(10)} + \Delta_1{}^{(10)} = \alpha^5 + V_1$$
$$S_{13} = S_{13}{}^{(10)} + \Delta_3{}^{(10)} = \alpha^{14} + V_1{}^2 \tau_1{}^{(10)} + V_1(\sigma_1{}^{(10)})^2$$
$$= \alpha^{14} + V_1{}^2 \alpha^7 + V_1 \alpha^{18}$$

Setting $S_{13} = S_{11}{}^4$ gives us a quartic equation in the unknown V_1.

$$\alpha^{14} + V_1 \alpha^{18} + V_1{}^2 \alpha^7 = (\alpha^5 + V_1)^4$$
$$\alpha^{10} + V_1 \alpha^{18} + V_1{}^2 \alpha^7 + V_1{}^4 = 0 \tag{10.62}$$

As we shall see in Sec. 11.1, this quartic equation has two solutions in GF(2^5). Either $V_1 = \alpha^{24}$, or $V_1 = \alpha^4$.

If $V_1 = \alpha^{24}$, then $\sigma(z) = 1 + \alpha^9 z + \alpha^{-1} z^2 + \alpha^{12} z^3 + \alpha^{14} z^4 + \alpha^{16} z^5 + \alpha^4 z^6$ and $1 + S(z) = \cdots + \alpha^{-1} z^{10} + \alpha^{16} z^{11} + \alpha^{12} z^{12} + \alpha^2 z^{13} + \alpha^{25} z^{14} + \alpha^{21} z^{15} + \alpha^{20} z^{16} + \alpha^{12} z^{17}$. Since we now have $S_{17} = \alpha^{12}$, $S_{17}{}^2 = \alpha^{24} \neq S_3 = \alpha^3$, we must reject the hypothesis that $V_1 = \alpha^{24}$.

Setting $V_1 = \alpha^4$ gives $\sigma(z) = 1 + \alpha^9 z + \alpha^7 z^2 + \alpha^{21} z^3 + \alpha^{12} z^4 + \alpha^{19} z^5 + \alpha^{15} z^6$. A Chien search reveals this error polynomial to be legitimate. In fact,

$$\sigma(z) = (1 + z)(1 + \alpha z)(1 + \alpha^2 z)(1 + \alpha^3 z)(1 + \alpha^4 z)(1 + \alpha^5 z)$$

In general, if one assumes that there are $t + u$ errors, then one can use the expressions for the generating function $\Delta^{(2t)}$ to obtain simultaneous equations for u unknown coefficients of the polynomials U and V. These equations may have no solutions, as may happen if the received word lies in a coset all of whose words have weight $> t + u$. Or there may be several legitimate solutions. Or there may be a unique solution.

Unfortunately, the simultaneous nonlinear equations appear to be quite complicated, and little is known about the conditions for any solutions, for a unique solution, or how to go about finding the solution(s) if it (they) exist. If one assumes that there are only $t + 1$ errors, then this extra error can be located by solving a single algebraic equation in the single unknown, V_1. However, if there are more than $t + 1$ errors, then the situation gets very complicated very quickly.

Even the correction of a single extra error requires the solution of an algebraic equation over some large finite field. In most cases, we cannot find the roots of this equation via a Chien search over the entire field unless we allow the total decoding time to be substantially increased. Fortunately, the Chien search can be avoided. There are better methods for solving algebraic equations of low degree over large finite fields. These methods will be investigated in Chap. 11.

PROBLEM

10.1. As in Example 10.2, consider the double-error-correcting BCH code over GF(4). Suppose that $1 + S = 1 + 0z + \alpha^{14}z^2 + \alpha z^3 + 0z^4 + \cdots$. Show that $\sigma = 1 + \alpha^2 z + \alpha^4 z^2$ and that $X_1 = \alpha^7$, $X_2 = \alpha^{12}$, $Y_1 = \xi$, $Y_2 = 1$.

11
Linearized Polynomials and Affine Polynomials

11.1 HOW TO FIND THEIR ROOTS

We have seen that there often arise situations in which the decoder must find the roots in $GF(p^m)$ of $f(z)$, a polynomial over $GF(p^m)$. One common approach to this problem is the Chien search, as described in Sec. 5.4. If p^m is large, this search may involve a large amount of computation.

There is a special class of polynomials whose roots can be found with much greater ease. These polynomials were first investigated by Ore (1933, 1934), who called them *p polynomials.* However, I use the term *linearized polynomials,* for reasons which will be placed in evidence by Theorem 11.12.

Definition 11.11 A polynomial $L(z)$, over $GF(p^m)$, is said to be a *linearized polynomial* iff $L(z) = \sum_i L_i z^{p^i}$.

If $Z_k \in \mathrm{GF}(p)$ and $\alpha^0, \alpha^1, \alpha^2, \ldots, \alpha^{m-1}$ are a standard basis for $\mathrm{GF}(p^m)$, then

$$\Big(\sum_k Z_k\alpha^k\Big)^{p^i} = \sum_k Z_k{}^{p^i}(\alpha^k)^{p^i} = \sum_k Z_k(\alpha^k)^{p^i}$$

We state this important conclusion as a theorem.

Theorem 11.12 If $L(z)$ is a linearized polynomial, and if $z = \sum_k Z_k\alpha^k$, $Z_k \in \mathrm{GF}(p)$, then $L(z) = \sum_k Z_k L(\alpha^k)$.

If we also use the standard representation to express the values of the field elements $L(\alpha^0), L(\alpha^1), \ldots, L(\alpha^{m-1})$, we may write $L(\alpha^i) = \sum_{j=0}^{m-1} \mathcal{L}_{i,j}\alpha^j$, $\mathcal{L}_{i,j} \in \mathrm{GF}(p)$. The coefficients of the standard representation for the value of the polynomial $L(z)$ may therefore be obtained by post-multiplying the row vector $\mathbf{Z} = [Z_0, Z_1, \ldots, Z_{m-1}]$ by the $m \times m$ matrix $\mathcal{L}$ over $\mathrm{GF}(p)$

$$[Z_0, Z_1, \ldots, Z_{m-1}] \begin{bmatrix} \mathcal{L}_{0,0} & \mathcal{L}_{0,1} & \mathcal{L}_{0,2} & \cdots & \mathcal{L}_{0,m-1} \\ \mathcal{L}_{1,0} & \mathcal{L}_{1,1} & \mathcal{L}_{1,2} & \cdots & \mathcal{L}_{1,m-1} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ \mathcal{L}_{m-1,0} & \cdots & \cdots & \cdots & \mathcal{L}_{m-1,m-1} \end{bmatrix}$$

For example, let us consider the polynomial

$$L(z) = z^4 + \alpha^7 z^2 + \alpha^{18} z$$

over $\mathrm{GF}(2^5)$, where α is a root of the primitive irreducible binary polynomial $x^5 + x^2 + 1$. Using the tables of logs and antilogs in Appendix A, we may calculate

$$\begin{aligned} L(1) &= 1 + \alpha^7 + \alpha^{18} &&= \alpha + \alpha^2 + \alpha^4 \\ L(\alpha) &= \alpha^4 + \alpha^9 + \alpha^{19} &&= \alpha^2 + \alpha^3 \\ L(\alpha^2) &= \alpha^8 + \alpha^{11} + \alpha^{20} &&= \alpha + \alpha^2 \\ L(\alpha^3) &= \alpha^{12} + \alpha^{13} + \alpha^{21} &&= \alpha + \alpha^3 \\ L(\alpha^4) &= \alpha^{16} + \alpha^{15} + \alpha^{22} &&= 1 + \alpha^4 \end{aligned}$$

so $\mathcal{L}$ is represented by the 5×5 binary matrix

$$\mathcal{L} = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

If, for example we wish to find the roots of the polynomial

$$z^4 + \alpha^7 z^2 + \alpha^{18} z + \alpha^{10} \tag{10.62}$$

we may notice that this is equivalent to the equation

$$L(z) = \alpha^{10} = 1 + \alpha^4$$

which is equivalent to the simultaneous linear binary equations

$$[Z_0, Z_1, Z_2, Z_3, Z_4] \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \end{bmatrix} = [10001]$$

We may solve this set of linear equations by the methods of Sec. 2.5. The solutions are $\mathbf{Z} = [00001]$ and $\mathbf{Z} = [01111]$. Therefore, the roots of Eq. (10.62) are α^4 and $\alpha + \alpha^2 + \alpha^3 + \alpha^4 = \alpha^{24}$. These are the only two roots in GF(2^5); the other two roots lie in GF(2^{10}).

The polynomial of Eq. (10.62) is the difference of a linearized polynomial and a constant. I call such polynomials *affine* polynomials.

Definition 11.13 A polynomial $A(z)$, over GF(p^m), is said to be an *affine polynomial* iff $A(z) = L(z) - u$, where $L(z)$ is a linearized polynomial and u is a constant in GF(p^m).

As we saw in the example, the roots in GF(p^m) of any affine polynomial over GF(p^m) may be found by solving m simultaneous linear equations in m unknowns over GF(p).

In the special case of quadratic equations over GF(2^m), the solution of simultaneous linear binary equations may be avoided. By appropriate change of variables, the general quadratic equation without repeated roots may be reduced to the form $y^2 + y = u$. The linearized part of this polynomial is the same for all quadratic equations over a given field, so the solution to all quadratics may be computed once and for all and stored in an appropriate matrix. According to Theorems 6.69 and 6.695, the quadratic equation $y^2 + y = u$ has solutions in GF(2^m) iff Tr $(u) = 0$. Let α be any field element of degree m over GF(2), and let α^k be an element with Tr $(\alpha^k) = 1$. Then for $i = 0, 1, 2, \ldots, m - 1$, we may find a y_i such that

$$y_i^2 + y_i = \begin{cases} \alpha^i & \text{if Tr } (\alpha^i) = 0 \\ \alpha^i + \alpha^k & \text{if Tr } (\alpha^i) = 1 \end{cases} \tag{11.14}$$

In order to solve the equation $y^2 + y = u = \sum_{i=0}^{m-1} U_i\alpha^i$, $U_i \in \mathrm{GF}(2)$, we simply set $y = \sum_{i=0}^{m-1} U_i y_i$. We then have

$$y^2 + y = \sum_{i=0}^{m-1} U_i(y_i^2 + y_i) = u2^k + \sum_{i=0}^{m-1} U_i \operatorname{Tr}(\alpha^i)$$

$$= u + 2^k \operatorname{Tr}\left(\sum_{i=0}^{m-1} U_i\alpha^i\right) = u2^k + \operatorname{Tr}(u)$$

Therefore, if $\operatorname{Tr}(u) = 0$, the two solutions of the quadratic equation $y^2 + y = u$ are given by $y = \sum_{i=0}^{m-1} U_i y_i$ and $y' = 1 + \sum_{i=0}^{m-1} U_i y_i$.

Example 11.15 Let us consider quadratic equations over $\mathrm{GF}(2^5)$. If α is a root of $x^5 + x^2 + 1$, then $\operatorname{Tr}(\alpha) = \operatorname{Tr}(\alpha^2) = \operatorname{Tr}(\alpha^4) = 0$, but $\operatorname{Tr}(1) = \operatorname{Tr}(\alpha^3) = 1$. With $\alpha^k = 1$, it is easily verified that a solution to Eqs. (11.14) is given by

$$\begin{aligned} y_0 &= 0 && = 00000 \\ y_1 &= \alpha^3 && = 00010 \\ y_2 &= y_1^2 = \alpha + \alpha^3 && = 01010 \\ y_3 &= \alpha^2 + \alpha^4 && = 00101 \\ y_4 &= y_2^2 = \alpha + \alpha^2 + \alpha^3 && = 01110 \end{aligned}$$

Therefore, the solutions of the equation $y^2 + y = u$ may be found

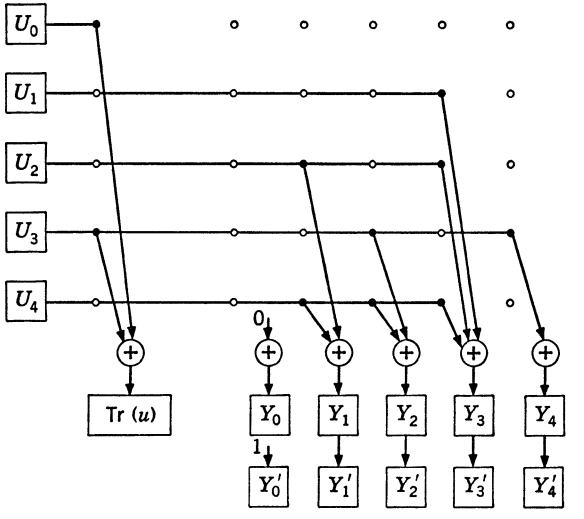


Fig. 11.1 Circuit for solving $y^2 + y = u$ in $\mathrm{GF}(2^5)$.

by the circuit of Fig. 11.1. If a 1 appears in the Tr (u) bit, then the quadratic equation $y^2 + y = u$ has no solutions. If a 0 appears in the Tr (u) bit, then the two solutions of the quadratic equation $y^2 + y = u$ appear in the y and y' registers. For example, if $u =$ 01001 as in Eq. (10.61), then the circuit of Fig. 11.1 computes Tr $(u) = 0$, $y = 01100 = \alpha^{19}$, $y' = 11100 = \alpha^{11}$.

11.2 THE LEAST AFFINE MULTIPLE

Unfortunately, most polynomials over GF(p^m) are not affine polynomials. Over fields of characteristic 2, quadratics are affine, but cubics are not. However, if we multiply the cubic equation by a suitably chosen linear factor, then we may obtain a quartic multiple which is affine. The roots of the cubic may then be found among the easily found roots of the affine quartic.

More generally, if we start with an arbitrary polynomial $f(z)$ of degree d, then we may find an affine multiple of $f(z)$ by Algorithm 11.21.

Algorithm 11.21 To find $L(z) - u$, an affine multiple of $f(z)$, a polynomial of degree d over GF(p^m), proceed as follows:

11.211.† Compute $z^k \bmod f(z)$ for $k = d, d+1, \ldots, (d-1)p$.

11.212. Using the results of step 11.211, compute the residues $r^{(0)}(z), r^{(1)}(z), \ldots, r^{(d-1)}(z)$, where $z^{p^j} \equiv r^{(j)}(z) \bmod f(z)$.

11.213. Solve the simultaneous linear equations

$$[u, L_0, L_1, L_2, \ldots, L_{d-1}] \begin{bmatrix} 0 & 0 & \cdots & 0 & -1 \\ r_{(d-1)}^{(0)} & \cdots & \cdots & r_1^{(0)} & r_0^{(0)} \\ r_{d-1}^{(1)} & \cdots & \cdots & r_1^{(1)} & r_0^{(1)} \\ r_{d-1}^{(d-1)} & \cdots & \cdots & r_1^{(d-1)} & r_0^{(d-1)} \end{bmatrix} = 0$$

This set of d equations in $d+1$ unknowns must always have at least one nonzero solution, which may be found by the methods described in Sec. 2.5. If $[u, L_0, L_1, \ldots, L_{d-1}]$ is a solution of these equations, then $A(z) = L(z) - u$ is an affine multiple of $f(z)$. If there are several solutions to the simultaneous linear equations at Step 11.213, we may select the monic solution for which the degree of $A(z)$ is smallest. This $A(z)$ is called the *least affine multiple* of $f(z)$.

†For very large p, it is easier to compute the residue of z^p by multiplying together an appropriate combination of the residues of $z, z^2, z^{2^2}, z^{2^3}, \ldots, z^{2^{\log p}}$. For $j = 1, 2, \ldots, d-1$, one then computes $r^{(j)}(z)$ from the congruence $r^{(j)}(z) \equiv (r^{(i-1)}(z))^p$.

Example 11.22 In GF(2^5) with $\alpha^5 + \alpha^2 + 1 = 0$, let

$$f(z) = z^5 + \alpha^{22}z^4 + \alpha^{18}z^3 + \alpha^{19}z^2 + \alpha^{16}z + \alpha^{13}$$

Modulo $f(z)$, we have

$$\begin{aligned}
z^5 &\equiv \alpha^{22}z^4 + \alpha^{18}z^3 + \alpha^{19}z^2 + \alpha^{16}z + \alpha^{13} \\
z^6 &\equiv \alpha^{22}z^5 + \alpha^{18}z^4 + \alpha^{19}z^3 + \alpha^{16}z^2 + \alpha^{13}z \\
&\equiv (\alpha^{18} + \alpha^{13})z^4 + (\alpha^{19} + \alpha^9)z^3 + (\alpha^{16} + \alpha^{10})z^2 \\
&\qquad + (\alpha^{13} + \alpha^7)z + \alpha^4 \\
&\equiv \alpha^{15}z^4 + \alpha^{13}z^3 + \alpha^6z^2 + \alpha^3z + \alpha^4 \\
z^7 &\equiv \alpha^{28}z^4 + \alpha^{12}z^3 + 0z^2 + \alpha^{10}z + \alpha^{28} \\
z^{2^3} = z^8 &\equiv \alpha^3z^4 + \alpha^{15}z^3 + \alpha^6z^2 + \alpha^6z + \alpha^{10} \\
z^{2^4} = z^{16} &\equiv \alpha^6z^8 + \alpha^{30}z^6 + \alpha^{12}z^4 + \alpha^{12}z^2 + \alpha^{20} \\
&\equiv \alpha^{29}z^4 + \alpha^{28}z^3 + \alpha^5z^2 + \alpha^6z + \alpha^{15}
\end{aligned}$$

$$[U,L_0,L_1,L_2,L_3,L_4]\begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ \alpha^3 & \alpha^{15} & \alpha^6 & \alpha^6 & \alpha^{10} \\ \alpha^{29} & \alpha^{28} & \alpha^5 & \alpha^6 & \alpha^{15} \end{bmatrix} = 0$$

whose solution is $[\alpha^4,\alpha^{20},\alpha^{18},\alpha^{30},\alpha^{13},1]$, so

$$U = \alpha^4 \qquad L(z) = \alpha^{20}z + \alpha^{18}z^2 + \alpha^{30}z^4 + \alpha^{13}z^8 + z^{16}$$

If we wish to find the roots in GF(2^5) of this affine polynomial, we compute

$$\begin{aligned}
L(1) &= \alpha^0 + \alpha^{13} + \alpha^{30} + \alpha^{18} + \alpha^{20} = 0 \\
L(\alpha) &= \alpha^{16} + \alpha^{21} + \alpha^3 + \alpha^{20} + \alpha^{21} = 1 + \alpha + \alpha^2 + \alpha^3 + \alpha^4 \\
L(\alpha^2) &= \alpha^1 + \alpha^{29} + \alpha^7 + \alpha^{22} + \alpha^{22} = 1 + \alpha + \alpha^2 + \alpha^3 + \alpha^4 \\
L(\alpha^3) &= \alpha^{17} + \alpha^6 + \alpha^{11} + \alpha^{24} + \alpha^{23} = 1 + \alpha + \alpha^2 + \alpha^3 \\
L(\alpha^4) &= \alpha^2 + \alpha^{14} + \alpha^{15} + \alpha^{26} + \alpha^{24} = 1 + \alpha + \alpha^2 + \alpha^3
\end{aligned}$$

so $\mathfrak{L}$ is represented by the 5×5 binary matrix

$$\mathfrak{L} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 \end{bmatrix}$$

The solutions of the equation $Z\mathfrak{L} = [0,0,0,0,1]$ are seen to be 00101, 00110, 01001, 01010, 10101, 10110, 11001, 11010, or $\log_\alpha z = 6, 7, 8, 17, 20, 22, 27, 30$. Among these 8 roots of $A(z)$ must lie all roots in GF(2^5) of $f(z)$. In this example, it happens that $f(z)$ has three roots in GF(2^5), namely, α^6, α^8, α^{17}.

★ 11.3 ABSTRACT PROPERTIES OF LINEARIZED AND AFFINE POLYNOMIALS

In this section we derive a number of properties of affine polynomials. In the subsequent sections of this chapter, we shall show how these results may be applied to certain questions in coding theory. The essential ideas of all of the results in this section were first presented by Ore (1933, 1934), although most of these theorems have recently been rediscovered by several coding theorists independently.

Theorem 11.31 A polynomial $L(z)$ is linearized iff its roots form a linear space over GF(p) and each root has the same multiplicity, which is a power of p.

Remark Note that the roots of $L(z)$ need not be in GF(p^m); they may be in some larger extension field.

Proof of "only if" If $L(z)$ is linearized and $r_1, r_2, \ldots$ are roots of $L(z)$, then $L(\Sigma c_k r_k) = \Sigma c_k L(r_k) = 0$, and $\Sigma c_k r_k$ is also a root, for any combination of $c_k \in \mathrm{GF}(p)$. Therefore the roots of $L(z)$ form a linear space over GF(p).

The derivative of $L(z) = \sum_i L_i z^{p^i}$ is L_0. If this is nonzero, then $L(z)$ has no repeated roots. On the other hand, if $L_0 = L_1 = \cdots = L_{k-1} = 0$, but $L_k \neq 0$, then

$$L(z) = \sum_i L_i z^{p^i} = \Big(\sum_i L_i{}^{p^{-k}} z^{p^{i-k}}\Big)^{p^k}$$

which is the p^k power of a linearized polynomial which has no repeated roots. In this case every root of $L(z)$ has multiplicity p^k.

Notice that if $L_i \in \mathrm{GF}(p^m)$, then $L_i{}^{p^{-k}}$ is uniquely defined in GF(p^m).

Proof of "if" If the roots of $L(z)$ form a linear space over GF(p), then we may select $r_1, r_2, \ldots, r_n$ as a basis of that space. Each distinct root of $L(z)$ may then be expressed uniquely as $r = \sum_{i=1}^{n} c_i r_i$ for some appropriate choice of the c_i's. Without loss of generality, we may assume that $L(z)$ is monic and write

$$L(z) = \Big[\prod_{c_n=0}^{p-1} \prod_{c_{n-1}=0}^{p-1} \cdots \prod_{c_1=0}^{p-1} \Big(z - \sum_{i=1}^{n} c_i r_i\Big)\Big]^{p^k}$$

We define

$$
\begin{aligned}
f^{(0)}(z) &= z \\
f^{(1)}(z) &= \prod_{c_1=0}^{p-1} (z - c_1 r_1) \\
f^{(2)}(z) &= \prod_{c_2=0}^{p-1} \prod_{c_1=0}^{p-1} (z - c_1 r_1 - c_2 r_2) \\
&= \prod_{c_2=0}^{p-1} [f^{(1)}(z - c_2 r_2)] \\
&\cdots\cdots\cdots\cdots\cdots\cdots \\
f^{(i+1)}(z) &= \prod_{c_{i+1}=0}^{p-1} \prod_{c_i=0}^{p-1} \cdots \prod_{c_1=0}^{p-1} \left(z - \sum_{j=1}^{i+1} c_j r_j\right) \\
&= \prod_{c_{i+1}=0}^{p-1} f^{(i)}(z - c_{i+1} r_{i+1})
\end{aligned}
$$

It is evident that $L(z) = [f^{(n)}(z)]^{p^k}$.

We now show by induction on i that $f^{(i)}(z)$ is a linearized polynomial. It is obviously true for $i = 0$. If $f^{(i)}(z)$ is linearized, then

$$f^{(i)}(z - c_{i+1} r_{i+1}) = f^{(i)}(z) - c_{i+1} f^{(i)}(r_{i+1})$$

and

$$
\begin{aligned}
f^{(i+1)}(z) &= \prod_{c_{i+1}=0}^{p-1} [f^{(i)}(z) - c_{i+1} f^{(i)}(r_{i+1})] \\
&= [f^{(i)}(z)]^p - f^{(i)}(z)[f^{(i)}(r_{i+1})]^{p-1}
\end{aligned}
$$

Since $f^{(i)}(z)$ is linearized, so is its scalar multiple $f^{(i)}(z)[f^{(i)}(r_{i+1})]^{p-1}$ and its p power, $[f^{(i)}(z)]^p$. Noting that the difference of two linearized polynomials is linearized, we deduce that $f^{(i+1)}(z)$ is linearized. By induction, $f^{(n)}(z)$ is linearized. Since the p^k power of a linearized polynomial is linearized, so is $L(z)$. Q.E.D.

The analogous result for affine polynomials is the following theorem.

Theorem 11.32 A polynomial $A(z)$ is an affine polynomial iff its roots form an affine space over GF(p) [that is, if $s_1, s_2, \ldots, s_n$ are roots, then $s_1 - \Sigma c_k(s_k - s_1)$ is also a root, for any $c_k \in \mathrm{GF}(p)$] and each root has the same multiplicity, which is a power of p.

Proof If $A(z)$ is affine, then $A(z) = L(z) - u$. If $s_1, s_2, \ldots, s_k, \ldots, s_n$ are roots of $A(z)$, then $L(s_k) = u$ and $s_k - s_1$ is a root of $L(z)$, for each $k = 1, 2, 3, \ldots, n$. Therefore $L(s_1 + \Sigma c_k(s_k - s_1)) = L(s_1) = u$, and $[s_1 + \Sigma c_k(s_k - s_1)]$ is a root of $A(z)$. This shows that the roots of an affine polynomial form an affine space.

The derivative of an affine polynomial $A(z) = L(z) - u$ is given by the coefficient of z, which is L_0. If this is nonzero, all roots of $A(z)$ are distinct. However, if $L_0 = L_1 = \cdots = L_{k-1} = 0$ but $L_k \neq 0$, then $A(z)$ is a p^k power of an affine polynomial with distinct roots, and all roots of $A(z)$ have multiplicity p^k.

On the other hand, if the roots of some polynomial $A(z)$ form an affine space, then the differences of these roots form a linear space. According to Theorem 11.31, the polynomial whose roots are the distinct elements of this linear space is a linearized polynomial $L(z)$. If s is any root of $A(z)$, we have $L(s - s_1) = 0$, or $L(s) - u = 0$, where $u = L(s_1)$. Hence, the roots of $A(z)$ are the roots of $L(z) - u$, and since every root of $A(z)$ has multiplicity p^k, $A(z)$ is a scalar multiple of $[L(z) - u]^{p^k} = [L(z)]^{p^k} - u^{p^k}$, which is an affine polynomial. Q.E.D.

The distinct roots of every affine polynomial form an affine space. The intersection of any two affine spaces is another affine space. Since the common roots of any two polynomials are the roots of the greatest common divisor of those two polynomials, we deduce that if A and B are affine polynomials with distinct roots, then the gcd of A and B is another affine polynomial with distinct roots. Furthermore, the gcd of A^{p^i} and B^{p^j} is the gcd of A and B raised to the p^i power or to the p^j power, whichever is the smaller. From this we deduce Theorem 11.331.

Theorem 11.331 The greatest common divisor of two affine polynomials is an affine polynomial.

If both of these affine polynomials have 0 as a root, then 0 is also a root of their gcd. This proves Theorem 11.332.

Theorem 11.332 The greatest common divisor of two linearized polynomials is a linearized polynomial.

If A and B are two affine polynomials and A divides B, then the roots of A are a subset of the roots of B. The affine root space of A is

therefore an affine subspace of the root space of B. The differences of pairs of elements in the affine root space of A are among the differences of pairs of elements of the affine root space of B. But, as seen in the proof of Theorem 11.32, each difference of a pair of roots of B is a root of the linearized part of B, and each difference of a pair of roots of A is a root of the linearized part of A. Since the multiplicity of the roots of the linearized part of any affine polynomial is equal to the multiplicity of the roots of the affine polynomial, we have the following result.

Theorem 11.34 If A and B are affine polynomials and A divides B, then the linearized part of A divides the linearized part of B. Conversely, if L and M are linearized polynomials in z, and if L divides M, then $L(z) - L(r) = L(z - r)$ divides $M(z - r) = M(z) - M(r)$ for any r in the coefficient field of a and b.

Often one would like to know whether or not all the roots of a given affine polynomial $A(z) = L(z) - u$ are in the field $\mathrm{GF}(p^m)$. If $A(z)$ divides $z^{p^m} - z$, then all roots of $A(z)$ are in $\mathrm{GF}(p^m)$, but if $A(z)$ does not divide $z^{p^m} - z$, then some of the roots of $A(z)$ are outside of $\mathrm{GF}(p^m)$.† According to Theorem 11.34, $A(z)$ cannot divide $z^{p^m} - z$ unless $L(z)$ divides $z^{p^m} - z$. If $L(z)$ does divide $z^{p^m} - z$, then $A(z)$ divides $z^{p^m} - z$ iff there exists an element r such that $L(r) = u$ and $r^{p^m} - r = 0$ or, equivalently, $r \in \mathrm{GF}(p^m)$. Evidently, for some u's, there exist r's in $\mathrm{GF}(p^m)$ such that $L(r) = u$. These u's form a linear space, called the *range space*, of the linearized polynomial L. If u belongs to this range space, then $L(z) - u$ divides $z^{p^m} - z$, and all roots of $L(z) - u$ are in $\mathrm{GF}(p^m)$. However, if u does not belong to this range space, then there is no r in $\mathrm{GF}(p^m)$ such that $L(r) = u$, and all roots of $L(z) - u$ lie outside of $\mathrm{GF}(p^m)$. For any given linearized polynomial $L(z)$, the question of which elements of $\mathrm{GF}(p^m)$ lie in the range space of $L(z)$ and which elements do not is answered by the following theorem.

Theorem 11.35 Let $L(z)$ be any monic linearized polynomial over $\mathrm{GF}(p^m)$ such that $L(z)$ divides $z^{p^m} - z$. Then there exists a unique monic linearized polynomial $L^*(z)$ which also divides $z^{p^m} - z$, such that:

11.351. $L^*(L(z)) = z^{p^m} - z$.
11.352. $L(L^*(z)) = z^{p^m} - z$.
11.353. $L(z) - u$ has roots in $\mathrm{GF}(p^m)$ iff $L^*(u) = 0$.
11.354. $L^*(z) - u$ has roots in $\mathrm{GF}(p^m)$ iff $L(u) = 0$.

I call the polynomials $L(z)$ and $L^*(z)$ *duals*.

†$A(z)$ is assumed to have distinct roots.

Example 11.355 In GF(2^5), let $L(z) = z^2 + z$; then

$$L^*(z) = z^{16} + z^8 + z^4 + z^2 + z$$

and we can verify that

$$L^*(L(z)) = (z^2 + z)^{16} + (z^2 + z)^8 + (z^2 + z)^4 + (z^2 + z)^2 + (z^2 + z) = z^{32} + z = z^{32} - z$$

$$L(L^*(z)) = (z^{16} + z^8 + z^4 + z^2 + z)^2 + (z^{16} + z^8 + z^4 + z^2 + z) = z^{32} + z = z^{32} - z$$

The equation $z^2 + z = u$ has roots in GF(2^5) iff $u^{16} + u^8 + u^4 + u^2 + u = 0$; that is $\mathrm{Tr}(u) = 0$. The equation $z^{16} + z^8 + z^4 + z^2 + z = u$ has roots in GF(2^5) iff $u^2 + u = 0$; that is, $u \in$ GF(2).

More generally, in GF(p^m), the linearized polynomials $z^p - z$ and $\sum_{k=0}^{m-1} z^{p^k}$ are duals of each other.

Example 11.356 As a more complicated example in GF(2^5), with α a root of $x^5 + x^2 + 1$,† we let

$$L(z) = z^4 + \alpha^9 z^2 + \alpha^{18} z = z(z - \alpha^7)(z - \alpha^{15})(z - \alpha^{27})$$

$$L^*(z) = z^8 + \alpha^{10} z^4 + \alpha^{17} z^2 + \alpha^{13} z = z(z - \alpha^5)(z - \alpha^7)(z - \alpha^{10})(z - \alpha^{14})(z - \alpha^{20})(z - \alpha^{21})(z - \alpha^{29})$$

After checking that $L^*(\alpha) = \alpha^{27}$, $L^*(\alpha^2) = \alpha^{15}$; $L(\alpha) = \alpha^{10}$, $L(\alpha^3) = \alpha^{29}$, $L(\alpha^4) = \alpha^{14}$, all cases of Claims 11.353 and 11.354 may be verified by summing appropriate combinations of these equations, for example, $L^*(\alpha + \alpha^2) = \alpha^{27} + \alpha^{15} = \alpha^7$.

Proof of Theorem 11.35 Let $r_1, r_2, \ldots, r_k$ be a basis for the root space of L, and let $r_1, r_2, \ldots, r_k, r_{k+1}, r_{k+2}, \ldots, r_m$ be a basis of GF(p^m) over GF(p). Define

$$L^*(z) = \prod_{c_m=0}^{p-1} \prod_{c_{m-1}=0}^{p-1} \cdots \prod_{c_{k+1}=0}^{p-1} \left[z - \sum_{i=k+1}^{m} c_i L(r_i) \right]$$

Then

$$L(z) = \prod_{c_k=0}^{p-1} \prod_{c_{k-1}=0}^{p-1} \cdots \prod_{c_1=0}^{p-1} \left(z - \sum_{i=1}^{k} c_i r_i \right)$$

† Tables of $\log_\alpha$ and $\mathrm{antilog}_\alpha$ are given in Appendix A.

and

$$z^{p^m} - z = \prod_{c_m=0}^{p-1} \prod_{c_{m-1}=0}^{p-1} \cdots \prod_{c_{k+1}=0}^{p-1} L\left(z - \sum_{i=k+1}^{m} c_i r_i\right)$$
$$= \prod_{c_m=0}^{p-1} \prod_{c_{m-1}=0}^{p-1} \cdots \prod_{c_{k+1}=0}^{p-1} \left[L(z) - \sum_{i=k+1}^{m} c_i L(r_i)\right]$$
$$= L^*(L(z))$$

This proves Claim 11.351. The equation

$$z^{p^m} - z = \prod_{c_m=0}^{p-1} \prod_{c_{m-1}=0}^{p-1} \cdots \prod_{c_{k+1}=0}^{p-1} \left[L(z) - \sum_{i=k+1}^{m} c_i L(r_i)\right]$$

gives a factorization of $z^{p^m} - z$ into factors of the form $L(z) - u$, where each u is a root of $L^*(z)$. This proves Claim 11.353. This argument also shows that the dual of L is unique, for the root space of the dual of L must be the range space of L.

To prove Claim 11.352, we compute $L(L^*(L(z))) = L(z^{p^m} - z) = L(z^{p^m}) - L(z)$. Since L is a linearized polynomial over $\mathrm{GF}(p^m)$, $L(z^{p^m}) = [L(z)]^{p^m}$ and we have $L(L^*(L(z))) = [L(z)]^{p^m} - L(z)$. Setting $y = L(z)$, this becomes $L(L^*(y)) = y^{p^m} - y$. This must be an identity in y, and we deduce that $L(L^*(z)) = z^{p^m} - z$, proving Claim 11.352.

To prove Claim 11.354, we let $r_1^*, r_2^*, \ldots, r_{m-k}^*$ be a basis for the root space of L^*, and we let $r_1^*, r_2^*, \ldots, r_{m-k}^*, \ldots, r_m^*$ be a basis of $\mathrm{GF}(p^m)$. The remainder of the proof of Claim 11.354 is the dual of the proof of Claim 11.353. Q.E.D.

If all the coefficients of a linearized polynomial lie in the ground field $\mathrm{GF}(p)$, then certain questions about this linearized polynomial may be answered by considering its conventional associate, defined as follows.

Definition 11.36 If $f_i \in \mathrm{GF}(p)$ for all i, then the polynomials $F(z) = \sum_i f_i z^{p^i}$ and $f(z) = \sum_i f_i z^i$ are said to be *associates*. F is the linearized associate of f, and f is the conventional associate of F.

The utility of this definition is indicated by the following theorem.

Theorem 11.37 Let F and G be linearized polynomials over $\mathrm{GF}(p)$, with conventional associates f and g. Then F divides G iff f divides g. If $g = hf$, then $G = H(F) = F(H)$, where H is the linearized associate of h.

Proof The equations

$$\sum_k g_k z^k = \sum_i h_i z^i \sum_j f_j z^j$$

and

$$\sum_k g_k z^{p^k} = \sum_i h_i \left(\sum_j f_j z^{p^j}\right)^{p^i} = \sum_i h_i \sum_j f_j z^{p^{j+i}}$$

are each true iff $g_k = \sum_j h_{k-j} f_j$ for every k.

Hence, if $g = hf$ then $G = H(F)$ and F divides G. Conversely, if F divides G, construct H using the method of the proof of Theorem 11.35 to obtain $G = H(F)$. Then $g = hf$. Q.E.D.

Corollary 11.38 If $L(z)$ is a linearized polynomial over GF(p), then L divides $z^{p^m} - z$ iff its conventional associate, $l(z)$, divides $z^m - 1$. If L divides $z^{p^m} - z$, then the conventional associate of L^*, the dual of L, is given by

$$l^*(z) = \frac{z^m - 1}{l(z)}$$

Example 11.381 Corresponding to the conventional factorization $z^m - 1 = (z - 1) \sum_{k=0}^{m-1} z^k$, one has the associated linearized factorization $z^{p^m} - z = (z^p - z)\left[\sum_{k=0}^{m-1} z^{p^k}\right]$. These two factors are duals.

Using this corollary, we may determine the number of elements of GF(q^m) which have m linearly independent conjugates over GF(q).† If $v \in \mathrm{GF}(q^m)$, then $v, v^q, v^{q^2}, \ldots, v^{q^{m-1}}$ are linearly dependent iff there exist $f_0, f_1, \ldots, f_{m-1} \in \mathrm{GF}(q)$ such that $\sum_i f_i v^{q^i} = 0$. Since v is a root of $\sum_i f_i z^{q^i}$ and of $z^{q^m} - z$, it also is a root of their gcd. The gcd of these two linearized polynomials is linearized. Thus, v has linearly dependent conjugates over GF(q) iff v is a root of some polynomial $\sum_i H_i z^{q^i}$ which divides $z^{q^m} - z$.

Let $m = p^j n$; gcd $(n,p) = 1$, so $z^m - 1 = (z^n - 1)^{p^j} = \prod_k (g^{(k)})^{p^j}$, where the polynomials $g^{(k)}(z)$ are the distinct irreducible factors of $z^n - 1$. We let d_k denote the degree of $g^{(k)}$. Clearly $\sum_k d_k = n$. The maximal

† At this point, if q is a prime power rather than a prime, we must redefine linearized polynomials as those of the form $\sum_i L_i z^{q^i}$ rather than $\sum_i L_i z^{p^i}$. All previous results still hold.

proper divisors of $z^m - 1$ are of the form $h^{(k)} = (z^m - 1)/g^{(k)}$. Every proper divisor of $z^m - 1$ divides at least one of these maximal divisors. Corresponding to each maximal divisor of $z^m - 1$ is a maximal linearized divisor of $z^{q^m} - z$, namely, $H^{(k)}(z) = \sum_i h_i^{(k)} z^{q^i}$; $\deg H^{(k)} = q^{[m-d_k]}$. An element $v \in \mathrm{GF}(q^m)$ has m linearly independent conjugates iff it is not a root of any of these polynomials, $H^{(k)}$.

The gcd of several maximal divisors of $z^m - 1$, say, $h^{(1)}, h^{(2)}$, and $h^{(3)}$, is $(z^m - 1)/g^{(1)}g^{(2)}g^{(3)}$, which has degree $m - (d_1 + d_2 + d_3)$. Correspondingly, the gcd of $H^{(1)}, H^{(2)}$, and $H^{(3)}$ is the linearized polynomial of degree $q^{[m-d_1-d_2-d_3]}$. If an element u is selected from $\mathrm{GF}(q^m)$ at random, the probability that $z - u$ divides $H^{(1)}$ is q^{-d_1}; the probability that $z - u$ divides $H^{(2)}$ is q^{-d_2}; the probability that $z - u$ divides both $H^{(1)}$ and $H^{(2)}$ is $q^{-d_1-d_2}$. Thus, dividing $H^{(1)}$ and dividing $H^{(2)}$ are independent events. Similarly, dividing $H^{(1)}$, dividing $H^{(2)}$, dividing $H^{(3)}$, ... are all independent events. The probability that the randomly selected $z - u$ does not divide $H^{(k)}$ is $1 - q^{-d_k}$. By statistical independence, we conclude that the probability that a randomly selected $z - u$ divides none of the H polynomials is given by $\prod_k (1 - q^{-d_k})$. This proves Theorem 11.39.

Theorem 11.39 The number of elements in $\mathrm{GF}(q^m)$ which have m linearly independent conjugates over $\mathrm{GF}(q)$ is given by

$$q^m \prod_k (1 - q^{-d_k})$$

where the d_k are the degrees of the distinct irreducible factors of $z^m - 1$ over $\mathrm{GF}(q)$.

Example 11.391 Consider $\mathrm{GF}(2^{14})$. Over $\mathrm{GF}(2)$,

$$z^{14} - 1 = (z^7 - 1)^2 = (Q^{(1)}Q^{(7)})^2$$

$Q^{(1)}$ is irreducible of degree 1, and $Q^{(7)}$ is the product of two irreducible cubics. So $d_1 = 1$, $d_2 = d_3 = 3$, and the number of elements of $\mathrm{GF}(2^{14})$ having 14 linearly independent binary conjugates is $2^{14}(\frac{1}{2})(\frac{7}{8})(\frac{7}{8}) = 6272$. These elements lie in $6272/14 = 448$ conjugate sets. There are 448 irreducible binary 14-ics whose roots are linearly independent.

Readers having a distaste for the probabilistic arguments used in the above derivation may prove the same result using the combinatorial

principle of inclusion and exclusion. In my opinion, the probabilistic approach makes the result more meaningful. The probabilistic approach to enumerations has also found application in a wide variety of other problems, as indicated by Kac (1959).

★ 11.4 TRANSFORMATIONS OF $\tilde{f}(z)$

Often it is possible to eliminate certain terms from an algebraic equation in z by a substitution $y = (az + b)/(cz + d)$, where a, b, c, and d are elements in GF(q). Usually the most judicious choices for the elements a, b, c, and d are most readily found by proceeding in smaller steps, each step consisting only of a translation ($y = z - a$), a reciprocation ($y = 1/z$), or a scaling ($y = az$). For example, suppose we start with the general quintic over a field of characteristic 2

$$z^5 + az^4 + bz^3 + cz^2 + dz + e = 0$$

If we translate by Δ, we obtain

$$\begin{array}{llllll} y^5 + \Delta y^4 + & & + & \Delta^4 y + & \Delta^5 & \\ + ay^4 & & & & + a\Delta^4 & \\ & + by^3 + b\,\Delta y^2 & & + b\,\Delta^2 y + & b\Delta^3 & \\ & + cy^2 & & & + c\Delta^2 & \\ & & + & dy + & d\Delta & \\ & & & & + e & \end{array}$$

We can eliminate the quartic term by the choice $\Delta = a$, or we can eliminate the quadratic term by the choice $\Delta = c/b$. The latter choice proves more fruitful. After eliminating the quadratic term, we can reciprocate to obtain an equation without a cubic term. By appropriate translations of this equation, we can eliminate either the quartic term or the linear term, without the cubic term's reappearing. If we eliminate the quartic term and then normalize the linear term to 1 by an appropriate choice of scale factor, we can reduce the general quintic to the form

$$u^5 + fu^2 + u + g = 0$$

or

$$u^5 + fu^2 + g = 0$$

There arises the question as to whether or not such preliminary transformations are of much value in simplifying the subsequent calculations of the least affine multiple and its roots. Since the reduced quintic has fewer terms than the original quintic, we might hope that its least affine multiple might have degree 8, instead of degree 16. Unfortunately this is not the case, as may be shown by a straightforward but

lengthy calculation which we shall omit. Neither does one succeed in guaranteeing a least affine multiple of degree 8 by choosing alternate reduced forms in which the quadratic and linear, or the quartic and quadratic, or the cubic and linear terms are missing. By eliminating the quartic term, one does guarantee a simplification of the least affine multiple, but unfortunately this simplification evidences itself in the vanishing of the lowest term rather than the highest term. The generalization of this phenomenon is the following theorem.

Theorem 11.41 If

$$\tilde{f}(z) = z^d + \sum_{k=1}^{d} f_k z^{d-k} \qquad f_k \in \mathrm{GF}(q)$$

is any factor of an affine polynomial $A(z)$ over $\mathrm{GF}(q)$, and if d is not a multiple of p, then f_1/d is a root of $A(z)$.

Remark Here $f_1/d = f_1 d^{-1}$, where $d^{-1} \in \mathrm{GF}(p)$ is the multiplicative inverse of d mod p.

Proof The roots of $A(z)$ include all elements of the affine space spanned by the roots of $\tilde{f}$, which we call $r_1, r_2, \ldots, r_d$. Since $\sum_{k=1}^{d} r_k = f_1$, we have

$$\frac{f_1}{d} = r_1 + \sum_{k=1}^{d} \frac{r_k - r_1}{d}$$

Thus f_1/d must be a root of $A(z)$. Q.E.D.

When the degree of $\tilde{f}$ is not a multiple of p, the elimination of f_1 is accomplished by the straightforward translation $y = z - f_1/d$. The least affine multiple of the reduced polynomial will then have 0 as a root, but its degree may still be as great as p^{d-1}. Since the transformation to eliminate f_1 is so straightforward, it is not surprising that it proves of so little value.

On the other hand, if the degree of $\tilde{f}$ is a multiple of p, then no translation will affect f_1. However, if we manage to find a translation which causes f_{d-1} to vanish, we may then reciprocate to obtain a reduced form in which $f_1 = 0$. The reward for this extra effort is guaranteed by the following theorem.

Theorem 11.42 If $\deg \tilde{f}$ is a multiple of p, and if $f_1 = 0$, then the least affine multiple of $\tilde{f}$ has degree at most f^{d-2}.

Proof If $\sum_{i=1}^{d} r_i = 0$, and d is a multiple of p, then

$$\sum_{i=1}^{d} (r_i - r_1) = 0 - dr_1 = 0 \qquad \text{and} \qquad r_d = r_1 - \sum_{i=1}^{d-1} (r_i - r_1)$$

Thus, the affine space spanned by the roots $r_1, r_2, \ldots, r_d$ is the same as the affine space spanned by the roots $r_1, r_2, \ldots, r_{d-1}$. The affine space spanned by any $d - 1$ elements has dimension at most $d - 2$, which proves the claim. Q.E.D.

As one example of this theorem, consider the general quartic equation over a field of characteristic 2. By appropriate translation, the linear term may be eliminated, after which a reciprocation results in a standard form with no cubic term. The least affine multiple of this reduced quartic, according to Theorem 11.42, has degree at most $2^{[4-2]} = 4$. This assertion is confirmed by the observation that the reduced quartic is itself an affine polynomial.

In general, the elimination of the linear term from a polynomial of even degree d over a field of characteristic 2 requires the solution of another algebraic equation of degree $(d - 2)/2$. For example, to eliminate the linear term of the sextic

$$z^6 + az^5 + bz^4 + cz^3 + dz^2 + ez + f$$

by the translation $z = y + \Delta$, one must choose Δ so that

$$a(\Delta^2)^2 + c(\Delta^2) + e = 0$$

If this equation has solutions in the original field, one can translate and reciprocate to a reduced sextic whose least affine multiple has degree no greater than 16; if this equation has no solutions in the field, then one has only the original sextic, whose least affine multiple may have degree as large as 32.

★ 11.5 ROOT COUNTING

After one has calculated A, the least affine multiple of $\tilde{f}$, and then solved for the roots of the least affine multiple in GF(q), there still remains the problem of deciding which of these roots of A are also roots of $\tilde{f}$. In general, there appears to be no alternative but a search among these roots.

However, certain general statements can be made about the *number* of roots of $\tilde{f}$ in GF(q), based only on the number of roots of A in GF(q) and the total number of roots of A. For example, suppose $\tilde{f}$ is a quintic over a field of characteristic 2 and that its least affine multiple A is of degree 16 and has 8 roots in the field. How many of these 8 roots of A are also roots of $\tilde{f}$? One might naïvely suspect that there could be any number from 0 to 5. According to the following theorem, however, the only possibilities are 1 and 3.

Root-counting theorem 11.51 Let $\tilde{f}$ be a polynomial over GF(q) of characteristic p, and suppose $\tilde{f}$ has no repeated roots. Let N be the number of distinct roots of $\tilde{f}$ outside of GF(q), and let A be the least affine multiple of $\tilde{f}$.

If A has 0 roots in GF(q), then $N = \deg \tilde{f}$. In this case $\tilde{f}$ and each of its irreducible factors have degrees which are multiples of p.

If the number of distinct roots of A in GF(q) is positive, it is a power of p, p^I. The total number of distinct roots of A is of the form p^{I+J}.

If $J = 0$, $N = 0$.

If $J \geq 1$, $N \geq J + 1$.

If $J = 1$, then N has a nonunit factor in common with $p(p - 1)$.

If $J = 2$, then either N has at least one nonunit factor $\leq p + 1$ or N is of the form $N = Bn_1 + Cn_2$, where B and C are positive integers and n_1 and n_2 each has a nonunit factor in common with $p(p - 1)$.

Proof If A has no roots in GF(q), then by Theorem 11.41 every factor of A over GF(q) has degree which is a multiple of p. On the other hand, if r is a root of $A(z)$ in GF(q), then we may translate $A(z)$ to $A(z - r)$, which translates $\tilde{f}(z)$ to $\tilde{f}(z - r)$. This translation moves roots in GF(q) to roots in GF(q) and roots outside of GF(q) to roots outside of GF(q). The advantage of the translation is that it translates the affine space spanned by the roots of $\tilde{f}(z)$ to the linear space spanned by the roots of $\tilde{f}(z - r)$.

The roots of $A(z - r)$ in GF(q) form a linear space over GF(p), and hence there are p^I such roots, having a basis which we may denote by $u_1, u_2, \ldots, u_I$. The set of all roots of A, including those in extension fields of GF(q), is also a vector space over GF(p). The linearly independent set of elements $u_1, u_2, \ldots, u_I$ may be augmented to form a basis for the space of all roots by including the additional basis vectors $v_1, v_2, \ldots, v_J$. Without loss of

generality, we may assume that the basis vectors $u_1, u_2, \ldots, u_I$, $v_1, v_2, \ldots, v_J$ are chosen as a subset of the roots of the polynomial $\tilde{f}(z - r)$. The space spanned by the vectors $u_1, u_2, \ldots, u_I$, $v_1, v_2, \ldots, v_J$ is identical to the space spanned by the roots of $\tilde{f}(z - r)$, which we denote by $s_1, s_2, \ldots, s_K, t_1, t_2, \ldots, t_N$, where $s_i \in \mathrm{GF}(q)$ and $t_i \notin \mathrm{GF}(q)$. Clearly if $J = 0$, then $N = 0$. If $J > 0$, then $N > 0$, and the space is also spanned by the set of elements $s_1, s_2, \ldots, s_K, \sum_{i=1}^{N} t_i, t_1, t_2, \ldots, t_{N-1}$. Since $\sum_{i=1}^{K} s_i + \sum_{i=1}^{N} t_i \in \mathrm{GF}(q)$, $\sum_{i=1}^{N} t_i \in \mathrm{GF}(q)$. Thus the space spanned by the roots of $\tilde{f}(z - r)$ has a basis consisting of at most $N - 1$ elements outside of $\mathrm{GF}(q)$. So $J \leq N - 1$ and $N \geq J + 1$, as claimed.

If $J = 1$, each of the N elements $t_1, t_2, \ldots, t_N$ must be of the form $t_i = C_i v + u_i$, where $C_i \in \mathrm{GF}(p)$, $u_i \in \mathrm{GF}(q)$, $v \notin \mathrm{GF}(q)$. Taking conjugates gives $t_i{}^{q^n} = C_i v^{q^n} + u_i$, which equals t_i iff v^{q^n} equals v. Thus, the number of conjugates of each root is the same as the number of conjugates of v. Since $t_i{}^q = t_j$, we have $C_i v^q + u_i = C_j v + u_j$ and $v^q = (C_j/C_i)v + (u_j - u_i)/C_i = Cv + u$. Taking conjugates gives $v^{q^2} = C^2 v + (C + 1)u$, and, in general, $v^{q^n} = C^n v + \left(\sum_{i=0}^{n-1} C^i\right) u$. If $C = 1$, then $v^{q^n} = v$ iff n is a multiple of p. In this case v has p conjugates. If $C \neq 1$, then $v^{q^n} = v$ iff $C^n = 1$. Since $C \in \mathrm{GF}(p)$, $C^{p-1} = 1$, and the number of conjugates of v is a divisor of $p - 1$.

Next suppose $J = 2$. We first suppose that $v_1{}^q = C_1 v_1 + u_1$ and that $v_2{}^q = C_2 v_2 + u_2$; $C_i \in \mathrm{GF}(p)$, $u_i \in \mathrm{GF}(q)$. Let n_1 be the number of conjugates of v_1, and let n_2 be the number of conjugates of v_2. Each root of $\tilde{f}(z - r)$ not in $\mathrm{GF}(q)$ must be of the form

$$t_i = C_i v_1 + D_i v_2 + u_i$$
$$t_i{}^{q^n} = C_i v_1{}^{q^n} + D_i v_2{}^{q^n} + u_i$$

so that the number of conjugates of t_i is given by n_1 if $D_i = 0$, or n_2 if $C_i = 0$, or the least common multiple of n_1 and n_2 if both C_i and D_i are nonzero. Evidently N must be of the form $Bn_1 + Cn_2$, for some positive integers B and C. By the previous part of this theorem (the case $J = 1$), it is clear that both n_1 and n_2 must each have a nonunit factor in common with $p(p - 1)$.

If there do not exist constants $C_1 \in \mathrm{GF}(p)$ and $u_1 \in \mathrm{GF}(q)$ such that $v_1{}^q = C_1 v_1 + u_1$, then the space spanned by $u_1, u_2, \ldots, u_I, v_1, v_2$ is the same as the space spanned by $u_1, u_2, \ldots, u_I, v_1, v_1{}^q$. Similarly, if there do not exist constants $C_2 \in \mathrm{GF}(p)$ and $u_2 \in \mathrm{GF}(q)$

such that $v_2{}^q = C_2v_2 + u_2$, then we may take $u_1, u_2, \ldots, u_I, v_2, v_2{}^q$ as a basis for the roots of $A(z - r)$. In either case, we may find a v in terms of which every root of $A(z - r)$ outside GF(q) may be expressed as

$$t_i = C_iv + D_iv^q + u_i \qquad C_i,D_i \in \mathrm{GF}(p) \qquad u_i \in \mathrm{GF}(q)$$
$$t_i{}^{q^n} = C_iv^{q^n} + D_iv^{q^{n+1}} + u_i$$

Consequently, every root of $\tilde{f}(z - r)$ outside of GF(q) has the same number of conjugates, which is equal to the number of conjugates of v. For each j we may write

$$v^{q^j} = A_jv^q + B_jv + u_j \qquad A_j,B_j \in \mathrm{GF}(p) \qquad u_j \in \mathrm{GF}(q)$$

Let k be that integer for which $A_1 \neq 0$, $A_2 \neq 0$, $\ldots$, $A_{k-1} \neq 0$, $A_k = 0$. For $2 \leq j < k$ we may write

$$v^{q^j} = C_jv^{q^{j-1}} + D_jv + u_j \qquad C_j,D_j \in \mathrm{GF}(p) \qquad u_j \in \mathrm{GF}(q)$$
$$C_j \neq 0$$
$$D_i \neq 0$$

Taking each of these equations to the q^{k+1-j} power gives

$$v^{q^{(k+1)}} = C_jv^{q^k} + D_jv^{q^{(k+1-j)}} + u_j \qquad \text{for all } j,\ 2 \leq j < k$$

If $C_i = C_j$ for some $i < j < k$, then we have

$$\begin{aligned} v^{q^{(k+1)}} &= C_jv^{q^k} + D_jv^{q^{(k+1-j)}} + u_j \\ &= C_iv^{q^k} + D_iv^{q^{(k+1-i)}} + u_i \end{aligned}$$

so

$$\begin{aligned} D_jv^{q^{k-(j-1)}} &= D_iv^{q^k} + u_i - u_j \\ &= D_i(B_kv + u_k) + u_i - u_j \end{aligned}$$

which contradicts the assumption that $A_{k-(j-i)} \neq 0$. So, for $2 \leq i < j < k$, $C_i \neq C_j$. There are only $p - 1$ nonzero elements of GF(p) from which to choose $C_2, C_3, \ldots, C_{k-1}$, so we conclude that $2 \leq k \leq p + 1$. Since the number of conjugates of each element is a multiple of k, so is N. Q.E.D.

It may be possible to strengthen this theorem slightly when $J = 2$ and $p \geq 11$, for not all values of $k \leq p + 1$ are necessarily possible. An exhaustive search reveals that, if $p = 2$, 3, or 5, then all values of k are permitted, but if $p = 7$, the value $k = 5$ is impossible. However, this particular exception does not exclude the possibility that $p = 7$, $J = 2$, $N = 5$, since we may have $N = 2 + 3$, with $v_1{}^q = -v_1 + u_1$ and $v_2{}^q = 2v_2 + u_2$.

For $J \geq 3$, no number theoretic constraints can be placed

upon N. As we saw in Theorem 11.39, there are many polynomials of each degree ≥ 2 over $GF(q)$ which have linearly independent conjugates, that is,

$$\sum_{i=0}^{d-1} g_i v^{q^i} \neq 0$$

for any sequence $g_i \in \mathrm{GF}(q)$. This implies that

$$\sum_{i=0}^{d-1} g_i v^{q^i} \notin \mathrm{GF}(q)$$

for any $g_i \in \mathrm{GF}(q)$, unless all g_i are equal.

Thus, if $J = 3$, we have $N = 4$ if $\tilde{f}(z - r)$ has an irreducible quartic factor with linearly independent roots. We may have $N = 6$ if $\tilde{f}(z - r)$ has an irreducible quartic factor with linearly independent roots v, v^q, v^{q^2}, v^{q^3} and an irreducible quadratic factor whose roots are $v + v^{q^2} + u$ and $v^q + v^{q^3} + u$, $u \in \mathrm{GF}(q)$. If $N \neq 4$, $N \neq 6$, we may write $N = 2B + 3C$; $B,C > 1$, and find N roots of $\tilde{f}(z - r)$ among B irreducible quadratics and C irreducible cubics.

It is relatively easy to determine the probability that a randomly chosen affine polynomial will have all its roots in the field. The number of affine polynomials of any given degree is just the number of ways of choosing the coefficients. The number which have all their roots in the field is given by the following theorem, which was known to Landsberg (1893).

Theorem 11.52 The number of k-dimensional subspaces of $\mathrm{GF}(q^m)$ over $\mathrm{GF}(q)$ is given by

$$\prod_{i=0}^{k-1} \frac{q^m - q^i}{q^k - q^i} = \prod_{i=0}^{k-1} \frac{q^{m-i} - 1}{q^{k-i} - 1} = \prod_{i=0}^{m-k-1} \frac{q^{m-i} - 1}{q^{m-k-i} - 1}$$

The number of k-dimensional affine subspaces of $\mathrm{GF}(q^m)$ over $\mathrm{GF}(q)$ is given by

$$q^{m-k} \prod_{i=0}^{k-1} \frac{q^{m-i} - 1}{q^{k-i} - 1}$$

Proof How many ways are there of choosing an ordered set of k elements in $\mathrm{GF}(q^m)$ which are linearly independent over $\mathrm{GF}(q)$? If i such vectors have already been chosen, they span a space of dimen-

sion i, which contains q^i vectors, leaving $q^m - q^i$ choices for the next. Thus, there are $(q^m - 1)(q^m - q)(q^m - q^2) \cdots (q^m - q^{k-1})$ ways of choosing an ordered set of k linearly independent elements in $\mathrm{GF}(q^m)$. Each subspace of dimension k has $(q^k - 1)(q^k - q) \cdots (q^k - q^{k-1})$ different bases, so the number of k-dimensional subspaces is the expression given in the theorem. Each k-dimensional subspace has q^{m-k} translates, each of which is an affine subspace, and each affine subspace is a translate of a subspace. Q.E.D.

★ 11.6 LOW-WEIGHT CODEWORDS IN CERTAIN CODES

There are various forms in which one can express codewords and error words. For some purposes, it is most convenient to express the codewords directly in terms of the digits which are transmitted over the channel: $\mathbf{C} = [C_0, C_1, \ldots, C_{n-1}]$. The error word may be expressed directly in terms of the added noise digits: $\mathbf{E} = [E_0, E_1, \ldots, E_{n-1}]$. However, for decoding purposes, it is more convenient to express the error word in terms of its error-locator polynomial

$$\sigma(z) = \prod_{\substack{X_i = \text{error} \\ \text{locations}}} (1 - X_i z) \tag{7.21}$$

For cyclic binary codes and for negacyclic nonbinary Lee metric codes, the error-locator polynomial alone determines the error word: for nonbinary Hamming metric codes, the error word depends on both the error-locator polynomial and the error-evaluator polynomial, $\omega(z) = \sigma(z) + \sum_i z Y_i X_i \prod_{j \neq i} (1 - X_j z)$. If all errors have the value $Y_i = 1$, then $\omega(z) = \sigma(z) - z\sigma'(z)$. The great advantage of expressing the error pattern in terms of the error-locator polynomial $\sigma(z)$, rather than the error word $E(x) = \sum_{i=0}^{n-1} E_i x^i$, is that the polynomial $\sigma(z)$ exhibits the weight of the error pattern explicitly; $\deg \sigma(z) = \text{weight } \mathbf{E}$.

Although we have not previously done so, we may also express any *codeword* in terms of its *codeword-locator polynomial*,

$$\sigma(z) = \prod_{\substack{X_i = \text{locations of nonzero} \\ \text{coefficients of } C(x)}} (1 - X_i z)$$

The advantage of expressing a codeword in this form is that low-weight codewords have low-degree codeword-locator polynomials. Thus, we can demonstrate the existence of low-weight codewords in certain codes by exhibiting a codeword-locator polynomial of low degree. To prove that the exhibited polynomial is indeed a codeword-locator polynomial, we must show that its reciprocal roots all lie among the locations of the

code and that the corresponding codeword satisfies all the code's parity-check equations. For cyclic codes, these parity checks are weighted power-sum symmetric functions of the reciprocal roots of $\sigma(z)$.

In the Lee metric, we may compute the power-sum symmetric functions of the reciprocal roots of the locator polynomial $\sigma(z)$ by Newton's identities.

$$S(z) = \frac{-z\sigma'(z)}{\sigma(z)} \tag{9.32}$$

This formula remains valid in the Hamming metric if all errors have value 1.

We shall now show that if certain coefficients of $\sigma(z)$ vanish, then certain coefficients of $S(z)$ must also vanish.

Theorem 11.61 Let $\tilde{\sigma}(z) = L(z) - u$, where $L(z) = \sum_{i=0}^{k} L_i z^{q^i}$.

Let $S(z) = \dfrac{-z\sigma'(z)}{\sigma(z)} = \sum_K \sum_J B_{K,J} z^K u^J$

where the constants $B_{K,J}$ depend only on the coefficients of $L(z)$. Let $w(K)$, the *weight of the integer* K, be defined by $w(K) = \sum_j K_j$, where K_j are the coefficients of the q-ary expansion of the integer $K = \Sigma K_j q^j$, $0 \leq K_j < q$. Then

$B_{K,J} = 0$

unless both

$$w(K) \geq k(q-1) + w(J) \geq k(q-1) \tag{11.611}$$

and

$$J + 1 \leq q^{-k}(K+1) \tag{11.612}$$

Proof

$$\tilde{\sigma}(z) = -u + \sum_{i=0}^{k} L_i z^{q^i} \qquad L_k = 1$$

$$\sigma(z) = -uz^{q^k} + \sum_{i=0}^{k} L_i z^{q^k - q^i}$$

$$\sigma'(z) = L_0(q^k - 1)z^{q^k-2} = -L_0 z^{q^k-2}$$

$$S(z) = \frac{-z\sigma'(z)}{\sigma(z)} = \frac{L_0 z^{q^k-1}}{1 + \sum_{i=0}^{k-1} L_i z^{q^k-q^i} - uz^{q^k}}$$

$$- L_0 z^{q^k-1} \sum_{N=0}^{\infty} \left(uz^{q^k} - \sum_{i=0}^{k-1} L_i z^{q^k-q^i}\right)^N$$

Expanding N in q-ary, $N = \sum_j N_j q^j$, we get

$$S(z) = L_0 \sum_{N=0}^{\infty} z^{q^k-1} \left(u z^{q^k} - \sum_{i=0}^{k-1} L_i z^{q^k-q^i} \right)^{\sum_j N_j q^j}$$

$$= L_0 \sum_{N=0}^{\infty} z^{q^k-1} \prod_j \prod_{m=1}^{N_j} \left(u^{q^j} z^{q^{k+j}} - \sum_{i=0}^{k-1} L_i^{q^j} z^{q^{k+j}-q^{i+j}} \right)$$

We now consider the expansion of the product

$$z^{q^k-1} \prod_j \prod_{m=1}^{N_j} \left(u^{q^j} z^{q^{k+j}-q^{j-\infty}} - \sum_{i=0}^{k-1} L_i^{q^j} z^{q^{k+j}-q^{i+j}} \right)$$

The power-series expansion of this product can contain terms in u^J only if $J = \sum_j J_j q^j$, where $0 \leq J_j \leq N_j$. In other words, every digit of the q-ary expansion of J must be dominated by the corresponding digit of the q-ary expansion of N. The expansion of the product can contain a term in $z^K u^J$ only if

$$K = q^k - 1 + \sum_j \sum_{m=1}^{N_j} (q^{k+j} - q^{j+i(j,m)})$$

where i is a function of j and m which assumes only the values $i(j,m) = 0, 1, 2, \ldots, k-1$, or $-\infty$, and $i(j,m) = -\infty$ for exactly J_j different values of m. Summing first over those m for which $i(j,m) = -\infty$, we have

$$K = q^k - 1 + \sum_j \left(J_j q^{k+j} + \sum_{m'=1}^{N_j - J_j} q^{k+j} - q^{j+i'(j,m')} \right)$$

$$= q^k - 1 + q^k J + \sum_j \sum_{m'} (q^{k+j} - q^{j+i'(j,m')})$$

where $i'(j,m') = 0, 1, 2, \ldots,$ or $k-1$. Therefore, $K \geq q^k - 1 + q^k J$. This proves Eq. (11.612). We may also write

$$K = q^k - 1 + q^k J + q^k(N - J) - \sum_j \sum_{m'} q^{j+i'(j,m')}$$

$$q^k - 1 + q^k N = K + \sum_j \sum_{m'} q^{j+i'(j,m')}$$

Since $w\left(\sum a_i\right) \leq \sum_i w(a_i)$ for any set of numbers a_i, we may write†

$$w(q^k - 1 + q^k N) \leq w(K) + \sum_j \sum_{m'} w(q^{j+i'(m,m')})$$

$$= w(K) + w(N - J) = w(K) + w(N) - w(J)$$

† The skeptic may find a proof of this assertion at Lemma 15.575.

Substituting $w(q^k - 1 + q^k N) = k(q-1) + w(N)$ gives

$$k(q-1) + w(N) \leq w(K) + w(N) - w(J)$$

or

$$w(K) \geq k(q-1) + w(J)$$

This proves Eq. (11.611). Q.E.D.

Theorem 11.61 enables us to exhibit low-weight codewords in many cyclic codes of block length $q^m - 1$. For example, in a cyclic binary code of block length 31, the coordinates may be labeled 1, α, α^2, α^3, α^4, α^5, . . . , α^{30}, where $\alpha^5 + \alpha^2 + 1 = 0$. Every element in GF($2^5$) may be written in the form $\sum_{i=0}^{4} Z_i \alpha^i$. The eight elements whose coordinates satisfy the equations $Z_0 + Z_1 + Z_2 = 0$, $Z_2 + Z_3 = 1$ form a three-dimensional affine subspace over GF(2). This affine subspace consists of the elements α^3, α^5, α^{16}, α^{19}, α^{21}, α^{22}, α^{27}, α^{28}, which are the roots of the affine polynomial $\tilde{\sigma}(z) = z^8 + \alpha^{14}z^4 + \alpha^6 z^2 + \alpha^4 z + \alpha^{17}$. According to Theorem 11.61, the Kth power-sum symmetric function of these eight elements vanishes unless $w(K) \geq 3$. In particular, $S_1 = S_3 = S_5 = 0$, so $\sigma(z) = 1 + \alpha^{14}z^4 + \alpha^6 z^6 + \alpha^4 z^7 + \alpha^{17} z^8$ is the locator polynomial of a codeword of weight 8 in the triple-error-correcting binary BCH code of block length 31. The corresponding codeword is

$$\mathbf{C} = [0001010000000000100101100001100]$$

Since $S_0 = 0$, this same codeword (extended by 0) appears in the triple-error-correcting extended binary BCH code of block length 32. This is one example of Theorem 11.62, which is an immediate consequence of Theorem 11.61.

Theorem 11.62 [Kasami, Lin, and Peterson (1966) and Camion (1966, 1968)] If α is a primitive element of GF(q^m), then the extended cyclic code of block length $N = q^m$ over GF(q^j) generated by $g(x)$ has codewords of weight q^k (in both the Hamming and Lee metrics) unless there exists an integer K for which $w(K) \geq k(q-1)$ and $g(\alpha^K) = 0$.

Corollary 11.63 The extended narrow-sense BCH code of designed distance q^k over GF(q) has codewords of weight q^k.

Proof of Corollary 11.63 The roots of the generator polynomial of the cyclic BCH code of designed distance q^k include only α^0, α^1, α^2, . . . , α^{q^k-2} and their conjugates. If $K < q^k - 1$, $w(K) <$

$k(q-1)$. If α^J is a conjugate of α^K, then $w(J) = w(K)$. Thus $g(\alpha^K) \neq 0$ unless $w(K) < k(q-1)$. Q.E.D.

Theorem 11.62 exhibited low-weight codewords whose nonzero coordinates were located in an affine subspace of GF(q^m) over GF(q). In some codes, all the low-weight codewords are of this type, as indicated by Theorem 11.64, which is a generalization of a theorem of Peterson.

Theorem 11.64 Let $g(x)$ be the generator polynomial of a code of length $n = q^m - 1$, and let $w(K)$ be the sum of the digits of the q-ary expansion of K. If

$$g(\alpha^K) = 0 \Rightarrow w(K) < k(q-1)$$

and

$$\left.\begin{array}{l} w(K) < k(q-1) \\ \text{and} \\ K < 2q^k \end{array}\right\} \Rightarrow g(\alpha^K) = 0$$

then

11.641. The locations of the nonzero coordinates of any codeword of weight q^k in the extended code form an affine subspace of dimension k over GF(q).

11.642. The values of the nonzero digits in any given codeword of weight q^k are identical.

Proof of 11.641 Let $\sigma(z)$ be the locator of a codeword of Hamming weight q^k in the extended code, and let $\omega(z)$ be the evaluator of this codeword. Then

$$\frac{\omega(z) - \sigma(z)}{\sigma(z)} = S(z) = \sum_{K=1}^{\infty} S_K z^K \tag{11.643}$$

or

$$\omega(z) - \sigma(z) = S(z)\sigma(z) \tag{11.644}$$

From Eq. (10.12),

$$\omega(z) - \sigma(z) = + \sum_i z X_i Y_i \prod_{j \neq i} (1 - X_j z)$$

The coefficient of z^{q^k} in this polynomial is seen to be

$$+\sum_i X_i Y_i \prod_{j \neq i} (-X_j) = \pm \left(\prod_j X_j\right)\left(\sum_i Y_i\right) = 0$$

because $\sum_i Y_i = 0$ for every codeword in the extended code. Therefore,

$$\deg [\omega(z) - \sigma(z)] \leq q^k - 1 \tag{11.645}$$

If $K < q^k - 1$, then $w(K) < k(q-1)$ so $g(\alpha^K) = 0$, and $S_K = 0$. Therefore, $S(z) \equiv 0 \bmod z^{q^k-1}$, so Eqs. (11.644) and (11.645) imply that

$$\omega(z) - \sigma(z) = z^{q^k-1} S_{q^k-1}$$

If $S_{q^k-1} = 0$, then we have the all-zero codeword. If $S_{q^k-1} \neq 0$, then we may define

$$T(z) = \frac{S(z)}{S_{q^k-1} z^{q^k-1}} \tag{11.646}$$

Equation (11.644) then becomes

$$T(z)\sigma(z) = 1$$

whence $\sigma_0 = 1$ and

$$\sigma_i = -\sum_{j=1}^{i} \sigma_{i-j} T_j \qquad \text{for } i = 1, 2, \ldots \tag{11.647}$$

If $K < 2q^k$, then $S_K = 0$ unless $w(K) \geq k(q-1)$, which happens iff $K = 2q^k - 1 - q^I$ for some I. From (11.646), this implies that $T_j = 0$ unless $j = q^k - q^I$. This enables us to replace Eq. (11.647) with

$$\sigma_i = -\sum_{I<k} \sigma_{i-(q^k-q^I)} T_{q^k-q^I} \qquad \text{for } i \leq q^k - 1 \tag{11.648}$$

If j is the greatest nonpower of q for which $\sigma_{q^k-j} \neq 0$, then according to (11.648), there must exist an $I \leq k-1$ and an $i \leq k$ such that $q^I - j = q^k - q^i$, or $j = q^I + q^i - q^k$, which is negative unless $j = 0$ or a power of q. We conclude that $\sigma_{q^k-j} = 0$ unless j is a power of q. Thus, $\tilde{\sigma}(z)$ is an affine polynomial, and according to Theorem 11.32, its roots form an affine subspace.

Proof of 11.642 Let $C^{(1)}$ be a codeword of weight q^k. According to Theorem 11.641, the nonzero digits of $C^{(1)}$ must occupy an affine subspace of GF(q^m). According to Theorem 11.61, the code con-

tains another codeword, $C^{(2)}$, whose only nonzero digits are ones which occur in these same q^k locations. If $C^{(1)}$ is not a scalar multiple of $C^{(2)}$, then the difference of $C^{(1)}$ and some scalar multiple of $C^{(2)}$ yields a codeword of Hamming weight less than q^k, which contradicts the BCH bound on the minimum weight of the code.

Q.E.D.

Theorem 11.61 is also useful for exhibiting low-weight codewords in certain negacyclic codes, as shown by Theorem 11.65.

Theorem 11.65 The negacyclic code of block length $(p^m - 1)/2$ whose generator's roots include only α, α^3, α^5, . . . , α^{2p-3} and their conjugates has codewords of Lee weight p.

Proof The only integer $K < 2p - 2$ for which $w(K) \geq p - 1$ is $K = p - 1$, which is even. Therefore we take $\sigma(z) = \prod_{i=0}^{p-1} [1 + (i\xi + \eta)z]$ where ξ and η are elements in GF(p^m) which are linearly independent over GF(p). To show that $\sigma(z)$ is legitimate, we must check that no negative of a reciprocal root is also a reciprocal root. If it were, then $i\xi + \eta = -(j\xi + \eta)$, $(i + j)\xi = -2\eta$, contradicting the linear independence of ξ and η over GF(p). Q.E.D.

As one example of Theorem 11.65, let $p = 5$, $m = 2$. Take α as in Appendix B. Let $\xi = \alpha$, $\eta = 1$, so the reciprocal roots of $\sigma(z)$ are α^0, α^8, $\alpha^{17} = -\alpha^5$, $\alpha^{21} = -\alpha^9$, and $\alpha^{22} = -\alpha^{10}$, so $\mathbf{C} = [1,0,0,0,0,-1,0,0,1,-1,-1,0]$ is a codeword in the negacyclic code whose generator's roots include α, α^3, α^5, α^7 and their conjugates.

Theorem 11.65 limits the extendibility of Theorem 9.34. The negacyclic code whose generator polynomial's roots include α, α^3, α^5, . . . , α^{p-2} can correct as many Lee errors as the code whose roots include α, α^3, α^5, . . . , α^{2p-3}, even though the redundancy of the latter code is almost twice the redundancy of the former.

Theorems 11.62 and 11.65 have exhibited low-weight codewords whose nonzero coordinates occur at points whose location numbers are an affine (or linear) subspace of GF(p^m). It is also possible to find codewords of low Hamming weight whose nonzero coordinates occur at points whose location numbers are several translates of an affine subspace of GF(p^m). In particular, we have Theorem 11.66.

Theorem 11.66 If the extended BCH code of designed distance d and block length q^m over GF(q) contains a codeword of weight d, and if the locations of the nonzero coordinates of this codeword lie in a subspace of dimension $\leq m - k$ over GF(q), then the extended BCH code of block length q^m and designed distance dq^k has codewords of weight dq^k.

Proof † Let u_i, $i = 1, 2, \ldots, d$ be the locations of the nonzero values of the codeword of weight d, and let Y_i be the values of the digits in the corresponding coordinates. Then $Y_i \in \mathrm{GF}(q)$, $u_i \in \mathrm{GF}(q^m)$, and $\sum_{i=1}^{d} Y_i u_i^J = 0$ for $J = 0, 1, 2, \ldots, d-2$. Let $L^*(z)$ be a q-linearized polynomial of degree q^{m-k} over GF(q^m), whose q^{m-k} roots all lie in GF(q^m) and include $u_1, u_2, \ldots, u_d$. Since the linear space spanned by $u_1, u_2, \ldots, u_d$ has dimension $\leq m - k$, a polynomial having the properties required of $L^*(z)$ exists. Let $L(z)$ be the dual polynomial of $L^*(z)$. According to Theorem 11.35, $L(z) - u_i = \prod_{j=1}^{q^k} (z - X_{i,j})$, where $X_{i,j} \in \mathrm{GF}(q^m)$. We now claim that the extended BCH code of designed distance dq^k has a codeword whose only dq^k nonzero values are located at $X_{i,j}$, $i = 1, 2, \ldots, d$; $j = 1, 2, \ldots, q^k$. We claim that the value of the nonzero digit at location $X_{i,j}$ may be taken as Y_i. We prove this by calculating

$$S_K = \sum_{i=1}^{d} \sum_{j=1}^{q^k} Y_i X_{i,j}^K$$

$$= \sum_{i=1}^{d} Y_i \sum_{j=1}^{q^k} X_{i,j}^K$$

Since $\sum_{j=1}^{q^k} X_{i,j}^K$ is an unweighted power-sum symmetric function of the $X_{i,j}$, we may apply Theorem 11.61 to write

$$S_K = \sum_{i=1}^{d} Y_i \sum_{J} B_{K,J} u_i^J$$

$$= \sum_{J} B_{K,J} \sum_{i=1}^{d} Y_i u_i^J$$

If $0 \leq J \leq d-2$, $\sum_{i=1}^{d} Y_i u_i^J = 0$. If $d - 1 \leq J$, then $B_{K,J} = 0$

† Another, independent proof of this theorem has been given by Peterson (1968).

unless $J + 1 \leq q^{-k}(K + 1)$, which implies that $K \geq dq^k - 1$. Therefore, $S_K = 0$ for $0 \leq K \leq dq^k - 2$. Q.E.D.

Theorems 11.61 to 11.66 show how the subspaces and affine subspaces of a finite field may be used to exhibit low-weight codewords in certain codes. In certain other codes, low-weight codewords may be found by other techniques, unrelated to affine subspaces. Theorem 11.67 is a simple example of such a result.

Theorem 11.67 [Kasami, Lin, and Peterson (1966)] If the BCH code of designed distance d and block length n_1 over GF(q) has codewords of weight d, and if n_1 divides n_2, then the BCH code of designed distance d and block length n_2 over GF(q) also has codewords of weight d.

Proof The location numbers of the first code are the n_1th roots of unity; the location numbers of the second code are the n_2th roots of unity. Since n_1 divides n_2, the locations of the first code are a subset of the locations of the second code. Thus, the low-weight codeword of the first code remains in the second code. Q.E.D.

Corollary 11.68 If $d|n$, the BCH code of designed distance d and block length n over GF(q) has codewords of weight d.

Proof The BCH code of designed distance d and block length d must have a codeword of weight d. Q.E.D.

Unfortunately, Theorems 11.61 to 11.67 allow us to find minimum-weight codewords in only a relatively small number of codes. The minimum distance of most codes, even most BCH codes, is not known. From Secs. 10.3 and 10.4, we know that the actual minimum distance must be at least as large as the designed distance, but in some cases it is even larger. In general, we do know that the actual distance of every primitive BCH code is within a factor of q of the designed distance, as guaranteed by Corollary 11.63. In practice, the error-correction capability of a BCH code is often limited by its designed distance rather than its actual distance because the decoding algorithms most commonly used are unable to correct any error pattern of weight greater than

half the designed distance. In Secs. 10.5, 10.6, and 11.1, we have presented refined algorithms by which the decoder may correct error patterns of slightly greater weight; but even these algorithms become infeasible if the decoder attempts to correct more than one or two additional errors beyond the number for which the code was designed.

From a practical viewpoint, the most important questions about any code are (1) "How good is it?" (2) "How easy is it to decode?" and (3) "What is its information rate?" · Questions (1) and (2) are engineering questions, because the words "good" and "easy" may be defined only in terms of channel noise statistics, which are never known exactly, or circuitry costs, which depend critically on the present state of electronics technology. If we measure "good" in terms of designed distance, then we may give the following approximate answers to questions (1) and (2) for narrow-sense BCH codes: (1) The designed distance may be arbitrarily specified. (2) The decoding is relatively easy, as shown in Chaps. 2, 7, and 10. These answers are both very encouraging. Unlike questions (1) and (2), question (3) is a precise mathematical question, to which we obtain a precise answer in Chap. 12. In Chap. 13, we show that this answer is disappointing in a certain sense, for there exist better long codes which have a much higher information rate, although these codes have no known feasible decoding algorithms.

PROBLEMS

11.1. Let α be a root of $x^5 + x^2 + 1$. Find all of the roots in GF(2^5) of each of the following polynomials:

(*a*) $z^3 + \alpha^{14}z^2 + \alpha^4 z + \alpha$

(*b*) $z^4 + \alpha^{18}z^3 + \alpha^{11}z^2 + \alpha^{24}z + 1$

(*c*) $z^5 + \alpha^{20}z^4 + \alpha^{18}z^3 + \alpha^3 z^2 + \alpha^2 z + \alpha^{17}$

11.2. Compute the gcd of $x^{2^5} - x$ and the linearized part of the least affine multiple of each of the polynomials of Prob. 11.1. Then compute the dual of each of these gcd's.

11.3. For $m = 1, 2, 3, \ldots, 20$, how many irreducible binary polynomials of degree m have linearly independent roots?

11.4. Find a primitive irreducible cubic over GF(3), let α be a root of that cubic, and express every power of α as a polynomial of degree <3 in α over GF(3). Show that the general cubic equation over GF(3^3) can be transformed into one of the standard affine forms

(*a*) $z^3 + z = u$

(*b*) $z^3 + \alpha z = u$

(*c*) $z^3 = u$

As in Example 11.15, design circuitry for solving each of these three standard affine cubics.

11.5. (*a*) [Landsberg (1893)] What fraction of the q^{mn} $m \times n$ matrices over GF(q) have row rank r? *Hint:* Review Theorem 11.52.

(*b*) For large n, what fraction of the square $n \times n$ binary matrices are nonsingular?

(*c*) Compare these results, the answers to Prob. 7.6, and the enumerations by rank which have been obtained for certain other classes of matrices [Carlitz and Hodges (1956*a*), (1956*b*); Berlekamp (1966)].

11.6. (*a*) [Ore (1933, 1934); Gleason and Marsh; Zierler (1958)] Let $f(z)$ be an irreducible polynomial over GF(q), with linearized associate $F(z)$. Prove that every irreducible factor of $F(z)/z$ has degree equal to the period of $f(z)$.

(*b*) Exhibit irreducible binary trinomials of degrees 3, 7, 15, 31, 63, and 127.

12
The Enumeration of Information Symbols in BCH Codes

12.1 CONVERTING THE PROBLEM TO AN ENUMERATION OF CERTAIN INTEGERS MOD n

Any cyclic code of block length n over GF(q) may be defined by its generator polynomial $g(x)$, which is some factor of $x^n - 1$ over GF(q), or by its check polynomial $h(x) = (x^n - 1)/g(x)$. The number of check digits in the code is given by the degree of $g(x)$; the number of information digits, by the degree of $h(x)$. We assume that n and q are relatively prime. It is most convenient to work in a particular extension field of GF(q), namely, GF(q^m), where m is the multiplicative order of q mod n. In this field, $x^n - 1$ factors into distinct linear factors

$$x^n - 1 = \prod_{i=1}^{n} (x - \alpha^i)$$

Here α is any primitive nth root of unity in GF(q^m); $\alpha^n = 1$. From the factorization $x^n - 1 = g(x)h(x)$, we see that every power of α is a

root of either $g(x)$ or of $h(x)$ but not both. Thus, a cyclic code partitions the powers of α into two sets: those powers which are roots of the generator polynomial and those powers which are roots of the check polynomial. If $g(x)$ and $h(x)$ were permitted to have coefficients in GF(q^m), then any partition of the powers of α would define a cyclic code. However, the coefficients of $g(x)$ and $h(x)$ must lie in the ground field GF(q). Consequently, if α^i is a root of $g(x)$, then so are the conjugates of α^i, namely, α^{iq}, α^{iq^2}, α^{iq^3}, ... Conversely, if all conjugates of roots of $g(x)$ are also roots of $g(x)$ and all conjugates of roots of $h(x)$ are also roots of $h(x)$, then all the coefficients of the polynomials $g(x)$ and $h(x)$ lie in GF(q).

The previous remarks hold for all cyclic codes.

A q-ary BCH code of block length n over GF(q) may be defined as the cyclic code whose generator's roots include only α, α^2, . . . , α^{d-1} and their conjugates. This code is capable of correcting any combination of fewer than $d/2$ errors (see Chap. 10); the minimum Hamming distance of this code is at least d. For this reason, d is called the *designed distance* of the code.

The first result on the number of information symbols in BCH codes is the following lemma.

Lemma 12.11 Let $I(q,n,d)$ be the number of information symbols in the q-ary BCH code of block length n and designed distance d.

Define $\lceil i \rceil$ by the conditions

$$i \equiv \lceil i \rceil \bmod n \qquad \text{and} \qquad 1 \leq \lceil i \rceil \leq n$$

Then $I(q,n,d)$ is the number of integers j, such that $1 \leq j \leq n$ and $\lceil jq^k \rceil \geq d$ for all k.

Proof α^j is a root of the generator polynomial of the BCH code iff there exists some $k(j)$ such that $\lceil jq^k \rceil < d$. Conversely, α^j is a root of the check polynomial iff $\lceil jq^k \rceil \geq d$ for all k. Q.E.D.

Lemma 12.11 enables one to compute the number of information symbols in any q-ary given BCH code without doing any calculation in GF(q) or its extensions. One need only enumerate certain types of residue classes mod n. In practice, this enumeration is still often laborious, particularly when n and d are large.

In order to obtain more tractable results for large n and d, we prefer to start from an alternate form of Lemma 12.11.

Lemma 12.12 Let $I(q,n,d)$ be the number of information symbols in the q-ary BCH code of block length n and designed distance d.

Define $\lfloor i \rfloor$ by the conditions

$$i \equiv \lfloor i \rfloor \bmod n \qquad \text{and} \qquad 0 \leq \lfloor i \rfloor \leq n-1$$

Then $I(q,n,d)$ is the number of integers i such that $0 \leq i \leq n-1$ and $\lfloor iq^k \rfloor < n+1-d$ for all k.

Proof $1 \leq j \leq n$ and $\lceil jq^k \rceil \geq d$ for all k iff $0 \leq n-j \leq n-1$ and $\lfloor (n-j)q^k \rfloor \leq n-d$ for all k. Let $i = n-j$. Q.E.D.

★ 12.2 CONVERTING THE PROBLEM FOR PRIMITIVE BCH CODES TO AN ENUMERATION OF CERTAIN q-ARY SEQUENCES

We shall now enumerate the information symbols in primitive BCH codes by reducing the problem to the enumeration of certain kinds of sequences over the alphabet consisting of the integers $0, 1, 2, \ldots, q-1$, as first suggested by Mann (1962). We begin by defining the appropriate manipulations with such sequences.

We shall always use *capital letters for sequences.* We let $(Q-1)$ denote the sequence consisting of the single letter $q-1$. Unless otherwise stated, we allow every sequence to be either finite or infinite.

Let $V = V_1V_2V_3 \cdots$ be any finite or infinite q-ary sequence, i.e., a sequence of numbers V_i, where V_i is an integer, $0 \leq V_i \leq q-1$. We let $\bar{V} = \bar{V}_1\bar{V}_2\bar{V}_3 \cdots$ denote the *complement* of V, defined by $\bar{V}_i = (q-1) - V_i$ for all i. If $W = W_1W_2 \cdots W_k$ is a finite q-ary sequence, then we may form the *cyclic shifts* of W: $W_2W_3 \cdots W_kW_1$, $W_3W_4 \cdots W_kW_1W_2$, ... If X is a finite q-ary sequence, $X = X_1X_2 \cdots X_j$, then we may form the *concatenation* $X * V = X_1X_2X_3 \cdots X_jV_1V_2V_3 \cdots$. This concatenation may be formed whenever V is a finite or infinite q-ary sequence. If V is a finite q-ary sequence, then $V * X$ is a cyclic shift of $X * V$.

The q-ary sequence Y is said to be a *prefix* of X iff $X = Y * Z$ for some Z; Y is said to be a *suffix* of X iff $X = Z * Y$ for some Z. A prefix must be a finite (or empty) sequence; a suffix may be empty, finite, or infinite. V is a *proper prefix* of X iff $X = V * Z$ and neither V nor Z is empty. Z is a *proper suffix* of X iff $X = V * Z$ and neither V nor Z is empty. If X is a finite q-ary sequence, $X = X_1X_2 \cdots X_k$, then we may form the *iterated concatenation* of X with itself, $\dot{X} = X_1X_2 \cdots X_kX_1X_2 \cdots X_k \cdots$. In particular, $(Q-1)$ denotes the infinite q-ary sequence all of whose letters are $q-1$.

We say $X < Y$ iff there exists a j such that $X_i = Y_i$ for $i = 1, 2, \ldots, j - 1$ but $X_j < Y_j$. If $X \not< Y$ and $Y \not< X$, then one is a prefix of the other.

This ordering is similar to the numerical ordering of q-ary fractions, but there are important differences. For example, $\frac{1}{4} = 0.01 < 0.0101 = \frac{5}{16}$, but the sequences 01 and 0101 are incomparable, because one is a prefix of the other. On the other hand, $0.0111111 \cdots = 0.1 = \frac{1}{2}$, yet $01111 \cdots < 1$. This type of example may be excluded by writing all fractions in their terminating form if they have one. We may then assert Theorem 12.21, whose proof is immediate.

Theorem 12.21 Let

$$u = \sum_i U_i q^{-i} \qquad v = \sum_i V_i q^{-i} \qquad U = U_1 U_2 U_3 \cdots$$

and $V = V_1 V_2 V_3 \cdots$, and suppose that $(Q - 1)$ is not a suffix of U or V. Then

$$U < V \Rightarrow u < v$$

$$u \leq v \Leftrightarrow \begin{cases} U < V \\ \text{or} \\ U \text{ is a prefix of } V \end{cases}$$

We say that X is an *immediate subordinate* of Y iff X is a finite sequence, $X = X_1 X_2 X_3 \cdots X_k$, and $X_1 = Y_1, X_2 = Y_2, \ldots, X_{k-1} = Y_{k-1}$ but $X_k < Y_k$. The sequence Y has Y_1 immediate subordinates of length 1, Y_2 immediate subordinates of length 2, Y_3 immediate subordinates of length 3, $\ldots$, Y_k immediate subordinates of length k. If the sequence Y has only a finite number of nonzeros, then we may define the *greatest immediate subordinate* of Y. If the last nonzero in the sequence $Y = Y_1 Y_2 \cdots$ is Y_k, then the greatest immediate subordinate of Y is $Y_1 Y_2 \cdots Y_{k-1}(Y_k - 1)$. If the sequence Y contains an infinite number of nonzeros, then Y has infinitely many immediate subordinates. All of them are less than Y itself, but none of them is the greatest immediate subordinate.

Similarly, we say that Y is an *immediate superior* of X if $Y = Y_1 Y_2 Y_3 \cdots Y_k$, where $Y_1 = X_1, Y_2 = X_2, \ldots, Y_{k-1} = X_{k-1}$ but $Y_k > X_k$. If $X = X_1 X_2 \cdots X_k$ and $X_k \neq (Q - 1)$, then the *least immediate superior* of X is $Y = Y_1 Y_2 \cdots Y_k$; $Y_i = X_i$ for $i = 1, 2, \ldots, k - 1$, and $Y_k = X_k + 1$. It should be evident that the least immediate superior is among the longest immediate superiors, and the greatest immediate subordinate is among the longest immediate subordinates.

Definition 12.22 If q is any integer, U is any infinite q-ary sequence, and m is any integer, we define $J(q,U,m)$ to be the number of q-ary m-tuples all of whose cyclic shifts are less than U.

Lemma 12.23 (Mann) If $n = q^m - 1$, $n + 1 - d = \sum_{i=1}^{m} U_i q^{m-i}$, $0 \leq U_i < q$, $U = U_1 U_2 \cdots U_m$, and Y is any q-ary sequence, then $I(q,n,d) = J(q,U * Y,m)$

Proof Lemma 12.23 reduces to Lemma 12.12 under the following correspondence. The q-ary m-tuple U corresponds to the integer $n + 1 - d$; another q-ary m-tuple $W = W_1 W_2 \cdots W_m$ corresponds to the integer $w = \sum_{i=1}^{m} W_i q^{m-i}$. The first cyclic shift of W is the sequence $W_2 W_3 \cdots W_m W_1$, which then corresponds to the integer $\sum_{i=2}^{m} W_i q^{m+1-i} + W_1 = qw - (q^m - 1)W_1$. Modulo $n = q^m - 1$, the integer corresponding to the first cyclic shift of W is seen to be congruent to qw. Therefore, the successive cyclic shifts of an m-digit q-ary sequence W correspond to the integers $\lfloor w \rfloor$, $\lfloor wq \rfloor$, $\lfloor wq^2 \rfloor$, . . . , $\lfloor wq^{m-1} \rfloor$. These integers are all $< n + 1 - d$ iff all cyclic shifts of W are $< U$, which is true iff all cyclic shifts of W are $< U * Y$, for any Y. Q.E.D.

The choice $Y = \dot{U}$ has an interesting interpretation

$$\sum_{i=1}^{\infty} \dot{U}_i q^{-i} - \sum_{k=0}^{\infty} \sum_{i=1}^{m} U_i q^{-(i+mk)} = \frac{\sum_{i=1}^{m} U_i q^{-i}}{1 - q^{-m}}$$

$$= \frac{\sum_{i=1}^{m} U_i q^{m-i}}{q^m - 1} = 1 - \frac{d-1}{n}$$

Thus, the sequence $\dot{U}$ is the q-ary expansion of $1 - (d-1)/n$. For this reason, we may investigate the behavior of $I(q,n,d)$ for large n and d with a fixed fractional error-correction capability, $(d-1)/2n$, by studying $J(q,\dot{U},m)$ as a function of m for fixed q and $\dot{U}$.

We shall temporarily ignore the periodicity of the $\dot{U}$ sequence and consider the function $J(q,V,m)$ for an arbitrary q-ary sequence V. We assume only that the sequence V has no terminal zeros.

From the definition of the immediate subordinates of V, it is clear that *if an m-digit q-ary sequence W is less than V, then some immediate subordinate of V is a prefix of W.* For if W is less than V, then there exists a k such that $W_i = V_i$ for $i = 1, 2, \ldots, k-1$, but $W_k < V_k$, and the sequence $W_1W_2 \cdots W_k$ is a prefix of W and an immediate subordinate of V.

Now suppose that some m-digit sequence W and all its cyclic shifts are less than V. Since W itself is less than V, some prefix of W must be an immediate subordinate of V. Are all possible immediate subordinates of V possible prefixes of W? In general, they are not, for some immediate subordinates may have suffixes which are greater than V. If $X * Y$ is an immediate subordinate of V and Y is greater than V, then $X * Y$ cannot be a prefix of W. For, if $W = X * Y * Z$, then one of the cyclic shifts of W is $Y * Z * X$, which is greater than V.

For example, consider the ternary sequence $V = 20212$. Its immediate subordinates are 0, 1, 200, 201, 2020, 20210, and 20211. The immediate subordinate 20210 has the suffix 210, which is greater than V. Therefore, if 20210 is the prefix of W, then the second cyclic left shift of W is greater than V. Similarly, V's immediate subordinate 20211 has the suffix 211, which is also greater than V.

For some sequences V, this diffculty arises only when V is less than one of its own proper suffixes. If V is not less than any of its own suffixes, then we have Theorem 12.3.

★ 12.3 SEQUENCE ENUMERATION THEOREMS

Let V be a q-ary sequence which is not less than any of its own proper suffixes. Then:

12.31. No immediate subordinate of V is a proper prefix of any other immediate subordinate of V.

12.32. Every **suffix** of every immediate subordinate of V is a concatenation of other immediate subordinates of V.

12.33. If W and all its cyclic shifts are less than V, then W can be uniquely decomposed into a concatenation of immediate subordinates of V, including a (possibly empty) end-around immediate subordinate. Specifically,

$$W = W^{(1)} * W^{(2)} * \cdots * W^{(i)} * W^{(i+1)} * W^{(i+2)} * \cdots * W^{(j-1)} * W^{(j)}$$

$W^{(1)}, W^{(2)}, \ldots, W^{(j-1)}$ are immediate subordinates of V; $W^{(j)} * W^{(1)} * W^{(2)} * \cdots * W^{(i)}$ is the end-around immediate subordinate. The end-around immediate subordinate has a prefix $W^{(j)}$, which is a suffix of W, and a suffix $W^{(1)} *$

$W^{(2)} * \cdots * W^{(i)}$, which is a prefix of W, as well as a concatenation of the shorter immediate subordinates $W^{(1)}$, $W^{(2)}$, $\ldots$, $W^{(i)}$.

12.34. Every concatenation of immediate subordinates of V, including a (possibly empty) end-around immediate subordinate, yields a sequence which has the property that all its cyclic shifts are less than V. No such sequence of length m can exceed the maximum m-digit concatenation of immediate subordinates of V. If Y is the maximum m-digit concatenation of immediate subordinates of V, and $Y \leq U \leq V$, then $J(q,V,m) = J(q,U,m)$.

12.35. $J(q,V,m) = mV_m + \sum_{k=1}^{m-1} V_k J(q,V, m-k)$, where V_j is taken as 0 if j exceeds the length of the sequence V.

12.36. Let

$$n = q^m - 1 \qquad d = \Sigma D_i q^{m-i} \qquad 0 \leq D_i < q$$
$$D = D_1 D_2 \cdots D_m$$

If

$$\bar{V} * (Q-1) < D \leq \text{least } m\text{-digit concatenation of immediate superiors of } \bar{V}$$

then

$$I(q,n,d) = J(q,V,m)$$

Proofs

12.31. This property of immediate subordinates does not even depend on the suffix condition on V. From the definition of immediate subordinates, each immediate subordinate must disagree with V only in the immediate subordinate's last digit, and hence no immediate subordinate can be a prefix of any other.

12.32. Let us first prove the following weaker assertion.

12.321. Every proper suffix of every immediate subordinate of V has a prefix which is a shorter immediate subordinate of V.

Let S be an immediate subordinate of V, and let $S^{(2)}$ be a suffix of S. We may write $S = S^{(1)} * S^{(2)}$. Since S differs from V only in its last digit, $S^{(1)}$ is a prefix of V, and $V = S^{(1)} * V^{(2)}$. Since $S < V$, $S^{(2)} < V^{(2)}$. Since V is not less than any of its own proper suffixes, $V^{(2)} < V$. Therefore, $S^{(2)} < V$. Therefore some prefix of $S^{(2)}$ is an immediate subordinate of V.

12.322. If every suffix of an immediate subordinate has a prefix which is an immediate subordinate, then every suffix of every immediate subordinate is a concatenation of immediate subordinates.

For, suppose F is a suffix on an immediate subordinate; then $F = B^{(1)} * F^{(2)}$, where $B^{(1)}$ is an immediate subordinate. Since $F^{(2)}$ is a suffix of F, it is also a suffix of an immediate subordinate, and $F^{(2)} = B^{(2)} * F^{(3)}$, where $B^{(2)}$ is an immediate subordinate $\cdots F = B^{(1)} * B^{(2)} * B^{(3)} * \cdots$.

12.33. Since $W < V$, it contains a prefix $W^{(1)}$ which is an immediate subordinate of V. After shifting this prefix around to the end, we may similarly identify $W^{(2)}, W^{(3)}, \ldots, W^{(j-1)}$, each of which is an immediate subordinate of V. The sequence $W^{(j)} * W^{(1)} * W^{(2)} * \cdots * W^{(j-1)}$ is a cyclic shift of W, and so it must have a prefix P which is an immediate subordinate of V. P is not a prefix of $W^{(j)}$, so $W^{(j)}$ must be a prefix of P. Suppose that $W^{(j)} * W^{(1)} * \cdots * W^{(i)}$ is a prefix of P but that $W^{(j)} * W^{(1)} * \cdots * W^{(i+1)}$ is not a prefix of P. (This defines i.) Then $P = W^{(j)} * W^{(1)} * \cdots * W^{(i)} * S$, where the (possibly empty) sequence S is a proper prefix of $W^{(i+1)}$. Since S is a suffix of P, which is an immediate subordinate of V, S is itself a concatenation of immediate subordinates of V. But no immediate subordinate of V is a proper prefix of any other immediate subordinate of V, so S must be empty.

12.34. This is the converse of (12.33). Suppose we are given the sequence $W = S^{(j)} * W^{(1)} * W^{(2)} * \cdots * W^{(j-1)} * P^{(j)}$, where $W^{(1)}, W^{(2)}, \ldots, W^{(j-1)}$ and $W^{(j)} = P^{(j)} * S^{(j)}$ are immediate subordinates of V. We must show that all cyclic shifts of W are less than V. Any cyclic shift is of the form

$$C = S^{(k)} * W^{(k+1)} * W^{(k+2)} * \cdots * W^{(j)} * W^{(1)} * W^{(2)} * \cdots * W^{(k-1)} * P^{(k)}$$

where $W^{(k)} = P^{(k)} * S^{(k)}$. If $S^{(k)}$ is empty, C has the prefix $W^{(k+1)}$, which is an immediate subordinate of V. If $S^{(k)}$ is not empty, by (12.32) it has a prefix which is an immediate subordinate of V, which is a prefix of C. In either case, C has a prefix which is an immediate subordinate of V. Therefore, $C < V$.

12.35. V has V_m immediate subordinates of length m, each of which has m distinct cyclic shifts. Thus, W may be chosen as a single end-around or terminal immediate subordinate of V in mV_m ways.

If W is a concatenation of several immediate subordinates of V, $W = W^{(1)} * W^{(2)} * \cdots * W^{(j-1)} * W^{(j)}$, where $W^{(1)}, W^{(2)}, \ldots, W^{(j-1)}$ are immediate subordinates of V and $W^{(j)}$ is a (possibly empty) proper prefix of the immediate subordinate $W^{(j)} * W^{(1)} * \cdots * W^{(i)}$, then the length of W is the length of $W^{(j-1)}$ plus the length of $W^{(1)} * W^{(2)} * \cdots * W^{(j-2)} * W^{(j)}$. For each k, there are V_k choices of $W^{(j-1)}$ of length k and $J(q, V, m - k)$ choices for $W^{(1)} * W^{(2)} * \cdots * W^{(j-2)} * W^{(j)}$.

12.36. *Least special case:* Suppose D is the least m-digit q-ary sequence greater than $\bar{V} * (Q \dot{-} 1)$. Letting

$$d = \Sigma D_i q^{m-i} \qquad v = \Sigma V_i q^{m-i} \qquad \bar{v} = \Sigma \bar{V}_i q^{m-i}$$

it is evident that $d + v = n + 1$ and $v = n + 1 - d$. According to Lemma 12.23, $I(q,m,d) = J(q,V,m)$.

12.36. *Greatest special case:* Let D be the least m-digit concatenation of immediate superiors of $\bar{V}$. Complementing, $\bar{D}$ is the greatest m-digit concatenation of immediate subordinates of V. In the notation of Theorem 12.34, $\bar{D} = Y$. Letting $\bar{d} = \Sigma \bar{D}_i q^{m-i}$, $\bar{d} = n - d$. Letting $n + 1 - d = \Sigma U_i q^{m-i}$, $U > Y$, because $n + 1 - d > n - d$. The claim follows from Theorem 12.34 and Lemma 12.23.

12.36. *The general case* follows because $J(q,U,m)$ is a monotonic function of U. Q.E.D.

Example 12.37 Let V be the binary sequence 1101. We compute Table 12.1

Here $J(q,V,m)$ is computed by Theorem 12.35. The designed distances are computed according to Theorem 12.36 using $\bar{V} = 0010$, with immediate superiors 1, 01, and 0011.

$$\bar{V} * (Q \dot{-} 1) = 001011111111 \cdots$$

Evidently, the binary BCH code of block length $2^{12} - 1$ and designed distance 768 is identical to the binary BCH code of block length $2^{12} - 1$ and designed distance 769, or 770, or . . . , or 819. This code has 854 information symbols. This code is distinct from the binary BCH code of block length $2^{12} - 1$ and designed distance 820. This is true in general, because the least m-digit concatenation of immediate superiors of $\bar{V}$ is necessarily minimum among all its own cyclic shifts. I call this *greatest designed distance* the *Bose distance.*

Table 12.1

		Bose distance†		*Designed distance*	
m	$J(q,V,m)$	*Binary*	*Decimal*	*Binary*	*Decimal*
1	1	1	1	1	1
2	3	01	1	01	1
3	4	011	3	010	2
4	11	0011	3	0011‡	3‡
5	16	00111	7	00110	6
6	30	001101	13	001100	12
7	50	0011011	27	0011000	24
8	91	00110011	51	00110000	48
9	157	001100111	103	001100000	96
10	278	0011001101	205	0011000000	192
11	485	00110011011	411	00110000000	384
12	854	001100110011	819	001100000000	768

† Defined at bottom of p. 281.
‡ This code is identical to the binary BCH code of block length 15 and designed distance 2.

It happens that the binary BCH code of block length $2^{12} - 1$ and designed distance 768 is also distinct from the binary BCH code of block length $2^{12} - 1$ and designed distance 767, because the 12-digit binary expansion of 767 is minimum among all its cyclic shifts. This, however, need not be true in general. For example, the binary BCH code of block length $2^4 - 1$ and designed distance 3 is not distinct from the binary BCH code of block length $2^4 - 1$ and designed distance 2, because the 4-digit binary expansion of $2 = 0010$ is not minimum among its cyclic shifts; the minimum is 0001.

In general, we would like to determine the number of information digits in the q-ary BCH code of block length $n = q^m - 1$ and designed distance $d = \Sigma D_i q^{m-i}$. Theorems 12.23 and 12.35 give us a solution to this problem if we can find a sequence V which is greater than all its own suffixes and has the property that

$$\bar{V} * (Q \dot{-} 1) < D \leq \text{least } m\text{-digit concatenation of immediate superiors of } \bar{V}$$

Complementing this condition gives

$$V > \bar{D} \geq \text{greatest } m\text{-digit concatenation of immediate subordinates of } V$$

or

$$V > \bar{D} * (Q \dot{-} 1) > (\text{greatest } m\text{-digit concatenation of immediate subordinates of } V) * \dot{0} > \dot{X}$$

where X is the greatest immediate subordinate of V. We may assume that V has no terminal zeros and that the length of V does not exceed the length of $\bar{D}$. Since X and V have the same length, X is a prefix of $\bar{D}$.

Since V is the least immediate superior of X, the problem of finding V is reduced to the problem of finding X, which is a prefix of $\bar{D}$. The solution is given by Theorem 12.38.

Theorem 12.38 Let X be the shortest nonempty prefix of $\bar{D}$ such that

$$\bar{D} = X * F \qquad F * (Q \dot{-} 1) \geq \bar{D} * (Q \dot{-} 1)$$

and let V be the least immediate superior of X. Then:

12.381. $\bar{V} * (Q \dot{-} 1) < D \leq$ least m-digit concatenation of immediate superiors of $\bar{V}$

12.382. V exceeds all its own proper suffixes.

Proof of 12.381 Since X is a shortest nonempty prefix of $\bar{D}$ and V is an immediate superior of X, V is an immediate superior of $\bar{D}$. So $V > \bar{D}$ and $V > \bar{D} * (Q - 1)$.

Complementing gives $\bar{V} * (Q \dot{-} 1) < D * \dot{0}$, so $V * (Q \dot{-} 1) < D$.

Let $X^{(k)} = \overbrace{X * X * \cdots * X}^{k}$. Then $F * (Q \dot{-} 1) \geq \bar{D} * (Q \dot{-} 1)$ is equivalent to $X^{(0)} * F * (Q \dot{-} 1) \geq X^{(1)} * F * (Q \dot{-} 1)$. Therefore, $X * X^{(0)} * F * (Q \dot{-} 1) \geq X * X^{(1)} * F * (Q \dot{-} 1)$, or $X^{(1)} * F * (Q \dot{-} 1) \geq X^{(2)} * F * (Q \dot{-} 1)$. By induction, $X^{(k)} * F * (Q \dot{-} 1) \geq X^{(k+1)} * F * (Q \dot{-} 1)$, and $\bar{D} * (Q \dot{-} 1) \geq X^{(k)} * F * (Q \dot{-} 1)$ for all k. Since this is true for arbitrarily large k, $\bar{D} * (Q \dot{-} 1) \geq \dot{X}$. Complementing, $D * \dot{0} \leq \dot{X} \leq$ any infinite concatenation of immediate superiors of $\bar{V}$. Therefore, $D \leq$ any m-digit concatenation of immediate superiors of $\bar{V}$.

Proof of 12.382 Let $X = Y * Z * L$, where Y and Z are arbitrary (possibly empty) and L is the final digit of X. We have

$$V = Y * Z * (L + 1)$$
$$\bar{D} = Y * Z * L * F$$
$$F * (Q \dot{-} 1) \geq Y * Z * L * F * (Q \dot{-} 1) = X * F * (Q \dot{-} 1)$$
$$= Y * Z * L * F * (Q \dot{-} 1) > Z * L * F * (Q \dot{-} 1)$$

else Y would be a shorter prefix than X, which satisfied the same conditions.

No proper suffix of V can equal V, for the suffix must be shorter.

If some proper suffix of V, say $Z * (L + 1)$ (Z possibly empty), exceeds V, then

$$Z * (L + 1) > Y * Z * (L + 1) > Y * Z * L = X$$

If $Z * L > X$, then $Z * L * F * (Q \dot{-} 1) > X * F * (Q \dot{-} 1)$, a contradiction. If $Z * L$ is a prefix of X, then $X = Z * L * G$, and from

$$X * F * (Q \dot{-} 1) > Z * L * F * (Q \dot{-} 1)$$

we have

$$Z * L * G * F * (Q \dot{-} 1) > Z * L * F * (Q \dot{-} 1)$$
$$G * F * (Q \dot{-} 1) > F * (Q \dot{-} 1) \geq X * F * (Q \dot{-} 1)$$

Now $Z * L$ is a shorter prefix than X, a contradiction. Therefore, $Z * (L + 1) < Y * Z * (L + 1)$; that is, V exceeds all its own proper suffixes. Q.E.D.

★ 12.4 EXAMPLES

Example 12.41 Let $q = 9, n = 728 = 9^3 - 1, d = 217$. Then $D = 261$, $\bar{D} = 627$, $\bar{D} * \dot{8} = 627888 \cdots$, $X = 62$, $V = 63$, $\bar{V} = 25$.

		Bose distance	
m	J†	*q-ary*	*Decimal*
1	6	3	3
2	42	26	24
3	270	263	219

† In this and subsequent tables we use the single letter J as an abbreviation for $J(q,V,m)$.

Example 12.42 Let $q = 2$, $n = 511$, $d = 185$. Then $D = 010111001$, $\bar{D} = 101000110$, $\bar{D} * \dot{1} = 101000110111111 \cdots$, $X = 101000$, $V = 101001$, $\bar{V} = 010110$. Immediate superiors are 010111, 011, 1.

m	J	*Bose distance*	*Smaller designed distance*
1	1	1	1
2	1	11	11
3	4	011	011
4	5	0111	0111
5	6	01111	01111
6	16	010111	010111
7	22	0101111	0101110
8	29	01011111	01011100
9	49	010111011	010111000

Example 12.43 Let $q = 2$, $n = 511 = 2^9 - 1$, $d = 187$. Then $D = 010111011$, $\bar{D} = 101000100 = X$, $V = 101000101$, $\bar{V} = 010111010$.

m	J	*Bose distance*
1	1	1
2	1	11
3	4	011
4	5	0111
5	6	01111
6	10	011011
7	22	0101111
8	29	01011111
9	49	010111011

The answer agrees with Example 12.42, although the recurrence is different. This illustrates the general nonuniqueness of V. Theorem 12.38 specifies one satisfactory method of finding V, but, as seen from this example, this V need not be unique. This simplest recurrence rule generally arises from the shortest possible V, which corresponds to the greatest V, or the least D. This can generally be found by first reducing D as much as possible.

Example 12.44 Let $n = 2^{11} - 1$, $d = 411$. Then $D = 00110011011$. We could take $\bar{D} = 11001100100 = X$ and proceed. However, we instead consider $d = 410$, $D = 00110011010$. Since D has a cyclic shift smaller than itself, the code is unchanged. But $\bar{D} = 11001100101$, $X = 1100110010$ does not look much easier, so we continue. Each prime marks the starting point of a smaller cyclic shift.

D
0011001101'0
00110011'001
00110011'000
0011'0010111
0011'0010110
.....
0011'0010000
0011'0001111
.....
0011'0000000
00101111111

Since 00101111111 has no cyclic shift less than itself, this designed distance is the Bose distance of a *different* BCH code. We must instead use $D = 00110000000$, $\bar{D} = 11001111111$, $X = 1100$, $V = 1101$. The recurrence is given in Example 12.37. $I(2, 2^{11} - 1, 411) = 485$. This same V is obtained if we started with $D = 00110011000$ or any D in the region

$$00110000000 \leq D \leq 00110011000$$

Example 12.45 Let $q = 2$, $n = 2^{15} - 1$, $D = 001010010100111$, $\bar{D} * \mathrm{i} = 1101011010110001111 \cdots$, $X = 110101101011000$, $V = 110101101011001$, $\bar{V} = 001010010100110$.

m	J	*Bose distance*
1	1	1
2	3	01
3	4	011
4	11	0011
5	16	00111
6	36	001011
7	64	0010101
8	115	00101011
9	211	001010011
10	378	0010100111
11	694	00101001011
12	1256	001010010101
13	2276	0010100101011
14	4112	00101001010111
15	7474	001010010100111

m	J	*Bose distance*
1	1	1
2	3	01
3	4	011
4	11	0011
5	21	00101
6	36	001011
7	64	0010101
8	115	00101011
9	211	001010011
10	383	0010100101
11	694	00101001011
12	1256	001010010101
13	2276	0010100101011
14	4126	00101001010011
15	7479	001010010100101

Although the brute-force method just used gives the right answer, a more devious approach proves easier. Instead of $D = 001010010100111$, let us consider $D = 001010010100101$, $X = 11010$, $V = 11011$, $\tilde{V} = 00100$. This yields a different set of codes (shown above) with a much simpler recurrence. The code with $D = 001010010100111$ has five fewer information digits than the code with $D = 001010010100101$, corresponding to the five distinct cyclic shifts of 001010010100101.

★ 12.5 THE ENUMERATION OF INFORMATION SYMBOLS IN NONPRIMITIVE BCH CODES

Although it is always possible to calculate the number of information symbols in any BCH code from Lemma 12.12, the calculation is often simplified by converting the problem to an enumeration of q-ary sequences. For primitive BCH codes, this is accomplished by Theorem 12.23, which then enables us to apply the theorems of Sec. 12.3. The comparable result for nonprimitive BCH codes is Theorem 12.51.

Theorem 12.51 Given q, n, and d, let

m = multiplicative order of q mod n

$j = \dfrac{q^m - 1}{n}$

D be the q-ary sequence whose digits are taken from the q-ary expansion of $jd = \sum_i D_i q^i$

V be defined from D by Theorem 12.38

Then $I(q,n,d)$ is the number of q-ary m-tuples W which satisfy both of the following conditions:

1. W is a concatenation of immediate subordinates of V, including a possibly empty end-around immediate subordinate.
2. The integer $w = \sum_i W_i q^i$ is divisible by j.

Proof According to Theorem 12.12, $I(q,n,d)$ is the number of integers i, such that $0 \leq i \leq n-1$ and $\lfloor iq^k \rfloor < n+1-d$ for all k. If we multiply this condition by j, we find that $I(q,n,d)$ is the number of integers w which are multiples of j and have the property that $0 \leq w \leq j(n-1)$ and $\lfloor wq^k \rfloor < j(n+1-d)$ for all k, where $\lfloor wq^k \rfloor$ is now defined by $\lfloor wq^k \rfloor \equiv wq^k \bmod n$ and $0 \leq \lfloor wq^k \rfloor \leq jn-1$. Since there is no multiple of j between $j(n-1)$ and jn or between $j(n+1-d)$ and $j(n-d)$, we may replace the latter conditions on w by $0 \leq w \leq jn-1 = q^m - 2$ and $\lfloor wq^k \rfloor < jn+1-jd = q^m - jd$. Since the m-digit q-ary expansion of $\lfloor wq^k \rfloor$ is a cyclic shift of the m-digit q-ary expansion of w, Theorem 12.51 follows from Theorems 12.38, 12.33, and 12.34. Q.E.D.

In order to enumerate the sequences of q-ary m-tuples which satisfy the conclusion of Theorem 12.51, it is helpful to introduce another definition.

Definition 12.52 Let $K^{(r)}(q,V,m,j)$ be the number of q-ary m-tuples which are concatenations of immediate subordinates of V and which represent integers that are congruent to $r \bmod j$.

let $V_k^{(r)}(j)$ be the number of immediate subordinates of V of length k which are congruent to $r \bmod j$.

For fixed q, j, and V, Theorem 12.53 gives us a recurrence by which we can compute $K^{(r)}(q,V,m,j)$ for all values of r for successive values of m. We adopt the convention that all arithmetic operations on the superscripts of K are to be performed mod j.

Theorem 12.53

$$K^{(r)}(q,V,m,j) = \begin{cases} 0 & \text{if } m < 0 \\ 0 & \text{if } m = 0 \text{ and } r \neq 0 \\ 1 & \text{if } m = 0 \text{ and } r = 0 \\ \sum_l \sum_k K^{(l)}(q, V, m-k, j) V_k^{(r-q^k l)}(j) & \text{otherwise} \end{cases}$$

Proof The empty sequence, of length 0, has remainder 0 when divided by j. For $m > 0$, there are $V_k^{(x)}(j)$ ways in which we may choose the last of the concatenated immediate subordinates in such a way that it has length k and remainder x. The preceding $(m - k)$-tuple must itself be a concatenation of immediate subordinates of V. If this concatenation of length $m - k$ has remainder l, then the concatenation of length m has remainder $q^k l + x$, which equals r iff we set $x = r - q^k l$. Q.E.D.

Theorem 12.54 With the hypotheses of Theorems 12.51 and 12.53 and Definition 12.52,

$$I(q,n,d) = \sum_l \sum_k kK^{(l)}(q, V, m - k, j)V_k^{(r-q^k l)}(j)$$

Proof Since $j|q^m - 1$, divisibility of a q-ary m-tuple by j does not depend on the phase of its end-around (or terminal) immediate subordinate. Theorem 12.54 then follows from Theorem 12.53 and the fact that there are k possible phases of any particular end-around (or terminal) immediate subordinate of length k. Q.E.D.

For small j, Theorems 12.51, 12.53, and 12.54 and Definition 12.52 provide a simple method for computing $I(q,(q^m - 1)/j,d)$. For large j, the calculation suggested by these theorems is more involved.

Example 12.55 How many information digits are there in the binary BCH code of length 1365 and designed distance 260?

Solution $n = 1365$, $m = 12$, $j = 3$, $3 \times 260 = 001100001100$ in binary, so $D = 001100001100$, $\bar{D} = 110011110011$, $X = 1100$, $V = 1101$. The immediate subordinates of V are 0, 10, and 1100, so $V_1^{(0)}(3) = V_2^{(2)}(3) = V_4^{(0)}(3) = 1$; all other $V_k^{(r)}(3) = 0$. We may then plot $V_k^{(r-2^k l)}(3)$:

$r = 0$			
4	1	0	0
3	0	0	0
2	0	1	0
1	1	0	0
k / l	0	1	2

$r = 1$			
4	0	1	0
3	0	0	0
2	0	0	1
1	0	0	1
k / l	0	1	2

$r = 2$			
4	0	0	1
3	0	0	0
2	1	0	0
1	0	1	0
k / l	0	1	2

We may then compute $K^{(r)}(2,1101,m,3)$:

m \ r	0	1	2
0	1	0	0
1	1	0	0
2	1	0	1
3	1	1	1
4	2	2	2
5	4	3	3
6	7	5	6
7	11	10	10
8	18	18	19
9	32	32	32
10	57	56	56
11	100	98	98

According to Theorem 12.54, $I(2,1365,260) = 100 + 2 \times 56 + 4 \times 18 = 284$.

To find the Bose distance of this code, we find the least m-digit concatenation of immediate superiors of $\bar{V}$, divide by j, and round up. In the present example, $\bar{V} = 0010$. The least 12-digit concatenation of its immediate superiors is 001100110011, which is 3×273, so the Bose distance is 273.

12.6 ASYMPTOTIC RESULTS

Let us define the enumerator

$$J(q,U;z) = \sum_{m=1}^{\infty} J(q,U,m)z^m$$

Given a sequence V which exceeds all its own proper suffixes, we may also define

$$V(z) = \sum_k V_k z^k$$

so that

$$zV'(z) = \sum_k kV_k z^k$$

The recurrence

$$J(q,V,m) = mV_m + \sum_{k=1}^{m-1} V_k J(q,\ V,\ m-k) \tag{12.35}$$

becomes

$$J(q,V;z) = zV'(z) + V(z)J(q,V;z)$$

whose solution is

$$J(q,V;z) = \frac{zV'(z)}{1 - V(z)} \tag{12.01}$$

Let $\rho_1, \rho_2, \ldots$ be the (not necessarily distinct) complex reciprocal roots of $1 - V(z)$. Then

$$1 - V(z) = \prod_i (1 - \rho_i z)$$

$$-V'(z) = -\sum_i \rho_i \prod_{j \neq i} (1 - \rho_j z)$$

$$\frac{zV'(z)}{1 - V(z)} = \sum_i \frac{\rho_i z}{1 - \rho_i z} = \sum_i \sum_{m=1}^{\infty} (\rho_i z)^m = \sum_{m=1}^{\infty} \sum_i \rho_i{}^m z^m$$

Therefore

$$J(q,V;z) = \sum_{m=1}^{\infty} \sum_i \rho_i{}^m z^m$$

so we have proved Theorem 12.62.

Theorem 12.62

$$J(q,V,m) = \sum_i \rho_i{}^m$$

where ρ_i are the complex numbers defined by the equation

$$1 - V(z) = \prod_i (1 - \rho_i z)$$

Although this gives an explicit expression for $J(q,V,m)$, the expression depends upon the complex numbers ρ_i. For finite values of m, it is usually easier to compute $J(q,V,m)$ directly from the recurrence relation of Theorem 12.35, since these calculations involve only integers. For asymptotic results, however, the above equation is very useful.

Definition 12.63 Let

$$\rho = \max_i |\rho_i|$$

and let

$$s = \log_q \rho$$

Since all coefficients of the polynomial $V(z)$ must be nonnegative integers not exceeding $q - 1$, it is easily seen that the ρ_i with the maximum absolute value is real and positive and $1 \leq \rho < q$. Clearly,

$$J(q,V,m) \sim \rho^m$$

for large m, in the sense that

$$\lim_{m \to \infty} \rho^{-m} J(q,V,m) = 1$$

Similarly,

$$\log_q J(q,V,m) \sim m \log_q \rho = ms$$

If

$$u = \sum_{i=1}^{\infty} U_i q^{-i}$$

and

$$\dot{X} \leq \bar{U} \leq V * \dot{0}$$

where V exceeds all its own suffixes and X is the maximum subordinate of V, then

$$I(q, q^m - 1, uq^m) \sim q^{ms}$$

In other words, if we fix the fraction $d/n = u$ and let n and d grow large, then

$$I \sim n^s$$

or, more precisely,

$$s(u) = \lim_{m \to \infty} \frac{\log_q I(q, q^m - 1, uq^m)}{m} \tag{12.64}$$

For given q, the function $s(u)$ is a rather complicated animal. To compute it, one must first write u in q-ary. If $\bar{U}$ exceeds all its proper suffixes, set $V = \bar{U}$; otherwise write $\bar{U} = X * F$, where X is the shortest prefix such that $\bar{U} \leq F$. V is then taken to be the least immediate superior of X. Then, s is defined as the logarithm (base q) of the maximum reciprocal root of $1 - V(z)$. For $q = 2$, $s(u)$ is plotted in Fig. 12.1.

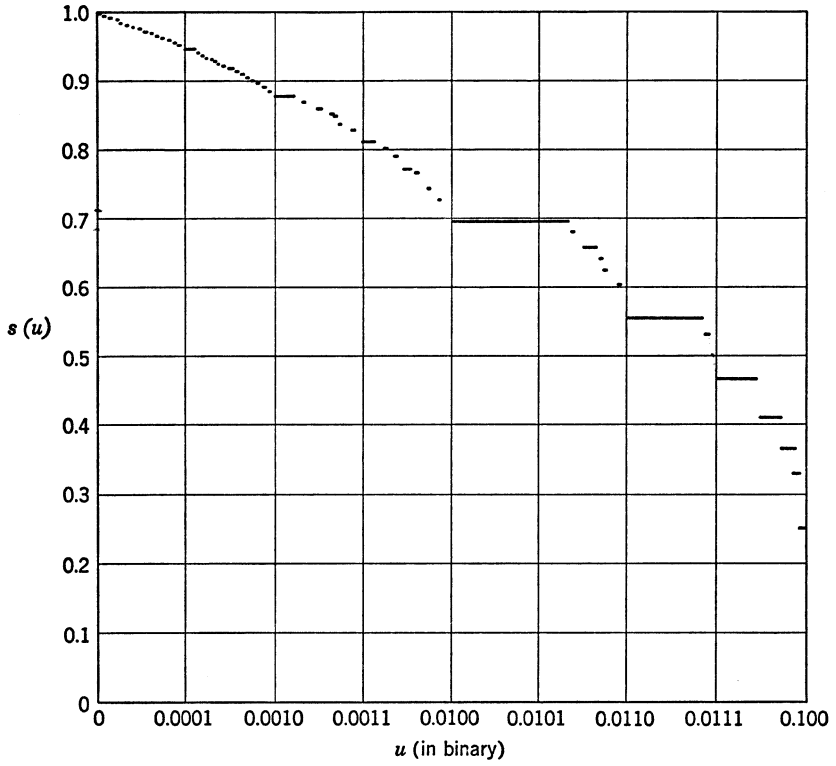


Fig. 12.1 Graph of $s(u)$ versus u.

It can easily be shown that s is a continuous, monotonic nonincreasing function of u. It can also be shown that the derivative of s with respect to u either is 0 or is undefined. There are two kinds of points at which the derivative $s'(u)$ is undefined. First, there are the endpoints of the intervals on which $s(u)$ is constant. u is a lower endpoint of such an interval iff $\bar{U}$ is a finite sequence which exceeds all its own proper suffixes; u is an upper endpoint of such an interval iff $\bar{U}$ is a periodic sequence equal to some of its suffixes but not less than any others. At these endpoints, $s(u)$ is undifferentiable because it has only a right derivative or a left derivative, but not both. There is only a countable number of points of this type.

The more interesting points are those at which $s(u)$ has neither a right derivative nor a left derivative. This happens iff $\bar{U}$ is an infinite sequence which exceeds all its own proper suffixes and $\dot{0}$ is not a suffix of $\bar{U}$.

The set of points u such that $s(u)$ is not differentiable is uncount-

able, but it has measure 0. Pitcher (1966) has also shown that this set has Hausdorff dimension 1. This appears to be entirely due to the large density of these points in the vicinity of $u = 0$. In general, I conjecture that the Hausdorff dimension of the set of points in the interval $a \leq u \leq b$ [where $0 \leq a \leq b \leq 1$, $s(a) \neq s(b)$] is $s(a)$. In some sense, almost all the nondifferentiable points in any interval seem to lie very near the leftmost cluster point of the interval.

When Mann first obtained results identical to those here in the special cases $u = q^{-k}$, he also showed that ρ is the only reciprocal root of $1 - V(z)$ with magnitude greater than 1. Thus, not only is

$$I \sim \rho^m$$

but in fact, for sufficiently large m,

$$I = \langle \rho^m \rangle$$

where $\langle \cdot \rangle$ denotes the nearest integer to $\cdot$. Unfortunately, this strengthened result is not true in general. For some values of u, $1 - V(z)$ has only one reciprocal root with magnitude greater than 1, but for other values of u, $1 - V(z)$ has many reciprocal roots with magnitude greater than 1. Little is known about the behavior of the smaller complex reciprocal roots of $1 - V(z)$ as a function of u, although Logan (1967) has obtained a few unpublished preliminary results in this area.

★ 12.7 ACTUAL DISTANCE

As one increases the designed distance, the number of information symbols in the resulting code must either remain constant or decrease. This proves Theorem 12.71.

Theorem 12.71 $I(q,n,d)$ is the maximum number of information symbols in any of the q-ary BCH codes with designed distance $\geq d$.

We must be careful to distinguish between I and $\check{I}$, defined as follows.

Definition 12.72 $\check{I}(q,n,d)$ is the maximum number of information symbols in any of the q-ary BCH codes with actual distance $\geq d$.

It is obvious that $\check{I}(q,n,d) \geq I(q,n,d)$.

For example, there are three binary BCH codes of block length 23, having 23, 12, and 1 information symbols. The reader should have no

trouble verifying that

$$I(2,23,d) = \begin{cases} 23 & \text{if } d = 1 \\ 12 & \text{if } 2 \leq d \leq 5 \\ 1 & \text{if } 6 \leq d \leq 23 \end{cases}$$

In Sec. 15.2, we shall show that

$$\check{I}(2,23,d) = \begin{cases} 23 & \text{if } d = 1 \\ 12 & \text{if } 2 \leq d \leq 7 \\ 1 & \text{if } q \leq d \leq 23 \end{cases}$$

Thus, $\check{I}(2,23,7) > I(2,23,7)$.

The known cases in which $\check{I}(q,n,d) > I(q,n,d)$ are relatively sparse. Kasami, Lin, and Peterson (1966) have conjectured that $\check{I}(2, 2^m - 1, d) = I(2, 2^m - 1, d)$ for all m and d, and they have proved it in certain special cases. Although this conjecture remains open, we can obtain certain results about the asymptotic behavior of $I(2, 2^m - 1, u2^m)$ from the known classes of special cases in which $\check{I}(2, 2^m - 1, d) = I(2, 2^m - 1, d)$. We would like to define

$$\check{s}(u) \stackrel{?}{=} \lim_{m \to \infty} \frac{\log_2 \check{I}(2, 2^m - 1, u2^m)}{m}$$

Unfortunately, however, we have no assurance that the limit exists. In order to discuss the asymptotic behavior of the best BCH codes, we use definition (12.73)

$$\check{s}(u) = \lim_{m \to \infty} \sup \frac{\log_2 \check{I}(2, 2^m - 1, u2^m)}{m} \tag{12.73}$$

Clearly $\check{s}(u) \geq s(u)$. Like $s(u)$, $\check{s}(u)$ must be a monotonic nonincreasing function of u, because if $d' > d$, the codewords of the q-ary BCH code of distance d' are a subset of the codewords of the q-ary BCH code of distance d.

We can prove that $\check{s}(u) = s(u)$ for certain values of u, as indicated by Theorem 12.74

Theorem 12.74 If $u \geq 2^{-k}$, then $\check{s}(u) \leq s(2^{-k})$.

Proof According to Corollary 11.63, if $u \geq 2^{-k}$, then

$$\check{I}(2, 2^{m-1}, u2^m) \leq \check{I}(2, 2^m - 1, 2^{m-k} - 1) = I(2, 2^m - 1, 2^{m-k} - 1)$$

for any $m \geq k$

Hence,

$$\frac{\log \check{I}(2, 2^m - 1, u2^m)}{m} \leq \frac{\log I(2, 2^m - 1, 2^{m-k} - 1)}{m}$$

So

$$\check{s}(u) \leq \lim_{m \to \infty} \frac{\log I(2, 2^m - 1, 2^{m-k} - 1)}{m} = s(2^{-k})$$

because $s(u)$ is continuous. Q.E.D.

This shows that $\check{s}(u) = s(u)$ if $u = \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \ldots$. By appropriate application of Theorems 11.66 and 13.12, it may also be shown that $\check{s}(u) = s(u)$ for various other values of u. I conjecture that $\check{s}(u) = s(u)$ for all u. This is a weakened form of Peterson's conjecture that $\check{I}(2, 2^m - 1, d) = I(2, 2^m - 1, d)$ for all m and d.

According to Eq. (12.73), the primitive BCH code of long block length n and designed distance $d = un$ has approximately $n^{s(u)}$ information symbols. The rate of this code is approximately $n^{[s(u)-1]}$. Since $s(u) < 1$ for all $u > 0$, the rate of long primitive BCH codes with fixed $u = d/n$ approaches 0 as n approaches ∞. Similarly, the rate of long primitive BCH codes with actual distance un approaches 0 as $n^{[\check{s}(u)-1]}$. Even though the function $\check{s}(u)$ is not known exactly, Theorem 12.74 assures us that $\check{s}(u) < 1$ for all $u > 0$.

In Chap. 13 we shall show that the "optimum" long block codes are asymptotically much better than long primitive BCH codes. The best long block codes with actual distance un have rate $R(u)$, which is independent of n. Although the exact asymptotic relationship between R and d/n is not known, these quantities are constrained by various bounds which we shall present in Chap. 13.

PROBLEMS

12.1. Show that if $j = 1$, then the results of Sec. 12.5 agree with the results of Sec. 12.3.

12.2. Using any convenient method, calculate the number of information symbols in the binary BCH codes with each of the following lengths and designed distances. Also calculate the Bose distance of each of these codes.

(*a*) $n = 511$, $d = 65$
(*b*) $n = 511$, $d = 88$
(*c*) $n = 1365$, $d = 200$
(*d*) $n = 341$, $d = 50$
(*e*) $n = 13107$, $d = 2180$
(*f*) $n = 819$, $d = 137$
(*g*) $n = 79$, $d = 11$

12.3. [Mann, (1962)] Let $V(z)$ and ρ_i be defined as in Sec. 12.6. Show that if $u = q^{-k}$, then only one ρ_i has magnitude greater than 1.

12.4. For $q = 3, 4, 5$, draw a rough sketch of

$$\lim_{m \to \infty} \frac{\log_q I(q, q^m - 1, uq^m)}{m}$$

as functions of u.

12.5. For $q = 2$, draw a graph of the tightest upper bound on $\check{s}(u)$ which you can obtain. Use Theorems 12.74, 11.66, and any other relevant results which you can prove.

13
The Information Rate of the Optimum Codes

13.1 THE HAMMING-RAO HIGH-RATE VOLUME BOUND

Both the Hamming metric and the Lee metric define a distance between any pair of points in the space of n-digit words over the alphabet of q letters. Using either metric, we may define the sphere of radius t and center $\mathbf{C}$ to be the set of all points whose distance from $\mathbf{C}$ is $\leq t$. The *surface area* of this sphere, denoted by $A_t^{(n)}$, is the number of points whose distance from $\mathbf{C}$ is exactly t; the *volume* of this sphere, denoted by $V_t^{(n)}$, is the number of points whose distance from $\mathbf{C}$ is $\leq t$. Clearly $V_t^{(n)} = \sum_{i=0}^{t} A_i^{(n)}$. Let $A^{(n)}(z) = \sum_i A_i^{(n)} z^i$. Since the distance is additive over the n coordinates, the generating function $A^{(n)}(z)$ is multiplicative over these coordinates, and $A^{(n)}(z) = [A^{(1)}(z)]^n$. Thus, for the Hamming metric, $A^{(n)}(z) = [1 + (q-1)z]^n$; for the Lee metric,

$$A^{(n)}(z) = \begin{cases} (1 + 2z + 2z^2 + 2z^3 + \cdots + 2z^{(q-1)/2})^n & \text{if } q \text{ is odd} \\ (1 + 2z + 2z^2 + \cdots + 2z^{(q-2)/2} + z^{q/2})^n & \text{if } q \text{ is even} \end{cases}$$

In either case, we can determine $A_t^{(n)}$ by expanding $A^{(n)}(z)$ as a power series in z. For the Hamming metric, $A^{(n)}(z) = [1 + (q-1)z]^n = \sum_{t=0}^{n} \binom{n}{t} (q-1)^t z^t$, so $A_t^{(n)} = \binom{n}{t}(q-1)^t$. The analogous formulas for the Lee metric are much more complicated.

If the minimum distance between any pair of codewords is $\geq 2t+1$, then no point can be within distance t of more than one code point. If the code has rate R, then it has q^{Rn} codewords. The spheres of radius t surrounding these q^{Rn} codewords must all be disjoint. Therefore, $q^{Rn} V_t^{(n)} \leq q^n$, or

$$V_t^{(n)} \leq q^{n(1-R)} \tag{13.11}$$

Inequality (13.11) is called the *volume bound* or the *sphere packing bound.* This inequality appeared in a paper on experimental designs by Rao (1947). It was first applied to error-correcting codes by Hamming (1950).

One application of the volume bound is Theorem 13.12, which proves that very high-rate binary BCH codes are essentially as good as any codes of the same rate and block length.

Theorem 13.12 If

$$\frac{(t+1)!}{(q-1)^{t+1}} + \frac{(t+1)(t+2)}{2} \leq q^m$$

then every code of block length $q^m - 1$ and rate $R \geq 1 - mt/n$ has codewords of Hamming weight $\leq 2t+2$.

Proof We claim that $V_{t+1}^{(q^m-1)} > A_{t+1}^{(q^m-1)} \geq q^{mt} \geq q^{n(1-R)}$. The only nontrivial claim here is the assertion that $A_{t+1}^{(q^m-1)} \geq q^{mt}$. Since $A_{t+1}^{(q^m-1)} = \binom{q^m-1}{t+1}(q-1)^{t+1}$,

$$\frac{A_{t+1}^{(q^m-1)}(t+1)!}{(q-1)^{t+1}q^{mt}} = \frac{(q^m-1)(q^m-2)(q^m-3)\cdots(q^m-t-1)}{q^{mt}}$$

$$= q^m \prod_{i=1}^{t+1} (1 - iq^{-m}) \geq q^m \left(1 - \sum_{i=1}^{t+1} iq^{-m}\right)$$

$$= q^m - \frac{(t+1)(t+2)}{2} \geq \frac{(t+1)!}{(q-1)^{t+1}}$$

by hypothesis of Theorem 13.12. Therefore, $A_{t+1}^{(q^m-1)} \geq q^{mt}$, so

$V_{t+1}^{(q^m-1)} > q^{n(1-R)}$ and Theorem 13.12 follows from Inequality (13.11). Q.E.D.

For some purposes, it is useful to rewrite Eq. (13.11) as

$$R \leq 1 - \frac{\log_q V_t^{(n)}}{n} \tag{13.13}$$

For large n, fixed $P = t/n$, this becomes

$$R \leq C(P) \tag{13.14}$$

where $C(P)$ is defined by

$$C(P) = 1 - \lim_{n \to \infty} \frac{\log_q V_{Pn}^{(n)}}{n} \tag{13.15}$$

In order to evaluate this limit, one must obtain an asymptotically tight estimate of the quantity $V_{Pn}^{(n)}$. One form of this estimate is given by Lemma 13.16, which is a modification of a more general bound due to Chernoff (1952) and Feller (1943).

Lemma 13.16 If $A^{(n)}(z) = [1 + (q-1)z]^n = \sum_{i=0}^{n} A_i^{(n)} z^i$, $V_t^{(n)} = \sum_{i=0}^{t} A_i^{(n)}$, and $t < (q-1)n/q$, then

$$\min_{0 \leq z \leq 1} \frac{z^{-t} A^{(n)}(z)}{n+1} \leq A_t^{(n)} \leq V_t^{(n)} \leq \min_{0 \leq z \leq 1} z^{-t} A^{(n)}(z)$$

Sketch of proof For any z, $0 \leq z \leq 1$, $V_t^{(n)} = \sum_{i=0}^{t} A_i^{(n)} \leq \sum_{i=0}^{t} A_i^{(n)} z^{i-t} = z^{-t} \sum_{i=0}^{t} A_i^{(n)} z^i \leq z^{-t} A^{(n)}(z)$.

We claim that there are some values of z for which

$$A_t^{(n)} z^t \geq A_i^{(n)} z^i \qquad \text{for all } i \neq t \tag{13.161}$$

Equation (13.161) implies that $A_t^{(n)} z^t \geq A_{t-1}^{(n)} z^{t-1}$ and $A_t^{(n)} z^t \geq A_{t+1}^{(n)} z^{t+1}$, which implies that

$$\frac{A_{t-1}^{(n)}}{A_t^{(n)}} \leq z \leq \frac{A_t^{(n)}}{A_{t+1}^{(n)}} \tag{13.162}$$

It can easily be verified that this interval is nonempty because $A_i^{(n)}/A_{i+1}^{(n)}$ is a monotonic nondecreasing function of i for $0 \leq i <$

$(q-1)n/q$. For this reason, (13.162) implies (13.161). Thus, there exists a particular value of z, say $\check{z}$, for which

$$A^{(n)}(\check{z}) = \sum_{i=0}^{n} A_i^{(n)}\check{z}^i \leq \sum_{i=0}^{n} A_t^{(n)}\check{z}^t = (n+1)A_t^{(n)}\check{z}^t$$

so

$$\min_{0\leq z\leq 1} \frac{z^{-t}A^{(n)}(z)}{n+1} \leq \frac{\check{z}^{-t}A^{(n)}(\check{z})}{n+1} \leq A_t^{(n)} \qquad \text{Q.E.D.}$$

The value of z which minimizes $z^{-Pn}[1+(q-1)z]^n$ and $z^{-P}[1+(q-1)z]$ is most easily found by setting the derivative to zero, obtaining

$$-Pz^{-P-1}[1+(q-1)z] + z^{-P}[q-1] = 0$$

from which

$$z = \frac{P}{(1-P)(q-1)} \qquad \text{and} \qquad A^{(n)}(z) = \left(\frac{1}{1-P}\right)^n$$

Substituting these expressions into Lemma 13.16, we obtain

$$\frac{[(q-1)^P P^{-P}(1-P)^{-(1-P)}]^n}{n+1} \leq V_{Pn}^{(n)} \leq [(q-1)^P P^{-P}(1-P)^{-(1-P)}]^n$$

Equation (13.15) then becomes

$$C(P) = 1 - P\log_q(q-1) + P\log_q P + (1-P)\log_q(1-P) \tag{13.17}$$

When $q = 2$, the reader who has studied information theory will recognize $C(P)$ as the capacity of the binary symmetric channel shown in Fig. 13.1.

Using analogous methods, one may also derive formulas for the asymptotic form of the volume bound for the Lee metric. However, the calculations are tedious, and the final results are rather complicated expressions which we prefer to omit.

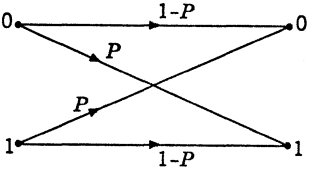


Fig. 13.1 The binary symmetric channel.

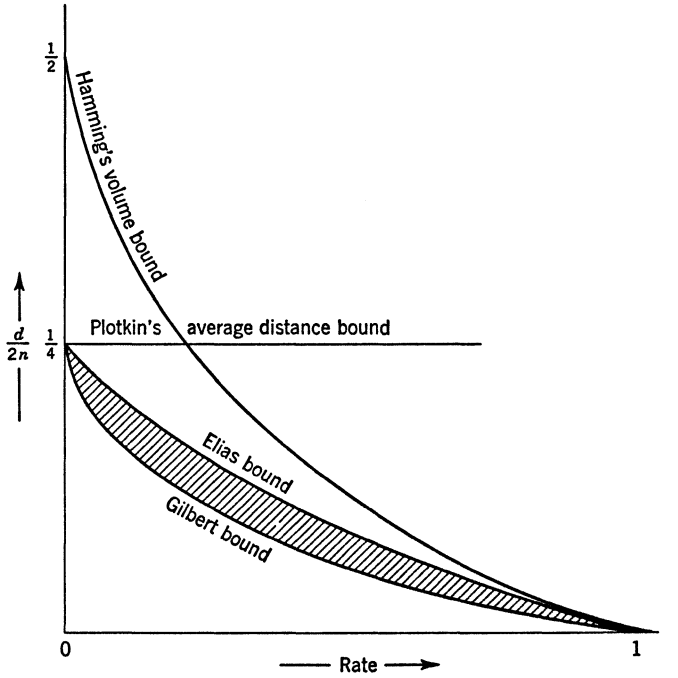


Fig. 13.2 Asymptotic bounds on error-correction capability of the binary symmetric channel without feedback.

In some applications, it is desirable to express Eq. (13.14) as a bound on t/n rather than a bound on R. This is accomplished by defining $\delta(R)$ implicitly as the smallest solution of the equation

$$R = C(\delta(R)) \tag{13.18}$$

Equation (13.14) then becomes

$$\frac{t}{n} \leq \delta(R) \qquad \text{or} \qquad \frac{d}{n} \leq 2\delta(R) \tag{13.19}$$

where $d = 2t + 1$ is the minimum distance between codewords. Equation (13.19) is the asymptotic form of the volume bound. For the binary symmetric channel of Fig. 13.1, this function is the upper curve plotted in Fig. 13.2.

★ 13.2 PERFECT CODES

Whenever one obtains any bound, it is natural to examine the cases, if any, when the bound is tight. Codes which satisfy the volume bound (13.11) with equality are called *perfect codes* or *close-packed codes.*

The *binary repetition code* of odd block length n is able to correct all patterns of up to $(n-1)/2$ errors. With $t = (n-1)/2$, $R = 1/n$, we have $V_t^{(n)} = \sum_{i=0}^{(n-1)/2} \binom{n}{i} = 2^{n-1} = 2^{(1-R)n}$. Thus, the binary repetition codes of odd block lengths are perfect. As we shall see in Sec. 13.6, all other perfect codes must have high rates or short block lengths or both. Theorem 13.21 exhibits an infinite class of perfect codes with high rate and low error-correction capability ($t = 1$); Theorem 13.25 exhibits an infinite class of perfect Lee metric codes of short block length ($n = 2$) and arbitrary error-correcting capability.

Many of the codes promised by Theorem 13.21 consist of the set of polynomials which are multiples of a generator polynomial $g(x)$ modulo $x^n - \xi$, where $\xi \in \mathrm{GF}(q)$. If $C_0, C_1, C, \ldots, C_{n-1}$ is a codeword of such a code, then its constacyclic shift, $\xi C_{n-1}, C_0, C_1, \ldots, C_{n-2}$, is also a codeword. For that reason, I call such codes *constacyclic* codes. The class of constacyclic codes includes as proper subclasses the class of cyclic codes and the class of negacyclic codes.

Theorem 13.21 A perfect systematic† single-Hamming-error-correcting code of block length n over the alphabet of q letters can exist only if

$$n = \frac{q^r - 1}{q - 1} \tag{13.22}$$

for some integer r.

If q is a prime power and n is given by (13.22), then there exists a perfect single-Hamming-error-correcting code‡ which is linear and constacyclic. There also exists a cyclic perfect single-Hamming-error-correcting code iff (13.22) holds and $\gcd(n, q-1) = 1$.

If $q > 2$, a perfect systematic single-Lee-error-correcting code of block length n over the alphabet of q letters can exist only if

$$n = \frac{q^r - 1}{2} \tag{13.23}$$

for some integer r. If q is any odd integer and n is given by (13.23), then there exists a perfect single-Lee-error-correcting code which is linear over the integers mod q. If q is prime, this code is negacyclic.

† A *systematic code* is one which has distinguishable information digits and check digits. Over a finite field, all linear codes and some nonlinear codes [e.g., Eq. (13.83)] are systematic. If q is prime power, van Lint has shown the length of a nonsystematic perfect code must also satisfy Eq. (13.22) or (13.23).

‡ These codes were constructed by Golay (1949, 1954, 1958), Zaremba (1952), and Cocke (1959).

Proof In the Hamming metric, $V_1^{(n)} = 1 + (q-1)n$, so the number of codewords in a perfect single-error-correcting code must be given by

$$q^{Rn} = \frac{q^n}{1 + (q-1)n}$$

Since Rn must be an integer, $1 + (q-1)n$ must be a power of q, and n must be given by (13.22).

In the Lee metric, $V_1^{(n)} = 1 + 2n$, so the number of codewords in a perfect single-error-correcting code must be given by

$$q^{Rn} = \frac{q^n}{1 + 2n}$$

Since Rn must be an integer, $1 + 2n$ must be a power of q, and n must be given by (13.23). If q is an odd prime, the existence of a perfect single-Lee-error-correcting negacyclic code of block length n is guaranteed by Theorem 9.34.

If q is an odd nonprime, then the $q^r - 1$ nonzero r-tuples over the integers mod q may be partitioned into $(q^r - 1)/2$ sets, each set consisting of an r-tuple and its negative. If the columns of an $\mathfrak{IC}$ matrix include one r-tuple from each of these $(q^r - 1)/2$ sets, then this $\mathfrak{IC}$ matrix is the parity-check matrix of a perfect linear single-Lee-error-correcting code of length $n = (q^r - 1)/2$ and rate $(n - r)/n$ over the integers mod q.

We next show that if q is a prime power, then a linear perfect single-Hamming-error-correcting code of block length n over GF(q) exists for every n of the form given by Eq. (13.22). Let α be a primitive element in GF(q^r), and let the code be defined by the parity-check matrix

$$\mathfrak{IC} = [1, \alpha, \alpha^2, \alpha^3, \ldots, \alpha^{n-1}] \tag{13.24}$$

Any nonzero syndrome may be represented as $\alpha^j = \alpha^{an+b}$, where $0 \leq a < q - 1$ and $0 \leq b < n$. Since $\alpha^n \in \mathrm{GF}(q)$, this syndrome is seen to correspond to a unique single error whose value is α^{an} and whose location is α^b.

If $q = 2$, this perfect single-error-correcting code is a primitive binary BCH code which is cyclic. However, if $q \neq 2$, this code is *not* cyclic, for $\alpha^n \neq 1$. The code defined by the parity-check matrix of Eq. (13.24) is in fact constacyclic; its codewords are polynomials in an ideal of polynomials modulo $x^n - \alpha^n$.

If $\gcd(n, q-1) = 1$, then there exists a cyclic perfect single-Hamming-error-correcting code of block length n over GF(q). The

roots of the generator polynomial of this code may be taken as $\beta, \beta^q, \beta^{q^2}, \ldots, \beta^{q^{r-1}}$, where $\beta \in \mathrm{GF}(q^r)$ and β is a primitive nth root of unity. In this form, the code is a BCH code with designed Hamming distance 2. However, we may set $\beta = \alpha^j$, where α is another primitive nth root of unity and j is a solution of the equation $(q-1)j \equiv 1 \bmod n$. We then have $\beta = \alpha^j, \beta^q = \alpha^{j+1}, \ldots$. The roots of the generator polynomial now include two consecutive powers of α, so the code is recognized as an alternate BCH code with Hamming distance 3, according to Sec. 10.3.

If $\gcd(n, q-1) = j \neq 1$, then no perfect single-Hamming-error-correcting code of block length n over $\mathrm{GF}(q)$ can be cyclic. The generator polynomial of such a code would have to be some irreducible factor of $x^n - 1$. But $x^n - 1 = \prod_{i=0}^{j} (x^{n/j} - \alpha^{(q-1)i/j})$, where α is a primitive element of $\mathrm{GF}(q)$. Therefore, any irreducible factor of $x^n - 1$ has a multiple of degree n/j which has Hamming weight 2. Q.E.D.

There also exist other classes of perfect single-error-correcting codes. Vasil'ev (1962) and Schönheim (1968) have constructed many systematic *nonlinear* perfect single-error-correcting codes. Lloyd (1957) and Roos (1965) have shown that if any other nonlinear perfect codes exist, then they must also have a certain algebraic structure. It is also conceivable that there may exist perfect single-Hamming-error-correcting codes over some alphabet whose order is not a prime power. No such codes are presently known. Golomb and Posner (1964) have proved that no single-Hamming-error-correcting code exists with $q = 6$ and $n = 7$. They attribute part of their proof to R. E. Block and M. Hall, Jr.

Theorem 13.25 demonstrates the existence of certain short perfect multiple-Lee-error-correcting codes over alphabets of composite order.

Theorem 13.25 For any given t, there exists a perfect t-Lee-error-correcting code of block length $n = 2$ over the alphabet of integers mod $q = 2t^2 + 2t + 1$. This code may be taken as the negacyclic code generated by $g(x) = x + (2t + 1)$.

Proof As shown in Fig. 13.3, the error patterns $[E_0, E_1]$ for which $|E_0| + |E_1| \leq t$ may be divided into five classes:

Class 1: $E_0 \geq 0 \quad E_1 > 0$
Class 2: $E_0 < 0 \quad E_1 \geq 0$

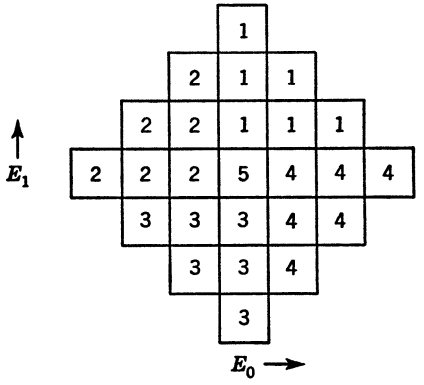


Fig. 13.3 A two-dimensional sphere of Lee radius 3.

Class 3: $E_0 \leq 0 \qquad E_1 < 0$
Class 4: $E_0 > 0 \qquad E_1 \leq 0$
Class 5: $E_0 = E_1 = 0$

The number of error patterns in Class 1, 2, 3, or 4 is $\sum_{i=0}^{t} i = t(t+1)/2$, so the total number of error patterns of Lee weight $\leq t$ is $V_t^{(2)} = 1 + 4t(t+1)/2 = q$. Therefore, any t-Lee-error-correcting code which has q codewords of length $n = 2$ over the alphabet of integers mod $q = 2t^2 + 2t + 1$ must be perfect.

We now show that the negacyclic code generated by $g(x) = x + (2t+1)$, $h(x) = x - (2t+1)$ has minimum weight $2t+1$. For suppose $[C_0,C_1]$ is a nonzero codeword of weight $\leq 2t$. We may assume that C_1 is the component of smaller magnitude, for if it is not, we may instead consider the negacyclic shift, $[-C_1,C_0]$. If $|C_0| + |C_1| \leq 2t$ and $|C_1| \leq |C_0|$, then $|C_1| \leq t$, so we write $|C_1| = t - a$, where $0 \leq a < t$. Since $C(x)$ is a multiple of $g(x)$, $C_0 \equiv (2t+1)C_1 \bmod q$. Since $(2t+1)(t-a) = q - (2a+1)t - a - 1$, either $|C_0| > 2t+1$ and $|C_0| + |C_1| > 2t+1$ or $|C_0| = (2a+1)t + a + 1$ and $|C_0| + |C_1| = (2a+2)t + 1 \geq 2t+1$, with equality iff $a = 0$. Therefore, the minimum weight of any nonzero codeword is $2t+1$, which is attained by the codeword $[-(t+1),t]$, its negative, and their negacyclic shifts. Q.E.D.

Corollary 13.26 If $2t^2 + 2t + 1$ divides q, there exists a perfect t-Lee-error-correcting code of length $n = 2$ over the alphabet of integers mod q.

Proof Let $q = k(2t^2 + 2t + 1)$. The code may be taken as the set of all $k^2(2t^2 + 2t + 1)$ codewords of the form $[C_0 + i(2t^2 + 2t + 1), C_1 + j(2t^2 + 2t + 1)]$, where $0 \leq i < k$, $0 \leq j < k$, and $\mathbf{C} = [C_0, C_1]$ is a codeword in the perfect t-Lee-error-correcting code over the alphabet of integers mod $2t^2 + 2t + 1$. Q.E.D.

Theorem 13.25 guarantees the existence of a perfect linear 6-Lee-error-correcting code mod 85, as well as perfect linear codes over several other modular rings which are not fields. Good linear codes over such alphabets are quite exceptional. Levy (1964) has shown that the best long linear codes over certain nonfields have minimum distances substantially below certain known lower bounds (Theorem 13.71) for long nonlinear codes over such alphabets.

Figure 13.4 gives a geometric interpretation of the perfect 3-Lee-error-correcting code of length 2 over the alphabet of integers mod 25. The 25 codewords are the boldface letters **A**, **B**, **C**, . . . , **Y**. (For example, **A** = [0,0]; **J** = [9,13].) Each of the 625 entries in the figure lists the message which the decoder would select if the corresponding word were received. For example, the received word [11,7] should be decoded as **M**, since the nearest codeword is [12,9] = **M**. Similarly, the received word [0,4] should be decoded as [−3,4] = [22,4] = **W**.

In Figure 13.4, the topmost row is adjacent to the bottommost row and the leftmost column is adjacent to the rightmost column. For this reason, the space represented by Fig. 13.4 is topologically a torus. Therefore, the problem of designing perfect Lee-error-correcting codes is equivalent to the problem of packing spheres into toruses. Figure 13.4 shows a perfect packing of 25 two-dimensional spheres of radius 3 into a 25^2 torus.

Golomb and Welch (1968) have shown that this sphere-packing problem is a special case of a more general polyomino-packing problem. They conjecture that no perfect Lee-error-correcting codes exist with $t > 1$, $n > 2$, and $q > 3$. They have proved several special cases of this conjecture.

Except for the binary repetition codes, there are only two known perfect codes with $t > 1$ and $n > 2$. Both are cyclic, and both are called *Golay codes*. The binary Golay code has block length 23, rate 12/23. It is, in fact, the double-error-correcting BCH code which we studied in Example 5.6. The BCH construction guarantees only that the distance of this code is ≥ 5. However, the actual minimum distance of this code is 7, and the code is a perfect triple-error-correcting code. The ternary Golay code has block length 11 and rate 6/11. It is a perfect double-error-correcting code over GF(3). Both Golay codes are quadratic-residue codes, which we shall study further in Sec. 15.2.

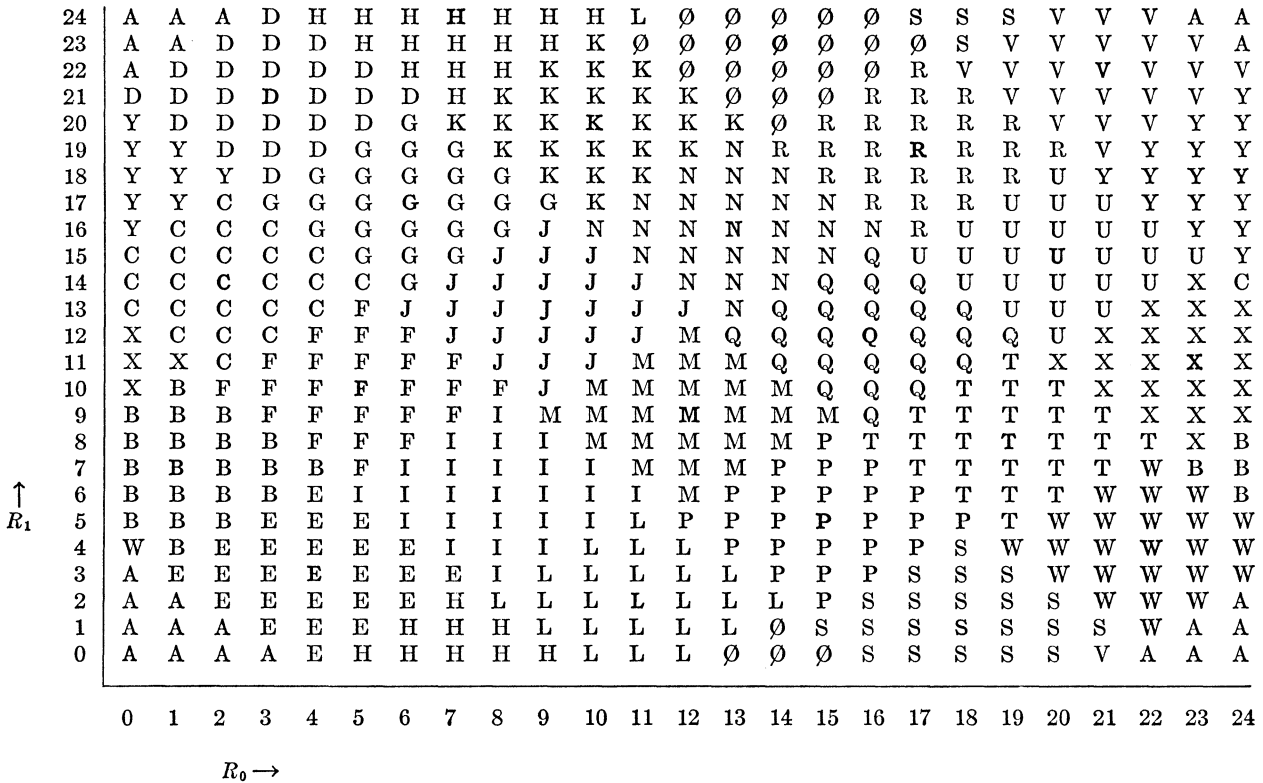


Fig. 13.4 Geometric interpretation of a perfect Lee metric negacyclic code. $n = 2, q = 25, t = 3$.

No other perfect codes are known. Integers t and n for which $V_t^{(n)}$ is a divisor of q^m are rare. Even when such values are found, there may be no corresponding perfect code. For example, over the binary alphabet, $V_2^{(90)} = 2^{12}$. However, Golay (1949), Cohen (1964), and Alter (1968) have proved that the ternary Golay code is the only nontrivial perfect double-Hamming-error-correcting code with $q = 2$, 3, 4, 5, or 7.

In a linear t-error-correcting perfect code, every word of weight $\leq t$ must be a coset leader, and every coset leader must have weight $\leq t$. If every word of weight $\leq t$ is a coset leader and every coset leader has weight $\leq t + 1$, the code is said to be *quasi-perfect.* Quasi-perfect binary codes appear to be much more numerous than perfect binary codes. Gorenstein, Peterson, and Zierler (1960) have shown that all double-error-correcting primitive binary BCH codes are quasi-perfect, and Wagner (1966*a*, 1967) has discovered several noncyclic quasi-perfect binary codes. As we shall see in Sec. 13.6, very long codes cannot be quasi-perfect unless they have very high rates.

★ 13.3 THE BOUND $d \leq n + 1 - k$

As shown in Sec. 1.2, any linear code of length n and dimension k may be defined in terms of a parity-check matrix which has n columns and $n - k$ linearly independent rows. If we annex to this matrix $k - 1$ additional rows, each of which is an n-dimensional unit vector, we obtain the parity-check matrix of a subcode of the original code. This subcode consists of those codewords which have zeros in a specified set of $k - 1$ positions. This subcode is a linear code of positive dimension. Since its only nonzero coefficients are confined to $n - (k - 1)$ positions, its Hamming distance is bounded by

$$d \leq n + 1 - k \tag{13.31}$$

This bound must also hold for the original code.

Let us now consider codes for which

$$d = n + 1 - k \tag{13.32}$$

Such codes are called *maximum distance separable codes.*† Any such code must also have certain additional properties. Given any set of d positions, the code must contain $q - 1$ codewords of weight d whose only nonzero coefficients lie in the given d positions. The subcode consisting of those codewords which have zeros in a specified set of $n - d$

† Some authors have defined codes with this property as *optimal codes* or *maximum codes.* I prefer to avoid restricting such general terms to so specialized a property.

positions has dimension 1; the larger subcode consisting of those codewords which have zeros in any specified set of $n - w$ positions ($w \geq d$) has dimension $w + 1 - d$.

Any code which satisfies Eq. (13.32) must contain exactly one codeword of weight d which has a one in any given position and nonzeros only in $d - 1$ other given positions. The code must contain exactly $n - (d - 1) = k$ codewords of weight d which have one in a given position and nonzeros in each of $d - 2$ other given positions. If any pair of these k codewords had the same nonzero coefficient in one of the $d - 2$ fixed positions, the difference of those two codewords would be a nonzero codeword of weight $< d$. Therefore, $k \leq q - 1$. We repeat:

$$\text{If } d = n + 1 - k > 2 \qquad \text{then } q \geq k + 1 \tag{13.33}$$

By a similar argument applied to the dual code,† Singleton (1964) has shown the following:

$$\text{If } d = n + 1 - k < n - 1 \qquad \text{then } q \geq n + 1 - k \tag{13.34}$$

According to Eq. (13.33), no binary code can satisfy Eq. (13.32) unless $k = 1$ (the binary repetition codes) or $d = 2$ (the single-parity-check codes). Although these trivial cases are the only binary codes with $d = n + 1 - k$, nontrivial codes with $d = n + 1 - k$ do exist over larger alphabets. One class of such codes is given by Theorem 13.35.

Theorem 13.35 The distance, length, and number of information symbols in any Reed-Solomon code (a BCH code with $q = n + 1$) are related by $d = n - k + 1$.

Proof The generator polynomial of any BCH code is defined by $g(x) = \prod_{i=1}^{d-1} M^{(i)}(x)$, where $M^{(i)}(x)$ is the minimal polynomial of α^i, α being a primitive nth root of unity in an extension field of GF(q). For RS codes, $\alpha \in \text{GF}(q)$, $M^{(i)}(x) = x - \alpha^i$, and $\deg g(x) = d - 1$. Q.E.D.

† The codewords of the *dual* of a linear code are the vectors in the row space of its parity-check matrix. It is easily verified that the dual of the dual of a code is the code itself.

Other classes of maximum distance separable codes have been presented by Singleton (1964). In Sec. 16.5, we shall see that it is possible to determine the complete weight structure of all of these codes.

13.4 PLOTKIN'S LOW-RATE AVERAGE DISTANCE BOUND

The minimum distance between any pair of codewords in a code cannot exceed the average distance between all pairs of different codewords. This observation, due to Plotkin, forms the basis for another bound on the minimum distance. This bound was first obtained for the Lee metric by Graham and Wyner (1968).

Suppose the code contains K codewords, $\mathbf{C}^{(1)}, \mathbf{C}^{(2)}, \mathbf{C}^{(3)}, \ldots, \mathbf{C}^{(K)}$, where $\mathbf{C}^{(i)} = [C_0^{(i)}, C_1^{(i)}, \ldots, C_{n-1}^{(i)}]$. Then the total distance between pairs of codewords is given by

$$\begin{aligned} d_{\text{tot}} &= \sum_{i=1}^{K} \sum_{j=1}^{K} d(\mathbf{C}^{(i)}, \mathbf{C}^{(j)}) \\ &= \sum_{i=1}^{K} \sum_{j=1}^{K} \sum_{m=0}^{n-1} d(C_m^{(i)}, C_m^{(j)}) \\ &= \sum_{m=0}^{n-1} \sum_{i=1}^{K} \sum_{j=1}^{K} d(C_m^{(i)}, C_m^{(j)}) \end{aligned}$$

For $I = 0, 1, 2, \ldots, q-1$, we let $J_I^{(m)}$ denote the number of occurrences of the Ith channel letter among the letters $C_m^{(1)}, C_m^{(2)}, C_m^{(3)}, \ldots, C_m^{(K)}$. The probability vector $\mathbf{P}^{(m)} = [J_0^{(m)}, J_1^{(m)}, \ldots, J_{q-1}^{(m)}]/K$ then denotes the composition of the mth letter of a codeword selected from the code at random. We then have

$$\begin{aligned} \sum_{i=1}^{K} \sum_{j=1}^{K} d(C_m^{(i)}, C_m^{(j)}) &= K^2 \sum_{i=0}^{q-1} \sum_{j=0}^{q-1} P_i^{(m)} P_j^{(m)} \mathfrak{D}_{i,j} \\ &= K^2 \mathbf{P}^{(m)} \mathfrak{D} \mathbf{P}^{(m)t} \end{aligned}$$

where $\mathfrak{D}_{i,j}$, the i,jth entry of the $\mathfrak{D}$ matrix, is the distance between the ith channel input letter and the jth channel input letter. For example, if $q = 5$, we have

$$\mathfrak{D} = \begin{bmatrix} 0 & 1 & 2 & 2 & 1 \\ 1 & 0 & 1 & 2 & 2 \\ 2 & 1 & 0 & 1 & 2 \\ 2 & 2 & 1 & 0 & 1 \\ 1 & 2 & 2 & 1 & 0 \end{bmatrix} \quad \text{for the Lee metric}$$

and

$$\mathfrak{D} = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 0 \end{bmatrix} \quad \text{for the Hamming metric}$$

In any case, the total distance is given by

$$d_{\text{tot}} = K^2 \sum_{m=0}^{n-1} \mathbf{P}^{(m)} \mathfrak{D} \mathbf{P}^{(m)t}$$

This total distance includes the sum over K^2 pairs of codewords. K of these pairs consist of the same codeword, and $K^2 - K$ pairs consist of different codewords. The average distance among the $K(K-1)$ pairs of different codewords is therefore given by

$$d_{\text{av}} = \frac{K^2}{K(K-1)} \sum_{m=0}^{n-1} \mathbf{P}^{(m)} \mathfrak{D} \mathbf{P}^{(m)t} \leq \frac{n}{1 - K^{-1}} \mathbf{P} \mathfrak{D} \mathbf{P}^t$$

where $\mathbf{P}$ represents the best possible column composition. Since the minimum distance cannot exceed the average distance,

$$d_{\min} \leq \frac{n}{1 - K^{-1}} \mathbf{P} \mathfrak{D} \mathbf{P}^t \tag{13.41}$$

Equation (13.41) is called the *Plotkin bound* on minimum distance.

From symmetry considerations, it is reasonable to conjecture that the best possible column composition is given by $\mathbf{P} = \mathbf{1}/q = [1,1, \ldots ,1]/q$. However, there are symmetric $\mathfrak{D}$ matrices for which $\mathbf{1}/q$ is *not* the best column composition. In general, we may decide whether or not $\mathbf{1}/q$ is an optimum composition for any given $\mathfrak{D}$ matrix by computing its eigenvalues. If the channel noise is additive mod q, then $\mathfrak{D}_{l,j} = \mathfrak{D}_{k,m}$ whenever $l - j \equiv k - m \bmod q$. This implies that the eigenvectors of $\mathfrak{D}$ are given by $\varrho^{(k)} = [1, e^{2\pi ik/q}, e^{4\pi ik/q}, \ldots]$ for $k = 0, 1, 2, \ldots, q-1$, where $i = \sqrt{-1}$, $\pi = 3.14159 \cdots$, and $e = 2.71828 \cdots$. The choice $k = 0$ corresponds to the eigenvector $\varrho^{(0)} = \mathbf{1}$. The eigenvalue corresponding to the eigenvector $\varrho^{(k)}$ is $\lambda_k = \sum_{j=0}^{q-1} \mathfrak{D}_{0,j} e^{2\pi ikj/q}$. It is easily verified that $\mathfrak{D}\varrho^{(k)} = \lambda_k \varrho^{(k)}$ for $k = 0, 1, 2, \ldots, q-1$. It is also easily verified that the eigenvectors $\varrho^{(k)}$ are orthogonal, in the sense that $\varrho^{(j)} \varrho^{*(k)t} = 0$ if $j \neq k$. Here $\varrho^{*(k)}$ denotes the complex conjugate of $\varrho^{(k)}$.

In general, $\lambda_{q-k} = \lambda_k^*$, the complex conjugate of λ_k. If the $\mathfrak{D}$ matrix is symmetric, so that $\mathfrak{D}_{k,j} = \mathfrak{D}_{j,k}$ for all j and k, then all eigenvalues are

real. In this case, we may use the real eigenvectors $\varrho^{(k)} + \varrho^{(q-k)}$ and $i(\varrho^{(k)} - \varrho^{(q-k)})$ instead of the complex eigenvectors $\varrho^{(k)}$ and $\varrho^{(q-k)}$, but there is little advantage in doing so.

The eigenvalues for the $\mathfrak{D}$ matrices corresponding to the Hamming metric and the Lee metric are easily calculated. Setting $u = e^{2\pi i/q}$, we find for the Hamming metric that

$$\lambda_k = \sum_{j=1}^{q-1} u^{kj} = \frac{u^{kq} - u^k}{u^k - 1} = -1 \qquad \text{if } k \not\equiv 0 \tag{13.42}$$

$$\lambda_0 = \sum_{j=1}^{q-1} 1 = q - 1 \tag{13.43}$$

The *principal* eigenvalue, i.e., the one having greatest magnitude, is λ_0.

To evaluate λ_k for the Lee metric, we must sum series of the form

$$\sum_{j=1}^{n} jz^j = z \sum_{j=0}^{n} jz^{j-1} = z \frac{d}{dz} \sum_{j=0}^{n} z^j = z \frac{d}{dz} \frac{z^{n+1} - 1}{z - 1}$$
$$= \frac{z[(n+1)z^n(z-1) - (z^{n+1} - 1)]}{(z-1)^2}$$

or

$$\sum_{j=1}^{n} jz^j = \frac{1 + (n+1)z^n(z-1) - z^{n+1}}{(z^{1/2} - z^{-1/2})^2} \tag{13.44}$$

For odd q, the secondary eigenvalues for the $\mathfrak{D}$ matrices of the Lee metric are given by $\lambda_k = 2 \operatorname{Re} \left(\sum_{j=1}^{(q-1)/2} ju^j \right)$ where $u = e^{2\pi ik/q}$. Since $u^{q/2} = (-1)^k$ and $u^{1/2} - u^{-1/2} = 2i \sin(\pi k/q)$, we may use Eq. (13.44) to obtain

$$\lambda_k = 2 \operatorname{Re} \left[\frac{1 + \frac{q+1}{2}(-1)^k 2i \sin \frac{\pi k}{q} - (-1)^k \left(\cos \frac{\pi k}{q} + i \sin \frac{\pi k}{q} \right)}{-4 \sin^2 (\pi k/q)} \right]$$
$$= \frac{-1 + (-1)^k \cos(\pi k/q)}{2 \sin^2 (\pi k/q)} \qquad \text{for } k \not\equiv 0 \tag{13.45}$$

The principal eigenvalue is given by

$$\lambda_0 = 2 \sum_{j=1}^{(q-1)/2} j = \frac{q-1}{2} \frac{q+1}{2} = \frac{q^2 - 1}{4} \tag{13.46}$$

For even q, the secondary eigenvalues for the $\mathfrak{D}$ matrices of the Lee

metric are given by

$$\begin{aligned}\lambda_k &= \operatorname{Re}\left(\frac{q}{2}u^{q/2} + 2\sum_{j=1}^{(q-2)/2} ju^j\right)\\ &= \operatorname{Re}\left\{(q/2)(-1)^k + \frac{2[1 + (q/2)(-1)^k(1 - u^{-1}) - (-1)^k]}{-4\sin^2(\pi k/q)}\right\}\\ &= \operatorname{Re}\left[\frac{2 - 2(-1)^k + (q/2)(-1)^k(u - u^{-1})}{-4\sin^2(\pi k/q)}\right]\\ &= \frac{-1 + (-1)^k}{2\sin^2(\pi k/q)} \end{aligned} \tag{13.47}$$

For even q, the principal eigenvalue for the $\mathfrak{D}$ matrix of the Lee metric is given by

$$\lambda_0 = \frac{q}{2} + 2\sum_{j=1}^{(q-2)/2} j = \frac{q}{2} + \frac{q}{2}\frac{q-1}{2} = \frac{q^2}{4} \tag{13.48}$$

From Eqs. (13.42), (13.45), and (13.47), we observe that the $\mathfrak{D}$ matrices corresponding to the Hamming and Lee metrics have an important property: *all secondary eigenvalues are nonpositive.* This assures us that the quadratic form $\mathbf{P}\mathfrak{D}\mathbf{P}^t$ is a *concave* function of the probability vector $\mathbf{P}$. This means that, if $a > 0$, $b > 0$, $a + b = 1$, $\mathbf{1P}^t = 1$, $\mathbf{1Q}^t = 1$, then

$$(a\mathbf{P} + b\mathbf{Q})\mathfrak{D}(a\mathbf{P} + b\mathbf{Q})^t \geq a\mathbf{P}\mathfrak{D}\mathbf{P}^t + b\mathbf{Q}\mathfrak{D}\mathbf{Q}^t$$

To prove this, we expand $\mathbf{P}$ and $\mathbf{Q}$ in terms of the real eigenvectors of $\mathfrak{D}$. We set $\mathbf{P} = \sum_{j=0}^{q-1} P_j\varrho^{(j)}$, $\mathbf{Q} = \sum_{j=0}^{q-1} Q_j\varrho^{(j)}$ so

$$(a\mathbf{P} + b\mathbf{Q})\mathfrak{D}(a\mathbf{P} + b\mathbf{Q})^t = q\sum_{j=0}^{q-1} \lambda_j(aP_j + bQ_j)^2$$

$$a\mathbf{P}\mathfrak{D}\mathbf{P}^t + b\mathbf{Q}\mathfrak{D}\mathbf{Q}^t = aq\sum_{j=0}^{q-1} \lambda_j P_j^2 + bq\sum_{j=0}^{q-1} \lambda_j Q_j^2$$

Since the eigenvectors are orthogonal and $\mathbf{1P}^t = \mathbf{1Q}^t = 1$, $P_0 = Q_0 = aP_0 + bQ_0 = 1/q$ and $\lambda_0(aP_0 + bQ_0)^2 = a\lambda_0 P_0^2 + b\lambda_0 Q_0^2$. The concavity assertion then reduces to the claim that

$$\sum_{j=1}^{q-1} \lambda_j(aP_j + bQ_j)^2 \geq \sum_{j=1}^{q-1} \lambda_j(aP_j^2 + bQ_j^2)$$

Since x^2 is a convex function of x, $(aP_j + bQ_j)^2 \leq aP_j^2 + bQ_j^2$ for all j.

Since all secondary eigenvalues are nonpositive,

$$\lambda_j(aP_j + bQ_j)^2 \geq \lambda_j(aP_j^2 + bQ_j^2) \qquad \text{for all } j \geq 1$$

and $\mathbf{P}\mathfrak{D}\mathbf{P}^t$ is a concave function of the probability vector $\mathbf{P}$.

Because of this concavity, any stationary point of the quadratic form $\mathbf{P}\mathfrak{D}\mathbf{P}^t$ must be a global maximum. By symmetry, the choice $\mathbf{P} = \mathbf{1}/q$ is a stationary point. Thus, we can determine the maximum value of $\mathbf{P}\mathfrak{D}\mathbf{P}^t$ in Eq. (13.41) from Eqs. (13.43), (13.46), and (13.48). We then have Theorem 13.49.

Theorem 13.49 The minimum distance of any code which has K codewords of block length n over the alphabet of q letters is bounded by

$$d_{\min} \leq \frac{\bar{D}n}{1 - K^{-1}}$$

where the constant $\bar{D} = \sum_{i=1}^{q-1} \mathfrak{D}_{0,i}/q$ is given by

$$\bar{D} = \begin{cases} \dfrac{q-1}{q} & \text{Hamming metric} \\ \dfrac{q^2-1}{4q} & \text{Lee metric, } q \text{ odd} \\ \dfrac{q}{4} & \text{Lee metric, } q \text{ even} \end{cases}$$

★ 13.5 EQUIDISTANT CODES

In order for a code to satisfy Theorem 13.49 with equality, its minimum distance must equal its average distance. This can happen only if the distance between every pair of codewords is the same. If the distance between every pair of codewords is the same, the code is said to be *equidistant.*

The most important class of equidistant codes is described by Definition 13.51 and Theorem 13.52.

Definition 13.51 A *maximal-length shift-register* code is a cyclic, negacyclic, or constacyclic code whose parity-check polynomial $h(x)$ is a primitive polynomial of degree k over GF(q). If the block length is n, $h(x)$ must divide $x^n - \xi$, where $\xi \in \text{GF}(q)$ and the order of ξ is $(q^k - 1)/n$.

Theorem 13.52 Every nonzero codeword of a cyclic, negacyclic, or constacyclic maximal-length shift-register code is a cyclic, negacyclic, or constacyclic shift of every other nonzero codeword. Therefore, all maximal-length shift-register codes are equidistant in the Hamming metric, and all cyclic and negacyclic maximal-length shift-register codes are equidistant in the Lee metric.

Proof The code contains $q^k - 1$ nonzero codewords, all of which are multiples of the code's generator polynomial, $g(x) = (x^n - \xi)/h(x)$. We must show that all $q^k - 1$ constacyclic shifts of the generator polynomial are distinct. If not, then $x^i g(x) \equiv x^m g(x) \bmod x^n - \xi$, and there exists a $j = m - i, j < q^k - 1$, such that $(x^j - 1)g(x) \equiv 0 \bmod x^n - \xi$. Since $x^n - \xi = g(x)h(x)$, $x^j - 1 \equiv 0 \bmod h(x)$. But $h(x)$ is a primitive polynomial, so it can divide $x^j - 1$ only if j is a multiple of $q^k - 1$. Therefore all $q^k - 1$ nonzero codewords are constacyclic shifts of the generator polynomial, and all have equal Hamming weight. If the codewords are all cyclic or negacyclic shifts of the generator polynomial, then they all have equal Lee weight. However, if $\xi \neq \pm 1$, then the constacyclic shifts of the generator polynomial have different Lee weights.

Q.E.D.

In some cases, Theorem 13.52 shows the actual minimum weight of certain maximal-length shift-register codes to be greater than what one might expect from other considerations. For example, consider the maximal-length shift-register negacyclic code of block length 63 over GF(127). This code appears in Table 9.1. According to Theorem 9.34, this code is able to correct all patterns of up to 63 Lee errors. Yet, according to Theorem 13.52, every nonzero codeword in this code actually has Lee weight 32×63. Therefore, the code is capable of correcting almost 16 times as many errors as promised by Theorem 9.34.

Given an equidistant code with K codewords of block length n, one may construct an equidistant code containing K codewords of block length jn by simply repeating each word of the original code j times. If the original code is linear, the repeated code is also linear; if the original code is cyclic, the repeated code is also cyclic, with the same parity-check polynomial.

The maximal-length shift-register codes over GF(2) have $K = 2^k$ codewords and block length $n = 2^k - 1$. For many other values of K, equidistant binary codes having K codewords of block length $n = K - 1$ may be constructed from *Hadamard matrices*. A Hadamard matrix is

an orthogonal $K \times K$ matrix whose elements are the real numbers $+1$ and -1. For example,

$$\mathfrak{IC}_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} + & + \\ + & - \end{bmatrix}$$

$$\mathfrak{IC}_4 = \begin{bmatrix} + & + & + & + \\ + & + & - & - \\ + & - & + & - \\ + & - & - & + \end{bmatrix}$$

$$\mathfrak{IC}_{12} = \begin{bmatrix} + & + & + & + & + & + & + & + & + & + & + & + \\ + & - & - & + & - & - & - & + & + & + & - & + \\ + & - & + & - & - & - & + & + & + & - & + & - \\ + & + & - & - & - & + & + & + & - & + & - & - \\ + & - & - & - & + & + & + & - & + & - & - & + \\ + & - & - & + & + & + & - & + & - & - & + & - \\ + & - & + & + & + & - & + & - & - & + & - & - \\ + & + & + & + & - & + & - & - & + & - & - & - \\ + & + & + & - & + & - & - & + & - & - & - & + \\ + & + & - & + & - & - & + & - & - & - & + & + \\ + & - & + & - & - & + & - & - & - & + & + & + \\ + & + & - & - & + & - & - & - & + & + & + & - \end{bmatrix}$$

There is no loss of generality in assuming that all the entries in the first row and the first column of a Hadamard matrix have value $+1$. By omitting the first column from a $K \times K$ Hadamard matrix, one obtains K rows of length $n = K - 1$. These rows form an equidistant binary code. If K is not a power of 2, the code is nonlinear.

It is easily shown that $K \times K$ Hadamard matrices can exist only if $K = 2$ or if K is a multiple of 4. It is conjectured that $K \times K$ Hadamard matrices exist for every such K, but this has not been proved. The first possible counterexample is $K = 188$. For all smaller K (and many larger K), Hadamard matrices can be constructed by methods discussed by Hall (1967) or the more recent papers by Spence (1967) and by Goethals and Seidel (1967).

Although the total distance of any linear code must be $\bar{D}K^2n$, where $\bar{D}$ is given by Theorem 13.49, a nonlinear code may have a smaller total distance. A nonlinear code may be equidistant even though its distance is much less than the Plotkin bound. For example, the binary code whose n codewords consist of the n rows of the identity matrix is equidistant, with distance 2. Although this is a rather poor code, one can do no better if every codeword is constrained to have weight 1. In general, if the weight of each codeword is sufficiently constrained, then

the average distance between pairs of codewords cannot be large. This observation forms the basis of the Elias bound, which we present in the next section.

13.6 THE ELIAS BOUND

We have seen that the volume bound is tight at high rates but weak at low rates and that the average distance bound is tight at low rates but weak at high rates. At medium rates, we may obtain a stronger bound by combining the volume bound and the average distance bound, using an ingenious unpublished argument first introduced by Elias in 1960. Certain refinements of this bound have been presented by Wyner (1965).

We give the Elias bound as a pair of lemmas. Lemma 13.61 uses a volume-bound argument to show that the code must contain a critical sphere which has many codewords. Lemma 13.62 then shows that the average distance among codewords in the critical sphere is small.

Lemma 13.61 Given an integer t and a code of length n and rate R, there exists a critical sphere of radius t which includes K codewords, where $K \geq V_t^{(n)}/q^{n(1-R)}$. By suitable translation of the code, this critical sphere may be centered at $\mathbf{0}$.

Proof Place a sphere of radius t about each of the q^n points in the space, and let the number of codewords in the ith sphere be denoted by K_i ($i = 1, 2, \ldots, q^n$). Each of the q^{Rn} codewords appears in $V_t^{(n)}$ spheres, so $\sum_{i=1}^{q^n} K_i = q^{Rn} V_t^{(n)}$, and $K = \max_i K_i \geq V_t^{(n)}/q^{(1-R)n}$. The sphere containing the most codewords is called the *critical sphere*. If the word at the center of the critical sphere is subtracted from each of the codewords, the center of the critical sphere is translated to $\mathbf{0}$, and the minimum distance of the code is unchanged. Q.E.D.

We next compute the average distance between pairs of codewords *within the critical sphere*. Since these words are all within a distance t of the center of the sphere, they must be close to each other. For this reason, the average distance between pairs of the codewords in the critical sphere is considerably smaller than the average distance between all pairs of codewords. The result of this calculation is Lemma 13.62.

Lemma 13.62 If each of K codewords has weight $\leq \bar{D}xn \leq \bar{D}n$, where $\bar{D} = \sum_{i=1}^{q-1} \mathfrak{D}_{0,i}/q$, then the distance between some pair of these K codewords must be no greater than $x(2 - x)\bar{D}n/(1 - K^{-1})$.

Proof As in Sec. 13.4, $d_{\min} \leq d_{\text{tot}}/(K^2 - K)$, where d_{tot} is the total distance among all K^2 ordered pairs of codewords. If the probability vector $\mathbf{P}^{(m)}$ denotes the composition of the mth column of the code, then

$$d_{\text{tot}} = \sum_{m=1}^{n} K^2 \mathbf{P}^{(m)} \mathfrak{D} \mathbf{P}^{(m)t}$$

where $\mathfrak{D}_{i,j}$ is the distance between the ith channel input letter and the jth channel input letter. The total weight of all of the codewords is given by $\sum_{m=1}^{n} K\mathbf{D}^{(0)}\mathbf{P}^{(m)t}$, where $\mathbf{D}^{(0)}$ is the first row of the $\mathfrak{D}$ matrix, namely, $\mathbf{D}^{(0)} = [\mathfrak{D}_{0,0}, \mathfrak{D}_{0,1}, \mathfrak{D}_{0,2}, \ldots, \mathfrak{D}_{0,q-1}]$. If each of the K codewords has weight $\leq \bar{D}xn$, then the total weight is at most $K\bar{D}xn$, and $\sum_{m=1}^{n} K\mathbf{D}^{(0)}\mathbf{P}^{(m)t} \leq K\bar{D}xn$.

We must now find the probability vectors $\mathbf{P}^{(1)}, \mathbf{P}^{(2)}, \ldots, \mathbf{P}^{(n)}$ which maximize $\sum_{m=1}^{n} \mathbf{P}^{(m)} \mathfrak{D} \mathbf{P}^{(m)t}$ subject to the constraint $\sum_{m=1}^{n} \mathbf{D}^{(0)}\mathbf{P}^{(m)t} \leq \bar{D}xn$. This maximization is performed in two steps. First, we maximize $\mathbf{P}^{(m)} \mathfrak{D} \mathbf{P}^{(m)t}$ subject to the constraint $\mathbf{D}^{(0)}\mathbf{P}^{(m)t} = \bar{D}x_m$, and then we choose the maximizing x_m's, subject to the constraint that $\sum_{m=1}^{n} x_m \leq xn$.

The constraints require that

$$\mathbf{1}\mathbf{P}^{(m)t} = 1 \tag{13.63}$$

and

$$\mathbf{D}^{(0)}\mathbf{P}^{(m)t} = \bar{D}x_m \tag{13.64}$$

where $\mathbf{1} = [1,1,1, \ldots, 1]$. Subject to these constraints, the objective function $\mathbf{P}^{(m)} \mathfrak{D} \mathbf{P}^{(m)t} = \sum_i \sum_j P_i^{(m)} P_j^{(m)} \mathfrak{D}_{i,j}$ is identical to the objective function

$$\sum_i \sum_j P_i^{(m)} P_j^{(m)} \mathfrak{D}_{i,j} + A\left(\bar{D}x_m - \sum_i \mathfrak{D}_{0,i} P_i^{(m)}\right) + B\left(1 - \sum_i P_i^{(m)}\right)$$

where Lagrange multipliers A and B are arbitrary constants. To find a stationary point of this objective function, we set the partial derivative with respect to $P_k^{(m)}$ to zero. Since $\mathfrak{D}_{k,k} = 0$, this gives $\sum_i P_i^{(m)}\mathfrak{D}_{i,k} + \sum_j P_j^{(m)}\mathfrak{D}_{k,j} - A\mathfrak{D}_{0,k} - B = 0$ for all k or

$$2\sum_i P_i^{(m)}\mathfrak{D}_{i,k} = A\mathfrak{D}_{0,k} + B \tag{13.65}$$

Setting

$$\boldsymbol{\delta} = [1,0,0, \ldots ,0]$$

we can verify that one solution to Eqs. (13.63) to (13.65) is given by

$$\begin{aligned} \mathbf{P}^{(m)} &= \frac{x_m\mathbf{1}}{q} + (1 - x_m)\boldsymbol{\delta} \\ A &= 2(1 - x_m) \\ B &= 2x_m\bar{D} \\ \mathbf{P}^{(m)}\mathfrak{D}\mathbf{P}^{(m)t} &= x_m(2 - x_m)\bar{D} \end{aligned} \tag{13.66}$$

According to Sec. 13.4, $\mathbf{P}\mathfrak{D}\mathbf{P}^t$ is a concave function of the probability vector $\mathbf{P}$. Therefore, the stationary point given by Eq. (13.66) must maximize $\mathbf{P}^{(m)}\mathfrak{D}\mathbf{P}^{(m)t}$ subject to constraints (13.63) and (13.64).

To conclude the proof, we must choose $x_1, x_2, \ldots, x_n$ which maximize $\sum_{m=1}^{n} \bar{D}x_m(2 - x_m)$ subject to the constraint $\sum_{m=1}^{n} x_m \leq nx$. Symmetry suggests that we choose all $x_m = x$. This is indeed a stationary point, and since $x(2 - x)$ is a concave function of x, this choice is correct.

Thus, the total distance, $d_{\text{tot}} = K^2 \sum_{m=1}^{n} \mathbf{P}^{(m)}\mathfrak{D}\mathbf{P}^{(m)t}$, is maximized subject to the constraint $\sum_{m=1}^{n} \mathbf{D}^{(0)}\mathbf{P}^{(m)t} \leq \bar{D}xn$ by choosing all $x_m = x$ and all $\mathbf{P}^{(m)}$ according to Eq. (13.66). The total distance is then $K^2x(2 - x)\bar{D}n$, and the average distance among the $K^2 - K$ pairs of nonidentical codewords is $K^2x(2 - x)\bar{D}n/K(K - 1)$.

Q.E.D.

In order to combine Lemmas 13.61 and 13.62 effectively, we must first choose t, the radius of the critical sphere. This t should be chosen in such a way as to minimize $\bar{D}xn(2 - x)/(1 - K^{-1})$, where $x = t/\bar{D}n$ and K is the least integer not less than $V_t^{(n)}/q^{n(1-R)}$. For large block

lengths, the volume $V_t^{(n)}$ increases so rapidly with t that the best choice of t is only slightly greater than the least integer for which $V_t^{(n)}$ exceeds $q^{n(1-R)}$.

According to Eqs. (13.15) and (13.18),

$$\lim_{n\to\infty} \frac{1}{n} \log \frac{V_{n\delta(R)}^{(n)}}{q^{n(1-R)}} = 0$$

Therefore, if the ratio t/n is held constant at any value $>\delta(R)$, then $V_t^{(n)}/q^{n(1-R)}$ goes to infinity with increasing n. We then have $K^{-1} \ll 1$. Ignoring this term in the denominator of Lemma 13.62 gives Theorem 13.67, the asymptotic form of the Elias bound.

Theorem 13.67 Given any rate R and any $\epsilon > 0$, then for sufficiently large n,

$$\frac{d_{\min}}{n} < \delta(R)\left[2 - \frac{\delta(R)}{\bar{D}}\right] + \epsilon$$

where $\delta(R)$ is given by Eqs. (13.18) and (13.15) and $\bar{D}$ is given by Theorem 13.49.

A comparison with the asymptotic form of the volume bound [Eq. (13.19)] shows the Elias bound to be uniformly stronger, although the difference becomes negligible at high rates. At low rates, the Elias bound approaches Plotkin's average distance bound, since $\lim_{R\to 0} \delta(R) = \bar{D}$. The asymptotic form of the Elias bound for binary codes is plotted in Fig. 13.2.

13.7 THE GILBERT BOUND

The Hamming bound, the Plotkin bound, and the Elias bound all provide upper bounds on the minimum distance of any block code, linear or nonlinear. The Gilbert bound, on the other hand, provides a *lower* bound on the minimum distance.

Theorem 13.71 [Gilbert (1952)] If

$$V_{d-1}^{(n)} < q^{n(1-R)}$$

then any code of block length n, minimum distance d, and rate R is a proper subset of another code with the same minimum distance.

Proof Surround each of the q^{nR} codewords with a sphere of radius $d-1$. The volume of each sphere is $V_{d-1}^{(n)}$; the total volume of all of the spheres is $q^{nR}V_{d-1}^{(n)}$. If this total volume is less than q^n, then

there exist points which are not contained in any of the spheres. The distance from any such point to any of the codewords is $\geq d$, so we may choose any one of these points as an additional codeword without decreasing the code's minimum distance. Q.E.D.

According to the Gilbert bound, if we start with any low-rate code of desired minimum distance d, we may continue to add more and more codewords to the code until the rate is increased to at least $1 - (1/n) \log_q V_{d-1}^{(n)}$. The distance of the augmented code will be the same as the distance of the original code. Unfortunately, however, the augmented code will generally have less mathematical structure than the original code. Whatever algebraic decoding algorithms were available for the original code may be inapplicable to the augmented code. The augmented code will not, in general, be cyclic, even if the original code was cyclic. If we proceed naïvely, we may obtain an augmented code which is nonlinear even though the original code was linear. If we are more careful, however, we can avoid this particular pitfall by following Lemma 13.72 and Theorem 13.73.

Lemma 13.72 From an original linear code containing q^{Rn} codewords, we may form a linear augmented code whose q^{1+Rn} codewords are the codewords of the original linear code and all nonzero scalar multiples of the words in any particular coset of the original code.

Proof Let G be the set of original codewords, and let $\mathbf{v}$ be the leader of the augmenting coset. Every word in the augmenting coset is of the form $\mathbf{v} + \mathbf{g}$, for some $\mathbf{g} \in G$, and every word in the augmented code is of the form $s(\mathbf{v} + \mathbf{g})$ or $s\mathbf{g}$, where s is a scalar. Since $s\mathbf{g} \in G$, a word is in the augmented code iff it is of the form $s\mathbf{v} + \mathbf{g}$, $\mathbf{g} \in G$. In this representation, the augmented code is seen to be linear. The augmented code has q^{1+Rn} codewords, corresponding to q choices of s and q^{Rn} choices of $\mathbf{g}$ in the expression $s\mathbf{v} + \mathbf{g}$. Q.E.D.

Theorem 13.73 If q is a prime power and

$$V_{d-1}^{(n)} < q^{n(1-R)}$$

in the Hamming metric or if q is an odd prime power and

$$V_{d-1}^{(n)} < 1 + \frac{2(q^{n(1-R)} - 1)}{q - 1}$$

in the Lee metric or if q is a power of two and

$$V_{d-1}^{(n)} < 1 + \frac{q^{n(1-R)} - 1}{q - 1}$$

in the Lee metric, then every linear code of distance d, rate R, and length n over GF(q) is a proper subset of an augmented linear code with the same minimum distance.

Proof Any code of rate R has $q^{n(1-R)}$ coset leaders, but the space of all q^n points contains only $V_{d-1}^{(n)}$ words of weight $<d$. Therefore, if $V_{d-1}^{(n)} < q^{n(1-R)}$ in the Hamming metric, the code contains a coset leader of weight $\geq d$. All scalar multiples of the words of this coset also have Hamming weight $\geq d$. By Lemma 13.72, we may form an augmented linear code by making this coset and its scalar multiples into codewords.

In the Lee metric, a coset of weight $\geq d$ may contain words which have scalar multiples of weight $<d$, so we must refine the argument. The $q^{n(1-R)} - 1$ proper cosets (excluding the coset consisting of the codewords) may be partitioned into sets, each set consisting of all $q - 1$ nonzero scalar multiples of any of its members. A member of each set with least weight may be designated as set leader. If q is odd, the negatives of set leaders may also be designated as set leaders; in this case each set has two leaders. If the number of leaders exceeds the number of nonzero words with Lee weight $\leq d - 1$, then there must be some leader with weight $\geq d$. The members of the set containing this leader may then be made codewords of the augmented code. Q.E.D.

For finite values of n, d, and R, Varsharmov (1957) and Sacks (1958) have obtained slightly stronger versions of Theorems 13.71 and 13.73. If the original code has distance d, then the spheres of radius $d - 1$ about each code point must overlap slightly, and the bound can be improved by taking this into account. However, none of the refinements yet proposed has resulted in any asymptotic improvement. From Theorems 13.71, 13.73, 13.15, and 13.18, we have Theorem 13.74.

Theorem 13.74 Over GF(q)† there exist linear codes of arbitrarily large block length n which have rate R and minimum distance d satisfying

$$\frac{d}{n} \geq \delta(R)$$

† See discussion on page 307 following proof of Corollary 13.26.

The asymptotic form of the Gilbert bound for binary codes is plotted in Fig. 13.2.

It is not known whether or not there exist arbitrarily long *cyclic* codes which satisfy Theorem 13.74.

From a practical standpoint, it is not very easy to construct good codes by Theorem 13.73. Although a given linear code may be known to contain cosets of weight greater than the minimum distance of the code, there is no easy way to find one of these heavy cosets. For large block lengths and moderate rates, the number of codewords and the number of cosets are both so large than an exhaustive search is not feasible.

13.8 ASYMPTOTIC ERROR BOUNDS AND FINITE SPECIAL CASES

Although minimum distance is certainly one measure of a code's ability to correct errors, it does not tell the entire story. Most codes of distance d are able to correct not only all patterns of fewer than $d/2$ errors but many patterns of more than $d/2$ errors as well. Often it may be possible to correct so many patterns of more than $d/2$ errors that the probability of decoding failure or decoding error is substantially less than the probability that the channel noise causes more than $d/2$ channel errors.

It is very difficult to compute the probability of error for most codes, including most BCH codes, if one uses a complete decoding algorithm. Such a decoding algorithm will decode correctly whenever the error pattern is a coset leader. For most codes, very little is known about the distribution of weights of the coset leaders. From a nonconstructive viewpoint, however, this problem turns out to be much more tractable. It is possible to get rather tight bounds on the probability of error of the "best" code, even though it is very difficult to ever find out what the best code is!

The situation is not unlike the graph shown in Fig. 13.5. With the naked eye, the observer can confidently assert that the minimum value of the function $f(x)$ is about 0.10. Yet he has considerable difficulty in determining the value of x at which the minimum is attained. He may be able to tell that $f(0.4) = 0.2$, but he cannot determine $f(0.5)$ or $f(0.6)$ with much precision.

Although the analogy between the probability of error over the ensemble of all codes and the function $f(x)$ of Fig. 13.5 should not be carried too far, I hope that it may help to emphasize the following points:

1. The ensemble of all codes of given rate and block length contains many codes each of which has a probability of error near the minimum.

2. It is very difficult to evaluate the probability of error of any code, except for certain codes which are unequivocally bad.
3. By clever arguments, it *is* possible to obtain an accurate estimate of the average probability of error of all the codes in a particular set of codes. If almost all the codes in this set have error probabilities near the minimum, then the average probability of error of the codes in this ensemble provides a good estimate of the minimum.

Thus, a great deal is known about the probability of error of the "best" code, even though essentially nothing is known about the best code itself. Most of this information about the probability of error of the best code has been obtained by nonconstructive statistical arguments. These arguments are not algebraic or combinatorial. They are often involved and lengthy, so I will give only a brief summary of some of the more important results. The interested reader may find the proofs of most of these results in the papers by Shannon, Gallager, and Berlekamp (1967) and the references listed there, or in the forthcoming book by Gallager (1968), or in the book by Jelinek (1968).

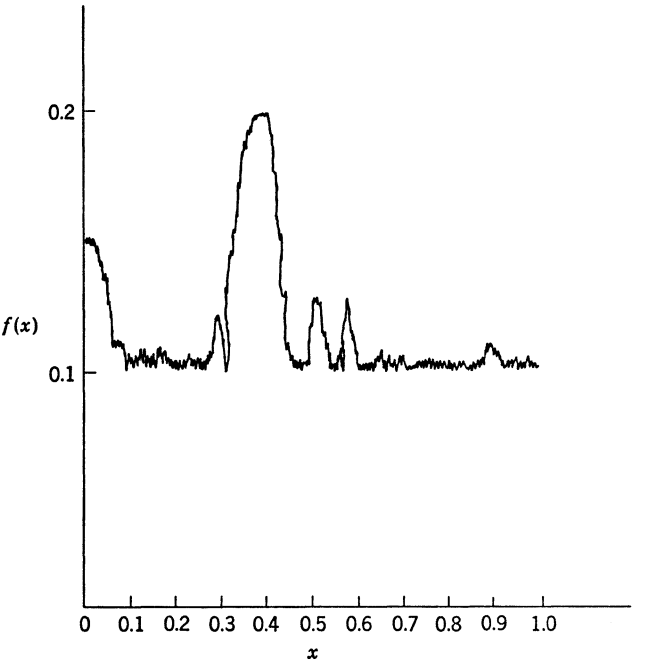


Fig. 13.5 Graphical analogy of the probability of error over the ensemble of codes.

For a given channel, let $P_e(N,M,1)$ denote the probability of error of the best code having M codewords of block length N. The function $P_e(N,M,1)$ is a special case of the function $P_e(N,M,L)$, which is the probability of error of the best code having M codewords of block length N and decoding list L. Using list decoding, the decoder is permitted to list all the possible transmitted codewords, in order of their a posteriori probabilities based on the received word. If the transmitted codeword appears anywhere among the top L words on the decoder's list, the decoder is scored as correct. The decoder is charged with a decoding error only if the codeword that was actually transmitted does not appear among the top L words on the decoder's list. List decoding for the binary symmetric channel was first considered by Elias (1957).

The rate, in natural units, of a code with M codewords, block length N, and list L is defined by

$$R = \frac{1}{N} \ln \frac{M}{L}$$

Most interest centers on the behavior of $P_e(N,M,L)$ when R is held fixed while N is allowed to become arbitrarily large. In 1949, Shannon proved his classic coding theorem, which states that for any discrete memoryless channel (and most other kinds of channels as well), there exists a capacity C which depends only on the noise statistics of the channel and not on the code. This capacity has the important property that if $R < C$,

$$\lim_{N\to\infty} P_e(N,[\exp RN],1) = 0$$

where $[\exp RN]$ is the least integer not exceeding $\exp RN$. Shannon (1948, 1949) also showed that if $R > C$, then

$$\lim_{N\to\infty} P_e(N,[\exp RN],1) > 0$$

Wolfowitz (1961) showed that if $R > C$, then

$$\lim_{N\to\infty} P_e(N,[\exp RN],1) = 1$$

Feinstein (1954, 1958) showed that if $R < C$, then

$$P_e(N,[\exp RN],1) < \exp\{-NE_r(R)\}$$

where $E_r(R)$ is a positive function of R which goes to 0 as R goes to C. Since then, Feinstein's bound has been refined and improved by Fano (1961), Gallager (1965), Shannon, Gallager, and Berlekamp (1967), and others. Considerable interest has centered on the *reliability function* $E(R,L)$, which we would like to define as

$$E(R,L) = \lim_{N\to\infty} -\frac{1}{N} \ln P_e(N,[L \exp RN],L) \tag{13.81}$$

Unfortunately, $E(R,L)$ cannot be rigorously defined by Eq. (13.81), for no one has yet been able to prove that the limit exists! Nevertheless, we shall use Eq. (13.81) as a *heuristic definition*, with the tacit understanding that all upper bounds on $E(R,L)$ are actually upper bounds on the lim sup, and all lower bounds on $E(R,L)$ are actually lower bounds on the lim inf.

The function $E(R,\infty)$, defined as $\lim_{L\to\infty} E(R,L)$, is known exactly at all rates, in the interval $R_\infty < R < C$, where R_∞ is a known rate below which $E(R,\infty) = \infty$. The known function $E(R,\infty)$, called the *sphere-packing bound*, serves as an upper bound to $E(R,L)$ for finite L. For sufficiently large R, this bound is attained. In fact, there exists a known sequence of critical rates, $0 \leq R_\infty < \cdots < R_2 < R_1 < R_0 = C$, such that $E(R,L) = E(R,\infty)$ for all R in the interval $R_L < R < C$. For rates in the interval $R_1 < R < C$, $E(R,L)$ is independent of L. However, it is known that $E(R,L)$ does depend on L at sufficiently low rates.

The function $E(R,1)$ is known exactly for all rates only for a few pathological channels. For most channels, the tightest known bounds on $E(R,1)$ differ for all R in the interval $0 < R < R_1$, although $E(R_1,1)$ and $E(0,1)$, defined as $E(0,1) = \lim_{R\to 0} E(R,1)$, are known. At very low rates, the error probability is related to the numbers E_M, defined by $E_M = \lim_{N\to\infty} -(1/N) \ln P_e(N,M,1)$. In contrast to the limit of Eq. (13.81), this limit is known to exist. The numbers E_M are not known exactly, except for a few simple channels including the binary symmetric channel. However, there is a simple known general expression for their limit, $E_\infty = \lim_{M\to\infty} E_M$. In fact, $E_\infty = E(0,1)$, but the only known proof of this fact is very involved.

All evidence indicates that $E(R,L)$ must be a convex function of R, although this has never been proved. The best known upper and lower bounds to $E(R,L)$ are both convex, and $E(R,\infty)$ is convex. It is also known that if $f(R,L)$ is an upper bound to $E(R,L)$, then the convex hull of $f(R,L)$ and $E(R,\infty)$ is also an upper bound to $E(R,L)$.

To indicate the connections between the asymptotic error bounds and the asymptotic distance bounds of Fig. 13.2, we introduce the function $e(R)$ with the heuristic definition

$$e(R) = \lim_{n\to\infty} \frac{d(n,[\exp Rn])}{2n} \tag{13.82}$$

where $d(n,M)$ is the minimum distance of the binary code which has the maximum minimum distance among all binary codes with M codewords of block length n. As in Definition 13.81, we interpret the limit in (13.82) to be lim sup or lim inf, according to the usage.

The function $E(R,L)$ depends implicitly on the channel. Let $E(R,L;P)$ be $E(R,L)$ for a binary symmetric channel with crossover probability P. Then

$$\lim_{P\to 0} \frac{E(R,1;P)}{-\ln P} = e(R)$$

This theorem shows that the asymptotic distance ratio plotted in Fig. 13.2 is actually a case of a reliability function. Indeed, as P goes to zero, $-E(R,\infty;P)/\ln P$ becomes the Hamming volume bound on $e(R)$; the zero rate point $-E(0,1;P)/\ln P$ becomes the Plotkin bound on $e(R)$; the Elias bound on $-E(R,1;P)/\ln P$ becomes the Elias bound on $e(R)$; the "expurgated" lower bound on $-E(R,1;P)/\ln P$ becomes the Gilbert bound on $e(R)$. It is often conjectured that the function $e(R)$ actually lies at the Gilbert bound. This is equivalent to the conjecture that $E(R,1)$ coincides with the expurgated bound for the binary symmetric channel. Both of these conjectures have been open for several years, and both appear to be very difficult.

Although the conventional function $e(R)$ is known only to within the bounds of Fig. 13.2, the corresponding function is known exactly if one allows noiseless, delayless feedback. This function is shown in Fig. 13.6; the details are indicated by Berlekamp (1964*a*, 1968*b*). Since the function of Fig. 13.6 exceeds the Elias bound of Fig. 13.2, the use of noiseless feedback enables one to obtain exponentially reduced error probabilities at all rates below capacity on sufficiently quiet binary symmetric channels, even though Shannon (1956) has shown that a noiseless, delayless feedback link cannot increase the forward capacity of any discrete memoryless channel.

The best known asymptotic bounds on $e(R)$, due to Gilbert and Elias, apply to both linear and nonlinear codes. It is conjectured that nonlinear codes are asymptotically no better than linear codes, but this has not yet been proved. For certain finite cases, Green (1966), Julin (1965), Nadler (1962*a*, *b*), and Nordstrom and Robinson (1967) have discovered nonlinear codes which are better than any linear code. Of special interest is Nordstrom's nonlinear "quadratic" code of length 15 over GF(2), which has eight information digits denoted by $X_0, X_1, \ldots, X_7$ and seven redundant digits denoted by $Y_0, Y_1, \ldots, Y_6$. The first check digit, Y_0, is defined by

$$\begin{aligned} Y_0 = {} & X_0 + X_1 + X_3 + X_6 + X_7 \\ & + (X_0 + X_4)(X_1 + X_2 + X_3 + X_5) \\ & + (X_1 + X_2)(X_3 + X_5) \end{aligned} \tag{13.83}$$

The remaining Y's are found by cyclically shifting X_0 through X_6; that is, to compute Y_j, substitute $X_{i+j(\text{mod } 7)}$ for X_i in Eq. (13.83).

Preparata (1968) and Reddy (1968) have shown that this code has distance 5. This code is of special interest because it has been shown that no binary code with $n = 15, d = 5$ can have more than 2^8 codewords and Wagner (1965) has shown that the best *linear* code with $n = 15, d = 5$ has only 2^7 codewords. There are indications that definitions similar to Eq. (13.83) might be used to construct other good nonlinear codes, but no general construction has yet been found.†

A great number of papers have been devoted to the determination of the function $w(n,k)$, the maximum minimum weight of any linear binary code of block length n and rate $R = k/n$. Many techniques have been presented for determining $w(n,k)$ for certain small values of n and k. Lower bounds on $w(n,k)$ may be obtained either by exhibiting specific good codes, or by using the Varsharmov (1957) and Sacks (1958) modifications of the Gilbert bound. Upper bounds on $w(n,k)$ have been derived by a large variety of arguments. For small $n - k$, the volume bound and its quasi-perfect variation yield tight results. For small k,

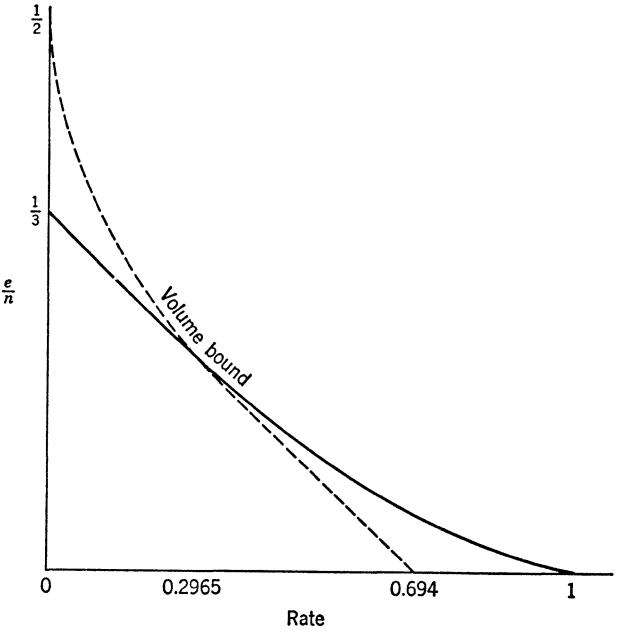


Fig. 13.6 The asymptotic error correction capability of the binary symmetric channel with feedback (solid line).

† A construction for $d = 5$, $n = 4^{j-1}$ was recently given by Preparata (1968): "A class of optimum nonlinear double-error correcting codes." Inform. Control 13, pp. 378–400.

MacDonald (1960) and Griesmer (1960) have obtained tight bounds by certain modifications of Plotkin's (1960) arguments. Solomon and Stiffler (1965) have exhibited a class of low-rate codes which attain the Greismer bound. An introduction to Solomon-Stiffler codes is given in Sec. 14.2.

For many values of n and k, the tightest upper bound on $w(n,k)$ is obtained by a technique introduced by Johnson (1962, 1963). In some cases, a bound introduced by Wax (1959) proves helpful. Calabi and Myrvaagnes (1964) have obtained certain results on the possible local variations of $w(n,k)$ for small changes in n and k. They have also collected a table of the known values of $w(n,k)$. Some entries in their table have been improved by Wagner (1965).

As newer and faster computers appear on the market, it seems probable that some programmers will continue to find gainful employment by computing $w(n,k)$ for yet another pair of n and k. But as long as the function $e(R)$ remains unknown, little can be said about $w(n,k)$ for large values of n and k.

14
Codes Derived by Modifying or Combining Other Codes

In many applications, the powers that be may dictate peculiar specifications within which the coding engineer must operate. For example, they may require that the block length n or the number of information digits k be set at some particular value other than those which the code designer would prefer. Although the coding theorist may find such constraints arbitrary and illogical, it may be impossible to change them. These constraints may have already been built into some other part of the communications system. Rather than redesign the entire system from scratch, it usually proves easier to remodel the code.

There are six basic ways in which codes can be modified, as illustrated in Fig. 14.1. The block length of the code can be increased by annexing more check digits or more message digits; the block length can be decreased by omitting check digits or omitting message digits; the block length may remain constant while the number of codewords is increased or decreased. I have chosen the respective words *extend, lengthen, puncture, shorten, augment,* and *expurgate* for these six basic

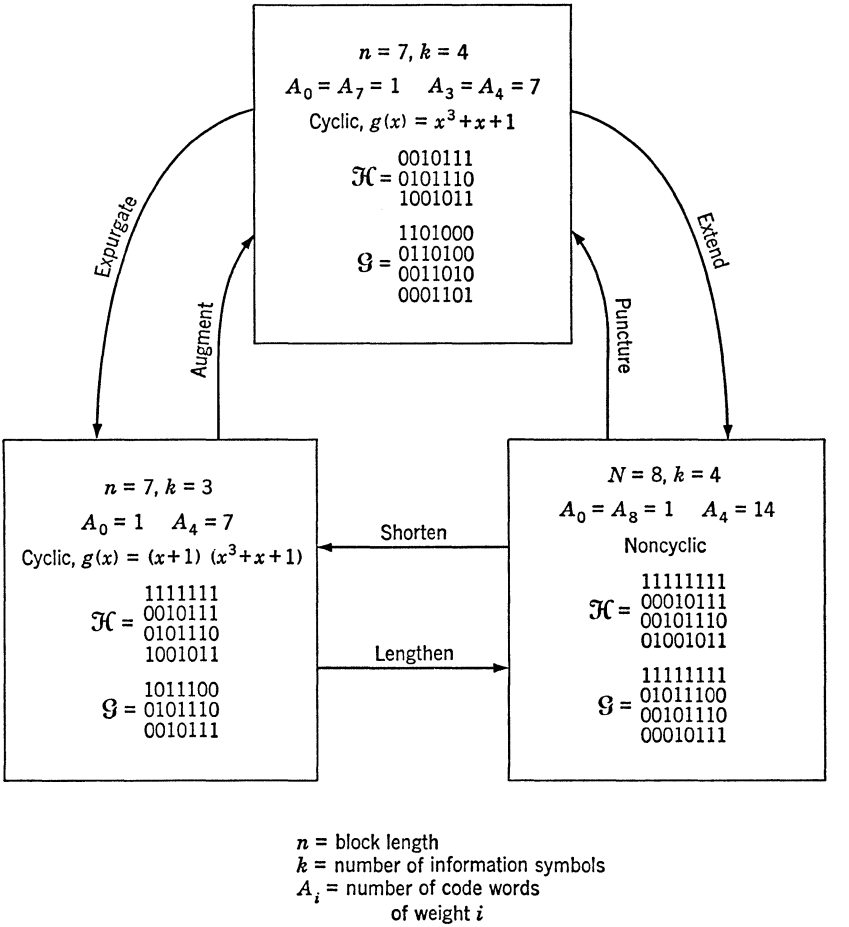


Fig. 14.1 Simple illustration of the terms extend, puncture, shorten, lengthen, expurgate, and augment. Here $\mathcal{G}$ is a matrix whose row space is the set of codewords. Such a matrix is called a *generator matrix* of the code.

types of modifications. The terms extend, puncture, shorten, and expurgate are already common in the literature, and lengthen is the obvious opposite of shorten. Unfortunately, the use of these words is not yet uniform in the literature; Kasami, Lin, and Peterson have used the term *modify* instead of the Solomon-Stiffler term *puncture*, which I use here. I prefer to use *modify* as a general term which includes all six basic modifications.

14.1 EXTENDING A CODE BY ANNEXING MORE CHECK DIGITS

Any code can be extended by annexing additional check digits, although the minimum weight of the extended code will not be greater than the minimum weight of the original code unless the extending check digits are carefully chosen. Annexing an overall parity check is the most common method by which an original code of block length n can be extended to a new code of block length $N = n + 1$. The weight of every codeword in an extended binary code must be even. If the original code is a linear binary code with odd minimum weight, then the minimum weight of the extended code is greater than the minimum weight of the original code.

The extended code's overall parity check C_∞ is equal to the negative of the remainder obtained by dividing the original codeword, $C(x) = \sum_{i=0}^{n-1} C_i x^i$, by the polynomial $x - 1$. In some cases, it may be advantageous to extend the code even further by annexing additional digits, equal to the remainder obtained by dividing the original codeword by certain other polynomials. For example, Andryanov and Saskovets (1966) have suggested that an original cyclic binary BCH code of block length n and designed distance d may be *multiply extended*† to a new code of block length $n + 1 + \deg M^{(d)}$. After transmitting the original codeword of length n, the encoder also transmits the remainder obtained by dividing the original codeword by $x + 1$ and the remainder obtained by dividing the original codeword by $M^{(d)}(x)$. The multiply extended code has distance $\geq d + 2$ because any pattern of no more than $(d + 1)/2$ channel errors may be correctly decoded by the following procedure.

First, the decoder attempts to decode the first $n + 1$ received digits, ignoring the last $\deg M^{(d)}$ received check digits. If these first $n + 1$ digits contain no more than $(d - 1)/2$ channel errors, this decoding attempt succeeds, and the extra check digits (including whatever errors they may contain) may be ignored. If the first $n + 1$ digits contain exactly $(d + 1)/2$ channel errors, then the initial decoding attempt fails. The decoder then assumes that the last $\deg M^{(d)}$ received check digits are correct. From these correct check digits, he may determine S_d, the sum of the dth powers of the error locations among the code's first $n + 1$ positions. With this additional information, the decoder may then correct any pattern of $(d + 1)/2$ channel errors in the code's first $n + 1$ positions by straightforward methods, as described in Chaps. 7 and 10.

† Here "multiply" is the adverbial opposite of "singly," not the verb denoting an arithmetic operation.

14.2 PUNCTURING A CODE BY DELETING CHECK DIGITS

Any code can be punctured by deleting some of its check digits, although the minimum weight of the punctured code will be considerably less than the minimum weight of the original code unless the punctured digits are carefully chosen. In general, the weight of each codeword in the original code will be reduced by its weight in the punctured positions. Thus, if one punctures p digits from a linear binary code of length n and minimum weight d, then the punctured code may have minimum weight as low as $d - p$. The punctured code will have minimum weight greater than $d - p$ iff no codeword of weight d in the original code has ones in all the punctured positions.

Solomon and Stiffler (1965) have shown that good linear codes of sufficiently low rates may be obtained by puncturing subspaces of a maximal-length shift-register code. For example, consider the maximal-length shift-register binary code of block length 31, whose generator matrix is given by

$$
\mathcal{G} = \begin{array}{c}
\begin{array}{ccccccccccccccccccccccccccccc}
X & X & Y & & & & & & Y & & & & & & & Y & Y & & X & & & & & & & & & Y & Y & Y
\end{array} \\
\begin{bmatrix}
0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 \\
0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 \\
0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\
0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\
1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0
\end{bmatrix}
\end{array}
\tag{14.21}
$$

The 31 columns of this matrix and the **0** column form an additive vector space of dimension 5 over GF(2). The set consisting of the three X positions and **0** forms a subspace of dimension 2; the set consisting of the seven Y positions and **0** forms a subspace of dimension 3. Every codeword of the original code has either zero 1s or two 1s among the three X positions and zero 1s or four 1s among the seven Y positions. In general, any codeword in a maximal-length shift-register code of length $2^k - 1$ has either zero 1s or 2^{l-1} 1s among the $2^l - 1$ positions corresponding to the nonzero points in a subspace of dimension l. Thus, if we puncture from the maximal-length shift-register code the points corresponding to linearly independent subspaces of dimensions $l_1, l_2, l_3, \ldots$, where

$$\Sigma l_i \leq k \tag{14.22}$$

then we change the block length and minimum distance as follows:

$$
\begin{aligned}
n &= 2^k - 1 \qquad n' = 2^k - 1 - \Sigma(2^{l_i} - 1) \\
d &= 2^{k-1} \qquad d' = 2^{k-1} - \Sigma 2^{l_i - 1}
\end{aligned}
\tag{14.23}
$$

For example, if we puncture the X and Y positions from the matrix of Eq. (14.21), we obtain the punctured code whose generator matrix is

given by

$$\mathcal{G}' = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix} \tag{14.24}$$

The punctured code generated by the matrix of Eq. (14.24) has length 21 and minimum distance 10.

Using the Griesmer bound, Solomon and Stiffler have shown that every linear code obtained by puncturing subspaces of different dimensions from an original maximal-length shift-register code is optimum in the sense that no linear code with the same number of codewords and the same minimum distance can have shorter length.

Many optimum codes can be obtained by the Solomon-Stiffler puncturing procedure, but the constraint of Eq. (14.22) restricts all these codes to low rates or short block lengths or both.

14.3 AUGMENTING ADDITIONAL CODEWORDS TO THE CODE

A cyclic code generated by the polynomial $g(x)$ may be augmented into any other cyclic code of the same length whose generator polynomial is a factor of $g(x)$. The most common augmented code is the one generated by $g(x)/(x - 1)$, as illustrated in Fig. 14.1.

According to the Gilbert bound (Lemma 13.72 and Theorem 13.73) and the known asymptotic properties of long BCH codes (Sec. 12.7), a large number of codewords may be augmented to any sufficiently long BCH code of moderate rate to form an augmented linear code with the same minimum distance. Unfortunately, virtually nothing is known about how to go about finding these additional codewords.

14.4 EXPURGATING CODEWORDS FROM THE CODE

A cyclic code generated by the polynomial $g(x)$ can be expurgated to form another cyclic code by multiplying any additional factor into the generator polynomial. The most common expurgated code is the one generated by $g(x)(x - 1)$. In the binary case, the codewords of this expurgated code are the even-weight codewords of the original code. The minimum weight of the expurgated code is often one greater than the minimum weight of the original code, but in some cases the minimum weight of the expurgated code is equal to the minimum weight of the original code, and in some cases the minimum weight of the expurgated code exceeds the minimum weight of the original code by more than one.

All of these phenomena occur among the cyclic binary codes of length 15 whose weight enumerators are given in Table 16.1.

Shannon (1959) and Gallager (1965) have derived reliability bounds (see Sec. 13.8) by expurgating all codewords of sufficiently low weight from a randomly chosen code. At low rates, these bounds are among the best known; at zero rate, they give the exact reliability. Like the technique of augmentation, which has also proved very useful in deriving nonconstructive bounds, the technique of expurgation has not yet been used constructively in algebraic coding theory in any nontrivial way.

14.5 LENGTHENING THE CODE BY ANNEXING MORE MESSAGE DIGITS

If the generator polynomial of an original binary cyclic code of length n contains the factor $x + 1$, then this cyclic code can be lengthened to a linear code of length $N = n + 1$. The lengthened code is obtained by annexing the all-one vector of length N to the code's generator matrix. As seen in Fig. 14.1, this lengthened code is identical to the code obtained by first augmenting the original cyclic code into the cyclic code generated by $g(x)/(x + 1)$ and then extending this augmented code by annexing an overall parity check. In most cases, the minimum weight of the lengthened code is identical to the minimum weight of the original code. In some cases, however, the minimum weight of the lengthened code is less than the minimum weight of the original code.

Although it is not difficult to formulate modified versions of the Gilbert bound which assure that any sufficiently long poor code of moderate rate may be lengthened into a better code, the technique of lengthening has not yet been used in algebraic coding theory in any nontrivial way.

14.6 SHORTENING THE CODE BY OMITTING MESSAGE DIGITS

Any code can be shortened by deleting message digits, although the minimum distance of the shortened code will be no greater than the minimum distance of the original code unless the message digits to be deleted are carefully chosen. From a theoretical standpoint, little is known about how to choose these message digits wisely. In practice, a cyclic code is usually shortened by deleting a sequence of consecutive message digits, since this leads to the simplest implementation.

14.7 SUBFIELD SUBCODES

Definition 14.71 If $\mathcal{C}$ is a code of length n over GF(p^s), where p is any prime power† and s is any positive integer, then the subcode of $\mathcal{C}$ consisting of those words all of whose coefficients lie in GF(p) is called the *subfield subcode of* $\mathcal{C}$ *over GF*(p).

† The case of principal interest is $p = 2$.

Properties

14.72. A subfield subcode of a linear code is linear.

14.73. Any permutation group which preserves $\mathcal{A}$ preserves any subfield subcode of $\mathcal{A}$.

14.74. If $\mathcal{A}$ is a cyclic code of length n over GF(p^s) with generator polynomial

$$g(x) = \prod_{k \in K} (x - \alpha^k) \tag{14.741}$$

where α is a primitive nth root of unity and K is a set of integers mod n, then the subfield subcode of $\mathcal{A}$ over GF(p) is cyclic, with generator polynomial

$$\check{g}(x) = \prod_{k \in \check{K}} (x - \alpha^k) \tag{14.742}$$

where $k \in \check{K}$ iff there exists an integer $i(k)$ for which $kp^i \in K$.

Proofs Let $\mathcal{P}$ be the set of n-dimensional vectors over GF(p). Then $\mathcal{B}$, the subfield subcode of $\mathcal{A}$ over GF(p), is given by $\mathcal{B} = \mathcal{A} \cap \mathcal{P}$. With this notation, we may now prove the claims.

14.72. $\mathbf{u},\mathbf{v} \in \mathcal{A} \Rightarrow \mathbf{u} \pm \mathbf{v} \in \mathcal{A}$
$\mathbf{u},\mathbf{v} \in \mathcal{P} \Rightarrow \mathbf{u} \pm \mathbf{v} \in \mathcal{P}$
$\mathbf{u},\mathbf{v} \in \mathcal{A} \cap \mathcal{P} \Rightarrow \mathbf{u} \pm \mathbf{v} \in \mathcal{A} \cap \mathcal{P}$

Therefore $\mathcal{B} = \mathcal{A} \cap \mathcal{P}$ is linear.

14.73. Let $\pi(\mathbf{u})$ be a permutation of $\mathbf{u}$, such that $\mathbf{u} \in \mathcal{A} \Rightarrow \pi(\mathbf{u}) \in \mathcal{A}$. Since π is a permutation, $\mathbf{u} \in \mathcal{P} \Rightarrow \pi(\mathbf{u}) \in \mathcal{P}$, so $\mathbf{u} \in \mathcal{B} \Rightarrow \pi(\mathbf{u}) \in \mathcal{B}$.

14.74. If $\mathcal{A}$ is a linear cyclic code, then $\mathcal{B}$ is a linear cyclic code by Properties 14.72 and 14.73, with π taken as the cyclic permutation. If we let the generator polynomials of $\mathcal{A}$ and $\mathcal{B}$ be given by Eqs. (14.741) and (14.742), then we need only show that $\check{K} = \bigcup_{i=0}^{\infty} p^i K$. Since $\mathcal{B} \subseteq \mathcal{A}$, $K \subseteq \check{K}$. Since $\mathcal{B} \subseteq \mathcal{P}$, $p\check{K} = \check{K}$, so $p^i K \subseteq \check{K}$ for all i, and

$$\check{K} \supseteq \bigcup_i p^i K = \bigcup_{i=0}^{\infty} p^i K \tag{14.743}$$

To show that $\check{K} = \bigcup_i p^i K$, we consider the polynomial

$$c(x) = \prod_{k \in \bigcup_i p^i K} (x - \alpha^k)$$

Since $\bigcup_i p^i K \supseteq K$, $c(x) \in \mathfrak{A}$; since $p\{\bigcup_i p^i K\} = \{\bigcup_i p^i K\}$, $c(x) \in \mathcal{P}$. Therefore, $c(x) \in \mathfrak{A} \cap \mathcal{P} = \mathfrak{B}$. Since every root of $\check{g}(x)$ must also be a root of $c(x)$,

$$\check{K} \subseteq \bigcup_i p^i K \tag{14.744}$$

From Eqs. (14.744) and (14.743), $\check{K} = \bigcup_i p^i K$. Q.E.D.

If we let $\bar{K}$ and $\check{\bar{K}}$ denote the complements of K and $\check{K}$, respectively, that is, $K \cup \bar{K} = \check{K} \cup \check{\bar{K}} = \{0, 1, 2, \ldots, n-1\}$ and $K \cap \bar{K} = \check{K} \cap \check{\bar{K}} = \emptyset$, then

$$\check{\bar{K}} = \bigcap_i p^i \bar{K}$$

In other words, α^k is a root of the check polynomial of the subfield subcode of $\mathfrak{A}$ over GF(p) iff α^k, α^{kp}, α^{kp^2}, $\ldots$, $\alpha^{kp^{s-1}}$ are *all* roots of the check polynomial of $\mathfrak{A}$.

We shall consider a certain special class of subfield subcodes in greater detail in Sec. 15.5.

14.8 DIRECT-PRODUCT CODES AND THEIR RELATIVES

14.81 DIRECT-PRODUCT CODES†

In most circumstances, it is convenient to write the coefficients of a codeword of length n in a $1 \times n$ array, $\check{\mathbf{C}} = [\check{C}_0, \check{C}_1, \check{C}_2, \ldots, \check{C}_l, \ldots, \check{C}_{n-1}]$.‡ However, if n is a composite number, $n = n_1 \times n_2$, it is sometimes advantageous to write the coefficients of $\mathbf{C}$ in an $n_1 \times n_2$ array,

$$\begin{bmatrix} C_{0,0} & C_{0,1} & \cdots & C_{0,n_2-1} \\ C_{1,0} & C_{1,1} & \cdots & C_{1,n_2-1} \\ \cdots & \cdots & \cdots & \cdots \\ C_{n_1-1,0} & C_{n_1-1,1} & \cdots & C_{n_1-1,n_2-1} \end{bmatrix}$$

If the digits of this codeword are to be transmitted consecutively, then the one-dimensional nature of time forces us to identify the components of this $n_1 \times n_2$ array with the components of a $1 \times n$ array. In order to identify $C_{i,j}$ with $\check{C}_l$, we must define i and j as functions of l. This can be done in various ways. The *canonical ordering* is obtained by choosing i and j as the quotient and remainder when l is divided by n_2. This

† Some authors use the equivalent term, *Kronecker product.*

‡ Throughout Sec. 14.8, I have used the accent ˇ for one-dimensional codewords and their components, to avoid confusion with the two-dimensional representation.

ordering gives the array

$$\begin{bmatrix} \check{C}_0 & \check{C}_1 & \cdots & \check{C}_{n_2-1} \\ \check{C}_{n_2} & \check{C}_{n_2+1} & \cdots & \check{C}_{2n_2-1} \\ \cdots & \cdots & \cdots & \cdots \\ \check{C}_{(n_1-1)n_2} & \check{C}_{(n_1-1)n_2+1} & \cdots & \check{C}_{n-1} \end{bmatrix}$$

The *cyclic ordering*, introduced by Burton and Weldon (1965), is defined by

$$l \equiv i \bmod n_1 \quad \text{and} \quad l \equiv j \bmod n_2 \tag{14.811}$$

For example, if $n_1 = 3$ and $n_2 = 7$, we have

$$\begin{bmatrix} \check{C}_0 & \check{C}_{15} & \check{C}_9 & \check{C}_3 & \check{C}_{18} & \check{C}_{12} & \check{C}_6 \\ \check{C}_7 & \check{C}_1 & \check{C}_{16} & \check{C}_{10} & \check{C}_4 & \check{C}_{19} & \check{C}_{13} \\ \check{C}_{14} & \check{C}_8 & \check{C}_2 & \check{C}_{17} & \check{C}_{11} & \check{C}_5 & \check{C}_{20} \end{bmatrix}$$

If n_1 and n_2 are relatively prime, the Chinese remainder theorem 2.16 assures us that the cyclic ordering maps only one value of l, $0 \leq l < n$, into any given pair (i,j), $0 \leq i < n_1$, $0 \leq j < n_2$. If n_1 and n_2 are not relatively prime, the cyclic ordering is undefined.

Of the many possible two-dimensional codes, the class which has been most extensively studied is the class of direct-product codes, first introduced by Elias (1954).

Definition 14.812 If $\mathfrak{A}$ is a linear code of length n_1 and $\mathfrak{B}$ is a linear code of length n_2, the *direct product* of codes $\mathfrak{A}$ and $\mathfrak{B}$ denoted by $\mathfrak{A} \times \mathfrak{B}$ is the two-dimensional code consisting of all length $n_1 n_2$ codewords which satisfy the following two conditions:

1. For all j, $[C_{0,j}, C_{1,j}, C_{2,j}, \ldots, C_{n_1-1,j}] \in \mathfrak{A}$.
2. For all i, $[C_{i,0}, C_{i,1}, C_{i,2}, \ldots, C_{i,n_2-1}] \in \mathfrak{B}$.

The following two properties of direct-product codes can be verified without difficulty:†

$$\dim(\mathfrak{A}\mathfrak{B}) = (\dim \mathfrak{A})(\dim \mathfrak{B}) \tag{14.813}$$

$$\text{Minimum distance } (\mathfrak{A} \times \mathfrak{B}) = (\text{minimum distance } \mathfrak{A})(\text{minimum distance } \mathfrak{B}) \tag{14.814}$$

The most obvious way in which to decode a canonically ordered direct-product code is to decode the component codes one at a time, as shown in Fig. 14.2. The first stage of the decoder consists of a decoder

† Here "dim" is an abbreviation for dimension. The *dimension* of a code is the number of its information symbols.

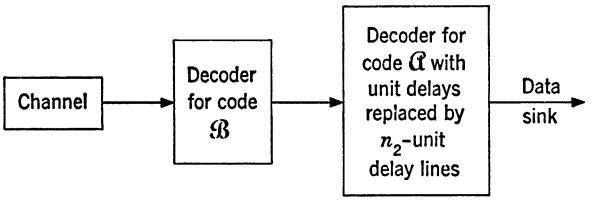


Fig. 14.2 Cascaded decoder for direct-product code transmitted in the canonical order.

for code $\mathcal{B}$. The second stage consists of a decoder for code $\mathcal{A}$, with every unit delay replaced by an n_2-unit delay line. This enables the second stage to decode all n_2 columns in phase. At any given instant of time, all the digits emerging from the delay lines in the second stage belong to a single column of the code. The first stage decodes each row of the code; the second stage decodes each column of the code. Even though the digits transmitted from the channel to the first stage may contain many errors, the digits transmitted from the first stage of the decoder to the second stage will (hopefully) contain fewer errors, and the digits transmitted from the second stage to the data sink will contain virtually no errors at all.

For memoryless channels, this simple decoding procedure leaves much to be desired. It fails to decode many of the error patterns which the code is capable of correcting. If the component codes have odd minimum distances d_1 and d_2, then there are error patterns of weight $(d_1 + 1)(d_2 + 1)/4$ which are incorrectly decoded by this decoding procedure, even though the code is capable of correcting all patterns of $(d_1d_2 - 1)/2$ or fewer errors.

However, the cascaded decoding procedure for direct-product codes provides an effective method for obtaining reliable communication across channels whose errors tend to occur in bursts, since most bursts will not affect too many rows of the code. If the direct-product code is transmitted in the cyclic order, then a burst will be distributed both among the code's rows and among the code's columns. For this reason, the cyclic order often proves preferable to the canonical order. Another interesting property of the cyclic order is discussed in Sec. 14.82.

14.82 CYCLIC PRODUCT CODES

In general, the direct product of two cyclic codes may not be cyclic. However, if the block lengths of the two original codes are relatively prime, and if the digits of the product code are ordered according to Eq. (14.811), then the product code is cyclic. This is the essence of Theorem 14.821.

Theorem 14.821 [Burton-Weldon (1965)] *If*

$n_1p_1 + n_2p_2 \equiv 1 \bmod n_1n_2$
$\mathcal{A}$ = cyclic code of length n_1 generated by $a(x)$
$\mathcal{B}$ = cyclic code of length n_2 generated by $b(x)$

then the cyclic ordering of the product code $\mathcal{A} \times \mathcal{B}$ is the cyclic code of length n_1n_2 generated by

$$g(x) = \gcd(a(x^{p_2n_2})b(x^{p_1n_1}), x^{n_1n_2} - 1)$$

Proof As suggested by Elspas (1967), the components of a two-dimensional codeword **C** may be considered to be the coefficients of a bivariate polynomial

$$C(x,y) = \sum_{i=0}^{n_1-1} \sum_{j=0}^{n_2-1} C_{i,j}x^iy^j$$

On the other hand, the components of **C** may also be considered to be the coefficients of the univariate polynomial

$$\check{C}(z) = \sum_{l=0}^{n_1n_2-1} \check{C}_l z^l$$

where $\check{C}_l$ and $C_{i,j}$ are related by the cyclic ordering of Eq. (14.811).

Let α be a primitive (n_1n_2)th root of unity; let $\beta = \alpha^{n_2}$; let $\gamma = \alpha^{n_1}$. Then we assert that the following statements are logically equivalent.

14.822. $\mathbf{C} \in \mathcal{A} \times \mathcal{B}$
14.823. $a(x)b(y) \mid C(x,y)$
14.824. $a(\beta^I)b(\gamma^J) = 0$ implies $C(\beta^I,\gamma^J) = 0$
14.825. $g(\beta^I\gamma^J) = 0$ implies $\check{C}(\beta^I\gamma^J) = 0$
14.826. $g(z) \mid C(z)$

The equivalence of Statements 14.822 and 14.823 follows from Definition 14.812. Statements 14.823 and 14.824 are equivalent because every root of $a(x)$ is a power of β and every root $b(y)$ is a power of γ.

With l defined by Eq. (14.811),

$$iIn_2 + jJn_1 \equiv lIn_2 + lJn_1 \bmod n_1n_2$$

We have

$$\begin{aligned} C(\beta^I,\gamma^J) &= \sum_i \sum_j C_{i,j}\alpha^{iIn_2+jJn_1} \\ &= \sum_{l=0}^{n_1n_2-1} \check{C}_l\alpha^{l(In_2+Jn_1)} \end{aligned}$$

So

$$C(\beta^I,\gamma^J) = \check{C}(\beta^I\gamma^J) \tag{14.827}$$

From the definition of $g(x)$,

$$g(\beta^I\gamma^J) = a(\beta^I)b(\gamma^J) \tag{14.828}$$

The equivalence of Statements 14.824 and 14.825 follows from Eqs. (14.827) and (14.828). Finally, the equivalence of Statements 14.825 and 14.826 follows from the fact that every root of $g(x)$ is an (n_1n_2)th root of unity, and every (n_1n_2)th root of unity can be written in the form $\beta^I\gamma^J$ for appropriate values of I and J.

Q.E.D.

Corollary 14.829 In the notation of Theorem 14.821

$$g(x) = \text{lcm } (d(x^{n_2}),f(x^{n_1}))$$

where $d(x) = \text{gcd } (a(x^{p_2}),x^{n_1}-1)$
$f(x) = \text{gcd } (b(x^{p_1}),x^{n_2}-1)$

Proof

$$\text{lcm } (d(\beta^{In_2}\gamma^{Jn_2}), f(\beta^{In_1}\gamma^{Jn_1})) = a(\beta^I)b(\gamma^J)$$ Q.E.D.

Let $\mathfrak{D}$ and $\mathfrak{F}$ denote the cyclic codes generated by $d(x)$ and $f(x)$. According to Sec. 5.8, $\mathfrak{D}$ and $\mathfrak{F}$ are permutations of $\mathfrak{A}$ and $\mathfrak{B}$; therefore, deg $d(x)$ = deg $a(x)$ and deg $f(x)$ = deg $b(x)$.

Because the cyclic ordering differs from the canonical ordering, the general outline of a decoder for a cyclic product code differs slightly from the cascaded decoder shown in Fig. 14.2. After $C_{0,0}$, the next digit in the first row of the code to be received from the channel is C_{0,n_1}, then $C_{0,2n_1}$, ... Therefore, as Abramson (1968) has observed, the first stage of the cascaded decoder should consist of a decoder for code $\mathfrak{F}$, with each unit delay replaced by an n_1-unit delay line. Similarly, the second stage of the cascaded decoder should consist of a decoder for code $\mathfrak{D}$, with each unit delay replaced by an n_2-unit delay line.

Theorem 14.821 and Corollary 14.829 can be used to determine whether or not it is possible to decompose a given code of length n_1n_2 into the product of a cyclic code of length n_1 and another cyclic code of length n_2. One need only write all the roots of the original generator polynomial in the form (I,J), examine the (I,J) pairs which occur, and apply Theorem 14.821 or Corollary 14.829 if possible. Many codes

which are not decomposable are subcodes of larger codes which are decomposable. Goethals (1967) has considered this decomposition problem from a more abstract viewpoint.

14.83 EXPECTED NUMBER OF ERRONEOUSLY DECODED DIGITS IN A HAMMING CODE

In order to evaluate the performance of the direct product of a Hamming code and another code, we must first calculate the expected number of erroneously decoded digits when the extended Hamming code of length 2^m is used on a memoryless binary symmetric channel with error probability P.

If there is an even number of channel errors among the 2^m digits in a given block, the Hamming decoder makes no changes; if there is an odd number of channel errors in this block, the decoder changes one digit. If there is only one channel error, this change will correct it. However, if there are several errors, the change may correct one of them, or it may insert another. The decoder will correct an error iff the channel-error pattern has weight $i + 1$ and lies at distance 1 from a codeword of weight i; it will insert an additional error iff the channel-error pattern has weight $i - 1$ and lies at distance 1 from a codeword of weight i.

Let $A_i^{(m)}$ be the number of codewords of weight i in the extended Hamming code of length 2^m, and let $A^{(m)}(z) = \sum_i A_i^{(m)} z^i$. Of the 2^m words which lie at distance 1 from any given codeword of weight i, i have weight $i - 1$, and $2^m - i$ have weight $i + 1$. Therefore, the enumerator for error patterns which will cause the decoder to insert an additional error is $\sum_i i A_i^{(m)} z^i = z A^{(m)\prime}(z)$, and the enumerator for error patterns which cause the decoder to correct an error is $\sum_i (2^m - i) A_i^{(m)} z^{i+1} = 2^m z A^{(m)}(z) - z^2 A^{(m)\prime}(z)$.

If the channel-error probability is P, then the expected number of channel errors is $e = 2^m P$. The probability that the decoder inserts an additional error is given by $\sum_i i A_i^{(m)} P^{i-1}(1 - P)^{2^m - i + 1} = (1 - P)^{2^m} \times A^{(m)\prime}(x)$, where $x = P/(1-P)$. Similarly, the probability that the Hamming decoder corrects an error is given by $(1 - P)^{2^m}[2^m x A^{(m)}(x) - x^2 A^{(m)\prime}(x)]$. From these probabilities of error correction and error insertion, we can calculate ϵ, the expected number of digit errors per block after decoding. The result, expressed in terms of e, the expected number of channel errors, is

$$\epsilon = e - (1 - e2^{-m})^{2m}\left[2^m x A^{(m)}(x) - (1 - x^2) A^{(m)\prime}(x)\right] \qquad (14.831)$$

where $x = e2^{-m}/(1 - e2^{-m})$.

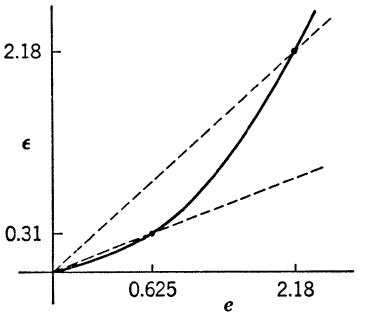


Fig. 14.3 Relationship between the expected number of digit errors/block before and after decoding a long extended binary Hamming code.

A formula for $A^{(m)}(z)$ will be derived in Sec. 16.2. I will not give this formula explicitly here, because I do not wish to distract the reader with the cumbersome details of computations which he cannot easily check without a computer.

Although the functional relationship between ϵ and e depends on m, this function approaches a limit as m goes to infinity. A graph of this limiting relationship between ϵ and e is shown in Fig. 14.3. Two important points on this graph are worthy of mention

$$\epsilon < e \qquad \text{iff } e < 2.1779 \tag{14.832}$$

$$\epsilon < \frac{e}{2} \qquad \text{iff } e < 0.6246 \tag{14.833}$$

According to (14.832), the decoder is more likely to insert an additional error than to correct an error if $2.1779 < e \ll 2^m$. This should not be too surprising, since the single-error-correcting decoder will change any channel-error pattern of weight 3 into a codeword of weight 4.

14.84 THE ELIAS CODES

The definition of direct product can be readily extended to any number of component codes. For example, the direct product of $\mathfrak{A}$, $\mathfrak{B}$, and $\mathfrak{C}$ might be defined as $(\mathfrak{A} \times \mathfrak{B}) \times \mathfrak{C}$.

Following an ingenious construction first introduced by Elias (1954), we now consider the direct product of the extended binary Hamming codes of lengths 2^i, 2^{i+1}, 2^{i+2}, ... Since the rate of a product code is the product of the rates of its component codes, the rate of this infinite direct product of extended Hamming codes is

$$R_i = \prod_{m=i}^{\infty} [1 - (m+1)2^{-m}]$$

It is easily seen that this product converges to a positive limit, because

$$\sum_{m=i+1}^{\infty} m2^{-(m-1)} = \left[\frac{d}{dz}\sum_{m=i+1}^{\infty} z^m\right]_{z=1/2} = \frac{i+2}{2^{i-1}} < \infty$$

The approximate values of the converging product are $R_8 = 0.92$, $R_7 = 0.86$, $R_6 = 0.77$, $R_5 = 0.63$, $R_4 = 0.43$, $R_3 = 0.21$, $R_2 = 0.05$.

The infinite direct-product code may be decoded by an infinite cascade of decoders, as shown in Fig. 14.4. The first stage consists of a decoder for the extended Hamming code of length 2^i; the second stage consists of a decoder for the extended Hamming code of length 2^{i+1}; . . . .

Let ϵ_m denote the expected number of digit errors per block at the output of the decoder of the Hamming code of length 2^m. Then the probability of error per digit is $\epsilon_m/2^m$, and the expected number of errors in a block of length 2^{m+1} is $e_{m+1} = 2^{m+1}\epsilon_m/2^m = 2\epsilon_m$. The locations of these errors are not statistically independent, for there is an even number of them within any block of the extended Hamming code of length 2^m. Furthermore, it is likely that these errors form a codeword. However, the errors in each block of this Hamming code are statistically independent of the errors in any other block of that code. Therefore, the errors within a block of 2^{m+1} digits which are the input to the next larger Hamming code are statistically independent. For this reason, we may use Eq. (14.831) to compute the error probability per digit after every stage of the cascaded decoder.

From (14.833) we deduce that if $e_m < 0.6246$ for any sufficiently large m, then $e_{m+1} = 2\epsilon_m < e_m$. In this case each successive Hamming decoder in the cascade reduces the expected number of errors in every block that it decodes by a factor of 2 or more. Even though the block sizes of the successive Hamming codes grow larger and larger, the expected number of errors per block approaches zero. On the other hand, if $0.6246 < e_m$ for some sufficiently large m, then $e_{m+1} > e_m$, and $e_{m+1} < e_{m+2} < e_{m+3} < \cdots$. Eventually, we shall find $e_{m+k} > 2.1779$

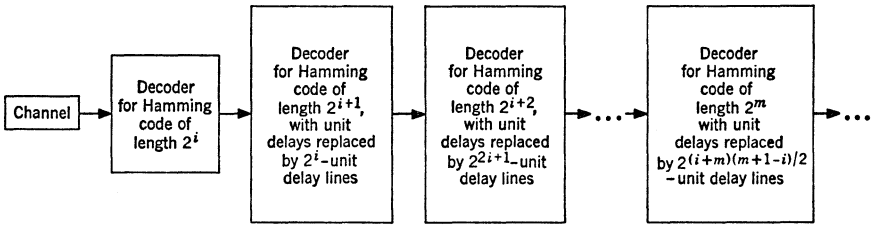


Fig. 14.4 Cascaded decoder for an Elias code.

for some k. When this happens, further decoding attempts will only increase the expected number of errors. Thus, for sufficiently large m, 0.6246 is the critical value of e_m. Working backward with the aid of Eq. (14.831) and a computer, I obtained the following critical values:

m	*Critical value of* e_m	*Critical value of* P_m
∞	0.6246	
11	0.6246	0.000305
10	0.6248	0.000610
9	0.6251	0.00122
8	0.6257	0.00244
7	0.6268	0.00489
6	0.6290	0.00983
5	0.6336	0.0192
4	0.6429	0.0402
3	0.6628	0.0828

According to these figures, the cascaded decoder for the infinite direct product of the extended Hamming codes of lengths 2^4, 2^5, . . . will achieve error-free decoding iff the channel-error probability per digit is ≤ 0.0402. If $P = 0.0402$, then the error probabilities per digit after successive stages of the cascaded decoder are 0.0192, 0.00983, ...

The Elias codes are the only explicitly known codes which achieve error-free coding on the one-way binary symmetric channel at positive rates.

14.85 TENSOR PRODUCT CODES

Any parity-check matrix with r rows and n columns over GF(q^m) may be expanded into a parity-check matrix with mr rows and n columns over GF(q), as shown in the example of Eq. (5.21), page 125, where $r = 2$, $q = 2$, $m = 4$, and $n = 15$. I call the code over GF(q) the *expansion* of the code over GF(q^m); I call the code over GF(q^m) the *contraction* of the code over GF(q). Every code over GF(q^m) has a unique expansion, but some codes over GF(q) may have several contractions over GF(q^m), particularly if the redundancy of the code over GF(q) is a nonmultiple of m.

If $\mathfrak{A}$ and $\mathfrak{B}$ are codes over GF(q^m) and $\mathfrak{B}$ is the expansion of $\mathfrak{C}$ over GF(q), then the *tensor product* $\mathfrak{A} \circledast \mathfrak{C}$ is the expansion of the dual of the direct product of the duals of $\mathfrak{A}$ and $\mathfrak{B}$.

If $\mathfrak{A}$ has length n_1 and redundancy r_1 and $\mathfrak{C}$ has length n_2 and redundancy r_2, then $\mathfrak{A} \circledast \mathfrak{C}$ has length $n_1 n_2$ and redundancy $r_1 r_2$. In practice, $\mathfrak{C}$ is usually taken to be a code with $r_2 = m$. $\mathfrak{A}$ is often taken to be an RS code. The decoder first decodes code $\mathfrak{A}$, treating the $r_1 m$

syndrome digits over GF(q) as r_1 syndrome digits over GF(q^m), and obtains n_1 "error values" in GF(q^m). Each "error value" is then taken as an r_2-digit syndrome over GF(q), and each such syndrome is used to decode a subblock of length n_2 to a codeword of code $\mathcal{C}$.

Although tensor product codes are not particularly attractive for use on one-way memoryless channels, Wolf (1965*a*) has shown that certain tensor product codes are able to correct multiple bursts of errors. Elspas and Wolf (1963) and Wolf (1965*b*) have suggested that certain tensor product codes might be used to locate subblocks within which errors occurred. In a two-way communication system, the receiver could then request the sender to retransmit the erroneously received subblocks.

14.86 DIRECT-SUM CODES

If $\mathcal{A}$ and $\mathcal{B}$ are two codes over the same field, then the *direct sum* $\mathcal{A} \oplus \mathcal{B}$ consists of the set of all words of the form $a * b$, where $a \in \mathcal{A}$, $b \in \mathcal{B}$, and $*$ represents concatenation, as in Chap. 12. The length of $\mathcal{A} \oplus \mathcal{B}$ is the length of $\mathcal{A}$ plus the length of $\mathcal{B}$; the dimension of $\mathcal{A} \oplus \mathcal{B}$ is the dimension of $\mathcal{A}$ plus the dimension of $\mathcal{B}$. The minimum distance of $\mathcal{A} \oplus \mathcal{B}$ is the minimum of the minimum distances of $\mathcal{A}$ and $\mathcal{B}$.

Although direct-sum codes bear little resemblance to direct-product codes, the two terms have appeared together so frequently that they must be categorized as relatives-in-law.

A code $\mathcal{D}$ is said to be *decomposable* iff it is equivalent under some permutation to a code $\mathcal{C}$, which can be written as a direct sum, $\mathcal{C} = \mathcal{A} \oplus \mathcal{B}$. Slepian (1960) has shown that no decomposable linear code has larger minimum distance or smaller probability of error than the best indecomposable linear code of the same rate and block length.

14.9 CONCATENATED CODES

If a direct-product code is encoded with a cascaded encoder and decoded with a cascaded decoder, as in Fig. 14.2, the resulting transmission system is a special case of the transmission system shown in Fig. 14.5. This figure also provides the general outline of a powerful class of codes which were introduced by Forney (1966*b*). This section is a brief introduction to Forney's work.

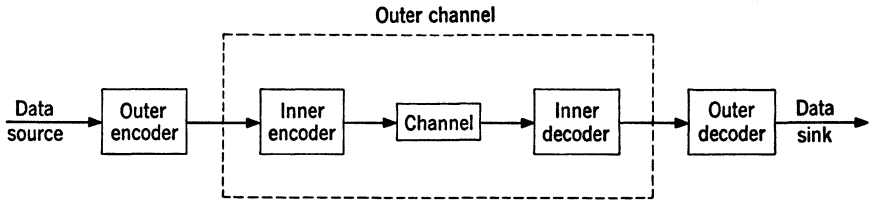


Fig. 14.5 Concatenated encoding and decoding system.

Let us assume that the designs of the inner encoder and the inner decoder, as well as the channel in Fig. 14.5, are fixed by decree from the powers that be. We are then faced with the problem of choosing the outer encoder and the outer decoder in such a way as to improve the performance of the outer channel.

The inner encoder partitions the data stream it receives into blocks of k digits and encodes each of these blocks into a longer block of n digits. Each such block is then transmitted across the inner channel and decoded by the inner decoder. The data stream which is emitted by the inner decoder consists of blocks of k consecutive digits, each of which is the set of message digits from a decoded block of n digits. Although the pattern of errors within any of these blocks is statistically independent of the pattern of errors within any other block, there are statistical dependencies between the error positions within any block. Most blocks of n decoded digits will contain no digit errors at all, but those few blocks which do contain digit errors are likely to have several digits in error.

Within any decoded block of n digits, the error pattern must form a codeword. Suppose, for example, that the inner code is a binary maximal-length feedback-shift-register code of length $n = 2^k - 1$. This code is equidistant; all its nonzero codewords have weight 2^{k-1}. Thus, if a block of n digits from the inner decoder contains any digit errors, it must contain exactly 2^{k-1} digit errors. Since the code is equidistant, the error pattern within the k message digits of a misdecoded block is equally likely to be any of the $2^k - 1$ nonzero binary k-tuples. In other words, each block of k message digits which the inner decoder transmits to the outer decoder is either completely correct or completely random.

This suggests that the outer decoder should regard each block of k message bits which it receives as a single letter in an outer alphabet of order 2^k. When the inner code is equidistant, all intersymbol errors in this outer alphabet are equiprobable.

More generally, if the inner code has k message digits per block over GF(q_{in}), then we may treat the inner encoder, the inner channel, and the inner decoder as a single outer channel whose inputs and outputs are elements in an alphabet of order $q_{\text{out}} = q_{\text{in}}{}^k$. The outer channel is memoryless. In general, its intersymbol error probabilities are difficult to evaluate, so we proceed to design the outer code as though all intersymbol errors were equally likely. We choose the outer code to be some Hamming-metric code over GF(q_{out}).

Since q_{out} is quite large, we choose a Reed-Solomon code as the outer code. By Sec. 13.3, the RS code has as large a minimum distance as any other linear code of the same rate and block length. From Chap. 10, we know that the RS code can be encoded and decoded by the efficient algebraic methods which are applicable to all BCH codes.

Taken together, the outer RS code over GF(q_{out}) and the inner code

over GF(q_{in}) form a very long concatenated supercode over GF(q_{in}). The information rate of this supercode is the product of the rate of the RS code and the rate of the inner code.

In order to construct a concatenated supercode of rate R and length N over GF(q_{in}), we must select the length and rate of the inner code and choose the inner code. For purposes of analysis, the inner code may be chosen at random. Even if this inner code must be decoded by an exhaustive search over all q_{in}^{k} possible codewords, this does not represent an excessive amount of computation in terms of N, the superblock length. If k is algebraic in log N, then the amount of computation per decoded inner block is algebraic in N. The total amount of computation per superblock performed by the inner decoder is also algebraic in N. And, of course, the amount of computation per superblock required to decode the outer RS code is likewise algebraic in N.

The evaluation of probability of decoding error per superblock, i.e., the probability that the concatenated decoder fails to decode *all* the transmitted message digits in the superblock correctly, depends on the known error bounds for randomly chosen codes as well as on the proper choices of rates and block lengths for the inner and outer codes. Forney (1966*b*) has obtained the remarkable result that the probability of decoding error per superblock approaches zero exponentially in N, the superblock length, at all rates less than channel capacity.

Forney (1966*b*) has also evaluated the improvement obtained by allowing the inner decoder to transmit varying amounts of additional information to the outer decoder. In order to attain the best possible concatenated exponent, the inner decoder must be permitted to transmit to the outer decoder both each decoded word and an estimate of the probability that each decoded word is correct. If the outer decoder is not given these estimates of the probabilities of inner-decoder errors, the overall exponent is one-half of the maximum concatenated exponent. If the inner decoder does not estimate its probability of error but does erase all sufficiently ambiguous inner blocks, then the overall exponent is two-thirds of the maximum concatenated exponent.

The reader who is interested in learning more about concatenated codes is referred to the monograph by Forney (1966*b*).

Forney's concatenated codes and Elias' iterated-product codes are not the only known methods of achieving reliable communication with a decoder whose complexity grows relatively slowly with increasing block length. Epstein (1958*a*) exhibited such a system for the binary erasure channel; Horstein (1963), Berlekamp (1964*a*), and Schalkwijk and Kailath (1966) have exhibited such systems for channels which allow noiseless feedback. Wozencraft and Reiffen (1961),† Fano (1963),† Gallager (1963), Ziv (1962, 1966, 1967), Pinsker (1965), and Falconer (1967) have designed such systems for nonerasure channels without feedback.

† See Sec. 15.65.

15
Other Important Coding and Decoding Methods

15.1 SRIVASTAVA CODES—NONCYCLIC CODES WITH AN ALGEBRAIC DECODING ALGORITHM

As we saw in Sec. 10.1, it is convenient to label the locations of a code over GF(q) with elements from GF(q^m). The decoding problem then consists of finding a minimal set of error locations, $X_1, X_2, \ldots, X_e[X_i \in \mathrm{GF}(q^m)]$, and a corresponding set of error values, $Y_1, Y_2, \ldots, Y_e[Y_i \in \mathrm{GF}(q)]$, which satisfy the code's parity-check equations. Rather than determine the X's and Y's directly, it proves easier for the BCH decoder first to find the error-locator polynomial

$$\sigma(z) = \prod_i (1 - X_i z) \tag{15.11}$$

and the alternate error-evaluator polynomial

$$\xi(z) = \sum_i Y_i X_i^l \prod_{j \neq i} (1 - X_j z) \tag{15.12}$$

The degree of this alternate error-evaluator polynomial is always *less* than the number of errors. Combining Eqs. (15.11) and (15.12) gives

$$\frac{\xi(z)}{\sigma(z)} = \sum_i Y_i \frac{X_i^l}{1 - X_i z} \tag{15.13}$$

Srivastava (1967) has proposed a new class of linear codes, defined by parity-check matrices of the form

$$\mathcal{H} = \begin{bmatrix} \frac{b_1^l}{1 - a_1 b_1} & \frac{b_2^l}{1 - a_1 b_2} & \frac{b_3^l}{1 - a_1 b_3} & \cdots & \frac{b_n^l}{1 - a_1 b_n} \\ \frac{b_1^l}{1 - a_2 b_1} & \frac{b_2^l}{1 - a_2 b_2} & \frac{b_3^l}{1 - a_2 b_3} & \cdots & \frac{b_n^l}{1 - a_2 b_n} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ \frac{b_1^l}{1 - a_{d-1} b_1} & \frac{b_2^l}{1 - a_{d-1} b_2} & \frac{b_3^l}{1 - a_{d-1} b_3} & \cdots & \frac{b_n^l}{1 - a_{d-1} b_n} \end{bmatrix} \tag{15.14}$$

where l is any integer, $a_1, a_2, \ldots, a_{d-1}$ are distinct elements from GF(q^m), and $b_1, b_2, \ldots, b_n$ are the elements in GF(q^m) $- 0 -$ $\{a_1^{-1}, a_2^{-1}, a_3^{-1}, \ldots, a_{d-1}^{-1}\}$. The block length of the code is $n = q^m - d$.

In view of Eq. (15.13), the parity-check equations of Srivastava's code reveal $\xi(a_i)/\sigma(a_i)$ for $i = 1, 2, \ldots, d - 1$. The decoder must then attempt to determine the polynomials $\xi(z)$ and $\sigma(z)$ from his knowledge of the value of their quotient at the points $a_1, a_2, \ldots, a_{d-1}$. There can be at most one pair of polynomials $\sigma(z)$ and $\xi(z)$ with $\deg \xi < \deg \sigma \leq (d - 1)/2$, for if there were two solutions [say $\delta(z)$ and $\varphi(z)$ as well as $\sigma(z)$ and $\xi(z)$], we would have

$$\delta(a_i)\xi(a_i) - \sigma(a_i)\varphi(a_i) = 0 \qquad \text{for } i = 1, 2, \ldots, d - 1 \tag{15.15}$$

According to Theorem 2.15, any polynomial of degree $< d - 1$ which has $d - 1$ roots must vanish identically. Since $\deg(\delta\xi - \sigma\varphi) < d - 1$, Eq. (15.15) implies that $\xi(z)/\sigma(z) = \varphi(z)/\delta(z)$. Since $\xi(z)$ and $\sigma(z)$ are relatively prime, this implies that there can be at most one pair of polynomials $\sigma(z)$ and $\xi(z)$ such that $\deg \xi < \deg \sigma \leq (d - 1)/2$ and $\xi(a_i)/\sigma(a_i)$ is equal to any given set of values for $i = 1, 2, \ldots, d - 1$. Therefore, the code defined by Eq. (15.14) is capable of correcting any pattern of up to $(d - 1)/2$ channel errors. The reader may easily verify that the minimum distance of the code is at least d.

Berlekamp (1967*c*) formulated an efficient decoding algorithm for Srivastava's codes directly in terms of sequences of polynomials which converge to $\sigma(z)$ and $\xi(z)$. It is also possible to formulate an efficient decoding procedure based on the numerical method of *divided differences*, as discussed by Milne (1949). Using either approach, it appears that

decoders for long Srivastava codes are only slightly more expensive than decoders for comparable BCH codes.

If $l = 1$, the binary Srivastava code defined by Eq. (15.14) is actually capable of correcting any pattern of up to $d - 1$ channel errors. For binary codes, all the nonzero Y_i have the value 1, and Eq. (15.12) becomes $\xi(z) = \sigma'(z)$ [the derivative of $\sigma(z)$ with respect to z] $= \tilde{\sigma}(z)/z$ [the odd part of $\sigma(z)$ divided by z]. The parity-check equations then reveal the values of the quotient $\tilde{\sigma}(z)/z\sigma(z)$ at the points $z = a_1, a_2, \ldots, a_{d-1}$. We claim that this determines at most one error-locator polynomial of degree $\leq d - 1$, for if there were two such polynomials [say $\sigma(z)$ and $\delta(z)$], then

$$\frac{\tilde{\sigma}(a_i)}{a_i[\hat{\sigma}(a_i) + \tilde{\sigma}(a_i)]} = \frac{\tilde{\delta}(a_i)}{a_i[\hat{\delta}(a_i) + \tilde{\delta}(a_i)]} \tag{15.16}$$

from which

$$\frac{\tilde{\sigma}(a_i)\hat{\delta}(a_i)}{a_i} + \frac{\tilde{\delta}(a_i)\hat{\sigma}(a_i)}{a_i} = 0 \qquad \text{for } i = 1, 2, \ldots, d - 1 \tag{15.17}$$

But

$$\frac{\tilde{\sigma}(z)\hat{\delta}(z)}{z} + \frac{\tilde{\delta}(z)\hat{\sigma}(z)}{z} = f(z^2) = [f(z)]^2 \tag{15.18}$$

where $\deg f < (d - 1)/2$. Therefore, Eq. (15.16) and Theorem 2.15 imply that $\delta(z) = \sigma(z)$.

Each of the $d - 1$ rows of the parity-check matrix of Eq. (15.14) represents an element in $\mathrm{GF}(q^m)$. When this parity-check matrix is expanded out in $\mathrm{GF}(q)$, it will have $(d - 1)m$ rows. If these rows are all linearly independent, the Srivastava code has $(d - 1)m$ check digits. In general, however, the rows of the $\mathfrak{IC}$ matrix will not be independent. The row rank of the $\mathfrak{IC}$ matrix may depend on the choice of the elements $a_1, a_2, \ldots, a_{d-1}$. No method for choosing these elements cleverly has yet been suggested.

Little is known about how many information symbols Srivastava's codes actually do contain. At worst, they are only slightly inferior to the BCH codes. At best, they may have a considerably better information rate.

The Srivastava codes definitely merit further research.

15.2 RESIDUE CODES—GOOD CODES WHICH ARE HARD TO DECODE

Definitions 15.21 If n is prime, and if e divides $n - 1$, then the integer r, $1 \leq r < n$, is said to be an eth *residue mod* n iff the equation $x^e \equiv r \bmod n$ has solutions.

If the order of $q \bmod n$ divides $(n-1)/e$, then the two eth *residue codes* of length n over GF(q) are defined as the cyclic codes whose generator polynomials are

$$g(x) = \begin{cases} \prod_{r \in R_0} (x - \alpha^r) & \text{augmented code} \\ (x-1) \prod_{r \in R_0} (x - \alpha^r) & \text{expurgated code} \end{cases}$$

where α is a primitive nth root of unity in an extension field of GF(q) and R_0 is the set of eth residues mod n.

The *extended* eth*-residue code* of length $N = n + 1$ may be obtained by annexing an overall parity check to the augmented eth-residue code.

Theorem 15.22 Let d be the minimum Hamming weight of the codewords which occur in the augmented eth-residue code but not in the expurgated eth-residue code of length n over GF(q), and assume $e \geq 2$. Then $d^e > n$.

Proof Since n is prime, the integers mod n form a finite field GF(n) which contains a primitive element a whose multiplicative order is $n - 1$. For $i = 0, 1, \ldots, e-1$, let R_i be the set of elements of the form a^{je+i}, where $0 \leq j < (n-1)/e$. The set R_0 consists of the eth residues mod n. Let $g^{(i)}(x) = \prod_{r \in R_i} (x - \alpha^r)$. Then

$$g^{(i)}(x) = \prod_{r \in a^i R_0} (x - \alpha^r) = \prod_{r \in R_0} [x - (\alpha^{a^i})^r]$$

The cyclic codes generated by the polynomials $g^{(0)}(x), g^{(1)}(x), \ldots, g^{(e-1)}(x)$ are all equivalent. For binary codes, this equivalence follows from Theorem 5.81 and Corollary 5.82; for nonbinary codes, this equivalence follows from a straightforward extension of those results. If $C^{(0)}(x)$ is a codeword in the augmented code but not in the expurgated code, then $C^{(0)}(x)$ is a multiple of $g^{(0)}(x)$ but a nonmultiple of $x - 1$. The equivalent codes generated by $g^{(0)}(x), g^{(1)}(x), \ldots, g^{(e-1)}(x)$ contain codewords $C^{(0)}(x), C^{(1)}(x), \ldots, C^{(e-1)}(x)$, which are permutations of the codeword $C^{(0)}(x)$. All these words are nonmultiples of $x - 1$. We now consider the product, $\prod_{i=0}^{e-1} C^{(i)}(x)$. This product is a multiple of $\prod_{i=0}^{e-1} g^{(i)}(x)$, but

it is not a multiple of $x - 1$. Therefore,

$$\prod_{i=0}^{e-1} C^{(i)}(x) = f(x) \sum_{i=0}^{n-1} x^i$$

where $f(1) = s$, a nonzero scalar in GF(q). Since

$$\prod_{i=0}^{e-1} C^{(i)}(x) \equiv sn \bmod x - 1$$

and

$$\prod_{i=0}^{e-1} C^{(i)}(x) \equiv 0 \bmod \sum_{i=0}^{n-1} x^i$$

the Chinese remainder theorem implies that

$$\prod_{i=0}^{e-1} C^{(i)}(x) \equiv s \sum_{i=0}^{n-1} x^i \bmod x^n - 1 \tag{15.221}$$

Since the Hamming weight of the product of several polynomials mod $x^n - 1$ is no greater than the product of their Hamming weights, Eq. (15.221) implies that

$$\prod_{i=0}^{e-1} d \geq n$$

Since n is prime, we must have strict inequality. Q.E.D.

The most important class of residue codes are those with $e = 2$. These codes are called *quadratic residue* (QR) *codes*. For QR codes, Theorem 15.22 asserts that $d^2 > n$. If $n \equiv -1 \bmod 4$, this bound can be improved by Theorem 15.23.

Theorem 15.23 [Mattson-Solomon (1961)] If n is a prime $\equiv -1 \bmod 4$, then the minimum Hamming weight of the codewords in the augmented QR code of length n that are not codewords in the expurgated QR code of length n is bounded by $d^2 - d + 1 \geq n$.

Proof As in the proof of Theorem 15.22, each word in the code generated by $g^{(0)}(x)$ corresponds to another word in the code generated by $g^{(1)}(x)$. Since $n = -1 \bmod 4$, -1 is a nonresidue mod n, and we may take $C^{(1)}(x) = \tilde{C}^{(0)}(x)$, the reciprocal polynomial of

$C^{(0)}(x)$. Equation (15.221) then becomes

$$C^{(0)}(x)\tilde{C}^{(0)}(x) \equiv s \sum_{i=0}^{n-1} x^i \bmod x^n - 1$$

If d is the weight of $C^{(0)}$ and $\tilde{C}^{(0)}$, then the weight of the product is at most $d^2 - d + 1$ because the product contains d terms in $x^{\deg C^{(0)}}$. Q.E.D.

According to Theorem 6.54, QR codes of length n over GF(2) exist iff n is a prime $\equiv \pm 1 \bmod 8$. Although the generator matrix of a binary QR code may be taken as the successive shifts of the code's generator polynomial, there are certain advantages to using another representation of this matrix, as illustrated in Fig. 15.01.

Theorem 15.24 Let $N = n + 1$; let R_0 be the set of nonzero quadratic residues mod n, and let R_1 be the set of nonresidues mod n. Then the extended binary QR code of length N is generated by the $N \times N$ matrix $\mathcal{G}$, defined by

$$\mathcal{G}_{\infty,v} = 1 \text{ for all } v, \text{ including } v = \infty$$

$$\mathcal{G}_{u,\infty} = \begin{cases} 0 & \text{if } n \equiv 1 \bmod 8 \\ 1 & \text{if } n \equiv -1 \bmod 8 \end{cases}$$

$$\mathcal{G}_{u,v} = \begin{cases} 1 & \text{if } v - u \in R_0 \\ 0 & \text{if } v - u \in R_1 \\ 0 & \text{if } v - u = 0 \end{cases}$$

Remarks Although this generator matrix has N rows, the dimension of its row space is only $N/2$.

Since every extended binary QR code is equivalent to its own dual, the matrix defined by Theorem 15.24 may also be taken as the parity-check matrix of an extended QR code.

Proof As seen in the example illustrated in Fig. 15.01, the rows of $\mathcal{G}$ (excluding the symbol labeled ∞) are the cyclic shifts of the polynomial $f(x) = \sum_{r \in R_0} x^r$. If α is a primitive nth root of unity, then $[f(\alpha)]^2 = f(\alpha)$, so $f(\alpha) \in \mathrm{GF}(2)$, and if $i \in R_0$, then $f(\alpha^i) = f(\alpha)$. If $i \in R_1$, then $f(\alpha^i) = 1 + f(\alpha)$, because $f(\alpha) + f(\alpha^i)$ is the sum of all primitive nth roots of unity, which is 1 because it is equal to the coefficient of x^{n-2} in the cyclotomic polynomial $Q^{(n)}(x)$. There-

	∞	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
∞	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	1	1	0	1	0	0	0	1	1	0	0	0	1	0	1	1
1	0	1	0	1	1	0	1	0	0	0	1	1	0	0	0	1	0	1
2	0	1	1	0	1	1	0	1	0	0	0	1	1	0	0	0	1	0
3	0	0	1	1	0	1	1	0	1	0	0	0	1	1	0	0	0	1
4	0	1	0	1	1	0	1	1	0	1	0	0	0	1	1	0	0	0
5	0	0	1	0	1	1	0	1	1	0	1	0	0	0	1	1	0	0
6	0	0	0	1	0	1	1	0	1	1	0	1	0	0	0	1	1	0
7	0	0	0	0	1	0	1	1	0	1	1	0	1	0	0	0	1	1
8	0	1	0	0	0	1	0	1	1	0	1	1	0	1	0	0	0	1
9	0	1	1	0	0	0	1	0	1	1	0	1	1	0	1	0	0	0
10	0	0	1	1	0	0	0	1	0	1	1	0	1	1	0	1	0	0
11	0	0	0	1	1	0	0	0	1	0	1	1	0	1	1	0	1	0
12	0	0	0	0	1	1	0	0	0	1	0	1	1	0	1	1	0	1
13	0	1	0	0	0	1	1	0	0	0	1	0	1	1	0	1	1	0
14	0	0	1	0	0	0	1	1	0	0	0	1	0	1	1	0	1	1
15	0	1	0	1	0	0	0	1	1	0	0	0	1	0	1	1	0	1
16	0	1	1	0	1	0	0	0	1	1	0	0	0	1	0	1	1	0

By suitably permuting rows and columns, this matrix can also be written as

	∞	0	1	2	4	8	16	15	13	9	3	6	12	7	14	11	5	10
∞	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0
1	0	1	0	1	0	0	1	0	0	1	1	0	0	0	1	0	1	1
2	0	1	1	0	1	0	0	1	0	0	1	1	0	0	0	1	0	1
4	0	1	0	1	0	1	0	0	1	0	1	1	1	0	0	0	1	0
8	0	1	0	0	1	0	1	0	0	1	0	1	1	1	0	0	0	1
16	0	1	1	0	0	1	0	1	0	0	1	0	1	1	1	0	0	0
15	0	1	0	1	0	0	1	0	1	0	0	1	0	1	1	1	0	0
13	0	1	0	0	1	0	0	1	0	1	0	0	1	0	1	1	1	0
9	0	1	1	0	0	1	0	0	1	0	0	0	0	1	0	1	1	1
3	0	0	1	1	1	0	1	0	0	0	0	0	1	1	0	1	1	0
6	0	0	0	1	1	1	0	1	0	0	0	0	0	1	1	0	1	1
12	0	0	0	0	1	1	1	0	1	0	1	0	0	0	1	1	0	1
7	0	0	0	0	0	1	1	1	0	1	1	1	0	0	0	1	1	0
14	0	0	1	0	0	0	1	1	1	0	0	1	1	0	0	0	1	1
11	0	0	0	1	0	0	0	1	1	1	1	0	1	1	0	0	0	1
5	0	0	1	0	1	0	0	0	1	1	1	1	0	1	1	0	0	0
10	0	0	1	1	0	1	0	0	0	1	0	1	1	0	1	1	0	0

Fig. 15.01 Generator matrices for the extended binary QR code of length 18.

fore, there exists a primitive nth root of unity, say α, such that $f(\alpha^r) = 0$ if $r \in R_0$ but $f(\alpha^r) = 1$ if $r \in R_1$.

The multiples of $f(x)$ are therefore identical to the multiples of the generator polynomial of the augmented QR code or the expurgated QR code, depending on whether $f(1) = 0$ or 1. If

$n \equiv 1 \bmod 8$, then $(n-1)/2$ is even, and $f(1) = 0$. In this case, the multiples of $f(x)$ are the codewords of the expurgated QR code of length n. The extended QR code of length N may be formed by lengthening this code by annexing the all-one vector of length N to the code's generator matrix. This gives the generator matrix defined in the statement of the theorem. If $n \equiv -1 \bmod 8$, then $(n-1)/2$ is odd, and $f(1) = 1$. In this case, the multiples of $f(x)$ are the codewords of the augmented QR code of length n. The extended QR code of length N is then formed by annexing an overall parity check to the code generated by $f(x)$. This results in the generator matrix defined in the statement of the theorem.

Q.E.D.

If r is the square of a primitive element in GF(n), and if s is any nonresidue in GF(n), then the N elements in GF(n) $\cup\ \infty$ may be arranged in the order $\infty, 0; 1, r, r^2, r^3, \ldots; s, sr, sr^2, \ldots$ or the order $\infty, 0; s, sr, sr^2, \ldots; 1, r, r^2, \ldots$. As in Fig. 15.01, an appropriate ordering of the coordinates leads to a generator matrix of the form

$$\mathcal{G} = \left[\begin{array}{c|c} \mathcal{A} & \mathcal{B} \\ \hline \mathcal{B}^t & \mathcal{C} \end{array}\right] \tag{15.241}$$

where $\mathcal{A}$, $\mathcal{B}$, and $\mathcal{C}$ are square circulant matrices of size $(n-1)/2$, and $\mathcal{A}$ and $\mathcal{C}$ are symmetric. If $\mathcal{A}$, $\mathcal{B}$, or $\mathcal{C}$ is invertible, then an appropriate linear transformation enables one to obtain a generator matrix of the form

$$\mathcal{G} = \left[\begin{array}{c|c} \mathcal{I} & \mathcal{D} \end{array}\right] \tag{15.242}$$

where $\mathcal{I}$ is an identity matrix of size $(n-1)/2$ and $\mathcal{D}$ is a square circulant matrix of same size. Karlin (1969) has found this form of the generator matrix very useful in his studies of the weight structure of binary QR codes. Since many of the best of the known binary codes can be generated by matrices of the form given by Eq. (15.242), MacWilliams (1968, 1970) has recently undertaken a search for new codes of this form.

Another advantage of the form of the generator matrix specified by Theorem 15.24 is the ease with which it enables us to prove Theorem 15.25.

Theorem 15.25 Every extended binary QR code is invariant under the permutation $C_u \leftrightarrow C_{-u^{-1}}$, where the arithmetic operations on the subscripts are performed in GF(n) $\cup\ \infty$ with $\pm 0^{-1} = \infty$, $\pm \infty^{-1} = 0$.

Proof We claim that

$$\mathcal{G}_{-u^{-1},-v^{-1}} = \begin{cases} \mathcal{G}_{u,v} + \mathcal{G}_{0,v} & \text{if } u \in R_1 \text{ or } u = \infty \\ & \text{and } n \equiv -1 \bmod 8 \\ \mathcal{G}_{u,v} + \mathcal{G}_{0,v} + \mathcal{G}_{\infty,v} & \text{if } u \in R_0 \cup 0 \text{ or } u = \infty \\ & \text{and } n \equiv 1 \bmod 8 \end{cases} \tag{15.251}$$

From Theorem 15.24, Eq. (15.251) follows directly if u or $v = 0$ or ∞ or if $u = v$. To check the other cases, we notice that $\mathcal{G}_{-u^{-1},-v^{-1}} + \mathcal{G}_{u,v} = 0$ iff $(v - u)(-v^{-1} + u^{-1}) \in R_0$. Since $(v - u)(-v^{-1} + u^{-1}) = -1 + u/v + v/u - 1 = (u/v)[(v/u) - 1]^2$, $\mathcal{G}_{-u^{-1},-v^{-1}} + \mathcal{G}_{u,v} = 0$ iff $u/v \in R_0$, which is equivalent to (15.251).

Since the matrix on the right of Eq. (15.251) is obtained from $\mathcal{G}_{u,v}$ by a linear transformation of its rows, and since the matrix on the left of Eq. (15.251) is obtained by a row permutation and the column permutation specified by the theorem, the theorem is proved. Q.E.D.

Gleason, Prange, Assmus, and Mattson have shown that if C_∞ is taken as a scalar multiple of the overall parity check, then the extended nonbinary QR code is invariant under the transformation $C_u \leftrightarrow \pm C_{-u^{-1}}$, where the sign depends on the quadratic character of u.

All the arithmetic operations (except $0 \times \infty$) may be defined in $\mathrm{GF}(n) \cup \infty$ in the obvious way. [If $a \in \mathrm{GF}(n)$, then $\infty + a = \infty$; if $a \in \mathrm{GF}(n) - 0$, then $a\infty = \infty$.] These arithmetic operations generate the *linear fractional group*, $u \to (au + b)/(cu + d)$, where $u \in \mathrm{GF}(n) \cup \infty$; $a,b,c,d \in \mathrm{GF}(n)$; $ad \neq cb$. This linear fractional group is *triply transitive;* any ordered triplet may be permuted into any other ordered triplet. To see this, we first observe that we may permute any ordered triplet into the positions $(\infty,0,1)$. To do so, we first move the element destined for ∞ to the 0 position by an appropriate addition; we then move that element to ∞ by a reciprocation. We then move the element destined for the 0 position to the 0 position by an appropriate addition. Finally, we move the symbol destined for the 1 position to the 1 position by an appropriate multiplication. Thus, the linear fractional group contains a permutation which moves any ordered triplet into the positions $(\infty,0,1)$. Since the group includes the inverse of each of its permutations, it also contains a permutation which moves the triplet from the positions $(\infty,0,1)$ to any three desired positions. Therefore, we can permute the triplet in the positions (u,v,w) into the positions (x,y,z) by the sequence $(u,v,w) \to (\infty,0,1) \to (x,y,z)$.

Since the identity is the only permutation in the linear fractional group which leaves the three symbols $(\infty,0,1)$ pointwise fixed, the order of the linear fractional group is seen to be $N(N - 1)(N - 2)$.

The linear fractional group contains a subgroup of index 2 and order $N(N-1)(N-2)/2$ which consists of those permutations

$$u \to (au + b)/(cu + d)$$

for which $ad - cb = 1$. This subgroup is called the *projective unimodular group*. It is generated by the permutations $u \to u + 1$, $u \to -u^{-1}$, and $u \to a^2u$, where a is a primitive element in GF(n).

Every extended binary cyclic code is invariant under the permutation $C_u \to C_{u+1}$, since this permutation is merely a cyclic shift. According to Theorem 15.24, an extended binary QR code is also invariant under the permutation $C_u \to C_{-u^{-1}}$; according to Theorem 5.81, this code is also invariant under all permutations of the form $C_u \to C_{ru}$, $r \in R_0$. This proves Theorem 15.26.

Theorem 15.26 Every extended binary QR code is invariant under the doubly transitive projective unimodular group, $C_u \to C_{(au+b)/(cu+d)}$, where $u \in \mathrm{GF}(n) \cup \infty$; $a,b,c,d \in \mathrm{GF}(n)$; $ad - bc = 1$.

Corollary 15.27 The minimum weight of the augmented binary QR code of length n is an odd number d, which is bounded by $d^2 > n$ if $n \equiv +1 \bmod 8$ or $d(d-1) \geq n - 1$ if $n \equiv -1 \bmod 8$.

Corollary 15.27 is an immediate consequence of Theorems 15.26, 10.39, 15.22, and 15.23.

Corollary 15.27 shows that the minimum distance of the augmented binary QR code of length 23 (the Golay code) is at least 7. According to the volume bound (Sec. 13.1), no binary code of this length and information rate can have distance >7, because $\binom{23}{0} + \binom{23}{1} + \binom{23}{2} + \binom{23}{3} = 2^{11}$. Golay's code is therefore a perfect code: it satisfies the volume bound with equality.

All extended QR codes have information rate $R = 1/2$. These codes include the extended single-error-correcting binary Hamming code of length 8, the extended binary Golay code of length 24, and the extended ternary Golay code of length 12. Although the minimum distance guaranteed by Corollary 15.27 increases slowly with N, QR codes often have minimum weights higher than this bound. For example, we shall see (Theorem 16.33) that the extended binary QR code of length 48 has minimum weight 12, even though Corollary 15.27 guarantees only that its weight is at least 10.

n	q	d
7	2	3
17	2	5
23	2	7
31	2	7
41	2	9
47	2	11
71	2	11
73	2	13
79	2	15
89	2	17
97	2	15
103	2	≤ 19
113	2	≤ 15
127	2	≤ 19
137	2	≤ 21
151	2	≤ 19
167	2	≤ 23
191	2	≤ 31
193	2	≤ 27
199	2	≤ 31
223	2	≤ 35
233	2	≤ 25
239	2	≤ 39
241	2	≤ 41

n	q	d
761	2	
1,361	2	
1,381	3	
6,841	2, 3	$< \frac{n-1}{10}$
8,821	3	
21,401	5	
30,941	7	
49,921	2	
2,521	2	
8,089	2	$< \frac{n-1}{12}$
47,521	3	
49,681	2	
8,221	5	
18,541	3	
22,621	5	$< \frac{n-1}{12}$
25,261	5	
44,269	7	
1,289	2	
6,301	3	$< \frac{n-1}{14}$
49,057	2	
4,801	2	
15,937	3	$< \frac{n-1}{16}$
47,713	2	
8,821	3	$< \frac{n-1}{20}$
9,601	2	
18,433	2	
21,313	2	
34,849	2	$< \frac{n-1}{24}$
37,057	2	
42,961	2	
48,817	2	

Fig. 15.02 Minimum Hamming distances of certain augmented QR codes of length n over GF (q). (This table was supplied by H. F. Mattson.)

Since the short QR codes are so good, a great deal of research has been devoted to the determination of the minimum weights of the longer binary QR codes. The best results known on this question have been obtained by Gleason, Prange (unpublished); Assmus, Mattson, and Turyn (1964, 1965, 1966); Pless (1963); and Karlin (1969). Most of their results are summarized in Fig. 15.02. The results for $n \geq 761$ were obtained by Assmus and Mattson (1966), using a sophisticated method based on cyclotomic number tables computed by J. B. Muskat. This method enabled them to find low weight codewords in certain long QR codes of lengths $n \equiv 1 \bmod 4$. No comparable low weight codewords are known in long QR codes of length $n \equiv -1 \bmod 4$.

In general, the minimum distances of QR codes of moderate lengths appear to be slightly better than the minimum distances of comparable BCH codes. Unfortunately, however, the QR codes are much more difficult to decode. Theorems 15.22, 15.23, 15.27, and 16.33 all give lower bounds on the minimum distances of certain QR codes, but the proofs of these theorems do not lead to feasible decoding algorithms.

In general, the decoder has no trouble in recognizing the channel errors if all these errors occur in the code's check positions. This observation is the basis of a decoding strategy called *permutation decoding*, introduced by MacWilliams (1964). The decoder successively permutes the digits of the received word until all the errors are permuted into the code's check positions, where they are recognized and corrected. Permutation decoding is best suited to codes which are invariant under a large group of permutations. As will be seen in Prob. 15.3, certain extended QR codes are invariant under a much larger permutation group than the projective unimodular group of Theorem 15.26. However, Shaughnessy (1968) has shown that certain other extended QR codes are invariant under only the simple† projective unimodular group.

Although permutation decoding is one of the best methods known for decoding certain QR codes, including the Golay code, permutation decoders are considerably more expensive than comparable BCH decoders. From a practical standpoint, the relatively large minimum distances of QR codes and the relative difficulties in decoding them make these codes best suited for applications in which an incomplete decoding algorithm is desirable.

15.3 REED-MULLER CODES—WEAK CODES WHICH ARE EASY TO DECODE

Although shortened Reed-Muller codes were first introduced in 1954, they were not known to be cyclic until recently. With the benefit of hindsight, I find it pedagogically advantageous to reverse the historical

† A *simple* group has no normal subgroups. For further discussion, see Dickson (1901).

sequence and to define these codes in terms of the roots of their generator polynomials and then prove their other properties, rather than vice versa.

The nonbinary generalization of the Reed-Muller codes is due to Kasami, Lin, and Peterson (1967*a*).

15.31 Definitions The *shortened rth-order generalized Reed-Muller code* (hereafter abbreviated GRM code) of length $n = q^m - 1$ over GF(q) is the cyclic code whose generator polynomial $g(x)$ is defined by

$$g(x) = \prod_{\substack{j, \\ 0 \le w(j) < (q-1)m - r \\ 0 \le j < q^m - 1}} (x - \alpha^j)$$

where $w(j)$ is the sum of the digits of the q-ary expansion of the integer j and α is a primitive element in GF(q^m).

The *rth-order GRM code* is obtained by annexing the all-one vector of length $N = q^m$ to the generator matrix of the shortened GRM code.

The *rth-order RM code* is the rth-order GRM code over GF(2).

15.32 Properties of the GRM Codes

15.321. The rth-order RM code is a subcode of the extended BCH code of designed distance $d = 2^{m-r}$.

Proof For $j = 1, 2, 3, \ldots, 2^{m-r} - 2$, $w(j) < m - r$.

15.322. The rth-order GRM code is a subcode of the extended BCH code of designed distance $d = (q - R)q^{(m-Q-1)}$, where Q and R are the quotient and the remainder obtained by dividing r by $q - 1$.

Proof Letting $r = Q(q - 1) + R$; $(q - 1)m - r = (q - 1)(m - Q - 1) + q - 1 - R$. $w([q - R]q^{[m-Q-1]} - 1) = (q - 1)(m - Q - 1) + (q - R - 1)$, but for $j = 1, 2, 3, \ldots, (q - R)q^{(m-Q-1)} - 2$, $w(j) < (q - 1)m - r$.

15.323. The dual of the rth-order GRM code is equivalent to a $[(q - 1)m - r - 1]$st-order GRM code.

Proof

$$\prod_{\substack{(q-1)m-r\le w(j)\le(q-1)m \\ 0\le j<q^m-1}} (x-\alpha^j) = \prod_{\substack{0<w(j)<r+1 \\ 0\le j<q^m-1}} (x-\alpha^{-j})$$

15.324. The number of information symbols in the rth-order RM code is given by

$$\sum_{i=0}^{r} \binom{m}{i}$$

Proof There are $\binom{m}{i}$ integers j such that $0 \le j < 2^m$ and $w(j) = i$.

15.325. The number of information symbols in the rth-order GRM code is given by

$$\sum_{i=0}^{r} P(i+m, m, q)$$

where $P(i+m, m, q)$ is the number of partitions of $i+m$ into exactly m parts, no part exceeding q.

Proof The number of integers $j = \sum_{k=0}^{m-1} j_k q^k$ such that $0 \le j_k < q$ for all k and $\Sigma j_k = i$ [or $\Sigma(1+j_k) = i+m$] is $P(i+m, m, q)$.

15.326. The zeroth-order GRM code is a repetition code.

15.327. The first-order GRM code is an extended maximal-length shift-register code.

15.328. The $[(q-1)m-2]$nd-order GRM code is an extended single-error-correcting BCH code.

15.329. The $[(m-k)(q-1)]$st-order GRM code has exactly

$$(q-1)q^{m-k} \prod_{i=0}^{k-1} \left(\frac{q^{m-i}-1}{q^{k-i}-1}\right)$$

codewords of Hamming weight q^k. Each of these codewords

may be represented as a scalar multiple of the indicator function of a k-dimensional affine subspace of GF(q^m) over GF(q). That is,

$$C(x) = \xi \sum_{\substack{i \\ \alpha^i \in A}} x^i$$

where $\xi \in \mathrm{GF}(q) - 0$ and A is a k-dimensional affine subspace of GF(q^m) over GF(q). If $\alpha^\infty \in A$, then the overall parity check is given by $C_\infty = \xi$; if $\alpha^\infty \notin A$, then $C_\infty = 0$.

Property 15.329 is a direct consequence of Theorems 11.64, 11.61, and 11.52. Readers who have skipped the starred sections of Chap. 11 must accept Property 15.329 without proof.

Property 15.329 shows that the minimum distance of every RM code (and certain GRM codes) is given by the BCH bound of Property 15.322. Using Theorem 11.61, it can be shown that the minimum distance of every GRM code is given by the BCH bound. Since the argument is directly parallel to Theorem 11.66, it is omitted.

According to Properties 15.327 and 15.328, the class of GRM codes includes some good codes, at both high rates and low rates. At medium rates, however, the GRM codes are disappointing. If m is odd, then the $[(m-1)/2]$nd-order RM code of length $N = 2^m$ has minimum distance $d = \sqrt{2N}$. The extended BCH code of length N and $d = \sqrt{2N}$ has only $1 + [(d-2)/2]\log_2 N = 1 + (\sqrt{N/2} - 1)\log_2 N$ check symbols, but the $[(m-1)/2]$nd-order RM code has $N/2$ check symbols. As $N \to \infty$, the RM codes of rate 1/2 have the same distances as BCH codes of rates approaching 1. Even for moderate N, the difference is significant. The third-order RM code of length 128 has rate 64/128; the extended 7-error-correcting BCH code of length 128 has rate 78/128. Both of these codes have minimum distance 16.

Nevertheless, RM codes occupy an important place in algebraic coding theory for two reasons: (1) the study of RM codes yields additional insight into certain BCH codes to which these RM codes are closely related, and (2) the low-rate RM codes of short to moderate block lengths have rates only slightly less than the comparable BCH codes, and these RM codes are somewhat easier to decode.

The codewords of any RM code are related to the codewords in the first-order RM code in an interesting way which we shall now investigate. Let $u_1, u_2, \ldots, u_m$ be any basis of GF(2^m) over GF(2), and for $i = 1, 2, \ldots, m$, let A_i be the $(m-1)$-dimensional subspace of GF(2^m) consisting of those elements of the form $\sum_{j=1}^{m} U_j u_j$, where $U_i = 0$ and $U_j \in$ GF(2) for all j. Let $\mathbf{C}^{(i)} = [C_0^{(i)}, C_1^{(i)}, \ldots, C_{n-1}^{(i)}, C_\infty^{(i)}]$ be the *indicator*

function of the set A_i, defined by

$$C_j^{(i)} = \begin{cases} 1 & \text{if } \alpha^j \in A_i \\ 0 & \text{if } \alpha^j \notin A_i \end{cases} \tag{15.331}$$

As usual, we take $n = 2^m - 1$ and conventionally define $\alpha^\infty = 0$.

I next define the *component-wise product*† of $\mathbf{C}^{(i)}$ and $\mathbf{C}^{(j)}$ as the word formed by component-by-component multiplication

$$\begin{aligned} \mathbf{C}^{(i)} &= C_0^{(i)}, \quad C_1^{(i)}, \ \ldots, \quad C_{n-1}^{(i)}, C_\infty^{(i)} \\ \mathbf{C}^{(j)} &= C_0^{(j)}, \quad C_1^{(j)}, \ \ldots, \quad C_{n-1}^{(j)}, C_\infty^{(j)} \\ \mathbf{C}^{(i)} \otimes \mathbf{C}^{(j)} &= C_0^{(i)}C_0^{(j)}, C_1^{(i)}C_1^{(j)}, \ \ldots, C_{n-1}^{(i)}C_{n-1}^{(j)}, C_\infty^{(i)}C_\infty^{(j)} \end{aligned} \tag{15.332}$$

Theorem 15.34 Every codeword in an rth-order RM code can be written as a binary polynomial of degree at most r in the variables $\mathbf{C}^{(1)}$, $\mathbf{C}^{(2)}$, . . . , $\mathbf{C}^{(m)}$.

Remarks The constant term in the polynomial must be taken as **1**, which is the all-one vector.

Kasami, Lin, and Peterson have also proved Theorem 15.34 (substituting q-ary for binary) for nonbinary GRM codes. In the nonbinary case, $\mathbf{C}^{(1)}$, $\mathbf{C}^{(2)}$, . . . , $\mathbf{C}^{(m)}$ must be taken as particular codewords of the first-order RM code; they cannot be defined by Eq. (15.331) although their product is still defined by Eq. (15.332).

Proof A word of the form

$$\mathbf{C}^{(i_1)} \otimes \mathbf{C}^{(i_2)} \otimes \cdots \otimes \mathbf{C}^{(i_j)} \tag{15.341}$$

is the indicator function of $A_{i_1} \cap A_{i_2} \cap \cdots \cap A_{i_j}$, which is an $(m - j)$-dimensional subspace of GF(2^m). If $j \leq r$, then according to Theorem 11.64 (or Property 15.329), a word of the form (15.341) is a codeword of the rth-order RM code. The number of codewords of this form is $\sum_{j=0}^{r} \binom{m}{j}$, which is identical to the number of information symbols in the rth-order RM code by Property 15.324. To show that all words of the form (15.341) are linearly independent, we notice that the indicator function of any element in GF(2^m), say $\sum_{i=1}^{m} U_i u_i$, may be expressed $\prod_{i=1}^{m} [\mathbf{C}^{(i)} + U_i \mathbf{1}]$,

† Kasami, Lin, and Peterson use the term *vector product.*

π_1: $A_0 + A_1\alpha + A_2\alpha^2 + A_3\alpha^3 \rightarrow A_3 + (A_0 + A_3)\alpha + A_1\alpha^2 + A_2\alpha^3$
$(\infty)(0,1,2,3,4,5,6,7,8,9,10,11,12,13,14)$
(Meaning $\alpha^\infty \rightarrow \alpha^\infty$, $\alpha^0 \rightarrow \alpha^1 \rightarrow \alpha^2 \rightarrow \cdots \rightarrow \alpha^{14} \rightarrow \alpha^0$)

π_2: $A_0 + A_1\alpha + A_2\alpha^2 + A_3\alpha^3 \rightarrow A_0 + A_3\alpha + (A_1 + A_3)\alpha^2 + A_2\alpha^3$
$(\infty)(0)(1,2,3,5,6,11,9)(4,8,14,10,13,12,7)$

π_3: $A_0 + A_1\alpha + A_2\alpha^2 + A_3\alpha^3 \rightarrow A_0 + A_1\alpha + A_3\alpha^2 + (A_2 + A_3)\alpha^3$
$(\infty)(0)(1)(4)(2,3,6)(5,9,11)(7,10,12)(8,14,13)$

π_4: $A_0 + A_1\alpha + A_2\alpha^2 + A_3\alpha^3 \rightarrow A_0 + A_1\alpha + A_2\alpha^2 + (A_3 + 1)\alpha^3$
$(\infty,3)(0,14)(1,9)(2,6)(4,7)(5,11)(8,13)(10,12)$

Fig. 15.03 Some permutations which preserve the RM codes of length 16.

which is a polynomial of degree $\leq m$ in the $\mathbf{C}^{(i)}$. Thus, all 2^m words of the form (15.341) are linearly independent, and the $\sum_{j=0}^{r} \binom{m}{j}$ of these words which have degree $\leq r$ form a set of generators for the rth-order RM code. Q.E.D.

Theorem 15.35 Every RM code of length $N = 2^m$ is invariant under any permutation which permutes all affine-subspaces of GF(2^m) over GF(2) into affine subspaces. The group of all such permutations is triply transitive, and its order is

$$2^m \prod_{i=0}^{m-1} (2^m - 2^i)$$

Proof Every codeword in an rth-order RM code is a sum of words which are indicator functions of affine subspaces of dimension $\geq m - r$, and every sum of indicator functions of affine subspaces of dimension $\geq m - r$ is a codeword. Therefore, every permutation which permutes affine subspaces of GF(2^m) into other affine subspaces must preserve all RM codes.

Any of the 2^m translations of GF(2^m) permutes affine subspaces into affine subspaces, so the order of the entire permutation group is 2^m times the order of its subgroup which leaves the 0 element of GF(2^m) fixed. This subgroup must permute all linear subspaces of GF(2^m) into other linear subspaces. Any given ordered basis of GF(2^m) must be permuted into another ordered basis. The order of the subgroup of permutations which leave 0 fixed is therefore equal to the number of ordered bases of an m-dimensional binary vector space, which is $\prod_{i=0}^{m-1} (2^m - 2^i)$, as in the proof of Theorem 11.52. Q.E.D.

Some examples of Theorem 15.35 may be found in Fig. 15.03, giving some of the permutations which preserve the RM codes of block length 16. These permutations are based on the representation of $GF(2^4)$ found in column 7 of Table 4.1.

15.4 THRESHOLD DECODING—THE BEST KNOWN ALGORITHM FOR DECODING CERTAIN CODES

Since the codewords of an RM code are a subset of the codewords of an extended BCH code, it is possible to decode the RM code by the algebraic decoding algorithms for BCH codes. However, it is easier to decode the low-order RM codes by another type of decoding algorithm, called *threshold decoding*. A variation of threshold decoding for the Reed-Muller codes was first proposed by Reed (1954). Threshold-decoding procedures for certain other classes of block codes were first introduced by Massey (1963).

We explain threshold decoding by a sequence of examples.

15.41 BINARY REPETITION CODES OF LENGTH $n = 2^m - 1$

We begin with the simplest case, the zeroth-order punctured RM code, which is a binary repetition code. To decode this code, we need only count the number of ones among the code's $n = 2^m - 1$ received digits. If there are more than $n/2$ received ones, we decide that the all-one codeword was sent; if there are fewer than $n/2$ received ones, we decide that the all-zero codeword was sent. The most direct method by which this decoding algorithm might be implemented is shown in Fig. 15.04. The decoder contains an m-bit counter and an m-stage primitive feedback shift register (FSR). The initial contents of the counter are zero; the initial contents of the FSR are 00 . . . 01. As the digits are received from the channel, they are added into the counter. When the contents

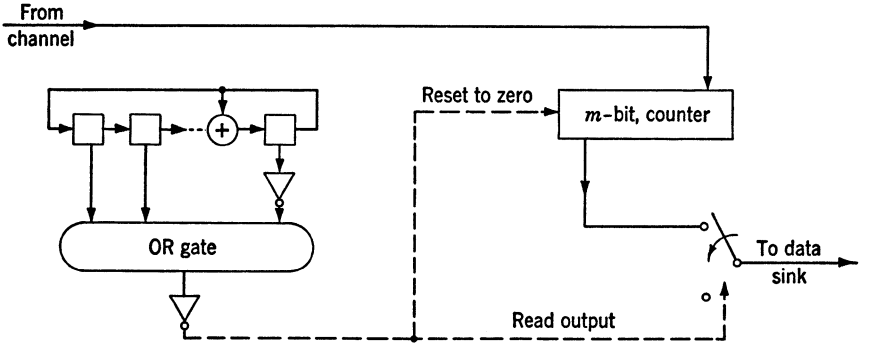


Fig. 15.04 Decoder for 0th-order punctured RM code.

of the FSR are again 00 . . . 01, all $2^m - 1$ channel digits have been received. The highest bit of the counter is a one iff the received digits include 2^{m-1} or more ones. The highest bit of the counter is emitted as output, and the counter is reset to zero in preparation for the next block of $2^m - 1$ digits.

The binary repetition code satisfies the following parity-check equations:

$$\begin{aligned} 0 &= C_0 + C_1 \\ 0 &= C_0 \quad + C_2 \\ 0 &= C_0 \quad\quad + C_3 \\ &\cdots\cdots\cdots\cdots \end{aligned} \tag{15.42}$$

Each of these parity-check sums includes C_0 and one other digit. Furthermore, no two of these parity-check sums include any common digits except C_0. For this reason, each of these equations provides an independent estimate of C_0, namely,

$$C_0 = \begin{cases} R_1 & \text{if } E_1 = 0 \\ R_2 & \text{if } E_2 = 0 \\ R_3 & \text{if } E_3 = 0 \end{cases}$$
$$\cdots\cdots\cdots\cdots$$

If no more than half of the error digits $E_0, E_1, \ldots, E_{n-1}$ have value 1, then C_0 may be determined by a majority vote of the R's.

Equations (15.42) are the simplest example of a set of orthogonalized parity-check equations. In general, we have the following definition.

Definition 15.43 A set of parity-check equations is said to be *orthogonal on the set of positions* $P = \{i,j, \ldots ,k\}$ iff

1. For each $i \in P$, E_i is included in the parity-check sum of each equation in the set.
2. For each $i \notin P$, E_i is included in at most one equation in the set.

15.44 MAXIMAL-LENGTH FSR CODES

We next consider the first-order shortened RM codes, which are identical to the maximal-length FSR codes. Since the dual of such a code is a perfect single-error-correcting Hamming code, the code satisfies the parity check $\sum_{i \in P} C_i = 0$ iff $\sum_{i \in P} \alpha^i = 0$. In particular, for any given pair of positions $\{i,j\}$, the code contains a parity-check set of weight 3 including these two positions, namely, $P = \{i,j,k\}$, where $\alpha^k = \alpha^i + \alpha^j$. Thus, we can find $(n - 1)/2$ parity checks of weight 3 which are orthogonal on any

given position. For example, suppose $m = 4$ and $\alpha^4 = \alpha + 1$. Then the following seven parity-check sets are orthogonal on position 0: {0,1,4}, {0,2,8}, {0,3,14}, {0,5,10}, {0,6,13}, {0,7,9}, {0,11,12}.

If there are no more than three channel errors, then C_0 may be determined by a majority vote of these eight estimates:

$$C_0 = \begin{cases} R_1 + R_4 & \text{if } E_1 + E_4 = 0 \\ R_2 + R_8 & \text{if } E_2 + E_8 = 0 \\ R_3 + R_{14} & \text{if } E_3 + E_{14} = 0 \\ R_5 + R_{10} & \text{if } E_5 + E_{10} = 0 \\ R_6 + R_{13} & \text{if } E_6 + E_{13} = 0 \\ R_7 + R_9 & \text{if } E_7 + E_9 = 0 \\ R_{11} + R_{12} & \text{if } E_{11} + E_{12} = 0 \\ R_0 & \text{if } E_0 = 0 \end{cases} \tag{15.45}$$

If five or more of these estimates agree, then that majority gives the more probable value of C_0. If four of these estimates give $C_0 = 1$ and the other four give $C_0 = 0$, then the naïve threshold-decoding procedure fails, detecting four or more channel errors.

In order to resolve such a deadlock, we consider the error probabilities of the various estimates of Eq. (15.45). On a binary symmetric channel with error probability P,

$$\Pr(E_0 = 0) = 1 - P$$
$$\Pr(E_i + E_j = 0) = (1 - P)^2 + P^2$$

If $0 < P < \frac{1}{2}$, $(1 - P)^2 + P^2 < 1 - P$, so the estimate $C_0 = R_0$ is more likely to be correct than any other estimate of Eq. (15.45). For this reason a more sophisticated threshold decoder will break a four-to-four

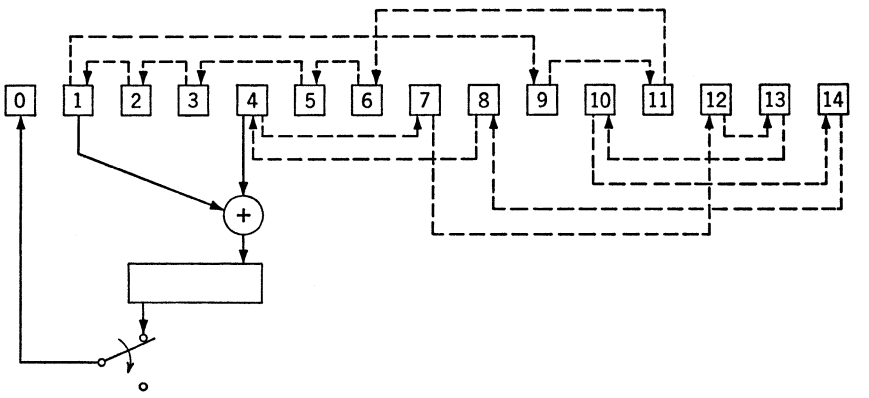


Fig. 15.05 Permutation-threshold decoder for maximal length FSR code, $n = 15$.

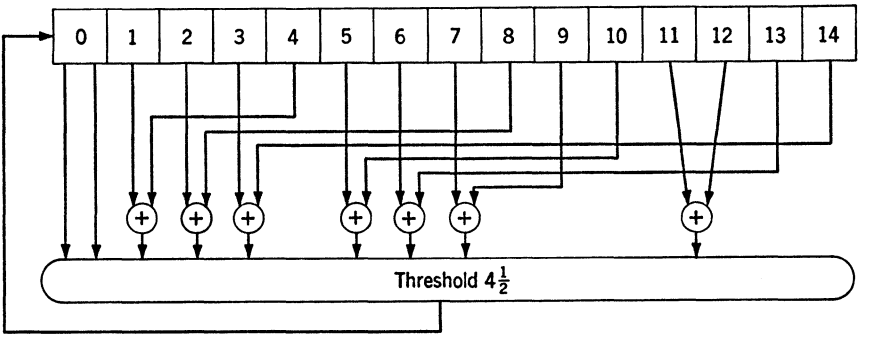


Fig. 15.06 Threshold decoder for maximal length FSR code, $n = 15$.

tie among the eight estimates of Eq. (15.45) by giving the estimate $C_0 = R_0$ an additional vote.

The decoder of Fig. 15.05 computes the eight estimates of Eq. (15.45) successively. First, $2R_0$ is added into an initially zero counter. Then, $R_1 + R_4$ is computed and added into the counter. The codeword is then permuted according to the permutation π_2^{-1} of Fig. 15.03. This permutation leaves position 0 fixed but moves positions 2 and 8 to positions 1 and 4. The new check sum is then computed and added into the counter. After all eight estimates have been counted, C_0 is determined as the majority decision. The codeword is then permuted by a cyclic shift, and the process is repeated to determine $C_1, C_2, \ldots, C_{n-1}$.

Although the decoding circuit of Fig. 15.05 requires only one sum of received digits, it requires seven permutations of the codeword for each decoded digit. A faster circuit (Fig. 15.06) computes all eight estimates simultaneously. The counter of Fig. 15.05 is replaced by a threshold element in Fig. 15.06. This is a memoryless circuit whose instantaneous output is a one iff more than T of its inputs are ones. Although the construction of a threshold element from the elementary AND, OR, and NOT elements of Fig. 2.01 is very expensive, requiring many components, in most technologies it is possible to obtain threshold elements as single components. For example, the threshold element may consist of a single iron torus. Each of its inputs may consist of a different wire which is wrapped around the torus a certain number of times. An additional bias wire is wrapped around the torus T times as many turns in the opposite direction. The bias wire always carries current. Each input wire carries current iff the input it represents has value 1. The output of the threshold element is then determined by the direction of flux in the torus.

In Fig. 15.07, the nine-input threshold element of Fig. 15.06 has

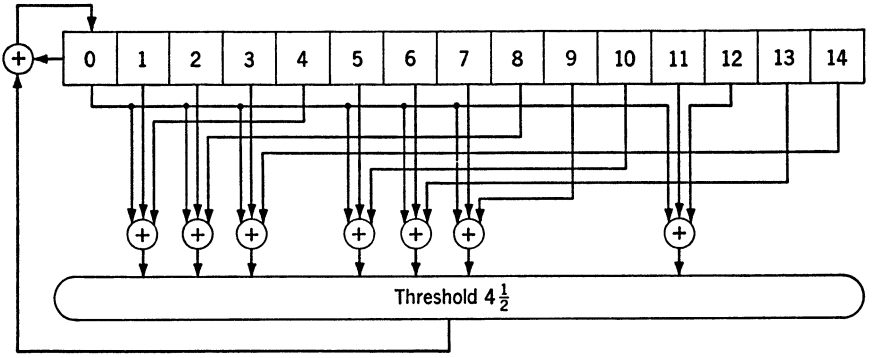


Fig. 15.07 Syndrome-threshold decoder for maximal length FSR code, $n = 15$.

been replaced by several additional adders and a seven-input threshold element whose output is a 1 iff five or more of its seven inputs are 1. The decoder of Fig. 15.07 is called a *syndrome threshold decoder* because the inputs to its threshold element are the syndrome digits $R_0 + R_1 + R_4$, $R_0 + R_2 + R_8$, . . . , $R_0 + R_{11} + R_{12}$, whose values are equal to $E_0 + E_1 + E_4, E_0 + E_2 + E_8$, . . . , $E_0 + E_4 + E_{12}$, respectively. The output of the threshold element is the decoder's decision of the value of E_0.

The t-error-correcting decoding algorithms of Figs. 15.04 to 15.07 depend on the existence of a set of $2t$ parity checks which are orthogonal on the position to be decoded. In most t-error-correcting codes, there is no set of $2t$ parity checks orthogonal on any given position. However, the following example will show that it is sometimes possible to determine the error pattern on the basis of majority decisions, if the decoder proceeds in several steps.

15.46 TWO-STEP THRESHOLD DECODING OF THE RM (31,16) CODE

Consider the 3-error-correcting second-order punctured RM code of length 31. We shall show that this code can be decoded by the syndrome threshold decoder of Fig. 15.08, where the connections within the "adding circuits" can be determined from Fig. 15.09 in a manner which we shall explain.

The purpose of all the circuitry shown in Fig. 15.08 is to obtain an estimate of E_0, the error in position 0. Once this is known, the digit in position 0 can be corrected, and a cyclic shift will enable the same circuitry to estimate $E_1, E_2, \ldots, E_{n-1}$.

The estimate of E_0 is obtained as the output of the final six-input threshold element. Each input to this final threshold element carries an estimate of the binary sum of four error values corresponding to one

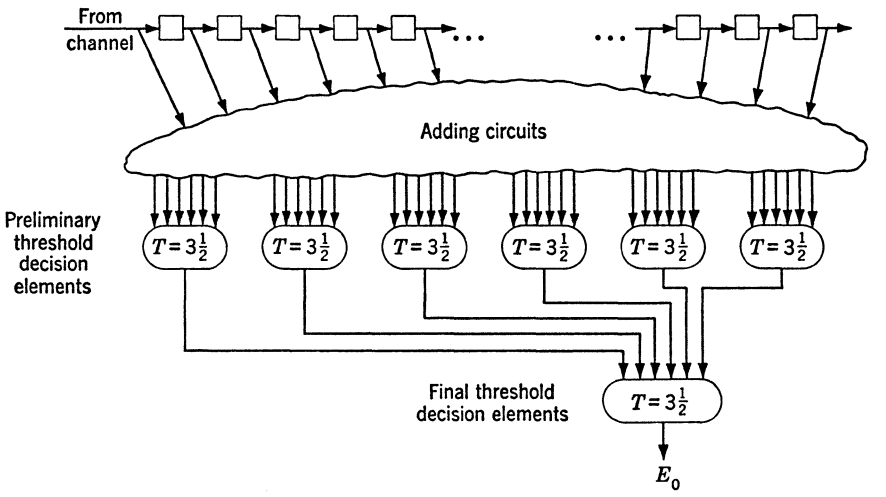


Fig. 15.08 Two-step threshold decoder for the RM (31,16) code.

of the sets listed in the leftmost column of the top half of Fig. 15.09. The first input carries an estimate of $E_0 + E_4 + E_{12} + E_{15}$; the second, an estimate of $E_0 + E_8 + E_{24} + E_{30}$; . . . ; the sixth, an estimate of $E_0 + E_9 + E_{11} + E_{13}$.

Each of the 4-tuples listed in Fig. 15.09 is an affine subspace of GF(2^5). Any two 4-tuples on the same row are translates of each other. Therefore, the union of any two 4-tuples on the same row of Fig. 15.09 is a three-dimensional affine subspace of GF(2^5). For example, the union of the first two 4-tuples in Fig. 15.09 is {0,4,12,15,1,17,8,13}, corresponding to the three-dimensional affine subspace

$$\{\alpha^0,\alpha^4,\alpha^{12},\alpha^{15},\alpha^1,\alpha^{17},\alpha^8,\alpha^{13}\}$$

The reader may verify that this is an affine subspace by reviewing Appendix A. According to Theorems 15.323 and 15.329, the code satisfies a parity-check equation over any three-dimensional affine subspace of GF(2^5) which does not include α^∞. In other words, the decoder can calculate the syndrome digits

$$s_1 = E_0 + E_4 + E_{12} + E_{15} + E_1 + E_{17} + E_8 + E_{13}$$

as

$$s_1 = R_0 + R_4 + R_{12} + R_{15} + R_1 + R_{17} + R_8 + R_{13}$$

Similarly, the decoder can calculate the binary sum of the error digits in positions {0,4,12,15,2,22,27,99}, {0,4,12,15,3,25,11,28}, . . . ,

{0,4,12,15,19,26,29,21}

Since these six parity-check sets are orthogonal on {0,4,12,15}, the output of the first preliminary threshold element will be $E_0 + E_4 + E_{12} + E_{15}$ unless there are more than three channel errors.

Each of the six preliminary threshold elements estimates the binary sum of the error digits in a set of positions listed in the leftmost column of Fig. 15.09. If there are no more than three channel errors, then all these estimates are correct, and the output of the final threshold element is the correct value of E_0. If there are four or more channel errors, the

Six 2-dimensional affine subspaces orthogonal on position 0	Their translates						The unused translate (including position ∞)
0 4 12 15	1 17 8 13	2 22 27 9	3 25 11 28	18 30 20 14	7 5 16 6	19 28 29 21	∞ 10 23 24
0 8 24 30	1 12 14 10	2 3 16 26	11 27 21 7	4 13 23 18	5 29 9 28	6 19 22 25	∞ 20 15 17
0 16 17 29	1 21 4 6	2 24 28 20	18 25 10 27	23 22 14 11	15 5 8 26	12 7 13 19	∞ 9 30 3
0 18 7 28	23 8 9 21	2 19 10 17	3 6 14 15	4 30 5 11	12 20 16 25	13 24 29 27	∞ 1 22 26
0 20 21 22	1 23 16 28	2 29 14 4	3 5 10 13	18 12 9 26	15 30 19 27	6 11 17 24	∞ 8 25 7
0 9 11 13	1 25 2 15	18 21 5 24	3 17 12 22	4 27 28 8	23 7 29 30	6 10 20 26	∞ 16 19 14
Three additional subspaces orthogonal on position 0							
0 1 3 27	23 20 19 5	2 11 8 12	7 26 14 24	4 17 25 9	15 13 28 22	10 30 21 16	∞ 18 29 6
0 2 6 23	1 11 29 20	14 21 28 17	3 8 18 19	4 22 16 24	15 9 7 10	13 25 26 30	∞ 5 27 12
0 5 14 25	1 19 24 9	23 27 17 26	3 20 7 4	6 12 28 30	15 16 18 11	8 29 10 22	∞ 2 13 21

Fig. 15.09 Affine subspaces of GF (2^5).

decoder of Fig. 15.08 may or may not decode correctly, as will be seen in Prob. 15.5.

The decoder of Fig. 15.08 is able to decode with two stages of threshold elements because the code is 2-step-orthogonalizable according to the following definition.

Definition 15.47 A t-error-correcting linear code is said to be 1-*step-orthogonalizable* iff for each $i = 0, 1, 2, \ldots, n-1$, the code satisfies a set of $2t$ parity checks which are orthogonal on position i.

A code is said to be *L-step-orthogonalizable* iff the code contains sets of positions $P^{(1)}, P^{(2)}, \ldots$ such that

1. For all i, the code contains $2t$ parity checks orthogonal on $P^{(i)}$.
2. The subcode of the original code which satisfies these additional parity checks $\left(\sum_{j \in P^{(i)}} C_j = 0 \text{ for all } i\right)$ is $(L-1)$-step-orthogonalizable.

Given a $(2^{m-r-1} - 1)$-error-correcting rth-order RM code of length 2^m, we may choose the sets $P^{(i)}$ to be the r-dimensional affine subspaces of GF(2^m). The subcode which satisfies the additional parity-check constraints is the $(r-1)$st-order RM code. The proof of Theorem 15.48 then follows immediately by induction on r.

Theorem 15.48 The $(2^{m-r-1} - 1)$-error-correcting rth-order RM code of length 2^m and the $(2^{m-r-1} - 1)$-error-correcting rth-order punctured RM code of length $2^m - 1$ are both $(r+1)$-step-orthogonalizable; the $(2^{m-r-1} - 1)$-error-correcting rth-order shortened RM code of length $2^m - 1$ is r-step-orthogonalizable.

As seen by the example of Sec. 15.46, some rth-order RM codes can be orthogonalized in fewer than $r+1$ steps. However, no shortened RM code with $r > 1$ can be 1-step-orthogonalized.

Threshold decoding proves inferior to algebraic decoding for certain codes which correct few errors and can be orthogonalized only in many steps. For example, it is possible to decode the Hamming codes by threshold decoding, since they are punctured RM codes. However, threshold decoders for the Hamming codes turn out to be more complicated than the decoder of Fig. 5.03.

On the other hand, threshold decoders prove decidedly easier to implement than algebraic decoders for many-error-correcting codes which can be easily orthogonalized. Unfortunately, most codes cannot be easily orthogonalized, and those which can often have minimum distances that are inferior to the minimum distances of comparable BCH codes. In many situations, however, these disadvantages for threshold decoding are outweighed by the advantage of easier implementation. This advantage is even greater in situations where one wishes to correct many patterns of more than t errors as well as all patterns of $\leq t$ errors. Algebraic decoding algorithms can do this only with extra embellishments, as indicated in Sec. 10.5. Threshold-decoding algorithms correct many patterns of $>t$ errors automatically.

15.5 ORTHOGONALIZABLE CODES BASED ON FINITE GEOMETRIES

The relative simplicity of certain threshold decoders has motivated the search for codes which can be orthogonalized in only a few steps. This search has led to the discovery of several important classes of orthogonalizable codes, including the Townsend-Weldon (1967) quasi-cyclic codes, the euclidean-geometry codes, and the projective-geometry codes.

E. Prange first studied certain projective-geometry codes in the late 1950s. He obtained many results on the distribution of weights of the codewords and coset leaders of certain binary codes based on projective geometries and on the permutation groups which preserve these codes. Unfortunately, Prange's work was never published. Weldon (1966) introduced difference-set codes, a subclass of the projective-geometry codes. The more general classes of code based on finite geometry were first presented by Rudolph (1964, 1967). Further results on codes based on finite geometries have been obtained by Chow (1967), Goethals and Delsarte (1968), Graham and MacWilliams (1966), MacWilliams and Mann (1968), Smith (1967, 1969), and Weldon (1968*a*,*b*).

Unfortunately, different authors have used the same terms with different meanings. The definitions used here represent only my own personal preference.

Definition 15.51 The (r,s)th-order *euclidean-geometry code* (EG code) of length $n = p^{sm}$ over GF(p) is the dual of the subfield subcode of the $[(q-1)(m-r-1)]$st-order GRM code over GF(q), where $q = p^s$.

Theorem 15.52 The (r,s)th-order EG code has minimum distance at least $d = (q^{m-r}-1)/(q-1)$. The code is $(r+1)$-step-orthogonalizable.

Proof According to Theorems 11.61 and 15.329, the indicator function of every $(r + 1)$-dimensional subspace of $GF(q^m)$ over $GF(q)$ is a codeword of the $[(q - 1)(m - r - 1)]$st-order GRM code. Since the only values of the coefficients of these codewords are 0 and 1, these codewords also lie in every subfield subcode of the GRM code. Therefore, the (r,s)th-order EG code satisfies a parity check over every $(r + 1)$-dimensional affine subspace.

Any given r-dimensional affine subspace of $GF(q^m)$ over $GF(q)$ has q^{m-r} translates. The given r-dimensional affine subspace and any of its $q^{m-r} - 1$ disjoint translates both lie in some $(r + 1)$-dimensional affine subspace which consists of the given r-dimensional affine subspace and $q - 1$ of its translates. Therefore, there are d different $(r + 1)$-dimensional affine subspaces which contain any given r-dimensional affine subspace. The indicator functions of these d affine subspaces form a set of d parity-check equations which are orthogonal on the given r-dimensional subspace.

Thus, if there are fewer than $d/2$ channel errors in the (r,s)th-order EG code, the decoder may determine the parity checks on all r-dimensional affine subspace with one level of threshold-decision elements. Theorem 15.52 then follows by induction of r. Q.E.D.

Definition 15.53 The (r,s)th-order p-ary *projective-geometry code* (PG code) of length $n = (q^m - 1)/(q - 1)$ is the dual of the subfield subcode of the cyclic code over $GF(q)$ whose generator's roots include $\alpha^{(q-1)i}$ iff $w_q([q - 1]i) > r(q - 1)$, where $q = p^s$, $w_q(x)$ is the sum of the digits of the q-ary representation of the integer x, and α is a primitive element of $GF(q^m)$.

Theorem 15.54 The (r,s)th-order p-ary PG code has minimum distance at least $d = (q^{r+1} - 1)/(q - 1)$, where $q = p^s$. The code is r-step-orthogonalizable.

Proof Let $q = p^s$ and $n = (q^m - 1)/(q - 1)$; let α be a primitive element of $GF(q^m)$; let $\gamma = \alpha^n$. Then γ is a primitive $(q - 1)$st root of unity. Every nonzero element in $GF(q^m)$ can be uniquely represented as $\alpha^i\gamma^j$ for appropriate choices of i and j, $0 \leq i < n$, $0 \leq j < q - 1$. Since γ is a primitive element of $GF(q)$, every i-dimensional linear subspace of $GF(q^m)$ over $GF(q)$ must be of the form $S = 0 \cup \{\alpha^i\gamma^k | j \in A, k = 0, 1, \ldots, q - 2\}$, where A is some subset of the integers between 0 and $n - 1$, inclusive. If $|S| = q^i$, then $|S - 0| =$

$q^i - 1$, and $|A| = (q^i - 1)/(q - 1) = \sum_{l=0}^{i-1} q^l$. The *projection* of S is the set of field elements $P = \{\alpha^j | j \in A\}$. A set P is said to be an *i-dimensional projective subspace* of GF(q^m) over GF(q) iff it is the projection of an $(i + 1)$-dimensional linear subspace of GF(q^m) over GF(q). In particular, if $0 \leq j < n$, α^j is a 0-dimensional projective subspace. The set consisting of $\{\alpha^0, \alpha^1, \alpha^2, \ldots, \alpha^{n-1}\}$ is an $(m - 1)$-dimensional projective subspace.

Just as the intersection of two linear subspaces is a linear subspace, so the intersection of two projective subspaces is a projective subspace. Just as any i-dimensional linear subspace is contained in $(q^{m-i} - 1)/(q - 1)$ distinct $(i + 1)$-dimensional linear subspaces, so any $(i - 1)$-dimensional projective subspace is contained in $(q^{m-i} - 1)/(q - 1)$ distinct i-dimensional projective subspaces.

If the decoder of a PG code could compute the parity check over the digits in the positions corresponding to any i-dimensional projective subspace, and if the received word contained no more than $(q^{m-i} - 1)/2(q - 1)$ errors, then the decoder could determine the parity check over any $(i - 1)$-dimensional projective subspace by a threshold decision among the known parity checks over the $(q^{m-i} - 1)/(q - 1)$ different i-dimensional projective subspaces which intersect only in the desired $(i - 1)$-dimensional projective subspace. Setting $i = m - r - 1$, we can then prove Theorem 15.54 from Lemma 15.541 by induction on r.

Lemma 15.541 The (r,s)th-order p-ary PG code of length $n = (q^m - 1)/(q - 1) = (p^{sm} - 1)/(p^s - 1)$ satisfies a parity-check equation over each $(m - 1 - r)$-dimensional projective subspace of GF(q^m) over GF(q).

Proof It suffices to show that the indicator function of any $(m - 1 - r)$-dimensional projective subspace of GF(q^m) over GF(q) is a codeword of the dual of the (r,s)th-order PG code. Let p be the set of field elements in an $(m - 1 - r)$-dimensional projective subspace of GF(q^m) over GF(q), and let S be the $(m - r)$-dimensional linear subspace of GF(q^m) over GF(q) of which P is the projection. Let $C^{(P)}(x)$ be the indicator function of P, defined by

$$C^{(P)}(x) = \sum_{\substack{i, \\ \alpha^i \in P}} x^i$$

Similarly, let $C^{(S)}(x)$ be the indicator function of $S - 0$ (S with the 0 point excluded), defined by

$$C^{(S)}(x) = \sum_{\substack{i \\ \alpha^i \in S-0}} x^i$$

According to Theorem 15.329, $C^{(S)}(x)$ is a codeword of the cyclic code of length $q^m - 1$ whose generator's roots include α^j for all j for which $0 < w_q(j) \leq r(q-1)$. This implies that if $w_q(j) \leq r(q-1)$, then

$$0 = C^{(S)}(\alpha^j) = \sum_{\substack{\xi \\ \xi \in S-0}} \xi^j$$

If $j = (q-1)i$, we have

$$\begin{aligned} 0 &= \sum_{\xi \in S} \xi^{(q-1)i} = \sum_{\substack{a, \\ a \in GF(q)-0}} \sum_{\eta \in P} (\eta a)^{(q-1)i} \\ &= \sum_{\substack{a, \\ a \in GF(q)-0}} \sum_{\substack{\eta, \\ \eta \in P}} \eta^{(q-1)i} = (q-1) \sum_{\substack{\eta, \\ \eta \in P}} \eta^{(q-1)i} \\ &= -\sum_{\substack{\eta, \\ \eta \in P}} \eta^{(q-1)i} = -C^{(P)}(\alpha^{(q-1)i}) \end{aligned}$$

We have shown that if P is any $(m-1-r)$-dimensional projective subspace of GF(q^m) over GF(q), with indicator function $C^{(P)}(x)$, and if $w_q([q-1]i) \leq r(q-1)$, then $C^{(P)}(\alpha^{(q-1)i}) = 0$. Therefore, $C^{(P)}(x)$ is a codeword of the dual of the (r,s)th-order PG code. Q.E.D.

Example 15.55 Let us consider the (1,2)th-order binary PG code of length 21, for which $p = 2$, $s = 2$, $q = 4$, $m = 3$, $n = (4^3 - 1)/(4 - 1)$. Let ∂ be a primitive element of GF(4), and let α be a root of the primitive cubic over GF(4), $x^3 + x^2 + \partial x + \partial^2$. Over GF(2), the minimal polynomial of α is $M^{(1)}(x) = (x^3 + x^2 + \partial x + \partial^2)(x^3 + x^2 + \partial^2 x + \partial) = x^6 + x + 1$. The minimal polynomial of α^3 is $M^{(3)}(x) = x^6 + x^4 + x^2 + x + 1$; the minimal polynomial of α^9 is $M^{(9)}(x) = x^3 + x^2 + 1$. The integers $3i$ (mod 63) for which $w_4(3i) > 3$ are (base 4) $3i = 033$, 123, 132, 222, and their cyclic shifts. Base 2, these numbers are 001111, 011011, 011110, 101010, and their cyclic double shifts. Thus, the (1,2)th-order binary PG code has $h(x) = M^{(15)}(x)M^{(27)}(x)M^{(21)}(x)$ and $g(x) = M^{(0)}(x)$

$M^{(3)}(x)M^{(9)}(x) = (x + 1)(x^6 + x^4 + x^2 + x + 1)(x^3 + x^2 + 1)$. The check polynomial of this code is given by $h(x) = (x^{21} + 1)/g(x) = 1 + x^2 + x^7 + x^8 + x^{11}$. Every nonzero element in GF(4^3) may be expressed in the form $\alpha^i = A_i\alpha^2 + B_i\alpha + C_i$, where A_i, B_i, $C_i \in$ GF(4). For $i = 0, 1, 2, \ldots, 20$, we have

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
A_i	0	0	1	1	∂^2	∂	0	1	0	∂	1	∂	1	1	∂	∂^2	∂^2	∂^2	0	∂^2	1
B_i	0	1	0	∂	1	∂	1	1	∂	∂^2	∂^2	∂^2	0	∂^2	1	0	0	∂^2	∂^2	∂	1
C_i	1	0	0	∂^2	∂^2	∂	1	0	∂^2	0	1	∂^2	1	∂^2	∂^2	1	∂	∂	∂	0	∂

The set consisting of those elements of GF(4^3) which are of the form $\beta\alpha + C$; $B,C \in$ GF(4) is a two-dimensional linear subspace of GF(4^3) over GF(4). The projection of this subspace consists of the one-dimensional projective subspace $P = \{\alpha^0,\alpha^1,\alpha^6,\alpha^8,\alpha^{18}\}$, whose indicator function is $C^{(P)}(x) = 1 + x + x^6 + x^8 + x^{18} = (1 + x + x^3 + x^5 + x^7)h(x^{-1})x^{11}$. Since $C^{(P)}(x)$ is a multiple of $h(x^{-1})$, the code satisfies a parity-check equation over the positions of the one-dimensional projective subspace P.

The threshold-decoding procedures for EG and PG codes depend only on the fact that the codes satisfy parity-check equations over all affine or projective subspaces of a given dimension. There are other codes which also satisfy parity-check equations over all affine or projective subspaces of a given dimension, but Delsarte, Goethals, and MacWilliams (1968) have shown that any such code is a subcode of the EG or PG code defined by Definition 15.51 or 15.53.

Historically, the indicator function of a projective subspace P has usually been denoted by $\theta(x)$ rather than by $C^{(P)}(x)$. This polynomial has some interesting properties, as indicated by the following theorem.

Theorem 15.56 Let P be a one-dimensional projective subspace of GF(q^m) over GF(q), and assume P includes the point α^0.

Let $n = (q^m - 1)/(q - 1)$.
Let $\theta(x) = \sum_{\substack{i, \\ \alpha^i \in P}} x^i$.

Then over the rational integers,

$$\theta(x)\theta(x^{-1}) \equiv q + 1 + \xi(x) \bmod x^n - 1$$

where $\xi(x)$ is the sum of $q(q + 1)$ distinct nonzero powers of x.

Proof

$$\theta(x)\theta(x^{-1}) = \sum_{\substack{i,\\ \alpha^i \in P}} x^{-i}\theta(x)$$

Mod $x^n - 1$, each of the polynomials $\theta(x)$, $x^{-1}\theta(x)$, $x^{-2}\theta(x)$, . . . is an indicator function of some one-dimensional projective subspace of GF(q^m) over GF(q). Since the intersection of any two one-dimensional projective subspaces is a zero-dimensional projective subspace consisting of a single point, $x^{-i}\theta(x)$ and $x^{-j}\theta(x)$ mod $x^n - 1$ can have at most one term in common unless $i \equiv j \bmod n$. If $\alpha^i, \alpha^j \in P$, then x^0 is the only common term in the polynomials $x^{-i}\theta(x)$ and $x^{-j}\theta(x)$. Q.E.D.

Corollary 15.561 With $m = 3$ and the hypothesis of Theorem 15.56,

$$\theta(x)\theta(x^{-1}) \equiv q + 1 + \sum_{i=1}^{n-1} x^i \bmod x^n - 1$$

Proof With $n = (q^3 - 1)/(q - 1) = q^2 + q + 1$, there are only $q(q + 1)$ distinct nonzero powers of x mod $x^n - 1$. Q.E.D.

With P, a projective subspace of GF(q^m) over GF(q), we may associate D, a set of integers mod n, by the definition

$$d \in D \quad \text{iff } \alpha^d \in P \tag{15.562}$$

$$\text{Since } \theta(x) = \sum_{\substack{d,\\ d\in D}} x^d,$$

$$\theta(x)\theta(x^{-1}) = \sum_{\substack{d,\\ d\in D}} \sum_{\substack{d',\\ d'\in D}} x^{d-d'}$$

Thus, according to Corollary 15.561, if P is a one-dimensional projective subspace of GF(q^3) over GF(q), then the differences of the $q(q + 1)$ distinct ordered pairs of elements of D must include every nonzero integer mod $q^2 + q + 1$ exactly once. For this reason, the set D is said to be a *perfect simple difference set*† mod $q^2 + q + 1$. If q is a prime power, then a perfect simple difference set mod $q^2 + q + 1$ may be constructed from a one-dimensional projective subspace of GF(q^3) over GF(q) via the definition (15.562). Difference sets of this type were first introduced by

† The equivalent term, *planar difference set,* is also frequently used.

Singer (1938) from a slightly different point of view. Surprisingly, the Singer difference sets include all known perfect simple difference sets. Evans and Mann (1951) have shown that there are no non-Singer difference sets with $|D| \leq 1{,}600$, but no one has yet been able to prove (or disprove) the nonexistence of a perfect simple difference set mod $q^2 + q + 1$ for certain large non-prime-power values of q.

Because of their relationship to perfect simple difference sets, $(1,s)$th-order p-ary PG codes of length $n = (p^{3s} - 1)/(p^s - 1)$ are called *difference-set codes.* Like all $(1,s)$th-order PG codes, difference set codes are one-step-orthogonalizable, and the set of parity checks orthogonal on a given digit may be taken as cyclic shifts of a single parity-check-generating polynomial $\theta(x)$.

Many EG and PG codes are proper subcodes of BCH codes with the same length and designed distance. In general, most EG and PG codes have fewer information symbols than comparable BCH codes. This loss in information rate is the price we must pay for the advantages of threshold decoding. Fortunately, at short block lengths, the price is not very high. All binary EG or PG codes of lengths ≤ 32 are identical to BCH codes. The comparison of punctured EG codes of length 63 with comparable BCH codes, due to Weldon (1968*b*), is as follows:

				Information symbols		*Check symbols*	
d	*r*	*s*	*m*	*EG*	*BCH*	*EG*	*BCH*
3	4	1	6	57	57	6	6
5	1	2	3	48	51	15	12
7	3	1	6	42	45	21	18
9	0	3	2	37	39	26	24
15	2	1	6	22	24	41	39
21	0	2	3	13	18	50	45
31	1	1	6	7	7	56	56

The EG and PG codes deteriorate much faster than BCH codes with increasing block lengths. In general, the enumeration of information symbols in EG and PG codes appears fairly difficult. We consider only the two special cases of greatest practical interest, a class of PG codes which includes the difference set codes, and the one-step-orthogonalizable EG codes. Another class of EG codes is considered in Prob. 15.8. In order to determine the number of information symbols in these codes, we must first derive certain properties of the weight function. In some cases, we must distinguish between $w_p(x)$, the sum of the digits of the p-ary expansion of the integer x, and $w_q(x)$, the sum of the digits of the q-ary expansion of the integer x.

Only Lemmas 15.571 to 15.573 are needed for Theorems 15.58 and 15.599. The remaining lemmas are included because they prove helpful in calculating the number of information symbols in certain higher-order EG and PG codes.

Lemma 15.571 If $q = p^s$, $0 \leq j < sm$, $0 \leq x < q - 1$, and $\lfloor xp^j \rfloor$ is defined by $0 \leq \lfloor xp^j \rfloor < q^m - 1$ and $\lfloor xp^j \rfloor \equiv xp^j \bmod q^m - 1$, then $w_q(\lfloor xp^j \rfloor) = w_q(xp^j)$.

Proof Let $x = \sum_{i=0}^{ms-1} X_i p^i$, $0 \leq X_i < p$, $xp^j = \sum_{i=0}^{ms-1} X_i p^{i+j} = \sum_{i=0}^{ms-j-1} X_i p^{i+j} + q^m \sum_{i=ms-j}^{ms-1} X_i p^{i+j-ms}$, so $w_q(\lfloor xp^j \rfloor) = \sum_{i=0}^{ms-j-1} X_i p^{i+j} + \sum_{i=ms-j}^{ms-1} X_i p^{i+j-ms} = w_q(xp^j)$. Q.E.D.

Lemma 15.572 If $q = p^s$, $0 \leq j < s$, and $x = \sum_{l=0}^{m-1} \sum_{k=0}^{s-1} X_{k,l} p^k q^l$, where $0 \leq X_{k,l} < p$, then $w_q(xp^j) = \sum_{k=s-j}^{s-1} Y_k p^{k+j-s} + \sum_{k=0}^{s-j-1} Y_k p^{k+j}$, where $Y_k = \sum_{l=0}^{m-1} X_{k,l}$.

Proof $xp^j = \sum_{l=0}^{m-1} \sum_{k=s-j}^{s-1} X_{k,l} p^{k+j-s} q^{l+1} + \sum_{l=0}^{m-1} \sum_{k=0}^{s-j-1} X_{k,l} p^{k+j-s} q^l$, so $w_q(xp^j) = \sum_{l=0}^{m-1} \sum_{k=s-j}^{s-1} X_{k,l} p^{k+j-s} + \sum_{l=0}^{m-1} \sum_{k=0}^{s-j-1} X_{k,l} p^{k+j}$. Q.E.D.

Lemma 15.573 $w_q(x) \equiv x \bmod q - 1$.

Proof Since $q^i \equiv 1 \bmod q - 1$ for all i, $\sum_i X_i \equiv \sum_i X_i q^i$. Q.E.D.

Lemma 15.574 $w_q(x) \leq x$, with equality iff $x < q$.

Proof Since $1 \leq q^i$, with equality iff $i = 0$, $\sum_i X_i \leq \sum_i X_{iq^i}$. Q.E.D.

Lemma 15.575 $w_q(x + y) \leq w_q(x) + w_q(y)$, with equality iff $X_i + Y_i < q$ for all i.

Proof Let $c_{-1} = 0$, and for $i \geq 0$, define c_i and Z_i by

$$c_i q + Z_i = c_{i-1} + X_i + Y_i \tag{15.5751}$$

where $0 \leq Z_i < q$ and $0 \leq c_i$. Multiplying Eq. (15.5751) by q^i and summing on i gives $\sum_i Z_i q^i = X + Y$, so $w_q(x + y) = \sum_i Z_i$. Summing Eq. (15.5751) directly gives $(q - 1) \sum_i c_i + \sum_i Z_i = \sum_i X_i + \sum_i Y_i$, so $w_q(x + y) = w_q(x) + w_q(y) - (q - 1) \sum_i c_i$. Q.E.D.

Lemma 15.576 If $x^{(0)}$, $x^{(1)}$, $x^{(2)}$, . . . are arbitrary integers, $x^{(i)} = \sum_j X_j^{(i)} q^j$, $0 \leq X_j^{(i)} < q$, then $w_q\left(\sum_{i=0}^{k} x^{(i)}\right) \leq \sum_{i=0}^{k} w_q(x^{(i)})$, with equality iff $\sum_{i=0}^{k} X_j^{(i)} < q$ for all j.

Proof Lemma 15.576 follows directly from Lemma 15.575 by induction on k. Q.E.D.

Lemma 15.577 If $q = p^s$, $w_p(x) \geq w_p(w_q(x))$.

Proof Let $x = \sum_j \sum_{i=0}^{s-1} X_{i,j} p^i q^i$, $0 \leq X_{i,j} < p$. Then $w_q(x) = \sum_j \sum_{i=0}^{s-1} X_{i,j} p^i$. According to Lemma 15.576, $w_p\left(\sum_j \sum_{i=0}^{s-1} X_{i,j} p^i\right) \leq \sum_j \sum_{i=0}^{s-1} w_p(X_{i,j} p^i) = \sum_j \sum_{i=0}^{s-1} X_{i,j} = w_p(x)$. Q.E.D.

Lemma 15.578 If $a > 0$ and $s > 0$, $w_p(a[p^s - 1]) \geq s(p - 1)$.

Proof Let $q = p^s$, let $x^{(0)} = a(p^s - 1)$, and let $x^{(i+1)} = w_q(x^{(i)})$. According to Lemma 15.574, $x^{(i+1)} \leq x^{(i)}$, with equality iff $x^{(i)} < q$. We define $x^{(\infty)} = \lim_{i \to \infty} x^{(i)}$. It is clear that for all sufficiently large i, $x^{(i)} = x^{(\infty)}$. According to Lemma 15.573, $x^{(i+1)} \equiv x^{(i)} \bmod (q - 1)$, so $x^{(\infty)} \equiv x^{(0)} \equiv 0 \bmod q - 1$. Since $x^{(i)} \neq 0$ implies that $x^{(i+1)} \neq 0$, $x^{(\infty)} \neq 0$, so $x^{(\infty)} = q - 1$. According to Lemma 15.577, $w_p(x^{(i+1)}) = w_p(w_q(x^{(i)})) \leq w_p(x^{(i)})$, so $w_p(x^{(\infty)}) \leq w_p(x^{(0)})$. Since $x^{(\infty)} = q - 1 = (p - 1) \sum_{i=0}^{s-1} p^i$, $w_p(x^{(\infty)}) = (p - 1)s$. Q.E.D.

Theorem 15.58 [Graham-MacWilliams (1966), MacWilliams-Mann (1968), Delsarte-Goethals (1968), and Smith (1967, 1969)]. The $(m - 1,s)$th-order p-ary PG code of length $n = (p^{sm} - 1)/(p^s - 1)$ has $1 + \binom{p + m - 2}{p - 1}^s$ check symbols and $n - 1 - \binom{p + m - 2}{p - 1}^s$ information symbols.

Proof Let $q = p^s$, and let β be a primitive nth root of unity, $\beta \in \mathrm{GF}(q^m)$. According to Definition 15.53, Theorem 14.74, and Lemma 15.571, β^i is a root of the code's generator polynomial iff

$$w_q([q - 1]ip^j) \leq q - 1 \qquad \text{for all } j \tag{15.581}$$

To count the number of check symbols in the code, we need only count the number of integers i mod n which satisfy condition (15.581). It is easier, however, to count the number of integers $x = [q - 1]i$, mod $q^m - 1$, which satisfy conditions (15.582)

$$\begin{aligned} &0 \leq x < q^m - 1 \\ &(q - 1) | x \\ &w_q(xp^j) \leq q - 1 \qquad \text{for all } j \end{aligned} \tag{15.582}$$

If $(q - 1)|x$, then $(q - 1)|xp^j$ for all j. According to Lemma 15.573, $(q - 1)|xp^j$ iff $w_q(xp^j) \equiv 0 \bmod q - 1$. We may therefore replace conditions (15.582) by conditions (15.583)

$$\begin{aligned} &0 \leq x < q^m - 1 \\ &w_q(xp^j) = q - 1 \text{ or } 0 \qquad \text{for all } j \end{aligned} \tag{15.583}$$

If $m > 1$, $x = q^m - 1$ does not satisfy $w_q(x) = q - 1$, so we may replace $0 \leq x < q^m - 1$ by $0 \leq x < q^m$. If $w_q(xp^j) = 0$ for

any j, then $x = 0$. Therefore, any nonzero solution of conditions (15.583) must also be a solution of conditions (15.584)

$$\begin{aligned} &0 \leq x < q^m \\ &w_q(xp^j) = q - 1 \qquad \text{for all } j \end{aligned} \tag{15.584}$$

We now let $x = \sum_{l=0}^{m-1} \sum_{k=0}^{s-1} X_{k,l} p^k q^l$, where $0 \leq X_{k,l} < p$. According to Lemma 15.572,

$$w_q(xp^j) = \sum_{k=s-j}^{s-1} Y_k p^{k+j-s} + \sum_{k=0}^{s-j-1} Y_k p^{k+j} \tag{15.585}$$

where

$$Y_k = \sum_{l=0}^{m-1} X_{k,l} \tag{15.586}$$

If, for some K, $Y_K \geq p$, then according to Eq. (15.585), $w_q(xp^{s-K-1}) \geq Y_K p^{s-1} \geq q$, violating conditions (15.584). Equation (15.585) also implies that $w_q(xp^{s-K}) \equiv Y_K \bmod p$. Since $q - 1 \equiv -1 \bmod p$, we deduce that $Y_K = p - 1$, for all K. Condition (15.584) is therefore equivalent to

$$\sum_{l=0}^{m-1} X_{k,l} = p - 1 \qquad \text{for } k = 0, 1, \ldots, s - 1 \tag{15.587}$$

We may represent the s-tuple $X_{k,0}; X_{k,1}; \ldots; X_{k,m-1}$ as a sequence of $X_{k,0}$ ones followed by a comma followed by $X_{k,1}$ ones followed by another comma followed by $X_{k,2}$ ones, ... Thus, the number of ways of selecting nonnegative integers $X_{k,0}; X_{k,1}; \ldots; X_{k,m-1}$ which sum to $p - 1$ is the number of ways of placing $p - 1$ ones and $m - 1$ commas into $p + m - 2$ positions. This number is $\binom{p+m-2}{p-1}$, so the number of x's which satisfy condition (15.587) is $\binom{p+m-2}{p-1}^s$, which is the number of nonzero solutions to conditions (15.583). Q.E.D.

To conclude this section, we present a procedure for obtaining an enumeration of the check symbols in (0,s)th-order EG codes.

15.59 ENUMERATING THE CHECK SYMBOLS IN (0,s)th-ORDER EG CODES

Let $q = p^s$, and let α be a primitive element of GF(q^m). According to Definition 15.51, Property 14.74, and Lemma 15.571, α^x is a root of

the generator polynomial of the punctured $(0,s)$th-order p-ary EG code of length $q^m - 1$ iff

$$w_q(xp^j) \geq q - 1 \qquad \text{for all } j \tag{15.591}$$

The number of check symbols in the punctured EG code is therefore the number of integers x such that x satisfies condition (15.591) and $0 \leq x < q^m - 1$. The unpunctured EG code also has an overall parity-check symbol, which we may associate with the integer $x = q^m - 1$. Since $w_q([q^m - 1]p^j) \geq q - 1$ for all j, the number of check symbols in the EG code is the number of integers x which satisfy condition (15.591) and $0 \leq x < q^m$. If we set $x = \sum_{l=0}^{m-1} \sum_{k=0}^{s-1} X_{k,l} p^k q^l$, where $0 \leq X_{k,l} < p$, then Lemma 15.572 transforms condition (15.591) and $0 \leq x < q^m$ to

$$\sum_{k=s-j}^{s-1} Y_k p^{k+j-s} + \sum_{k=0}^{s-j-1} Y_k p^{k+j} \geq q - 1 \qquad \text{for all } j \tag{15.592}$$

$$Y_k = \sum_{l=0}^{m-1} X_{k,l} \qquad 0 \leq X_{k,l} < p \tag{15.593}$$

Let B_y be the number of m-tuples $X_0, X_1, \ldots, X_{m-1}$ which solve the equations

$$y = \sum_{l=0}^{m-1} X_l \qquad 0 \leq X_l < p \qquad \text{for } l = 0, 1, 2, \ldots, m-1$$

The number B_y is implicitly dependent on p and m. If $p = 2$, $B_y = \binom{m}{y}$. If $y < p$, then $B_y = \binom{y + m - 1}{y}$, as in the concluding paragraph of the proof of Theorem 15.58. If $2 < p \leq y$, the explicit expression for B_y is more complicated. Recursive formulas and generating functions for B_y are given by Riordan (1958), where B_y is called the number of compositions of $y + m$ into exactly m parts, no part exceeding p.

For a given sequence of nonnegative integers $Y_{s-1}, Y_{s-2}, \ldots, Y_0$ which satisfies condition (15.592), there are $\prod_{i=0}^{s-1} B_{Y_i}$ choices of X_k which satisfy condition (15.593).

Condition (15.592) is actually a statement about the sequence $Y_{s-1}, Y_{s-2}, \ldots, Y_0$ and all its cyclic shifts. As in Chap. 12, we consider this sequence as a concatenation of several basic sequences, defined by Defition 15.594.

Definition 15.594 A sequence $Y_{k-1}, Y_{k-2}, \ldots, Y_0$ is said to be *deficient* iff $\sum_{i=1}^{j} Y_{k-i}p^{j-i} < p^j - 1$ for $j = 1, 2, \ldots, k$. If $Y_{k-1}, Y_{k-2}, \ldots, Y_0$ is deficient, its *deficiency* is defined as $p^k - 1 - \sum_{i=1}^{k} Y_{k-i}p^{k-i}$. A *basic* sequence is a nondeficient sequence all of whose proper nonempty prefixes are deficient.

Let $D_j^{(i)}$ denote the number of choices of $X_{k,l}$; $0 \leq l < m$; $0 \leq k < j$, for which the resulting sequence $Y_{j-1}, Y_{j-2}, \ldots, Y_0$ [defined by (15.593)] has deficiency i. Let $D^{(i)}(z) = \sum_{j=0}^{\infty} D_j^{(i)} z^j$. Corresponding to the fact that the empty sequence has deficiency 0, we define

$$D^{(0)}(z) = 1 \tag{15.595}$$

If $Y_{j-1}, Y_{j-2}, \ldots, Y_1$ has deficiency i and $Y_0 < pi + p - 1$, then $Y_{j-1}, Y_{j-2}, \ldots, Y_1, Y_0$ has deficiency l iff $l = pi + p - 1 - Y_0$. From this we deduce that for $l \geq 1$

$$D^{(i)}(z) = z \sum_{i} B_{pi+p-1-l} D^{(i)}(z) \tag{15.596}$$

For any given p and m, we may find the generating functions $D^{(1)}(z), D^{(2)}(z), \ldots$ by solving the simultaneous linear equations (15.595) and (15.596).

If V_j denotes the number of choices of $X_{k,l}$; $0 \leq l < m$; $0 \leq k < j$, for which the resulting sequence $Y_{j-1}, Y_{j-2}, \ldots, Y_0$ [defined by (15.593)] is a basic sequence, then the generating function $V(z) = \sum_{j=1}^{\infty} V_j z^j$ is evidently given by

$$\begin{aligned} V(z) &= z \sum_{i=0}^{\infty} \Big(\sum_{j=pi+p-1}^{\infty} B_j \Big) D^{(i)}(z) \\ &= z \Big(\sum_{j=0}^{\infty} B_j - \sum_{j=0}^{p-2} B_j \Big) + z \sum_{i=1}^{\infty} \Big(\sum_{j=ip+p-1}^{\infty} B_j \Big) D^{(i)}(z) \\ &= z \left[p^m - \binom{m+p-2}{m} \right] + z \sum_{i=1}^{\infty} \Big(\sum_{j=ip+p-1}^{\infty} B_j \Big) D^{(i)}(z) \end{aligned} \tag{15.597}$$

From Definition 15.594, we may deduce Lemma 15.598, whose proof is directly analogous to the proof of Theorem 12.33.

Lemma 15.598 Any sequence of nonnegative integers $Y_{s-1}, Y_{s-2}, \ldots, Y_0$ which satisfies condition (15.592) can be uniquely decomposed into a concatenation of basic sequences, including an end-around sequence.

Just as Theorem 12.33 leads directly to Theorem 12.35 [or Eq. (12.61), in generating-function notation], so Lemma 15.598 leads directly to Theorem 15.599.

Theorem 15.599 If $r(p,m,\gamma,s)$ denotes the number of check symbols in the (γ,s)th-order p-ary EG code of length $n = p^{sm}$, then

$$\sum_{s=0}^{\infty} r(p,m,0,s)z^s = \frac{zV'(z)}{1 - V(z)}$$

where $V(z)$ is given by Eq. (15.597) and $V'(z) = dV/dz$.

EXAMPLES:

If $m = 1$,

$$V(z) = z \qquad r(z) = \sum_{s=1}^{\infty} z^s$$

If $m = 2$,†

$$V(z) = z\left(p^2 - \binom{p}{2}\right) = z\binom{p+1}{2}$$

$$r(z) = \sum_{s=1}^{\infty} \binom{p+1}{2}^s z^s$$

If $m = 3$ and $p = 2$,

$$D^{(1)}(z) = \frac{z}{1 - 3z}$$

$$V(z) = 7z + \frac{z^2}{1 - 3z}$$

$$r(z) = \frac{7z - 40z^2 + 60z^3}{(1 - 10z + 20z^2)(1 - 3z)} = z\left(\frac{10 - 40z}{1 - 10z + 20z^2} - \frac{3}{1 - 3z}\right)$$

15.6 CONVOLUTIONAL CODES—A SURVEY

15.61 INTRODUCTION

An encoder for a block code of rate R and length n partitions the digits it receives from the data source into blocks of length $k = Rn$. Each block is encoded into a longer block of length n, and the entire block is then

† This case coincides with a special case of Prob. 15.8.

transmitted across the noisy channel. An encoder for another type of code, called a *convolutional code* or *recurrent code*, avoids any partitioning of the digits from the data source into large blocks. Instead, the digits from the data source are partitioned into small blocks of length k. After accepting k digits from the data source, the encoder transmits n digits, which depend not only on the k data digits which were just received but on many previously received data digits as well. Specifically, if the input to the encoder is the data sequence [over GF(q)]

$$I_1^{(1)}, I_1^{(2)}, \ldots, I_1^{(k)}, I_2^{(1)}, I_2^{(2)}, \ldots, I_2^{(k)}, I_3^{(1)}, \ldots$$

then the output of the encoder is the sequence

$$C_1^{(1)}, C_1^{(2)}, \ldots, C_1^{(n)}, C_2^{(1)}, C_2^{(2)}, \ldots, C_2^{(n)}, C_3^{(1)}, \ldots$$

where

$$C_i^{(j)} = \begin{cases} I_i^{(j)} & \text{if } 1 \leq j \leq k \\ \sum_{l=1}^{k} \sum_{h=0}^{m} H_{l,h}^{(j)} I_{i-h}^{(l)} & \text{if } k < j \leq n \end{cases}$$

(We assume $I_{i-j}^{(l)} = 0$ if $j \geq i$.) As in the case of linear block codes, we denote the received digits by $R_i^{(j)}$, the channel errors by $E_i^{(j)} = R_i^{(j)} - C_i^{(j)}$. For $j = k+1, k+2, \ldots, n$, we define the *syndrome digits* by

$$S_i^{(j)} = R_i^{(j)} - \sum_{l=1}^{k} \sum_{h=0}^{m} H_{l,h}^{(j)} R_{i-h}^{(l)}$$

Clearly,

$$S_i^{(j)} = E_i^{(j)} - \sum_{l=1}^{k} \sum_{h=0}^{m} H_{l,h}^{(j)} E_{i-h}^{(l)}$$

I call the polynomials

$$H^{(j)}(y,z) = \sum_{l=1}^{k} \sum_{h=0}^{m} H_{l,h}^{(j)} y^l z^h$$

the *check polynomials* of the code. (Some authors call them generator polynomials.) The *memory* of the encoder is m blocks; the *constraint length* is $N = n(m+1)$. The constraint length of a convolutional code plays a role roughly comparable to the block length of a block code.

Historically, convolutional codes were first introduced by Elias (1955). They have also been studied by Hagelbarger (1959), Massey (1963), Wyner and Ash (1963), Sullivan (1966), Reddy (1968), and others.

Example 15.62 [Hagelbarger (1959)] Let $q = 2$, $R = \frac{1}{2}$, $k = 1$, $n = 2$, $H^{(2)}(y,z) = y(1 + z^2 + z^5 + z^6)$. The memory is six blocks; the constraint length is 14 digits. An encoder for this code is shown in Fig. 15.10.

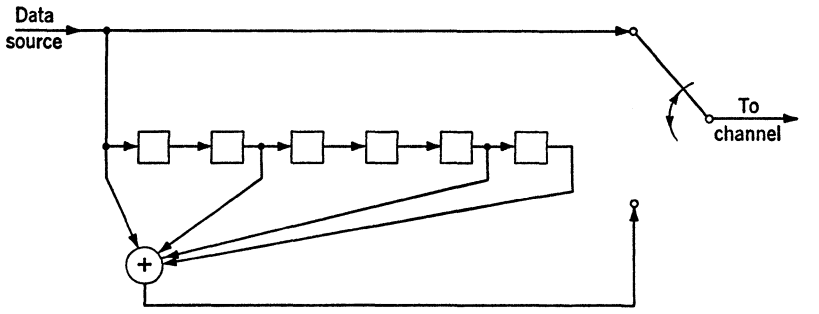


Fig. 15.10 Encoder for a convolutional code.

15.63 THRESHOLD DECODING; DIRECT DECODING VERSUS FEEDBACK DECODING

Among the parity-check equations satisfied by the code of Example 15.62 are

$$\begin{bmatrix} S_i^{(2)} \\ S_{i+2}^{(2)} \\ S_{i+5}^{(2)} \\ S_{i+6}^{(2)} \end{bmatrix} = \begin{bmatrix} E_i^{(2)} \\ E_{i+2}^{(2)} \\ E_{i+5}^{(2)} \\ E_{i+6}^{(2)} \end{bmatrix} + \begin{bmatrix} 1100101000000 \\ 0011001010000 \\ 0000011001010 \\ 0000001100101 \end{bmatrix} \begin{bmatrix} E_{i-6}^{(1)} \\ E_{i-5}^{(1)} \\ E_{i-4}^{(1)} \\ \cdot \cdot \cdot \\ E_{i+6}^{(1)} \end{bmatrix}$$

Since these checks are orthogonal on $E_i^{(1)}$, this code may be decoded by the threshold decoder shown in Fig. 15.11. As the message digit $R_i^{(1)}$ is received from the channel, it is read into the message buffer. As the check digit $R_i^{(2)}$ is received from the channel, it is added to $\sum_{j=1}^{6} H_{1,j} R_{i-j}^{(1)}$ and stored in the syndrome buffer. When $R_i^{(1)}$ is about to leave the message buffer, the majority-decision element† estimates $E_i^{(1)}$ from a majority vote among the four syndrome digits $S_i^{(2)}$, $S_{i+2}^{(2)}$, $S_{i+5}^{(2)}$, $S_{i+6}^{(2)}$. If three or four of these digits are 1, then $E_i^{(1)}$ is taken as 1; if none or only one of these digits is 1, then $E_i^{(1)}$ is taken as 0. The decoder will decode $R_i^{(1)}$ correctly unless there are at least three errors among the 26 consecutive digits $R_{i-6}^{(1)}$, $R_{i-6}^{(2)}$, $R_{i-5}^{(1)}$, . . . , $R_{i+6}^{(2)}$. This decoder is called a *direct decoder* because its decision of $E_i^{(1)}$ is based directly on this finite sequence of received digits and not on the decoder's own past history.

A different type of decoder, called a *feedback decoder*‡ is obtained

† A *majority-decision element* (MDE) is a threshold element whose threshold is half the number of its inputs.

‡ This term, used by Robinson (1968) and Reddy (1968) to denote feedback within the decoder, should not be confused with coding-decoding schemes which allow feedback from the decoder to the encoder.

by adding to Fig. 15.11 the connections shown by the dotted lines. The new connections allow the decoder to correct errors in the syndrome digits as well as errors in the message digits. These corrected syndrome digits improve the decoder's probability of decoding subsequent digits correctly. Thus, if the feedback decoder of Fig. 15.11 has correctly estimated $E_{i-6}^{(1)}$, $E_{i-5}^{(1)}$, . . . , $E_{i-1}^{(1)}$, then its estimate of $E_i^{(1)}$ depends only on the 14 consecutive digits $R_i^{(1)}$, $R_i^{(2)}$, $R_{i+1}^{(1)}$, . . . , $R_{i+6}^{(2)}$. If these 14 digits contain no more than two errors and the feedback decoder has not previously erred, then the feedback decoder's estimate of $E_i^{(1)}$ will be correct. However, if the feedback decoder does misdecode a digit, then this error may cause the decoder to misdecode subsequent digits as well. This *error-propagation effect* cannot occur in a direct decoder, because a direct decoder's present estimates are not affected by its past mistakes.

Using trial-and-error techniques, Massey (1963) has found many convolutional codes which can be decoded by simple threshold decoders, similar to the decoder shown in Fig. 15.11. Using a more systematic technique based on difference sets,† Robinson and Bernstein (1967) and Reddy (1968) have found many more codes of this type.

Although threshold decoding proves very effective for decoding many particular codes of short to moderate constraint lengths, it is a poor method for decoding codes with long constraint lengths. Massey (1963) has shown that the probability of error of the best threshold decoders is bounded above zero, independent of constraint length.

† The alert reader may have noticed that the exponents of z in the polynomial of Example 15.62 are $\{0,2,5,6\}$, a perfect simple difference set mod 13.

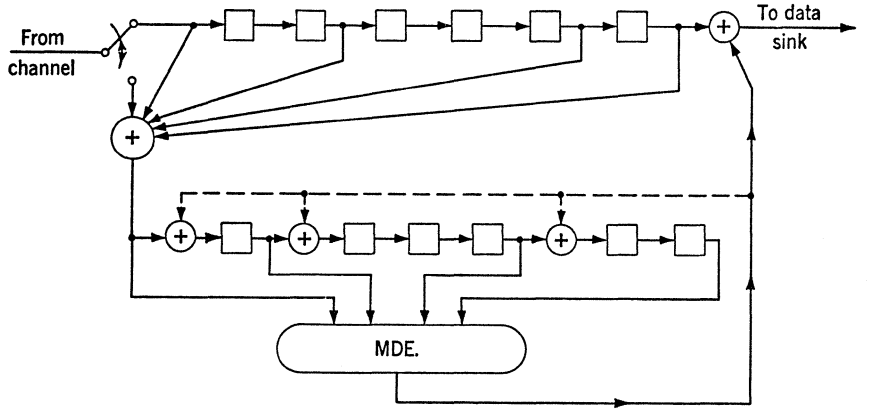


Fig. 15.11 Threshold decoder for code of Fig. 15.10. (Direct decoder without dotted connections; feedback decoder with dotted connections.)

15.64 FINDING "GOOD" CONVOLUTIONAL CODES

The known algebraic decoding procedures for block codes are all based on certain mathematical structures which are designed into the code. Unfortunately, very few classes of convolutional codes with any known structure have been proposed. Berlekamp (1963) has exhibited a class of erasure-burst-correcting convolutional codes which have an elaborate structure, but these codes do not appear to be very useful for correcting random errors on memoryless channels. Wyner and Ash (1963) have exhibited a class of convolutional codes which correct single errors with the minimum guard space. These codes are analogous to the Hamming single-error-correcting block codes. Unfortunately, no convolutional codes analogous to the multiple-error-correcting BCH block codes are known. Epstein (1958*b*) and Bussgang (1965) have found the "best" binary convolutional codes of certain rates and short constraint lengths, but no such results are known for $N > 32$.

Despite the lack of known methods for constructing "good" convolutional codes, a great deal is known about the "best" convolutional codes. This knowledge has been obtained from various studies of randomly chosen convolutional codes. Elias (1954) has shown that such codes can be used to achieve arbitrarily small probability of decoding error at all rates below Shannon's capacity. Massey (1963) has shown that they satisfy a bound similar to the Gilbert bound for block codes (see Sec. 13.7). Yudkin (1964) and Viterbi (1967) have obtained tight bounds on their probability of error. Viterbi's results show that the probability of error of a randomly chosen high-rate convolutional code of constraint length N is considerably less than the probability of error of the best block code of length N.

15.65 SEQUENTIAL DECODING

In addition to threshold decoding and the other few constructive decoding methods known for certain convolutional codes, there is another important decoding algorithm called *sequential decoding*. Introduced by Wozencraft and Reiffen (1961), this algorithm has since been refined and modified by Fano (1963).

Unlike the constructive decoding methods, which are based on the mathematical structures of the codes they decode, sequential decoding is a probabilistic algorithm which works almost all the time for almost all randomly chosen convolutional codes. Roughly speaking, a sequential decoder makes a tentative decoding decision on each received digit. It then attempts to decode subsequent digits on the basis of its previous tentative decisions. If the subsequent digits prove difficult to decode, i.e., all choices are improbable, the decoder backtracks and alters some of its tentative decisions.

The sequential decoder of Fano (1963) decodes a convolutional code with infinite constraint length. With probability 1, any given digit will eventually be decoded correctly and will remain unaltered forever thereafter. Unfortunately, the amount of computation required to decode any given digit is a random variable which depends on the channel noise. Bounds obtained by Jacobs and Berlekamp (1967) and by Jelinek (1969)† show that the distribution of this random variable is approximately given by

$$\Pr(\text{computations} \geq L) \approx L^{-\rho}$$

where ρ is a function of the rate of the code and the detailed noise statistics of the channel. If the rate is sufficiently low, $\rho > 1$, and the average number of computations per decoded digit is bounded. However, the variance may still be infinite. Even if the variance is finite, the number of computations necessary to decode a particular digit will be too large too often. It is this computational problem, rather than the error probability, which limits the performance of sequential decoders.

Some of the undesirable properties of the distribution of computation in sequential decoders can be circumvented by hybrid decoding schemes, which combine convolutional encoding and sequential decoding with another form of encoding and decoding. Such schemes have been proposed by Pinsker (1965) and Falconer (1967).

A detailed discussion of sequential decoding may be found in Wozencraft and Jacobs (1965) or in Jelinek (1968).

15.66 CONVOLUTIONAL CODES FOR CORRECTING BURSTS

Although no methods are known for constructing the best convolutional codes to correct random errors on a memoryless channel, there do exist explicit constructions for finding the best convolutional codes for correcting bursts.

The transmission of information across certain channels is hindered by the occasional occurrence of long periods during which the signal fades. This fading erases a sequence of consecutive digits, called an *erasure burst.* When the erasure burst ends, it is again possible to transmit information across the channel without errors or erasures until another deep fade causes another erasure burst. A coding system for such a channel must attempt to correct each erasure burst before the next one begins. If an erasure burst affecting E digits is followed by a quiet spell of length Q, then correction cannot be made until the number of check digits received during the quiet interval is at least as great as the number of erased message digits. This implies that the rate of the code must be sufficiently small so that $ER \leq Q(1 - R)$. Berlekamp (1963) has shown that for any given R, it is possible to construct a convolutional code which allows

† Jelinek's bound is based on a lemma of Yudkin (1964). Special cases of this bound were proved by Savage (1967*a*,*b*) and Falconer (1967).

correction whenever Q and E satisfy the necessary inequality. In the binary case, this construction gives the *only* code with this property. These erasure-burst-correcting convolutional codes can be decoded by a very simple decoding algorithm based on the structure of the code. The optimum erasure-burst-correcting convolutional codes are also uniquely optimum in their ability to *detect* error bursts as soon as possible.

The first efforts to design convolutional codes capable of correcting error bursts were due to Hagelbarger (1959). Wyner and Ash (1963) reformulated the problem in a more mathematical framework and obtained bounds on the minimum guard space between bursts of a given length which can be corrected by the best burst-correcting convolutional code of a given rate. Berlekamp (1964*a*) and Preparata (1964) then constructed a class of codes which attain the Wyner-Ash bound for bursts of a given phase relative to the blocks of the convolutional code. Massey (1965) discovered a simple decoding algorithm for these codes. Ekonomi (1965) and Hsu (1969) have constructed certain burst-correcting convolutional codes which have fewer phase restrictions than the Berlekamp-Preparata codes, but no general method of constructing phase-independent burst-correcting convolutional codes which satisfy the Wyner-Ash bound has yet been found.

Gallager (1966) has shown that the Wyner-Ash bound applies to block codes as well as convolutional codes. He has also exhibited a class of convolutional codes which correct many (but not all) bursts of a given length with considerably less guard space than the minimum required by the Wyner-Ash bound.

PROBLEMS

15.1. The expurgated binary QR code of length 31 may be generated by

$$M^{(1)}(x)M^{(5)}(x)M^{(7)}(x) = 1 + x^3 + x^8 + x^9 + x^{13} + x^{14} + x^{15}$$

(*a*) List the 7 codewords of weight 8 in the extended binary QR code of length 32 which have ones in positions $\{\infty,0,1\}$.

Answer: $\{\infty,0,1\} \cup \{26,13,29,9,16\}$ or $\{26,23,6,7,5\}$ or $\{26,17,30,20,25\}$ or $\{26,21,27,2,18\}$ or $\{21,23,17,9,14\}$ or $\{27,17,29,7,19\}$ or $\{12,29,21,5,25\}$.

(*b*) How many of the 2^{16} cosets of the extended binary QR code of length 32 have unique leaders of weight 4? How many cosets contain 2, 3, 4, 5, . . . words of weight 4?

(*c*) How many of the 2^{16} cosets of the second order RM code of length 32 contain 1, 2, 3, . . . words of weight 4?

15.2. Using the results of Prob. 15.1*a*, show that every permutation which preserves the extended binary QR code of length 32 is contained in the projective unimodular group.

Hint: The positions 1, 2, . . . , 30 may be partitioned into two sets, such that if there exist three or more codewords of weight 8 through $\{\infty,0,x,y\}$, then x and y are in opposite sets. Consequently no code-preserving permutation can fix positions ∞ and 0 and exchange residues and nonresidues. Furthermore, any code-preserving

permutation which pointwise fixes $\{\infty,0,1\}$ must also fix 26, 12, 28, 17, 7, 19, 14, 9, ... In other words, it must be the identity.

15.3. [Assmus-Mattson (1966)] The group of all permutations on 24 symbols which preserve the extended binary QR code (the extended Golay code) is called the *Mathieu group.*

(*a*) Show that the Mathieu group is 5-fold transitive.

(*b*) Show that its order is $16 \cdot 15 \cdot 14 \cdot 12 \cdot 8 \cdot 23 \cdot 11 \cdot 3$.

Hint: Let A and B be subsets of positions corresponding to the 0s and 1s of a particular codeword of weight 8. Then $|A| = 16$; $|B| = 8$. Let a be a particular element of A; let b be a particular element of B. Let $\mathcal{G}$ be the subgroup of the Mathieu group which setwise fixes A and B. Let $\mathcal{K}$ be the subgroup of $\mathcal{G}$ which pointwise fixes B, and let $\mathcal{S}$ be the subgroup of $\mathcal{G}$ which fixes a. Let $\mathcal{G}_A$, $\mathcal{K}_A$, $\mathcal{S}_A$ and $\mathcal{G}_B$, $\mathcal{K}_B$, $\mathcal{S}_B$ be the projections of $\mathcal{G}$, $\mathcal{K}$, and $\mathcal{S}$ onto A and B, respectively. $\mathcal{K}_B$ is the identity.

Each of the 2^{12} codewords in the extended Golay code may be projected onto A and B. The weight of the codeword is the sum of the weight of its projection onto A and the weight of its projection onto B. The projection of any codeword onto A must have weight 0 or ≥ 4. The 2^{11} distinct projections of extended Golay codewords onto A form a linear code of minimum weight ≥ 4 and length 16. Since this set of projections must be equivalent to an extended Hamming code, the set A may be coordinatized as a 4-dimensional vector space over GF(2), the **0** vector corresponding to the point a. The set of the 2^7 distinct projections of extended Golay codewords onto B must be equivalent to a single-parity-check code of length 8. If the projection of a codeword onto B is **0**, then its projection onto A is a codeword of the first-order RM code of length 16.

According to Theorem 15.35, $\mathcal{G}_A$ and $\mathcal{K}_A$ must be subgroups of $\mathcal{A}$, the group of all invertible affine transformations of GF(2^4) over GF(2). $\mathcal{S}_A$ must be a subgroup of $\mathcal{L}$, the group of all invertible linear transformations of GF(2^4) over GF(2). $\mathcal{A}$ has order $16 \times 15 \times 14 \times 12 \times 8$; $\mathcal{L}$ has order $15 \times 14 \times 12 \times 8 = 8!/2$. It is relatively easy to show that $\mathcal{K}_A$ is the group of additive translations on GF(2^4), which has order 16. It is harder to show that $\mathcal{S}_A = \mathcal{L}$, and that $\mathcal{S}_B = \mathcal{G}_B =$ the alternating group on 8 symbols. (These facts may be verified by examination of a listing of all 35 extended Golay codewords of weight 8 which have ones in positions a and b.) It follows that $\mathcal{G}$ has the same order as $\mathcal{A}$.

Since the Golay code is perfect, the extended Golay code must contain a codeword of weight 8 having ones in any given 5 positions. Altogether, it contains $24 \times 23 \times 22 \times 21 \times 20/8 \times 7 \times 6 \times 5 \times 4 = 23 \times 11 \times 3$ codewords of weight 8. It is not difficult to show that any of these $23 \times 11 \times 3$ codewords of weight 8 may be permuted into any other by some permutation in $\mathcal{P}$, the projective unimodular group. Theorem 15.26 asserts that $\mathcal{P}$ is a subgroup of the Mathieu group. If we wish to find a permutation π which permutes $x_1 \to y_1$, $x_2 \to y_2$, . . . , $x_5 \to y_5$, we first find the sets $X = \{x_1,x_2, \ldots ,x_5,x_6,x_7,x_8\}$ and $Y = \{y_1,y_2, \ldots ,y_8\}$ which correspond to the positions containing ones in codewords of weight 8. We may then set $\pi = \pi_1\pi_2\pi_3$, where $\pi_1 \in \mathcal{P}$ and π_1 permutes $X \to B$, $\pi_3 \in \mathcal{P}$ and π_3^{-1} permutes $Y \to B$, and π_2 is an appropriate permutation in $\mathcal{G}$.

15.4. Determine the order of the group of all permutations which preserve the extended binary QR code of length 18.

15.5. Using Fig. 15.09, decide which of the following error patterns would be correctly decoded by the decoder of Fig. 15.08.

(*a*) {0, 1, 2, 3}

(*b*) {0, 1, 2, 3, 4}

(*c*) {0, 1, 2, 3, 4, 5}
(*d*) {0, 1, 2, 3, 4, 5, 6}
(*e*) {0, 1, 2, 3, 4, 30}
(*f*) {0, 1, 3, 27}

Answers: OK, OK, OK, OK, fail, fail, fail

15.6. It is suggested that the decoder of Fig. 15.08 might be improved by including the nine 6-input threshold elements on the first level, one to estimate each of the nine subspaces orthogonal on position 0, as listed in Fig. 15.09. The estimate of E_0 would then be calculated by a 9-input threshold element, with $T = 5$. Comment on this suggestion.

15.7. Construct perfect simple difference sets mod 7,13,21,31,57,73. Which of these sets may be used to construct difference set codes over GF(2)?

15.8. [MacWilliams-Mann (1968) and Smith (1967)] In the notation of Theorem 15.599, show that $r(p, m, m-2, s) = \binom{p+m-1}{m}^s$.

16
Weight Enumerators†

16.1 THE RELATIONSHIP BETWEEN WEIGHT ENUMERATORS AND THE PROBABILITY OF DECODING FAILURE

In some situations decoding errors are very serious while decoding failures are only minor nuisances. In such circumstances, it is wise to use a very incomplete decoding algorithm, which decodes only if the received word is a codeword. This decoding algorithm will decode correctly iff the channel-error pattern is 0; it will decode incorrectly iff the channel-error pattern is a nonzero codeword; it will fail to decode iff the channel-error pattern is not a codeword. For the binary symmetric channel, the probability that the channel-error pattern will match any particular codeword of weight i in a code of length n is equal to $P^i(1 - P)^{n-i}$, where P is the channel crossover probability. If the code contains $A_0 = 1$ word of weight 0, A_1 words of weight 1, A_2 words of weight 2, . . . , A_i words of weight i, then the probability that the channel-error pattern is equal to a codeword is $\sum_{i=0}^{n} A_i P^i(1 - P)^{n-i}$. It is convenient to introduce the code's

† Additional results on weight distributions may be found through these references: Berlekamp, E. R., "Weight Enumeration Theorems," Sixth Allerton Conference on Circuit and System Theory, University of Illinois Press, Urbana, Illinois 1968.
Kasami and Tokura, "Some Remarks on BCH Bounds and Minimum Weights of Binary Primitive BCH Codes," *IEEE Trans. Inform. Theory*: IT15: 408–413.

weight enumerator, $A(z) = \sum_{i=0}^{n} A_i z^i$. In terms of the weight enumerator, the probability that the channel-error pattern is equal to a codeword is given by $(1 - P)^n A(P/(1 - P))$. The probability of correct decoding is $(1 - P)^n$; the probability of decoding error is $(1 - P)^n[A(P/(1 - P)) - 1]$; the probability of decoding failure is $1 - (1 - P)^n A(P/(1 - P))$. Thus, the probabilities of correct decoding, of decoding error, and of decoding failure for very incomplete decoding algorithms which decode only when the received word is a codeword are all easily expressed in terms of the code's weight enumerator.

The code's weight enumerator may also be used to express the probabilities of decoding error and decoding failure for decoding algorithms which correct all patterns of up to t channel errors and nothing more. In this case, the probability of correct decoding is given by $\sum_{i=0}^{t} \binom{n}{i} P^i(1 - P)^{n-i}$, which is the probability that the channel-error pattern has weight $\leq t$. The probability of decoding failure is the probability that the channel-error pattern lies in a coset of weight $> t$; the probability of decoding error is the probability that the channel-error pattern is a nonleader of a coset of weight $\leq t$. Let $W_{j,k}^{(i,n-i)}$ denote the number of words of weight k which are at distance j from any given word of weight i and length n. Then there are $\sum_i A_i W_{j,k}^{(i,n-i)}$ words of weight k at distance j from codewords. If the code has distance $\geq 2t + 1$, then no word can be within distance t of more than one codeword, so the number of words of weight k in cosets of weight $\leq t$ is then given by $\sum_{j=0}^{t} \sum_{i=0}^{n} A_i W_{j,k}^{(i,n-i)}$. The probability of decoding (either correctly or incorrectly) is then given by

$$\sum_{k=0}^{n} \sum_{j=0}^{t} \sum_{i=0}^{n} A_i W_{j,k}^{(i,n-i)} P^k (1 - P)^{n-k} \tag{16.11}$$

Let $W_j^{(i,n-i)}(y) = \sum_k W_{j,k}^{(i,n-i)} y^k$, and let $W^{(i,n-i)}(x,y) = \sum_j \sum_k W_{j,k}^{(i,n-i)} x^j y^k$. Since $W_{j,k}^{(i,n-i)}$ is additive over the code's n coordinates, the generating function $W^{(i,n-i)}(x,y)$ is multiplicative over these coordinates, and $W^{(i,n-i)} = (W^{(1,0)})^i (W^{(0,1)})^{n-i} = (x + y)^i (1 + xy)^{n-i}$. We have

$$\sum_{k=0}^{n} \sum_{j=0}^{n} \sum_{i=0}^{n} A_i W_{j,k}^{(i,n-i)} P^k (1 - P)^{n-k} x^j = (1 - P)^n \sum_{i=0}^{n} A_i \left(x + \frac{P}{1 - P}\right)^i \left(1 + \frac{xP}{1 - P}\right)^{n-i} \tag{16.12}$$

$$\sum_{k=0}^{n}\sum_{j=0}^{n}\sum_{i=0}^{n} A_i W_{j,k}^{(i,n-i)} P^k (1-P)^{n-k} x^j$$

$$= \sum_{i=0}^{n} A_i (x - xP + P)^i (1 - P + xP)^{n-i}$$

$$= (1 - P + xP)^n A\left(\frac{x - xP + P}{1 - P + xP}\right) \tag{16.13}$$

Let

$$f^{(j)}(x) = \frac{d^j}{dx^j}(1 - P + xP)^n A\left(\frac{x - xP + P}{1 - P + xP}\right) \tag{16.14}$$

According to Eqs. (16.12) and (16.13), $f^{(0)}(x)$ is a polynomial of degree at most n in x. We may expand it in a Taylor series, obtaining

$$f^{(0)}(x) = \sum_{j=0}^{n} \frac{f^{(j)}(0)}{j!} x^j \tag{16.15}$$

Combining Eqs. (16.12) to (16.15) with (16.11), we obtain an expression for the probability of decoding failure

$$1 - \sum_{k=0}^{n}\sum_{j=0}^{t}\sum_{i=0}^{n} A_i W_{i,k}^{(i,n-i)} P^k (1-P)^{n-k} = 1 - \sum_{j=0}^{t} \frac{f^{(j)}(0)}{j!} \tag{16.16}$$

Expression (16.16), which is due to MacWilliams (1963), gives the probability of decoding failure whenever t is less than half the code's minimum distance. The choice $t = 0$ corresponds to the very incomplete decoding algorithm which decodes only when the received word is a codeword.

Unfortunately, the weight enumerator does not reveal everything that we would like to know about the code. It does not, for example, tell us the probability of error of the complete decoding algorithm, which decodes every received word into a leader of its coset. For example, the expurgated binary QR code of length 31 and the shortened second-order RM code of length 31 have identical weight enumerators, even though Prob. 15.1 shows that the weight distributions of the coset leaders of these two codes are different.

Nevertheless, the weight enumerator does contain a good deal of information about a code, including its minimum distance, the number of codewords of each weight, and the probabilities of decoding error and failure if one decodes all patterns of up to t errors and nothing more. For this reason, a considerable amount of research has been devoted to the study of weight enumerators. We shall devote this chapter to some of the more important results which have been obtained.

All the known weight-enumerator results deal only with Hamming weight enumerators, and most of them deal only with binary codes.

With the exception of a few relatively trivial theorems, such as Theorem 13.52, virtually nothing is known about Lee weight enumerators.

The weight enumerators of a number of specific binary codes are given in Table 16.1; formulas for the weight enumerators of certain classes of low-rate or high-rate binary codes are given in Tables 16.4 and 16.5.†

16.2 THE MACWILLIAMS-PLESS EQUATIONS FOR THE WEIGHT ENUMERATORS OF DUAL CODES

For codes which have only a small number of codewords or only a small number of equivalence classes of codewords under some known permutation group, we can determine the code's weight enumerator by an exhaustive examination of all the equivalence classes. This approach proves quite feasible for certain classes of codes with short block lengths and/or low rates. In fact, the weight enumerators of most of the codes of rate $<1/2$ in Table 16.1 were first determined in this way, using the cyclic permutation group and the permutation group which maps $X_i \rightarrow X_i^2$ (Corollary 5.82).

Unfortunately, the number of codewords in most high-rate codes of moderate to long block lengths is many orders of magnitude greater than the order of the largest permutation group which leaves the code invariant. Thus, we cannot determine the weight enumerators of such codes directly, even with the aid of a large high-speed computer. However, even though q^{Rn} may be large, $q^{(1-R)n}$ may be relatively small. Thus, it may be comparatively easy to determine the weight enumerator of the *dual* code, whose codewords are the parity checks of the original code. In fact, it is generally easier to determine the weight enumerator of the dual code whenever the rate of the original code is $>1/2$, because the dual code is invariant under the same permutation group as the original code. In particular, the dual of a cyclic code with generator polynomial $g(x)$ is the cyclic code with generator polynomial $\tilde{h}(x)$, where $h(x) = (x^n - 1)/g(x)$ and $\tilde{h}(x)$ is the reciprocal of $h(x)$. Similarly, if $g(x)$ is not divisible by $x - 1$, then the dual of the extension of the cyclic code generated by $g(x)$ is the extension of the cyclic code generated by the reciprocal of $(x^n - 1)/g(x)(x - 1)$.

Once we have determined the weight enumerator of the dual code, we can find the weight enumerator of the original code by the following remarkable formula, due to MacWilliams (1963).

Theorem 16.21 (MacWilliams) Let $\mathcal{C}$ be a linear code of dimension k over GF(q), with Hamming weight enumerator $A(z)$, and let $\mathcal{C}$

† Tables 16.1 to 16.6 are grouped at the end of the chapter.

be its dual code, with Hamming weight enumerator $B(z)$. Then

$$A(z) = q^k(q^{-1} + \rho z)^n B\left(\frac{1 - z}{1 + (q - 1)z}\right)$$

where $\rho = (q - 1)/q$.

Remark The MacWilliams formula is symmetric; the substitution $y = (1 - z)/[1 + (q - 1)z]$ results in the formula

$$B(y) = q^{n-k}(q^{-1} + \rho y)^n A\left(\frac{1 - y}{1 + (q - 1)y}\right)$$

The proof of MacWilliams' theorem is based on Lemma 16.213, which requires Lemma 16.212 and Definitions 16.211.

Definitions 16.211 Let $\mathbf{v}_1, \mathbf{v}_2, \ldots \mathbf{v}_n$ be a basis of $GF(q^n)$ over $GF(q)$.

Let the dot product be defined by $\left(\sum_{i=1}^{n} a_i\mathbf{v}_i\right) \cdot \left(\sum_{i=1}^{n} b_j\mathbf{v}_j\right) = \sum_{i=1}^{n} a_i b_i$.

Let script variables represent subspaces of $GF(q^n)$ over $GF(q)$.

Let $\mathcal{F}^\perp$ be the dual of $\mathcal{F}$, defined by $\mathbf{x} \in \mathcal{F}^\perp$ iff $\mathbf{x} \cdot \mathbf{y} = 0$ for all $\mathbf{y} \in \mathcal{F}$.

Let $\mathcal{F} \oplus \mathcal{G}$ denote the direct sum of $\mathcal{F}$ and $\mathcal{G}$, defined by $\mathbf{x} \in \mathcal{F} \oplus \mathcal{G}$ iff $\exists \mathbf{y} \in \mathcal{F}$ and $\mathbf{z} \in \mathcal{G}$ such that $\mathbf{x} = \mathbf{y} + \mathbf{z}$.

Lemma 16.212

$$\mathcal{A}^\perp \oplus \mathcal{F}^\perp = (\mathcal{A} \cap \mathcal{F})^\perp$$

Proof

$$\mathcal{A} \cap \mathcal{F} \subseteq \mathcal{A}, \mathcal{F}$$
$$\mathcal{A}^\perp, \mathcal{F}^\perp \subseteq (\mathcal{A} \cap \mathcal{F})^\perp$$

so

$$(\mathcal{A}^\perp \oplus \mathcal{F}^\perp) \subseteq (\mathcal{A} \cap \mathcal{F})^\perp$$

On the other hand,

$$\begin{aligned}
&\mathcal{A}^\perp, \mathcal{F}^\perp \subseteq \mathcal{A}^\perp \oplus \mathcal{F}^\perp \\
&(\mathcal{A}^\perp \oplus \mathcal{F}^\perp)^\perp \subseteq \mathcal{A}^{\perp\perp}, \mathcal{F}^{\perp\perp} \\
&(\mathcal{A}^\perp \oplus \mathcal{F}^\perp)^\perp \subseteq \mathcal{A}^{\perp\perp} \cap \mathcal{F}^{\perp\perp} = \mathcal{A} \cap \mathcal{F} \\
&(\mathcal{A} \cap \mathcal{F})^\perp \subseteq ((\mathcal{A}^\perp \oplus \mathcal{F}^\perp)^\perp)^\perp = \mathcal{A}^\perp \oplus \mathcal{F}^\perp \qquad \text{Q.E.D.}
\end{aligned}$$

Lemma 16.213 Let $\mathcal{F}$ be any subspace of $\mathrm{GF}(q^n)$ of dimension j over $\mathrm{GF}(q)$, and let $\mathcal{G} = \mathcal{F}^\perp$. Let $\mathcal{A}$ be a subspace of dimension k over $\mathrm{GF}(q)$, and let $\mathcal{B} = \mathcal{A}^\perp$. Then

$$|\mathcal{G} \cap \mathcal{A}| = q^{k-j}|\mathcal{F} \cap \mathcal{B}|$$

Proof

$$\dim (\mathcal{F}^\perp \cap \mathcal{A}) = n - \dim (\mathcal{F}^\perp \cap \mathcal{A})^\perp$$

By Lemma 16.212, $(\mathcal{F}^\perp \cap \mathcal{A})^\perp = \mathcal{F}^{\perp\perp} \oplus \mathcal{A}^\perp = \mathcal{F} \oplus \mathcal{A}^\perp$,

$$\dim (\mathcal{F} \oplus \mathcal{A}^\perp) = \dim \mathcal{F} + \dim \mathcal{A}^\perp - \dim (\mathcal{F} \cap \mathcal{A}^\perp)$$

so

$$\begin{aligned}
\dim (\mathcal{G} \cap \mathcal{A}) &= n - \dim \mathcal{F} - \dim \mathcal{B} + \dim (\mathcal{F} \cap \mathcal{B}) \\
&= k - j + \dim (\mathcal{F} \cap \mathcal{B}) \qquad \text{Q.E.D.}
\end{aligned}$$

Proof of Theorem 16.21 For $m = 1, 2, \ldots, \binom{n}{j}$, let $\mathcal{F}_m$ be the particular j-dimensional subspace of $\mathrm{GF}(q^m)$ which consists of all those vectors which have zeros in a particular set of $n - j$ coordinates. Then $\mathcal{G}_m$, the dual of $\mathcal{F}_m$, consists of all the vectors which have zeros in the complementary set of j coordinates. If $\mathbf{a}$ is a particular vector of weight i in code $\mathcal{A}$, then $\mathbf{a} \in \mathcal{G}_m$ iff the set of j coordinates in which $\mathcal{G}_m$ is 0 is a subset of the $n - i$ coordinates in which $\mathbf{a}$ is 0. If we let

$$|\mathbf{a} \cap \mathcal{G}_m| = \begin{cases} 0 & \text{if } \mathbf{a} \notin \mathcal{G}_m \\ 1 & \text{if } \mathbf{a} \in \mathcal{G}_m \end{cases}$$

we have

$$\sum_{m=1}^{\binom{n}{j}} |\mathbf{a} \cap \mathcal{G}_m| = \binom{n-i}{j}$$

and

$$\sum_{\mathbf{a} \in \mathcal{A}} \sum_{m=1}^{\binom{n}{j}} |\mathbf{a} \cap \mathcal{G}_m| = \sum_i A_i \binom{n-i}{j} \tag{16.214}$$

Similarly,

$$\sum_{\mathfrak{b}\in\mathfrak{B}} \sum_{m=1}^{\binom{n}{j}} |\mathfrak{b} \cap \mathfrak{F}_m| = \sum_i B_i \binom{n-i}{n-j} \tag{16.215}$$

Combining Eqs. (16.213) to (16.215) gives

$$\begin{aligned}\sum_i A_i \binom{n-i}{j} &= \sum_{\mathfrak{a}\in\mathfrak{A}} \sum_{m=1}^{\binom{n}{j}} |\mathfrak{a} \cap \mathfrak{G}_m| \\ &= \sum_{m=1}^{\binom{n}{j}} |\mathfrak{A} \cap \mathfrak{G}_m| = q^{k-j} \sum_{m=1}^{\binom{n}{j}} |\mathfrak{B} \cap \mathfrak{F}_m| \\ &= q^{k-j} \sum_{\mathfrak{b}\in\mathfrak{B}} \sum_{m=1}^{\binom{n}{j}} |\mathfrak{b} \cap \mathfrak{F}_m| \\ &= q^{k-j} \sum_i B_i \binom{n-i}{n-j} \end{aligned} \tag{16.216}$$

Since j is arbitrary, Eq. (16.216) must be valid for all j. Multiplying by x^j and summing on j gives

$$\begin{aligned}\sum_j \sum_i A_i \binom{n-i}{j} x^j &= \sum_j q^{k-j} \sum_i B_i \binom{n-i}{n-j} x^j \\ &= q^k q^{-n} x^n \sum_i B_i \sum_j \binom{n-i}{n-j} \left(\frac{q}{x}\right)^{n-j} \\ &= \sum_i A_i (1+x)^{n-i} = q^k \left(\frac{x}{q}\right)^n \sum_i B_i \left(1 + \frac{q}{x}\right)^{n-i}\end{aligned}$$

Dividing through by $(1 + x)^n$ gives

$$\sum_i A_i \left(\frac{1}{1+x}\right)^i = q^k \left(\frac{x+q}{q+qx}\right)^n \sum_i B_i \left(\frac{x}{x+q}\right)^i$$

Setting $z = 1/(1 + x)$, $x = (1 - z)/z$, $\rho = (q - 1)/q$ gives

$$\sum_i A_i z^i = q^k (q^{-1} + \rho z)^n \sum_i B_i \left(\frac{1-z}{1+(q-1)z}\right)^i \qquad \text{Q.E.D.}$$

The MacWilliams formula enables us to obtain the weight distribution of any linear code from the weight distribution of its dual code. For example, the dual of an extended binary Hamming code of block

length 2^m is a first-order RM code, whose weight enumerator is obviously

$$B(z) = 1 + (2^{m+1} - 2)z^{2^{m-1}} + z^{2^m}$$

The weight enumerator of the extended Hamming code is therefore

$$A(z) = 2^{-m-1}2^{-2^m}(1+z)^{2^m}\left[1 + (2^{m+1} - 2)\left(\frac{1-z}{1+z}\right)^{2^{m-1}} + \left(\frac{1-z}{1+z}\right)^{2^m}\right] \quad (16.22)$$

The MacWilliams formula is a system of $n + 1$ equations relating the $n + 1$ coefficients of $A(z)$ to the $n + 1$ coefficients of $B(z)$. If we know either $A(z)$ or $B(z)$, the MacWilliams formula enables us to determine the other. In many cases, however, it may be difficult to determine either all the A's or all the B's, although it may be relatively easy to determine some of the A's and some of the B's. For example, we may know A_j for all but s particular values of j, and we may know the values of B_0, B_1, . . . , B_{s-1}. The MacWilliams formula then becomes a system of $n + 1$ equations in s unknown A's and $n + 1 - s$ unknown B's. To solve this system of equations, we first find s equations relating the s unknown values of A_j and the known values B_0, B_1, . . . , B_{s-1}. This is accomplished by the power-moment identities, which we shall now derive. These identities, due to Pless (1963), express the rth translated moment of the A's, $\sum_{j=0}^{n} (w - j)^r A_j$, in terms of the B's. The constant w is usually taken to be 0, $n/2$, $(n + 1)/2$, or n.

To compute the rth translated moment, we write the MacWilliams formula as

$$\sum_{j=0}^{n} A_j z^j = q^k \sum_{j=0}^{n} B_j(q^{-1} + \rho z)^{n-j}(q^{-1} - zq^{-1})^j$$

Multiplying through by $\exp 2wx$ and setting $z = \exp(-2x)$ gives

$$\sum_{j=0}^{n} A_j \exp[2(w - j)x] = q^k \sum_{j=0}^{n} B_j[q^{-1} + \rho \exp(-2x)]^{n-j}[q^{-1} - q^{-1}\exp(-2x)]^j \exp 2wx$$

We then take the rth derivative of each side with respect to x and set $x = 0$. This gives

$$\sum_{j=0}^{n} (2w - 2j)^r A_j = q^k \sum_{j=0}^{n} B_j F_r^{(j)}(n) \quad (16.225)$$

where $F_r^{(j)}(n)$ is a polynomial in n, defined by

$$F_r^{(j)}(n) = \left(\frac{d^r}{dx^r}\{[q^{-1} + \rho \exp(-2x)]^{n-j} [q^{-1} - q^{-1}\exp(-2x)]^j \exp 2wx\}\right)_{x=0} \quad (16.23)$$

If $r < j$, then $F_r^{(j)}(n) = 0$. Therefore, we may rewrite Eq. (16.225) as

$$\boxed{\sum_{j=0}^{n} (2w - 2j)^r A_j = q^k \sum_{j=0}^{r} B_j F_r^{(j)}(n) \qquad \text{for all } r} \quad (16.24)$$

Equation (16.24) is called the *Pless power-moment identities.* This equation may be written in terms of generating functions as

$$\sum_{r=0}^{\infty}\sum_{j=0}^{n} (2w - 2j)^r z^r A_j = q^k \sum_{j=0}^{\infty} B_j F^{(j)}(n;z) \quad (16.241)$$

where

$$F^{(j)}(n;z) = \sum_{r=0}^{\infty} F_r^{(j)}(n) z^r \quad (16.242)$$

If $B_0, B_1, B_2, \ldots, B_{s-1}$ are known and only s A's are unknown, the Pless identities (16.24) constitute a system of s equations in s unknowns. These equations are virtually identical to the equations of Sec. 10.1, where the numbers $2w - 2j$ now correspond to the known error locations and the A_j correspond to the unknown error values. Equation (16.24) can therefore be solved by Eq. (10.14). In some cases, however, it is possible to reduce Eq. (16.24) to an even simpler form, as we shall see in Sec. 16.4.

In general, the computation of the polynomial $F_r^{(j)}(n)$ from Eq. (16.23) is a rather tedious process. The answers may be obtained in closed form by the use of Stirling numbers or Bell polynomials, whose properties are discussed by Riordan (1958, pp. 47–48). In the binary case, the choice $w = n/2$ leads to a relatively straightforward recurrence. With $w = n/2$, $q = 2$, and $\rho = \frac{1}{2}$, Eq. (16.23) becomes

$$F_r^{(j)}(n) = \left(\frac{d^r}{dx^r} \cosh^{n-j} x \sinh^j x\right)_{x=0} \quad (16.243)$$

In most applications, $B_j = 0$ for most small positive values of j, so the only relevant $F_r^{(j)}(n)$ are those where $j = 0$, r, or $r - 2$. When needed, the other $F_r^{(j)}(n)$ may be expressed in terms of $F_r^{(0)}(n)$ in various ways. For example, the Taylor series expansions

$$\tanh^j x = \left(x - \frac{x^3}{3} + \cdots\right)^j = x^j - \frac{jx^{j+2}}{3} + \cdots$$

and

$$\cosh^n x = 1 + \frac{nx^2}{2} + \cdots$$

$$\cosh^n x \tanh^j x = x^j + \left(\frac{n}{2} - \frac{j}{3}\right) x^{j+2} + \cdots$$

lead immediately to the formulas

$$\begin{aligned} F_r^{(r)}(n) &= r! \\ F_r^{(r-2)}(n) &= r!\left(\frac{n}{2} - \frac{r-2}{3}\right) \\ &\cdots\cdots\cdots\cdots\cdots \end{aligned} \tag{16.244}$$

To compute

$$F^{(0)}(n;z) = \sum_{r=0}^{\infty} F_r^{(0)}(n) z^r$$

we write

$$\begin{aligned} F^{(0)}(n;z) &= 1 + \sum_{r=2}^{\infty} \left(z^r \frac{d^r}{dx^r} \cosh^n x\right)_{x=0} \\ &= 1 + z^2 \sum_{r=0}^{\infty} \left(z^r \frac{d^r}{dx^r} \frac{d^2}{dx^2} \cosh^n x\right)_{x=0} \\ &= 1 + z^2 \sum_{r=0}^{\infty} z^r \frac{d^r}{dx^r} [n^2 \cosh^n x - n(n-1) \cosh^{n-2} x]_{x=0} \\ &= 1 + z^2 n^2 F^{(0)}(n;z) - n(n-1) z^2 F^{(0)}(n-2;z) \end{aligned} \tag{16.245}$$

Therefore,

$$F^{(0)}(n;z) = \begin{cases} 1 & \text{if } n = 0 \\ \dfrac{1 - n(n-1)z^2 F^{(0)}(n-2;z)}{1 - z^2 n^2} & \text{if } n \geq 1 \end{cases} \tag{16.25}$$

It turns out to be easier to express the polynomial $F_r^{(0)}(n)$ not directly in terms of the powers of n but indirectly in terms of the falling factorials, $n_{(i)} = n(n-1)(n-2) \cdots (n+1-i)$. Setting $F_r^{(0)}(n) = \sum_i E_{r,i} n_{(i)}$, Eq. (16.245) becomes

$$\sum_i E_{r,i} n_{(i)} = n^2 \sum_i E_{r-2,i} n_{(i)} - \sum_i E_{r-2,i} n_{(i+2)} \qquad \text{for } r > 0$$

Using the identity $n^2 = (n-1-i)(n-i) + (2i+1)(n-i) + i^2$ gives

$$\sum_i E_{r,i} n_{(i)} = \sum_i (2i+1) E_{r-2,i} n_{(i+1)} + \sum_i i^2 E_{r-2,i} n_{(i)}$$

We thus obtain the recursive definition of $E_{r,i}$.

$$E_{r,i} = \begin{cases} 1 & \text{if } i = 0 \text{ and } r = 0 \\ 0 & \text{if } i = 0 \text{ and } r \neq 0 \text{ or} \\ & \text{if } r = 0 \text{ and } i \neq 0 \\ (2i-1)E_{r-2,i-1} + i^2 E_{r-2,i} & \text{otherwise} \end{cases} \tag{16.26}$$

It is obvious that $E_{r,i} = 0$ if r is odd. The values of $E_{r,i}$ for $r = 0, 2, 4, \ldots, 12$ are given in Table 16.2. The generating function for the ith column of this table is

$$E^{(i)}(z) = \sum_r E_{r,i} z^r$$

According to Eq. (16.26), $E^{(0)}(z) = 1$, $E^{(1)}(z) = z^2/(1-z^2)$, $E^{(2)}(z) = 3z^4/(1-z^2)(1-4z^2)$, $\ldots$, $E^{(i)}(z) = (2i-1)z^2E^{(i-1)}(z)/(1-i^2z^2)$, and

$$E^{(i)}(z) = z^{2i} \prod_{k=1}^{i} \left(\frac{2k-1}{1-k^2z^2} \right) \tag{16.27}$$

★ 16.3 WEIGHT RESTRICTIONS

If we know A_j for all but s values of j, and if we know $B_0, B_1, \ldots, B_{s-1}$, then we can determine the s unknown A's from the Pless identities and the remaining B's from the MacWilliams formula. For most codes, however, the first unknown value of B_s occurs for a relatively low value of s, and most of the $n+1$ A's are unknown. Before we can determine the last s unknown A's from the Pless identities, we must determine $n+1-s$ of the A's by some other means.

There are several methods by which we can determine some of the A's for certain classes of codes. It is obvious that $A_0 = 1$. If the code can be shown to have minimum distance $\geq d$, then $A_j = 0$ for $j = 1, 2, \ldots, d-1$. The BCH bound (Sec. 7.4) and the residue bound (Theorem 15.22) both provide useful bounds on the minimum distance, but in some cases these bounds can be improved by a powerful technique introduced by Mattson and Solomon (1961).

Theorem 16.31 (Mattson-Solomon) Let q be a prime power and let n be relatively prime to q. In any cyclic code of length n over GF(q), the Hamming weight of a codeword $C(x)$ whose power-sum symmetric functions are given by $S_i = C(\alpha^i)$ (for $i = 0, 1, 2, \ldots, n$) is $n - r$, where r is the number of nth roots of unity which are also roots of the *Mattson-Solomon* (MS) *polynomial*

$\frac{1}{n} \sum_{j=1}^{n} S_j x^{n-j}$, and α is a primitive n^{th} root of unity over GF(q).

Proof In general,

$$S_k = C(\alpha^k) \tag{16.311}$$

Multiplying Eq. (16.311) by α^{-kj} and summing on k gives

$$\sum_k S_k \alpha^{-jk} = \sum_i C_i \sum_k \alpha^{(i-j)k} \tag{16.312}$$

From the equation $0 = 1 - \alpha^{sn} = (1 - \alpha^s)(1 + \alpha^s + \cdots + \alpha^{s(n-1)})$ we see that

$$\sum_k \alpha^{(i-j)k} = \begin{cases} 0 & \text{if } i \not\equiv j \mod n \\ n & \text{if } i = j \end{cases} \tag{16.313}$$

Combining Eqs. (16.313) and (16.312) gives

$$\sum_k S_k \alpha^{-jk} = C_j n$$

or

$$C_j = \frac{1}{n}\left(\sum_k S_{-k} x^k\right)_{x=\alpha^j} = \frac{1}{n}\left(\sum_k S_k x^{n-k}\right)_{x=\alpha^j} \tag{16.314}$$

Equation (16.314), which is called the *Reed inversion formula*, expresses the coefficients of the codeword $C(x)$ as values assumed by the Mattson-Solomon polynomial at the nth roots of unity. Theorem 16.31 follows immediately. Q.E.D.

Corollary 16.315 Every nonzero codeword for which $S_1, S_2, \ldots, S_{d_{\text{BCH}}-1} = 0$ has weight $\geq d_{\text{BCH}}$.

Proof The Mattson-Solomon polynomial cannot have more roots than its degree, which is no greater than $n - d_{\text{BCH}}$. Q.E.D.

Corollary 16.315 gives an elegant proof of the BCH bound on minimum distance, but unfortunately this proof does not lead to any feasible decoding procedure. However, the Mattson-Solomon approach sometimes enables us to improve the BCH bound on minimum distance. Theorem 16.32 shows that for certain low-rate codes, the question of whether or not the BCH bound is attained can be resolved by calculations in GF(q) rather than by calculations in the extension field which includes the nth roots of unity.

Theorem 16.32 (Mattson-Solomon-Cerveira) Let q be a prime or prime power; let n be a multiple of $q - 1$ which is relatively prime to q; let α be a primitive nth root of unity in an extension field of GF(q). Let J be the set of residue classes of the form $q^i d_{\text{BCH}} \bmod n$ for $i = 0, 1, 2, \ldots$. Let d_{BCH} be the least element in J.

For example, if $q = 2$, $n = 23$, we might have

$$d_{\text{BCH}} = 5 \qquad J = \{5,10,20,17,11,22,21,19,15,7,14\}$$

Let $T(z) = \sum_{j \in J} z^{n-j}$. Let d be the actual minimum distance of the cyclic code whose check polynomial is $h(x) = (x - 1) \prod_{j \in J} (x - \alpha^j)$. If d_{BCH} and n are relatively prime, then $d = d_{\text{BCH}}$ iff there exists a nonzero s in GF(q) such that $1 + sT(-z)$ divides $1 - z^n$.

Proof In general, a nonzero codeword can have weight d_{BCH} iff its Mattson-Solomon polynomial has degree $n - d_{\text{BCH}}$ and all the roots of the MS polynomial are nth roots of unity. According to the hypothesis, $h(x) = (x - 1) \prod_{j \in J} (x - \alpha^j)$ so $S_i = 0$ unless $i \in J \cup 0$. Thus each codeword has an MS polynomial of the form

$$\text{MS}(x) = \frac{S_0 + T(\gamma x)}{n}$$

where $\gamma^{-d_{\text{BCH}}} = S_{d_{\text{BCH}}}$. If $S_0 = 0$, then the roots of the MS polynomial include 0, which is not an nth root of unity, so we assume that $S_0 \neq 0$ and

$$S_0 + T(\gamma x) = S_0 \prod_{i=1}^{n - d_{\text{BCH}}} (1 - \xi_i x) \tag{16.321}$$

where the ξ_i (like their reciprocals) are nth roots of unity. Equating the leading coefficients of these polynomials gives

$$\gamma^{n - d_{\text{BCH}}} = S_0 \prod_{i=1}^{n - d_{\text{BCH}}} (-\xi_i)$$

Since each ξ_i is an nth root of unity, so is their product. Since $S_0 \in \text{GF}(q)$ and n is a multiple of $q - 1$, S_0 is an nth root of unity. Therefore $(-\gamma)^{n - d_{\text{BCH}}}$ is an nth root of unity. According to the hypothesis of the theorem, $n - d_{\text{BCH}}$ and n are relatively prime, so $-\gamma$ must also be an nth root of unity. Substituting $z = -\gamma x$,

$\beta_i = -\xi_i/\gamma$, $s = S_0^{-1}$ into Eq. (16.321) gives

$$1 + sT(-z) = \prod_{i=1}^{n-d_{\text{BCH}}} (1 - \beta_i z)$$

Since the β_i are distinct nth roots of unity, $1 + sT(-z)$ divides $1 - z^n$. Q.E.D.

Corollary 16.322 Let n be prime, let m be the order of 2 mod n, and let d_{BCH} be the BCH bound on the binary BCH code of length n and rate $(m + 1)/n$. If $d_{\text{BCH}} < m < n - 1$, then $d_{\text{BCH}} < d$.

Proof If $1 + T(x)$ divided $x^n + 1$, the quotient would have degree d_{BCH}. But the degree of every factor of $(x^n + 1)/(x + 1)$ is a multiple of m, which exceeds d_{BCH}. The only other possibility is $d_{\text{BCH}} = 1$, which is excluded by the hypothesis that $m < n - 1$. Q.E.D.

Corollary 16.322 gives another proof that the Golay code has distance greater than 5. It also shows that the binary BCH code with length 43, 15 information digits, and 28 check digits has distance greater than 7, which is the BCH bound.

The minimum-distance bounds help us to obtain weight enumerations in two ways. The minimum-distance bounds on code $\mathfrak{A}$ eliminate some of the unknown A's, and the minimum-distance bounds on code $\mathfrak{B}$ increase the minimum value of s for which B_s is unknown. Even with the best minimum-distance bounds, however, it is still often not possible to determine the weight enumerators from the Pless identities. There are too many unknown A's. Fortunately, however, it is sometimes possible to eliminate some of these remaining A's by other methods, which we shall now consider.

If $A_n = 1$ in a binary code of odd block length, then $A_j = A_{n-j}$ for all j. Since $n - j$ is even if j is odd and vice versa, we can determine the weight enumerator of such a code directly from the weight enumerator of the expurgated code, which consists of those codewords of the original code which have even weight. In the expurgated code, $A_j = 0$ for all odd j.

An investigation of Table 16.1 reveals several codes (including the expurgated Golay code with $n = 23$, $k = 11$) whose nonzero weights satisfy an even stronger constraint. Not only is $A_j = 0$ unless $j \equiv 0 \bmod 2$, but $A_j = 0$ unless $j \equiv 0 \bmod 4$. Using arguments based on projective

geometry, Gleason proved that this property is shared by all QR codes of length $\equiv -1 \bmod 8$. This result enables one to determine the weight enumerators of the extended QR codes of block lengths 8, 24, 32, and 48 via the Pless identities. For example, the minimum odd weight of the QR code of length 47 is known to be ≥ 9 by Corollary 15.27. Since every codeword in the extended QR code has weight divisible by 4, the only unknowns are $A_{12} = A_{36}$, $A_{16} = A_{32}$, $A_{20} = A_{28}$, and A_{24}. Since the dual of an extended QR code is equivalent to the code itself, $B_i = 0$ for $i = 1, 2, \ldots, 11$, and the A's can be determined by the Pless identities.

Theorem 16.33, due to Solomon and McEliece (1966), is a generalization of Gleason's theorem. As usual, I use the notation $\tilde{g}(x)$ to denote the reciprocal polynomial of $g(x)$.

Theorem 16.33 (Solomon-McEliece) If $x^n - 1$ divides $g(x)\tilde{g}(x)$, then the weight of every codeword of the binary cyclic code generated by $g(x)$ is a multiple of 4.

Proof According to the hypothesis of the theorem, if $g(\alpha^k) \neq 0$, then $g(\alpha^{-k}) = 0$ and $S_{-k} = 0$. Therefore,

$$S_k S_{-k} = 0 \qquad \text{for all } k \tag{16.331}$$

If $C(x)$ is any codeword, its weight $w(C)$ may be represented in binary as

$$w(C) = \sum_i w_i(C) 2^i$$

Theorem 16.33 asserts that Eq. (16.331) implies that $w_0(C) = w_1(C) = 0$ for all codewords C. To prove this assertion, we must obtain an expression for w_i in terms of the S's. To this end, Solomon (1962) has defined

$$\Gamma_j = \sum_{i_1} \sum_{i_1 < i_2} \cdots \sum_{i_{j-1} < i_j} C_{i_1} C_{i_2} \cdots C_{i_j} \qquad \text{sum in GF(2)} \tag{16.332}$$

It is evident that

$$\binom{w(C)}{j} = \sum_{i_1} \sum_{i_1 < i_2} \cdots \sum_{i_{j-1} < i_j} C_{i_1} C_{i_2} \cdots C_{i_j} \qquad \text{real sum}$$

so that $\Gamma_j(C) \equiv \binom{w(C)}{j} \bmod 2$. According to Lucas' theorem 4.71,

$$\binom{w(C)}{2^i} \equiv w_i(C) \bmod 2$$

so that $w_i(C) = \Gamma_{2^i}(C)$. We must now express $\Gamma_{2^i}(C)$ in terms of the S's. Clearly,

$$\Gamma_1(C) = \sum_i C_i = \sum_i C_i\alpha^0 = S_0 = 0$$

by Eq. (16.331). From Eq. (16.332), $\Gamma_2(C) = \sum_{i<j}\sum C_iC_j$.

Using Eq. (16.314), this becomes

$$\begin{aligned}\Gamma_2(C) &= \sum_{i<j}\sum \left(\sum_k S_k\alpha^{-ik}\right)\left(\sum_K S_K\alpha^{-jK}\right) \\ &= \sum_k\sum_K S_kS_K \sum_{i<j}\sum \alpha^{-(ik+jK)}\end{aligned} \tag{16.333}$$

Setting

$$\sum_k\sum_K = \sum_{k<K}\sum + \sum_{k>K}\sum + \sum_{k=K}$$

Eq. (16.333) becomes

$$\Gamma_2(C) = \sum_{k<K}\sum S_kS_K \sum_{i<j}\sum (\alpha^{-(ik+jK)} + \alpha^{-(iK+jk)}) + \sum_k S_k^2 \sum_{i<j}\sum \alpha^{-k(i+j)} \tag{16.334}$$

As the pair i, j $(i < j)$ assumes each of the permissible $\binom{n}{2}$ values, the sum $i + j$ assumes each of its possible values (mod n) exactly $(n - 1)/2$ times. Therefore,

$$\sum_{i<j}\sum \alpha^{-k(i+j)} = \frac{n-1}{2}\sum_{J=0}^{n-1}\alpha^{-kJ} = \begin{cases} 0 & \text{if } k \not\equiv 0 \\ \dfrac{(n-1)n}{2} & \text{if } k = 0 \end{cases}$$

and

$$\sum_k S_k^2 \sum_{i<j}\sum \alpha^{-k(i+j)} = S_0^2\frac{(n-1)n}{2} = 0$$

by Eq. (16.311). To evaluate the other term in Eq. (16.334), we notice that

$$\sum_{i<j}\sum (\alpha^{-(ik+jK)} + \alpha^{-(iK+jk)}) = \sum_{i\neq j}\sum (\alpha^{-(ik+jK)}) = \left(\sum_i \alpha^{-ik}\right)\left(\sum_j \alpha^{-jK}\right) + \sum_i \alpha^{-(k+K)i}$$

Substituting this expression into Eq. (16.334) gives

$$\Gamma_2(C) = \sum_{k<K} \sum S_k S_K \sum_i \alpha^{-(k+K)i}$$
$$= \sum_{k=1}^{(n-1)/2} S_k S_{-k} = 0$$

by Eq. (16.331). Q.E.D.

Solomon and McEliece have also determined explicit formulas for $\Gamma_4(C)$, $\Gamma_8(C)$, . . . in terms of the S's. Their formula for $\Gamma_4(C)$ has proved helpful in determining the weights of several codes, but the formulas for most of the Γ's are so complicated that they are difficult to apply in practice.

As a corollary to Theorem 16.33, we might notice that if $r \leq [(m-1)/2]$, then the weight of every codeword of the rth-order RM code is a multiple of 4. To see this, we need only observe that if $w(j)$ (the number of ones in the binary representation of the integer j) is no greater than $m/2$, then $g(\alpha^j) = 0$, whereas if $w(j) \geq m/2$, then $w(2^m - 1 - j) = m - w(j) \leq m/2$, so that $g(\alpha^{-j}) = 0$. Actually, it is possible to place even stronger restrictions on the weights of the codewords in a RM code. The only possible weights in the first-order RM code are 0, 2^{m-1}, and 2^m. According to Theorem 16.34, the weights of the second-order RM code are also severely restricted.

Theorem 16.34 [Kasami (1968)] The weight of every codeword in any subcode of the second-order RM code of block length 2^m is of the form

$$w = 2^{m-1} + \epsilon 2^{(m+i)/2-1}$$

where i is some nonnegative integer $\equiv m \bmod 2$ and $\epsilon = 0, +1$, or -1, depending on the codeword.

Remarks According to Theorem 15.34, every codeword in a second-order RM code can be expressed in the form

$$\sum_i \sum_j B_{i,j} x_i x_j + \sum B_i x_i + B \tag{16.341}$$

where $B_{i,j}$, B_i, and $B \in \mathrm{GF}(2)$, $x_i = \mathbf{C}^{(i)}$, and $x_i x_j = \mathbf{C}^{(i)} \otimes \mathbf{C}^{(j)}$. To simplify expression (16.341), we invoke Theorem 16.35, which is due

to Dickson (1901).† We postpone the proof of Theorem 16.34 until after we have proved Dickson's theorem and its corollary. Although the first term in expression (16.341) is a quadratic form over GF(2), we prove Dickson's theorem over GF(2^m). The proof for arbitrary m is only slightly more difficult than the proof for $m = 1$.

Theorem 16.35 (Dickson) Any quadratic form in I variables

$$F = \sum_{i=1}^{I} \sum_{j=1}^{I} B_{i,j} x_i x_j \qquad B_{i,j} \in \mathrm{GF}(2^m)$$

can be reduced by an invertible linear transformation of the x's to one of the canonical forms

$$F = \begin{cases} x_k^2 + \sum_{i=1}^{(k-1)/2} x_{2i-1} x_{2i} & k \text{ odd} \\ \lambda(x_k^2 + x_{k-1}^2) + \sum_{i=1}^{k/2} x_{2i-1} x_{2i} & k \text{ even} \end{cases}$$

where k is an integer $\leq I$ and $\lambda = 0$ or a particular value in GF(2^m) with Tr $(\lambda) = 1$.

Proof Define k as the least number of variables in which F can be expressed by an invertible linear transformation of the x's.

We shall first prove that if F depends on x_1 and $k \geq 3$, there is no loss of generality in assuming that $B_{1,1} = 0$. If $B_{1,1} \neq 0$ and $B_{2,2} = 0$, we can exchange x_1 and x_2. If $B_{i,j} + B_{j,i} = 0$ for all $i \neq j$, then F is a perfect square, and $k = 1 < 3$. Thus, we may assume that $B_{2,2} \neq 0$ and $B_{2,3} + B_{3,2} \neq 0$. Replacing $B_{i,j} + B_{j,i}$ by $\check{B}_{i,j}$, we have‡

$$F = B_{2,2} x_2^2 + x_2 \sum_{i \neq 2} B_{i,2} x_i + \sum_{\substack{i \leq j \\ i \neq 2, j \neq 2}} \sum B_{i,j} x_i x_j$$

Letting $\check{x}_3 = \sum_{i \neq 2} B_{i,2} x_i$, $\check{x}_i = x_i$ for $i \neq 3$ tranforms F into

$$\sum_{i \leq j} \sum B_{i,j} x_i x_j$$

† See p. 197 of the 1958 reprinted edition.

‡ Of course, we really have

$$F = B_{2,2} x_2^2 + x_2 \Sigma \check{B}_{i,2} x_i + \Sigma\Sigma \check{B}_{i,j} x_i x_j$$

But since only the form is important, we immediately drop the hats. This practice is continued throughout this proof.

where $B_{1,2} = 0$. Letting $\check{x}_2 = \sqrt{B_{1,1}}\, x_1 + \sqrt{B_{2,2}}\, x_2$ and $\check{x}_i = x_i$ for $i \neq 2$ eliminates $B_{1,1}$. Thus, there is no loss of generality in assuming that if F depends on x_1 and $k \geq 3$, then $B_{1,1} = 0$.

If $B_{1,j} = 0$ for all j, then F is independent of x_1, so we may assume that $B_{1,2} \neq 0$. Setting $\check{x}_2 = \sum_j B_{1,j}x_j$, $\check{x}_i = x_i$ for $i \neq 2$ reduces F to $x_1x_2 + \sum_{2 \leq i \leq j} \sum B_{i,j}x_ix_j$. Setting $\check{x}_1 = x_1 + \sum_j B_{2,j}x_j$ reduces F to $x_1x_2 + \check{F}$, where $\check{F}$ is a quadratic form in $x_3, x_4, \ldots, x_m$.

Thus, we have shown that any quadratic form which depends on three or more variables can be reduced to the form $F = x_1x_2 + \check{F}$, where $\check{F}$ is independent of x_1 and x_2. If $\check{F}$ depends on three or more variables, we can continue to reduce $\check{F}$ to $x_3x_4 + \check{\check{F}}$, etc., until we eventually reduce F to a quadratic form which depends on fewer than three variables. The only quadratic form in zero variables is 0; the only quadratic form in one variable is x^2; the only quadratic form in two variables (x and y) is $Ax^2 + Bxy + Cy^2$. If $AC = 0$, this form can be reduced to xy by an obvious substitution; if $AC \neq 0$, we replace $\sqrt{A}\, x$ by x and $\sqrt{C}\, y$ by y, obtaining $x^2 + Bxy + y^2$. If $B = 0$, this is a perfect square. If $B \neq 0$, we set $\check{y} = By$, $F = x^2 + xy + y^2/B^2$. The substitution $x = \check{x} + uy$ changes F to $x^2 + xy + (u^2 + u + 1/B^2)y^2$. If $\mathrm{Tr}\,(1/B^2) = 0$, then u may be selected so that $u^2 + u + 1/B^2 = 0$; if $\mathrm{Tr}\,(1/B^2) = 1$, then u may be selected so that $u^2 + u + 1/B^2 = \lambda^2$. Setting $\check{x} = x/\sqrt{\lambda}$, $\check{y} = y\sqrt{\lambda}$ changes F to $xy + \lambda(x^2 + y^2)$. Q.E.D.

Corollary 16.351 Any polynomial of degree ≤ 2 in a finite number of variables over a finite field of characteristic 2 can be reduced by an invertible affine transformation of its variables to one of the canonical forms

$$F = \begin{cases} x_k^2 + \sum_{i=1}^{(k-1)/2} x_{2i-1}x_{2i} + B_kx_k + B & \\ x_k^2 + \sum_{i=1}^{(k-1)/2} x_{2i-1}x_{2i} + B_{k+1}x_{k+1} + B & B \times B_{k+1} = 0 \\ \lambda(x_k^2 + x_{k-1}^2) + \sum_{i=1}^{k/2} x_{2i-1}x_{2i} + B_{k+1}x_{k+1} + B & B \times B_{k+1} = 0 \end{cases}$$

Proof The identity

$$x_{2i-1}x_{2i} + B_{2i-1}x_{2i-1} + B_{2i}x_{2i} = (x_{2i-1} + B_{2i})(x_{2i} + B_{2i-1}) + B_{2i-1}B_{2i}$$

enables us to eliminate all the other linear terms by appropriate translations. Q.E.D.

Proof of Theorem 16.34 Since affine transformations preserve polynomials of degree ≤ 1, they transform codewords of the first-order RM code into other codewords of the first-order RM code. For example, a typical affine transformation might take $\mathbf{C}^{(1)}$ into $\mathbf{C}^{(2)} + \mathbf{C}^{(5)} + \mathbf{1}$. In view of Corollary 16.351 and the fact that $\mathbf{C}^{(i)} \otimes \mathbf{C}^{(i)} = \mathbf{C}^{(i)}$, we can reduce expression (16.341) to one of the forms

$$\sum_{k=1}^{j} \mathbf{C}^{(2k-1)}\mathbf{C}^{(2k)} + \begin{cases} \mathbf{0}, & j \leq m/2 \\ \mathbf{1}, & j \leq m/2 \\ \mathbf{C}^{(2j+1)}, & j < m/2 \end{cases} \tag{16.352}$$

where $j \leq m/2$. The weight of the codeword $\sum_{k=1}^{j} \mathbf{C}^{(2k-1)}\mathbf{C}^{(2k)}$ is the number of coordinates, $i = 0, 1, 2, \ldots, n-1, \infty$, such that $\sum_{k=1}^{j} C_i^{(2k-1)}C_i^{(2k)} = 1$, which is the number of ways in which the m binary coordinates $U_1, U_2, \ldots, U_m$ can satisfy the equation

$$\sum_{k=1}^{j} U_{2k-1}U_{2k} = i \tag{16.353}$$

if $i = 1$. For $i = 0, 1$, let N_i be the number of solutions of Eq. (16.353). Then

$$\begin{aligned} N_0 &= 2^{m-2j} \sum_{i \text{ even}} \binom{j}{i} 3^i \\ N_1 &= 2^{m-2j} \sum_{i \text{ odd}} \binom{j}{i} 3^i \\ N_0 + N_1 &= 2^{m-2j}(3+1)^j = 2^m \\ N_0 - N_1 &= 2^{m-2j}(3-1)^j = 2^{m-j} \\ N_0 &= 2^{m-1} + 2^{m-j-1} \\ N_1 &= 2^{m-1} - 2^{m-j-1} \end{aligned} \tag{16.354}$$

We then have

$$w\left(\sum_{k=1}^{j} \mathbf{C}^{(2k-1)}\mathbf{C}^{(2k)}\right) = N_1 = 2^{m-1} - 2^{m-j-1}$$

$$w\left(\sum_{k=1}^{j} \mathbf{C}^{(2k-1)}\mathbf{C}^{(2k)} + \mathbf{1}\right) = N_0 = 2^{m-1} + 2^{m-j-1}$$

$$w\left(\sum_{k=1}^{j} \mathbf{C}^{(2k-1)}\mathbf{C}^{(2k)} + \mathbf{C}^{(2j+1)}\right) = 2^{m-1}$$

because exactly half of the 2^m binary m-tuples satisfy the equation

$$\sum_{k=1}^{j} U_{2k-1}U_{2k} = U_{2j+1}$$

Setting $j = (m - i)/2$ in these expressions of the weights results in Theorem 16.34. Q.E.D.

★ 16.4 KASAMI'S WEIGHT ENUMERATORS FOR CERTAIN SUBCODES OF THE SECOND-ORDER RM CODE

16.41 REDUCTION AND TRIANGULARIZATION OF THE PLESS IDENTITIES

For subcodes of the second-order RM code Kasami's theorem 16.34 assures us that the only possible nonzero A's are A_0, A_N, $A_{N/2}$, $A_{f(i)}$, and $A_{N-f(i)}$, where $i \equiv m \bmod 2$ and $f(i) = 2^{m-1} + 2^{(m+i)/2-1}$. The Pless identities (16.24) with $w = 2^{m-1}$ then become

$$(A_0 + A_N)2^{mr} + \sum_{\substack{i \equiv m \\ \bmod 2}} (-2^{(m+i)/2})^r A_{f(i)} + \sum_{\substack{i \equiv m \\ \bmod 2}} (2^{(m+i)/2})^r A_{N-f(i)} = 2^k \sum_{j=0}^{r} B_j F_r^{(j)}(N) \quad (16.411)$$

where I denote the block length by $N = 2^m$ to avoid confusion with the block length of the shortened codes, $n = 2^m - 1$. If we assume that $A_N = 1$, then $A_j = A_{N-j}$ for all j. The all-one vector is an overall parity check on the dual code, so $B_j = 0$ for all odd j. Replacing r by $2r$ and j by $2j$, Eq. (16.411) becomes

$$2\left(2^{2mr} + \sum_{\substack{i \equiv m \\ \bmod 2}} 2^{mr+ir} A_{f(i)}\right) = 2^k \sum_{j=0}^{r} B_{2j} F_{2r}^{(2j)}(N) \qquad r > 0 \quad (16.412)$$

I define

$$v = \frac{k-1}{m} \quad (16.413)$$

Using Eq. (16.413) and $N = 2^m$, Eq. (16.412) becomes

$$\sum_{\substack{i \equiv m \\ \bmod 2}} 2^{ir} A_{f(i)} = N^{v-r} \sum_{j=0}^{r} B_{2j} F_{2r}^{(2j)}(N) - N^r \quad (16.414)$$

If we multiply Eq. (16.414) by z^{2r} and sum on r from 1 to ∞, we obtain

$$\sum_{\substack{i \equiv m \\ \bmod 2}} \frac{2^i z^2 A_{f(i)}}{1 - 2^i z^2} = N^v \sum_j B_{2j} \left[F^{(2j)}\left(N; \frac{z}{\sqrt{N}}\right) - 1 \right] - \frac{Nz^2}{1 - Nz^2} \quad (16.415)$$

From Eq. (16.242) and Table 16.2, we can compute

$$F^{(0)}\left(N;\frac{z}{\sqrt{N}}\right)-1=\frac{N_{(1)}z^2}{N}+\frac{[3N_{(2)}+N_{(1)}]z^4}{N^2}+\frac{[15N_{(3)}+15N_{(2)}+N_{(1)}]z^6}{N^3}+\cdots$$
$$=z^2+\left(3-\frac{2}{N}\right)z^4+\left(15-\frac{30}{N}+\frac{16}{N^2}\right)z^6+\left(105-\frac{420}{N}+\frac{588}{N^2}-\frac{272}{N^3}\right)z^8+\cdots \quad (16.416)$$

In order to eliminate $A_{f(i_1)}, A_{f(i_2)}, \ldots, A_{f(i_I)}$ from Eq. (16.415), we can multiply Eq. (16.415) by $\prod_{j=1}^{I}(1-2^{i_j}z^2)$ and then equate the coefficients of z^I. For example, if we multiply Eq. (16.415) by $1-2z^2$ and use Eq. (16.244), we obtain

$$S(z)(1-2z^2)=N^v\left[z^2+\left(3-\frac{2}{N}\right)z^4+\cdots+2!B_2\left(\frac{z^2}{N}+\cdots\right)+4!B_4\left(\frac{z^4}{N^2}+\cdots\right)\right]-Nz^2-(N^2-2N)z^4-\cdots$$

Equating the coefficients of z^4 gives

$$\sum_{\substack{i\equiv m\\ \bmod 2}}2^i(2^i-2)A_{f(i)}=N(N-2)(N^{v-2}-1)+4!N^{v-2}B_4 \qquad \text{if } B_2=0$$

This is the third equation in Table 16.3. By similar calculations, we can obtain all the equations listed in Table 16.3. These equations are the reduced Pless identities (16.414) in their triangularized form.

16.42 ODD m, SOME LOW-RATE BCH CODES

We begin with the first-order RM code, as listed in the first row of Table 16.4. The weight distribution for this code was known to Reed and Muller. The shortened first-order RM code is cyclic. Its check polynomial has m roots, which are $\alpha^{-1}, \alpha^{-2}, \alpha^{-4}, \ldots$ or, equivalently, $\alpha^{n-1}, \alpha^{n-2}, \alpha^{n-4}, \ldots$, where $n=2^m-1$. The $\log_\alpha$ of these roots are represented in binary by the cyclic shifts of $01111\cdots$, as indicated in the first column of Table 16.4. This cyclic code is an expurgated BCH code of rate $(m+1)/n$, which is a punctured first-order RM code. As is well known, the weight distribution of the first-order RM code is $A_0=A_N=1; A_{N/2}=2N-2$.

We next consider the extended BCH code of rate $(2m+1)/N$, as listed in the second row of Table 16.4. The weight distribution for this code was first found by Kasami. The check polynomial of the expur-

gated BCH code has $2m$ roots, whose $\log_\alpha$ are represented in binary by the cyclic shifts of 011111111 and 011101111, as indicated in the first column of the table. According to the BCH bound, the minimum distance of the extended BCH code of rate $(2m + 1)/N$ is at least $2^{m-1} - 2^{(m-1)/2}$, so $A_{f(i)} = 0$ for all $i > 1$. Since this code is a supercode of the first-order RM code, its dual is a subcode of the dual of the first-order RM code. Thus, since $B_2 = 0$ on the previous line, the dual of the first-order RM code (which is in fact the extended Hamming code) has no codewords of weight 2, so the dual of the extended BCH code of rate $(2m + 1)/N$ cannot have any codewords of weight 2 either. In other words, $B_2 = 0$. We are therefore able to determine $A_{f(1)}$ directly from Eq. (T2) and $A_{N/2}$ from (T1) in Table 16.3. According to Eq. (T3), $0 = 0 + 4!B_4$, from which we conclude that $B_4 = 0$.

Before continuing with the extended BCH code of the next higher rate, we notice that for sufficiently large m, there are many low-rate extended BCH codes which are subcodes of the second-order RM code. Indeed, every m-digit binary sequence with three or more zeros must have a cyclic shift whose leading digit is 0 and whose next 0 occurs no later than the $[m/3] + 1$ place in the sequence. For example, if $m = 15$, the extreme case is 011110111101111. If $v \leq (m + 1)/2 - ([m/3] + 1) + 2$, then the v m-digit binary sequences 011111111111111, 011111101111111, 011111011111111, . . . , and 011110111111111 and all their cyclic shifts must exceed at least one cyclic shift of any m-digit binary sequence with three or more 0's. Therefore, if $v \leq (m + 1)/2 - [m/3] + 1$ and m is odd, then the extended binary BCH code of rate $(mv + 1)/2^m$ is a subcode of the second-order RM code.

We next consider the extended BCH code of rate $(3m + 1)/N$, $m \geq 7$, as listed in the third row of Table 16.4. According to the BCH bound, the minimum distance of this code is at least $2^{m-1} - 2^{(m+1)/2}$, so $A_{f(i)} = 0$ for all $i > 3$. Since this code is a supercode of the extended BCH code of rate $(2m + 1)/N$, its dual is a subcode of the dual of the extended BCH code of rate $(2m + 1)/N$. Therefore, since $B_4 = 0$ on the previous line, we still have $B_4 = 0$. This enables us to determine $A_{f(3)}$ from Eq. (T3); $A_{f(1)}$ from (T2); and $A_{N/2}$ from (T1). According to (T4), $0 = 0 + 6!B_6$, from which we conclude that $B_6 = 0$.

We next consider the extended BCH code of rate $(4m + 1)/N$, $m \geq 11$. By the BCH bound, $A_{f(i)} = 0$ for all $i > 5$. From the previous code, $B_6 = 0$. We can therefore determine $A_{f(5)}$ from Eq. (T4); $A_{f(3)}$ from (T3); $A_{f(1)}$ from (T2); $A_{N/2}$ from (T1). According to (T5), $0 = -30N(N - 2)(N - 1)(N - 4) + 8!B_8$, from which we can find B_8.

We next consider the extended BCH code of rate $(5m + 1)/N$, $m \geq 17$. By the BCH bound, $A_{f(i)} = 0$ for all $i > 7$. From the previous code, $B_8 \leq N(N - 1)(N - 2)(N - 4)/(8 \times 7 \times 6 \times 4)$. On

the other hand, since this code is a subcode of the second-order RM code, its dual must be a supercode of the dual of the second-order RM code, which has exactly $N(N-1)(N-2)(N-4)/(8 \times 7 \times 6 \times 4)$ codewords of weight 8 by Theorem 15.329. We conclude that $B_8 = N(N-1)(N-2)(N-4)/(8 \times 7 \times 6 \times 4)$. We then calculate $A_{f(7)}$ from Eq. (T5); $A_{f(5)}$ from (T4); $A_{f(3)}$ from (T3); $A_{f(1)}$ from (T2); $A_{N/2}$ from (T1). According to (T6),

$$0 = N(N-2)[150(N-1)(N-4)(25N+48)] - 8!(125N+240)B_8 + 10!B_{10}$$

from which $B_{10} = 0$.

We next consider the extended BCH code of rate $(6m+1)/N$, $m \geq 23$. By the BCH bound, $A_{f(i)} = 0$ for all $i > 9$. From the previous code, $B_{10} = 0$. We can therefore determine $A_{f(9)}$ from Eq. (T6); $A_{f(7)}$ from (T5); . . . .

. . . . . .

At each code in this sequence, we have computed the next B_j. Thanks to extremely good luck, this next B_j has always turned out to be identical to the number of words of weight j in the dual of the second-order RM. Having thus bounded this B_j for the next code in the sequence between these two identical values, we were able to continue the computations and find the weight enumerator for yet another sequence of low-rate BCH codes. Our computations ended with the BCH codes of rate $(6m+1)/N$, $m \geq 23$, not because we ran out of luck, but because I ran out of patience. Indeed, I cannot resist the conjecture that any so auspicious an event which occurs five times in a row must continue to occur forever.

CONJECTURE 16.43

If $m \geq 6v - 13$, the number of words of weight $2v$ in the dual of the extended BCH code of rate $(mv+1)/N$ is identical to the number of words of weight $2v$ in the dual of the second-order RM code.

This conjecture implies that for odd m, the weight enumerators for the entire sequence of low-rate BCH codes which are subcodes of the second-order RM code can be found by the power-moment identities alone. As an optimist, I would hope not only for a proof of Conjecture 16.43 but for a tractable, explicit expression for all the enumerators. From Table 16.4, the number of codewords of least nonzero weight in the extended BCH code of rate $(vm+1)/2^m$, m odd, $m \geq 6v - 13$, is evidently given by

$$A_{f(i)} = \frac{(N-1)N}{2^i} \prod_{j=1}^{(i-1)/2} \frac{N - 2^{2j-1}}{2^i - 2^{2j-1}}$$

where $i = 2v - 3$. It would be nice to find a direct proof of this formula (or for that matter, any general proof). It would also be nice to find a characterization of these low-weight codewords, analogous to Theorem 15.329.

16.44 ODD m, HIGH-RATE BCH CODES

For fixed m, only three extended BCH codes have duals which are subcodes of the second-order RM code. These are the 1-, 2-, and 3-error-correcting BCH codes. If $t \geq 4$, then the dual of the t-error-correcting extended binary BCH code cannot be a subcode of the second-order RM code, because α^7 must be a root of the generator of the BCH code and 7 in binary is 111, which has weight 3.

For odd m, the weight enumerators of the duals of the extended double- and triple-error-correcting binary BCH codes can be determined from the formulas in Table 16.3. For the dual of the double-error-correcting code, $v = 2$, and the BCH bound assures that $B_2 = B_4 = 0$. Equation (T3) then becomes

$$\sum_{i \text{ odd}} 2^i(2^i - 2)A_{f(i)} = 0$$

Since $2^i(2^i - 2)A_{f(i)} \geq 0$ for all odd i, we deduce that $2^i(2^i - 2)A_{f(i)} = 0$ for all odd i, and $A_{f(i)} = 0$ for all $i > 1$. We can then determine $A_{f(1)}$ from Eq. (T2) and $A_{N/2}$ from (T1).

We next consider the dual of the extended triple-error-correcting binary BCH code. Since $B_2 = B_4 = B_6 = 0$ by the BCH bound, Eq. (T4) becomes

$$\sum_{i \text{ odd}} 2^i(2^i - 2)(2^i - 8)A_{f(i)} = 0$$

This implies that $A_{f(i)} = 0$ for all $i > 3$. We then determine $A_{f(3)}$, $A_{f(1)}$, and $A_{N/2}$ from Eqs. (T3), (T2), and (T1). The results are given in Table 16.5.

From these known weight distributions of the duals of the extended double- and triple-error-correcting BCH codes, we can compute the weight distributions of the extended double- and triple-error-correcting BCH codes from the MacWilliams formula (16.21). We omit this straightforward but tedious calculation.

For odd m, the derivation of the weight enumerators of the extended binary BCH codes which are subcodes (or duals of subcodes) of the second-order RM code was greatly simplified by the appearance of the terms $N^{v-2} - 1$ and $N^{v-3} - 1$ in Eqs. (T3) and (T4), respectively. For even m, the comparable equations [(T7) and (T8)] have no such terms, and the derivations of the weight enumerators are much more complicated. Far

fewer cases have been solved, and the solutions to these cases have been obtained only with the aid of results based on a more detailed study of the codes. Theorems 16.45 to 16.46 are such results. As in Sec. 4.7, I use the notation $M^{(j)}(x)$ to denote the minimal polynomial of α^j.

Theorem 16.45 (Kasami) The binary cyclic code of block length $2^m - 1$ whose generator polynomial is $g(x) = M^{(1)}(x) \prod_i M^{(2^{e_i}+1)}(x)$ has exactly $a_3 = (2^m - 1)(2^{\text{gcd}} - 2)/3!$ codewords of weight 3, where gcd denotes the greatest common divisor of all the e_i and m.

Proof Since the code is invariant under the transitive permutation group consisting of the cyclic shifts, the number of codewords of weight 3 is $(2^m - 1)/3$ times the number of codewords of weight 3 which have a one at location α^0. If $1, x$, and y are the locations of the three ones in a codeword of weight 3, then $x,y \in \text{GF}(2^m) - \text{GF}(2)$, and

$$x + y + 1 = 0 \tag{16.451}$$
$$x^{2^{e_i}+1} + y^{2^{e_i}+1} + 1 = 0 \qquad \text{for all } i \tag{16.452}$$

Substituting Eq. (16.451) into Eq. (16.452) yields

$$(y + 1)^{2^{e_i}+1} + y^{2^{e_i}+1} = 1 \qquad \text{for all } i$$

Expanding gives

$$y^{2^{e_i}+1} + y^{2^{e_i}} + y + 1 + y^{2^{e_i}+1} = 1 \qquad \text{for all } i$$
$$y^{2^{e_i}} + y = 0 \qquad \text{for all } i$$
$$y \in \text{GF}(2^{e_i}) - \text{GF}(2) \qquad \text{for all } i$$
$$y \in \text{GF}(2^{\text{gcd}}) - \text{GF}(2)$$

There are thus exactly $2^{\text{gcd}} - 2$ choices for the ordered pair of elements (x,y) or $(2^{\text{gcd}} - 2)/2$ choices for the unordered pair (x,y). Q.E.D.

Corollary 16.453 The code of block length $N = 2^m$ which is the extension of the code of Theorem 16.45 has exactly

$$A_4 = N(N - 1)\frac{2^{\text{gcd}} - 2}{4!}$$

codewords of weight 4.

Proof According to Theorem 10.37, the extended code is invariant under the affine permutation group, which is transitive. Therefore, $a_3 = 4A_4/N$, as in Theorem 10.38. Q.E.D.

Theorem 16.46 [Welch (1967)] The equation $\mathrm{Tr}\,(x^{2^a+1}) = 0$ has exactly $2^{2ab-1} + (-1)^{b+1}2^{ab+a-1}$ solutions in $\mathrm{GF}(2^{2ab})$.

Proof Let $u_1, u_2, \ldots, u_{2ab}$ be a basis of $\mathrm{GF}(2^{2ab})$ over $\mathrm{GF}(2)$. Then $x = \sum_{i=1}^{2ab} X_i u_i$, $X_i \in \mathrm{GF}(2)$,

$$x^{2^a+1} = (x^{2^a})x = \sum_{i=1}^{2ab}\sum_{j=1}^{2ab} X_i X_j u_i u_j^{2^a}$$

$$\mathrm{Tr}\,(x^{2^a+1}) = \sum_{i=1}^{2ab}\sum_{j=1}^{2ab} X_i X_j \,\mathrm{Tr}\,(u_i u_j^{2^a})$$

This is a quadratic form in the X_i. According to Dickson's theorem 16.35, there exists a basis $u_1, u_2, \ldots, u_{2ab}$, an integer c, $c \leq ab$, and an element $\lambda \in \mathrm{GF}(2)$ such that

$$\mathrm{Tr}\,(x^{2^a+1}) = X_{2c+1}^2 + \sum_{i=1}^{c} X_{2i-1}X_{2i} \tag{16.461}$$

or

$$\mathrm{Tr}\,(x^{2^a+1}) = \lambda(X_{2c-1}^2 + X_{2c}^2) + \sum_{i=1}^{c} X_{2i-1}X_{2i} \tag{16.462}$$

In the first case, $\mathrm{Tr}\,(x^{2^a+1})$ is independent of $2ab - 2c - 1$ coordinates; in the second case, it is independent of $2ab - 2c$ coordinates. To find c, we must find the number of v's such that

$$\mathrm{Tr}\,[(w+v)^{2^a+1}] = \mathrm{Tr}\,(w^{2^a+1}) \qquad \text{for all } w \in \mathrm{GF}(2^{2ab})$$

We have

$$\begin{aligned}\mathrm{Tr}\,[(w+v)^{2^a+1} - w^{2^a+1}] &= \mathrm{Tr}\,(w^{2^a}v + v^{2^a}w + v^{2^a+1})\\ &= \mathrm{Tr}\,(w^{2^a}v) + \mathrm{Tr}\,(v^{2^a}w) + \mathrm{Tr}\,(v^{2^a+1})\\ &= \mathrm{Tr}\,(w^{2^a}v) + \mathrm{Tr}\,(v^{2^{2a}}w^{2^a}) + \mathrm{Tr}\,(v^{2^a+1})\\ &= \mathrm{Tr}\,[w^{2^a}(v^{2^{2a}} + v)] + \mathrm{Tr}\,(v^{2^a+1})\end{aligned}$$

This is independent of w iff $v^{2^{2a}} + v = 0$, which happens iff $v \in \mathrm{GF}(2^{2a})$. If $v \in \mathrm{GF}(2^{2a}) - 0$, then $(v^{2^a+1})^{(2^a-1)} = 1$, so $v^{2^a+1} \in \mathrm{GF}(2^a)$ and $\mathrm{Tr}\,(v^{2^a+1}) = \sum_{i=1}^{2ab} (v^{2^a+1})^{2^i} = 2b \sum_{i=1}^{a} (v^{2^a+1})^{2^i} = 0$. Therefore, $\mathrm{Tr}\,[(w+v)^{2^a+1}] = \mathrm{Tr}\,(w^{2^a+1})$ for all $w \in \mathrm{GF}(2^{2ab})$ iff $v \in \mathrm{GF}(2^{2a})$. Since $\mathrm{Tr}\,(x^{2^a+1})$ is independent of $2a$ coordinates, we deduce that $2a = 2ab - 2c$ and $\mathrm{Tr}\,(x^{2^a+1})$ is given by Eq. (16.462) rather than Eq. (16.461).

According to Eq. (16.354), N_i, the number of ways in which

$2ab$ binary numbers $X_1, X_2, \ldots, X_{2ab}$ can satisfy the equation

$$\sum_{k=1}^{c} X_{2k-1}X_{2k} = i \qquad c = ab - a$$

is given by

$$N_0 = 2^{2ab-1} + 2^{ab+a-1}$$
$$N_1 = 2^{2ab-1} - 2^{ab+a-1}$$

Therefore, if $\lambda = 0$, the equation $\mathrm{Tr}\,(x^{2^a+1}) = 0$ has $2^{2ab-1} + 2^{ab+a-1}$ solutions in $\mathrm{GF}(2^{2ab})$; if $\lambda = 1$, the equation

$$\begin{aligned} 0 &= \mathrm{Tr}\,(x^{2^a+1}) \\ &= X_{2c-1}X_{2c} + X_{2c-1}^2 + X_{2c}^2 + \sum_{i=1}^{c-1} X_{2c-1}X_{2i} \\ &= 1 + (X_{2c} + 1)(X_{2c-1} + 1) + \sum_{i=1}^{c-1} X_{2c-1}X_{2i} \end{aligned}$$

has $2^{2ab-1} - 2^{ab+a-1}$ solutions in $\mathrm{GF}(2^{2ab})$. To determine whether $\lambda = 0$ or 1, we note that the number of nonzero solutions of the equation $\mathrm{Tr}\,(x^{2^a+1}) = 0$ must be a multiple of $2^a + 1$. Therefore, we have

$$2^{2ab-1} + (-1)^{\lambda}2^{ab+a-1} - 1 \equiv 0 \bmod 2^a + 1$$

and

$$2^{2ab} + (-1)^{\lambda}2^{ab+a} \equiv 2 \bmod 2^a + 1$$

Since $2^a \equiv -1$, this becomes

$$\begin{aligned} 1 + (-1)^{\lambda}(-1)^{(b+1)} &\equiv 2 \bmod 2^a + 1 \\ (-1)^{(\lambda+b+1)} &\equiv 1 \\ (-1)^{\lambda} &= (-1)^{b+1} \end{aligned}$$

Q.E.D.

16.47 EVEN m, LOW-RATE EXTENDED BCH CODES

We next consider the extended binary BCH code of rate $(3m/2 + 1)/N$, as listed in Table 16.5. According to the BCH bound, the minimum distance of this code is at least $2^{m-1} - 2^{(m-2)/2}$, so $A_{f(i)} = 0$ for all $i > 0$. Since this code is a supercode of the first-order RM code, its dual is a subcode of the Hamming code, and $B_2 = 0$. With $v = 3/2$, we determine $A_{f(0)}$ and $A_{N/2}$ from Eqs. (T2) and (T1) in Table 16.3.

We next consider the extended binary BCH code of rate $(5m/2 + 1)/N$. According to the BCH bound, $A_{f(i)} = 0$ for all $i > 2$. Since the greatest common divisor of m, $m/2$, and $m/2 - 1$ is 1, Corollary

16.453 guarantees that $B_4 = 0$. With $v = 5/2$, we determine $A_{f(2)}$, $A_{f(0)}$, and $A_{N/2}$ from Eqs. (T7), (T2), and (T1).

Unfortunately, the weight distributions of the binary BCH codes of rates $(7m/2 + 1)/N$, $(9m/2 + 1)/N$, . . . are not known.

16.48 EVEN m, DOUBLE-ERROR-CORRECTING EXTENDED BCH CODES

In order to enumerate the weights in the double-error-correcting BCH codes, I adopt a relatively indirect approach. I first exhibit a complete decoding algorithm for these codes. This enables us to count the words of weight 3 in cosets of weight 3, and the words of weight 3 in cosets of weight 2. From this, it is easy to determine the number of codewords of weight 5 in the (cyclic) BCH code of length $n = 2^m - 1$. This information proves just sufficient to determine the weight enumerator via the formulas in Table 16.3. Kasami has given a more direct derivation of the weight enumerator for double-error-correcting BCH codes. However, the complete decoding algorithm and the coset enumerator are of considerable interest in their own right, so I use the more indirect derivation.

If there are no more than two errors, then, according to Sec. 1.4 (or Algorithm 7.4), the error-locator polynomial for a double-error-correcting BCH code is given by

$$\sigma(z) = \begin{cases} 1 & \text{if there are 0 errors} \\ 1 + S_1 z & \text{if there is 1 error} \\ 1 + S_1 z + \dfrac{\Delta_1 z^2}{S_1} & \text{if there are 2 errors} \end{cases}$$

where $\Delta_1 = S_3 + S_1 S_2 = S_3 + S_1^3$. If $\mathrm{Tr}\,(\Delta_1/S_1^3) \neq 0$, then, according to Theorem 6.695, this $\sigma(z)$ does not have all its roots in GF$(n + 1)$. This decoding algorithm then fails to decode, detecting three or more errors.

If we assume three errors, located in positions X_1, X_2, and X_3, then the parity-check equations tell us S_1 and S_3, where

$$\begin{aligned} X_1 + X_2 + X_3 &= S_1 \\ X_1^3 + X_2^3 + X_3^3 &= S_3 \end{aligned}$$

Setting $w = X_3 + S_1$, we can eliminate X_3 to obtain

$$\begin{aligned} X_1 + X_2 &= w \\ X_1^3 + X_2^3 &= S_3 + S_1^3 + S_1^2 w + S_1 w^2 + w^3 \end{aligned}$$

Adding the cube of the former equation to the latter and then dividing by the former gives

$$X_1 X_2 = \frac{\Delta_1 + S_1^2 w + S_1 w^2}{w}$$

The elements $y_1 = X_1/w$ and $y_2 = X_2/w$ satisfy the equation

$$y^2 + y = \frac{\Delta_1}{w^3} + \left(\frac{S_1}{w}\right)^2 + \frac{S_1}{w}$$

This equation has solutions in the field iff

$$\begin{aligned} 0 &= \mathrm{Tr}\left(\frac{\Delta_1}{w^3} + \left(\frac{S_1}{w}\right)^2 + \frac{S_1}{w}\right) \\ &= \mathrm{Tr}\left(\frac{\Delta_1}{w^3}\right) + \mathrm{Tr}\left(\frac{S_1}{w}\right)^2 + \mathrm{Tr}\left(\frac{S_1}{w}\right) \\ &= \mathrm{Tr}\left(\frac{\Delta_1}{w^3}\right) \end{aligned}$$

Defining $u = \Delta_1/w^3$, we can decode by starting with an arbitrary element u, whose trace is known to be 0. If we can extract the cube root, $w = (\Delta_1/u)^{1/3}$, we can then determine the error locator as

$$\sigma(z) = [1 + (S_1 + w)z]\left[1 + wz + \left(S_1^2 + S_1 w + \frac{\Delta_1}{w}\right)z^2\right]$$

In order to be able to extract the cube root of Δ_1/u for an arbitrary choice of Δ_1, we must be prepared with three different values of u. These values are chosen such that $\mathbf{Tr}(u_1) = \mathbf{Tr}(u_2) = \mathbf{Tr}(u_3) = 0$ and $u_1^{n/3} = 1$; $u_2^{n/3} = \xi$; $u_3^{n/3} = \partial$; where ξ and ∂ are the two elements in $\mathrm{GF}(2^2) - \mathrm{GF}(2)$. Once chosen, the values of the elements u_1, u_2, and u_3 may be wired into the decoder, which then decodes according to the following algorithm.

16.481 Complete decoding algorithm for double-error-correcting binary BCH codes

Calculate $\Delta_1 = S_1^3 + S_1 S_2$.
If $\Delta_1 = S_1 = 0$, set $\sigma(z) = 1$.
If $S_1 \neq 0$ and $\mathrm{Tr}\,(\Delta_1/S_1^3) = 0$, set $\sigma(z) = 1 + S_1 z + (\Delta_1/S_1)z^2$.
If $S_1 = 0$ and $\Delta_1 \neq 0$ or if $\mathrm{Tr}\,(\Delta_1/S_1^3) \neq 0$, set $w = (\Delta_1/u_i)^{1/3}$† and then set

$$\sigma(z) = [1 + (S_1 + w)z]\left[1 + wz + \left(S_1^2 + S_1 w + \frac{\Delta_1}{w}\right)z^2\right]$$

† To find w, the decoder first attempts to extract the cube root of Δ_1/u_1 by the methods of Sec. 11.1. If Δ_1/u_1 has three cube roots in $\mathrm{GF}(2^m)$, any of these roots may be arbitrarily chosen as w; if Δ_1/u_1 has no cube roots in $\mathrm{GF}(2^m)$, the decoder then attempts to extract the cube roots of Δ_1/u_2, etc.

If m is odd, these problems do not arise. Any nonzero field element with trace 0 may be chosen as u_1. There is no need for the elements u_2 and u_3 because Δ_1/u_1 always has a unique cube root in $\mathrm{GF}(2^m)$.

The number of possible choices of the elements u_1, u_2, and u_3 depends on the number of nonzero cubes and noncubes in GF(2^m) with trace 0. These numbers, obtained from Welch's theorem 16.46, are as follows:

The number of nonzero cubes with Trace 0 is

$$\frac{n - 1 \pm 2\sqrt{n+1}}{6}$$

The number of nonzero cubes with Trace 1 is

$$\frac{n + 1 \mp 2\sqrt{n+1}}{6}$$

The number of noncubes with Trace 0 is

$$\frac{n - 1 \mp \sqrt{n+1}}{3}$$

The number of noncubes with Trace 1 is

$$\frac{n + 1 \pm \sqrt{n+1}}{3}$$

Here we define $\pm\sqrt{n+1}$ as $-(-2)^{m/2}$. In any case, there should be no confusion, for the wrong choice of sign will lead to numbers in the various categories which are not integers. With the aid of these results, we can categorize the cosets of the double-error-correcting binary BCH code of block length $n = 2^m - 1$ as in Table 16.6.

For example, we consider the category of syndromes where $S_1 \neq 0$ and Δ_1 is a noncube such that Tr $(\Delta_1/S_1^3) = 1$. There are n choices for S_1, and as many choices for Δ_1 as there are choices for Δ_1/S_1^3, which is the number of noncubes having Trace 1. As we have seen, GF$(n+1)$ contains $(n + 1 \pm \sqrt{n+1})/3$ such elements. This accounts for the entry in the "Number of cosets in category" column. The weight of cosets of this type is obviously 3.

In how many ways can the coset be decoded to an error pattern of weight 3? Well, first we must choose the element u, having Trace 0, such that the cube root of Δ_1/u exists. There are $(n - 1 \mp \sqrt{n+1})/3$ noncubes in the field with Trace 0. Since the squares of the legitimate choices for u are illegitimate, and vice versa, exactly half of these $(n - 1 \mp \sqrt{n+1})/3$ noncubes with Trace 0 are legitimate choices for u. Thus, u may be chosen in $(n - 1 \mp \sqrt{n+1})/6$ different ways. Although there are three ways of choosing w as the cube root of Δ_1/u, each cubic error polynomial may also arise from three different choices of w (not necessarily corresponding to identical choices of u). Therefore, these two factors of 3 cancel, and the number of words of weight 3 in each coset in this category is $(n - 1 \mp \sqrt{n+1})/6$. This accounts for the

entry in the "Multiplicity of minimum weight elements in coset" column of Table 16.6. The other entries in Table 16.6 can be derived in a similar fashion.

Notice that no coset has weight >3. This fact, first discovered by Gorenstein, Peterson, and Zierler (1960), proves that double-error-correcting binary BCH codes of length $2^m - 1$, m even, are quasi-perfect (see Sec. 13.2). From Algorithm 16.48I, it is easily seen that binary BCH codes of length $2^m - 1$, m odd, are also quasi-perfect.

Several questions remain. How many words of weight 3 lie in cosets of weight 3? The answer is

$$\frac{n}{3}\left(\frac{n-1\pm 2\sqrt{n+1}}{6}\right)+\frac{2n}{3}\left(\frac{n-1\mp\sqrt{n+1}}{6}\right)$$
$$+n\left(\frac{n+1\pm\sqrt{n+1}}{3}\right)\left(\frac{n-1\mp\sqrt{n+1}}{6}\right)$$
$$+n\left(\frac{n+1\mp 2\sqrt{n+1}}{6}\right)\left(\frac{n-1\pm 2\sqrt{n+1}}{6}\right)=\frac{n}{3}\frac{n^2-5}{4}$$

The total number of words of weight 3 in all cosets is $\binom{n}{3} = n(n-1)(n-2)/6$. The number of words of weight 3 in cosets of weight 2 must be $n(n-1)(n-2)/6 - n(n^2-5)/12 = n(n-3)^2/12$. For every codeword of weight 5, there are $\binom{5}{3} = 10$ triple-error patterns in cosets of weight 2, so the code must contain $n(n-3)^2/5!$ codewords of weight 5.

Since the double-error-correcting BCH code has $b_5 = n(n-3)^2/5!$ codewords of weight 5, we see from Theorem 10.38 that the extended BCH code has $B_6 = (n+1)n(n-3)^2/6! = N(N-1)(N-4)^2/6!$ codewords of weight 6. Equation (T8) of Table 16.3 then becomes

$$\Sigma 2^i(2^i-1)(2^i-4)A_{f(i)} = 0$$

from which we conclude that $A_{f(i)} = 0$ for all $i > 2$. We then determine $A_{f(2)}$, $A_{f(0)}$, and $A_{N/2}$ from Eqs. (T7), (T2), and (T1). These results are listed in Table 16.5.

16.49 EVEN m, TRIPLE-ERROR-CORRECTING EXTENDED BCH CODES

We conclude this section by considering the dual of the extended triple-error-correcting binary BCH code, with $v = 3$. According to the BCH bound, $B_2 = B_4 = B_6 = 0$. According to Eq. (T8),

$$2^4(2^4-1)(2^4-4)A_{f(4)} + 2^6(2^6-1)(2^6-4)A_{f(6)} + \cdots = 3N(N-1)(N-4)$$

or

$$A_{f(4)} + 84A_{f(6)} + \cdots = \frac{N(N-1)(N-4)}{64\times 15}$$

Kasami (1968) has shown that $A_{f(i)}$ must be divisible by $N(N-1)/16$ and therefore that if $84 < (N-4)/60$, then $A_{f(i)} = 0$ for all $i > 4$. This condition is met if $N \leq 2^{12}$. For these cases, we can find $A_{f(4)}$, $A_{f(2)}$, $A_{f(0)}$, and $A_{N/2}$ from Eqs. (T8), (T7), (T2), and (T1). According to (T9), $B_8 = N(N-1)(N-4)(N^2+6N+68)/8!$. This equation has been proved for $m \leq 12$, and it appears to be a fairly safe conjecture for all m. The conjectured formula for B_8 is equivalent to the assertion that $A_{f(i)} = 0$ for all $i > 4$. A proof of either of these assertions would prove the conjectured weight enumerator for the dual of the extended triple-error-correcting binary BCH code. It is my opinion that the conjectured formula for B_8 can be proved by a coset enumeration on the triple-error-correcting BCH code, along the lines of Sec. 16.48. However, it appears that such an enumeration will be rather involved, and it may be possible to find a more direct proof.

★ 16.5 THE WEIGHT ENUMERATOR FOR THE REED-SOLOMON CODES

According to Theorem 13.35, the Reed-Solomon codes satisfy the general bound $d \leq n + 1 - k$ with equality. This fact enables us to determine the weight enumerator for these codes from Theorem 16.51, which was independently proved by three groups of researchers: Assmus, Mattson, and Turyn (1965); Forney (1966), and Kasami, Lin, and Peterson. Parts of the proofs are attributed to Gleason and Kohlenberg.

Theorem 16.51 If a code of length n and dimension k over GF(q) has minimum Hamming distance $d = n + 1 - k$, then for $w \geq d$, the number of codewords of Hamming weight w is given by

$$A_w = \binom{n}{w} \sum_{j=0}^{w-d} (-1)^j \binom{w}{j} (q^{w-d-j+1} - 1)$$

$$= \binom{n}{w} (q-1) \sum_{i=0}^{w-d} (-1)^i \binom{w-1}{i} q^{w-d-i}$$

Proof Let R, S, and T denote subsets of the code's n positions, with cardinalities $|R|$, $|S|$, and $|T|$ and complements $\bar{R}$, $\bar{S}$, and $\bar{T}$. Let M_R denote the number of codewords which have nonzeros in the positions of set R, and 0 in the positions in $\bar{R}$. With the nonzeros of the all-zero codeword lying on the empty set $\emptyset$, we have $M_\emptyset = 1$. Then the number of codewords which have zeros in the positions in $\bar{S}$ is given by

$$\sum_{\substack{R, \\ R \subseteq S}} M_R = f(S) \tag{16.52}$$

where

$$f(S) = \begin{cases} 1 & \text{if } |S| \leq d-1 \\ q^{|S|+1-d} & \text{if } d \leq |S| \end{cases} \tag{16.53}$$

Equation (16.53) follows from the fact that there exists a unique codeword which takes on arbitrary values on any specified set of k positions, as shown in Sec. 13.3. To solve Eq. (16.52) for an arbitrary $f(S)$, we multiply Eq. (16.52) by some function $\mu(S,T)$ and sum on $S \subseteq T$, obtaining

$$\sum_{\substack{R,\\ R\subseteq S\subseteq T}} \sum_{\substack{S,\\ S\subseteq T}} \mu(S,T)M_R = \sum_{\substack{S,\\ S\subseteq T}} \mu(S,T)f(S) \tag{16.54}$$

If

$$\sum_{\substack{S,\\ R\subseteq S\subseteq T}} \mu(S,T) = \begin{cases} 1 & \text{if } R = T \\ 0 & \text{if } R \subset T \end{cases} \tag{16.55}$$

then Eq. (16.54) becomes

$$M_T = \sum_{\substack{S\\ S\subseteq T}} \mu(S,T)f(S) \tag{16.56}$$

Thus, the problem of solving Eq. (16.52) reduces to the problem of choosing the function $\mu(S,T)$ which solves Eq. (16.55). The judicious choice is

$$\mu(S,T) = (-1)^{|T|-|S|} \tag{16.57}$$

To check that Eq. (16.57) solves Eq. (16.55), we observe that there are $\binom{|T|-|R|}{j}$ choices of S such that $R \subseteq S \subseteq T$ and $|S| = |R| + j$, so

$$\begin{aligned} \sum_{\substack{S,\\ R\subseteq S\subseteq T}} (-1)^{|T|-|S|} &= \sum_{j=0}^{|T|-|R|} \binom{|T|-|R|}{j} (-1)^{|T|-|R|-j} \\ &= (1-1)^{|T|-|R|} = 0 \qquad \text{if } |R| < |T| \end{aligned}$$

Therefore, Eq. (16.57) solves Eq. (16.55), and the solution to Eq. (16.52) is given by (16.56). However, a simpler form of the solution is obtained if we replace M_R by M_R^*, defined as $M_R^* = M_R$ if $R \neq \emptyset$, but $M_\emptyset^* = 0$. Equation (16.52) then becomes

$$\sum_{\substack{R,\\ R\subseteq S}} M_R^* = \begin{cases} q^{|S|+1-d} - 1 & \text{if } d \leq |S| \\ 0 & \text{otherwise} \end{cases}$$

According to (16.56), we then have

$$M_T^* = \sum_{\substack{S, \\ S \subseteq T \\ d \leq |S|}} (-1)^{|T|-|S|}(q^{|S|+1-d} - 1)$$

Setting $i = |S|$, this becomes

$$M_T^* = \sum_{i=d}^{|T|} (-1)^{|T|-i} \binom{|T|}{i} (q^{i+1-d} - 1)$$

Setting $j = |T| - i$ gives

$$M_T^* = \sum_{j=0}^{|T|-d} (-1)^j \binom{|T|}{j} (q^{|T|-d-j+1} - 1) \tag{16.58}$$

If $w \neq 0$,

$$A_w = \sum_{\substack{T, \\ |T|=w}} M_T^*$$

Since there are $\binom{n}{w}$ choices for T, the first form of Theorem 16.51 follows directly from Eq. (16.58). To obtain the second form, we write

$$\begin{aligned}
\frac{A_w}{\binom{n}{w}} &= \sum_{j=0}^{w-d} (-1)^j \left[\binom{w-1}{j} + \binom{w-1}{j-1} \right] (q^{w-d-j+1} - 1) \\
&= \sum_{j=0}^{w-d} (-1)^j \binom{w-1}{j} (q^{w-d-j+1} - 1) \\
&\quad + \sum_{j=0}^{w-d+1} (-1)^j \binom{w-1}{j-1} (q^{w-d+1-j} - 1) \\
&= \sum_{i=0}^{w-d} (-1)^i \binom{w-1}{i} (qq^{w-d-i} - 1) - \sum_{i=0}^{w-d} \binom{w-1}{i} (q^{w-d-i} - 1)(-1)^i \\
&= \sum_{i=0}^{w-d} (-1)^i \binom{w-1}{i} (q-1)q^{w-d-i} \qquad \text{Q.E.D.}
\end{aligned}$$

The reader may notice certain similarities between the function $\mu(S,T)$ which solves Eq. (16.55) and the function $\mu(k)$ of Theorem 3.41. Rota (1964) has shown that both are special cases of a Moebius function which may be defined on an arbitrary partially ordered set.

Table 16.1 Weight enumerators of some cyclic binary codes (divided by block length)

	Length	Information symbols	Check polynomial	Weight														
				0	2	4	6	8	10	12	14	16	18	20	22	24	26	28
BCH	7	3	$M^{(1)}$	1/7	0	1												
JMG[a]	9	6	$M^{(1)}$	1/6	1	3	3											
JMG BCH	15	6	$M^{(1)}M^{(5)}$	1/15	0	0	2	1	6/15	0								
JMG	15	6	$M^{(3)}M^{(5)}$	1/15	0	0	5/15	2	1/15	1/15								
JMG	15	8	$M^{(1)}M^{(3)}$	1/15	0	1	20/15	5	4	1/15								
JMG	15	8	$M^{(1)}M^{(7)}$	1/15	0	2	4	7	4	0								
JMG BCH	15	10	$M^{(1)}M^{(3)}M^{(5)}$	1/15	0	7	56/15	29	56/15	7/15								
FJM[b] QR	17	8	$M^{(1)}$	1/17	0	0	4	5	4	2								
JMG	21	6	$M^{(1)}$	1/21	0	0	0	1	0	2	0	0	0					
JMG	21	5	$M^{(3)}M^{(7)}$	1/21	0	0	0	0	1	1/3	1/7	0	0					
JMG	21	8	$M^{(1)}M^{(7)}$	1/21	0	0	1	1	6	2	15/7	0	0					
JMG	21	9	$M^{(7)}M^{(9)}$	1/21	0	0	0	10	0	49/3	0	1	0					
JMG	21	9	$M^{(1)}M^{(9)}$	1/21	0	1	0	7	0	49/3	0	0	0					
JMG	21	8	$M^{(3)}M^{(7)}M^{(9)}$	1/21	0	0	1	1	5	14/3	1/7	0	1/3					
FJM	21	11	$M^{(1)}M^{(3)}M^{(7)}$	1/21	0	0	8	10	48	49/3	120/7	1	0					
JMG	21	12	$M^{(1)}M^{(3)}M^{(9)}$	1/21	0	1	9	4	57	185/3	19	4	1/3					
JMG	21	12	$M^{(1)}M^{(5)}$	1/21	0	3	10	35	60	61	26	0	0					
JMG	21	14	$M^{(1)}M^{(3)}M^{(7)}M^{(9)}$	1/21	0	4	44	142	276	629/3	652/7	13	4/3					
JMG	21	15	$M^{(1)}M^{(3)}M^{(5)}$	1/21	10	10	78	308	518	1339/3	170	31	2					
FJM QR	23	11	$M^{(1)}$	1/23	0	0	0	22	0	56	0	11	0					
JMG	25	20	$M^{(1)}$	1/25	2	41	440	2,610	8,300	13,050	11,000	5,125	1,250	125	0	0	0	0
JMG	27	18	$M^{(1)}$	1/27	1	19	84	378	1,134	2,268	2,916	2,187	729	0	0	0	0	0
JMG	31	10	$M^{(1)}M^{(3)}$	1/31	0	0	0	0	0	10	0	17	0	6	0	0	0	0
JMG BCH	31	10	$M^{(1)}M^{(5)}$	1/31	0	0	0	0	0	10	0	17	0	6	0	0	0	0
JMG	31	10	$M^{(1)}M^{(15)}$	1/31	0	0	0	0	1	5	10	7	5	5	0	0	0	0
JMG BCH	31	15	$M^{(1)}M^{(3)}M^{(5)}$	1/31	0	0	0	15	0	280	0	589	0	168	0	5	0	0
JMG QR	31	15	$M^{(1)}M^{(5)}M^{(7)}$	1/31	0	0	0	15	0	280	0	589	0	168	0	5	0	0
JMG	31	15	$M^{(1)}M^{(3)}M^{(15)}$	1/31	0	0	1	10	36	140	270	303	180	92	25	0	0	0
JMG	31	15	$M^{(1)}M^{(3)}M^{(7)}$	1/31	0	0	0	10	41	140	260	303	190	92	20	0	1	0
JMG BCH	31	20	$M^{(1)}M^{(3)}M^{(5)}M^{(15)}$	1/31	0	0	26	255	1,342	4,600	8,100	9,741	6,300	2,760	610	85	6	0
JMG	31	20	$M^{(1)}M^{(3)}M^{(5)}M^{(7)}$	1/31	0	0	26	255	1,342	4,600	8,100	9,741	6,300	2,760	610	85	6	0
FJM	31	20	$M^{(1)}M^{(3)}M^{(7)}M^{(15)}$	1/31	0	0	27	245	1,387	4,480	8,310	9,489	6,510	2,640	655	75	7	0

					0	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36
	BCH	31	25	$M^{(1)}M^{(3)}M^{(5)}M^{(7)}M^{(11)}$	1/31	0	35	728	7,995	44,616	142,415	267,120	303,165	207,760	85,449	20,280	2,665	168	5				
JMG		33	10	$M^{(1)}$	1/33	0	0	0	0	0	5	5	5	10	5	1	0	0	0				
JMG		33	12	$M^{(1)}M^{(11)}$	1/33	0	0	0	0	5	12	15	35	35	16	6 7/11							
JMG		33	12	$M^{(3)}M^{(11)}$	1/33	0	0	5/3	0	0	11	15	42	44	5	1/11	5						
JMG		35	7	$M^{(5)}M^{(7)}$	1/35	0	0	0	0	0	0	3/7	1	2	1/5	0	0	0	1/7				
JMG		35	10	$M^{(5)}M^{(7)}M^{(15)}$	1/35	0	0	0	0	0/5	0	3/7	11	10	3	3	0	0	1/7	1/5			
JMG		35	12	$M^{(1)}$	1/35	0	0	0	2	0	12	0	43	0	60	0	0	0	0	0			
JMG		35	15	$M^{(1)}M^{(5)}$	1/35	0	0	0	2	0	93	0	439	0	1786/5	0	44	0	1	0			
JMG		35	15	$M^{(1)}M^{(15)}$	1/35	0	1	0	14	0	98	0	343	0	2401/5	0	0	0	0	0			
JMG		39	12	$M^{(1)}$	1/39	0	0	0	0	0	4	0	27	0	52	0	22	0	0	0			
JMG		39	14	$M^{(1)}M^{(13)}$	1/39	0	0	0	0	3	4	36	27	130	52	132	22	14	0	0			
JMG		39	14	$M^{(3)}M^{(13)}$	1/39	0	0	2	0	0	55/3	1	22	143	132	55	39	1/13	0	2 2/3			
JMG	QR	41	20	$M^{(1)}$	1/41	0	0	0	0	32	185	810	2,379	4,760	6,226	5,660	3,570	1,464	405	74	10		
JMG		43	14	$M^{(1)}$	1/43	0	0	0	0	0	0	8	28	49	77	93	77	35	7	7	0		
FJM		47	23	$M^{(1)}$	1/47	0	0	0	0	0	276	0	7,590	0	49,588	0	81,720	0	35,420	0	3,795	0	92
JMG		51	16	$M^{(1)}M^{(5)}$	1/51	0	0	0	0	0	0	0	30	0	160	0	504	0	480	0	111	0	0

					0	12	16	20	24	28	32	36	40	44	48	52	56	60
JMG		55	20	$M^{(1)}$	1/55	0	0	1,048	5,296	8,080	3,947	600	24					
JMG		63	12	$M^{(1)}M^{(5)}$	1/63	00	0	0	10	0	49	0	6					
JMG		63	12	$M^{(1)}M^{(3)}$	1/63	0	0	0	19/3	24	17	5 6/3	2					
VP[c]	QR	71	35	$M^{(1)}$	1/71	35	2,345	186,186	4,340,910	37,861,505	129,893,225	181,404,764	103,914,580	24,093,685	2,170,455	716,100	670	7
JMG		89	11	$M^{(1)}$	1/89	0	0	0	0	0	0	0	11	0	11	0	1	
JMG		91	12	$M^{(1)}$	1/91	0	0	0	0	0	0	4	6	12	15	8		

[a] Due to Goethals.
[b] Due to MacWilliams.
[c] Due to Pless.

Table 16.2 Values of $E_{r,i}$

$$\left(\frac{d^r}{dx^r}\cosh^n x\right)_{x=0} = \sum_i E_{r,i} n_{(i)} = F_r^{(0)}(n)$$

	i						
r	0	1	2	3	4	5	6
0	1	0	0	0	0	0	0
2	0	1	0	0	0	0	0
4	0	1	3	0	0	0	0
6	0	1	15	15	0	0	0
8	0	1	63	210	105	0	0
10	0	1	255	2,205	3,150	945	0
12	0	1	1,023	21,120	65,835	51,975	10,395

Table 16.3 Reduced and triangularized power-moment identities for subcodes of the second-order RM code†

$$\tfrac{1}{2}A_{N/2} + \Sigma A_{f(i)} = N^v - 1 \tag{T1}$$

$$\Sigma 2^i A_{f(i)} = N(N^{v-1} - 1) + 2!N^{v-1}B_2 \tag{T2}$$

If $B_2 = 0$, then

$$\Sigma 2^i(2^i - 2)A_{f(i)} = N(N - 2)(N^{v-2} - 1) + 4!N^{v-2}B_4 \tag{T3}$$

If $B_2 = B_4 = 0$, then

$$\Sigma 2^i(2^i - 2)(2^i - 8)A_{f(i)} = N(N - 2)(N - 8)(N^{v-3} - 1) + 6!N^{v-3}B_6 \tag{T4}$$

If $B_2 = B_4 = B_6 = 0$, then

$$\Sigma 2^i(2^i - 2)(2^i - 8)(2^i - 32)A_{f(i)} = N(N - 2)[-N^{v-4}(29N^2 - 110N - 136) - (N^2 - 40N + 256)] + 8!N^{v-4}B_8 \tag{T5}$$

If $B_2 = B_4 = B_6 = 0$, then

$$\Sigma 2^i(2^i - 2)(2^i - 8)(2^i - 32)(2^i - 128)A_{f(i)} = N(N - 2)[31N^{v-5}(121N^3 - 378N^2 - 504N - 128) - N^3 + 168N^2 - 5{,}376N + 32{,}768] + N^{v-5}\left[10!\frac{N}{2} - \frac{8}{3} - 170 \times 8!N\right]B_8 + 10!N^{v-5}B_{10} \tag{T6}$$

If $B_2 = 0$, then

$$\Sigma 2^i(2^i - 1)A_{f(i)} = N(N - 1)(2N^{v-2} - 1) + 4!N^{v-2}B_4 \tag{T7}$$

If $B_2 = B_4 = 0$, then

$$\Sigma 2^i(2^i - 1)(2^i - 4)A_{f(i)} = N(N - 1)(N - 4)(4N^{v-3} - 1) + 6!N^{v-3}B_6 \tag{T8}$$

If $B_2 = B_4 = B_6 = 0$, then

$$\Sigma 2^i(2^i - 1)(2^i - 4)(2^i - 16)A_{f(i)} = -N(N - 1)(N - 4)[2N^{v-4}(11N + 34) + (N - 16)] + 8!N^{v-4}B_8 \tag{T9}$$

† $f(i) = 2^{m-1} + 2^{(m+i)/2-1}$; $N = 2^m$ = block length; k = number of information bits; $v = (k - 1)/m$. All sums are taken over nonnegative i which are congruent to m mod 2.

Table 16.4 Weight enumerators for low-rate BCH codes of block length $N = 2^m$, m odd

$f(i) = 2^{m-1} - 2^{(m+i)/2-1}$

Roots of check polynomial of punctured code (one entry/cycle)	v	*Initial observation*	$\cdots A_{f(11)}$	$A_{f(9)}$	$A_{f(7)}$
†01111111111111111111111	1	(First order RM)	$\cdots 0$	0	0
‡01111111111111111111111 01111111111011111111111	2	$B_2 = 0$	$\cdots 0$	0	0
01111111111111111111111 ‡01111111111011111111111 01111111110111111111111	3	$B_4 = 0$	$\cdots 0$	0	0
01111111111111111111111 ‡01111111111011111111111 01111111110111111111111 01111111101111111111111	4	$B_6 = 0$	$\cdots 0$	0	0
01111111111111111111111 01111111111011111111111 01111111110111111111111 01111111101111111111111 01111111011111111111111	5	$B_8 = \dfrac{N(N-1)(N-2)(N-4)}{8 \times 7 \times 6 \times 4}$	$\cdots 0$	0	$\dfrac{N(N-1)(N-2)(N-8)(N-32)}{128 \times 126 \times 120 \times 96}$
01111111111111111111111 01111111111011111111111 01111111110111111111111 01111111101111111111111 01111111011111111111111 01111110111111111111111	6	$B_{10} = 0$	$\cdots 0$	$\dfrac{N(N-1)(N-2)(N-8)(N-32)(N-128)}{512 \times 510 \times 504 \times 480 \times 384}$	§
	7	$? \leq B_{12} \leq ?$	(?)		

v	$A_{f(5)}$	$A_{f(3)}$	$A_{f(1)}$	$\frac{1}{2}A_{N/2}$	*Concluding observation*
1	0	0	0	$N-1$	$B_2 = 0$
2		0	$\frac{N(N-1)}{2}$	$\frac{N(N+2)}{4}$	$B_4 = 0$
3		$\frac{N(N-1)(N-2)}{8 \times 6}$	$\frac{N(N-1)(5N+8)}{2 \times 6}$	$\frac{(N-1)(9N^2+6N+16)}{16}$	$B_6 = 0$
4	$\frac{N(N-1)(N-2)(N-8)}{32 \times 30 \times 24}$	$\frac{N(N-1)(N-2)(23N-32)}{9 \times 128}$	$\frac{N(N-1)(151N^2+200N+256)}{45 \times 8}$	§	$B_8 = \frac{30N(N-1)(N-2)(N-4)}{8!}$
5	§	§	§	§	$B_{10} = 0$
6	§	§	§	§	$B_{12} = ?$
7					

† Known to Reed and Muller.
‡ Found by Kasami.
§ These entries are obtainable from Table 16.3 by straightforward calculation.

Table 16.5 Kasami's weight enumerators for certain other subcodes of the second-

$f(i) = 2^{m-1} - 2^{(m+i)/2-1}$

Roots of check polynomial of punctured code (one entry/cycle)		v	*Remarks*	$\cdots A_{f(3)}$	$A_{f(4)}$	$A_{f(3)}$
Duals of extended high-rate BCH codes, m odd	011111111 001111111	2		$\cdots 0$		0
	011111111 001111111 010111111	3		$\cdots 0$		$\frac{N(N-1)(N-2)}{8 \times 6}$
Low-rate extended BCH code, m even	0111111111 0111101111	1½			$\cdots 0$	
	0111111111 0111101111 0111011111	2½			$\cdots 0$	
	.....	3½	Unknown			
Duals of extended high-rate BCH codes, m even	01111111 00111111	2	$B_6 = \frac{N(N-1)(N-4)^2}{6!}$		$\cdots 0$	
	01111111 00111111 01010111	3	$B_8 = \frac{N(N-1)(N-4)(N^2+6N+68)}{8!}$ (Proof via Carlitz-Uchiyama (1957))		$\frac{3N(N-1)(N-4)}{16 \times 15 \times 12}$	
Arbitrary subcode of rate $(2m+1)/N$, m odd or even	$(2^m-1)-1$ (2^m-1) $-(2^e+1)$	2	Kasami's proof not included in this book gcd = gcd (m,e)		If gcd odd, then $A_{f(\text{gcd})} = \frac{N(N-1)}{2^{\text{gcd}}}$ If gcd even, then $A_{f(\text{gcd})} = \frac{N(N-1)}{2^{\text{gcd}}(\sqrt{2^{\text{gcd}}}+1)}$	

order RM code of length $N = 2^m$

$A_{f(2)}$	$A_{f(1)}$	$A_{f(0)}$	$\frac{1}{2}A_{N/2}$
	$\frac{N(N-1)}{2}$		$\frac{(N-1)(N+2)}{2}$
	$\frac{N(N-1)(5N+8)}{2\times 6}$		$\frac{(N-1)(9N^2+6N+16)}{8}$
0		$N(\sqrt{N}-1)$	$N-1$
$\frac{N(N-1)(2\sqrt{N}-1)}{4\times 3}$		$\frac{N(\sqrt{N}-1)(N+2\sqrt{N}+4)}{3}$	$\frac{(\sqrt{N}-1)(2N^2+N^{3/2}-N+4\sqrt{N}+4)}{4}$
$\frac{N(N-1)}{4\times 3}$		$\frac{2N(N-1)}{3}$	$\frac{(N-1)(N+4)}{4}$
$\frac{7N^2(N-1)}{4\times 4\times 3}$		$\frac{2N(N-1)(3N+8)}{15}$	$\frac{(N-1)(29N^2-4N+68)}{64}$

$(N-1)(N-N/2^{\text{gcd}}+1)$

$(N-1)\left(\frac{\sqrt{2^{\text{gcd}}}-1}{2^{\text{gcd}}}N+1\right)\frac{\sqrt{2^{\text{gcd}}}N(N-1)}{(\sqrt{2^{\text{gcd}}}+1)}$

Table 16.6 Coset enumeration for double-error-correcting binary BCH code of length $n = 2^m - 1$, m even

Category	*Number of cosets in category*	*Multiplicity of minimum weight elements in coset*	*Weight of coset leader*
$S_1 = 0, \Delta_1 = 0$	1	1	0
$S_1 = 0, \Delta_1 =$ nonzero cube	$\frac{n}{3}$	$\frac{n - 1 \pm 2\sqrt{n+1}}{6}$	3
$S_1 = 0, \Delta_1 =$ noncube	$\frac{2n}{3}$	$\frac{n - 1 \mp \sqrt{n+1}}{6}$	3
$S_1 \neq 0, \Delta_1 = 0$	n	1	1
$S_1 \neq 0, \Delta_1 =$ noncube, $\mathrm{Tr}\left(\frac{\Delta_1}{S_1^3}\right) = 0$	$\frac{n(n - 1 \mp \sqrt{n+1})}{3}$	1	2
$S_1 \neq 0, \Delta_1 =$ noncube, $\mathrm{Tr}\left(\frac{\Delta_1}{S_1^3}\right) = 1$	$\frac{n(n - 1 \pm \sqrt{n+1})}{3}$	$\frac{n - 1 \mp \sqrt{n+1}}{6}$	3
$S_1 \neq 0, \Delta_1 =$ nonzero cube, $\mathrm{Tr}\left(\frac{\Delta_1}{S_1^3}\right) = 0$	$\frac{n(n - 1 \pm 2\sqrt{n+1})}{6}$	1	2
$S_1 \neq 0, \Delta_1 =$ nonzero cube, $\mathrm{Tr}\left(\frac{\Delta_1}{S_1^3}\right) = 1$	$\frac{n(n + 1 \mp 2\sqrt{n+1})}{6}$	$\frac{n - 1 \pm 2\sqrt{n+1}}{6}$	3

Appendix A

Logs and antilogs in $GF(2^5)$, Base α, a root of $x^5 + x^2 + 1$

		$1\alpha\alpha^2\alpha^3\alpha^4$	$\alpha^4\alpha^3\alpha^2\alpha 1$	$1\alpha\alpha^2\alpha^3\alpha^4$		$\alpha^4\alpha^3\alpha^2\alpha 1$	
-31	0	10000	00001	00000	∞	00000	∞
-30	1	01000	00010	00001	4	00001	0
-29	2	00100	00100	00010	3	00010	1
-28	3	00010	01000	00011	21	00011	18
-27	4	00001	10000	00100	2	00100	2
-26	5	10100	00101	00101	7	00101	5
-25	6	01010	01010	00110	20	00110	19
-24	7	00101	10100	00111	13	00111	11
-23	8	10110	01101	01000	1	01000	3
-22	9	01011	11010	01001	30	01001	29
-21	10	10001	10001	01010	6	01010	6
-20	11	11100	00111	01011	9	01011	27
-19	12	01110	01110	01100	19	01100	20
-18	13	00111	11100	01101	28	01101	8
-17	14	10111	11101	01110	12	01110	12
-16	15	11111	11111	01111	24	01111	23
-15	16	11011	11011	10000	0	10000	4
-14	17	11001	10011	10001	10	10001	10
-13	18	11000	00011	10010	29	10010	30
-12	19	01100	00110	10011	25	10011	17
-11	20	00110	01100	10100	5	10100	7
-10	21	00011	11000	10101	22	10101	22
-9	22	10101	10101	10110	8	10110	28
-8	23	11110	01111	10111	14	10111	26
-7	24	01111	11110	11000	18	11000	21
-6	25	10011	11001	11001	17	11001	25
-5	26	11101	10111	11010	27	11010	9
-4	27	11010	01011	11011	16	11011	16
-3	28	01101	10110	11100	11	11100	13
-2	29	10010	01001	11101	26	11101	14
-1	30	01001	10010	11110	23	11110	24
0	31	10000	00001	11111	15	11111	15

Appendix B

$GF(5^2)$, in terms of α, a root of $x^2 + x + 2$

	Conjugate	*Minimal polynomial*	*Order*
$\alpha^0 = \alpha^{-24} = 0\alpha + 1$	α^0	$x - 1$	1
$\alpha^1 = \alpha^{-23} = 1\alpha + 0$	α^5	$x^2 + x + 2$	24
$\alpha^2 = \alpha^{-22} = -1\alpha - 2$	α^{10}	$x^2 - 2x - 1$	12
$\alpha^3 = \alpha^{-21} = -1\alpha + 2$	α^{15}	$x^2 - 2$	8
$\alpha^4 = \alpha^{-20} = -2\alpha + 2$	α^{20}	$x^2 - x + 1$	6
$\alpha^5 = \alpha^{-19} = -1\alpha - 1$	α^1	$x^2 + x + 2$	24
$\alpha^6 = \alpha^{-18} = 0\alpha + 2$	α^6	$x - 2$	4
$\alpha^7 = \alpha^{-17} = 2\alpha + 0$	α^{11}	$x^2 + 2x - 2$	24
$\alpha^8 = \alpha^{-16} = -2\alpha + 1$	α^{16}	$x^2 + x + 1$	3
$\alpha^9 = \alpha^{-15} = -2\alpha - 1$	α^{21}	$x^2 + 2$	8
$\alpha^{10} = \alpha^{-14} = 1\alpha - 1$	α^2	$x^2 - 2x - 1$	12
$\alpha^{11} = \alpha^{-13} = -2\alpha - 2$	α^7	$x^2 + 2x - 2$	24
$\alpha^{12} = \alpha^{-12} = 0\alpha - 1$	α^{12}	$x + 1$	2
$\alpha^{13} = \alpha^{-11} = -1\alpha + 0$	α^{17}	$x^2 - x + 2$	24
$\alpha^{14} = \alpha^{-10} = 1\alpha + 2$	α^{22}	$x^2 + 2x - 1$	12
$\alpha^{15} = \alpha^{-9} = 1\alpha - 2$	α^3	$x^2 - 2$	8
$\alpha^{16} = \alpha^{-8} = 2\alpha - 2$	α^8	$x^2 + x + 1$	3
$\alpha^{17} = \alpha^{-7} = 1\alpha + 1$	α^{13}	$x^2 - x + 2$	24
$\alpha^{18} = \alpha^{-6} = 0\alpha - 2$	α^{18}	$x + 2$	4
$\alpha^{19} = \alpha^{-5} = -2\alpha + 0$	α^{23}	$x^2 - 2x - 2$	24
$\alpha^{20} = \alpha^{-4} = 2\alpha - 1$	α^4	$x^2 - x + 1$	6
$\alpha^{21} = \alpha^{-3} = 2\alpha + 1$	α^9	$x^2 + 2$	8
$\alpha^{22} = \alpha^{-2} = -1\alpha + 1$	α^{14}	$x^2 + 2x - 1$	12
$\alpha^{23} = \alpha^{-1} = 2\alpha + 2$	α^{19}	$x^2 - 2x - 2$	24

Factorable quadratics over GF(5):

$$x^2 + 1 = (x + 2)(x - 2)$$
$$x^2 - 1 = (x + 1)(x - 1)$$
$$x^2 \pm x - 1 = (x \mp 2)^2$$
$$x^2 \pm 2x + 1 = (x \pm 1)^2$$
$$x^2 \pm 2x + 2 = (x \mp 1)(x \mp 2)$$
$$x^2 \pm x - 2 = (x \mp 1)(x \pm 2)$$

REFERENCES

Abramson, N. M., 1959 A Class of Systematic Codes for Non-independent Errors, *IEEE Trans. Inform. Theory,*† **IT5**:150–157; *Math. Rev.*, **23B**:*2105.*‡

———, 1968 Cascade Decoding of Cyclic Product Codes, *IEEE Trans. Inform. Theory,* **IT14**: in press.

Alanen, J. D., and D. E. Knuth, 1964 Tables of Finite Fields, *Sankhya (Calcutta)*, **26**:305–328; *Math. Rev.*, **32**:*4122.*

Albert, A., 1956 "Fundamental Concepts of Higher Algebra," The University of Chicago Press, Chicago; *Math. Rev.*, **20**:*5190.*

Alter, R., 1968 On the Non-existence of Close-packed Double Hamming Error-correcting Codes on $q = 7$ Symbols, J. Computer & System Sciences, 2: 169–176; *Math. Rev.*, 39:*1227.*

Andryanov, V. I., and V. N. Saskovets, 1966 Decycling Codes, *Akad. Nauk Ukr. SSR Kibernetika*, pt. 1.

Artin, E., 1942 "Galois Theory," North State Press, South Bend, Ind.; *Math. Rev.*, **4**:66.

Ash, R. B., 1965 "Information Theory," John Wiley & Sons, Inc., New York; *Math. Rev.*, 37:*5049.*

Assmus, E. F., Jr., and H. F. Mattson, 1966 Perfect Codes and the Mathieu Groups, *Arch. Math.*, **17**:121–135; *Math. Rev.*, 34:*4050.*

———, ———, and R. Turyn, 1965 Cyclic Codes, *Air Force Cambridge Res. Lab. Sum. Rept.* 4.

———, ———, and ———, 1966 Cyclic Codes, *Air Force Cambridge Res. Lab. Final Rept.*

Bartee, T. C., and D. I. Schneider, 1963 Computation with Finite Fields, *Inform. Control,* **6**:79–98; *Math. Rev.*, **28**:*5854.*

Berlekamp, E. R., 1963 A Class of Convolution Codes, *Inform. Control,* **6**:1–13, *Math. Rev.*, **28**:*2935.*

———, 1964*a* "Block Coding with Noiseless Feedback," Ph.D. thesis, Department of Electrical Engineering, Massachusetts Institute of Technology, Cambridge, Mass.

———, 1964*b* Note on Recurrent Codes, *IEEE Trans. Inform. Theory,* **IT10**:257–258.

———, 1966*a* Distribution of Cyclic Matrices in a Finite Field, *Duke Math. J.*, **33**:45–48; *Math. Rev.*, **32**:*2405.*

———, 1966*b* Practical BCH Decoders, unpublished preliminary version of chaps. 7 and 10 of "Algebraic Coding Theory."

———, 1967*a* Factoring Polynomials over Finite Fields, *Bell System Tech. J.*, **46**:1853–1859; *Math. Rev.*, **36**:*2314.*

———, 1967*b* The Enumeration of Information Symbols in BCH Codes, *Bell System Tech. J.*, **46**:1861–1880; *Math. Rev.*, 36:*2427.*

———, 1967*c* Decoding Algorithms for Srivastava's Codes, unpublished notes.

† Prior to 1963, the journal hereinafter cited as *IEEE Trans. Inform. Theory* was called *IRE Trans. Inform. Theory.* Between 1953 and 1954 several issues were published without volume numbers: the boldface number in the citation thus refers to an issue number rather than a volume number. Most libraries have bound all 1953–1954 issues into one volume.

‡ Citations to *Math. Rev.* mean that papers (and books) are abstracted there. For vols. 1 to 19, references are to pages; subsequent volumes use review numbers (set in italics).

———, 1968*a* Negacyclic Codes for the Lee Metric, chap. 17 in R. C. Bose and T. A. Dowling (eds.), "Proceedings of the Conference on Combinatorial Mathematics and Its Applications (April 10–14, 1967)," The University of North Carolina Press, Chapel Hill, N.C.†; *Math. Rev.*, **40**:*3970.*

———, 1968*b* Block Coding for the Binary Symmetric Channel with Noiseless, Delayless Feedback, "Proceedings of the Symposium on Error-correcting Codes at the University of Wisconsin (May 6–8, 1968)," pp. 61–88. John Wiley & Sons, Inc., New York; *Math. Rev.*, **38**:*3072.*

———, 1968*c* A Construction for Partitions which Avoid Long Arithmetic Progressions, *Can. Math. Bull.*, **11**, No. 3, pp. 409–414; *Math. Rev.*, **38**:*1066.*

———, H. Rumsey, and G. Solomon, 1967 On the Solution of Algebraic Equations over Finite Fields, *Inform. Control*, **10**:553–564; *Math. Rev.*, **37**:*6266.*

Birkhoff, G., and S. MacLane, 1965 "A Survey of Modern Algebra," 3d ed., The Macmillan Company, New York; *Math. Rev.*, **31**:*2250.*

Bose, R. C., and D. K. Ray-Chaudhuri, 1960 On a Class of Error Correcting Binary Group Codes, *Inform. Control*, **3**:68–79, 279–290; *Math. Rev.*, **22**:*3619.*

Brillhart, J., and J. L. Selfridge, 1967 Some Factorizations of $2^n \pm 1$ and Related Results, *Math. Computation*, **21**:87–96; *Math. Rev.*, **37**:*131.*

——— and N. Zierler, 1969 On Irreducible Trinomials Modulo 2, *Inform. Control*, **14**:*566–569*; *Math. Rev.*, **39**:*5521.*

Burton, H. O., and E. J. Weldon, Jr., 1965 Cyclic Product Codes, *IEEE Trans. Inform. Theory*, **IT11**:433–439; *Math. Rev.*, **32**:*7340.*

Bussgang, J. J., 1965 Some Properties of Binary Convolutional Code Generators, *IEEE Trans. Inform. Theory*, **IT11**:90–100; *Math. Rev.*, **32**:*3953.*

Calabi, L., and E. Myrvaagnes, 1964 On the Minimal Weight of Binary Group Codes, *IEEE Trans. Inform. Theory*, **IT10**:385–387.

Camion, P., 1966 Codes correcteurs d'erreurs, *Rev. CETHEDEC*, **3** (num. spéc.).

———, 1968 A Proof of Some Properties of Reed-Muller Codes by Means of the Normal Basis Theorem, chap. 22 in R. C. Bose and T. A. Dowling (eds.), "Proceedings of the Conference on Combinatorial Mathematics and Its Applications (April 10–14, 1967)," The University of North Carolina Press, Chapel Hill, N.C. *Math. Rev.* **35**: *5251.*

Carlitz, L., 1953 A Theorem of Stickelberger, *Math. Scand.*, **1**:82–84; *Math. Rev.*, **15**:13.

——— and J. H. Hodges, 1956*a* Distribution of Bordered Symmetric, Skew and Hermitian Matrices in a Finite Field, *J. Reine Angew. Math.*, **195**:192–201; *Math. Rev.*, **17**:828.

——— and J. H. Hodges, 1956*b* Distribution of Matrices in a Finite Field, *Pacific J. Math.*, **6**:225–230; *Math. Rev.*, **18**:554.

Cerveira, A. G., 1968 On a Class of Wide Sense Binary BCH Codes Whose Minimum Distances Exceed the BCH Bound, *IEEE Trans. Inform. Theory*, **IT14**: *784–85.*

Chernoff, H., 1952 A Measure of Asymptotic Efficiency for Tests of a Hypothesis Based on the Sum of Observations, *Ann. Math. Statist.*, **23**:493–507; *Math. Rev.*, **15**:241.

Chien, R. T., 1964 Cyclic Decoding Procedures for Bose-Chaudhuri-Hocquenghem Codes, *IEEE Trans. Inform. Theory*, **IT10**:357–363.

Chow, D. K., 1967 A Geometric Approach to Coding Theory with Application to Information Retrieval, *Coordinated Sci. Lab., Rept.* R-368, University of Illinois, Urbana.

Cocke, J. 1959 Lossless symbol coding with nonprimes, *IEEE Trans. Inform. Theory*, **5**:33–34; *Math. Rev.*, **22**:*13355.*

† These proceedings were actually published in 1969.

Cohen, E. L., 1964 A Note on Perfect Double Error-correcting Codes on q Symbols, *Inform. Control*, **7**:381–384; *Math. Rev.*, **29**:*5656*.

Cunningham, A. J. C., and H. J. Woodall, 1925 "Factorizations of $(y^n \mp 1)$," F. Hodgson, London.

Dalen, K., 1955 On a Theorem of Stickelberger, *Math. Scand.*, **3**:124–126; *Math. Rev.*, **17**:130.

Davenport, W. B., Jr., and W. L. Root, 1958 "Introduction to the Theory of Random Signals and Noise," McGraw-Hill Book Company, New York; *Math. Rev.*, **19**:*1090*.

Daykin, D. E., 1960 Distribution of Bordered Persymmetric Matrices in a Finite Field, *J. Reine Angew. Math.*, **203**:47–54; *Math. Rev.*, **22**:*3734*.

———,1965 Generation of Irreducible Polynomials over a Finite Field, *Am. Math. Monthly*, **72**:646–648.

Delsarte, P., J. M. Goethals, and E. J. MacWilliams, 1968 Unpublished paper.

Dickson, L. E., 1911 "Linear Groups with an Exposition of the Galois Field Theory," republished in paperback by Dover Publications, Inc., New York, 1958.

Ekonomi, E. A., 1965 "Phase Independent Burst Correcting Codes," M.S. plan II project report, University of California, Berkeley, Calif., June, 1965.

Elias, P., 1954 Error-free Coding, *IEEE Trans. Inform. Theory*, **IT4**:29–37; *Math. Rev.*, **19**:721.

———, 1955 Coding for Noisy Channels, *IRE Conv. Record*, pt. 4, pp. 37–46.

———, 1957 List Decoding for Noisy Channels, *M.I.T. Res. Lab. Electron. Rept.* 335; *Math. Rev.*, **20**:*5702*.

Elspas, B., 1967 Talk presented at *IEEE International Symposium on Information Theory* at San Remo, Italy, September 1967.

Epstein, M. A., 1958*a* Algebraic Decoding for a Binary Erasure Channel, *M.I.T. Res. Lab. Electron. Rept.* 340.

———, 1958*b* Construction of Convolutional Codes by Suboptimization, *M.I.T. Res. Lab. Electron. Rept.* 341.

Evans, T. A., and H. B. Mann, 1951 On Simple Difference Sets, *Sankhya (Calcutta)*, **11**:357–364; *Math. Rev.*, **13**:899.

Falconer, D. D., 1967 "A Hybrid Sequential and Algebraic Decoding Scheme," Ph.D. thesis, Department of Electrical Engineering, Massachusetts Institute of Technology, Cambridge, Mass.

Fano, R. M., 1961 "Transmission of Information," The M.I.T. Press, Cambridge, Mass.; *Math. Rev.*, **24B**:*442*.

———, 1963 A Heuristic Discussion of Probabilistic Decoding, *IEEE Trans. Inform. Theory*, **IT9**:64–74; *Math. Rev.*, **29**:*3283*.

Feinstein, A., 1954 A New Basic Theorem of Information Theory, *IEEE Trans. Inform. Theory*, **IT4**:2–22; *Math. Rev.*, **19**:516.

———, 1958 "Foundations of Information Theory," McGraw-Hill Book Company, New York; *Math. Rev.*, **20**:*1594*.

Feller, W., 1943 Generalization of a Probability Limit Theorem of Cramer, *Trans. Am. Math. Soc.*, **54**:361–372; *Math. Rev.*, **5**:125.

Fire, P., 1959 A Class of Multiple-error-correcting Codes for Non-independent Errors, *Sylvania Reconnaissance Systems Lab. Rept.* RSL-E-2.

Fisher, R. A., 1942 The Theory of Confounding in Factorial Experiments in Relation to the Theory of Groups, *Ann. Eugenics*, **11**:341–353; *Math. Rev.*, **4**:127.

Forney, G. D., Jr., 1965 On Decoding BCH Codes, *IEEE Trans. Inform. Theory*, **IT11**:549–557; *Math. Rev.*, **32**:*7341*.

———, 1966*a* Generalized Minimum Distance Decoding, *IEEE Trans. Inform. Theory*, **IT12**:125–131; *Math. Rev.*, **34**:*7272*.

———, 1966*b* "Concatenated Codes," Monograph 37, The M.I.T. Press, Cambridge, Mass.

Gallager, R. G., 1963 "Low Density Parity Check Codes," The M.I.T. Press, Cambridge, Mass. (abbreviated version published in *IEEE Trans. Inform. Theory*, **IT8**:21–28); *Math. Rev.*, **24B**:*2048*.

———, 1965 A Simple Derivation of the Coding Theorem and Some Applications, *IEEE Trans. Inform. Theory.*, **IT11**:3–18; *Math. Rev.*, **32**:*3951*.

———, 1966 Binary Codes for Burst Error Corrections, *IEEE Trans. Inform. Theory*, **IT12**:273.

———, 1968 "Information Theory & Reliable Communication," John Wiley & Sons, Inc., New York.

Gilbert, E. N., 1952 A Comparison of Signalling Alphabets, *Bell System Tech. J.*, **31**:504–522.

Gillies, D. B., 1964 Three New Mersenne Primes and a Statistical Theory, *Math. Computation*, **18**:93–97; *Math. Rev.*, **28**:*2990*.

Goethals, J. M., 1965 Algebraic Structure and Weight Distribution of Binary Cyclic Codes, *MBLE Res. Lab. (Belgium) Rept.* R37.

———, 1966 Analysis of Weight Distribution in Binary Cyclic Codes, *IEEE Trans. Inform. Theory*, **IT12**:401–402.

———, 1967 Factorization of Cyclic Codes, *IEEE Trans. Inform. Theory*, **IT13**:242–246. *Math. Rev.* **35**, *5252*.

——— and P. Delsarte, 1968 On a Class of Majority Logic Decodable Cyclic Codes, *IEEE Trans. Inform. Theory*, **IT14**:182–188; *Math. Rev.*, 40:*5321*.

——— and J. J. Seidel, 1967 Orthogonal Matrices with Zero Diagonal, *Can. J. Math.* **19**:1001–1010; *Math. Rev.*, **36**:*5013*.

Golay, M. J. E., 1949 Notes on Digital Coding, *Proc. IRE*, **37**:657.

———, 1954 Binary Coding, *IEEE Trans. Inform. Theory*, **IT4**:23–28; *Math. Rev.*, **19**:622.

———, 1958 Notes on the Penny-weighing Problem, Lossless Symbol Coding with Nonprimes, etc., *IEEE Trans. Inform. Theory*, **IT4**:103–109; *Math. Rev.*, **22**:*13354*.

Golomb, S. W., 1967 "Shift Register Sequences," Holden-Day, Inc., San Francisco, Calif.; *Math. Rev.*, **39**:*3906*.

———, 1968 On the Cross-correlation of Shift Register Sequences, *IEEE Trans. Inform. Theory*, **IT14**: 835.

——— and E. C. Posner, 1964 Rook Domains, Latin Squares, Affine Planes, and Error-distributing Codes, *IEEE Trans. Inform. Theory*, **IT10**:196–208; *Math. Rev.*, **29**:*5657*.

——— and L. R. Welch, 1970 Perfect Codes in the Lee Metric and the Packing of Polyominoes, *J. Soc. Ind. Appl. Math.*, **18**:302–317; *Math. Rev.*, **41**:*1422*.

Gorenstein, D. C., W. W. Peterson, and N. Zierler, 1960 Two-error Correcting Bose-Chaudhuri Codes Are Quasi-perfect, *Inform. Control*, **3**:291–294; *Math. Rev.*, **22**:*9350*.

——— and N. Zierler, 1961 A Class of Error-correcting Codes in p^m Symbols, *J. Soc. Indus. Appl. Math.*, **9**:207–214; *Math. Rev.*, **24B**:*450*.

Graham, R. L., and F. J. MacWilliams, 1966 On the Number of Information Symbols in Difference-set Cyclic Codes, *Bell System Tech. J.*, **45**:1057–1070; *Math. Rev.*, **34**:*1102*.

Green, M. W., 1966 Two Heuristic Techniques for Block-code Construction, *IEEE Trans. Inform. Theory*, **IT12**:273.

Griesmer, J. H., 1960 A Bound for Error-correcting Codes, *IBM J. Res. Develop.*, **4**:532–542; *Math. Rev.*, **23B**:*3081*.

Gross, A. J., 1963 Augmented Bose-Chaudhuri Codes Which Correct Single Bursts of Errors, *IEEE Trans. Inform. Theory*, **IT9**:121.

Hagelbarger, D. W., 1959 Recurrent Codes: Easily Mechanized Burst-correcting, Binary Codes, *Bell System Tech. J.*, **38**:969–984; *Mech. Rev.*, **21**:*7812*.

Hall, M., Jr., 1967 "Combinatorial Theory," Blaisdell Publishing Company, Waltham, Mass; *Math. Rev.*, **37**:*80*.

Hamming, R. W., 1950 Error Detecting and Error Correcting Codes, *Bell System Tech. J.*, **29**:147–160; *Math. Rev.*, **12**:35.

Hocquenghem, A., 1959 Codes correcteurs d'erreurs, *Chiffres (Paris)*, **2**:147–156; *Math. Rev.*, **22**:*652*.

Horstein, M., 1963 Sequential Transmission Using Noiseless Feedback, *IEEE Trans. Inform. Theory*, **IT9**:136–143.

Hsu, H. T., 1968 A New Class of Recurrent Codes, *IEEE Trans. Inform. Theory*, **IT14**: 592–597; *Math. Rev.*, **42**:*4315*.

Huffman, D. A., 1956 The Synthesis of Linear Sequential Coding Networks, pp. 77–95, in C. Cherry (ed.), "Information Theory," Academic Press Inc., New York.

Jacobs, I. M., and E. R. Berlekamp, 1967 A Lower Bound to the Distribution of Computation for Sequential Decoding, *IEEE Trans. Inform. Theory*, **IT13**: 167–174.

Jelinek, F., 1968 "Probabilistic Information Theory: Discrete and Memoryless Models," McGraw-Hill Book Company, New York.

Johnson, S. M., 1962 A New Upper Bound for Error-correcting Codes, *IEEE Trans. Inform. Theory*, **IT8**:203–207; *Math. Rev.*, **25**:*1067*.

———, 1963 Improved Asymptotic Bounds for Error-correcting Codes, *IEEE Trans. Inform. Theory*, **9**:198–205; *Math Rev.*, **29**:*1089*.

Julin, D., 1965 Two Improved Block Codes, *IEEE Trans. Inform. Theory*, **IT11**:459.

Kac, M., 1959 "Statistical Independence in Probability, Analysis, and Number Theory," Carus Monograph No. 12, Mathematical Association of America and John Wiley & Sons, Inc., New York; *Math Rev.*, **22**:*996*.

Karlin, M., 1968 Circulants in the Calculation and Representation of Cyclic Ideals and Group Codes, in press.

Kasami, T., 1968 Weight Distribution of BCH Codes, chap. 20 in R. C. Bose and T. A. Dowling (eds.), "Proceedings of the Conference on Combinatorial Mathematics and Its Applications (April 10–14, 1967)," The University of North Carolina Press, Chapel Hill, N.C.; *Math. Rev.*, **40**:*5325*.

———, S. Lin, and W. W. Peterson, 1966 Some Results on Weight Distributions of BCH Codes, *IEEE Trans. Inform. Theory*, **IT12**:274.

———, ———, and ———, 1967*a* Some Results on Cyclic Codes which are Invariant under the Affine Group and their Applications, *Inform. Control*, **11**:475–496; *Math. Rev.*, **37**:*5051*.

———, ———, and ———, 1967*b* Weight Distribution of BCH Codes, *J. Inst. Commun. Eng. Japan*, September, 1967.

———, ———, and ———, 1968*b* New Generalizations of the Reed-Muller Codes, Part I: Primitive Codes, *IEEE Trans. Inform. Theory*, **14**:189–198 ; *Math. Rev.*, **43**:*1742*.

———, ———, and ———, 1968*c* Polynomial Codes, *IEEE Trans. Inform. Theory*, **IT14**: 807–814; *Math. Rev.*, **40**:*2426*.

Knuth, D. E., 1968– "The Art of Computer Programming," vols. I–VII, Addison-Wesley Publishing Company, Inc., Reading, Mass.

Landsberg, G., 1893 Über eine Anzahlbestimmung und eine damit zusammenhangende Reihe, *J. Reine Angew. Math.*, **111**:87–88.

Lee, C. Y., 1958 Some Properties of Nonbinary Error-correcting Codes, *IEEE Trans. Inform. Theory*, **IT4**, 77–82; *Math. Rev.*, **22**:*13353*.

Lehmer, E., 1936 On the Magnitude of the Coefficients of the Cyclotomic Polynomial, *Bull. Am. Math. Soc.*, **42**:389–392.

Levy, J., 1964 "Capabilities of Parity-check Codes for Nonprime Alphabets," M.S. thesis, Electrical Engineering Department, Massachusetts Institute of Technology.

Lin, S., 1968 Some Codes Which Are Invariant under a Transitive Permutation and Their Connection with Balanced Incomplete Group Block Design, chap. 24 in R. C. Bose and T. A. Dowling (eds.), "Proceedings of the Conference on Combinatorial Mathematics and Its Applications (April 10–14, 1967)," The University of North Carolina Press, Chapel Hill, N.C.; *Math. Rev.*, **40**:*2430*.

Lloyd, S. P., 1957 Binary Block Coding, *Bell System Tech. J.*, **36**:517–535; *Math. Rev.*, **19**:465.

Logan, B. F., 1967 Private communication.

Lucas, M. E., 1878 Sur les congruences des nombres euleriennes et des coefficients différentiels des functions trigonométriques, suivant un-module premier, *Bull. Soc. Math. France*, **6**:49–54.

MacDonald, J. E., 1960 Design Methods for Maximum Minimum-distance Error-correcting Codes, *IBM J. Res. Develop.*, **4**:43–57; *Math. Rev.*, **22**:*1471*.

MacWilliams, F. J., 1963 A Theorem on the Distribution of Weights in a Systematic Code, *Bell System Tech. J.*, **42**:79–94; *Math. Rev.*, **26**:*7462*.

———, 1964 Permutation Decoding of Systematic Codes, *Bell System Tech. J.*, **43**:485–505.

———, 1965 The Structure and Properties of Binary Cyclic Alphabets, *Bell System Tech. J.*, **44**:303–332; *Math. Rev.*, **33**:*3843*.

———, 1968*a* Codes and Ideals in Group Algebras, chap. 18 in R. C. Bose and T. A. Dowling (eds.), "Proceedings of the Conference on Combinatorial Mathematics and Its Applications (April 10–14, 1967)," The University of North Carolina Press, Chapel Hill, N.C.; *Math. Rev.*, **40**:*7011*.

———, 1968*b* Orthogonal Circulant Matrices over Finite Fields, *J. Combinatorial Theory*, **10**:1–17; *Math. Rev.*, **39**:*230*.

——— and H. B. Mann, 1968 On the p-rank of the Design Matrix of a Difference Set, *Inform. Control*, **12**:474–488; *Math. Rev.*, **39**:*4026*.

Mandelbrot, B., 1965 Self-similar Error Clusters in Communication Systems and the Concept of Conditional Stationarity, *IEEE Trans. Commun. Tech.*, **ComT13**: 71–90.

Mann, H. B., 1962 On the Number of Information Symbols in Bose-Chaudhuri Codes, *Inform. Control*, **5**:153–162; *Math Rev.*, **26**:*4847*.

———, 1965 "Addition Theorems," John Wiley & Sons, Inc., New York; *Math. Rev.*, **31**:*5854*.

———, 1967 Recent Advances in Difference Sets, *Am. Math. Monthly*, **74**:229–235, *Math. Rev.* **36**:*6307*.

Marsh, R. W., 1957 Table of Irreducible Polynomials over GF(2) through Degree 19, National Security Agency, Washington, D.C.

Massey, J. L., 1963 "Threshold Decoding," The M.I.T. Press, Cambridge, Mass.; *Math. Rev.*, **27**:*4709*.

———, 1964 Reversible Codes, *Inform. Control*, **7**:369–380; *Math. Rev.*, **29**:*5660*.

———, 1965 Implementation of Burst-correcting Convolutional Codes, *IEEE Trans. Inform. Theory*, **IT11**:416–422.

———, 1969 Shift Register Synthesis and BCH Decoding, *IEEE Trans. Inform. Theory*, IT**15**:122–127; *Math. Rev.*, **39**:*3887*.

Mattson, H. F., and E. F. Assmus, Jr., 1964 Research Program to Extend the Theory of Weight Distribution and Related Problems for Cyclic Error-correcting Codes, *Air Force Cambridge Res. Lab. Sum. Rept.* 3.

——— and G. Solomon, 1961 A New Treatment of Bose-Chaudhuri Codes, *J. Soc. Indus. Appl. Math.*, **9**:654–669; *Math. Rev.*, **24B**:*1705*.

McCoy, N. H., 1965 "The Theory of Numbers," The Macmillan Company, New York; *Math. Rev.*, **32**:*78*.

Milne, W. E., 1949 "Numerical Calculus," Princeton University Press, Princeton, N.J.; *Math. Rev.*, **10**:483.

Muller, D. E., 1954 Application of Boolean Algebra to Switching Circuit Design and to Error Detection, *IEEE Trans. Electron. Computers*, **EC3**:6–12; *Math. Rev.*, **16**:99.

Nadler, M., 1962*a* A 32-point $n = 12$, $d = 5$ Code, *IEEE Trans. Inform. Theory*, **IT8**:58.

———, 1962*b* "Topics in Engineering Logic, Pergamon Press, New York.

Nagell, T., 1951, 1964 Introduction to Number Theory, Chelsea Publishing Company, New York; *Math. Rev.*, **13**:207.

Nordstrom, A. W., and J. P. Robinson, 1967 An Optimum Nonlinear Code, *Inform. Control*, **11**:613–616.

Ore, O., 1933 On a Special Class of Polynomials, *Trans. Am. Math. Soc.*, **35**:559–584; **36**:275.

———, 1934 Contributions to the Theory of Finite Fields, *Trans. Am. Math. Soc.*, **36**:243–274.

Peterson, W. W., 1961 "Error-correcting Codes," The M.I.T. Press, Cambridge, Mass.; *Math. Rev.*, **22**:*12003*.

———, 1968 Some New Results on Finite Fields and Their Applications to the Theory of BCH Codes, chap. 19 in R. C. Bose and T. A. Dowling (eds.), "Proceedings of the Conference on Combinatorial Mathematics and Its Applications (April 10–14, 1967)," The University of North Carolina Press, Chapel Hill, N.C.; *Math. Rev.*, **40**:*7012*.

Pinsker, M., 1965 On Decoding Complexity, *Problemy Peredachi Informatsii* (*Moscow*), **1**:113–116.

Pitcher, T., 1966 Private correspondence.

Pless, V., 1963 Power Moment Identities on Weight Distributions in Error Correcting Codes, *Inform. Control*, **6**:147–152.

———, 1965 On the Invariants of a Vector Subspace of a Vector Space over a Field of Characteristic Two, *Proc. Am. Math. Soc.*, **16**:1062–1067; *Math. Rev.*, **32**:*110*.

Plotkin, M., 1960 Binary Codes with Specified Minimum Distances, *IEEE Trans. Inform. Theory*, **IT6**:445–450 (also published as *Univ. Penn. Res. Div. Rept.* 51-20, January, 1951); *Math. Rev.*, **22**:*13361*.

Prange, E., 1957 Cyclic Error-correcting Codes in Two Symbols, *Air Force Cambridge Res. Center Tech. Note* 57-103.

———, 1958 Some Cyclic Error-correcting Codes with Simple Decoding Algorithms, *Air Force Cambridge Res. Center Tech. Note* 58-156.

———, 1959 The Use of Coset Equivalence in the Analysis and Decoding of Group Codes, *Air Force Cambridge Res. Center Tech. Rept.* 59-164.

———, 1962 The Use of Information Sets in Decoding Cyclic Codes, *IEEE Trans Inform. Theory*, **8**:S5–S9; *Math. Rev.*, **26**:*1226*.

Preparata, F. P., 1964 Systematic Construction of Optimal Linear Recurrent Codes for Burst Error Correction, *Calcolo*, **1**:147–153; *Math. Rev.*, **30**:2954.

Rao, C. R., 1947 Factorial Experiments Derivable from Combinatorial Arrangements of Arrays, *J. Roy. Statist. Soc. Suppl.*, **9**:128–139; *Math Rev.*, **9**:264.

Reddy, S. M., 1968 A Class of Convolutional Codes and a New Decoding Algorithm, Ph.D. thesis, Electrical Engineering Department, University of Iowa, Iowa City.

Reed, I. S., 1954 A Class of Multiple-error-correcting Codes and the Decoding Scheme, *IEEE Trans. Inform. Theory*, **IT4**:38–49; *Math. Rev.*, **19**:721.

——— and G. Solomon, 1960 Polynomial Codes over Certain Finite Fields, *J. Soc. Indus. Appl. Math.*, **8**:300–304; *Math. Rev.*, **23B**:*510.*

Riordan, J., 1958 "An Introduction to Combinatorial Analysis," John Wiley & Sons, Inc., New York; *Math. Rev.*, **20**:*3077.*

Robinson, J. P., 1965 An Upper Bound on the Minimum Distance of a Convolutional Code, *IEEE Trans. Inform. Theory*, **IT11**:567–571; *Math. Rev.*, **32**:*7351.*

———, 1968 Analysis of Nordstrom's Optimum Quadratic Code, "Proceedings of the Hawaii International Conference on System Sciences," pp. 157–160.

——— and A. J. Bernstein, 1967 A Class of Binary Recurrent Codes with Limited Error Propagation, *IEEE Trans. Inform. Theory*, **IT13**:106–113.

Roos, J. E., 1965 An Algebraic Study of Group and Nongroup Error-correcting Codes, *Inform. Control*, **8**:195–214; *Math. Rev.*, **32**:*9147.*

Rota, G. C., 1964 On the Foundations of Combinatorial Theory, I. Theory of Moebius Functions, *Z. Wahrscheinlichkeitstheorie*, **2**:340–368; *Math. Rev.*, **30**:*4688.*

Rudolph, L. D., 1964 Geometric Configuration and Majority Logic Decodable Codes, M.E.E. Thesis, University of Oklahoma, Norman, Okla.

———, 1967 A Class of Majority Logic Decodable Codes, *IEEE Trans. Inform. Theory*, **IT13**:305–307.

Ryser, H. J., 1963 "Combinatorial Mathematics," Carus Monograph No. 14, Mathematical Association of America and John Wiley & Sons, Inc., New York; *Math Rev.*, **27**:*51.*

Sacks, G. E., 1958 Multiple Error Correction by Means of Parity Checks, *IEEE Trans. Inform. Theory*, **IT4**:145–147; *Math. Rev.*, **22**:*13356.*

Savage, J. E., 1966*a* The Distribution of the Sequential Decoding Computation Time, *IEEE Trans. Inform. Theory*, **IT12**:143–147.

———, 1966*b* Sequential Decoding—The Computation Problem, *Bell System Tech. J.*, **45**:149–175.

Schalkwijk, J. P. M., and T. Kailath, 1966 A Coding Scheme for Additive Noise Channels with Feedback, *IEEE Trans. Inform. Theory*, **IT12**:172–189; *Math. Rev.*, **40**:*3991.*

Schönheim, J., 1968 On Linear and Nonlinear, Single-error-correcting q-nary Perfect Codes, *Inform. Control*, **12**(1):23–26; *Math. Rev.*, **38**:*5517.*

Shannon, C. E., 1948 A Mathematical Theory of Communication, *Bell System Tech. J.*, **27**:379–423, 623–656; *Math. Rev.*, **10**:133.

———, 1949 Communication in the Presence of Noise, *Proc. IRE*, **37**:10–21; *Math. Rev.*, **10**:464.

———, 1956 The Zero Error Capacity of a Noisy Channel, *IEEE Trans. Inform. Theory*, **2**:8–19; *Math. Rev.*, **19**:623.

———, 1959 Probability of Error for Optimal Codes in a Gaussian Channel, *Bell System Tech. J.*, **38**:611–656; *Math. Rev.*, **21**:*1920.*

———, R. G. Gallager, and E. R. Berlekamp, 1967 Lower Bounds to Error Probability for Coding on Discrete Memoryless Channels, *Inform. Control*, **10**:65–103, 522–552. *Math. Rev.* **35**:*1405* and *7728.*

——— and W. Weaver, 1949 "A Mathematical Theory of Communication," The University of Illinois Press, Urbana, Ill.; *Math. Rev.*, **11**:258.

Shaughnessy, E. P., 1968 Codes with Simple Automorphism Groups, *Notices Am. Math. Soc.*, **15**:480.

Singer, J., 1938 A Theorem in Finite Projective Geometry and Some Applications to Number Theory, *Trans. Am. Math. Soc.*, **43**:377–385.

Singleton, R. C., 1964 Maximum Distance Q-nary Codes, *IEEE Trans. Inform. Theory*, **IT10**:116–118; *Math. Rev.*, **29**:*2118*.

Slepian, D., 1956 A Class of Binary Signaling Alphabets, *Bell System Tech. J.*, **35**:203–234; *Math. Rev.*, **17**:1100.

———, 1960 Some Further Theory of Group Codes, *Bell System Tech. J.*, **39**:1219–1252; *Math. Rev.*, **22**:*13351*.

Smith, K. J. C., 1967 Ph.D. Thesis, Department of Statistics, University of North Carolina, Chapel Hill, N.C.

———, 1968 On the p-rank of the Incidence Matrix of Points and Hyperplanes in a Finite Projective Geometry, *J. Combinatorial Theory*, **7**:122–129; *Math. Rev.*, **40**:*4855*.

Solomon, G., 1962 A Weight Formula for Group Codes, *IEEE Trans. Inform. Theory*, **IT8**:S1–S4; *Math. Rev.*, **26**:*2340*.

——— and R. McEliece, 1966 Weights of Cyclic Codes, *J. Combinatorial Theory*, **1**:459–475; *Math. Rev.*, **38**:*1940*.

——— and J. J. Stiffler, 1965 Algebraically Punctured Cyclic Codes, *Inform. Control*, **8**:170–179; *Math. Rev.*, **30**:*5847*.

Spence, E., 1967 A New Class of Hadamard Matrices, *Glasgow J.*, **8**:59–62; *Math. Rev.* **35**:*1496*.

Srivastava, J. N., 1967 Unpublished remarks at the Combinatorial Symposium, University of North Carolina, Chapel Hill, N.C., April 10–14.

Stickelberger, L., 1897 Über eine neue Eigenschaft der diskriminanten algebraischer Zahlkörper, *Verhandl. Ersten Intern. Math.-Kong. Zürich*, 1897.

Stiffler, J. J., 1969 "Synchronization in Communication Systems," Prentice-Hall, Inc., Englewood Cliffs, N.J.

Storer, T., 1967 Cyclotomy and Difference Sets, *Lectures Adv. Math.* 2, Markham Publishing Co., Chicago.

Sullivan, D. D., 1966 Control of Error Propagation in Convolutional Codes, *Univ. Notre Dame Tech. Rept.* EE667.

Swan, R. G., 1962 Factorization of Polynomials over Finite Fields, *Pacific J. Math.*, **12**:1099–1106; *Math. Rev.*, **26**:*2432*.

Townsend, R. L., and E. J. Weldon, Jr., 1967 Self-orthogonal Quasi-cyclic Codes, *IEEE Trans. Inform. Theory*, **IT13**:183–195.

Varsharmov, R. R., 1957 Estimate of the Number of Signals in Error Correcting Codes, *Dokl. Akad. Nauk SSSR*, **117**:739–741; *Math. Rev.* **20**:*1596*.

Vasil'ev, Ju. L., 1962 On Nongroup Close-packed Codes, *Probl. Cybernetics*, **8**:337–339; *Math. Rev.*, **29**:*5661*.

Viterbi, A. J., 1967 Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm, *IEEE Trans. Inform. Theory*, **IT13**:260–269.

Waerden, B. L. van der, 1931 "Moderne Algebra," Springer-Verlag OHG, Berlin (republished in English as "Modern Algebra," Frederick Ungar Publishing Co., New York, 1940.); *Math. Rev.*, **23A**:*167*.

Wagner, T. J., 1965 A Remark Concerning the Minimum Distance of Binary Group Codes, *IEEE Trans. Inform. Theory*, **IT11**:458.

———, 1966*a* A Search Technique for Quasi-perfect Codes, *Inform. Control*, **9**:94–99; *Math. Rev.*, **32**:*9151*.

———, 1966*b* A Remark Concerning the Existence of Binary Quasi-perfect Codes, *IEEE Trans. Inform. Theory*, **IT12**:401.

———, 1967 Some Additional Quasi-perfect Codes, *Inform. Control*, **10**:334.

Wax, N., 1959 On Upper Bounds for Error Detecting and Error Correcting Codes

of Finite Length, *IEEE Trans. Inform. Theory*, **IT5**:168–174; *Math. Rev.*, **22**:*1472*.

Welch, L. R., 1967 Private communication.

Weldon, E. J., Jr., 1966 Difference-set Cyclic Codes, *Bell System Tech. J.*, **45**:1045–1055; *Math. Rev.*, **34**:*1101*.

———, 1968*a* New Generalizations of the Reed-Muller Codes, Part II: Nonprimitive Codes, *IEEE Trans. Inform. Theory*, **14**:199–205; *Math. Rev.*, **43**:*1748*.

———, 1968*b* Euclidean Geometry Cyclic Codes, chap. 23 in R. C. Bose and T. A. Dowling (eds.), "Proceedings of the Conference on Combinatorial Mathematics and Its Applications (April 10–14, 1967)," The University of North Carolina Press, Chapel Hill, N.C.; *Math. Rev.*, **40**:*5331*.

Wolf, J. K., 1965*a* On Codes Derivable from the Tensor Product of Check Matrices, *IEEE Trans. Inform. Theory*, **11**:281–284; *Math. Rev.*, **32**:*9152*.

———, 1965*b* On an Extended Class of Error-locating Codes, *Inform. Control*, **8**:163–169; *Math. Rev.*, **30**:*5848*.

——— and B. Elspas, 1963 Error-locating Codes—A New Concept in Error Control, *IEEE Trans. Inform. Theory*, **9**:113–117; *Math. Rev.*, **29**:*4623*.

Wolfowitz, J., 1961 "Coding Theorems of Information Theory," Springer-Verlag OHG, Berlin; *Math. Rev.*, **26**:*1227*.

Wozencraft, J. M., and I. M. Jacobs, 1965 "Principles of Communication Engineering," John Wiley & Sons, Inc., New York.

——— and B. Reiffen, 1961 "Sequential Decoding," The M.I.T. Press, Cambridge, Mass; *Math. Rev.*, **19**:824.

Wyner, A. D., 1965 Capabilities of Bounded Discrepancy Decoding, *Bell System Tech. J.*, **44**:1061–1122; *Math. Rev.*, **31**:*4652*.

——— and R. B. Ash, 1963 Analysis of Recurrent Codes, *IEEE Trans. Inform. Theory*, **IT9**:143–156; *Math. Rev.*, **29**:*1088*.

Wyner, A. D., and R. L. Graham, 1968 An Upper Bound on the Minimum Distance for a K-ary Code, *Inform. control*, **3**:46–52; *Math. Rev.*, **38**:*4202*.

Yudkin, H. L., 1964 Unpublished correspondence.

Zaremba, S. K., 1952 Covering Problems Concerning Abelian Groups, *J. London Math. Soc.*, **27**:242–246; *Math. Rev.*, **13**:817.

Zierler, N., 1958 On the Theorem of Gleason and Marsh, *Proc. Am. Math. Soc.*, **9**:236–237; *Math. Rev.*, **20**:*851*.

Ziv, J., 1962 Coding and Decoding for Time-discrete Amplitude Continuous Memoryless Channels, *IEEE Trans. Inform. Theory*, **IT8**:S199–S205.

———, 1966 Further Results on the Asymptotic Complexity of an Iterative Coding Scheme, *IEEE Trans. Inform. Theory*, **IT12**:168–171.

———, 1967 Asymptotic Performance and Complexity of a Coding Scheme for Memoryless Channels, *IEEE Trans. Inform. Theory*, **IT13**:356–359.

SECOND EDITION REFERENCES

Aho, A., J. Hopcroft, and J. Ullman, 1974 "The Analysis and Design of Computer Algorithms," Addison-Wesley Publishing Co., Reading, Massachusetts; *Math. Rev.*, 54:1706.

Anderson, J. B., and D. P. Taylor, 1978 "A Bandwidth Efficient Class of Signal-Space Codes," *IEEE Trans. Inform. Theory*, IT24:703-712.

Assmus, E. F., Jr., and H. F. Mattson, Jr., 1969 "New 5-Designs," *J. Combinatorial Theory*, 6:122-151; *Math. Rev.*, 42:7528.

Baumert, L. D., and R. J. McEliece, 1972 "Weights of Irreducible Cyclic Codes," *Inform. Control*, 20:158-175; *Math. Rev.*, 58:15665.

Baumert, L. D., R. J. McEliece, and H. C. A. van Tilborg, 1979 "Symbol Synchronization in Convolutionally Coded Systems (Corresp.)," *IEEE Trans. Inform. Theory*, IT25:362-365.

Baumert, L. D., and R. J. McEliece, 1973 "A Note on the Griesmer Bound (Corresp.)," *IEEE Trans. Inform. Theory*, IT19: 134-135; *Math. Rev.*, 52:12959.

Baumert, L. D., and R. J. McEliece, 1978 "Soft Decision Decoding of Block Codes," *DSN Progress Report* 42-47, Jet Propulsion Laboratory, California Institute of Technology, Pasadena, California.

Berger, T., 1971 "Rate Distortion Theory," Prentice-Hall, Englewood Cliffs, New Jersey.

Berlekamp, E. R. (ed.), 1974 "Key Papers in the Development of Coding Theory," *IEEE Press*, New York.

Berlekamp, E. R., (not published), "Merging and Sorting on Cray I," Institute for Defense Analysis, Working Paper No. 464.

Berlekamp, E. R., (not published), "Speedup for the CDC 6600," Institute for Defense Analysis, Working Paper No. 463.

Berlekamp, E. R., (not published), "Speedup for the Cray I," Institute for Defense Analysis, Working Paper No. 465.

Berlekamp, E. R., 1968 "Algebraic Coding Theory," McGraw-Hill, New York; *Math Rev.*, 38:6873.

Berlekamp, E. R., 1968 "Nonbinary BCH Decoding (Abstr.)," *IEEE Trans. Inform. Theory*, IT14:242.

Berlekamp, E. R., 1968 "Weight Enumeration Theorems," pp. 161-170 in Proc. 6th Annu. Allerton Conf. Circuit and Syst. Theory, Univ. Illinois Press, Urbana, Illinois; *Math. Rev.*, 41:8139.

Berlekamp, E. R., 1969 "On Subsets with Intersections of Even Cardinality," *Can. Math. Bull.*, 12:471-474; *Math. Rev.*, 40:2549.

Berlekamp, E. R., 1969 "Survey of Coding Theory," pp. 3-11 in Proceedings of the Waterloo Conference on Combinatorial Mathematics, *Recent Progress in Combinatorics*, Academic Press, Inc., New York.

Berlekamp, E. R., 1970 "A Survey of Coding Theory for Algebraists and Combinatorialists," CISM Publications, Udine, Italy.

Berlekamp, E. R., 1970 "Factoring Polynomials Over Large Finite Fields," *Math. of Computation*, 24:713-735; *Math. Rev.*, 48:1943.

Berlekamp, E. R., 1970 "On Sets of Ternary Vectors Whose Only Linear Dependencies Involve an Odd Number of Vectors," *Can. Math. Bull.*, 13:363-366; *Math. Rev.*, 43:3277.

Berlekamp, E. R., 1970 "Some Mathematical Properties of a Scheme for Reducing the Bandwidth of Motion Pictures by Hadamard Smearing," *Bell System Tech. J.*, 49:969-986; *Math. Rev.*, 42:5698.

Berlekamp, E. R., 1970 "The Weight Enumerators for Certain Subcodes of the Sec-

ond Order Binary Reed-Muller Codes," *Inform. Control*, 17:485-500; *Math. Rev.*, 44:8036.

Berlekamp, E. R., 1971 "Coding Theory and the Mathieu Groups," *Inform. Control*, 18:40-64; *Math. Rev.*, 43:1734.

Berlekamp, E. R., 1972 "Factoring Polynomials," pp. 1-7 in F. Hoffman, R. B. Levow, and R. S. D. Thomas (eds.) Proc. of the 3rd Southeastern Conference on Combinatorics, Graph Theory & Computing, Florida, Atlantic Univ., Boca Raton; *Math. Rev.*, 50:2134.

Berlekamp, E. R., 1972 "Long Primitive Binary BCH Codes Have Distance d ~ 2n ln R-1/log n . . . ," *IEEE Trans. Inform. Theory*, IT18:415-426; *Math. Rev.*, 48:13447.

Berlekamp, E. R., 1972 "Some Recent Results on the Combinatorial Game Called Welter's Nim," pp. 203-204 in Proceedings of the Sixth Annual Conference on Information Sciences and Systems, Princeton, New Jersey.

Berlekamp, E. R., 1972 "Survey of Coding Theory," *J. Royal Stat. Soc. Series A*, 135:44-73; *Math. Rev.*, 56:8163.

Berlekamp, E. R., 1973 "Engineering and Public Service," Bridge, Eta Kappa Nu, 69:1-3.

Berlekamp, E. R., 1973 "Goppa Codes," *IEEE Trans. Inform. Theory*, IT19: 590-592; *Math. Rev.*, 51:15157.

Berlekamp, E. R., 1973 "Review of 'Coding Theory' by J. H. van Lint," *IEEE Trans. Inform. Theory*, IT19:138.

Berlekamp, E. R., 1974 "The Hackenbush Number System for the Compression of Numerical Data," *Inform. Control*, 24:134-140; *Math. Rev.*, 50:6622.

Berlekamp, E. R., 1975 "Algebraic Codes for Improving the Reliability of Tape Storage," pp. 497-499 in Proceedings of the AFIPS Conference.

Berlekamp, E. R., 1975 "The Design of Slowly Shrinking Labelled Squares," *Math. of Computation*, 29:25-27; *Math. Rev.*, 51:10133.

Berlekamp, E. R., 1976 "An Analog to the Discriminant Over Fields of Characteristic Two," *Journal of Algebra*, 38:315-317; *Math. Rev.*, 53:8000.

Berlekamp, E. R., 1976 "Cooperative Bridge Bidding (Corresp.)," *IEEE Trans. Inform. Theory*, IT22:753-756.

Berlekamp, E. R., 1976 "Making Change," *Mathematics Magazine*, 49:195-198; *Math. Rev.*, 53:14589.

Berlekamp, E. R., 1977 "Long Block Codes Which Use Soft Decisions and Correct Erasure Bursts Without Interleaving," pp. 1-2 in Proc. of the National Telecommunications Conference, Los Angeles, California.

Berlekamp, E. R., 1980 "The Technology of Error-Correcting Codes," *Proc. of the IEEE*, 68:564-593.

Berlekamp, E. R., R. G. Gallager, and C. E. Shannon, 1967 "Lower Bounds to Error Probability for Coding on Discrete Memoryless Channels II," *Inform. Control*, 10:522-552; *Math. Rev.*, 35:7728.

Berlekamp, E. R., R. G. Gallager, and C. E. Shannon, 1967 "Lower Bounds to Error Probability for Coding on Discrete Memoryless Channels I," *Inform. Control*, 10:65-103; *Math. Rev.*, 35:1405.

Berlekamp, E. R., E. N. Gilbert, and F. W. Sinden, 1965 "A Polygon Problem," *Am. Math. Monthly*, 72:233-241; *Math. Rev.*, 31:3925.

Berlekamp, E. R., F. J. MacWilliams, and N. J. A. Sloane, 1972 "Gleason's Theorem on Self-Dual Codes," *IEEE Trans. Inform. Theory*, IT18:409-414; *Math. Rev.*, 48:13446.

Berlekamp, E. R., H. M. Frederickson, and R. C. Proto, 1974 "Minimum Conditions for Uniquely Determining the Generator of a Linear Sequence," *Utilitas*

Mathematicae, 5:305-315; *Math. Rev.*, 54:285.

Berlekamp, E. R., J. H. Conway, and R. K. Guy, 1982 "Winning Ways, Vol. I & II," Academic Press, London.

Berlekamp, E. R., J. H. van Lint, and J. J. Seidel, 1973 "A Strongly Regular Graph Derived from the Perfect Ternary Golay Code," Chapter 3 in J. N. Srivastava (ed.), A Survey of Combinatorial Theory, North-Holland/American Elsevier, Amsterdam; *Math. Rev.*, 51:270.

Berlekamp, E. R., 1982 "Bit-Serial Reed-Solomon Encoders," *IEEE Trans. Inform. Theory*, IT28:869-874.

Berlekamp, E. R., 1983 "The Construction of Fast High-Rate Soft-Decision Block Decoders," *IEEE Trans. Inform. Theory*, IT29372-377.

Berlekamp, E. R., R. J. McEliece, and H. C. A. van Tilborg, 1978 "On the Inherent Intractability of Certain Coding Problems (Corresp.)," *IEEE Trans. Inform. Theory*, IT24:384-386; *Math. Rev.*, 58:13912.

Berlekamp, E. R., and D. D. Sullivan, 1970 "Some Results on Normalized Truncation Groups" pp. 1-1-1 in Proc. of the University of Missouri at Rolla M. J. Kelly Communications Conference.

Berlekamp, E. R., and F. K. Hwang, 1972 "Constructions for Balanced Howell Rotations for Bridge Tournaments," *J. Combinatorial Theory*, 12:159-165; *Math. Rev.*, 46:8862.

Berlekamp, E. R., and J. Justesen, 1974 "Some Long Cyclic Linear Binary Codes are Not So Bad," *IEEE Trans. Inform. Theory*, IT20:351-356; *Math. Rev.*, 52:2709.

Berlekamp, E. R., and J. L. Ramsey, 1978 "Readable Erasures Improve the Performance of Reed-Solomon Codes (Corresp.)," *IEEE Trans. Inform. Theory*, IT24:632-633.

Berlekamp, E. R., and L. R. Welch, 1972 "Weight Distributions of the Cosets of the (32,6) Reed-Muller Code," *IEEE Trans. Inform. Theory*, IT18:203-207; *Math. Rev.*, 52:16844.

Berlekamp, E. R., and L. R. Welch, submitted to IEEE Trans. Inform. Theory, "A New Reed-Solomon Decoding Algorithm."

Berlekamp, E. R., and N. J. A. Sloane, 1969 "Restrictions on Weight Distribution and Reed-Muller Codes," *Inform. Control*, 14:442-456; *Math. Rev.*, 39:5215.

Berlekamp, E. R., and O. Moreno, 1973 "Extended Double-Error-Correcting Binary Goppa Codes are Cyclic (Corresp.)," *IEEE Trans. Inform. Theory*, IT19:817-818; *Math. Rev.*, 51:5157.

Berlekamp, E. R., and P. Tong, submitted to IEEE Trans. Inform. Theory, "Interleaving."

Berlekamp, E. R., and R. J. McEliece, presented at the NATO Advanced Study Institute in France, July 11-22, 1983 "Average-Case Optimized Buffered Decoders," to be published in the Proceedings of the NATO Advanced Study Institute.

Berlekamp, E. R., and R. L. Graham, 1970 "Irregularities in the Distribution of Finite Sequences," *J. Number Theory*, 2:152-161; *Math. Rev.*, 42:4500.

Blahut, R. E., 1979 "Transform Techniques for Error Control Codes," *IBM J. of Res. Dev.*, 23:299-315.

Blake, I. F., 1971 "The Leech Lattice as a Code for the Gaussian Channel," *Inform. Control*, 19:66-74; *Math. Rev.*, 47:1537.

Brillhart, J., and N. Zierler, 1968 "On Primitive Trinomials (mod 2)," *Inform. Control*, 13:541-554.

Carlitz, L., and S. Uchiyama, 1957 "Bounds for Exponential Sums," *Duke Math. J.*, 24:37-41.

Carlitz, L., D. P. Roselle, and R. A. Scoville, 1971 "Some Remarks on Ballot-Type Sequences of Positive Integers," *J. Combinatorial Theory*, 11:258-271; *Math. Rev.*, 43:7351.

Chase, D., 1972 "A Class of Algorithms for Decoding Block Codes with Channel Measurement Information," *IEEE Trans. Inform. Theory*, IT18:170-182; *Math. Rev.*, 51:15161.

Chien, R. T., 1969 "Review of 'Algebraic Coding Theory' by E. R. Berlekamp," *IEEE Trans. Inform. Theory*, IT15:509-510.

Conway, J. H., 1969 "A Group of Order 8, 315, 553, 613, 086, 720" *Bull. London Math. Soc.*, 1:79-88; *Math. Rev.*, 40:1470.

Cover, T. M., R. J. McEliece, and E. C. Posner, 1981 "Asynchronous Multiple-Access Channel Capacity," *IEEE Trans. Inform. Theory*, IT27:409-413.

Davisson, L D., R. J. McEliece, M. B. Pursly, and M. S. Wallace, 1981 "Efficient Universal Noiseless Source Codes," *IEEE Trans. Inform. Theory*, IT26:269-280.

Delsarte, P., 1975 "On Subfield Subcodes of Modified Reed-Solomon Codes," *IEEE Trans. Inform. Theory*, IT21:575-576; *Math. Rev.*, 53:15547.

Delsarte, P., and R. J. McEliece, 1976 "Zeroes of Functions in Finite Abelian Group Algebras," *Amer. J. of Math*, 98:197-224; *Math. Rev.*, 53:2908.

Dobrusin, R. L., 1972 "Survey of Soviet Research in Information Theory," *IEEE Trans. Inform. Theory*, IT18:703-724; *Math. Rev.*, 52:2737.

Edmonds, J., 1965 "Paths, Trees, and Flowers," *Can. J. Math*, 17:449-467; *Math. Rev.*, 31:2165.

Forney, G. D., Jr., 1970 "Convolutional Codes I: Algebraic Structure," *IEEE Trans. Inform. Theory*, IT16:720-738; *Math. Rev.*, 43:1741.

Forney, G. D., Jr., 1972 "Convolutional Codes II: Maximum Likelihood Decoding," and "Convolutional Codes III: Sequential Decoding," *Tech. Report 7004-1*, Inform. Syst. Lab., Stanford Univ., Stanford, California; *Math. Rev.*, 52:16852, 52:16853.

Goethals, J. M., 1971 "On the Golay Perfect Binary Code," *J. Combinatorial Theory*, 2:178-186.

Goethals, J. M, 1973 "Some Combinatorial Aspects of Coding Theory," pp. 189-208 in J. N. Srivatava (ed.), A Survey of Combinatorial Theory, North-Holland/American Elsevier, Amsterdam.

Goethals, J. M., and J. J. Seidel, 1970 "Strongly Regular Graphs Derived from Combinatorial Designs," *Can. J. Math*, 22:597-614.

Goethals, J. M., and S. L. Snover, 1972 "Nearly Perfect Binary Codes," *Discrete Math.*, 3:65-88.

Goppa, V. D., 1971 "Rational Presentation of Codes and (L, g) Codes," *Problemy Peredachi Informatsii (Moscow)*, 7:41-49; *Math. Rev.*, 45:3116.

Goppa, V. D., 1970 "A New Class of Linear Error-Correcting Codes," *Problemy Peredachi Informatsii (Moscow)*, 6:24-30; *Math. Rev.*, 45:3115.

Gore, W. C., 1973 "Transmitting Binary Symbols with Reed-Solomon Codes," pp. 495-597 in Proc. of the Princeton Conference on Information Sciences and Systems, Princeton, New Jersey.

Heller, J. A., and I. M. Jacobs, 1971 "Viterbi Decoding for Satellite and Space Communications," *IEEE Trans. Commun. Technol.*, COM19:835-848.

Heller, R. M., 1967 "Forced-Erasure Decoding and the Erasure Reconstruction Spectra of Group Codes," *IEEE Trans. Commun. Technol.*, COM15:390-397.

van der Horst, J., and T. Berger, 1976 "Complete Decoding of Triple-Error-Correcting Binary BCH Codes," *IEEE Trans. Inform. Theory*, IT22:138-147; *Math. Rev.*, 53165.

Justesen, J., 1972 "A Class of Constructive Asymptotically Good Algebraic Codes,"

IEEE Trans. Inform. Theory, IT18:652-656; *Math. Rev.*, 52:5190.

Justesen, J., 1976 "On the Complexity of Decoding Reed-Solomon Codes," *IEEE Trans. Inform. Theory*, IT22:237-238; *Math. Rev.*, 57:5404.

Karlin, M., 1969 "New Binary Coding Results by Circulants," *IEEE Trans. Inform. Theory*, IT15:81-92; *Math. Rev.*, 40:2425.

Karlin, M., 1970 "Decoding of Circulant Codes," *IEEE Trans. Inform. Theory*, IT16:797-802; *Math. Rev.*, 42:9064.

Karp, R., 1972 "Reducibility Among Combinatorial Problems," in R.. E. Miller, J. W. Thatcher, and J. Bohlinger (eds.) Proc. Symp. Complexity of Combinatorial Comput.., Plenum, New York; *Math. Rev.*, 51:14644.

Kasami, T., 1971 "The Weight Enumerators for Several Classes of Subcodes of the Second Order Binary Reed-Muller Codes," *Inform. Control*, 18:369-394; *Math. Rev.*, 43:7253.

Kasami, T., S. Lin, and W. W. Peterson, 1967 "Some Results on Cyclic Codes Which are Invariant Under the Affine Group and Their Applications," *Inform. Control*, 11:475-496; *Math. Rev.*, 37:5051.

Kasami, T., and N. Tokura, 1970 "On the Weight Structure of Reed-Muller Codes," *IEEE Trans. Inform. Theory*, IT16:752-759; *Math. Rev.*, 43:3036.

Kerdock, A. M., 1972 "A Class of Low-Rate Nonlinear Binary Codes," *Inform. Control*, 20:182-187; *Math. Rev.*, 49:10438.

Knuth, D., 1968 "The Art of Computer Programming—Fundamental Algorithms, Vol. 1," Addison-Wesley Publishing Co., Reading, Massachusetts.

Knuth, D., 1969 "The Art of Computer Programming—Seminumerical Algorithms, Vol. 2," Addison-Wesley Publishing Co., Reading, Massachusetts.

Knuth, D., 1973 "The Art of Computer Programming—Sorting and Searching, Vol. 3," Addison-Wesley Publishing Co., Reading, Massachusetts.

Leech, J., and N. J. A. Sloane, 1971 "Sphere Packings and Error-Correcting Codes," *Can. J. Math.*, 23:718-745; *Math. Rev.*, 44:3211.

Lempel, A. and S. Winograd, 1977 "A New Approach to Error Correcting Codes," *IEEE Trans. Inform. Theory*, IT23:503-508; *Math. Rev.*, 57:5406.

Leung, K. S., and L. R. Welch, 1981 "Erasure Decoding in Burst-Error Channels," *IEEE Trans. Inform. Theory*, IT27:160-167.

Levinson, N., 1970 "Coding Theory: A Counterexample to G. H. Hardy's Conception of Applied Mathematics," *Amer. Math. Monthly*, 77:249-258; *Math. Rev.*, 40:8491.

van Lint, J. H., 1971 "Coding Theory," Lecture Notes in Mathematics, No. 201, Springer-Verlag, Berlin; *Math. Rev.*, 44:3764.

van Lint, J. H., 1972 "A New Description of the Nadler Code (Corresp.)," *IEEE Trans. Inform. Theory*, IT18:825-826; *Math. Rev.*, 52:2721.

van Lint, J. H., 1973 "A Survey of Perfect Codes," *Rocky Mountain J. Math*, 5:199-244.

Lin, J., 1972 "On the Number of Information Symbols in Polynomial Codes," *IEEE Trans. Inform. Theory*, IT18:785-794.

Lin, S., and D. J. Costello, Jr., 1983 "Error Control Coding," Prentice-Hall, Englewood Cliffs, New Jersey.

Lindsey, W. C., and M. K. Simon, 1973 "Telecommunications Systems Engineering," Table 5-1, pp. 199-209, Prentice-Hall, Englewood Cliffs, New Jersey.

Longo, G., 1979 "Review of 'The Theory of Information and Coding: A Mathematical Framework for Communication' by R. J. McEliece," *IEEE Trans. Inform. Theory*, IT25:367-368.

MacWilliams, F. J., and N. J. A. Sloane, 1977 "The Theory of Error-Correcting Codes," North-Holland Publishing Co., Amsterdam; *Math. Rev.*, 57:5408b.

McEliece, R. J., 1970 "On the Symmetry of Good Nonlinear Codes," *IEEE Trans. Inform. Theory*, IT16:609-611.

McEliece, R. J., 1973 "Comments on 'A Class of Codes for Axisymmetric Channels and A Problem from the Additive Theory of Numbers' by R. R. Varshanov," *IEEE Trans. Inform. Theory*, IT19:137; *Math Rev.*, 53:10477.

McEliece, R. J., 1977 "The Theory of Information and Coding: A Mathematical Framework for Communication," Addison-Wesley, Reading, Massachusetts.

McEliece, R. J., 1977 "Theory of Information and Coding," Chap. 9 in Encyclopedia of Mathematics and Its Applications, Vol. 3, Addison-Wesley, Reading, Massachusetts; *Math. Rev.*, 57:9301.

McEliece, R. J., 1981 "Practical Codes for Photon Communication," *IEEE Trans. Inform. Theory*, IT27:393-398.

McEliece, R. J., E. R. Rodemich, H. Rumsey, Jr., and L. R. Welch, 1977 "New Upper Bounds on the Rate of a Code via the Delsarte-MacWilliams Inequalities," *IEEE Trans. Inform. Theory*, IT23:157-166; *Math. Rev.*, 55:12296.

McEliece, R. J., and H. Rumsey, Jr., 1972 "Euler Products, Cyclotomy and Coding," *J. Number Theory*, 4:302-311; *Math. Rev.*, 45:8638.

McEliece, R. J., and J. K. Omura, 1977 "An Improved Upper Bound on the Block Coding Error Exponent for Binary-Input Discrete Memoryless Channels (Corresp.)," *IEEE Trans. Inform. Theory*, IT23:611-613; *Math. Rev.*, 56:15161.

McEliece, R. J., 1975 "Irreducible Cyclic Codes and Gauss Sums," pp. 185-202 in M. Hall, Jr., and J. H. van Lind (eds.) "Combinatorics," D. Reidel Publishing Co., Dordrecht, Holland; *Math. Rev.*, 50:16095.

Moenck, R., 1973 "Fast Computation of GCD's," pp. 142-151 in Proc. of the Fifth Annual ACM Symposium on the Theory of Computing; *Math. Rev.*, 56:13766.

Murakami, H., I. S. Reed, and L. R. Welch, 1977 "A Transform Decoder for Reed-Solomon Codes in Multiple-User Communication Systems," *IEEE Trans. Inform. Theory*, IT23:675-683; *Math. Rev.*, 57:9282.

Patterson, N., 1975 "The Algebraic Decoding of Goppa Codes," *IEEE Trans. Inform. Theory*, IT21:203-207; *Math. Rev.*, 51:15175.

Peterson, W., and E. Weldon, 1972 "Error-Correcting Codes," 2nd ed., M.I.T. Press, Cambridge, Massachusetts; *Math. Rev.*, 49:12164.

Pless, V., 1969 "On a New Family of Symmetry Codes and Related New Five-Designs," *Bull. Am. Math. Soc.*, 75:1339-1342; *Math. Rev.*, 39:6763.

Preparata, F. P., 1968 "A Class of Optimum Nonlinear Double-Error-Correcting Codes," *Inform. Control*, 13:378-400; *Math. Rev.*, 39:3894.

Preparata, F. P., 1968 "Weight and Distance Structure of Nordstrom-Robinson Quadratic Codes," *Inform. Control*, 12:466-473; *Math. Rev.*, 39:3893.

Reed, I. S., R. A. Scholtz, T. K. Truong, and L. R. Welch, 1978 "The Fast Decoding of Reed-Solomon Codes Using Fermat Theoretic Transforms and Continued Fractions," *IEEE Trans. Inform. Theory*, IT24:100-106; *Math. Rev.*, 58:20794.

Reed, I. S., T. K. Truong, and L. R. Welch, 1978 "The Fast Decoding of Reed-Solomon Codes Using Fermat Transforms (Corresp.)," *IEEE Trans. Inform. Theory*, IT24:497-499; *Math. Rev.*, 58:20795.

Scholtz, R. A., and L. R. Welch, 1970 "Mechanization of Codes with Bounded Synchronization Delays," *IEEE Trans. Inform. Theory*, IT16:438-446; *Math. Rev.*, 42:7401.

Scholtz, R. A., and L. R. Welch, 1978 "Group Characters: Sequences with Good Correlation Properties," *IEEE Trans. Inform. Theory*, IT24:537-545.

Shearer, J. B., and R. J. McEliece, 1977 "There is no MacWilliams Identity for Convolutional Codes (Corresp.)," *IEEE Trans. Inform. Theory*, IT23:775-776.

Sidel'nikov, V. M., 1971 "On Weight Spectrum of Binary BCH Codes," *Problemy*

Peredachi Informatsii (Moscow), 7:14-22.

Slepian, D. (ed.). 1974 "Key Papers in the Development of Information Theory," IEEE Press, New York.

Sloane, N. J. A., 1972 "A Survey of Constructive Coding Theory, and a Table of Binary Codes of Highest Known Rate," *Discrete Math.*, 3:265-294; *Math. Rev.*, 46:10494.

Sloane, N. J. A., and E. R. Berlekamp, 1970 "Weight Enumerator for Second-Order Reed-Muller Codes," *IEEE Trans. Inform. Theory*, IT16:745-751; *Math. Rev.* 42:9071.

Sugino, M., Y. Ienaga, N. Tokura and T. Kasami, 1971 "Weight Distribution of (128, 64) Reed-Muller Code (Corresp.)," *IEEE Trans. Inform. Theory*, IT17:627-628.

Sugiyama, Y., M. Kasahara, S. Hirasawa, and T. Namekawa, 1975 "A Method for Solving Key Equation for Decoding Goppa Codes," *Inform. Control*, 27:87-99; *Math. Rev.*, 52:2728.

Tietavainen, A., 1973 "On the Nonexistence of Perfect Codes Over Finite Fields," *SIAM J. Appl. Math.*, 24:88-96; *Math. Rev.*, 48:3609.

van Tilborg, H. C. A., 1971 "On Weights in Codes," pp. 1-76 in T. H. Rep. 71-WSK-03, Dept. Math., Technological Univ. Eindhoven, The Netherlands; *Math. Rev.*, 48:10659.

Viterbi, A. J., 1971 "Convolutional Codes and Their Performance in Communication Systems," *IEEE Trans. Commun. Technol.*, COM19:751-772; *Math. Rev.*, 52:16858.

Viterbi, A. J., and J. K. Omura, 1978 "Digital Communication and Coding," McGraw-Hill Book Co., New York.

Welch, L. R., 1974 "Lower Bounds on the Maximum Cross Correlation of Signals (Corresp.)," *IEEE Trans. Inform. Theory*, IT20:397-399.

Welch, L. R., and R. A. Scholtz, 1979 "Continued Fractions and Berlekamp's Algorithm," *IEEE Trans. Inform. Theory*, IT25:19-27.

Welch, L. R., and R. J. McEliece, H. Rumsey, Jr., 1974 "A Low-Rate Improvement on the Elias Bound (Corresp.)," *IEEE Trans. Inform. Theory*, IT20:676-678; *Math. Rev.*, 51:7720.

Witt, E., 1938 "Die 5-Fach Transitiven Gruppen von Mathieu," *Abh. Math. Sem. Univ. Hamburg*, 12:256-264.

Wu, W., 1976 "New Convolutional Codes—Part III," *IEEE Trans. Commun.* COM24:946-955.

Zetterberg, L. H., 1962 "Cyclic Codes from Irreducible Polynomials for Correction of Multiple Errors," *Proc. of IEEE Trans. Inform. Theory*, IT8:13:20; *Math. Rev.*, 28:1029.

Ziablov, V. V., 1971 "A Bound on the Complexity of the Construction of Binary Linear Concatenated Codes," *Problemy Peredachi Informatsii (Moscow)*, 7:5-13.

This page intentionally left blank

Index

Abbreviated logical-circuit diagrams, 35–36
Abramson, N. M., 144, 342
Actual distance, 294–296, 409–410
Addition theorems [*see* Mann (1965)]
Affine permutation group, 226, 422
Affine polynomial, 243–272
Affine subspace, 266, 364, 366
  enumeration, 261
  of GF(2^5), 373
Affine transformation of quadratic forms, 415–416
Albert, A., 106*n*.
Algebraic closure, 111–112
Algebraic decoders (*see* Decoders)
Algebraic equations for correcting additional errors, 240
Algebraic factor of $2^m - 1$, 173
Algebraic structure of finite fields, 96–104
Algorithm, factorization, 146–150
  for solving the key equation, 184–188
  (*See also* Decoders)
† Alphabets, inner and outer, 347–349
  whose order is not a prime power, 305, 307
  size, 207, 305
Alter, R., 309
Alternate BCH codes, 223–224
American Telephone and Telegraph Company, xi
Amplitude modulation, 202–203
Amplitude signals, 202–203
AND gate, 30
Andryanov, V. I., 333
Annex check digits, 333
Antilogs and logs, 48–50
  GF(16), 108
  GF(25), 442
  GF(31), 217
  GF(32), 441
Arithmetic, ones-complement, 109
Arithmetic operations, 21
Arithmetic progressions, partitions which avoid, 118
Ash, R. B., ix, 389, 392, 394
  (*See also* Wyner)
Assmus, E. F., 358, 361, 395, 429
  (*See also* Mattson)
Associates, linearized and conventional, 252–255
* (*see* Concatenation)
Asymptotic bounds, on BCH codes, 290–294
  on error-correction capability of optimum codes, 302
  on error probability of optimum codes, 324–329
  (*See also* Bounds)
Augment, 331–332, 335
‡ Augmented codes, 322–323, 331–332, 335, 353
Augmented matrix, 60–62
Augmented-matrix equation, 60–62
Auxiliary equation, 181
Average distance bound, 311–318

$B(k)$, 184
Bartee, T. C., 47
Basic sequence, 387
Baumert, L. D., xi
BCH (Bose, Chaudhuri, Hocquenghem) (*see under* BCH)
BCH bound, 407
  elegant proof of, 408
  improvements, 409–410
§ BCH codes, 12, 307, 309, 333, 352
  asymptotic information symbols, 290–294
  asymptotic rate, 290–294
  binary, 176
  decoding (*see* Decoders)
  designed distance, 274
  double-error-correcting, complete decoding algorithm, 426
    coset weight enumeration, 427–428, 440
  vs. EG codes, 381
  extended, 223–229
  high-rate, 299
    weight enumerators, even m, 425–429
    odd m, 421
  information symbols, 273–297
  of length 31, 176–177, 239
  low-rate, weight enumerators, even m, 424–425
    odd m, 418–421, 436–437
  low-weight codewords, 265, 269, 270
  nonprimitive, 176

† Some authors have used the term "alphabet" in place of the more conventional term "code" and the term "letter" in place of the term "codeword."

‡ Some authors have used the term "augmented codes" in the same sense that I use the term "extended codes." See pages 331–332.

§ Following Mattson & Solomon (1961), some authors have mistakenly used the term "BCH codes" to mean "cyclic codes." In fact, only a small subset of cyclic codes are BCH codes. Although all BCH codes are cyclic, this fact was not mentioned in the original Bose-Chaudhuri papers.

BCH codes, perfect, 305
vs. PG codes, 381
primitive, 176
vs. QR codes, 361
rate, 273–297
vs. RM codes, 364
short, weight enumerators, 432–433
weight enumerators of certain, 417–429
high-rate, even m, 425–429
odd m, 421
low-rate, even m, 424–425
odd m, 418–421, 436–437
BCH decoders (*see* Decoders)
Bell polynomials, 405
Bell Telephone Laboratories, Murray Hill, xi
Berkeley, University of California, xi
Berlekamp, E. R., 118, 170*n*., 272, 325, 326, 328, 349, 351, 392–394
(*See also* Jacobs; Shannon)
Berlekamp, Jennifer, v
Berlekamp-Preparata codes, 394
Bernstein, A. J. (*see* Robinson)
Binary adder, 31, 32
Binary BCH codes (*see* BCH codes)
Binary codes (*see* Codes)
Binary erasure channel, 349
Binary polynomial, 13
enumeration, 76–80
division, 121
(*See also* Polynomials)
Binary repetition codes (*see* Codes)
Binary sum, 3
Binary symmetric channel, 301, 327, 397
capacity, 301
Binary trinomials, 272
Binomial coefficient, 97
congruences, 113, 227, 411
Binomials, resultant of, 163
Birkhoff, G., viii, ix, 88
Bivariate generating function, 86
Bivariate polynomial, 341
Block, R. E., 305
Block code (code in which all codewords have same length), 1–8
vs. convolutional code, 388–389
Block designs [*see* Hall (1967)]
Block length n, 1–8
determination from generator polynomial, 151–152
Bookkeeping via generating functions, 73
Bordered symmetric matrices [*see* Carlitz and Hodges (1956*a*)]
Bose, R. C., vii, xi, 11, 208
(*See also under* BCH)
Bose distance, 281
Bounded discrepancy decoding [*see* Wyner (1965)]
Bounds, asymptotic, on error-correction capability, 302
on probability of error, 324–329
average distance, 311–318
BCH, 229–231, 409–410
Bounds, Chernoff, 300–301
Elias, 318–324
error, asymptotic, 324–329
for concatenated codes, 348
with feedback, 328–329
Gilbert, 321–324, 328–329, 335–336, 392
Griesmer, 329, 335
Hamming-Rao, 298–302
Johnson, 328–329
Plotkin, 312, 321, 328
Rao, 298–302
residue, 353–355
from restrictions, 407–413
Sacks, 323, 329
sphere-packing, 327
Varsharmov, 323, 329
volume, 298–302
Wax, 329
† Brackets (greatest integer less than or equal to), 22
Brillhart, J., xi, 153, 154
Burst-correcting codes, 144, 340, 346–347, 394
Burst noise, models of [*see* Mandelbrot (1965)]
Bursts, of erasures, 392–394
of errors, 347, 393–394
of noise, 340
and permutations, 143–144
Burton, H. O., 339, 341
Bussgang, J. J., 392

Calabi, L., 330
California, University of, Berkeley, xi
Camion, P., 265
Canonical forms of matrices, 52–59
Canonical ordering of product code, 338–346
Capacity, vii, 326, 392
of binary symmetric channel, 301
Carlitz, L., 170*n*., 272
Cascaded decoder, 340
Cell, 56
Cerveira, A. G., 409–410
Chakravarti, I. M., xi
Channel, 326
binary symmetric, 301, 327
inner and outer, 347–349
Channel input alphabet, 207
Character of 2, cubic, 159
quadratic, 158
quartic, 159
Characteristic, 96
Chaudhuri, D. K. Ray-, vii, 11, 208
(*See also under* BCH; Bose)
Chebyshev polynomials over GF(2), 118
Check digits, 1–8
(*See also* Information symbols)
Check polynomials of convolutional code, 211, 389
Chernoff, H., 300
Chernoff bound, 300–301

† Of course, brackets are also used simply as additional parentheses in some situations.

Chien, R. T., 132
Chien search, 132–135, 216–217
Chinese remainder theorem, 339
  for integers, 28
  for polynomials, 29
Chow, D. K., 375
Circulant matrices, orthogonal [*see* MacWilliams (1968*b*)]
Classic coding theorem, Shannon's, vii, 326
Clock signal, 33
Close-packed codes, 302–309
  (*See also* Perfect codes)
Cocke, J., 303*n*.
Codes, 2
  alternate, 223–224
  asymptotically error-free, 344–349
    explicit, 344–346
  augmented, 322–323, 331–332, 335
  BCH (*see* BCH codes)
  Berlekamp-Preparata, 394
  binary repetition, 1–4, 310, 363
  Bose-Chaudhuri (*see* BCH codes)
  burst-correcting, 144, 340, 346–347, 394
  cascaded, 338–349
  close-packed, 302–309
  concatenated, 347–349
  constacyclic, 303–305
  convolutional, 388–394
  cyclic product, 340–343
  direct-product, 338–346
  direct-sum, 347
  EG, 375–376, 381, 385–388
  Elias, 344–346
  equidistant, 315–318
  euclidean-geometry, 375–376, 381, 385–388
  explicit, asymptotically error-free, 344–346
  expurgated, 331–332, 335–336, 353, 410
  extended, 223–229
  with feedback, 328–329, 349
  formidable-looking, 210–211
  geometric, 375–388
  geometric interpretation of, 308
  Golay (*see* Golay codes)
  Gray, 206
  Hamming (*see* Hamming codes)
  inner and outer, 347–349
  invariance under permutation groups, 226
  iterated direct-product, 344–346
  Kronecker product, 338*n*.
  length, 15, 328
  lengthened, 331–332, 336
  long (*see* Asymptotic bounds)
  maximal-length shift-register, 315–316, 334, 348, 363
  maximum, 309*n*.
  maximum distance separable, 309–311
  modified, 331–338
  multidimensional, 338–346
  negacyclic, 207–217, 268
  noncyclic but linear, 324, 350–352
  nonlinear, systematic, 305, 328
Codes, nonprimitive negacyclic, 213–214
  optimal, 309*n*.
  outer and inner, 347–349
  perfect, 302–309
    Golay (*see* Golay codes)
    single-error-correcting, 303–305
  PG, 375–385
  primitive, BCH, 176
    negacyclic, 213–214
  product, cyclic, 340–343
    direct, 338–346
    Kronecker, 338*n*.
    tensor, 346–347
  projective geometry, 375–385
  punctured, 331–332, 334
  QR (*see* QR codes)
  quadratic nonlinear, 328
  quadratic residue (*see* QR codes)
  quasi-perfect, 309, 428
  quinary, 209–211
  recurrent, 388–394
  Reed-Muller (*see* RM codes)
  Reed-Solomon, 218, 232*n*., 310, 346–349, 429–431
  repetition (*see* Repetition codes)
  residue, 352–361
    (*See also* QR codes)
  RM (*see* RM codes)
  RS, 218, 232*n*., 310, 346–349, 429–431
  shortened, 331–332, 336
  single-parity-check, 3–4, 310
  Srivastava, 350–352
  systematic, 303*n*.
  tensor product, 346–347
Codeword, 2
  with low weights, 262–272
  weight of, binary, 6–7
Codeword-locator polynomial, 262
Codeword weight enumerators, 397–440
Cohen, E. L., 309
Column-reduced standard echelon matrix, 52*n*.
Combinatorial analysis [*see* Riordan (1958)]
Combinatorial mathematics [*see* Ryser (1963)]
Combinatorial theory [*see* Hall (1967)]
  foundations of [*see* Rota (1964)]
Communication theory (*see* Information theory)
Complement, 275
Complete decoding algorithm, 2, 123, 233, 324
  for double-error-correcting binary BCH codes, 426
Component-wise product, 365, 413
Composite moduli, 28–29
Compositions, number of, 386
Computation problem of sequential decoding, 393
Computer programming [*see* Knuth (1967)]
Concatenated codes, 347–349
Concatenation, of codewords, 347
  of sequences, 275
Concave function, 314

Confounding in factorial experiments [*see* Fisher (1942)]
Congruences, 14
of binomial coefficients, 113
of generating functions, 73
Conjectures, 294
(*See also* Unsolved problems)
Conjugate constraints, 236
Conjugates, 101
number of, 100
Constacyclic codes, 303–305
Constraint length of convolutional codes, 389
Continued fractions, 23–27, 36–44
Continuous nondifferentiable function, 293
Contraction of code, 346
Control circuitry, 43
Control signals, 34
Conventional associate, 252–255
Convergents of continued fractions, 25
Convex function, 314
Convex hull, 327
Convolutional codes (recurrent codes), 388–394
construction of, 391–392
Coset, 6–7, 204
Coset enumeration for double-error-correcting binary BCH codes, 440
Coset leader, 6–7
weight distribution of, 324, 427–428
Cosh, 405–407, 434
Counting (*see* Enumerations)
Cramer, H. [*see* Feller, (1943)]
Critical rates, 327
Critical sphere, 318
Cubic, discriminant, 169
irreducibility, 169
over GF(3), 271
Cubic character of 2, 159
Cubic transformation, 115–117
Cunningham, A. J. C., 174
Cyclic code, 129
encoder, 128, 132
some weight enumerators, 432–433
Cyclic constraints, 236
Cyclic ordering, 339–346
Cyclic product codes, 340–343
Cyclic shifts, 47, 275
Cyclotomic polynomials, 90–95
degrees of factors, 92, 156–159
Cyclotomy, 159

$\bar{D}$, 315
$D(k)$, 184
$d(n, M)$, 327
Dalen, K., 170*n.*
Davenport, W. B., 205
Daykin, D. E., 115, 199
Decoders, algebraic vs. threshold, 375
for BCH codes, 178–180, 184–188
alternate codes, 223–224
binary codes, 192–195
complete and incomplete, 231–240, 425–426
Decoders, for BCH codes, cost, 198–199
double-error-correcting, 425–426
errata-correcting, 229–231
examples, 195, 221–223, 237
extended, 224–229
implementation, 195–199
more than t errors, 231–240
multiply-extended, 333
nonbinary, 210–221
speed, 198–199
bounded discrepancy [*see* Wyner (1965)]
cascaded, 340
complete and incomplete, 2–3
errata-correcting, 229–231
inner and outer, 347–349
for Lee metric codes, 184–188
list, 326
for more than t errors, 231–240, 367–375
for negacyclic Lee metric codes, 184–188
permutation, 361
permutation-threshold, 369
probabilistic, 392–393
for reversible double-error-correcting codes, 141
sequential, 392–393
for Srivastava codes, 351
threshold, 367–375
Decoding error vs. decoding failure, 2
probability of, 397–400
Decomposition, into concatenation, 347
of product codes, 342–343
Deficiency, 387
Deg (degree of), 25
of cyclotomic polynomial, 92
of element in finite field, 101
of factors of cyclotomic polynomials, 156–159
in terms of order, 100
of 0, 28, 183
Delete check digits, 334
Delsarte, P., 375, 379, 384
(*See also* Goethals)
$\Delta^{(2i)}(z)$, 236
Delta function, inverse, 83
Derivatives, 216
Design constraints, 31
Designed distance of BCH codes, 274
Detection of error bursts, 394
Dickson, L. E., ix, 106*n.*, 361*n.*, 414, 423
Difference of generating functions, 73
Difference set, 380–381
perfect simple, 396
Difference-set codes, 381
Dim (dimension) (*see* Dimension)
Dimension, of code, 339*n.*
of direct-product code, 339
of direct-sum code, 347
Hausdorf, 294
Direct decoder for convolutional codes, 390–391
Direct-product codes, 338–346
Direct sum of subspaces, 401
Direct-sum codes, 347

Discriminant, 160–171, 233
cubic, 169
mod 8, 168–171
and Moebius function, 165
quadratic, 169
in terms of resultant, 161
trinomial, 163–164
Distance, actual, 294–296, 409–410
average, 311–318
BCH, 409–410
Bose, 281
designed, 274
Hamming, 204–205
Lee, 204–205
Distance bounds (*see* Bounds)
Distinct irreducible factors, even or odd, 79
Distribution of computation in sequential decoding, 393
Divided differences, 351
Division algorithm, 15, 25–26, 121
Division circuit, 122, 124
Dot (*see* Iterated concatenation)
Dot product, 401
Dowling, T. A., xi
Duals, of linear code, 310*n*.
weight relations, 400–407
of linearized polynomials, 250–255, 271
of subspace, 401

E_M, 327
$E(R,L)$, 326
EG (euclidean geometry) codes, 376, 381, 385–388
Ekonomi, E. A., 394
Elementary symmetric functions (σ), (*see* Error-locator polynomial)
Elias, P., 318, 326, 339, 389, 392
Elias bound, 318–324, 328
Elias codes, 344–346
Elspas, B., 347
Empty set (null set), 429
Encoder, 2–4
for convolutional code, 390
for cyclic code, 128, 132
for Hamming code, 125
inner and outer, 347–349
Encoding problem, 2
Ensemble of all codes, 324–325
Enumeration [*see* Riordan (1958)]
Enumerations, of check symbols (*see* Information symbols)
of codewords by weight, 397–440
of compositions, 386
of information symbols (*see* Information symbols)
of irreducible polynomials by degree, 70–86
of matrices by rank, 271–272
persymmetric, 199
of necklaces, 86
of polynomials, with distinct irreducible factors, 85–86
with an even number of factors, 79
irreducible, 70–86
formula, 84
Enumerations, of polynomials, without linear factors, 85–86
linearized completely factorable, 261
with linearly independent roots, 254
with an odd number of factors, 79
perfect squares, 85–86
primitive, 95, 106
of primitive field elements, 95
of q-ary sequences, 275
of roots of polynomials, 257–262
of solutions of the equation $\mathrm{Tr}\,(x^{2^e+1}) = 0$, 423–424
of solutions of the equation $\sum_{k=1}^{j} U_{2k-1}U_{2k} = i$, 416
of subspaces of $\mathrm{GF}(q^m)$, 261
of syndromes, 85
of weights of codewords (*see* Weight enumerators)
Epstein, M. A., 349, 392
Equations, formidable-looking, 210–211
Equidistant codes, 315–318
Equivalent codes, 141–144, 353
Erasure-burst-correcting convolutional codes, 392
Erasure bursts, 392–394
Erasure locator, 229, 230
Erasures, 203, 229
Errata (erasure and/or error), 229
Errata decoding, 231
Errata evaluator, 230
Errata locator, 229
Error-correction capability, 302
Error-distributing codes [*see* Golomb and Posner (1964)]
Error-evaluator polynomial, 219, 350–352
Error-free codes, 344–349
Error-locator polynomial, 18–19, 178, 207–209, 219, 229, 350–352
Errors, bounds, 324–329
limit as P approaches zero, 328
bursts, 347, 393–394
vs. erasures and errata, 229–231
evaluation of, 219, 350–352
vs. failure, 2–3
probability of, 397–400
locations, 18–19, 178, 207–209, 219, 350–352
multiple, 209
probabilities, decoding, 379–400
intersymbol, 205
propagation of, 391
values, 209, 219
η, 230
Euclidean geometry codes, 376, 381, 385–388
Euclid's algorithm, 15, 21–30
Euler's criterion, 171–172
Euler's phi function, 92–93, 95
Evaluator, legitimate, 232–240
Evans, T. A., 381
Even generating function, 74, 212–217
Even number of distinct irreducible factors, 79, 85–86, 159–171, 233

Exchange operation, 57
Existence, of finite fields, 80
of good codes, distance, 321–324, 328
error probability, 324–328
of irreducible polynomials, 80
with linearly independent roots, 253–254
primitive, 89, 106
of primitive field elements, 89
Expansion of code, 346
Expansions, partial fraction, 86
Expected number, of erroneously decoded digits, 343–344
of irreducible factors, 86
Exponential bounds (*see* Bounds)
Expurgated codes, 331–332, 335–336, 353, 410
Extended codes, 331–333
BCH codes, 223–229
Hamming codes, 10–11
multiply-, 333
residue codes, 353

Factorial experiments [*see* Rao (1947)]
Factorization, of polynomials, algorithm for, 146–150
linearized, 253
uniqueness of, 27
of $x^4 - x$, 105
of $x^n - 1$, 154–156
of $x^{q^k} - x$, 103
of $2^m - 1$, 152–153, 173
Falconer, D. D., 349, 393
Falling factorials, 406
Fano, R. M., 326, 349, 392
Feedback coding strategies, 328–329, 349
Feedback decoder for convolutional code, 390–391
Feedback shift register (FSR), 48, 367
synthesis via key equation, 180–181, 199
Feinstein, A., 326
Feller, W., 300
Fermat's theorem and its generalizations, 90, 97–102
Field, 15*n*.
characteristic of, 96
of characteristic 2, reciprocal generating functions in, 192
definition, 87
finite (*see* Finite field)
locator, 218
order of, 88
symbol, 218
Finite field (Galois field), 88, 104
abstract theory of [*see* Albert (1956)]
algebraic structure, 96–104
enumerations (*see* Enumerations)
examples, GF(4), 105
GF(16), 105–111
GF(25), 442
GF(31), 217
GF(32), 441
GF(n) $\cup$ ∞, 357–361
GF($p^{\infty!}$), 111
Finite field (Galois field), existence of, 80
representations of, 104, 108
subfield, 104
tables of [*see* Alanen and Knuth (1964)]
uniqueness, 104
Finite geometries, 375–388
Fire, P., 144–145
Fisher, R. A., 8*n*., 11
Flip-flop, 31, 32
Formal derivative, 74–76
Formidable-looking codes and equations, 210–211
Forney, G. D., xi, 203, 230, 347–349, 429
Foundations, of combinatorial theory [*see* Rota (1964)]
of information theory [*see* Feinstein (1958)]
Frequency modulation (*see* Orthogonal signals), 202
FSR [*see* Feedback shift register (FSR)]
Function, singular, 293

$\mathcal{G}$ (generator matrix), 332, 334, 355–358
Galois field (*see* Finite field)
Galois theory [*see* Artin (1942); Dickson (1900)]
Gallager, R. G., ix, 325, 326, 336, 349, 394 (*See also* Shannon)
Γ_j, 411
$\gamma(z)$, 181
Gates, AND and OR, 30
Gauss' quadratic reciprocity, 171–173
Gaussian noise, 205
gcd (greatest common divisor), 15, 22–30
of affine polynomials, 249
General decoder for cyclic binary codes, 136–138
General solution of key equation, 185–187
Generalized Reed-Muller code, 362
Generating functions, 73–76
Generator matrix, 332, 334, 355–358
Generator polynomial, 129, 211
Geometric codes, 375–388
Geometric interpretation of Lee metric code, 308
GF (Galois field) (*see* Finite field)
GF(n) $\cup$ ∞, 357–361
GF($p^{\infty!}$), 111
Gilbert, E. N., xi, 321–324
Gilbert bound, 321–324, 328–329, 335–336, 392
Gillies, D. B., 153*n*.
Gleason, A. M., 154, 272, 358, 361, 411, 429
Goethals, J. M., 143, 317, 343, 375, 379, 384, 432–433
Golay, M. J. E., 8*n*., 303*n*., 309
Golay codes, binary, 138–139, 294–295, 307, 359, 395, 410
ternary, 307
Golomb, S. W., ix, xi, 115, 154, 305, 307

Gorenstein, D. C., vii, 209, 218, 309, 428
Graham, R. L., xi, 311, 375, 384
Graphical analogy, 325
Gray codes, 206
Greatest common divisor, 15, 22–30
Greatest designed distance, 281
Greatest immediate subordinate, 276
Greatest integer, less than or equal to x, 22
  less than x, 158
Green, M. W., 328
Griesmer, J. H., 329, 335
GRM (generalized Reed-Muller code), 362
Gross, A. J., 199
Group code (linear block code), 4–8
  definition, 7
Groups of permutations, affine, 366
  Mathieu, 395
  projective unimodular, 359, 395
  quintuply transitive, 395
  transitive, 358, 395
  triply transitive, affine, 366
    linear fractional, 358
    Mathieu, 395
  simple, 361
Guard space between bursts, 392–394

$\mathcal{H}$ (parity-check matrix), 5–8
Hadamard matrices, 316–317
Hagelbarger, D. W., 389, 394
Hall, M., ix, 159, 305, 317
Hamming, R. W., 8*n*., 204, 298–299
Hamming codes, 8–11, 343–344, 374
  implementation, 119–125
  weight enumerator of, 404
Hamming distance, 204–205
Hamming metric, 204–205, 298–330
Hamming-Rao bound, 298
Hamming weight, 204–205
Hausdorff dimension, 294
Hermitian matrices, enumeration of [*see* Carlitz and Hodges (1956*a*)]
Heuristic, definition, 327
Heuristic solution of the key equation, 180–184
Higher algebra, [*see* Albert (1956)]
High-rate binary BCH codes, 299
  attaining volume bound, 299–300
  weight enumerators, even m, 425–429
    odd m, 421
Hocquenghem, A., vii, 11, 208
  (*See also under* BCH)
Hodges, J. H., 272
  (*See also* Carlitz)
Horstein, M., 349
Hsu, H. T., 394
Hybrid decoding, 393

$\check{I}$, 294
$\mathcal{I}$ (identity matrix), 147
$I(q,n,d)$, 274
$\check{I}(q,n,d)$, 294
Idempotent, 53
Identities, Newton's, 212
IEEE (Institute of Electrical and Electronics Engineers), 443*n*.
Iff (if and only if), 7*n*.
Illegitimate error evaluator, 232–240
Illegitimate error locator, 232–240
Immediate subordinate, 276
Immediate superior, 276
Implementation, of binary BCH decoders, 195
  of solution of quadratic equation, 244
Inclusion and exclusion, 255
Incomplete decoding algorithm, 2, 361
Indecomposable, into concatenation of codes, 347
  into product of codes, 342–343
Indeterminate Euclid's algorithm, 149
Indicator function, 364
Infinite field, 88
  of characteristic p, 97
Information rate, 3
  with list decoding, 326
Information retrieval [*see* Chow (1967)]
Information symbols, in BCH codes, 273–297
    long codes, 290–294
    nonbinary, 284
    nonprimitive, 287
  in EG codes, 381, 385–388, 396
  in optimum codes, 298–330
  in PG codes, 381, 384–385
  in RM codes, 363
Information theory, foundations of [*see* Feinstein (1958)]
  [*See also* Ash (1965); Gallager (1968); Jelinek (1968); Shannon (1948, 1949)]
Inner and outer, 347–349
Input alphabet, 207
Institute of Electrical and Electronics Engineers (IEEE), 443*n*.
Institute of Radio Engineers (IRE), 443*n*.
Integers, field, 96–99
Interlaced codes (direct-product codes), 338–346
Intersymbol error probabilities, 205
Intersymbol interference, 205
Invariants of a vector subspace [*see* Pless (1965)]
Inverse of the delta function, 83
Inversion, multiplicative, 36–44
  (*See also* Reciprocals)
Inversion formulas, Moebius, 81–85
Inverter, 30
IRE (Institute of Radio Engineers), 443*n*.
Irreducible polynomials, 27
  construction of, 16
    via affine polynomials, 272
    via Chebysheff polynomials (problem 4.6), 118
    via cyclotomic polynomials (problem 6.6), 174
    via elimination, 70–72
    via linear transformation, 112–114
    via power transformation, 115–117
    via reciprocal transformation, 114

Irreducible polynomials, cubics, 169
enumeration by degree, 70–86
existence, 80
expected number of, 86
linearly independent roots, 253–254
parity of number dividing $f(x)$, 159–171
primitive, 106
quadratics, 169
over GF(5), 422
tables of, binary [*see* Marsh (1957); Peterson (1961)]
nonbinary [*see* Alanen and Knuth (1964)]
trinomials, 170, 272
Iterated concatenation, 275
† Iterated direct-product codes, 344–346

J, 284*n*.
$J(q,U,m)$, 277
Jacobs, I. M., ix, 205, 393
Jelinek, F., ix, 325, 393
Jet Propulsion Laboratory, Pasadena, California, xi
Johnson, S. M., 329, 330
Julin, D., 328

$K^{(r)}(q,V,m,j)$, 288
Kac, M., 255
Kailath, T., 349
(*See also* Schalkwijk)
Karlin, M., 357, 361
Kasami, T., 226, 228, 265, 270, 295, 332, 362, 365, 413, 417, 418, 422, 425, 429, 437*n*., 438
Key equation, 178–184
algorithmic solution, 184–188
characteristic 2, 192–195
errata-correcting BCH codes, 30
general solution, 185–187
heuristic solution, 180–184
simplifications over field of characteristic 2, 192–195
Knuth, D. E., 150
[*See also* Alanen and Knuth (1964)]
Kohlenberg, A., 429
Kronecker product, 338*n*.

Label, 12
Lagrange multipliers, 319–320
λ, 229
Landsberg, G., 261, 271
Latin squares [*see* Golomb and Posner (1964)]
lcm (least common multiple), 151
Least affine multiple, 245–248
Least immediate superior, 276
Lebesgue measure, 293
Lee, C. Y., 204
Lee metric, 204–205, 207–217, 298–330
Lee metric codes, perfect, 305–309
Lee weight, 204–205, 316, 400
Legendre symbol, 171–173
Legitimacy tests, 233–240
Legitimate error evaluator, 232–240
Legitimate error locator, 232–240
Lehmer, E., 117
Length of block code n, 1–8
determination from generator polynomial, 151–152
Lengthen, 331–332, 336
‡ Letter, 204
(*See also* Symbol)
Level signals (*see* Amplitude signals)
Levy, J., 307
Lin, S., 228, 265, 270, 295, 332, 362, 365, 429
(*See also* Kasami)
Linear block codes, 4–8, 328
definition, 7
Linear convolutional codes, 388–389
Linear equations, 51–69
Linear fractional group, 358
Linear groups [*see* Dickson (1900)]
Linear polynomial, 28
Linear sequential networks, synthesis of [*see* Huffman (1956)]
Linear transformation of quadratic forms, 414
Linearized polynomials, 241–272
associates, 252–255
which completely factor, 261
duals, 250–255
factorization, 253
gcd, 249
List decoding, 326
Lloyd, S. P., 305
Location α^{∞}, 225–226
Location number, 119
Locator, 19, 207–209, 229
erasure, 220
errata, 229
error, 229
legitimate, 232–240
Locator field, 218, 232
Logan, B. F., 294
Logical circuitry, 30–36
Logs and antilogs, 48–50
GF(16), 108
GF(25), 442
GF(31), 217
GF(32), 441
Long codes, 324
BCH codes, 335
perfect and quasi-perfect, 309
(*See also* Bounds, asymptotic)
Lossless codes (*see* Perfect codes)
Low-rate codes, BCH, as punctured first-order RM, 418
as repetition codes, 177

† The term "iterated" should not be confused with the "iteration matrices" which Golay (1958) used for his special construction of single-error-correcting codes.

‡ Some authors have used the term "letter" in place of the more conventional term "codeword," and the term "alphabet" in place of the term "code."

Low-rate codes, BCH, weight enumerators, even m, 424–425
odd m, 418–421, 436–437
Hadamard codes, 316–318
maximal length FSR codes, 315
as shortened RM codes, 363
Reed-Muller (*see* RM codes)
Low-weight codewords, 262–272, 361
Lucas, M. E., 113, 153*n*., 227, 411
Lucas' theorem, 113

MacDonald, J. E., 329
MacLane, S., viii, ix, 88
(*See also* Birkhoff)
MacRae, R. E., 154
MacWilliams, Mrs. F. J., xi, 143, 357, 361, 375, 379, 384, 396, 399–401, 403, 404, 407, 432–433
(*See also* Delsarte; Graham)
Magnitudes of cyclotomic polynomials, 117
Majority-decision element, 390*n*.
Majority-rule decoding algorithm, 177
Mann, H. B., 275, 277, 294, 297, 375, 381, 384, 396
(*See also* Evans; MacWilliams)
Marker register, 42–43
Marsh, R. W., 272
(*See also* Zierler)
Massey, J. L., xi, 145, 179, 367, 389, 391, 392, 394
Master control unit, 134–135
Mathematical Review, 443*n*.
Mathieu group, 395
Matrices, augmented, 60–62
enumeration by rank, 271–272
persymmetric, 199
generator, 332, 334, 355–358
Hadamard, 316–317
parity-check, 5–8
persymmetric, 199
reduction of, 52–59
algorithm, 55
Matrix-decoding methods, 188–192
Mattson, H. F., xi, 354–355, 358, 360, 361, 395, 407–410, 429
(*See also* Assmus)
Mattson-Solomon polynomial, 407–410
Maximal-length shift-register, 315–316
Maximal-length FSR code, 315–316, 334, 348, 363
threshold decoding, 368
Maximum distance separable codes, 309–311
weight enumerator, 429–431
Maximum codes, 309*n*.
McCoy, N. H., ix, 25, 173
McEliece, R., 411, 413
(*See also* Solomon)
MDE (majority-decision element), 390*n*.
Measure, Lebesque, 293
Memory of convolutional code, 389
Mersenne number, 153*n*.
Mersenne prime, 153*n*.
Message digits, 1–8
Metrics, Hamming, 204–205
Lebesque, 293
Lee, 204–205, 207–217
Milne, W. E., 351
Minimal polynomials, 101
determination of, 112
elements in GF(2^4), 108
elements in GF(2^5), 116–117
tables of [*see* Marsh; Peterson (1961)]
Minimization by ratio test, 301
Minimum distance of decomposable code, 347
(*See also* Bounds)
Mobius (*see* Moebius functions; Moebius inversion formulas)
Modern algebra [*see* Birkhoff and MacLane (1965); Waerden (1931)]
Modify, 331–332
Modulation schemes, 200–204
amplitude (*see* Level signals)
frequency (*see* Orthogonal signals)
phase, 202
Moebius functions, 83
and discriminant, 165
for divisors of integers, 83
for subsets of a given set, 430–431
[*See also* Rota (1964)]
Moebius inversion formulas, 81–85, 91
multiplicative, 84
Moments of weight distribution, 404–405
Monic, 27
MS polynomial, 407–410
Muller, D. E., 418, 437*n*.
Muller (*see* RM codes)
Multidimensional codes, 338–346
Multiple error, 209
Multiple root, 209
Multiplication, 44–51
by constant, 44–46
of two registers, 47–48
Multiplicative inversion, 36–44
algorithm, 41
Multiplicative Moebius inversion, 84
Multiplicative structure of finite fields, 88–90
Multipliers, Lagrange, 319–320
Multiply extended, 333
Muskat, J. B., 361
Myrvaagnes, E., 330
(*See also* Calabi)

$n_{(i)}$, falling factorial, 406
Nadler, M., 328
Nagell, T., ix, 173
Narrow-sense codes (*see* Primitive BCH codes)
Natural integer (rational integer), 96
Necklace enumeration, 86
Negacyclic codes, 211–217
low-weight codewords in, 268
primitive and nonprimitive, 213–214
Negative, 87
Negative errors, 216
Newton's identities, 212, 263
Noise bursts, 143–144, 340, 392–394

Noise power, 205
Noise word, 5
Noncyclic codes, 324, 350–352
Nondeficient sequence, 387
Nondifferentiable but continuous function, 293
Nongroup codes (*see* Nonlinear codes)
Nonlinear codes, 316, 317, 328
  perfect, 305
Nonprimitive BCH codes, 176
  information symbols, 287
Nonprimitive negacyclic codes, 213–214
Nonresidue, quadratic, 171
Nordstrom, A. W., 328
  (*See also* NR codes; Preparata)
North Carolina, University of, xi
NOT, 30
NR codes (Nordstrom-Robinson codes), 328
Null set (empty set), 429
Nullity of traditional decoding matrix, 189–195
Number, of distinct irreducible factors, 150, 159–171
  of smaller integers coprime to n, 95
Number theory [*see* Nagell (1951)]
Numerology, 154

Odd generating function, 74, 212–217
Odd number of distinct irreducible factors, 79, 85–86, 159–171, 233
$\omega(z)$, 179
Ones-complement arithmetic, 109
Open conjectures (*see* Unsolved problems)
Optimal codes, 309*n*.
Optimum codes, 298–330, 335
OR gate, 30
Order, of α, 88
  of field, 88, 102
  of q mod composite, 157
  of q mod prime, 158
  of q mod prime power, 157
  of 2 mod prime, 158–159
  of 2 mod prime squared, 157
Ordering, canonical, 338–346
Ore, O., 241, 247, 272
Orthogonal circulant matrices [*see* MacWilliams (1968*b*)]
Orthogonal eigenvectors, 314
Orthogonal matrices [*see* Goethals and Seidel (1967)]
Orthogonal signals, 202–203
Orthogonalizability of RM codes, 374
Orthogonalizable codes, 374–388
Orthogonalized parity-check equations, 368
Outer and inner, 347–349

p polynomials, 241
$P_e(N,M,L)$, 326
Parity of number of distinct irreducible factors, 79, 85–86, 159–171, 233
  of binary trinomial, 170
Parity-check matrix, 5–8, 355–358
  definition, 7
Partial-fraction expansions, 86
Partitions, which avoid long arithmetic progressions, 118
  of integers, 363
Penny-weighing problem [*see* Golay (1958)]
Perfect codes, 302–309, 359
  Golay (*see* Golay codes)
  Lee metric, 303–305
  nonlinear, 305
  single-error-correcting, 303–305
Perfect simple difference set, 380–381, 396
Perfect squares enumeration, 85–86
Period of a polynomial, 150–153
  composite, 151
  $f(x^t)$, 153
  irreducible, 151–152
Permutation decoding, 361
Permutation groups (*see* Groups of permutations)
Permutation-threshold decoder, 369
Permutations, and bursts, 143–144
  of codes, 141–144, 342, 357–361, 366, 394
Persymmetric, 199
Peterson, W. W., vii, 47, 226–229, 265, 266, 269*n*., 270, 295, 296, 309, 332, 362, 365, 428, 429
  (*See also* Gorenstein; Kasami)
PG (projective geometry) (*see* PG codes)
PG codes, 375–385
$\emptyset$ (empty set or null set), 429, 430
φ function, Euler's, 92
$\Phi(z)$, 215
Π (product sign), 28
Pinsker, M., 349, 393
Pitcher, T., 294
Planar difference set, 380–381
Pless, V., 361, 400, 404–405, 407, 432–433
Pless power-moment identities, 405, 410–411
  triangularization of, 417–418, 435
Plotkin, M., 311
Plotkin bound, 312, 321, 328
Pollak, H. O., xi
Polynomials, affine, 243–272
  algorithmic factorization, 146–150
  algorithmic root-finding, via affine polynomials, 241–246
    via Chien search, 132–135
  binomial resultant, 163
  bivariate, 341
  Chebyshev, 118
  Chinese remainder theorem for, 29
  cyclotomic, 90–95, 156–159
  discriminant of, 160–171
  with distinct irreducible factors, 79, 85–86, 159–171, 233
  enumeration (*see* Enumerations)
  Euclid's algorithm, 21–27
  with an even number of distinct irreducible factors, 79, 85–86, 159–171, 233

Polynomials, factorization, via algorithm, 146–150
is unique, 27
gcd of, 22–30
of affine polynomials, 249
irreducible (*see* Irreducible polynomials)
least affine multiple, 245–248
linearized, 241–272
computation of roots, 241–245
with an odd number of distinct irreducible factors, 79, 85–86, 159–171, 233
order of, 150–153
p, 241–272
parity of number of distinct irreducible factors, 79, 85–86, 159–171, 233
period of, 150–153
primitive, 106
quadratics, 169, 243–245
random, 86
resultant of, 160–163
root-finding, via affine polynomials, 241–246
via Chien search, 132–135
trinomials, 46, 153–154, 163–164, 170–171
Tschebycheff, 118
unique factorization of, 27
weight of, 46
weight three, 46, 153–154, 163–164, 170–171
Polyominoes, packing of [*see* Golomb and Welch (1968)]
Positive errors, 216
Posner, E. C., 305
(*See also* Golomb)
Power-moment identities, Pless, 405, 410–411, 435
Power-sum symmetric functions, unweighted, 209, 212–217
weighted, 219
Powers, 49, 88
Powers that be, 331, 348
Prange, E., 228, 358, 361, 375
Prefix, 275
Preparata, F. P., 328–329, 394
′ (*see* Derivatives)
Prime to n, relatively, 95
Primitive BCH codes, 176
Primitive factor of $2^m - 1$, 173
Primitive field element, 49, 89
enumerator, 95
existence, 89
Primitive mod n, 174
Primitive negacyclic codes, 213–214
Primitive nth root, 89
of unity, 89
Primitive polynomial, 106
tables of [*see* Peterson (1961)]
Primitive polynomials (*see* Irreducible polynomials)
Primitive roots, 89
Principal eigenvalue, 313
Probability, of decoding error, 324–329, 349, 397–400
of decomposable code, 347
of decoding failure, 397–400
of error and failure, 2, 397–400
of iterated Hamming codes, 343–344
of intersymbol error, 205
Probabilistic decoding, 392–393
Probabilistic Information theory [*see* Jelinek (1968)]
Probabilistics proof, 254
Problems, unsolved (*see* Unsolved problems)
Product, component-wise, 365
dot, 401
vector, 365n.
Product codes, cyclic, 340–343
decomposition, 342–343
direct, 338–346
Kronecker, 338n.
tensor, 346–347
Products of generating functions, 73
Products enumeration, 85–86
Programming computers [*see* Knuth (1967)]
Projection, of set, 377
into shortened code, 395
Projective geometry, 376–381, 410–411
Projective-geometry codes, 375–385
Projective subspace, 377, 379–381
Projective unimodular group, 359
Proper prefix, 275
Proper suffix, 275
Pseudorandom bits, 154
Puncture, 331–332, 334

Q, 146
q-ary polynomials, enumeration, 76–80
q-conjugates, 104
QR codes, 352–361
vs. BCH codes, 361
invariance under permutations, 357–359, 361, 394–395
minimum distance table, 360
vs. RM codes, 394, 399
short length 11 (*see* Golay codes, ternary)
short length 17, 139–141, 356
short length 23 (*see* Golay codes, binary)
short length 31, 394, 399
weight enumerators, 411, 432–433
Quadratic character of 2, 158
Quadratic codes, nonlinear, 328
Quadratic equation over field of characteristic 2, 238, 243–245
Quadratic form, 314
over field of characteristic 2, 414–417
Quadratic nonresidue, 171
Quadratic polynomials, characteristic 2, 238
discriminant, 169
over field of characteristic 2, 243–245
over GF(5), 422
irreducibility, 169

Quadratic reciprocity, 171–173
Quadratic residue, 171
Quadratic-residue code (*see* QR codes)
Quartic character of 2, 159
Quartic equation over field of characteristic 2, 239
Quasi-perfect codes, 309, 428
Question mark (*see* Erasure)
Quinary codes, 209–211
Quotients of generating functions, 73

R (rate), 3
Radakrishnan-Rao (*see* Rao)
Random polynomial, 86
Range space, 250
Rank of traditional decoding matrix, 189–195
Rao, C. R., 298–299
Rate, 3
  BCH codes, 273
  critical, 327
  with list decoding, 326
  long BCH codes, 290–294
  optimum codes, 298–330
Rational integer (natural integer), 96
Ray-Chaudhuri, D. K. (*see* Chaudhuri)
Received word, 5
Reciprocal polynomial, 106, 220
Reciprocals, of field elements, 18, 87
  of generating function in a field of characteristic 2, 192
Recurrent codes (convolutional codes), 388–394
Reddy, S. M., 329, 389–391
Reduced triangular idempotent matrix, 52*n*.
Reed, I. S., vii, 218, 367, 408, 418, 429, 437*n*.
  (*See also* RM codes; RS codes)
Reed inversion formula, 408
Reed-Muller codes (*see* RM codes)
Reed-Solomon codes, 218, 232*n*., 310, 346–349, 429–431
  weight enumerator, 429–431
References, 443–452
Register, 33, 34
Reiffen, B., 349, 392
  (*See also* Wozencraft)
Relatively prime number of smaller integers, 95
Reliability function, 326
Repeated factor, 75–76
Repetition codes, 1–4
  as BCH codes, 177
  decoding, 367–368
  as GRM codes, 363
  as maximum distance separable codes, 310
  as perfect codes, 303
  threshold decoding, 367–368
Representation, Gray code, 206
Representations of Galois fields, 104
Research topics (*see* Unsolved problems)
Residue, quadratic, 171
Residue bound, 407
Residue classes, 88*n*.
Residue codes, 352–361
  (*See also* QR codes)
Residue mod n, 352
Resultant, 160
  of binomials, 163
  of product, 162
Reversible codes, 145
  decoding, 141
ρ, complex reciprocal roots, 291
Riordan, J., ix, 386, 405
RM codes, 239, 361–367
  vs. BCH codes, 364
  first-order, weight enumerator of, 404
  information symbols, 363
  vs. QR codes, 394, 399
  second-order, subcodes of, 413–429, 436–439
    weight restrictions, 413–417
  threshold decoding, 367–375
Robinson, J. P., 328, 390*n*., 391
  (*See also* NR codes; Preparata)
Rook domains [*see* Golomb and Posner (1964)]
Roos, J. E., 305
Root, W. L., 205
  (*See also* Davenport)
Roots, 28, 49
  computation of, 132–135, 241–246
  enumeration of, 257–262
  multiple, 209, 216
  space spanned by, 248
Rota, G. C., 83, 431
Rotate, 54
RS codes, 218, 232*n*., 310, 346–349, 429–431
Rudolph, L. D., 375
Rumsey, H., 170*n*.
  (*See also* Berlekamp)
Ryser, H. J., ix

$\mathbb{S}$ (the squaring matrix), 50–51
S (power-sum symmetric function), 209, 212–217, 219
$S^{(2t)}(z)$, 233
$s(u)$, 292
$\check{s}(u)$, 295
$S(z)$, 179
Sacks, G. E., 323, 329
Saskovets, V. N., 333
  (*See also* Andryanov)
Savage, J. E., 393*n*.
Schalkwijk, J. P. M., 349
Schneider, D. I., 47
  (*See also* Bartee)
Schonheim, J., 305
Secondary eigenvalues, 313
Second-order RM codes, subcodes of, 413–429, 436–439
  weight restrictions, 413–417
Seidel, J. J., 317
  (*See also* Goethals)
Selfridge, J. L., 153
Separable codes, maximum distance, 309–311

Sequence enumeration theorems, 278
Sequential decoding, 392–393
Set command, 31
Shannon, C. E., vii, 8*n*., 325, 326, 328, 336, 392
Shaughnessy, E. P., 361
Shift register sequences [*see* Golomb (1967)]
Shift registers (*see* Feedback shift registers)
Shifts, cyclic, 47
  left, 34
Shorten, 331–332, 336
Shorthand logical diagrams, 35–36
σ (elementary symmetric function) (*see* Error-locator polynomial)
Σ (summation sign), 29
Simple groups, 361
Simultaneous congruences, 29
Simultaneous linear equations, 51–69
Singer, J., 381
Single-error-correcting codes, perfect, 303–305
Single-error decoder, 9–11
Single-parity-check codes, 1–4, 310
Singleton, R. C., 310–311
Singular function, 293
Sinh, 405–407
Skew matrices, enumeration of [*see* Carlitz and Hodges (1956*a*)]
Slave multipliers, 134–135
Slepian, D., 7, 347
Sloane, N. J. A., xi
Smith, K. J. C., 375, 384, 396
Solomon, G., vii, xi, 170*n*., 218, 329, 332, 334–335, 354–355, 407–410, 411–413, 429
  (*See also* Berlekamp; Mattson; Reed; RS codes)
Specifications, 331
Spence, E., 317
Spencer, J. H., xi
Spheres, critical, 318
  in Hamming metric, 298
  in Lee metric, 298
  packing bound, 299, 327
  surface area, 298–302
  volume, 298–302
Squares and square roots, computation of, 50–51
Srivastava, J. N., 350–352
Srivastava codes, 350–352
Standard array, 7
$\star$, 30*n*.
Stickelberger, L., 160–171
Stickelberger's theorem, 233, 243–245
  generalization for characteristic 2, 166–171
  over GF(2), 170
  odd characteristic, 164
Stiffler, J. J., viii, 330, 332, 334–335
  (*See also* Solomon)
Stirling numbers, 405
Storer, T., 159
Subblocks, 206
Subcodes, of second-order RM codes, 413–429
  subfield, 336–338
Subfield, 104
  of integers, 97
Subfield subcodes, 336–338
Subordinates, 276
Subspaces, direct sum of, 401
  dual of, 401
  enumeration, 261
Suffix, 275
Sullivan, D. D., 389, 390
Sum of generating functions, 73
Supercode, 348–349
Superior, 276
Surface area, 298
Swan, R. G., 154, 160*n*., 162–171, 175
Symbol, 200–205
Symbol error, 203
Symbol field, 218, 232
Synchronization, viii
Syndrome, 5–7, 204
  for convolutional code, 389
  definition, 7
  enumeration, 85
Syndrome-threshold decoder, 371
Synthesis of linear sequential networks [*see* Huffman (1956)]
Systematic codes, 303*n*.

t-error-correcting codes, 11
$T(z)$, 230
Table 4.1, 108
Table 4.2, 115
Table 4.3, 116
Table 9.1, 214
Table 16.1, 432–433
Table 16.2, 434
Table 16.3, 435
Table 16.4, 436–437
Table 16.5, 438
Table 16.6, 440
Tables, of finite fields [*see* Alanen and Knuth (1964)]
  of irreducible polynomials [*see* Alanen and Knuth (1964); Marsh (1957); Peterson (1961)]
Tanh, 405–407
$\tau(z)$, 181
Taylor series, 399, 405
Tensor product codes, 346–347
Thesis topics (*see* Unsolved problems)
Thompson-Ramo-Wooldridge, xi
Threshold decoders, 367–388
  vs. algebraic decoders, 375
  for convolutional code, 390–391
  multistep, 371–375
Threshold element, 370
Time quantization, 200
Torus, 307–308
Townsend, R. L., 375
Tr (trace), 111, 166–171, 243–245, 425, 426
Tradeoffs, 2
Transformation of quadratic forms, 414–417

Transitive permutation groups, 228
(*See also* Triply transitive groups)
Translated moment of weight distribution, 404–405
Transmitter, 1
Transpose, 5
Tree codes (*see* Convolutional codes)
Triangular idempotent form, 52–59
Trinomials, 46, 153–154, 163–164, 170–171
discriminant of, 163
irreducible over GF(2), 272
parity of number of irreducible factors, 170
Triply transitive groups, affine group, 366
linear fractional, 358–359, 395
Mathieu group, 395
Tschebycheff polynomials over GF(2), 118
Turyn, R., 361, 429
(*See also* Assmus)
Two-dimensional codes, 338–346
Two-phase circuit, 34

$U(z)$, 185
Uncountable set of measure zero, 293–294
Undifferentiable but continuous function, 293
Unique factorization of polynomials, 27
Uniqueness of GF(q), 104
University of California, Berkeley, xi
University of North Carolina, Chapel Hill, xi
University of North Carolina Press, xi
Unsolved problems, 210–211, 217, 294, 296–297, 324, 327, 352, 400, 420–421, 425, 428–429

$V_k^{(r)}(j)$, 288
$V(z)$ (enumerator of basic components), 290
$V(z)$ (polynomial in the general solution of key equation), 185
van der Waerden, B. L., viii, 88, 160*n.*, 169
Varsharmov, R. R., 323, 329
Vasil'ev, Ju. L., 305
Vector (word), 5*n.*
Vector product, 365*n.*
Vector subspace, invariants of [*see* Pless (1965)]
Viterbi, A. J., 392
Volume, 298
Volume bound, 298–302, 328, 359

Waerden, B. L., viii, 88, 160*n.*, 169
Wagner, T. J., 309, 329, 330
Wax, N., 330
Weaver, W. (*see* Shannon)
Weight, binary, 6–7
of coset leaders, 6–7, 324, 399, 427–428, 440
Hamming, 204–205
of integer, 263, 381–388
Lee, 204–205
of polynomial, 46
Weight enumerators, 397–440
of BCH codes (*see* BCH codes)
of cyclic short codes, 432–433
of dual codes, 400–407
of Hamming codes, 404
Lee metric, 400
restrictions on, 407–417
of RM codes, first-order, 404
second-order subcodes, 417–429, 436–439
of RS codes, 429–431
of short cyclic codes, 432–433
Weighted power-sum symmetric functions, 219
Welch, L. R., 307, 423, 427
(*See also* Golomb)
Weldon, E. J., Jr., 339, 341, 375, 381
(*See also* Burton; Townsend)
Wide-sense codes (primitive and/or nonprimitive codes), 176
Wolf, J. K., 347
Wolfowitz, J., ix, 326
Woodall, H. J. (*see* Cunningham)
Word (vector), 5*n.*
Wozencraft, J. M., ix, 205, 349, 392, 393
Wyner, A. D., 311, 318, 389, 392, 394
(*See also* Graham)

X (error location), 178

Y (error value), 219
Yudkin, H. L., 392, 393

Zaremba, S. K., 303*n.*
Zero-error capacity [*see* Shannon (1956)]
Zierler, N., vii, 154, 209, 218, 272, 309, 428
(*See also* Brillhart; Gorenstein)
Ziv, J., 349